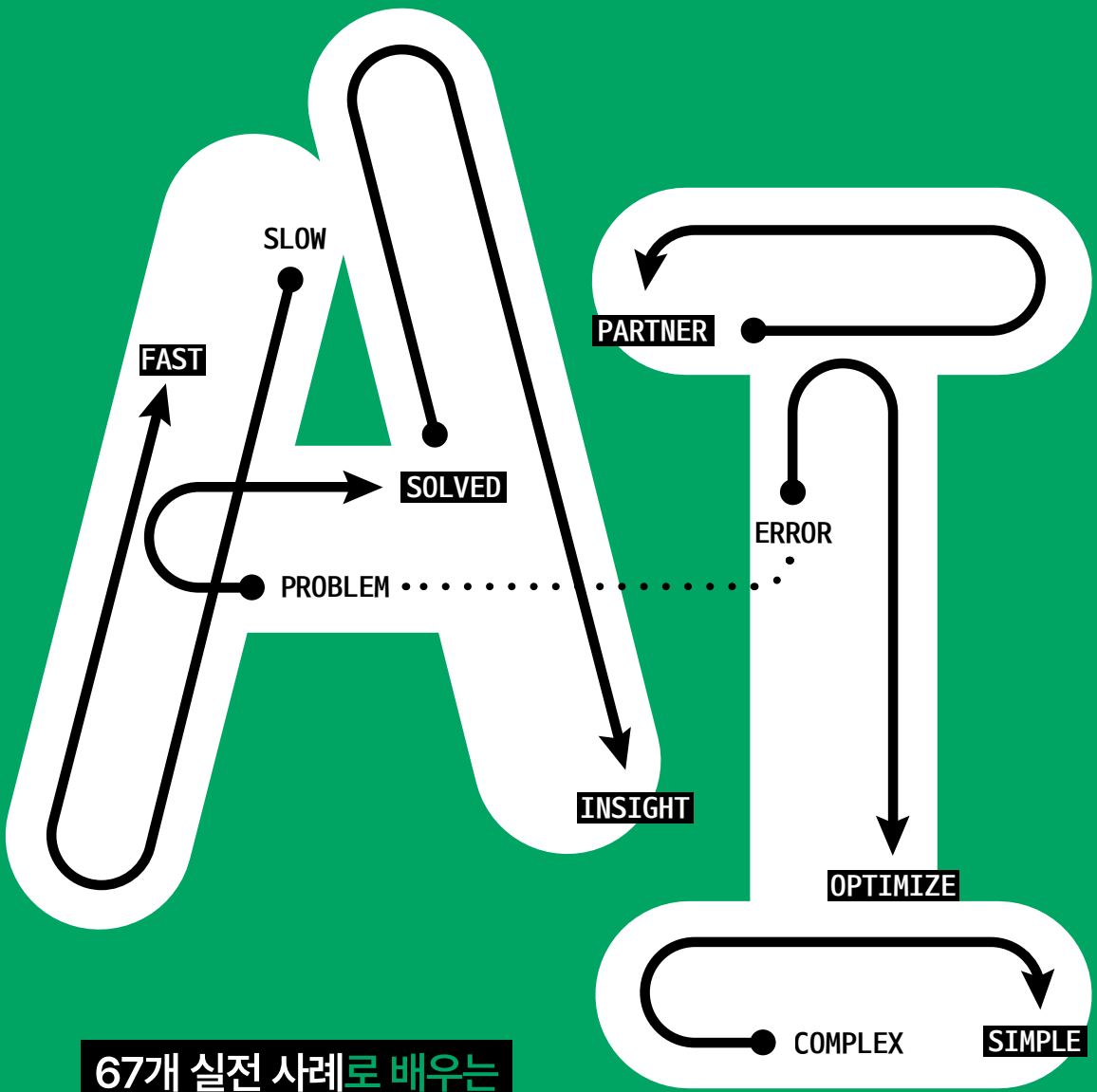
GitHub Copilot 채팅과 함께하는 SQL 튜닝 가이드

저자
김성식
이병찬



67개 실전 사례로 배우는
AI 기반 Microsoft
SQL Server 성능 최적화

exem

저자 소개

김성식

(주)엑셈의 통합기술센터 이사로 재직 중이며, SQL Server 2000 시절부터 현재까지 데이터베이스 유지보수와 다양한 DB 컨설팅(성능 최적화, DR, 부하 분산, 이중화 구축 등)을 수행해왔다. SQL 튜닝을 보다 쉽게 전달하고자 집필한 『SQL Server 튜닝 가이드』에 이어, SQL Server를 효율적으로 활용할 수 있는 방안을 제시하기 위해 AI 기반 GitHub Copilot 채팅 관련 내용을 집필하였다.

이병찬

성공회대학교에서 소프트웨어공학과 컴퓨터공학을 복수 전공하였다. 과거에는 SaaS 기반 생성형 AI 챗봇 솔루션 개발 팀에서 근무했으며, 현재는 엑셈에서 SQL Server 컨설팅과 데이터베이스 성능 모니터링 솔루션 지원을 담당하고 있다. 다양한 현장 경험을 바탕으로 생성형 AI 개발 경험과 SQL Server 튜닝 노하우를 접목해 이 책을 집필하였다.

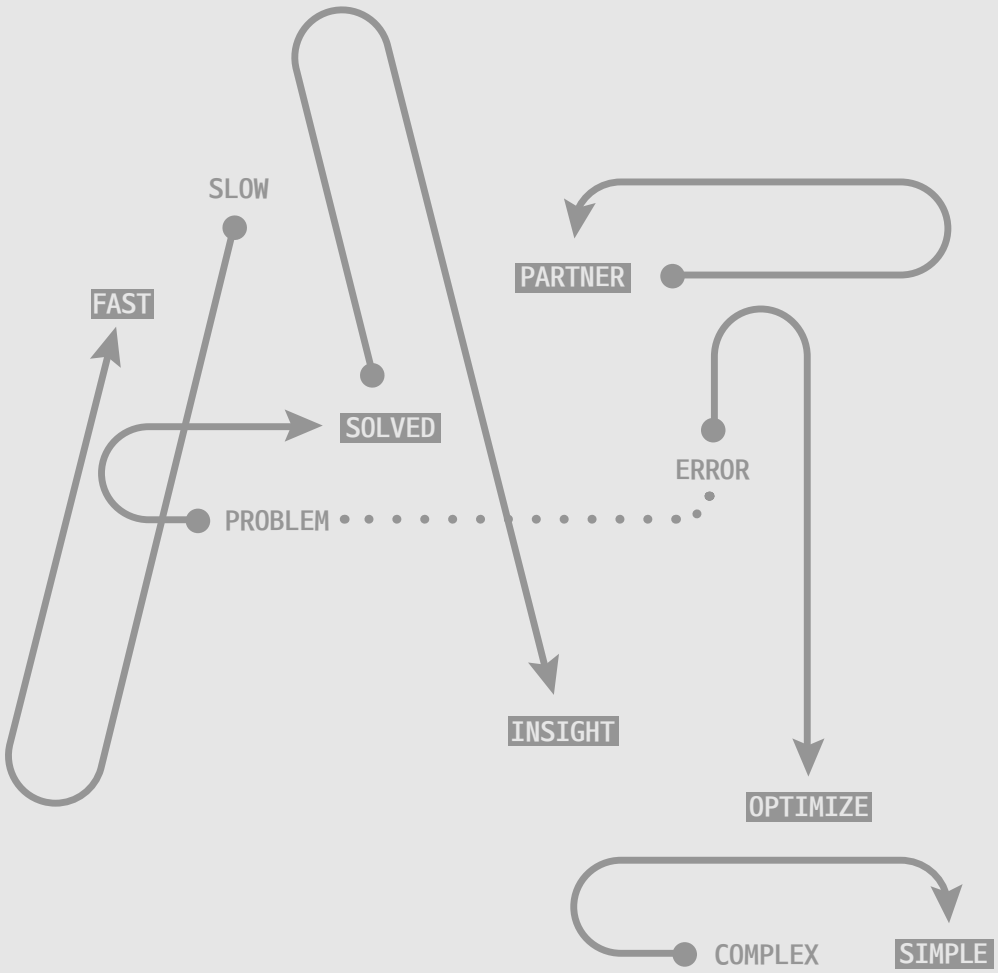
(주)엑셈

엑셈은 AI 기반 IT 통합 성능관리 전문기업입니다. 국내 DB 성능관리 1위 솔루션 MaxGauge부터 E2E 거래추적 APM 솔루션 InterMax, 클라우드, AI, 빅데이터 플랫폼까지 엑셈의 폭넓은 글로벌 기술력은 29개국 1,000여 개 고객사가 증명합니다. 빅데이터 분석과 AI 기술로 한 발 앞서 장애를 방지하는 시스템 모니터링으로 금융, 공공, 제조 등 다양한 산업군을 아우르며IT 성능 관리의 새로운 기준을 제시하고 있습니다.

exem

GitHub Copilot 채팅과 함께하는 SQL 튜닝 가이드

67개 실전 사례로 배우는
AI 기반 Microsoft SQL Server 성능 최적화 ✦



김성식 · 이병찬 지음

exem

목차 구성

CHAPTER 01

들어가며 06

Preface 1. 저자의 말 08

Preface 2. 머리말 10

Preface 3. 추천사 11

CHAPTER 02

GitHub Copilot 채팅 활용 가이드 12

Guide 1. GitHub Copilot 채팅 사용을 위한 필수 요소 14

Guide 2. GitHub Copilot 채팅의 기본 사용법 16

Guide 3. GitHub Copilot 채팅에 효과적으로 질문하는 방법 22

Guide 4. 좋은 질문 vs 모호한 질문 예시 23

CHAPTER 03

실전 사례집 32

Section 1. 인덱스와 접근 경로 최적화 34

- 사례 01. RID LOOKUP에 의한 성능 이슈 - 1 34
- 사례 02. RID LOOKUP에 의한 성능 이슈 - 2 38
- 사례 03. KEY LOOKUP에 의한 성능 이슈 42
- 사례 04. 인덱스 부재로 인한 성능 이슈 46
- 사례 05. 인덱스 키 누락으로 인한 성능 이슈 50
- 사례 06. 인덱스 페이지 분할에 의한 성능 이슈 54
- 사례 07. 인덱스를 사용하지 못하는 이슈 60
- 사례 08. CROSS APPLY를 이용한 부분 범위 처리 66
- 사례 09. CROSS JOIN으로 반복 사용된 테이블 통합 72
- 사례 10. 내부 조인의 스캔 범위 개선 77
- 사례 11. BETWEEN의 인덱스 스캔 범위 개선 - 1 83
- 사례 12. BETWEEN의 인덱스 스캔 범위 개선 - 2 88
- 사례 13. IN 조건절의 성능 이슈 94
- 사례 14. 조건절 컬럼이 가공됐을 때의 영향 101

Section 2. 실행 계획 및 연산 처리 최적화 106

- 사례 15. SPOOL 연산자 성능 개선 - 1 106
- 사례 16. SPOOL 연산자 성능 개선 - 2 111
- 사례 17. SPOOL 연산자 성능 개선 - 3 116
- 사례 18. TOP EXPRESSION (SUBQUERY) 121
- 사례 19. TOP EXPRESSION (NOT EXISTS) 126
- 사례 20. TOP EXPRESSION (TOP N) 130

Section 3. 쿼리 구조 및 서브쿼리 최적화 134

- 사례 21. 중복된 서브쿼리 사용으로 테이블을 여러 번 읽는 이슈 134
- 사례 22. 스칼라 서브쿼리 성능 이슈 138
- 사례 23. NOT IN 조건으로 사용된 서브쿼리 성능 이슈 144
- 사례 24. WITH 절의 성능 이슈 149
- 사례 25. EXCEPT 구문 성능 이슈 155
- 사례 26. INTERSECT 구문 성능 이슈 160
- 사례 27. 데이터 중복 제거에 대한 스캔 범위 개선 165

Section 4. 변수 및 임시 객체 활용 이슈 170

- 사례 28. 변수 테이블의 통계 개선 - 1 170
- 사례 29. 변수 테이블의 통계 개선 - 2 175
- 사례 30. 변수 테이블의 기능 제약 개선 179
- 사례 31. 변수가 사용된 OR 절의 성능 이슈 183
- 사례 32. 변수 사용에 따른 카디널리티 추정 이슈 188

Section 5. 커서와 절차형 처리 개선 196

- 사례 33. CURSOR 성능 이슈 - 1 196
- 사례 34. CURSOR 성능 이슈 - 2 201
- 사례 35. CURSOR 성능 이슈 - 3 208

Section 6. 집계 및 함수 처리 최적화 216

- 사례 36. COUNT 함수의 OVER 절 성능 이슈 - 1 216
- 사례 37. COUNT 함수의 OVER 절 성능 이슈 - 2 221
- 사례 38. FUNCTION 성능 이슈 - 1 226
- 사례 39. FUNCTION 성능 이슈 - 2 234
- 사례 40. FUNCTION 성능 이슈 - 3 240
- 사례 41. FUNCTION 성능 이슈 - 4 246
- 사례 42. ISNULL() 함수가 사용된 조건절의 성능 이슈 254
- 사례 43. MAX() 함수의 성능 개선 - 1 258
- 사례 44. MAX() 함수의 성능 개선 - 2 261
- 사례 45. MIN() 함수의 성능 개선 266
- 사례 46. NULL 값을 포함하지 않는 함수 개선 272
- 사례 47. 데이터 형식 우선순위에 의한 암시적 CONVERT 276
- 사례 48. 함수를 이용한 문자열 분할 성능 개선 281

Section 7. 동시성 및 잠금 287

- 사례 49. 스캔 범위에 따른 블로킹 이슈 - 1 287
- 사례 50. 스캔 범위에 따른 블로킹 이슈 - 2 293
- 사례 51. DEADLOCK 이슈 - 1 302
- 사례 52. DEADLOCK 이슈 - 2 309
- 사례 53. 잠금이 전환될 때 발생하는 DEADLOCK 이슈 319

Section 8. DML 최적화 325

- 사례 54. DELETE 구문의 성능 개선 - 1 325
- 사례 55. DELETE 구문의 성능 개선 - 2 330
- 사례 56. DELETE 구문의 성능 개선 - 3 336
- 사례 57. UPDATE / INSERT를 MERGE 문으로 통합 340
- 사례 58. 저장 프로시저 성능 이슈 345

Section 9. 통계 정보 351

- 사례 59. 잘못된 통계 정보로 인한 성능 이슈 - 1 351
- 사례 60. 잘못된 통계 정보로 인한 성능 이슈 - 2 356
- 사례 61. 통계 부재로 인한 성능 이슈 364

Section 10. 파티션 및 원격 쿼리 370

- 사례 62. REMOTE QUERY - 1 370
- 사례 63. REMOTE QUERY - 2 377
- 사례 64. REMOTE QUERY - 3 384
- 사례 65. PARTITION을 활용한 성능 개선 - 1 390
- 사례 66. PARTITION을 활용한 성능 개선 - 2 396
- 사례 67. PARTITION을 활용한 성능 개선 - 3 402

CHAPTER 04

마치며 408

Afterword. 총정리 & 에필로그 410

들어가며

01

Preface 1. 저자의 말 08

Preface 2. 머리말 10

Preface 3. 추천사 11

저자의 말

우리는 지금, 기술의 전환점 한가운데에 서 있습니다.

과거에는 더 빠른 CPU와 디스크, 더 큰 메모리, 더 정교한 아키텍처가 생산성 향상의 핵심이었다면, 이제는 그 중심에 'AI'가 자리하고 있습니다. AI는 단순한 자동화 도구를 넘어, 인간의 사고를 보조하고 확장하는 새로운 협업 파트너로 자리 잡고 있습니다. 결국 이 변화의 본질은 하나로 귀결됩니다. AI는 생산성을 극적으로 향상시키는 도구라는 점입니다.

데이터베이스 분야 역시 예외가 아닙니다. 오랜 시간 동안 데이터베이스 운영과 관리는 숙련된 DBA의 경험과 직관에 크게 의존해 왔습니다. 인덱스를 어떻게 설계할 것인지, 쿼리를 어떻게 튜닝할 것인지, 장애를 어떻게 분석할 것인지 등은 사람의 영역이었습니다. 그러나 이제 그 패러다임이 바뀌고 있습니다. AI는 방대한 실행 계획을 분석하고, 병목 지점을 찾아내며, 최적의 개선안을 제시합니다. 더 나아가 스스로 학습하여 반복적인 운영 작업을 자동화하고, 예측 기반의 관리까지 수행하는 단계로 나아가고 있습니다.

데이터베이스 운영과 관리 또한 AI와 함께하는 시대가 된 것입니다.

이러한 변화는 단순한 편의성의 문제가 아닙니다. 경쟁력의 문제입니다. 동일한 환경에서 누군가는 AI를 활용해 문제를 빠르게 해결하고, 더 나은 설계를 만들어내며, 더 높은 품질의 결과를 만들어냅니다. 반면, AI를 활용하지 않는다면 점점 더 많은 시간과 노력이 필요해질 것입니다. 격차는 서서히, 그러나 확실하게 벌어집니다.

그래서 이제는 분명히 말할 수 있습니다.

AI와의 협력은 더 이상 선택이 아니라 필수입니다.

이 책은 그러한 변화의 흐름 속에서 집필되었습니다. 특히 SQL 성능 문제 해결이라는 실전 영역에서, AI 기반 GitHub Copilot 채팅과의 협업이 어떻게 새로운 가능성을 만들어내는지를 보여주고자 합니다. 단순히 AI가 답을 대신해 주는 것이 아니라, 문제를 함께 분석하고, 다양한 관점을 제시하며, 더 빠르고 정확한 의사결정을 돕는 과정 자체가 핵심입니다.

결국 중요한 것은 도구가 아니라 활용 방식입니다. AI를 단순한 보조 수단으로 사용할 것인지, 아니면 함께 사고하는 파트너로 받아들일 것인지는 우리의 선택에 달려 있습니다. 이 책이 독자 여러분이 AI와 보다 효과적으로 협력하고, 데이터베이스 운영과 성능 튜닝의 새로운 기준을 만들어가는 데 작은 도움이 되기를 바랍니다.

기술은 계속 진화합니다. 그리고 그 변화 속에서 앞서 나가는 사람은 언제나 새로운 도구를 받아들이고, 그것을 자신의 것으로 만드는 사람입니다. 이제 그 도구는 AI입니다.

저자 대표 | **김성식**

머리말

오늘날의 개발 환경은 빠르게 변화하고 있습니다. 개발은 더 이상 코드를 작성하는 데 그치지 않고, 데이터 처리와 성능 관리까지 포함하는 복합적인 영역으로 확장되었습니다. 특히 데이터베이스는 대부분의 시스템에서 핵심적인 역할을 담당하며, SQL 성능 최적화는 시스템의 품질과 사용자 경험을 좌우하는 중요한 요소로 자리 잡고 있습니다.

이러한 흐름 속에서 GitHub Copilot 채팅은 새로운 가능성을 제시하고 있습니다. 단순한 코드 생성 도구를 넘어, 문제를 이해하고 해결 방향을 함께 탐색하는 협업 파트너로서의 역할을 수행하기 시작한 것입니다. SQL 튜닝 과정에서도 쿼리 구조 분석, 인덱스 설계 제안, 실행 계획 해석 등 다양한 영역에서 실질적인 도움을 제공합니다.

이 책은 GitHub Copilot 채팅의 사용법을 설명하는 데 목적이 있지 않습니다. SQL 성능 최적화라는 실제 문제를 중심으로, 이를 어떻게 활용할 수 있는지를 사례를 통해 보여주는 데 초점을 맞추고 있습니다. 독자는 GitHub Copilot 채팅이 제시하는 다양한 가능성을 바탕으로 성능 병목을 파악하고, 최적화 방향을 탐색하며, 문제를 해결하는 과정을 자연스럽게 경험하게 될 것입니다. 내용은 GitHub Copilot 채팅의 기본적인 활용 방법을 시작으로, 총 67개의 실전 사례를 기반으로 핵심을 정리한 예시를 통해 다양한 성능 문제 해결 과정을 다루고 있습니다. 각 사례는 단순한 결과 제시에 그치지 않고, 문제의 원인과 해결 과정, 그리고 그 과정에서 GitHub Copilot 채팅이 어떻게 활용되었는지를 함께 담고 있습니다.

초보 개발자부터 숙련된 DBA까지, SQL 성능 최적화에 관심 있는 모든 독자를 대상으로 합니다. 다만 GitHub Copilot 채팅은 정답을 제공하는 도구가 아닙니다. 여러 가능성을 제시하고 선택을 돕는 조력자에 가깝습니다. 결국 최종적인 판단과 책임은 여전히 사용자에게 있습니다. 최근 LLM 모델의 발전은 문제 해결 방식에도 변화를 가져오고 있습니다. 과거에는 질문의 수준이 결과의 품질을 크게 좌우했다면, 이제는 비교적 단순한 질문에도 높은 수준의 답변을 얻을 수 있는 환경이 되었습니다. 이로 인해 더욱 중요해진 것은 질문을 정교하게 만드는 능력보다, 결과를 해석하고 올바른 결정을 내리는 능력입니다. 다양한 제안 속에서 무엇을 선택할 것인지 판단하는 과정이 성능 최적화의 핵심이 되었기 때문입니다.

데이터베이스 성능 개선과 개발 생산성을 높이는 실질적인 파트너로 이해하는 데 도움이 되기를 바랍니다. 또한 변화하는 개발 환경 속에서 보다 효과적으로 문제를 해결하는 새로운 접근 방식을 발견하는 계기가 되기를 기대합니다.

추천사

급변하는 정보기술 환경 속에서 데이터베이스의 안정적 운영과 성능 최적화는 조직의 업무 연속성과 서비스 품질을 좌우하는 핵심 요소입니다. 특히 SQL Server는 다양한 업무 시스템의 기반으로 활용되고 있는 만큼, 이를 체계적으로 이해하고 효율적으로 관리하기 위한 실무 지침이 무엇보다 중요합니다.

『GitHub Copilot 채팅과 함께하는 SQL 튜닝 가이드』는 저자의 전작 『SQL Server 튜닝 가이드』에서 다룬 풍부한 실무 사례를 기반으로, 최신 AI 도구인 GitHub Copilot 채팅을 접목하여 성능 최적화의 새로운 가능성을 제시하는 뜻깊은 자료입니다. 현장에서 바로 적용할 수 있는 실질적인 경험과 유익한 내용을 담아내어, SQL Server 운영 및 성능 개선을 담당하는 실무자들에게 명확한 기준과 방향을 제공할 것입니다.

무엇보다 오랜 실무 경험과 전문적 식견을 바탕으로 이 책을 집필해 주신 엑셈 김성식 저자께 깊이 감사드립니다. 저자의 정성 어린 노력과 풍부한 지식이 담긴 본 자료는 현업 담당자들에게 깊이 있는 통찰과 실질적인 도움을 제공하는 소중한 지침이 될 것입니다. 다시 한번 감사의 뜻을 전하며, 본 가이드가 SQL Server를 운영하는 많은 실무자들에게 든든한 길잡이가 되기를 바랍니다.

신원의료재단 정보사업부 | **장기주 상무**

GitHub Copilot 채팅을 효과적으로 활용하기 위해서는 단순한 사용을 넘어, 준비부터 질문 방식까지 전반적인 이해가 필요하다. 본 가이드에서는 설치 및 환경 설정을 시작으로, 기본 사용법과 채팅 활용 방식을 단계적으로 살펴본다. 또한 원하는 결과를 이끌어내기 위한 질문 방법과, 좋은 질문과 모호한 질문의 차이를 함께 다룬다. 이를 통해 GitHub Copilot 채팅을 실무에 보다 효율적으로 적용할 수 있도록 돕는다.

* 본 가이드의 내용은 모두 SSMS(SQL Server Management Studio) 22.4.1 기준으로 진행되었다.

GitHub Copilot 채팅 활용 가이드

02

Guide 1. GitHub Copilot 채팅 사용을 위한 필수 요소 14

Guide 2. GitHub Copilot 채팅의 기본 사용법 16

Guide 3. GitHub Copilot 채팅에 효과적으로 질문하는 방법 22

Guide 4. 좋은 질문 vs 모호한 질문 예시 23

Guide 01

GitHub Copilot 채팅 사용을 위한 필수 요소

GitHub Copilot 채팅을 제대로 활용하기 위해서는 먼저 환경 구성이 필요하다. 이 장에서는 설치부터 기본 설정까지, 바로 사용할 수 있는 준비 과정을 안내한다.

1 사전 준비 사항

1. 필수 소프트웨어

SQL Server Management Studio (SSMS) 22 이상 버전

2. 계정 권한

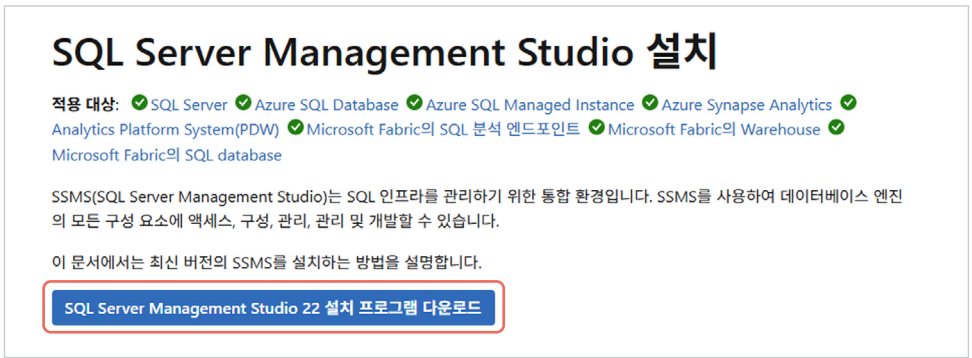
GitHub Copilot 채팅 액세스 권한이 활성화된 GitHub 계정

2 GitHub Copilot 채팅 설치 방법

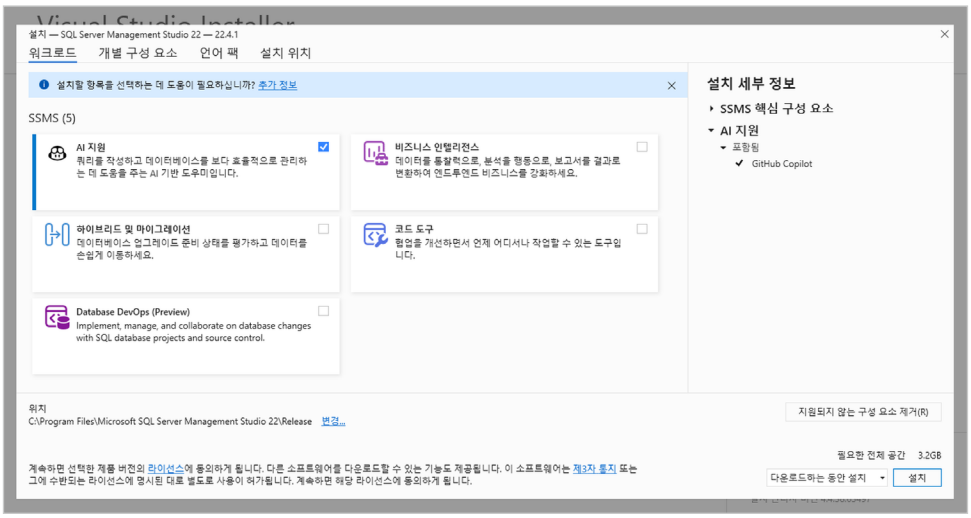
SSMS 환경에 GitHub Copilot 채팅 기능을 추가하려면 Visual Studio 설치 관리자를 통해 'AI 지원'을 추가해야 한다.

1 공식 홈페이지에서 SQL Server Management Studio 22 설치 프로그램을 다운받는다.

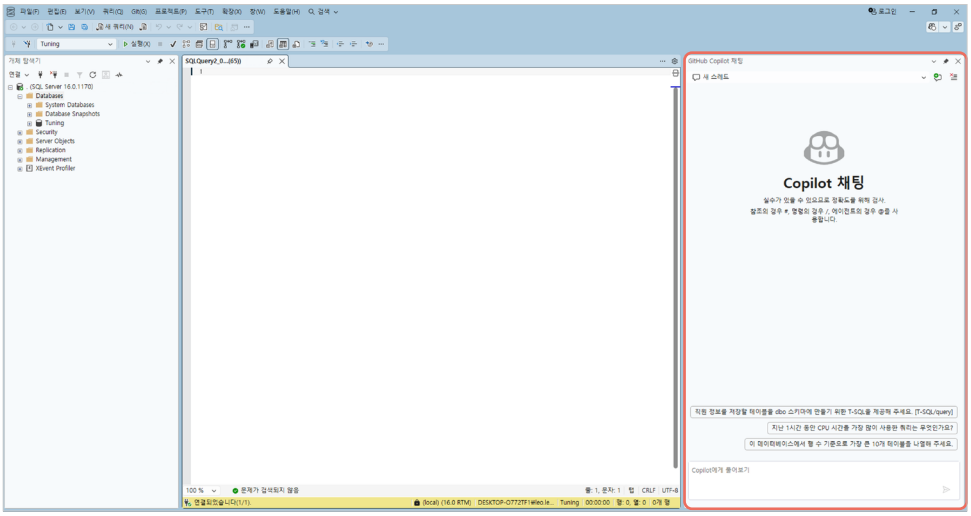
https://learn.microsoft.com/ko-kr/ssms/install/install



2 다운받은 설치 프로그램을 실행하여, AI 지원 선택 후 설치를 진행한다.



3 설치 완료 후 SSMS를 통해 GitHub Copilot 채팅 기능을 사용할 수 있다.



Guide
02

GitHub Copilot 채팅의 기본 사용법

GitHub Copilot 채팅은 단순한 도구가 아니라 '대화형 인터페이스'다. 이 장에서는 GitHub Copilot 채팅을 사용하는 기본적인 흐름과 핵심 기능을 살펴본다.

1 GitHub Copilot 채팅 사용을 위한 필수 요소

GitHub Copilot 채팅의 사용 방법과 각 부분의 기능을 알아보자.

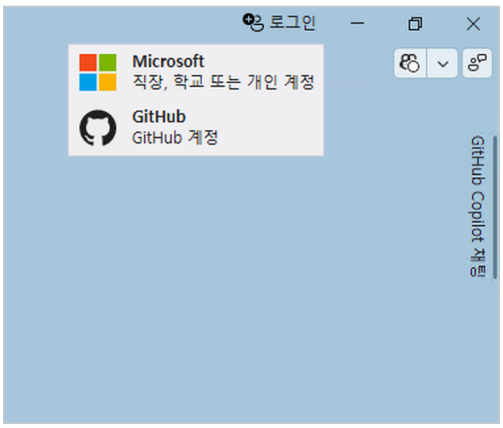
1. 계정 연결 및 관리

로그인

GitHub Copilot 채팅 사용 권한이 있는 GitHub 계정으로 로그인한다.

다중 계정

여러 개의 계정을 등록할 수 있으며, 현재 GitHub Copilot 채팅을 사용하는 계정은 [활성(Active)] 상태로 표시된다.



[로그인 화면]

[다중 계정 활성 표시 화면]

2. GitHub Copilot 채팅 인터페이스 구성 요소

GitHub Copilot 채팅에게 질문을 할 수 있는 방법은 2가지로 GitHub Copilot 채팅에서 직접 질문하거나, 코드 내에서 인라인으로 질문하는 것이다. GitHub Copilot 채팅은 사용자의 편의성과 효율적인 작업을 위해 직관

적인 인터페이스를 제공한다. 화면에 표시된 각 번호의 세부 기능은 다음과 같다.

❶ 대화 이력
과거에 진행했던 대화 기록을 확인하거나, 이전 작업의 컨텍스트를 다시 불러와 중단된 작업을 이어갈 수 있도록 지원한다.

❷ 새 스레드 시작
기존 대화의 문맥을 초기화하고, 완전히 새로운 주제나 독립적인 쿼리 작업에 대해 AI와 대화를 시작할 때 사용한다.

❸ 스레드 지우기
현재 진행 중인 대화 내용을 삭제하여 작업 공간을 정리한다.

❹ 참조
현재 편집기에서 활성화된 SQL 스크립트나 문서를 AI가 참조할 수 있도록 첨부하는 기능이다. 필요에 따라 자동 또는 수동으로 활성 문서를 추가하여, AI가 현재 작업에 적합한 답변을 생성하도록 유도할 수 있다.

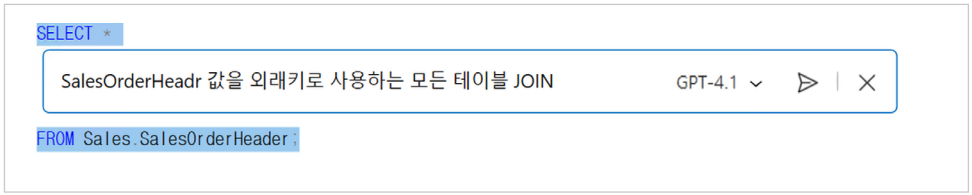
❺ 모델 선택
답변 생성에 사용할 AI 모델을 선택한다. 작업의 복잡도나 사용자의 선호도에 맞춰 가장 적합한 모델로 유연하게 변경할 수 있다.

❻ 보내기
입력창에 작성한 프롬프트를 선택한 AI 모델로 전달하여 답변 생성을 요청한다.

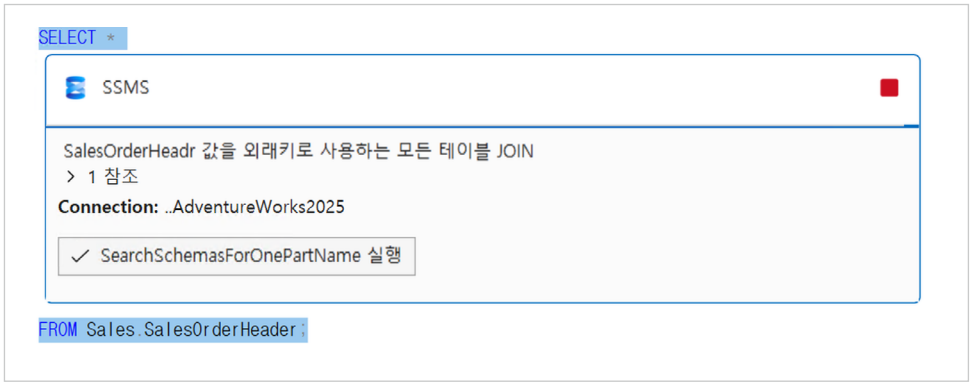
3. 인라인 채팅

SSMS의 인라인 채팅 기능은 편집 창 안에서 AI 보조 기능에 직접 질문하고, 그 응답을 바로 확인할 수 있도록 지원한다. 이를 통해 사용자는 별도의 대화 화면으로 이동하지 않고도, 현재 열려 있는 SQL 스크립트를 유지한 채 쿼리 작성, 코드 해석, 오류 점검 등의 작업을 수행할 수 있다. 사용 방법은 다음과 같다.

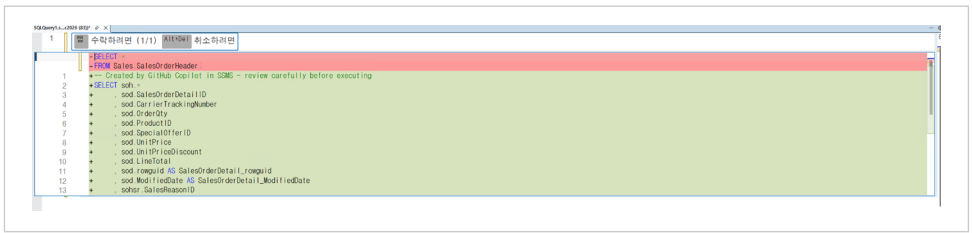
1 SSMS에서 [Alt+/] 키를 선택하여 편집기에서 GitHub Copilot 채팅의 인라인 채팅 보기를 표시한다.



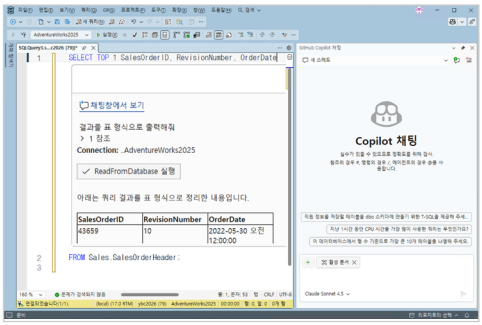
2 질문 텍스트 상자에 T-SQL 관련 질문을 입력한 다음 Enter 키 또는 보내기를 선택하여 질문을 한다.



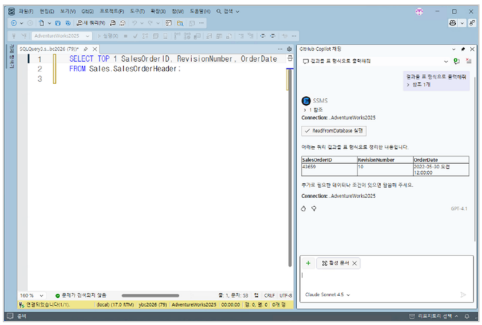
3 GitHub Copilot 채팅의 코드 제안이 diff 보기에 표시된다. 제안된 내용을 검토하고 구체화하고 Tab 키를 사용하여 코드에 변경 내용을 적용하거나 Alt+Delete를 사용하여 삭제할 수 있다.



4 채팅 창에서 보기를 선택하여 인라인 채팅 스레드를 채팅 창으로 승격할 수 있다. 이렇게 하면 대화의 레코드와 컨텍스트가 유지되며 채팅 창에서 계속할 수 있다.



[인라인 채팅 화면]



[채팅창에서 보기]

5 Esc를 선택하여 인라인 채팅 보기를 닫는다.

4. '활성 문서' 참조 기능

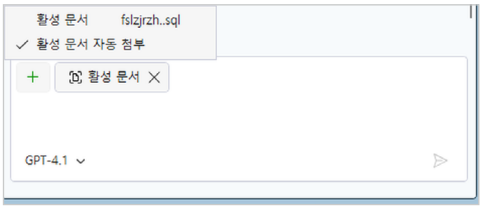
AI 모델은 사용자의 프롬프트를 중심으로 응답을 생성하지만, GitHub Copilot 채팅 환경에서는 현재 작업 중인 코드나 대화 이력과 같은 추가 컨텍스트가 함께 활용될 수 있다. 활성 문서 참조 기능은 현재 새 쿼리 탭에서 작업 중인 SQL 스크립트 등의 내용을 GitHub Copilot 채팅이 참고할 수 있도록 하여, 코드 문맥에 맞는 설명이나 제안을 생성하는 데 도움을 주며 이를 통해 사용자는 더 적합한 답변을 받을 수 있다.

· 활성 문서 첨부 방식

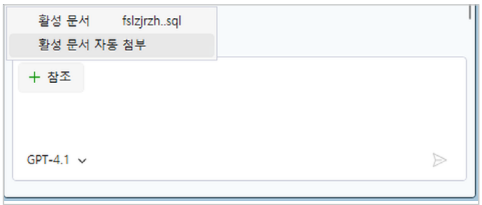
AI에게 현재 작업 중인 SQL 구문을 컨텍스트(참고 자료)로 전달하여 답변의 품질을 높이는 것은 GitHub Copilot 채팅 활용의 핵심이다. SSMS에서는 제어가 가능하도록 여러 첨부 방식을 제공한다.

1 자동 첨부(기본 동작)

사용자가 현재 편집기에서 포커스하고 있는 스크립트 탭의 내용이 AI 모델에게 자동으로 전달된다. 별도의 조작 없이도 AI가 현재 작성 중인 쿼리를 실시간으로 읽어내어 질문에 더 적합한 답변을 제안한다. 필요에 따라 클릭하여 이 기능을 비활성화하면 활성 문서를 첨부되지 않게 할 수 있다.



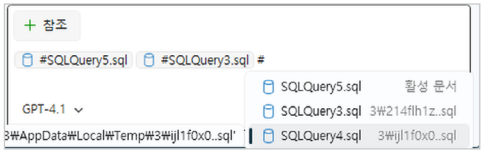
[활성 상태]



[비활성 상태]

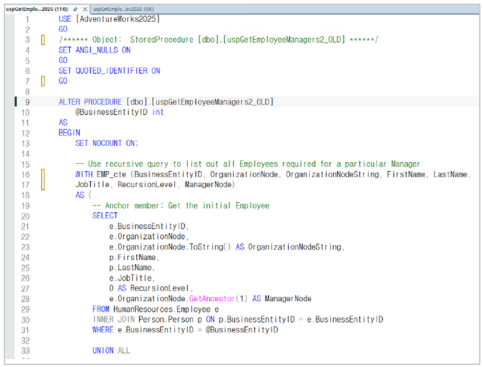
2 다중 참조

채팅 창에서 # 기호를 입력하면 현재 SSMS에 열려 있는 활성 문서들의 목록이 나타난다. 이 기능을 활용하면 사용자가 원하는 특정 내용만 명시적으로 선택하여 모델에게 전달할 수도 있고, 여러 개의 문서를 동시에 참조하게 만들어 더욱 복잡한 분석을 지시할 수 있다.



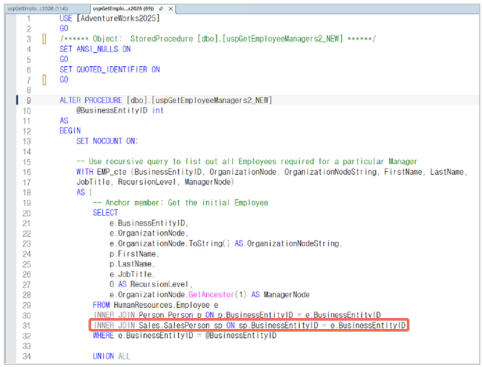
다중 참조 활용: 두 프로시저 간의 변경 점 비교 분석

기존 프로시저에서 변경한 내용을 두 참조를 통해 비교해서 찾아내는 사례



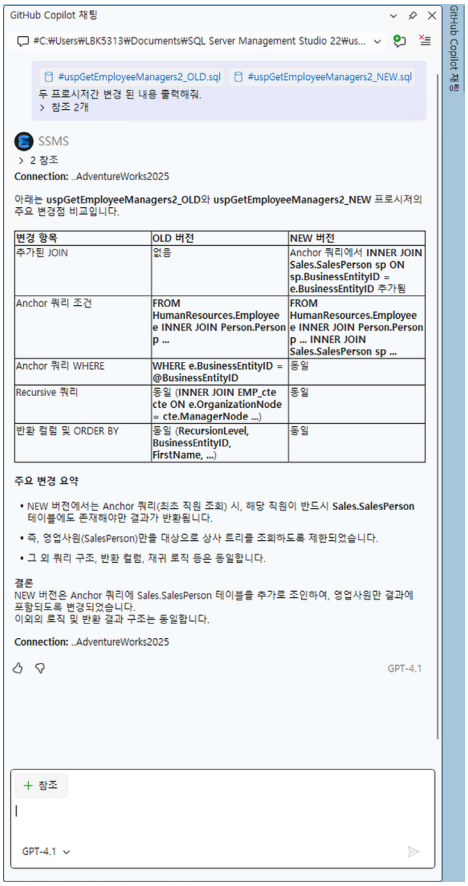
```
USE [AdventureWorks2025]
GO
/****** Object:  StoredProcedure [dbo].[uspGetEmployeeManagers2_OLD] ******/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO

ALTER PROCEDURE [dbo].[uspGetEmployeeManagers2_OLD]
    @BusinessEntityID int
AS
BEGIN
    SET NOCOUNT ON;

    -- Use recursive query to list out all Employees required for a particular Manager
    WITH EMP_cte (BusinessEntityID, OrganizationNode, OrganizationNodeString, FirstName, LastName,
    JobTitle, RecursionLevel, ManagerNode)
    AS (
        -- Anchor member: Get the initial Employee
        SELECT
            e.BusinessEntityID,
            e.OrganizationNode,
            e.OrganizationNode.ToString() AS OrganizationNodeString,
            p.FirstName,
            p.LastName,
            e.JobTitle,
            0 AS RecursionLevel,
            e.OrganizationNode.GetAncestor(1) AS ManagerNode
        FROM HumanResources.Employee e
        INNER JOIN Person.Person p ON p.BusinessEntityID = e.BusinessEntityID
        WHERE e.BusinessEntityID = @BusinessEntityID

        UNION ALL
```

[AS-IS]



```
USE [AdventureWorks2025]
GO
/****** Object:  StoredProcedure [dbo].[uspGetEmployeeManagers2_NEW] ******/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO

ALTER PROCEDURE [dbo].[uspGetEmployeeManagers2_NEW]
    @BusinessEntityID int
AS
BEGIN
    SET NOCOUNT ON;

    -- Use recursive query to list out all Employees required for a particular Manager
    WITH EMP_cte (BusinessEntityID, OrganizationNode, OrganizationNodeString, FirstName, LastName,
    JobTitle, RecursionLevel, ManagerNode)
    AS (
        -- Anchor member: Get the initial Employee
        SELECT
            e.BusinessEntityID,
            e.OrganizationNode,
            e.OrganizationNode.ToString() AS OrganizationNodeString,
            p.FirstName,
            p.LastName,
            e.JobTitle,
            0 AS RecursionLevel,
            e.OrganizationNode.GetAncestor(1) AS ManagerNode
        FROM HumanResources.Employee e
        INNER JOIN Person.Person p ON p.BusinessEntityID = e.BusinessEntityID
        INNER JOIN Sales.SalesPerson sp ON sp.BusinessEntityID = e.BusinessEntityID
        WHERE e.BusinessEntityID = @BusinessEntityID

        UNION ALL
```

[TO-BE]



[답변]

5. 모델 선택 및 사용 정책

SSMS의 *GitHub Copilot 채팅 기능은 작업 목적에 따라 서로 다른 모델을 선택해 사용할 수 있도록 구성되어 있다. 사용자는 질문에 필요한 응답 방식에 맞춰 적절한 모델을 선택할 수 있으며, 인터페이스의 드롭다운 목록을 통해 모델을 변경할 수 있다.

*GitHub Copilot 채팅은 여러 AI 모델을 지원하며, 각 모델은 할당된 승수에 따라 사용량을 차감한다.

· GitHub Copilot 채팅에서 지원되는 AI 모델

https://docs.github.com/ko/copilot/reference/ai-models/supported-models

· AI 모델 비교 - GitHub 문서

https://docs.github.com/ko/copilot/reference/ai-models/model-comparison

Guide
03

GitHub Copilot 채팅에 효과적으로 질문하는 방법

기본적으로 채팅 창에는 활성 쿼리 편집기를 기반으로 연결된 데이터베이스에 대한 컨텍스트가 있다. 프롬프트를 작성할 때는 모호한 질문으로 인해 모호한 응답이 발생하므로 요청에 따라 구체적으로 설명한다.

1 프롬프트 작성 시 권장 사항

1. 자연스럽고 이해하기 쉽게 작성

특수 어휘를 사용할 필요는 없지만 비공식적인 문구, 속어 및 전문 용어를 피하려고 해야 한다.

2. 명확하고 구체적으로 요청

GitHub Copilot 채팅이 요청하는 내용에 대해 명확하게 알 수 있도록 충분한 세부 정보를 제공한다.

3. 컨텍스트(참고 자료) 제공

SSMS의 GitHub Copilot 채팅에는 쿼리 편집기 연결을 기반으로 하는 연결 및 데이터베이스 컨텍스트가 있지만 시간 또는 특정 데이터베이스 개체와 같은 추가 컨텍스트를 제공할 수 있다.

4. 예제 사용

'이전 쿼리를 수정해서 결과를 CustomerID 기준으로 정렬해 주세요.'와 같이 이전 응답을 기반으로 빌드한다.

5. 출력 형식 정의

GitHub Copilot 채팅은 텍스트, 테이블 또는 목록 형식으로 정보를 제공할 수 있다. 기본적으로 적절한 형식을 스스로 선택하지만, 원하는 형식이 있다면 해당 형식으로 반환하도록 명시적으로 요청할 수 있다.

Guide
04

좋은 질문 vs 모호한 질문 예시

같은 도구라도 질문 방식에 따라 결과는 크게 달라진다. 이 장에서는 좋은 질문과 모호한 질문을 비교하여 차이를 직관적으로 이해할 수 있도록 한다.

1. 자연스럽고 이해하기 쉽게 작성	**비권장**	매출 데이터를 뽑을 수 있게 뭔가 멋지게 하나 만들어줘
	권장	Sales.SalesOrderDetail 테이블에서 가장 최근 주문 10건을 조회하는 SQL 쿼리를 작성해 줘.
2. 명확하고 구체적으로 요청	**비권장**	그러니까 어제 데이터로 뭔가 보고서나 결과물이 필요하긴 한데, 너무 최근 가입자는 아니고 그렇다고 너무 오래된 사용자도 아닌 신규 사용자만 대상으로 했으면 좋겠어요. 무슨 말인지 아시죠?
	권장	어제 가입했고 아직 한 번도 구매하지 않은 사용자를 조회하는 쿼리를 작성해줘.
3. 컨텍스트 (참고 자료) 제공	**비권장**	많이 팔린 제품을 알려줘
	권장	2023년 한 해 동안 Sales.SalesOrderDetail.OrderQty 합계가 가장 높은 제품(ProductID) 10개를 보여줘
4. 예제 사용	**비권장**	최근 고객 정보를 가져오는 쿼리를 작성해 줘
	권장	지난 10 영업일 동안의 고객 ID, 이름, 총 주문 수를 조회하고, 총 주문 수 기준 내림차순으로 정렬하는 쿼리를 작성해줘.
5. 출력 형식 정의	**비권장**	가장 바쁜 영업사원 알려줘.
	권장	가장 많은 매출을 기록한 상위 10명의 영업사원을 표로 보여줘. 표에는 SalesPersonID, Name, TotalSalesAmount 컬럼이 포함되어야해.

1. 자연스럽고 이해하기 쉽게 작성

■ 비권장

· Model

GPT-4.1

· Prompt

> 매출 데이터를 뽑을 수 있게 뭔가 멋지게 하나 만들어줘

· 답변

매출 데이터 추출을 위해 가장 직관적이고 활용도가 높은 뷰와 테이블 정보를 기반으로, 아래와 같이 매출 요약(주문별, 날짜별, 영업사원별 등)을 쉽게 조회할 수 있는 T-SQL 스크립트를 제공합니다.

```
-- Created by GitHub Copilot in SSMS - review carefully before executing
SELECT soh.SalesOrderID, soh.OrderDate, soh.SalesOrderNumber, soh.CustomerID,
       sp.FirstName + ' ' + sp.LastName AS SalesPersonName, soh.TotalDue AS
       OrderTotal, SUM(sod.LineTotal) AS LineTotalSum
FROM Sales.SalesOrderHeader AS soh
LEFT JOIN Sales.SalesOrderDetail AS sod
ON soh.SalesOrderID = sod.SalesOrderID
LEFT JOIN Sales.vSalesPerson AS sp
ON soh.SalesPersonID = sp.BusinessEntityID
GROUP BY soh.SalesOrderID, soh.OrderDate, soh.SalesOrderNumber, soh.CustomerID,
          sp.FirstName, sp.LastName, soh.TotalDue
ORDER BY soh.OrderDate DESC;
```

컬럼명	설명
SalesOrderID	주문 ID
OrderDate	주문일
SalesOrderNumber	주문번호
CustomerID	고객 ID
SalesPersonName	영업사원 이름
OrderTotal	주문별 총액
LineTotalSum	주문별 상세 합계

이 쿼리는 주문별 매출, 주문일, 영업사원, 고객 등 주요 정보를 한 번에 확인할 수 있습니다. 필요에 따라 WHERE 절로 기간, 영업사원, 고객 등 필터링을 추가해 활용할 수 있습니다.특정 집계 방식이나 추가 정보가 필요하면 말씀해 주세요.

권장

· Model

GPT-4.1

· Prompt

> Sales.SalesOrderDetail 테이블에서 가장 최근 주문 10건을 조회하는 SQL 쿼리를 작성해 줘.

· 답변

아래는 **Sales.SalesOrderDetail** 테이블에서 가장 최근 주문 10건을 조회하는 SQL 쿼리입니다.

```sql
SELECT TOP 10 *
FROM Sales.SalesOrderDetail
ORDER BY ModifiedDate DESC, SalesOrderID DESC, SalesOrderDetailID DESC;
```

ModifiedDate를 기준으로 내림차순 정렬하여 최신 주문부터 조회합니다.
동일한 날짜가 있을 경우 SalesOrderID, SalesOrderDetailID로 추가 정렬합니다.

결과는 다음과 같이 반환됩니다.

SalesOrde-rID	SalesOrderDetai-lID	ProductID	OrderQty	UnitPrice	Modified-Date
75123	121317	712	1	8.99	2025-06-29 오전 12:00:00

원하는 컬럼만 선택하려면 SELECT 절에 컬럼명을 지정하면 됩니다.

2. 명확하고 구체적으로 요청

비권장

· Model

GPT-4.1

· **Prompt**

> 그러니까 어제 데이터로 뭔가 보고서나 결과물이 필요하긴 한데, 너무 최근 가입자는 아니고 그렇다고 너무 오래된 사용자도 아닌 신규 사용자만 대상으로 했으면 좋겠어요. 무슨 말인지 아시죠?

· **답변**

네, 이해했습니다.
요약하면 “어제 가입한 사용자”가 아니라, “너무 최근도, 너무 오래전도 아닌” 일정 기간 내에 가입한 신규 사용자만을 대상으로 보고서나 결과물을 원하신다는 의미로 이해했습니다.

다음 단계 안내:

1. “신규 사용자”의 기준(예: 최근 7일, 30일 등 가입일 범위)을 명확히 정해야 합니다.
2. 사용자 정보가 저장된 테이블(예: 회원, 사용자 등)과 가입일 컬럼명을 확인해야 합니다.
3. 해당 조건에 맞는 사용자만 추출하여 원하는 보고서나 결과를 생성할 수 있습니다.

신규 사용자의 기준(며칠 이내 가입자 등)을 알려주시면, 정확한 쿼리나 결과를 제공해 드릴 수 있습니다. 또한, 사용자 테이블의 정확한 이름이나 가입일 컬럼명이 필요하니, 혹시 알고 계시면 알려주세요.

모르실 경우, 제가 직접 스키마를 탐색해 확인하겠습니다.

■ **권장**

· **Model**

GPT-4.1

· **Prompt**

> 어제 가입했고 아직 한 번도 구매하지 않은 사용자를 조회하는 쿼리를 작성해줘.

· **답변**

어제에 가입했고 아직 한 번도 구매하지 않은 사용자를 조회하는 쿼리는 아래와 같습니다.

```sql
-- Created by GitHub Copilot in SSMS - review carefully before executing
SELECT p.BusinessEntityID, p.FirstName, p.LastName, c.CustomerID
FROM Person.Person p
JOIN Sales.Customer c ON p.BusinessEntityID = c.PersonID
LEFT JOIN Sales.SalesOrderHeader soh ON c.CustomerID = soh.CustomerID
WHERE soh.CustomerID IS NULL
  AND CAST(p.ModifiedDate AS date) = DATEADD(day, -1, CAST(GETDATE() AS date));

Person.Person 테이블의 ModifiedDate가 어제인 사용자 중
Sales.Customer로 등록되어 있으나
Sales.SalesOrderHeader에 구매 이력이 없는 사용자만 조회합니다.
```

3. 컨텍스트(참고 자료) 제공

■ 비권장

· Model

GPT-4.1

· Prompt

> 많이 팔린 제품을 알려줘

· 답변

```
"많이 팔린 제품"을 확인하려면,
"판매 수량" 기준인지
"판매 금액" 기준인지
정확히 어떤 기준으로 집계할지 명확히 해주시면 더 정확한 결과를 드릴 수 있습니다.

어떤 기준으로 집계할까요?

예시:
판매 수량 기준
판매 금액(매출) 기준
원하시는 기준을 알려주시면 바로 결과를 제공하겠습니다.
```

■ 권장

· Model

GPT-4.1

· Prompt

> 2023년 한 해 동안 Sales.SalesOrderDetail.OrderQty 합계가 가장 높은 제품(ProductID) 10개를 보여줘

· 답변

2023년 한 해 동안 Sales.SalesOrderDetail.OrderQty 합계가 가장 높은 제품 10개는 아래와 같습니다.

순위	ProductID	OrderQty 합계
1	863	2,380
2	715	2,032
3	712	1,951
4	862	1,526
5	711	1,475
6	852	1,475
7	854	1,432
8	708	1,355
9	760	1,311
10	770	1,291

4. 예제 사용

■ 비권장

· Model

GPT-4.1

· Prompt

> 최근 고객 정보를 가져오는 쿼리를 작성해 줘

· 답변

아래는 최근 고객 정보 10건을 조회한 결과입니다.

Busi-nes-sEntityID	First-Name	LastName	EmailAddress	PhoneNumber	City	StateProv-inceName	CountryRe-gionName
20777	Crystal	Hu	crys-tal21@adventure-works.com	1 (11) 500 555-0126	Darling-hurst	New South Wales	Australia
20776	Crystal	Zheng	crys-tal20@adventure-works.com	1 (11) 500 555-0171	Ver-sailles	Yveline	France
20775	Crystal	He	crys-tal19@adventure-works.com	813-555-0148	Bremer-ton	Washington	United States
20774	Isabella	Richardson	isabel-la91@adventure-works.com	910-555-0166	Torrance	California	United States
20773	Crystal	Guo	crys-tal18@adventure-works.com	1 (11) 500 555-0171	W. York	England	United King-dom
20772	Crystal	Liang	crys-tal17@adventure-works.com	1 (11) 500 555-0120	London	England	United King-dom
20771	Isabella	Bailey	isabel-la90@adventure-works.com	808-555-0174	Beverly Hills	California	United States
20770	Crystal	Gao	crys-tal16@adventure-works.com	1 (11) 500 555-0136	Duesseldorf	Hessen	Germany
20769	Crystal	Zhu	crys-tal15@adventure-works.com	1 (11) 500 555-0180	Ham-burg	Hamburg	Germany
20768	Isabella	Murphy	isabel-la89@adventure-works.com	1 (11) 500 555-0117	Metz	Moselle	France

권장

· Model

GPT-4.1

· Prompt

> 지난 10 영업일 동안의 고객 ID, 이름, 총 주문 수를 조회하고, 총 주문 수 기준 내림차순으로 정렬하는 쿼리를 작성해줘.

· 답변

최근 10 영업일 동안의 고객 ID, 이름, 총 주문 수를 조회하는 쿼리 예시입니다.
아래는 T-SQL 템플릿과 실제 예시입니다.

일반 템플릿

```sql
-- Created by GitHub Copilot in SSMS - review carefully before executing
SELECT c.CustomerID, p.FirstName + ' ' + p.LastName AS CustomerName,
       COUNT(o.OrderID) AS TotalOrders
FROM customer_table c
JOIN person_table p
ON c.PersonID = p.PersonID
JOIN order_table o
ON c.CustomerID = o.CustomerID
WHERE o.OrderDate >= @StartDate AND o.OrderDate <= @EndDate
GROUP BY c.CustomerID, p.FirstName, p.LastName
ORDER BY TotalOrders DESC;
```

AdventureWorks2025 실제 예시

```sql
-- Created by GitHub Copilot in SSMS - review carefully before executing
SELECT c.CustomerID, p.FirstName + ' ' + p.LastName AS CustomerName,
       COUNT(soh.SalesOrderID) AS TotalOrders
FROM Sales.Customer c
LEFT JOIN Person.Person p
ON c.PersonID = p.BusinessEntityID
LEFT JOIN Sales.SalesOrderHeader soh
ON c.CustomerID = soh.CustomerID
AND soh.OrderDate >= DATEADD(day, -14, '2026-03-26')
AND DATENAME(weekday, soh.OrderDate) NOT IN ('Saturday', 'Sunday')
GROUP BY c.CustomerID, p.FirstName, p.LastName
HAVING COUNT(soh.SalesOrderID) > 0
ORDER BY TotalOrders DESC;
```

위 쿼리는 최근 14일 중 주말을 제외한 날짜(영업일 기준)로 주문을 집계합니다.
실제 데이터가 없을 경우 결과가 없을 수 있습니다.
필요에 따라 날짜 계산 로직을 조정할 수 있습니다.

5. 출력 형식 정의

■ 비권장

· Model

GPT-4.1

· Prompt

> 가장 바쁜 영업사원 알려줘.

· 답변

"가장 바쁜 영업사원"을 찾으려면, "바쁨"의 기준이 필요합니다.

예를 들어,
가장 많은 매출(판매 금액)
가장 많은 거래(판매 건수)
특정 기간 내 실적
등이 있습니다.

어떤 기준으로 집계할까요?
예시: "가장 많은 매출을 올린 영업사원" 또는 "가장 많은 거래를 한 영업사원" 등
원하시는 기준을 알려주시면 정확한 결과를 제공하겠습니다.

권장

· Model

GPT-4.1

· Prompt

> 가장 많은 매출을 기록한 상위 10명의 영업사원을 표로 보여줘. 표에는 SalesPersonID, Name, TotalSalesAmount 컬럼이 포함되어야해.

· 답변

아래는 가장 많은 매출을 기록한 상위 10명의 영업사원 목록입니다.

SalesPersonID	Name	TotalSalesAmount
276	Linda Mitchell	4,251,368.55
289	Jae Pak	4,116,871.23
275	Michael Blythe	3,763,178.18
277	Jillian Carson	3,189,418.37
290	Ranjit Varkey Chudukatil	3,121,616.32
282	José Saraiva	2,604,540.72
281	Shu Ito	2,458,535.62
279	Tsvi Reiter	2,315,185.61
288	Rachel Valdez	1,827,066.71
284	Tete Mensa-Annan	1,576,562.20

각 영업사원의 ID, 이름, 총 매출액이 포함되어 있습니다.

실제 운영 환경에서 자주 발생하는 성능 이슈를 바탕으로, 핵심 문제만을 추려 단순화한 SQL을 중심으로 구성되어 있다. 각 사례는 먼저 사람이 직접 성능을 분석하고 최적화하는 과정을 살펴보고, 이어서 GitHub Copilot 채팅을 활용한 AI 기반 최적화 결과를 비교하는 방식으로 진행된다. 이를 통해 두 접근 방식의 차이와 공통점을 이해하고, GitHub Copilot 채팅이 실무에서도 활용 가능한 수준의 신뢰도를 갖추고 있음을 확인할 수 있다. 독자는 이러한 흐름을 따라가며 성능 최적화의 사고 과정을 간접적으로 체험하고, 이를 실제 업무에 적용할 수 있는 인사이트를 얻을 수 있다.

* 모든 실습은 SQL Server 2017(RTM, 14.0.1000)에서 진행되었다.

실전 사례집

03

Section 1. 인덱스와 접근 경로 최적화 34

Section 2. 실행 계획 및 연산 처리 최적화 106

Section 3. 쿼리 구조 및 서브쿼리 최적화 134

Section 4. 변수 및 임시 객체 활용 이슈 170

Section 5. 커서와 절차형 처리 개선 196

Section 6. 집계 및 함수 처리 최적화 216

Section 7. 동시성 및 잠금 287

Section 8. DML 최적화 325

Section 9. 통계 정보 351

Section 10. 파티션 및 원격 쿼리 370

Section 01

인덱스와 접근 경로 최적화

> 쿼리 성능의 출발점은 적절한 인덱스와 효율적인 접근 경로 선택이다. 이 섹션에서는 인덱스 설계와 활용 방식, 그리고 옵티마이저의 접근 경로 선택을 중심으로 성능 개선 방법을 살펴본다.

1 RID LOOKUP에 의한 성능 이슈 - 1

구문에 사용된 컬럼의 일부가 인덱스에 존재하지 않아 RID Lookup 과정에서 많은 I/O가 발생한 사례이다.

1. 사전 구성 스크립트

```
SELECT rownum AS seq
      , CASE WHEN rownum/999<1 THEN 'SE'
      WHEN rownum/999<14 THEN 'CS'
      WHEN rownum/999<26 THEN 'QA' ELSE 'SA'
END AS class_cd
      , NEWID() AS main_code
      , DATEADD(ss, rownum, '2020-01-01 00:00:00.000') AS date
      , NEWID() AS no2 INTO TB_Maind5
FROM (SELECT ROW_NUMBER() OVER (ORDER BY a.number) AS rownum, a.number
FROM master..spt_values a
JOIN master..spt_values b ON a.type = 'P' AND b.type = 'P'
AND a.number <=1000 AND b.number <= 1000 ) a

CREATE INDEX IDX_Maind501 ON TB_Maind5(class_cd)
GO
```

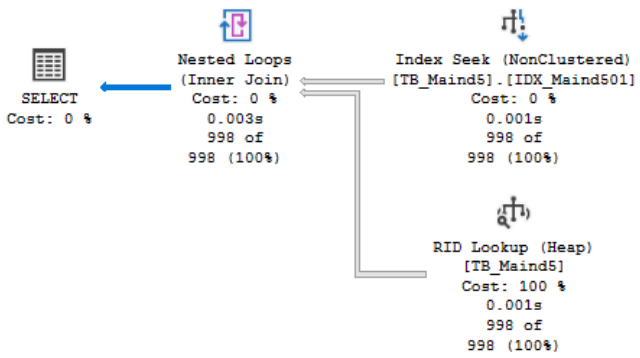
2. [AS-IS] SQL 구문 및 실행 계획

1) SQL 구문

```
SELECT seq, class_cd, main_code
FROM TB_Maind5
WHERE class_cd = 'SE'
GO
```

2) 실행 계획

Rows	Executes	StmtText
998	1	SELECT [seq],[class_cd],[main_code] FROM [TB_Maind5] WHERE [class_cd]=@1
998	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([Bmk1000], [Expr1004]) WITH UNORDERED PREFETCH)
998	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_Maind5].[IDX_Maind501]), SEEK:([Tuning].[dbo].[TB_Maind5].[class_cd]='SE') Ordered Forward)
998	998	¦--RID Lookup(OBJECT:([Tuning].[dbo].[TB_Maind5]), SEEK:([Bmk1000]=[Bmk1000]) LOOKUP Ordered Forward)




```
테이블 'TB_Maind5'. 스캔 수 1, 논리적 읽기 1005
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 2ms
```

3. [AS-IS] 통계 정보

- **인덱스 정보**

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Maind5	IDX_Maind501	NONCLUSTERED	0	class_cd	-

4. [AS-IS] 구문 설명

[TB_Maind5] 테이블에서 [class_cd] 컬럼 값이 'SE' 인 데이터를 조회하는 구문이다. 조건절에 사용된 [class_cd] 컬럼을 인덱스로 탐색하지만 SELECT 절에서 요구하는 [seq], [main_code] 컬럼이 사용된 인덱스에 포함되어 있지 않아 탐색된 결과 집합 수만큼 RID Lookup이 반복 수행되면서 높은 I/O 비용이 발생된다.

5. [TO-BE] 튜닝 포인트

RID Lookup은 구문에서 요구하는 컬럼들 중 일부가 인덱스에 존재하지 않을 때 테이블에서 부족한 컬럼을 가져오기 위해 수행된다. 인덱스로 탐색된 결과 집합 수만큼 NL Join으로 반복 수행되기 때문에 탐색된 행이 많을수록 높은 I/O를 유발한다. 요구되는 컬럼을 인덱스의 포괄(Include) 열로 추가하면 RID Lookup의 과정이 제거되기 때문에 I/O를 감소시킬 수 있다.

6. [TO-BE] 개선 방안

1) 포괄(Include) 열 인덱스 생성

RID Lookup 연산이 요구되지 않고 인덱스 탐색으로만 데이터를 조회할 수 있도록 한다.

■ 인덱스 생성 구문

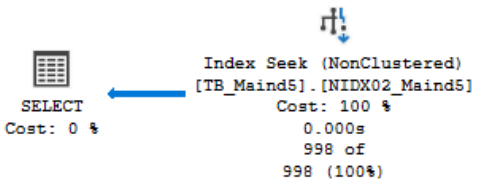
```
CREATE INDEX NIDX02_Maind5 ON TB_Maind5(class_cd) INCLUDE (seq, main_code)
GO
```

■ SQL 구문

```
SELECT seq, class_cd, main_code
FROM TB_Maind5
WHERE class_cd = 'SE'
GO
```

· 실행 계획

Rows	Executes	StmtText
998 998	1 1	SELECT [seq],[class_cd],[main_code] FROM [TB_Maind5] WHERE [class_cd]=@1 |--Index Seek(OBJECT:([tuning].[dbo].[TB_Maind5].[NIDX02_Maind5]), SEEK:([tuning].[dbo].[TB_Maind5].[class_cd]='SE') Ordered Forward)




```
테이블 'TB_Maind5'. 스캔 수 1, 논리적 읽기 9
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	0	2	1,005
TO-BE	0	0	9

AI GitHub Copilot 채팅 결과

Claude Sonnet 4.5

1. RID LOOKUP에 의한 성능 이슈 - 1

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT seq, class_cd, main_code
FROM TB_Maind5
WHERE class_cd = 'SE';
```

2. 분석 및 주요 최적화

1) 커버링 인덱스 생성

· 현재 IDX_Maind501은 class_cd만 포함
· seq, main_code를 INCLUDE 절에 추가하면 Key Lookup 방지
· 인덱스만으로 처리 가능 (테이블 접근 불필요)

2) 성능 향상 예상

· 현재: Index Seek + Key Lookup (테이블 접근)
· 개선 후: Index Seek만으로 완료 (I/O 감소)

3. 개선안

```
-- 더 나은 성능을 위한 커버링 인덱스 생성 (권장)

CREATE NONCLUSTERED INDEX IDX_Maind502
ON dbo.TB_Maind5(class_cd)
INCLUDE (seq, main_code);
```

2 RID LOOKUP에 의한 성능 이슈 - 2

구문에 사용된 컬럼의 일부가 인덱스에 존재하지 않아 RID Lookup이 수행되면서 높은 I/O를 사용한 사례이다.

1. 사전 구성 스크립트

```
SELECT CASE WHEN LEFT(CONVERT(VARCHAR,a.number),1) <= 1 THEN 'A' ELSE 'B' END ID
      , CONVERT(VARCHAR,a.number) ProductNumber
      , CONVERT(BIGINT,b.number)*100 AS Price
      , CONVERT(date,DATEADD(DD,a.number+b.number,'2010-01-01'),0) ModifiedDate
INTO TB_RIDLookup02
FROM master..spt_values a, (
      SELECT number
      FROM master..spt_values
      WHERE type = 'P'
) b
WHERE a.type = 'P'

CREATE NONCLUSTERED INDEX NIDX01_RIDLookup02 ON TB_RIDLookup02(ModifiedDate)
GO
```

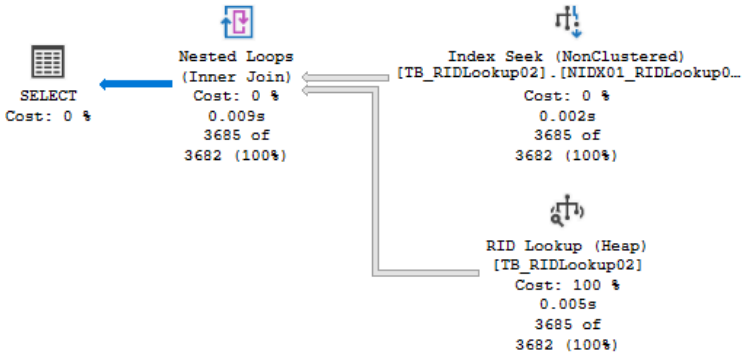
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT ProductNumber, Price, ModifiedDate
FROM TB_RIDLookup02
WHERE ModifiedDate IN ('2015-01-01', '2015-02-01')
GO
```

· 실행 계획

Rows	Executes	StmtText
3685	1	SELECT ProductNumber, Price, ModifiedDate FROM TB_RIDLookup02 WHERE ModifiedDate IN ('2015-01-01', '2015-02-01')
3685	1	┦--Nested Loops(INNER JOIN, OUTER REFERENCES:([Bmk1000], [Expr1004]) WITH UNORDERED PREFETCH)
3685	1	┦--Index Seek(OBJECT:([Tuning].[dbo].[TB_RIDLookup02].[NIDX01_RIDLookup02]), SEEK:([Tuning].[dbo].[TB_RIDLookup02].[ModifiedDate]='2015-01-01' OR Tuning].[dbo].[TB_RIDLookup02].[ModifiedDate]='2015-02-01') Ordered Forward)
3685	3685	┦--RID Lookup(OBJECT:([Tuning].[dbo].[TB_RIDLookup02]), SEEK:([Bmk1000]=[Bmk1000]) LOOKUP Ordered Forward)




```
테이블 'TB_RIDLookup02'. 스캔 수 2, 논리적 읽기 3701
SQL Server 실행 시간:
CPU 시간 = 16ms, 경과 시간 = 27ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_RIDLookup02	NIDX01_RIDLookup02	NONCLUSTERED	0	ModifiedDate	-

4. [AS-IS] 구문 설명

[TB_RIDLookup02] 테이블에서 [ModifiedDate] 컬럼 값이 '2015-01-01', '2015-02-01' 인 데이터를 조회하는 구문이다. 조건절에 사용된 [ModifiedDate] 컬럼을 인덱스로 탐색하지만 SELECT 절에서 요구하는 [Price], [ProductNumber] 컬럼이 사용된 인덱스에 포함되어 있지 않아 탐색된 결과 집합 수만큼 RID Lookup이 반복 수행되면서 높은 I/O 비용이 발생된다.

5. [TO-BE] 튜닝 포인트

조건절을 비클러스터형 인덱스로 탐색한 3,685건 결과 집합에 대해 힙 테이블로 RID Lookup을 반복 수행하면서 많은 I/O가 사용된다. 조건절에 사용한 컬럼을 키로 하여 클러스터형 인덱스를 생성하면 Leaf Page에 모든 컬럼이 포함되기 때문에 RID Lookup 연산에 의한 비용을 제거할 수 있다.

6. [TO-BE] 개선 방안

1) [ModifiedDate] 컬럼이 키로 구성된 클러스터형 인덱스 생성

클러스터형 인덱스를 통해 RID Lookup 연산에 의한 비용이 제거되어 I/O가 개선된다.

■ 인덱스 생성 구문

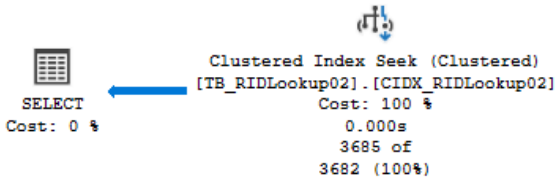
```
DROP INDEX NIDX01_RIDLookup02 ON TB_RIDLookup02
CREATE CLUSTERED INDEX CIDX_RIDLookup02 ON TB_RIDLookup02(ModifiedDate)
GO
```

■ SQL 구문

```
SELECT ProductNumber, Price, ModifiedDate
FROM TB_RIDLookup02
WHERE ModifiedDate IN ('2015-01-01', '2015-02-01')
GO
```

· 실행 계획

Rows	Executes	StmtText
3685	1	SELECT ProductNumber, Price, ModifiedDate FROM TB_RIDLookup02 WHERE ModifiedDate IN ('2015-01-01', '2015-02-01')
3685	1	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_RIDLookup02].[CIDX_RIDLookup02]), SEEK:([Tuning].[dbo].[TB_RIDLookup02].[ModifiedDate]='2015-01-01' OR [Tuning].[dbo].[TB_RIDLookup02].[ModifiedDate]='2015-02-01') Ordered Forward)




```
테이블 'TB_RIDLookup02'. 스캔 수 2, 논리적 읽기 25
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 1ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	16	27	3,701
TO-BE	0	1	25

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT ProductNumber, Price, ModifiedDate
FROM TB_RIDLookup02
WHERE ModifiedDate IN ('2015-01-01', '2015-02-01');
```

2. 분석 및 주요 최적화

1) 현재 테이블/인덱스 구조

항목	값
테이블	dbo.TB_RIDLookup02
구조	HEAP (클러스터드 인덱스 없음)
행 수	약 419만 건
넌클러스터드 인덱스	NIDX01_RIDLookup02 (ModifiedDate)

컬럼	데이터 타입
ID	varchar(1)
ProductNumber	varchar
Price	bigint
ModifiedDate	date

2) 비효율 원인 분석

인덱스 NIDX01_RIDLookup02는 ModifiedDate만 포함합니다. 쿼리는 ProductNumber, Price도 SELECT하므로, 인덱스에서 조건에 맞는 행을 찾은 뒤 매 행마다 HEAP으로 돌아가서 나머지 컬럼을 읽어야 합니다 (*RID Lookup, 가장 큰 비효율).

* HEAP 테이블의 RID Lookup은 클러스터드 인덱스의 Key Lookup보다 I/O 비용이 더 크며, 조건에 매칭되는 행이 많을수록 성능이 급격히 저하됩니다.

3. 개선안

```
-- 커버링 인덱스 생성 (RID Lookup 제거)
-- 기존 인덱스에 ProductNumber와 Price를 **포함 열(INCLUDE)**로 추가하면,
-- 인덱스만으로 쿼리를 완전히 처리할 수 있습니다 (Index Only Scan).

-- 기존 인덱스를 커버링 인덱스로 재생성

DROP INDEX NIDX01_RIDLookup02 ON dbo.TB_RIDLookup02;

CREATE NONCLUSTERED INDEX NIDX01_RIDLookup02
ON dbo.TB_RIDLookup02 (ModifiedDate)
INCLUDE (ProductNumber, Price);
```

3 KEY LOOKUP에 의한 성능 이슈

테이블 조회를 위해 인덱스로 탐색할 때 일부 컬럼이 인덱스에 포함되지 않아 Key Lookup이 발생하여 I/O가 증가되는 사례이다.

1. 사전 구성 스크립트

```
SELECT ROW_NUMBER() OVER (ORDER BY a.number) AS seq
      , a.type
      , b.number%10 AS no1
      , a.number*b.number AS no2
      , newid() AS tid
INTO TB_Keyup1
FROM master..spt_values a JOIN master..spt_values b
ON a.type = b.type
WHERE a.number BETWEEN 1 AND 1000
AND b.number BETWEEN 1 AND 1000

CREATE CLUSTERED INDEX CIDX_Keyup1 on TB_Keyup1(seq)
CREATE INDEX NIDX01_Keyup1 on TB_Keyup1(no1, type)
GO
```

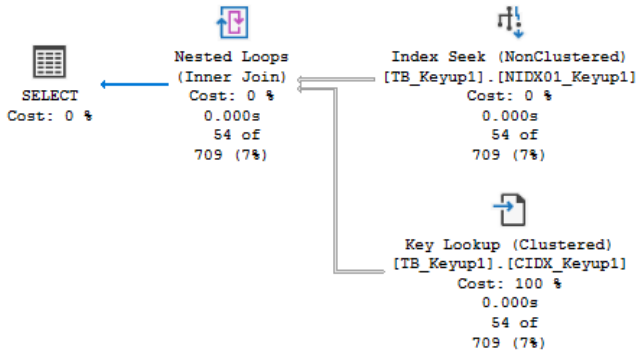
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT type, no1, tid
FROM TB_Keyup1
WHERE no1 = 4 AND type = 'J'
GO
```

· 실행 계획

Rows	Executes	StmtText
54	1	SELECT [type],[no1],[tid] FROM [TB_Keyup1] WHERE [no1]=@1 AND [type]=@2
54	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([Uniq1001], [tuning].[dbo].[TB_Keyup1].[seq], [Expr1005]) WITH UNORDERED PREFETCH)
54	1	¦--Index Seek(OBJECT:([tuning].[dbo].[TB_Keyup1].[NIDX01_Keyup1]), SEEK:([tuning].[dbo].[TB_Keyup1].[no1]=(4) AND [tuning].[dbo].[TB_Keyup1].[type]=N'J') Ordered Forward)
54	54	¦--Clustered Index Seek(OBJECT:([tuning].[dbo].[TB_Keyup1].[CIDX_Keyup1]), SEEK:([tuning].[dbo].[TB_Keyup1].[seq]=[tuning].[dbo].[TB_Keyup1].[seq] AND [Uniq1001]=[Uniq1001]) LOOKUP Ordered Forward)




```
테이블 'TB_Keyup1'. 검색 수 1, 논리적 읽기 수 177
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Keyup1	CIDX_Keyup1	CLUSTERED	0	seq	-
TB_Keyup1	NIDX01_Keyup1	NONCLUSTERED	0	no1, type	-

4. [AS-IS] 구문 설명

[TB_Keyup1] 테이블에서 [no1] 컬럼 값이 4이면서 [type] 컬럼 값이 'J' 인 데이터를 조회하는 구문이다. [NIDX01_Keyup1] 인덱스를 이용한 탐색을 수행하지만, SELECT 절에서 요구하는 [tid] 컬럼이 인덱스에 포함되어 있지 않다. 따라서 탐색된 결과 집합 수만큼 Key Lookup이 반복 수행되면서 높은 I/O 비용이 발생된다.

5. [TO-BE] 튜닝 포인트

Key Lookup은 인덱스를 탐색한 뒤 부족한 컬럼을 가져오기 위해 클러스터형 인덱스에 조인하는 작업이다. 이때 NL Join이 반복 수행되기 때문에 인덱스로 탐색된 행이 많을수록 높은 I/O가 사용된다. 인덱스에 포함되어 있지 않은 [tid] 컬럼을 인덱스의 포괄(Include) 열로 추가하면 필요한 모든 컬럼이 인덱스 내에 포함되기 때문에 Key Lookup의 과정이 제거되어 I/O를 감소시킬 수 있다.

6. [TO-BE] 개선 방안

1) 포괄(Include) 열이 있는 인덱스 생성

[tid] 컬럼을 인덱스 포괄 열로 포함하면 필요한 모든 컬럼이 인덱스에 포함되기 때문에 Key Lookup 과정 없이 인덱스 탐색으로만 수행된다.

■ 인덱스 생성 구문

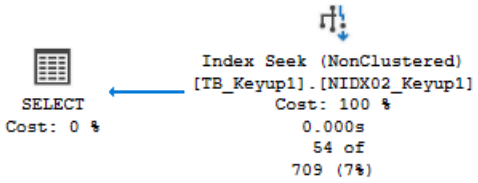
```
CREATE INDEX NIDX02_Keyup1 ON TB_Keyup1(no1, type) INCLUDE (tid)
GO
```

■ SQL 구문

```
SELECT type, no1, tid
FROM TB_Keyup1
WHERE no1 = 4 AND type = 'J'
GO
```

· 실행 계획

Rows	Executes	StmtText
54 54	1 1	SELECT [type],[no1],[tid] FROM [TB_Keyup1] WHERE [no1]=@1 AND [type]=@2 ¦--Index Seek(OBJECT:([tuning].[dbo].[TB_Keyup1].[NIDX02_Keyup1]), SEEK:([tuning].[dbo].[TB_Keyup1].[no1]=CONVERT_IMPLICIT(int,[@1],0) AND [tuning].[dbo].[TB_Keyup1].[type]=CONVERT_IMPLICIT(nvarchar(4000),[@2],0)) Ordered Forward)




```
테이블 'TB_Keyup1'. 검색 수 1, 논리적 읽기 수 3
 SQL Server 실행 시간:
 CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	0	0	177
TO-BE	0	0	3

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT type, no1, tid
FROM TB_Keyup1
WHERE no1 = 4 AND type = 'J';
```

2. 분석 및 주요 최적화

1) 현재 상태

- 테이블: dbo.TB_keyup1
- 기존 인덱스: NIDX01_Keyup1 (no1, type)
- WHERE 조건: no1 = 4 AND type = 'J'

2) 주요 이슈

- 현재 쿼리는 이미 적절한 인덱스(NIDX01_Keyup1)를 활용할 수 있지만, Key Lookup 문제가 있습니다.
- tid 컬럼이 인덱스에 포함되어 있지 않아 클러스터드 인덱스를 추가로 조회해야 합니다.

3. 개선안

```
-- tid를 INCLUDE 컬럼으로 추가한 Covering Index 생성

DROP INDEX NIDX01_Keyup1 ON dbo.TB_keyup1;

CREATE NONCLUSTERED INDEX NIDX01_Keyup1
ON dbo.TB_keyup1(no1, type)
INCLUDE (tid);
```

4 인덱스 부재로 인한 성능 이슈

인덱스 부재로 인해 테이블을 전체 스캔하면서 I/O가 발생하는 사례이다.

1. 사전 구성 스크립트

```
SELECT rownum AS seq
      , CASE WHEN rownum/3=0 THEN 'SE' WHEN rownum/3=1 THEN 'CS'
      WHEN rownum/3=2 THEN 'QA' ELSE 'SA' END AS class_cd
      , NEWID() AS main_code
      , DATEADD(ss, rownum, '2020-01-01 00:00:00.000') AS date
      , NEWID() AS no2 INTO TB_Maind4
FROM (SELECT ROW_NUMBER() OVER (ORDER BY a.number) AS rownum, a.number
      FROM master..spt_values a
      JOIN master..spt_values b ON a.type = 'P' AND b.type = 'P'
      AND a.number <=1000 AND b.number <= 1000
) a
GO
```

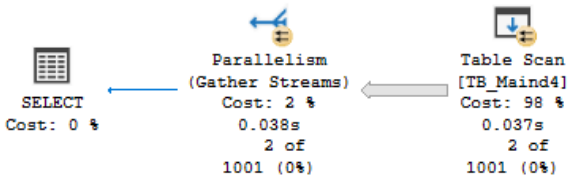
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT seq, class_cd, main_code
FROM TB_Maind4
WHERE class_cd = 'SE'
GO
```

· 실행 계획

Rows	Executes	StmtText
2 2 2	1 1 8	SELECT [seq],[class_cd],[main_code] FROM [TB_Maind4] WHERE [class_cd]=@1 ¦--Parallelism(Gather Streams) ¦--Table Scan(OBJECT:([tuning].[dbo].[TB_Maind4]), WHERE:([tuning].[dbo].[TB_Maind4].[class_cd]=[@1]))




```
테이블 'TB_Maind4'. 스캔 수 9, 논리적 읽기 7833
SQL Server 실행 시간:
CPU 시간 = 236ms, 경과 시간 = 34ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보 없음

4. [AS-IS] 구문 설명

[TB_Maind4] 테이블에서 [class_cd] 컬럼 값이 'SE' 인 데이터를 조회하는 구문이다. [TB_Maind4] 테이블의 전체 데이터는 100만 건이며 이 중 [class_cd] = 'SE' 조건에 부합하는 데이터는 2건에 해당한다. 조건절에 사용된 [class_cd] 컬럼이 키로 구성된 인덱스가 존재하지 않기 때문에 100만 건의 데이터 전체를 스캔하면서 높은 I/O 비용이 발생된다.

5. [TO-BE] 튜닝 포인트

인덱스 없이 테이블을 스캔할 경우 테이블의 모든 데이터를 읽고 조건절에 일치하는 값을 추출한다. 이때 불필요한 데이터까지 모두 읽게 되어 높은 I/O 비용이 발생한다. 조건절에 있는 컬럼 기준으로 필요한 데이터만 읽도록 인덱스를 생성하면 I/O를 감소시킬 수 있다.

6. [TO-BE] 개선 방안

1) [class_cd] 컬럼이 키로 구성된 신규 인덱스 생성

조건에 해당하는 데이터만 읽도록 [class_cd] 컬럼이 키로 구성된 인덱스를 생성한다.

■ 인덱스 생성 구문

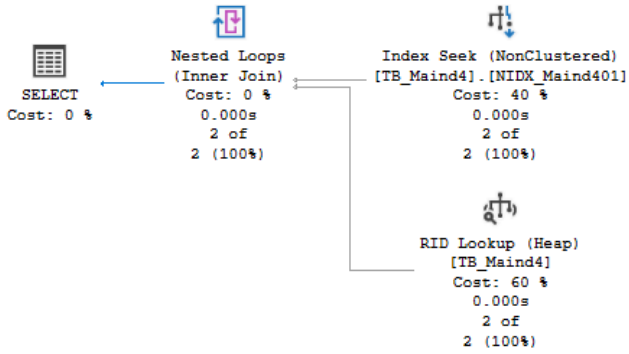
```
CREATE INDEX NIDX_Maind401 ON TB_Maind4(class_cd)
GO
```

■ SQL 구문

```
SELECT seq, class_cd, main_code
FROM TB_Maind4
WHERE class_cd = 'SE'
GO
```

· 실행 계획

Rows	Executes	StmtText
2	1	SELECT [seq],[class_cd],[main_code] FROM [TB_Maind4] WHERE [class_cd]=@1
2	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([Bmk1000]))
2	1	¦--Index Seek(OBJECT:([tuning].[dbo].[TB_Maind4].[NIDX_Maind401]), SEEK:([tuning].[dbo].[TB_Maind4].[class_cd]='SE') Ordered Forward)
2	2	¦--RID Lookup(OBJECT:([tuning].[dbo].[TB_Maind4]), SEEK:([Bmk1000]=[Bmk1000]) LOOKUP Ordered Forward)




```
테이블 'TB_Maind4'. 스캔 수 1, 논리적 읽기 5
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	236	34	7,833
TO-BE	0	0	5

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

4. 인덱스 부재로 인한 성능 이슈

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT seq, class_cd, main_code
FROM TB_Maind4
WHERE class_cd = 'SE';
```

2. 분석 및 주요 최적화

1) 현재 테이블/인덱스 구조

항목	값
테이블	dbo.TB_Maind4
총 행 수	1,002,001
인덱스	없음 (Heap 테이블)

2) class_cd 데이터 분포

class_cd	건수	비율
SA	1,001,993	99.999%
QA	3	~0%
CS	3	~0%
SE	2	~0%

3) 문제점

class_cd = 'SE'는 1,002,001건 중 단 2건만 해당하는 매우 선택적인 조건입니다. 그러나 인덱스가 전혀 없어 매번 100만 건 이상을 Full Table Scan 해야 합니다.

3. 개선안

```
-- Covering Nonclustered Index를 생성하면, Index Seek로 2건만 직접 찾아갈
-- 수 있습니다.
-- seq와 main_code를 INCLUDE 컬럼에 포함시켜 테이블 접근(Key Lookup) 없이
-- 인덱스만으로 결과를 반환하는 커버링 인덱스를 만듭니다.

CREATE NONCLUSTERED INDEX IX_TB_Maind4_class_cd
ON dbo.TB_Maind4 (class_cd) INCLUDE (seq, main_code);
```

· 예상 효과

항목	현재 (인덱스 없음)	인덱스 적용 후
액세스 방식	Table Scan (1,002,001건 전체)	Index Seek (2건만)
논리적 I/O	매우 높음	극소량
커버링 여부	N/A	✔ (Key Lookup 없음)

5 인덱스 키 누락으로 인한 성능 이슈

조건절에서 사용된 컬럼의 일부가 인덱스에 포함되지 않아 스캔 범위가 많아지면서 불필요한 I/O가 발생되는 사례이다.

1. 사전 구성 스크립트

```
SELECT number, type, status, LEFT(type,1)+CAST(number AS VARCHAR) AS uid
INTO TB_Store
FROM master..spt_values
WHERE type = 'P' AND number BETWEEN 1 AND 10

SELECT *, DATEADD(HH, -a.seq, '2020-09-19 00:00:00.000') AS logtime
INTO TB_Storelog
FROM (SELECT ROW_NUMBER() OVER (ORDER BY b.number) AS seq
      FROM master..spt_values b
      JOIN master..spt_values c
      ON b.type = c.type
      WHERE b.type = 'P' AND b.number BETWEEN 1 AND 1000
            AND c.type = 'P' AND c.number BETWEEN 1 AND 100
) a, TB_Store

CREATE INDEX NIDX01_Storelog ON TB_Storelog(number) include(logtime)
GO
```

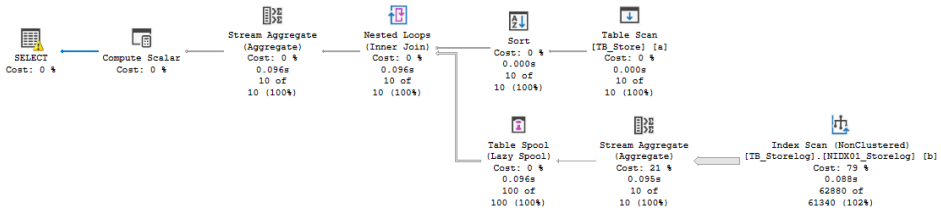
2. [AS-IS] SQL 구문 및 실행 계획

· SQL 구문

```
SELECT a.uid, COUNT(b.number) AS cnt
FROM TB_Store a JOIN TB_Storelog b ON a.number = b.number
WHERE b.logtime BETWEEN '2020-01-01' AND '2020-12-31'
GROUP BY a.uid
GO
```

· 실행 계획

Rows	Executes	StmtText
10	1	SELECT a.uid, COUNT(b.number) AS cnt FROM TB_Store a JOIN TB_Storelog b ON a.number = b.number WHERE b.logtime BETWEEN '2020-01-01' AND '2020-12-31' GROUP BY a.uid
0	0	¦--Compute Scalar(DEFINE:([Expr1004]=CONVERT_IMPLICIT(int,[globalagg1006],0)))
10	1	¦--Stream Aggregate(GROUP BY:([a].[uid]) DEFINE:([globalagg1006]=SUM([partialagg1005])))
10	1	¦--Nested Loops(INNER JOIN, WHERE:([tuning].[dbo].[TB_Storelog].[number] as [b].[number] =[tuning].[dbo].[TB_Store].[number] as [a].[number]))
10	1	¦--Sort(ORDER BY:([a].[uid] ASC))
10	1	¦ ¦--Table Scan(OBJECT:([tuning].[dbo].[TB_Store] AS [a]))
100	10	¦--Table Spool
10	1	¦--Stream Aggregate(GROUP BY:([b].[number]) DEFINE:([partialagg1005]=Count(*)))
62880	1	¦--Index Scan(OBJECT:([tuning].[dbo].[TB_Storelog].[NIDX01_Storelog] AS [b]), WHERE:([tuning].[dbo].[TB_Storelog].[logtime] as [b].[logtime]>='2020-01-01 00:00:00.000' AND [tuning].[dbo].[TB_Storelog].[logtime] as [b].[logtime]<='2020-12-31 00:00:00.000') Ordered Forward)




```
테이블 'TB_Storelog'. 검색 수 1, 논리적 읽기 수 3227
테이블 'Worktable'. 검색 수 1, 논리적 읽기 수 39
테이블 'TB_Store'. 검색 수 1, 논리적 읽기 수 1
SQL Server 실행 시간:
CPU 시간 = 94ms, 경과 시간 = 105ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Storelog	NIDX01_Storelog	NONCLUSTERED	0	number	logtime

4. [AS-IS] 구문 설명

[TB_Store] 테이블 10건의 데이터를 [TB_Storelog] 테이블의 2020년도 데이터와 조인하여 [uid] 컬럼을 기준으로 집계한 데이터 행 수를 조회하는 구문이다. [TB_Storelog] 테이블의 조인 조건으로 사용된 [number] 컬럼에 인덱스가 존재함에도 전체 데이터를 읽어내면서 불필요한 I/O를 사용하는데, 원인은 [number] 컬럼으로만 인덱스 탐색을 하게 되면 [logtime] 조건절에 의해 결과 집합에 포함되지 않는 불필요한 데이터를 반복적으로 읽어내기 때문이다. 이를 대신하여 처음 조인이 시도될 때 전체 데이터를 [number] 컬럼을 기준으로 미리 집계하고 Spool에 저장하여 이후 조인에 대해선 Spool과 조인하도록 수행된 것이다.

5. [TO-BE] 튜닝 포인트

[TB_Storelog] 테이블의 조건절로 사용된 [logtime] 컬럼이 인덱스 키에 포함되지 않아 탐색 범위가 많아지는 것이 원인이다. 조인 조건과 조건절이 모두 키로 구성된 인덱스를 생성하면 필요한 범위만 읽어내면서 I/O를 개선할 수 있다.

6. [TO-BE] 개선 방안

1) [TB_Storelog] 테이블의 [number] 컬럼과 [logtime] 컬럼이 키로 구성된 신규 인덱스 생성

[number], [logtime] 컬럼이 키로 구성된 인덱스를 생성하면 조인이 수행될 때마다 필요한 범위만 인덱스로 탐색하기 때문에 I/O가 개선된다.

■ 인덱스 생성 구문

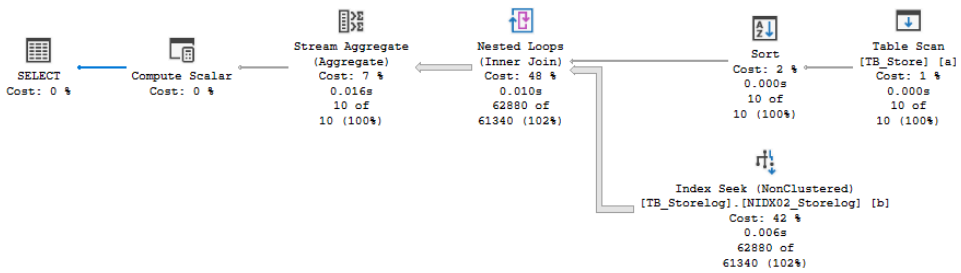
```
CREATE INDEX NIDX02_Storelog ON TB_Storelog(number, logtime)
GO
```

■ SQL 구문

```
SELECT a.uid, COUNT(b.number) AS cnt
FROM TB_Store a
JOIN TB_Storelog b ON a.number = b.number
WHERE b.logtime BETWEEN '2020-01-01' AND '2020-12-31'
GROUP BY a.uid
GO
```

· 실행 계획

Rows	Executes	StmtText
10	1	SELECT a.uid, COUNT(b.number) AS cnt FROM TB_Store a JOIN TB_Storelog b ON a.number = b.number WHERE b.logtime BETWEEN '2020-01-01' AND '2020-12-31' GROUP BY a.uid
0	0	¦--Compute Scalar(DEFINE:([Expr1004]=CONVERT_IMPLICIT(int,[Expr1007],0)))
10	1	¦--Stream Aggregate(GROUP BY:([a].[uid]) DEFINE:([Expr1007]=Count(*)))
62880	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([a].[number]))
10	1	¦--Sort(ORDER BY:([a].[uid] ASC))
10	1	¦ ¦--Table Scan(OBJECT:([tuning].[dbo].[TB_Store] AS [a]))
62880	10	¦--Index Seek(OBJECT:([tuning].[dbo].[TB_Storelog].[NIDX02_Storelog] AS [b]), SEEK: ([b].[number]=[tuning].[dbo].[TB_Store].[number] as [a].[number] AND [b].[logtime] >= '2020-01-01 00:00:00.000' AND [b].[logtime] <= '2020-12-31 00:00:00.000') Ordered Forward)




```
테이블 'TB_Storelog'. 검색 수 10, 논리적 읽기 수 242
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_Store'. 검색 수 1, 논리적 읽기 수 1
SQL Server 실행 시간:
CPU 시간 = 15ms, 경과 시간 = 15ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	94	105	3,267
TO-BE	15	15	243

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

5. 인덱스 키 누락으로 인한 성능 이슈

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT a.uid, COUNT(b.number) AS cnt
FROM TB_Store a JOIN TB_Storelog b ON a.number = b.number
WHERE b.logtime BETWEEN '2020-01-01' AND '2020-12-31'
GROUP BY a.uid;
```

2. 분석 및 주요 최적화

1) 분석 결과 테이블 현황

항목	dbo.TB_Store	dbo.TB_Storelog
행 수	10	1,000,000
저장 구조	Heap (인덱스 없음)	Heap
기존 인덱스	없음	NIDX01_Storelog (number) INCLUDE (logtime)

2) WHERE 조건 선택도

항목	값
전체 행	1,000,000
2020년 해당 행	62,880 (약 6.3%)

3) 현재 문제점

기존 인덱스 구조 비효율: NIDX01_Storelog는 number가 키, logtime이 INCLUDE 컬럼입니다. JOIN 시 number로 Seek한 후 logtime 필터는 **잔여 조건(Residual Predicate)**으로 처리되어, number당 약 100,000건을 읽고 나서 걸러냅니다.

3. 개선안

```
-- 기존 비효율 인덱스 제거 후 최적 인덱스 재생성

DROP INDEX IF EXISTS NIDX01_Storelog ON dbo.TB_Storelog;
CREATE NONCLUSTERED INDEX NIDX01_Storelog
ON dbo.TB_Storelog (number, logtime);
```

*logtime을 INCLUDE에서 키 컬럼으로 승격시켜 (number, logtime) 복합 인덱스로 변경합니다.

6 인덱스 페이지 분할에 의한 성능 이슈

인덱스 키값을 업데이트할 때 발생하는 페이지 분할로 인해 I/O가 증가되는 사례이다.

1. 사전 구성 스크립트

```
SELECT B.number AS NUM, CONVERT(NVARCHAR(10), A.number )AS CODE
INTO TB_PGSPLIT
FROM master..spt_values A, (SELECT number
                            FROM master..spt_values B
                             WHERE type ='P' AND number <= 500) B
WHERE A.type = 'P'

CREATE INDEX NIDX01_PGSPLIT ON TB_PGSPLIT(NUM, CODE)
GO
```

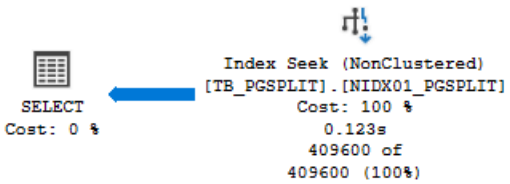
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT NUM, CODE
FROM TB_PGSPLIT
WHERE NUM>300

UPDATE A
SET CODE = ABS(NUM-CODE)
FROM TB_PGSPLIT A

SELECT NUM, CODE
FROM TB_PGSPLIT
WHERE NUM>300
GO
```

· 실행 계획 (UPDATE 전 SELECT 구문)

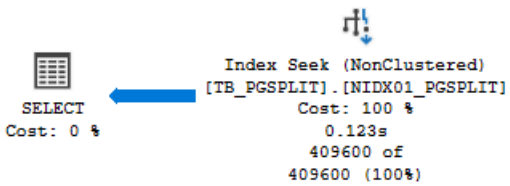
Rows	Executes	StmtText
409600 409600	1 1	SELECT [NUM],[CODE] FROM [TB_PGSPLIT] WHERE [NUM]>@1 ┆--Index Seek(OBJECT:([Tuning].[dbo].[TB_PGSPLIT].[NIDX01_PGSPLIT]), SEEK:([Tuning].[dbo].[TB_PGSPLIT].[NUM] > CONVERT_IMPLICIT(int,[@1],0)) Ordered Forward)



테이블 'TB_PGSPLIT'. 검색 수 1, 논리적 읽기 수 1482
SQL Server 실행 시간:
CPU 시간 = 125ms, 경과 시간 = 3377ms

· 실행 계획 (UPDATE 후 SELECT 구문)

Rows	Executes	StmtText
409600 409600	1 1	SELECT [NUM],[CODE] FROM [TB_PGSPLIT] WHERE [NUM]>@1 ┆--Index Seek(OBJECT:([Tuning].[dbo].[TB_PGSPLIT].[NIDX01_PGSPLIT]), SEEK:([Tuning].[dbo].[TB_PGSPLIT].[NUM] > CONVERT_IMPLICIT(int,[@1],0)) Ordered Forward)



테이블 'TB_PGSPLIT'. 검색 수 1, 논리적 읽기 수 4595
SQL Server 실행 시간:
CPU 시간 = 125ms, 경과 시간 = 3663ms

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_PGSPLIT	NIDX01_PGSPLIT	NONCLUSTERED	0	NUM, CODE	-

4. [AS-IS] 구문 설명

[TB_PGSPLIT] 테이블의 [NUM] 컬럼이 300 초과인 데이터를 조회할 때 [CODE] 컬럼을 업데이트하기 전과 후의 I/O를 비교하는 구문이다. UPDATE 수행 이후에 SELECT를 수행할 때 많은 양의 I/O가 사용되는데, 원인은 SELECT할 때 사용된 인덱스의 [CODE] 컬럼이 UPDATE로 인해 값이 변경될 때 정렬된 값을 유지하기 위해 페이지 중간에 삽입되면서 분할이 발생되고 이로 인해 페이지 양이 많아졌기 때문이다.

5. [TO-BE] 튜닝 포인트

[CODE] 컬럼은 SELECT 구문에서 조건절로 사용되지 않고 조회용으로만 사용되기 때문에 포괄 열로 하는 인덱스 생성을 고려할 수 있다. 포괄 열로 구성하게 되면 정렬을 하지 않기 때문에 UPDATE 구문에 의해 페이지 내에 중간에 삽입될 확률이 낮아지면서 페이지 분할 발생량이 적어진다.

6. [TO-BE] 개선 방안

1) [CODE] 컬럼을 포괄 열로 하는 인덱스를 생성한다.

[CODE] 컬럼을 포괄 열로 하는 [NIDX02_PGSPLIT] 인덱스를 신규로 생성한 뒤 업데이트하면 [CODE] 컬럼에 정렬이 필요하지 않아 페이지 분할이 적게 발생되고, 이후 SELECT 구문을 수행해도 해당 인덱스를 통해 적은 I/O가 사용된다.

■ 인덱스 생성 구문

```
CREATE INDEX NIDX02_PGSPLIT ON TB_PGSPLIT(NUM) INCLUDE (CODE)
GO
```

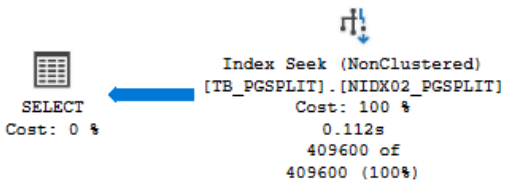
■ SQL 구문

```
SELECT NUM, CODE
FROM TB_PGSPLIT
WHERE NUM>300
```

```
UPDATE A
SET CODE = ABS(NUM-CODE)
FROM TB_PGSPLIT A

SELECT NUM, CODE
FROM TB_PGSPLIT
WHERE NUM>300
GO
```

· 실행 계획 (UPDATE 전 SELECT 구문)

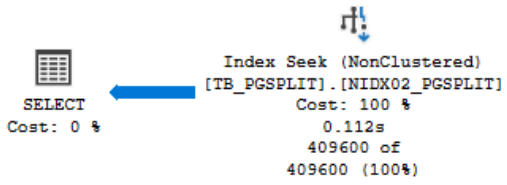
Rows	Executes	StmtText
409600 409600	1 1	SELECT [NUM],[CODE] FROM [TB_PGSPLIT] WHERE [NUM]>@1 ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_PGSPLIT].[NIDX02_PGSPLIT]), SEEK:([Tuning].[dbo].[TB_PGSPLIT].[NUM] > CONVERT_IMPLICIT(int,[@1],0)) Ordered Forward)



테이블 'TB_PGSPLIT'. 검색 수 1, 논리적 읽기 수 1475
SQL Server 실행 시간:
CPU 시간 = 62ms, 경과 시간 = 3399ms

· 실행 계획 (UPDATE 후 SELECT 구문)

Rows	Executes	StmtText
409600 409600	1 1	SELECT [NUM],[CODE] FROM [TB_PGSPLIT] WHERE [NUM]>@1 ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_PGSPLIT].[NIDX02_PGSPLIT]), SEEK:([Tuning].[dbo].[TB_PGSPLIT].[NUM] > CONVERT_IMPLICIT(int,[@1],0)) Ordered Forward)



테이블 'TB_PGSPLIT'. 검색 수 1, 논리적 읽기 수 1743
SQL Server 실행 시간:
CPU 시간 = 78ms, 경과 시간 = 3458ms

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	125	3,663	4,595
TO-BE	78	3,458	1,743

AI

GitHub Copilot 채팅 결과

Claude Opus 4.6

6. 인덱스 페이지 분할에 의한 성능 이슈

1. 프롬프트

> [AS-IS]
> 구문에 대해 성능을 극대화할 수 있는 최적화 방안을 제안해줘.

```
[AS-IS]
SELECT NUM,CODE
FROM TB_PGSPLIT
WHERE NUM>300

UPDATE A
SET CODE = ABS(NUM-CODE)
FROM TB_PGSPLIT A

SELECT NUM,CODE
FROM TB_PGSPLIT
WHERE NUM>300;
```

2. 분석 및 주요 최적화

1) 분석 결과 테이블 현황

항목	값
테이블	dbo.TB_PGSPLIT
총 행 수	1,026,048
저장 구조	Heap (클러스터드 인덱스 없음)
컬럼	NUM (int), CODE (nvarchar)
기존 인덱스	NIDX01_PGSPLIT (NUM, CODE) – Nonclustered

2) WHERE 조건 선택도

조건	해당 행 수	비율
NUM > 300	409,600	39.92%

3) 식별된 문제점

1 Heap 테이블 (클러스터드 인덱스 없음)

- 테이블이 Heap 상태이므로 UPDATE 시 RID Lookup 및 전방위 랜덤 I/O가 발생합니다.

100만 건 전체 UPDATE에 매우 불리합니다.

2 넌클러스터드 인덱스 키 컬럼 전체 갱신

- CODE가 인덱스 키 컬럼이므로, UPDATE 시 100만 건 전체에 대해 B-tree 키 재정렬이 필요합니다.
- INCLUDE 컬럼이라면 리프 페이지만 수정하면 되지만, 키 컬럼이면 위치 변경까지 수반됩니다.

3. 개선안

```sql
-- [1단계] 기존 비효율 넌클러스터드 인덱스 제거

DROP INDEX IF EXISTS NIDX01_PGSPLIT ON dbo.TB_PGSPLIT;


-- [2단계] 클러스터드 인덱스 생성 (Heap 제거)

CREATE CLUSTERED INDEX CIX_TB_PGSPLIT_NUM ON dbo.TB_PGSPLIT (NUM);


-- [3단계] 커버링 넌클러스터드 인덱스 재생성 (CODE를 INCLUDE로 변경)

CREATE NONCLUSTERED INDEX NIDX01_PGSPLIT ON dbo.TB_PGSPLIT (NUM)
INCLUDE (CODE);
```

변경 사항	변경 전	변경 후
저장 구조	Heap	Clustered Index (NUM)
NCI 키 구조	(NUM, CODE)	(NUM) INCLUDE (CODE)
UPDATE 시 NCI 영향	B-tree 키 재정렬	리프 페이지 in-place 수정

7 인덱스를 사용하지 못하는 이슈

조건절에 사용된 컬럼을 키로 하는 인덱스가 존재함에도 테이블을 전체 스캔하여 과다한 I/O를 사용한 사례이다.

1. 사전 구성 스크립트

```
SELECT num
     , num%10 AS no
     , CONVERT(NVARCHAR(36), NEWID()) AS txt
 , GETDATE() AS date INTO TB_Data01
FROM (SELECT ROW_NUMBER() OVER ( ORDER BY a.number ) AS num
      FROM master..spt_values a, master..spt_values b
      WHERE a.type = 'P'
      AND a.number BETWEEN 1 AND 3
      AND b.type = 'P'
      AND b.number BETWEEN 1 AND 1000
) c

SELECT c.num
     , c.num%10 AS no
     , CONVERT(NVARCHAR(36), NEWID()) AS txt
, GETDATE() AS date
INTO TB_Data02
FROM (SELECT ROW_NUMBER() OVER ( ORDER BY a.number) num
      FROM master..spt_values a, master..spt_values b
      WHERE a.type = 'P'
      AND a.number BETWEEN 1 AND 1000
      AND b.type = 'P'
      AND b.number BETWEEN 1 AND 5
) c, master..spt_values d
WHERE d.type = 'P'
AND d.number BETWEEN 1 AND 100

CREATE INDEX NIDX01_Data02 ON TB_Data02(NUM)
GO
```

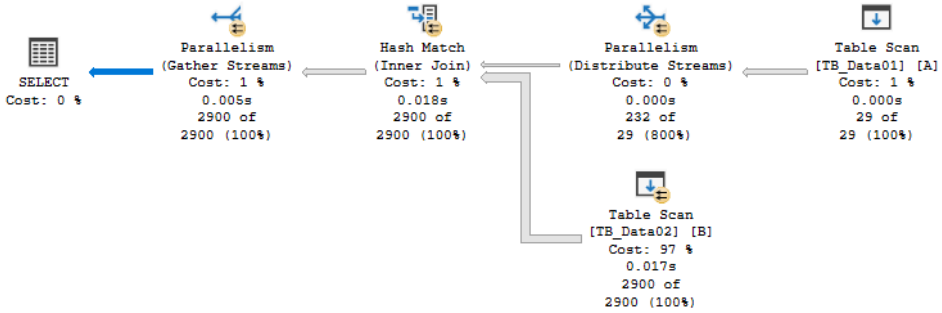
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT A.*, B.num, B.no
FROM TB_Data01 A INNER JOIN TB_Data02 B
ON A.num=B.num AND B.num<30
GO
```

· 실행 계획

Rows	Executes	StmtText
2900	1	SELECT A.*, B.num, B.no FROM TB_Data01 A INNER JOIN TB_Data02 B ON A.num=B.num AND B.num<30
2900	1	¦--Parallelism(Gather Streams)
2900	12	¦--Hash Match(INNER JOIN, HASH:([A].[num])=([B].[num]), RESIDUAL:([Tuning].[dbo].[TB_Data01].[num] as [A].[num]=[Tuning].[dbo].[TB_Data02].[num] as [B].[num]))
348	12	¦--Parallelism(Distribute Streams, Broadcast Partitioning)
29	1	¦ ¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Data01] AS [A]), WHERE:([Tuning].[dbo].[TB_Data01].[num] as [A].[num]<(30)))
2900	12	¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Data02] AS [B]), WHERE:([Tuning].[dbo].[TB_Data02].[num] as [B].[num]<(30)))




```
테이블 'TB_Data01'. 검색 수 1, 논리적 읽기 수 41
테이블 'Workfile'. 검색 수 0, 논리적 읽기 수 0
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_Data02'. 검색 수 13, 논리적 읽기 수 6765
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
SQL Server 실행 시간:
CPU 시간 = 155ms, 경과 시간 = 158ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Data02	NIDX01_Data02	NONCLUSTERED	0	NUM	-

4. [AS-IS] 구문 설명

[TB_Data02] 테이블의 [num] 컬럼 값이 30 미만인 데이터를 [TB_Data01] 테이블과 조인하는 구문이다. [TB_Data02] 테이블에는 필터 조건인 [num] 컬럼을 키로 하는 인덱스가 존재하지만 인덱스 탐색이 아닌 테이블 전체 스캔으로 실행 계획이 생성되었다. 원인은 SELECT 절에서 [no] 컬럼을 사용하지만 인덱스에는 [no] 컬럼이 포함되어 있지 않아 RID Lookup 연산이 추가적으로 필요하기 때문에 인덱스를 사용하는 것이 더 비효율적이라 판단한 것이다.

5. [TO-BE] 튜닝 포인트

다음 구문을 개선하기 위해 두 가지 방안을 고려할 수 있다.
첫째, 기존 인덱스에 포괄(Include) 열을 추가하여 RID Lookup이 발생되지 않도록 한다.
둘째, 클러스터형 인덱스를 생성하여 기존 인덱스에 키값이 포함되도록 한다.

첫 번째 방법은 필요한 컬럼을 기존 인덱스의 포괄 열로 추가시키는 방법이다. 포괄 열을 사용하면 해당 컬럼은 정렬되지 않은 상태로 인덱스에 데이터만을 포함한다. [no] 컬럼을 포괄 열로 추가하면 RID Lookup 이 필요하지 않기 때문에 인덱스 탐색으로 I/O를 개선할 수 있다.

두 번째 방법은 [no] 컬럼을 클러스터형 인덱스 키로 생성하는 것이다. 클러스터형 인덱스를 생성하게 되면 모든 비클러스터형 인덱스는 클러스터형 인덱스의 키가 포함된다. 첫 번째 방법과 동일한 맥락으로 Key Lookup이 필요하지 않기 때문에 인덱스 탐색으로 I/O를 개선할 수 있다.

6. [TO-BE] 개선 방안

1) 기존 인덱스에 포괄(Include) 열을 추가하여 RID Lookup이 발생되지 않도록 한다.

테이블 [TB_Data02]의 기존 인덱스 [NIDX01_Data02]에 [no] 컬럼을 포괄 열로 포함하여 재생성한다. 별도의 RID Lookup 연산이 요구되지 않기 때문에 인덱스 탐색을 통해 I/O 비용을 개선할 수 있다.

■ 인덱스 생성 구문

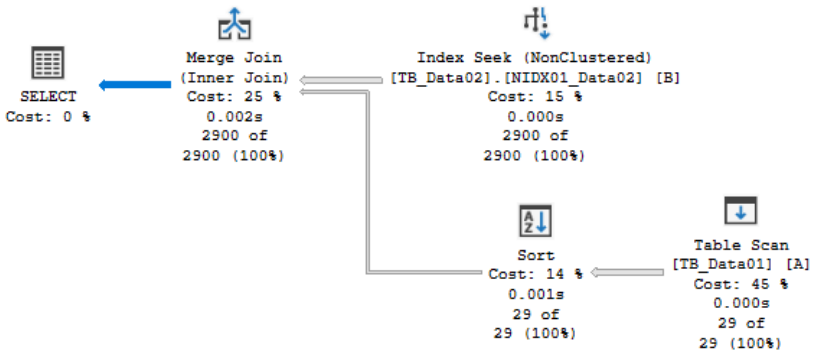
```
DROP INDEX NIDX01_Data02 ON TB_Data02
CREATE INDEX NIDX01_Data02 ON TB_Data02(num) INCLUDE (no)
GO
```

■ SQL 구문

```
SELECT A.*, B.num, B.no
FROM TB_Data01 A INNER JOIN TB_Data02 B
ON A.num=B.num AND B.num<30
GO
```

- 실행 계획

Rows	Executes	StmtText
2900	1	SELECT A.*, B.num, B.no FROM TB_Data01 A INNER JOIN TB_Data02 B ON A.num=B.num AND B.num<30
2900	1	¦--Merge Join(INNER JOIN, MANY-TO-MANY MERGE:([B].[num])=([A].[num]), RESIDUAL:([Tuning].[dbo].[TB_Data01].[num] as [A].[num]=[Tuning].[dbo].[TB_Data02].[num] as [B].[num]))
2900	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_Data02].[NIDX01_Data02] AS [B]), SEEK:([B].[num] < (30)) Ordered Forward)
29	1	¦--Sort(ORDER BY:([A].[num] ASC))
29	1	¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Data01] AS [A]), WHERE:([Tuning].[dbo].[TB_Data01].[num] as [A].[num]<(30)))




```
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_Data01'. 검색 수 1, 논리적 읽기 수 41
테이블 'TB_Data02'. 검색 수 1, 논리적 읽기 수 14
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 155ms
```

2) 클러스터형 인덱스를 생성하여 기존 인덱스에 키값이 포함되도록 한다.

[no] 컬럼을 키로 클러스터형 인덱스를 생성한다. 비클러스터형 인덱스는 클러스터형 인덱스의 키를 포함하고 있어 [no] 컬럼 값을 얻기 위한 Key Lookup 비용이 별도로 발생하지 않기 때문에 인덱스 탐색을 통해 I/O를 개선할 수 있다.

■ 인덱스 생성 구문

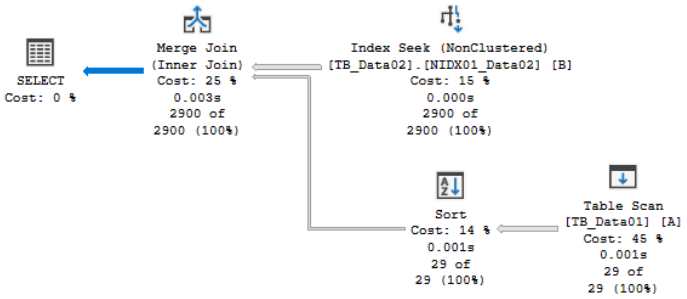
```
DROP INDEX NIDX01_Data02 ON TB_Data02 --TOBE(1)에서 생성한 인덱스 삭제
CREATE INDEX NIDX01_Data02 ON TB_Data02(num) --ASIS 기존 인덱스 생성

CREATE CLUSTERED INDEX CIDX_Data02 ON TB_Data02 (no)
GO
```

■ SQL 구문

```
SELECT A.*, B.num, B.no
FROM TB_Data01 A INNER JOIN TB_Data02 B
ON A.num=B.num AND B.num<30
GO
```

· 실행 계획

Rows	Executes	StmtText
2900 2900 2900 29 29	1 1 1 1 1	SELECT A.*, B.num, B.no FROM TB_Data01 A INNER JOIN TB_Data02 B ON A.num=B.num AND B.num<30 ¦--Merge Join(INNER JOIN, MANY-TO-MANY MERGE:([B].[num])=([A].[num]), RESIDUAL:([Tuning].[dbo].[TB_Data01].[num] as [A].[num]=[Tuning].[dbo].[TB_Data02].[num] as [B].[num])) ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_Data02].[NIDX01_Data02] AS [B]), SEEK:([B].[num] < (30)) Ordered Forward) ¦--Sort(ORDER BY:([A].[num] ASC)) ¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Data01] AS [A]), WHERE:([Tuning].[dbo].[TB_Data01].[num] as [A].[num]<(30)))



테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_Data01'. 검색 수 1, 논리적 읽기 수 41
테이블 'TB_Data02'. 검색 수 1, 논리적 읽기 수 14
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 155ms

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	155	158	6,806
TO-BE (1)	0	155	55
TO-BE (2)	0	155	55

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT A.*, B.num, B.no
FROM TB_Data01 A INNER JOIN TB_Data02 B
ON A.num = B.num AND B.num<30;
```

2. 분석 및 주요 최적화

1) 테이블 현황

항목	dbo.TB_Data01	dbo.TB_Data02
행 수	3,000	500,000
저장 구조	Heap (인덱스 없음)	Heap
기존 인덱스	없음	NIDX01_Data02 (num)

2) WHERE 조건 선택도 (num < 30)

테이블	해당 행	비율
TB_Data01	29	0.97%
TB_Data02	2,900	0.58%

*매우 선택적인 조건이므로 인덱스 Seek가 효과적입니다.

3) 식별된 문제점

1 TB_Data01 — Heap, 인덱스 없음

- 인덱스가 전혀 없어 JOIN 시 항상 3,000건 전체 Table Scan이 발생합니다.

2 TB_Data02 — 기존 인덱스 불완전

- NIDX01_Data02 (num)은 num만 키 컬럼이고 no가 포함되어 있지 않습니다. B.no를 SELECT 하므로 **매번 RID Lookup(Heap 접근)**이 필요합니다.

3. 개선안

```
-- [1단계] TB_Data01: Heap 제거 + num 클러스터드 인덱스

CREATE CLUSTERED INDEX CIX_TB_Data01_num ON dbo.TB_Data01 (num);


-- [2단계] TB_Data02: 기존 인덱스 재생성 (no 포함 → 커버링)

DROP INDEX IF EXISTS NIDX01_Data02 ON dbo.TB_Data02;

CREATE NONCLUSTERED INDEX NIDX01_Data02
ON dbo.TB_Data02 (num)
INCLUDE (no);
```

8 CROSS APPLY를 이용한 부분 범위 처리

조인 조건으로 전달받는 값마다 상위 N개의 데이터를 출력할 때 많은 스캔 범위로 인하여 불필요한 I/O가 발생되는 사례이다.

1. 사전 구성 스크립트

```
SELECT number id
      , CASE WHEN number <= 50 THEN 1 ELSE 2 END gid
      , DATEADD(dd,number, '2020-01-01') date
INTO  TB_CAFirst
FROM master..spt_values
WHERE type = 'P'
AND number BETWEEN 1 AND 100
GO

ALTER TABLE TB_CAFirst ADD CONSTRAINT PK_CAFirst PRIMARY KEY(id)
GO

WITH w_values AS
      (SELECT number
      FROM master..spt_values
      WHERE type = 'P'
      AND number BETWEEN 1 AND 100)
SELECT a.number id
      , DATEADD(dd,B.number,'2020-01-01') date
      , NEWID() code INTO TB_CASecond
FROM w_values a, w_values b, w_values c
```

```
GO

ALTER TABLE TB_CASecond ADD CONSTRAINT FK_CASecond FOREIGN KEY (ID)
      REFERENCES TB_CAFirst (ID)
GO

CREATE INDEX NIDX01_CAFirst ON TB_CAFirst (gid) INCLUDE (id)
CREATE INDEX NIDX01_CASecond ON TB_CASecond (id,date) INCLUDE (code)
GO
```

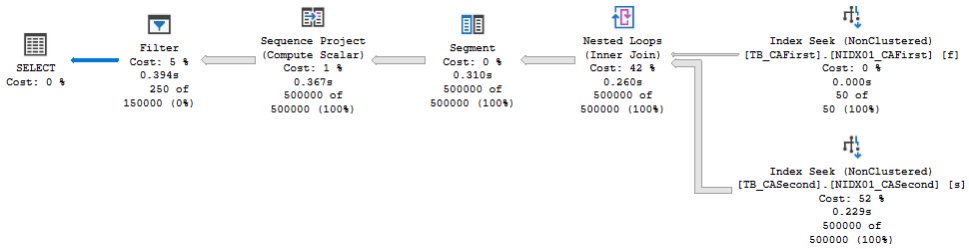
2. [AS-IS] SQL 구문 및 실행 계획

SQL 구문

```
SELECT id, code, date
FROM (SELECT f.id
            , s.code
            , s.date
            , ROW_NUMBER() OVER (PARTITION BY s.id ORDER BY s.date DESC) rn
       FROM TB_CAFirst f
       INNER  JOIN TB_CASecond s
       ON f.id = s.id
       WHERE f.gid = 1
) a
WHERE a.rn <= 5
GO
```

- **실행 계획**

Rows	Executes	StmtText
250	1	SELECT id, code, date FROM (SELECT f.id, s.code, s.date, ROW_NUMBER() OVER (PARTITION BY s.id ORDER BY s.date DESC) rn FROM TB_CAFirst f INNER JOIN TB_CASecond s ON f.id = s.id WHERE f.gid = 1) a WHERE a.rn <= 5
250	1	¦--Filter(WHERE:([Expr1003]<=(5)))
500000	1	¦--Sequence Project(DEFINE:([Expr1003]=row_number))
500000	1	¦--Segment
500000	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([f].[id], [Expr1004]) WITH ORDERED PREFETCH)
50	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_CAFirst].[NIDX01_CAFirst] AS [f]), SEEK:([f].[gid]=(1)) Ordered Forward)
500000	50	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_CASecond].[NIDX01_CASecond] AS [s]), SEEK:([s].[id]=[Tuning].[dbo].[TB_CAFirst].[id] as [f].[id]) ORDERED Backward)




```
테이블 'TB_CASecond'. 검색 수 50, 논리적 읽기 수 2854
테이블 'TB_CAFirst'. 검색 수 1, 논리적 읽기 수 2
SQL Server 실행 시간:
CPU 시간 = 188ms, 경과 시간 = 199ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_CAFirst	NIDX01_CAFirst	NONCLUSTERED	0	gid	id
TB_CAFirst	PK_CAFirst	CLUSTERED	1	id	-
TB_CASecond	NIDX01_CASecond	NONCLUSTERED	0	id, date	code

· 외래 키 정보

Table_Name	Constraint_Type	Constraint_Name	Constraint_Keys
TB_CASecond	FOREIGN KEY	FK_CASecond	id (REFERENCES Tuning.dbo.TB_CAFirst (id))

4. [AS-IS] 구문 설명

[TB_CAFirst] 테이블의 [gid] = 1인 데이터와 [TB_CASecond] 테이블을 조인한 결과 중 [id] 별로 가장 최근의 데이터 5건씩만 출력시키는 구문이다. 조인 조건에 사용된 [id] 컬럼은 [TB_CAFirst] 테이블과 [TB_CASecond] 테이블 간에 1:N 관계를 가지며 [TB_CASecond] 테이블은 [id] 값마다 10,000건의 행이 존재한다. 이 구문의 선행되는 테이블에서 [gid] = 1인 조건에 해당하는 데이터는 50건이므로 최종적으로 출력되는 데이터는 최대 250(50건*5)건이 된다. [id] 컬럼의 값마다 5건의 데이터만 요구되지만 두 테이블의 조인이 완료된 뒤 Filter 연산자를 통해 5개의 값을 추출하기 때문에 499,750(50*10000-250)건만큼의 불필요한 데이터까지 읽어내는 문제가 발생된다.

5. [TO-BE] 튜닝 포인트

[id] 컬럼 하나의 값에 대해 10,000건의 데이터를 읽고 상위 5건의 데이터를 필터 하는 방식이 아닌 TOP N 쿼리를 통해서 상위 5건만 읽어낼 수 있도록 구문을 변경하는 것이 좋다. SQL Server에서는 CROSS APPLY 조인 방식을 지원하는데 이는 인라인 뷰(FROM 절의 서브쿼리)에서 조인 조건으로 전달받을 값을 뷰 안에서 사용할 수 있도록 지원하는 조인 방식이다. 이 조인 방식을 통해 TOP N 쿼리를 이용하면 상위 5건의 데이터만 읽고 처리하는 것이 가능해진다.

6. [TO-BE] 개선 방안

1) CROSS APPLY로 변경하여 TOP N 쿼리를 활용한다.

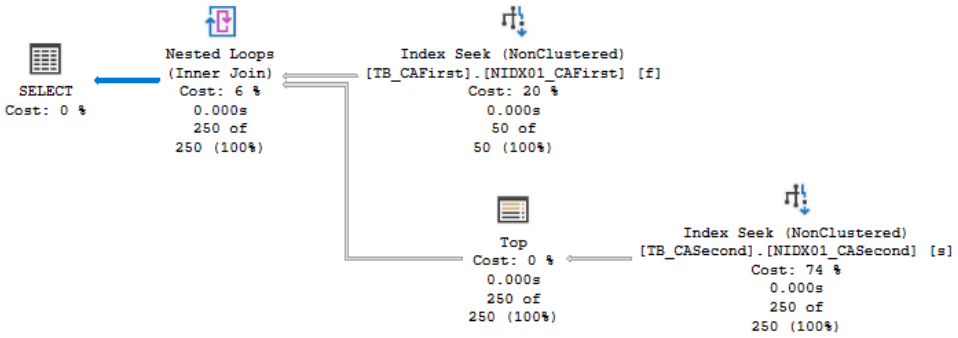
[TB_CASecond] 테이블을 인라인 뷰로 정의하여 [date] 컬럼의 값이 가장 높은 데이터 기준으로 TOP 5건을 읽어내도록 변경한다. 이때 CROSS APPLY를 사용하면 [TB_CAFirst] 테이블의 [id] 값을 인라인 뷰 내부에 전달하여 [id] 값마다 TOP 5건의 데이터를 출력할 수 있다.

■ 변경된 SQL 구문

```
SELECT f.id, s.code,s.date
FROM TB_CAFirst f CROSS APPLY (SELECT TOP 5 code,date
                               FROM TB_CASecond s
                               WHERE f.id = s.id
                               ORDER BY date DESC) s
WHERE f.gid = 1
GO
```

· 실행 계획

Rows	Executes	StmtText
250	1	SELECT f.id, s.code,s.date FROM TB_CAFirst f cross apply (SELECT TOP 5 code,date FROM TB_CASecond s WHERE f.id = s.id ORDER BY date DESC) s WHERE f.gid = 1
250	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([f].[id], [Expr1003]) WITH UNORDERED PREFETCH)
50	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_CAFirst].[NIDX01_CAFirst] AS [f]), SEEK:([f].[gid]=(1)) Ordered Forward)
250	50	¦--Top(TOP EXPRESSION:((5)))
250	50	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_CASecond].[NIDX01_CASecond] AS [s]), SEEK:([s].[id]=[Tuning].[dbo].[TB_CAFirst].[id] as [f].[id]) ORDERED Backward)




```
테이블 'TB_CASecond'. 검색 수 50, 논리적 읽기 수 186
테이블 'TB_CAFirst'. 검색 수 1, 논리적 읽기 수 2
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

- **리소스 사용률 비교**

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	188	199	2,856
TO-BE	0	0	188

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

8. CROSS APPLY를 이용한 부분 범위 처리

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT id, code, date
FROM (SELECT f.id, s.code, s.date,
             ROW_NUMBER() OVER (PARTITION BY s.id ORDER BY s.date DESC) rn
      FROM TB_CAFirst f
      INNER JOIN TB_CASecond s ON f.id = s.id
      WHERE f.gid = 1) a
WHERE a.rn <= 5;
```

2. 분석 및 주요 최적화

1) 테이블 현황

항목	dbo.TB_CAFirst	dbo.TB_CASecond
행 수	100	1,000,000
저장 구조	Clustered (PK_CAFirst: id)	Heap
NCI	NIDX01_CAFirst (gid) INCLUDE (id)	NIDX01_CASeconde (id, date) INCLUDE (code)

2) 데이터 분포

항목	값
gid = 1 해당 행 (TB_CAFirst)	50건
id별 TB_CASecond 행 수	~10,000건 (균등)

3) 현재 실행 흐름 (ROW_NUMBER 방식)

단계	처리 행 수
① TB_CAFirst gid = 1 필터	50건
② TB_CASecond JOIN (50 × 10,000)	500,000건
③ ROW_NUMBER 계산 (전체 대상)	500,000건
④ rn <= 5 필터 (50 × 5)	250건 반환

4) 핵심 문제점

1. ROW_NUMBER의 과도한 중간 결과셋
2. ROW_NUMBER()는 **모든 조인 결과(500,000건)**에 대해 파티션별 정렬과 번호 할당을 수행한 후 rn <= 5로 필터합니다.
3. id 50개 × 10,000건씩 전부 읽어야만 상위 5건을 선별할 수 있습니다.

3. 개선안

```sql
-- 최적화 방안: CROSS APPLY + TOP 변환
-- 기존 인덱스 NIDX01_CASecond (id, date) INCLUDE (code)를 그대로 활용하면서,
-- 각 id당 5건만 읽는 방식으로 변환합니다.

SELECT f.id, ca.code, ca.date
FROM dbo.TB_CAFirst f
CROSS APPLY (SELECT TOP 5 s.code, s.date
             FROM dbo.TB_CASecond s
             WHERE s.id = f.id
             ORDER BY s.date DESC
) ca
WHERE f.gid = 1;
```

9 CROSS JOIN으로 반복 사용된 테이블 통합

기간별 합계를 구하기 위해 동일한 테이블을 반복 조인하여 불필요한 I/O가 발생된 사례이다.

1. 사전 구성 스크립트

```
SELECT ROW_NUMBER() OVER (ORDER BY a.number) seq
      , CONVERT(INT, RIGHT(a.number*b.number*a.high,2)) AS value
      , NEWID() txt
      , DATEADD(dd,b.number,'2020-01-01') date
INTO TB_Sum
FROM master..spt_values a, master..spt_values b
WHERE a.type = 'P'
AND b.type = 'P'
AND a.number BETWEEN 1 AND 1000
AND b.number BETWEEN 1 AND 1000

CREATE CLUSTERED INDEX CIDX_CNT ON TB_Sum(date)
GO
```

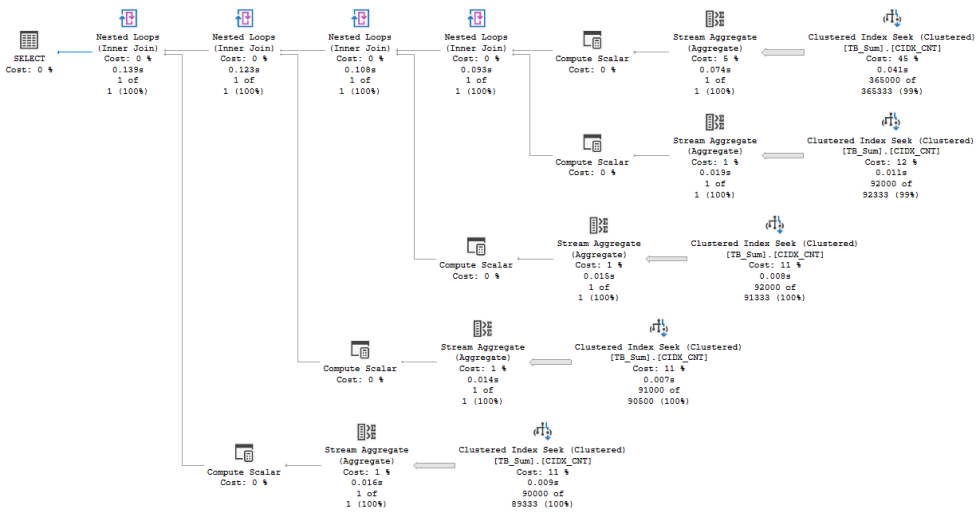
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT sum_total, sum_1, sum_2, sum_3, sum_4
FROM (SELECT SUM(value) sum_total
      FROM TB_Sum
      WHERE date BETWEEN '2020-01-01' AND '2020-12-31') a
CROSS JOIN
     (SELECT SUM(value) sum_1
      FROM TB_Sum
      WHERE date BETWEEN '2020-01-01' AND '2020-03-31') b
CROSS JOIN
     (SELECT SUM(value) sum_2
      FROM TB_Sum
      WHERE date BETWEEN '2020-04-01' AND '2020-06-30') c
CROSS JOIN
     (SELECT SUM(value) sum_3
      FROM TB_Sum
      WHERE date BETWEEN '2020-07-01' AND '2020-09-30') d
CROSS JOIN
     (SELECT SUM(value) sum_4
      FROM TB_Sum
      WHERE date BETWEEN '2020-10-01' AND '2020-12-31') e
GO
```

· 실행 계획

Rows	Executes	StmtText
1	1	SELECT sum_total, sum_1, sum_2, sum_3, sum_4 FROM (SELECT SUM(value) sum_total FROM TB_Sum WHERE date BETWEEN '2020-01-01' AND '2020-12-31') a CROSS JOIN (SELECT SUM(value) sum_1 FROM TB_Sum WHERE date BETWEEN '2020-01-01' AND '2020-03-31') b CROSS JOIN (SELECT SUM(value) sum_2 FROM TB_Sum WHERE date BETWEEN '2020-04-01' AND '2020-06-30') c CROSS JOIN (SELECT SUM(value) sum_3 FROM TB_Sum WHERE date BETWEEN '2020-07-01' AND '2020-09-30') d CROSS JOIN (SELECT SUM(value) sum_4 FROM TB_Sum WHERE date BETWEEN '2020-10-01' AND '2020-12-31') e
1	1	¦--Nested Loops(INNER JOIN)
1	1	¦--Nested Loops(INNER JOIN)
1	1	¦ ¦--Nested Loops(INNER JOIN)
1	1	¦ ¦ ¦--Nested Loops(INNER JOIN)
0	0	¦ ¦ ¦ ¦--Compute Scalar(DEFINE:([Expr1003]=CASE WHEN [Expr1040]=(0) THEN NULL ELSE [Expr1041] END))
1	1	¦ ¦ ¦ ¦ ¦--Stream Aggregate(DEFINE:([Expr1040]=COUNT_BIG([Tuning].[dbo].[TB_Sum].[value]), [Expr1041]=SUM([Tuning].[dbo].[TB_Sum].[value])))
365000	1	¦ ¦ ¦ ¦ ¦ ¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Sum].[CIDX_CNT]), SEEK:([Tuning].[dbo].[TB_Sum].[date] >= '2020-01-01 00:00:00.000' AND [Tuning].[dbo].[TB_Sum].[date] <= '2020-12-31 00:00:00.000') Ordered Forward)
0	0	¦ ¦ ¦ ¦--Compute Scalar(DEFINE:([Expr1019]=CASE WHEN [Expr1042]=(0) THEN NULL ELSE [Expr1043] END))
1	1	¦ ¦ ¦ ¦ ¦--Stream Aggregate(DEFINE:([Expr1042]=COUNT_BIG([Tuning].[dbo].[TB_Sum].[value]), [Expr1043]=SUM([Tuning].[dbo].[TB_Sum].[value])))
92000	1	¦ ¦ ¦ ¦ ¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Sum].[CIDX_CNT]), SEEK:([Tuning].[dbo].[TB_Sum].[date] >= '2020-10-01 00:00:00.000' AND [Tuning].[dbo].[TB_Sum].[date] <= '2020-12-31 00:00:00.000') Ordered Forward)
0	0	¦ ¦ ¦--Compute Scalar(DEFINE:([Expr1015]=CASE WHEN [Expr1044]=(0) THEN NULL ELSE [Expr1045] END))
1	1	¦ ¦ ¦ ¦--Stream Aggregate(DEFINE:([Expr1044]=COUNT_BIG([Tuning].[dbo].[TB_Sum].[value]), [Expr1045]=SUM([Tuning].[dbo].[TB_Sum].[value])))
92000	1	¦ ¦ ¦ ¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Sum].[CIDX_CNT]), SEEK:([Tuning].[dbo].[TB_Sum].[date] >= '2020-07-01 00:00:00.000' AND [Tuning].[dbo].[TB_Sum].[date] <= '2020-09-30 00:00:00.000') Ordered Forward)
0	0	¦ ¦--Compute Scalar(DEFINE:([Expr1011]=CASE WHEN [Expr1046]=(0) THEN NULL ELSE [Expr1047] END))
1	1	¦ ¦ ¦--Stream Aggregate(DEFINE:([Expr1046]=COUNT_BIG([Tuning].[dbo].[TB_Sum].[value]), [Expr1047]=SUM([Tuning].[dbo].[TB_Sum].[value])))
91000	1	¦ ¦ ¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Sum].[CIDX_CNT]), SEEK:([Tuning].[dbo].[TB_Sum].[date] >= '2020-04-01 00:00:00.000' AND [Tuning].[dbo].[TB_Sum].[date] <= '2020-06-30 00:00:00.000') Ordered Forward)
0	0	¦--Compute Scalar(DEFINE:([Expr1007]=CASE WHEN [Expr1048]=(0) THEN NULL ELSE [Expr1049] END))
1	1	¦--Stream Aggregate(DEFINE:([Expr1048]=COUNT_BIG([Tuning].[dbo].[TB_Sum].[value]), [Expr1049]=SUM([Tuning].[dbo].[TB_Sum].[value])))
90000	1	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Sum].[CIDX_CNT]), SEEK:([Tuning].[dbo].[TB_Sum].[date] >= '2020-01-01 00:00:00.000' AND [Tuning].[dbo].[TB_Sum].[date] <= '2020-03-31 00:00:00.000') Ordered Forward)



테이블 'TB_Sum'. 검색 수 5, 논리적 읽기 수 4840
SQL Server 실행 시간:
CPU 시간 = 32ms, 경과 시간 = 114ms

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Sum	CIDX_CNT	CLUSTERED	0	date	-

4. [AS-IS] 구문 설명

[TB_Sum] 테이블에서 2020년 전체에 해당하는 SUM(value) 값과 1, 2, 3, 4분기별 SUM(value) 값을 한 번에 출력하기 위해 CROSS JOIN을 활용한 구문이다. 이 구문에서는 각 기간의 값을 구하기 위해 테이블을 총 5회 읽어내고 있는데, 각 분기의 데이터는 2020년 전체 데이터에 일부로 포함되는 데이터임에도 중복된 데이터를 읽어내는 I/O가 추가로 발생된다.

5. [TO-BE] 튜닝 포인트

각 분기별 데이터를 모두 합치면 1년간의 데이터가 되기 때문에 2020년 전체 데이터를 출력하기 위해 테이블을 읽는 것은 불필요한 I/O를 사용하는 것이다. 2020년 전체 데이터를 읽어내는 인 라인 뷰를 제거하고 각 분기별 데이터를 합산하여 연간 데이터를 출력하면 I/O를 개선시킬 수 있다.

6. [TO-BE] 개선 방안

1) 각 분기 데이터를 합산하여 연간 데이터를 추출

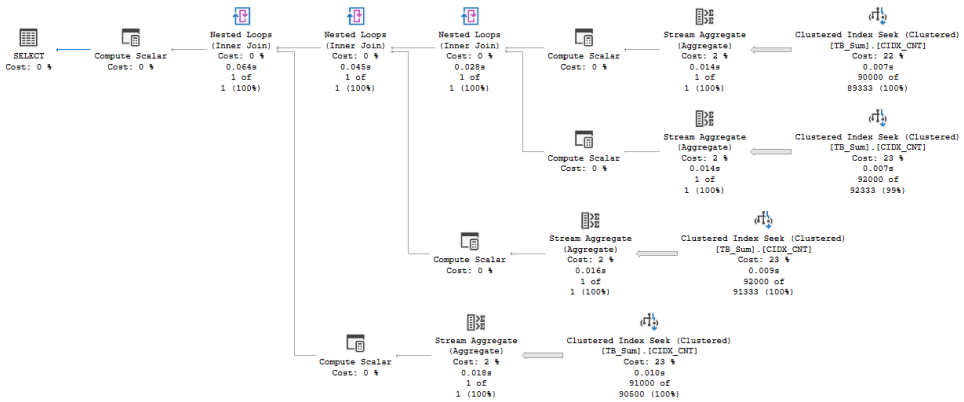
2020년 전체 데이터를 출력하는 인 라인 뷰를 제거하고 분기 별로 출력한 데이터를 모두 합산하여 연간 데이터를 출력하도록 변경한다.

■ 변경된 SQL 구문

```
SELECT sum_1+sum_2+sum_3+sum_4 as sum_total, sum_1, sum_2, sum_3, sum_4
FROM (SELECT SUM(value) sum_1
       FROM TB_Sum
       WHERE date BETWEEN '2020-01-01' AND '2020-03-31') b
CROSS JOIN (SELECT SUM(value) sum_2
             FROM TB_Sum
             WHERE date BETWEEN '2020-04-01' AND '2020-06-30') c
CROSS JOIN (SELECT SUM(value) sum_3
             FROM TB_Sum
             WHERE date BETWEEN '2020-07-01' AND '2020-09-30') d
CROSS JOIN(SELECT SUM(value) sum_4
            FROM TB_Sum
            WHERE date BETWEEN '2020-10-01' AND '2020-12-31') e
GO
```

· 실행 계획

Rows	Executes	StmtText
1	1	SELECT sum_1+sum_2+sum_3+sum_4 as sum_total, sum_1, sum_2, sum_3, sum_4 FROM (SELECT SUM(value) sum_1 FROM TB_Sum WHERE date BETWEEN '2020-01-01' AND '2020-03-31') b CROSS JOIN (SELECT SUM(value) sum_2 FROM TB_Sum WHERE date BETWEEN '2020-04-01' AND '2020-06-30') c CROSS JOIN (SELECT SUM(value) sum_3 FROM TB_Sum WHERE date BETWEEN '2020-07-01' AND '2020-09-30') d CROSS JOIN (SELECT SUM(value) sum_4 FROM TB_Sum WHERE date BETWEEN '2020-10-01' AND '2020-12-31') e
0	0	¦--Compute Scalar(DEFINE:([Expr1016]=[Expr1003]+[Expr1007]+[Expr1011]+[Expr1015]))
1	1	¦--Nested Loops(INNER JOIN)
1	1	¦--Nested Loops(INNER JOIN)
1	1	¦ ¦--Nested Loops(INNER JOIN)
0	0	¦ ¦ ¦--Compute Scalar(DEFINE:([Expr1003]=CASE WHEN [Expr1033]=(0) THEN NULL ELSE [Expr1034] END))
1	1	¦ ¦ ¦ ¦--Stream Aggregate(DEFINE:([Expr1033]=COUNT_BIG([Tuning].[dbo].[TB_Sum].[value]), [Expr1034]=SUM([Tuning].[dbo].[TB_Sum].[value])))
90000	1	¦ ¦ ¦ ¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Sum].[CIDX_CNT]), SEEK:([Tuning].[dbo].[TB_Sum].[date] >= '2020-01-01 00:00:00.000' AND [Tuning].[dbo].[TB_Sum].[date] <= '2020-03-31 00:00:00.000') Ordered Forward)
0	0	¦ ¦ ¦--Compute Scalar(DEFINE:([Expr1015]=CASE WHEN [Expr1035]=(0) THEN NULL ELSE [Expr1036] END))
1	1	¦ ¦ ¦--Stream Aggregate(DEFINE:([Expr1035]=COUNT_BIG([Tuning].[dbo].[TB_Sum].[value]), [Expr1036]=SUM([Tuning].[dbo].[TB_Sum].[value])))
92000	1	¦ ¦ ¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Sum].[CIDX_CNT]), SEEK:([Tuning].[dbo].[TB_Sum].[date] >= '2020-10-01 00:00:00.000' AND [Tuning].[dbo].[TB_Sum].[date] <= '2020-12-31 00:00:00.000') Ordered Forward)
0	0	¦ ¦--Compute Scalar(DEFINE:([Expr1011]=CASE WHEN [Expr1037]=(0) THEN NULL ELSE [Expr1038] END))
1	1	¦ ¦--Stream Aggregate(DEFINE:([Expr1037]=COUNT_BIG([Tuning].[dbo].[TB_Sum].[value]), [Expr1038]=SUM([Tuning].[dbo].[TB_Sum].[value])))
92000	1	¦ ¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Sum].[CIDX_CNT]), SEEK:([Tuning].[dbo].[TB_Sum].[date] >= '2020-07-01 00:00:00.000' AND [Tuning].[dbo].[TB_Sum].[date] <= '2020-09-30 00:00:00.000') Ordered Forward)
0	0	¦--Compute Scalar(DEFINE:([Expr1007]=CASE WHEN [Expr1039]=(0) THEN NULL ELSE [Expr1040] END))
1	1	¦--Stream Aggregate(DEFINE:([Expr1039]=COUNT_BIG([Tuning].[dbo].[TB_Sum].[value]), [Expr1040]=SUM([Tuning].[dbo].[TB_Sum].[value])))
91000	1	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Sum].[CIDX_CNT]), SEEK:([Tuning].[dbo].[TB_Sum].[date] >= '2020-04-01 00:00:00.000' AND [Tuning].[dbo].[TB_Sum].[date] <= '2020-06-30 00:00:00.000') Ordered Forward)



테이블 'TB_Sum'. 검색 수 4, 논리적 읽기 수 2426
SQL Server 실행 시간:
CPU 시간 = 62ms, 경과 시간 = 119ms

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	32	114	4,840
TO-BE	62	119	2,426

GitHub Copilot 채팅 결과

Claude Opus 4.6

9. CROSS JOIN으로 반복 사용된 테이블 통합

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT sum_total, sum_1, sum_2, sum_3, sum_4
FROM (SELECT SUM(value) sum_total
       FROM TB_Sum
       WHERE date BETWEEN '2020-01-01' AND '2020-12-31') a
CROSS JOIN
      (SELECT SUM(value) sum_1
      FROM TB_Sum
      WHERE date BETWEEN '2020-01-01' AND '2020-03-31') b
CROSS JOIN
      (SELECT SUM(value) sum_2
      FROM TB_Sum
      WHERE date BETWEEN '2020-04-01' AND '2020-06-30') c
CROSS JOIN
      (SELECT SUM(value) sum_3
      FROM TB_Sum
      WHERE date BETWEEN '2020-07-01' AND '2020-09-30') d
CROSS JOIN
      (SELECT SUM(value) sum_4
      FROM TB_Sum
      WHERE date BETWEEN '2020-10-01' AND '2020-12-31') e;
```

2. 분석 및 주요 최적화

1) 현재 문제점

항목	AS-IS	비고
테이블 스캔 횟수	5회	CROSS JOIN 서브쿼리마다 1회씩
대상 행 수	1,000,000건	dbo.TB_Sum
클러스터형 인덱스	CIDX_CNT (date)	존재
BETWEEN 사용	datetime에 대해 경계값 포함	경계값 정밀도 이슈 가능

*동일 테이블을 5번 반복 스캔하는 것이 가장 큰 성능 병목입니다.

2) 최적화 방안: 조건부 집계 (Conditional Aggregation)

CASE 식을 활용하면 테이블을 1회만 스캔하면서 모든 분기 합계를 동시에 구할 수 있습니다.

3. 개선안

```
-- [TO-BE] 조건부 집계(Conditional Aggregation)로 테이블을 1회만 스캔

SELECT
  SUM(value) AS sum_total,
  SUM(CASE WHEN date >= '2020-01-01' AND date < '2020-04-01' THEN value END) AS sum_1,
  SUM(CASE WHEN date >= '2020-04-01' AND date < '2020-07-01' THEN value END) AS sum_2,
  SUM(CASE WHEN date >= '2020-07-01' AND date < '2020-10-01' THEN value END) AS sum_3,
  SUM(CASE WHEN date >= '2020-10-01' AND date < '2021-01-01' THEN value END) AS sum_4
FROM dbo.TB_Sum
WHERE date >= '2020-01-01' AND date < '2021-01-01';
```

변경	설명
5개 서브쿼리 → 단일 쿼리	CROSS JOIN 제거, SUM(CASE ...) 패턴으로 1회 스캔
BETWEEN → 반개방 범위	datetime 타입에서 BETWEEN '...' AND '2020-12-31'은 2020-12-31 00:00:00까지만 포함. date < '2021-01-01'로 변경하여 12월 31일 전체를 정확히 포함
스키마 명시	dbo.TB_Sum 2-part name 사용

10 내부 조인의 스캔 범위 개선

조인된 결과에서 중복 제거된 값을 출력하기 위해 내부 조인을 사용하면서 과다한 스캔 범위로 I/O가 많아진 사례이다.

1. 사전 구성 스크립트

```
SELECT *, CONVERT(INT, RIGHT(num,1)) AS code
INTO TB_DATA11
FROM (SELECT
          ROW_NUMBER() OVER (ORDER BY A.number) AS num
        , LEFT(CONVERT(CHAR(8), DATEADD(HH, A.number,'2020-01-01'), 112), 6) AS date
      FROM master..spt_values A, (SELECT NUMBER
                                  FROM master..spt_values B
                                  WHERE type = 'P' AND number <= 500) B
      WHERE A.type = 'P'
) C
```

```
SELECT number AS id, number +2 AS code
INTO TB_CODE
FROM master..spt_values B
WHERE type = 'P' AND number BETWEEN 1 and 10

CREATE INDEX NIDX01_DATA11 ON TB_DATA11(code,date)
CREATE INDEX NIDX01_CODE ON TB_CODE(code)
GO
```

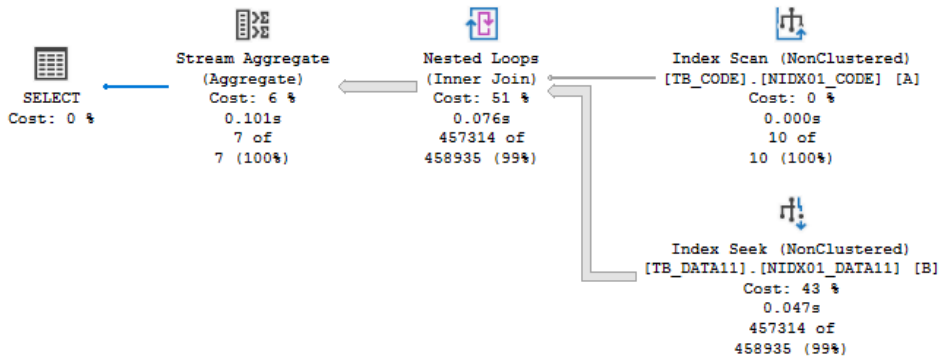
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT A.code
FROM TB_CODE A INNER JOIN TB_DATA11 B
ON A.code=B.code
AND B.date >'20200101'
GROUP BY A.code
ORDER BY A.code
GO
```

· 실행 계획

Rows	Executes	StmtText
7 7 457314 10 457314	1 1 1 1 10	A.code FROM TB_CODE A INNER JOIN TB_DATA11 B ON A.code=B.code AND B.date >'20200101' GROUP BY A.code ORDER BY A.code ¦--Stream Aggregate(GROUP BY:([A].[code])) ¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([A].[code])) ¦--Index Scan(OBJECT:([Tuning].[dbo].[TB_CODE].[NIDX01_CODE] AS [A]), Ordered Forward) ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_DATA11].[NIDX01_DATA11] AS [B]), SEEK:([B].[code]= [Tuning].[dbo].[TB_CODE].[code] as [A].[code] AND [B].[date] > '20200101') Ordered Forward)



테이블 'TB_DATA11'. 검색 수 10, 논리적 읽기 수 1627
테이블 'TB_CODE'. 검색 수 1, 논리적 읽기 수 2
SQL Server 실행 시간:
CPU 시간 = 141ms, 경과 시간 = 160ms

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_DATA11	NIDX01_DATA11	NONCLUSTERED	0	code, date	-
TB_CODE	NIDX01_CODE	NONCLUSTERED	0	code	-

4. [AS-IS] 구문 설명

[TB_CODE] 테이블을 기준으로 [TB_DATA11] 테이블과 [code] 컬럼으로 조인한 뒤 조인에 성공한 결과 집합에서 중복 값을 제거하고 조회하는 구문이다. [TB_CODE] 테이블의 데이터만을 출력하며 최종 행 수가 7건뿐임에도 [TB_DATA11] 테이블과 내부 조인이 수행되면서 약 45만 건의 행을 읽고 이를 GROUP BY 절로 중복 값을 제거하여 출력하게 된다.

5. [TO-BE] 튜닝 포인트

[TB_CODE] 테이블의 [code] 컬럼 값이 [TB_DATA11] 테이블에 존재하는지 여부만 확인하면 되기 때문에 EXISTS 절을 고려해 볼 수 있다. EXISTS는 Semi Join으로 수행되며 메인 쿼리 하나의 행에 대해서 서브쿼리의 모든 행과 조인하는 것이 아닌 처음 조인에 성공하는 행을 만나면 스캔을 멈추고 해당 행을 최종 결과 셋에 포함한다. EXISTS 절을 활용하면 후행 테이블의 스캔 범위를 감소시킬 수 있고 선행 테이블의 대해서만 중복을 제거하면서 I/O를 개선할 수 있다.

6. [TO-BE] 개선 방안

1) INNER JOIN을 EXISTS 절로 변경한다.

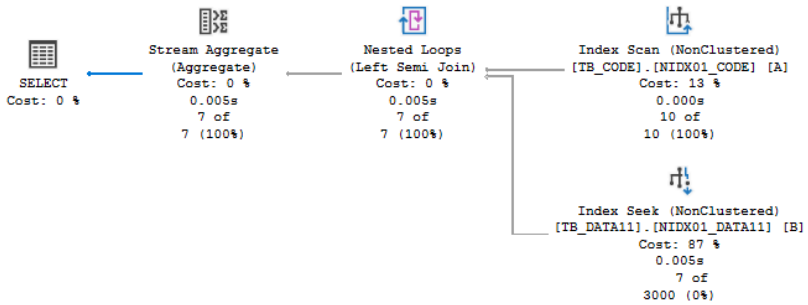
[TB_CODE] 테이블과 [TB_DATA11] 테이블을 내부 조인하여 집계하는 구문을 EXISTS 절로 변경한다. 메인 쿼리 10건의 행을 서브쿼리와 조인하면서 존재 유무만 확인하면 되기 때문에 I/O가 개선된다.

■ 변경된 SQL 구문

```
SELECT A.code
FROM TB_CODE A
WHERE EXISTS (SELECT '1'
              FROM TB_DATA11 B
              WHERE A.code=B.code
              AND B.date >'20200101')
GROUP BY A.code
ORDER BY A.code
GO
```

· 실행 계획

Rows	Executes	StmtText
7	1	SELECT A.code FROM TB_CODE A WHERE EXISTS (SELECT '1' FROM TB_DATA11 B WHERE A.code=B.code AND B.date >'20200101') GROUP BY A.code ORDER BY A.code
7	1	¦--Stream Aggregate(GROUP BY:([A].[code]))
7	1	¦--Nested Loops(Left Semi Join, OUTER REFERENCES:([A].[code]))
10	1	¦--Index Scan(OBJECT:([tuning].[dbo].[TB_CODE].[NIDX01_CODE] AS [A]), Ordered Forward)
7	10	¦--Index Seek(OBJECT:([tuning].[dbo].[TB_DATA11].[NIDX01_DATA11] AS [B]), SEEK:([B].[code]=[tuning].[dbo].[TB_CODE].[code] as [A].[code] AND [B].[date] > '20200101') Ordered Forward)




```
테이블 'TB_DATA11'. 검색 수 10, 논리적 읽기 수 30
테이블 'TB_CODE'. 검색 수 1, 논리적 읽기 수 2
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	141	160	1,629
TO-BE	0	0	32

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

10. 내부 조인의 스캔 범위 개선

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT A.code FROM TB_CODE A
INNER JOIN TB_DATA11 B ON A.code = B.code AND B.date > '20200101'
GROUP BY A.code
ORDER BY A.code;
```

2. 분석 및 주요 최적화

1) 현재 테이블 구조와 인덱스 현황을 정리하면 다음과 같습니다.

테이블	행수	인덱스	키 컬럼	클러스터드
dbo.TB_CODE	10	NIDX01_CODE	code	X (Heap)
dbo.TB_DATA11	1,026,048	NIDX01_DATA11	code, date	X (Heap)

2) 분석된 I/O 문제점

1 불필요한 GROUP BY:

- INNER JOIN으로 100만+ 행을 모두 매칭한 후 GROUP BY로 중복 제거 → 대량의 I/O 발생

단, *TB_CODE 테이블 code열에 중복 값이 없는 경우에 유효합니다.

2 EXISTS로 전환 가능:

- 각 code에 대해 매칭 행이 하나만 존재하면 충분하므로, EXISTS를 사용하면 조기 종료(short-circuit)로 I/O를 크게 줄일 수 있습니다

*TB_CODE는 10행뿐이므로, 각 code에 대해 TB_DATA11의 인덱스 NIDX01_DATA11(code, date)를 Index Seek하여 조건 충족 행이 1건만 발견되면 즉시 중단합니다.

3. 개선안

```
-- 최적화: INNER JOIN + GROUP BY → EXISTS (조기 종료로 I/O 감소)

SELECT A.code
FROM dbo.TB_CODE A
WHERE EXISTS (SELECT 1 FROM dbo.TB_DATA11 B
              WHERE B.code = A.code AND B.date > '20200101')
ORDER BY A.code;
```

· 변경 사항 요약

항목	변경 전	변경 후	효과
조인 방식	INNER JOIN + GROUP BY	EXISTS 서브쿼리	매칭 1건 발견 시 즉시 중단 → I/O 대폭 감소
스키마	미지정	dbo. 명시	이름 확인(Name Resolution) 오버헤드 제거
GROUP BY	GROUP BY A.code	제거	불필요한 집계 연산 제거

· 왜 I/O가 줄어드는가?

- 기존: TB_DATA11에서 date > '20200101' 조건을 만족하는 모든 행(잠재적으로 수십만 행)을 읽은 후 GROUP BY로 중복 제거

- 변경 후: 기존 인덱스 NIDX01_DATA11(code, date)를 활용하여, 각 code별로 조건을 만족하는 행이 1건만 발견되면 탐색 중단 (Semi Join)

4. 추가 프롬프트

> a.code 값이 중복이면 결과 건수가 달라지는데, 왜 GROUP BY를 제거한 거지?

지적이 정확합니다. dbo.TB_CODE의 code 컬럼은 PK도 아니고 UNIQUE 제약도 없으므로
중복이 존재할 수 있습니다.

현재 데이터에는 중복이 없지만, 스키마상 중복이 가능하므로
원본 쿼리의 GROUP BY 의미를 반드시 보존해야 합니다.
수정된 최적화 쿼리는 다음과 같습니다:

```
SELECT DISTINCT A.code
FROM dbo.TB_CODE A
WHERE EXISTS (SELECT 1
              FROM dbo.TB_DATA11 B
              WHERE B.code = A.code AND B.[date] > '20200101'
) ORDER BY A.code;
```

· 변경 사항 요약

항목	이전 최적화	수정된 최적화
중복 제거	✗ 누락	✓ DISTINCT 추가
Semi Join	✓ EXISTS	✓ EXISTS (유지)

11 BETWEEN의 인덱스 스캔 범위 개선 - 1

BETWEEN 조건을 사용했을 때 잘못된 스캔 범위로 인해 비효율적인 I/O가 발생되는 사례이다.

1. 사전 구성 스크립트

```
SELECT id
      , DATEADD(ss,id-1,'2020-01-01') date
      , NEWID() code
INTO TB_Date
FROM (SELECT ROW_NUMBER() OVER (ORDER BY a.number) id
      FROM master..spt_values a , (SELECT number
                                   FROM master..spt_values b
                                   WHERE type = 'P'
                                   AND number <= 500) b
       WHERE a.type = 'P'
) c

CREATE INDEX NIDX01_Date ON TB_Date (date) INCLUDE (id,code)
GO
```

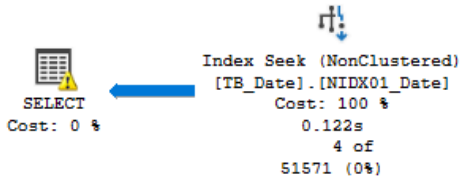
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT id, date,code
FROM TB_Date
WHERE date BETWEEN '2020-01-01' AND '2020-01-04'
AND SUBSTRING(CONVERT(NVARCHAR, date, 121), 12, 12) = '00:00:00.000'
GO
```

· 실행 계획

Rows	Executes	StmtText
4	1	SELECT id, date,code FROM TB_Date WHERE DATE BETWEEN '2020-01-01' AND '2020-01-04' AND SUBSTRING(CONVERT(NVARCHAR,date,121),12,12) = '00:00:00.000'
4	1	¦--Index Seek(OBJECT:([tuning].[dbo].[TB_Date].[NIDX01_Date]), SEEK:([tuning].[dbo].[TB_Date].[date] >= '2020-01-01 00:00:00.000' AND [tuning].[dbo].[TB_Date].[date] <= '2020-01-04 00:00:00.000'), WHERE:(substring(CONVERT(nvarchar(30),[tuning].[dbo].[TB_Date].[date],121),(12),(12))=N'00:00:00.000') Ordered Forward)




```
테이블 'TB_Date'. 검색 수 1, 논리적 읽기 수 1481
SQL Server 실행 시간:
CPU 시간 = 109ms, 경과 시간 = 112ms
```

3. [AS-IS] 통계 정보

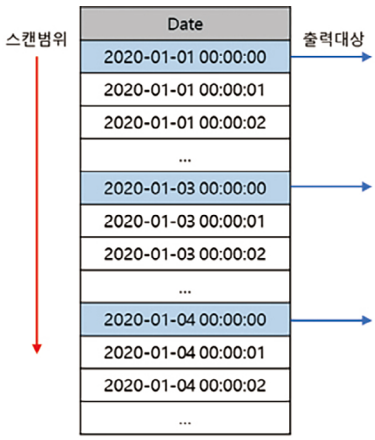
· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Date	NIDX01_Date	NONCLUSTERED	0	date	id, code

4. [AS-IS] 구문 설명

[TB_Date] 테이블에서 '2020-01-01'부터 4일간의 00시 정각 데이터를 조회하는 구문이다. 조건으로 사용된 컬럼은 초 단위로 데이터가 저장되어 있어 날짜와 시간을 별도의 조건을 주어 조회한다. 구문에 사용된 [NIDX01_Date] 인덱스에는 [Date] 컬럼이 키로 구성되어 있지만 시간을 구분하는 조건절이 함수로 가공되어 있어 인덱스 탐색 조건에 사용되지 못하고 날짜 간 사이에 존재하는 모든 초 단위 데이터가 스캔 범위에 포함되는 비효율이 생긴다.

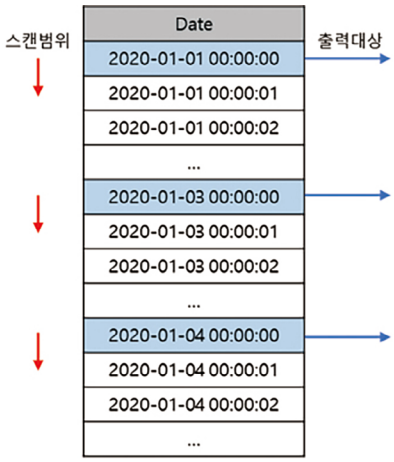
다음 그림은 [Date] 컬럼의 인덱스 탐색에 대한 스캔 범위를 나타낸다.



5. [TO-BE] 튜닝 포인트

이 구문은 특정 날짜와 시간만을 요구하기 때문에 IN 절로 변경하여 Equal 조건으로 탐색하는 것을 고려할 수 있다. IN 절 사용 시 나열된 조건 개수에 따라 인덱스를 재탐색해야 하는 비효율이 존재하지만 요구되는 날짜의 범위가 길지 않기 때문에 성능상의 이점을 가질 수 있다.

다음은 개선된 인덱스 스캔 범위를 나타낸다.



6. [TO-BE] 개선 방안

1) 날짜와 시간의 조건을 하나의 IN 절로 통합하여 구문 변경

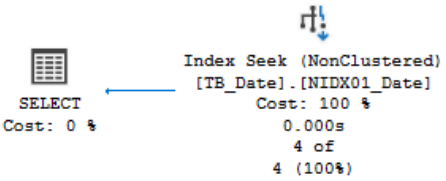
날짜와 시간을 하나의 조건으로 통합하고 IN 절의 값으로 나열한다. 각 조건마다 OR 절로 구분되어 Equal 조건으로 수행되며 인덱스 스캔 범위를 효과적으로 개선할 수 있다.

■ 변경된 SQL 구문

```
SELECT id,date,code
FROM TB_Date
WHERE date IN ('2020-01-01 00:00:00.000'
               ,'2020-01-02 00:00:00.000'
               ,'2020-01-03 00:00:00.000'
               ,'2020-01-04 00:00:00.000')
GO
```

· 실행 계획

Rows	Executes	StmtText
4	1	SELECT id,date,code FROM TB_Date WHERE date IN ('2020-01-01 00:00:00.000', '2020-01-02 00:00:00.000', '2020-01-03 00:00:00.000', '2020-01-04 00:00:00.000')
4	1	┆--Index Seek(OBJECT:([tuning].[dbo].[TB_Date].[NIDX01_Date]), SEEK:([tuning].[dbo].[TB_Date].[date]='2020-01-01 00:00:00.000' OR [tuning].[dbo].[TB_Date].[date]='2020-01-02 00:00:00.000' OR [tuning].[dbo].[TB_Date].[date]='2020-01-03 00:00:00.000' OR [tuning].[dbo].[TB_Date].[date]='2020-01-04 00:00:00.000') Ordered Forward)



테이블 'TB_Date'. 검색 수 4, 논리적 읽기 수 12
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	109	112	1,481
TO-BE	0	0	12

AI

GitHub Copilot 채팅 결과

Claude Opus 4.6

11. BETWEEN의 인덱스 스캔 범위 개선 - 1

1. 프롬프트

> [AS-IS]
> 구문을 수행하면 결과는 4건이지만, I/O 비용이 과도하게 발생하는 문제가 있다.
> 이 원인을 분석하고 성능을 개선해줘.

```
[AS-IS]
SELECT id, date, code
FROM TB_Date
WHERE date BETWEEN '2020-01-01' AND '2020-01-04'
AND SUBSTRING(CONVERT(NVARCHAR, date, 121), 12, 12) = '00:00:00.000';
```

2. 분석 및 주요 최적화

1) 현재 테이블 구조와 인덱스 현황을 정리하면 다음과 같습니다.

항목	값
테이블 전체 행 수	1,026,048
BETWEEN 범위 내 행 수	259,201
최종 반환 행 수	4
논리적 읽기	1,483
인덱스	NIDX01_Date (비클러스터형, date 컬럼)
클러스터형 인덱스	없음 (Heap)

2) 원인: Non-SARGable 조건식

```
SUBSTRING(CONVERT(NVARCHAR, date, 121), 12, 12) = '00:00:00.000'
```

이 조건의 의도는 시간 부분이 자정(00:00:00)인 행만 필터링하는 것입니다. 하지만 date 컬럼에 CONVERT → SUBSTRING 함수를 중첩 적용했기 때문에 인덱스 Seek가 불가능합니다. 옵티마이저는 BETWEEN 범위에 해당하는 259,201건을 모두 읽은 후 하나씩 함수를 적용해 4건을 걸러내고 있어, I/O가 비정상적으로 높습니다.

3. 개선안

```
-- BETWEEN을 제거하고 자정 날짜를 직접 지정하면, 인덱스Seek로 정확한 값만 조회할 수
-- 있습니다.

SELECT id, date, code
FROM dbo.TB_Date
WHERE date IN ('2020-01-01T00:00:00.000',
               '2020-01-02T00:00:00.000',
               '2020-01-03T00:00:00.000',
               '2020-01-04T00:00:00.000');


-- 범위가 동적이어서 날짜를 일일이 나열하기 어렵다면, CTE로 자정 날짜를 생성하여
-- 조인하는 방식을 사용하세요

WITH DateRange AS (
    SELECT CAST('2020-01-01' AS DATETIME) AS dt
```

```
    UNION ALL
    SELECT DATEADD(DAY, 1, dt)
    FROM DateRange
    WHERE dt < '2020-01-04'
)
SELECT t.id, t.date, t.code
FROM dbo.TB_Date t
INNER JOIN DateRange d ON t.date = d.dt;
```

12 BETWEEN의 인덱스 스캔 범위 개선 - 2

범위 조건을 사용할 때 인덱스 키 순서에 따라서 탐색하는 범위의 비효율이 발생된다.

1. 사전 구성 스크립트

```
SELECT *, CASE WHEN RIGHT(Num,1) = 1 THEN 1 ELSE 2 END Code
INTO TB_DATA10
FROM (SELECT ROW_NUMBER() OVER (ORDER BY A.number) Num
           , LEFT(CONVERT(CHAR(8), DATEADD(hh, A.number, '2020-01-01'), 112), 6) Date
      FROM master..spt_values A, (SELECT number
                                  FROM master..spt_values B
                                  WHERE type = 'P' AND number <= 500) B
      WHERE A.type ='P'
) C

CREATE INDEX NIDX01_DATA10 ON TB_DATA10(Date, Code)
GO
```

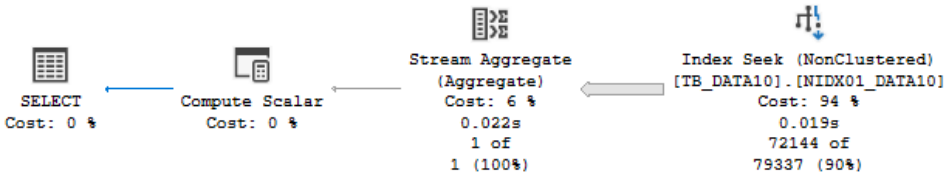
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT COUNT(*)
FROM TB_DATA10
WHERE Code ='1' AND Date BETWEEN '202001' AND '202002'
GO
```

· 실행 계획

Rows	Executes	StmtText
1 0 1 72144	1 0 1 1	SELECT COUNT(*) FROM [TB_DATA10] WHERE [Code]=@1 AND [Date]>=@2 AND [Date]<=@3 ¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(int,[Expr1005],0))) ¦--Stream Aggregate(DEFINE:([Expr1005]=Count(*))) ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_DATA10].[NIDX01_DATA10]), SEEK:(([Tuning].[dbo].[TB_DATA10].[Date], [Tuning].[dbo].[TB_DATA10].[Code]) >= ([@2], CONVERT_IMPLICIT(int,[@1],0)) AND ([Tuning].[dbo].[TB_DATA10].[Date], [Tuning].[dbo].[TB_DATA10].[Code]) <= ([@3], CONVERT_IMPLICIT(int,[@1],0))), WHERE:([Tuning].[dbo].[TB_DATA10].[Code]=CONVERT_IMPLICIT(int,[@1],0)) Ordered Forward)




```
테이블 'TB_DATA10'. 검색 수 1, 논리적 읽기 수 1420
SQL Server 실행 시간:
CPU 시간 = 31ms, 경과 시간 = 32ms
```

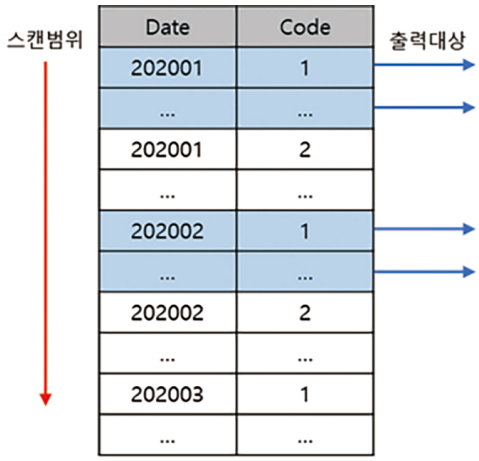
3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_DATA10	NIDX01_DATA10	NONCLUSTERED	0	Date, Code	-

4. [AS-IS] 구문 설명

[TB_DATA10] 테이블에서 2020년 1월, 2월 데이터 중 [Code] 컬럼의 값이 1인 행의 수를 구하는 구문이다. 조건에 사용된 모든 컬럼은 [NIDX01_DATA10] 인덱스의 키로 포함되어 있지만 선두 컬럼인 [Date]가 범위 조건으로 사용되고 있어 [Code] 컬럼은 인덱스 탐색 조건으로 사용될 수 없는 문제가 생긴다. 따라서 그림과 같이 검색 조건에 해당하지 않는 데이터까지 읽는 비효율이 발생한다.



5. [TO-BE] 튜닝 포인트

다음 구문을 개선하기 위해 두 가지 방안을 고려할 수 있다.
첫째, BETWEEN 조건을 IN 절로 변경하여 인덱스 스캔 범위를 줄인다.
둘째, Equal 조건으로 사용된 컬럼을 선두 키로 하여 신규 인덱스를 생성한다.

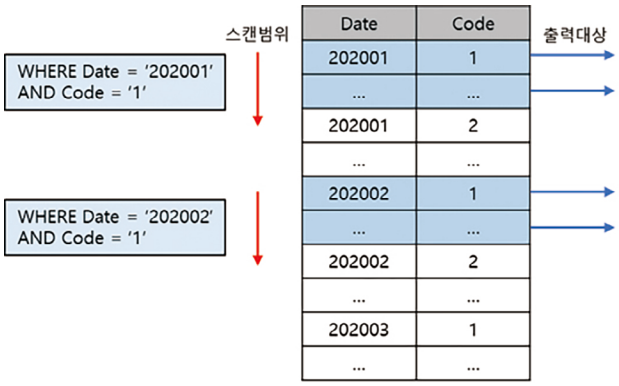
첫 번째 방법은 인덱스 선두 컬럼인 [Date]의 BETWEEN 조건을 IN 절로 변경하여 [Code] 컬럼을 인덱스 탐색 조건에 포함되도록 하는 방법이다. IN 절은 OR 연산자를 통해 나열된 항목마다 각각 탐색되지만 Equal 조건으로 수행되기 때문에 다음에 오는 [Code] 컬럼도 인덱스 탐색에 포함시킬 수 있다.

두 번째 방법은 Equal 조건으로 사용된 [Code] 컬럼을 선두 컬럼으로 하여 인덱스를 생성하는 방법이다. 범위 조건이 인덱스 선두 컬럼으로 사용되면 후행으로 오는 컬럼은 인덱스 탐색 조건으로 사용되지 못한다. 이를 고려하여 키 컬럼 순서가 변경된 인덱스를 생성하면 두 컬럼 모두 탐색 조건으로 사용되어 I/O를 개선할 수 있다.

6. [TO-BE] 개선 방안

1) BETWEEN 조건을 IN 절로 변경하여 인덱스 스캔 범위를 줄인다.

[Code] 컬럼이 인덱스 탐색에 사용될 수 있도록 선행되는 [Date] 컬럼을 BETWEEN 범위 조건에서 IN 절로 변경한다. IN 절에 포함된 조건 '202001' 과 '202002'를 각각 탐색하기 때문에 총 2회의 탐색이 발생되지만 [Code] 컬럼이 탐색 조건에 포함되면서 필요한 데이터만 읽어 내기 때문에 I/O를 개선시킬 수 있다.

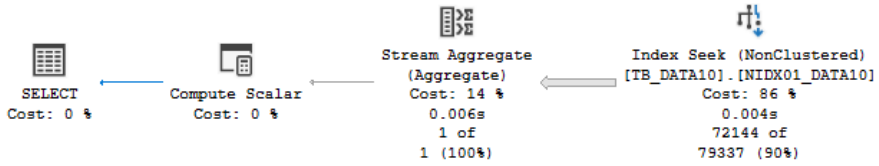


변경된 SQL 구문

```
SELECT COUNT(*)
FROM TB_DATA10
WHERE Code ='1' AND Date IN ('202001','202002')
GO
```

· 실행 계획

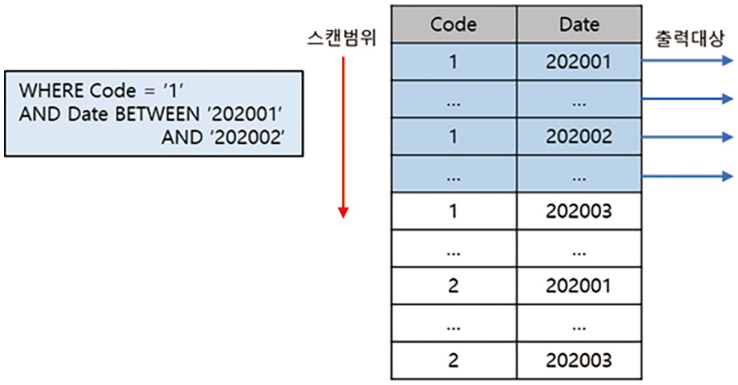
Rows	Executes	StmtText
1 1 1 72144	1 1 1 1	SELECT COUNT(*) FROM TB_DATA10 WHERE Code ='1' AND Date in ('202001','202002') ¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(int,[Expr1006],0))) ¦--Stream Aggregate(DEFINE:([Expr1006]=Count(*))) ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_DATA10].[NIDX01_DATA10]), SEEK:([Tuning].[dbo].[TB_DATA10].[Date]='202001' AND [Tuning].[dbo].[TB_DATA10].[Code]=(1) OR [Tuning].[dbo].[TB_DATA10].[Date]='202002' AND [Tuning].[dbo].[TB_DATA10].[Code]=(1)) Ordered Forward)




```
테이블 'TB_DATA10'. 검색 수 2, 논리적 읽기 수 258
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 3ms
```

2) Equal 조건으로 사용된 컬럼을 선두 키로 하여 신규 인덱스를 생성한다.

기존 쿼리의 변경 없이 신규 인덱스 생성만으로 개선하는 방법이다. [NIDX01_DATA10] 인덱스는 선두 컬럼 [Date]가 범위 조건으로 사용되었기 때문에 후행으로 오는 [Code] 컬럼이 인덱스의 탐색 조건으로 사용되지 못했다. Equal 조건으로 사용되는 [Code] 컬럼을 선두 키로 하여 신규 인덱스를 생성하면 두 조건 모두 인덱스를 탐색하게 되어 I/O를 개선시킬 수 있다.



인덱스 생성 구문

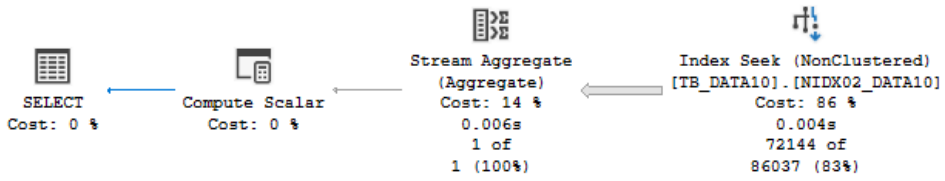
```
CREATE INDEX NIDX02_DATA10 ON TB_DATA10(Code,Date)
GO
```

SQL 구문

```
SELECT COUNT(*)
FROM TB_DATA10
WHERE Code ='1' AND Date BETWEEN '202001' AND '202002'
GO
```

· 실행 계획

Rows	Executes	StmtText
1 0 1 72144	1 0 1 1	SELECT COUNT(*) FROM [TB_DATA10] WHERE [Code]=@1 AND [Date]>=@2 AND [Date]<=@3 ¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(int,[Expr1005],0))) ¦--Stream Aggregate(DEFINE:([Expr1005]=Count(*))) ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_DATA10].[NIDX02_DATA10]),SEEK:([Tuning].[dbo].[TB_DATA10].[Code]=CONVERT_IMPLICIT(int,[@1],0) AND [Tuning].[dbo].[TB_DATA10].[Date] >= [@2] AND [Tuning].[dbo].[TB_DATA10].[Date] <= [@3]) Ordered Forward)



테이블 'TB_DATA10'. 검색 수 1, 논리적 읽기 수 254
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 5ms

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	31	32	1,420
TO-BE (1)	0	3	258
TO-BE (2)	0	5	254

AI

GitHub Copilot 채팅 결과

Claude Opus 4.6

12. BETWEEN의 인덱스 스캔 범위 개선 - 2

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT COUNT(*)
FROM TB_DATA10
WHERE Code = '1' AND Date BETWEEN '202001' AND '202002';
```

2. 분석 및 주요 최적화

1) 현황 분석

항목	값
총 행 수	1,026,048
date 고유값 수	3개
code 고유값 수	2개
date BETWEEN 조건 매칭	721,440행 (70.3%)
code = 1 조건 매칭	102,605행 (10%)
두 조건 모두 매칭	72,144행 (7%)
테이블 유형	HEAP (클러스터드 인덱스 없음)
현재 인덱스	NIDX01_DATA10(date, code)

2) 문제 원인

· 인덱스 키 순서가 비효율적 (핵심 원인)

· 현재 인덱스는 (date, code) 순서입니다.

· SQL Server는 선행 키인 date부터 Seek합니다.

· 등치(=) 조건을 선행 키, 범위(BETWEEN) 조건을 후행 키가 인덱스 설계의 핵심 원칙입니다.

· 키 순서만 바꿔도 I/O가 약 10배 감소합니다.

인덱스 순서	Seek 방식	읽는 행 수
(date, code) 현재	date 범위로 721,440행 스캔 → code 필터	~721,440
(code, date) 권장	code=1 등치로 102,605행 → date 범위 스캔	~72,144

3. 개선안

```
-- 최적 인덱스 생성 (code, date 순서)
-- 기존 비효율 인덱스 삭제

DROP INDEX NIDX01_DATA10 ON dbo.TB_DATA10;


-- 등치 조건(code) 선행, 범위 조건(date) 후행으로 재생성

CREATE NONCLUSTERED INDEX NIDX01_DATA10
ON dbo.TB_DATA10 (code, date);
```

13 IN 조건절의 성능 이슈

두 테이블이 NL로 조인되는 과정에서 후행 테이블에 존재하는 IN 조건절로 인해 인덱스로 탐색하는 횟수가 과다하게 증가되는 사례이다.

1. 사전 구성 스크립트

```
CREATE TABLE TB_ORFirst (seq INT, date DATETIME)
CREATE TABLE TB_ORSecond (seq INT , gid INT, txt NVARCHAR(36) DEFAULT (NEWID() ) )

INSERT INTO TB_ORFirst
SELECT ROW_NUMBER() OVER (ORDER BY a.number) seq
      , DATEADD(d, a.number, '2020-01-01') date
```

```
FROM master..spt_values a (NOLOCK) ,(SELECT number
                                     FROM master..spt_values (NOLOCK)
                                     WHERE type ='p' AND number < 20) b
WHERE type = 'p'
AND a.number <= 365

INSERT INTO TB_ORSecond
SELECT b.seq, a.number, NEWID()
FROM master..spt_values a (NOLOCK), TB_ORFirst b
WHERE type = 'P'
AND number%2 = 1
AND number <= 100

CREATE CLUSTERED INDEX CIDX01_ORFirst ON TB_ORFirst(date)
CREATE INDEX NIDX01_ORSecond ON TB_ORSecond(seq, gid) INCLUDE (txt)
GO
```

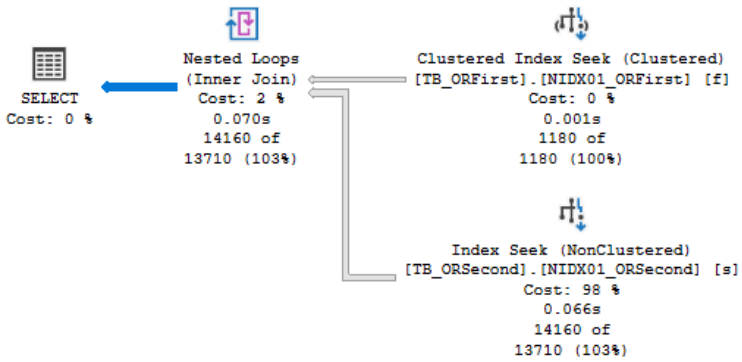
2. [AS-IS] SQL 구문 및 실행 계획

SQL 구문

```
SELECT f.date, s.txt
FROM TB_ORFirst f INNER JOIN TB_ORSecond s
ON f.seq = s.seq
WHERE f.date BETWEEN '2020-01-01' AND '2020-02-28'
AND s.gid IN (1,2,3,7,8,9,13,14,15,19,20,21,25,26,27,31,32,33)
GO
```

- 실행 계획

Rows	Executes	StmtText
14160	1	SELECT f.date, s.txt FROM TB_ORFirst f INNER JOIN TB_ORSecond s ON f.seq = s.seq WHERE f.date BETWEEN '2020-01-01' AND '2020-02-28' AND s.gid IN (1,2,3,7,8,9,13,14,15,19,20,21,25,26,27,31,32,33)
14160	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([f].[seq], [Expr1004]) WITH UNORDERED PREFETCH)
1180	1	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_ORFirst].[NIDX01_ORFirst] AS [f]), SEEK:([f].[date] >= '2020-01-01 00:00:00.000' AND [f].[date] <= '2020-02-28 00:00:00.000') Ordered Forward)
14160	1180	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_ORSecond].[NIDX01_ORSecond] AS [s]), SEEK:([s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq] AND [s].[gid]=(1) OR [s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq] AND [s].[gid]=(2) OR [s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq] AND [s].[gid]=(3) OR [s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq] AND [s].[gid]=(7) OR [s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq] AND [s].[gid]=(8) OR [s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq] AND [s].[gid]=(9) OR [s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq] AND [s].[gid]=(13) OR [s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq] AND [s].[gid]=(14) OR [s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq] AND [s].[gid]=(15) OR [s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq] AND [s].[gid]=(19) OR [s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq] AND [s].[gid]=(20) OR [s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq] AND [s].[gid]=(21) OR [s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq] AND [s].[gid]=(25) OR [s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq] AND [s].[gid]=(26) OR [s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq] AND [s].[gid]=(27) OR [s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq] AND [s].[gid]=(31) OR [s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq] AND [s].[gid]=(32) OR [s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq] AND [s].[gid]=(33)) Ordered Forward)




```
테이블 'TB_ORSecond'. 검색 수 21240, 논리적 읽기 수 64397
테이블 'TB_ORFirst'. 검색 수 1, 논리적 읽기 수 7
SQL Server 실행 시간:
CPU 시간 = 47ms, 경과 시간 = 56ms
```

3. [AS-IS] 통계 정보

- **인덱스 정보**

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_ORFirst	CIDX01_ORFirst	CLUSTERED	0	date	-
TB_ORSecond	NIDX01_ORSecond	NONCLUSTERED	0	seq, gid	txt

4. [AS-IS] 구문 설명

선행 테이블 [TB_ORFirst]과 후행 테이블 [TB_ORSecond]이 NL Join으로 수행될 때 후행 테이블의 조인 조건과 IN 조건절 컬럼이 인덱스 키로 생성되어 있어 인덱스로 탐색하는 횟수가 과다하게 많아지는 현상이다. NL Join으로 인해 선행 테이블의 결과 집합 건수 (1,180) 만큼 후행 테이블을 참조하며 그 결과 집합은 1건당 IN 절의 조건 개수 (18) 만큼 인덱스를 탐색하게 되면서 다음과 같은 테이블 검색 수를 보이게 된다.

> [테이블 검색 수 = 선행 테이블의 데이터 건수 * IN 조건절 개수 = 1,180 * 18 = 21,240]

5. [TO-BE] 튜닝 포인트

다음 구문을 개선하기 위한 두 가지 방안을 고려할 수 있다.

첫째, IN 조건절이 인덱스 탐색 조건의 대상이 되지 못하도록 컬럼을 고의적으로 가공한다.

둘째, Hash Join으로 유도하여 테이블 검색 수를 최소화한다.

첫 번째 방법은 [seq] 컬럼으로만 인덱스 탐색하는 것이 유리할 때 고려할 수 있다. IN 절의 조건들은 OR 절로 동작하여 각 조건마다 매번 Page를 새로 탐색하게 되고, 인덱스의 트리 구조의 깊이가 깊어질수록 더 많은 I/O를 유발하게 된다. 여기서 [seq] 컬럼만을 인덱스로 탐색한 데이터와 IN 조건절까지 모두 탐색한 데이터의 차이가 크지 않다면 IN 절에 사용된 컬럼을 가공하여 [seq] 컬럼으로만 탐색하도록 하는 것이 오히려 I/O가 더 감소될 수 있다.

두 번째 방법은 조인 방식을 Hash Join으로 변경하여 각 테이블이 참조되는 수를 줄이는 방법이다. Hash Join으로 변경할 경우 후행 테이블은 IN 절에 사용된 조건 수만큼만 인덱스 탐색을 수행하고, 선행 테이블 데이터 건수의 영향을 받지 않기 때문에 반복해서 읽는 과정이 제거되어 I/O를 감소시킬 수 있다.

6. [TO-BE] 개선 방안

1) IN 조건절이 인덱스 탐색의 대상이 되지 못하도록 컬럼을 고의적으로 가공한다.

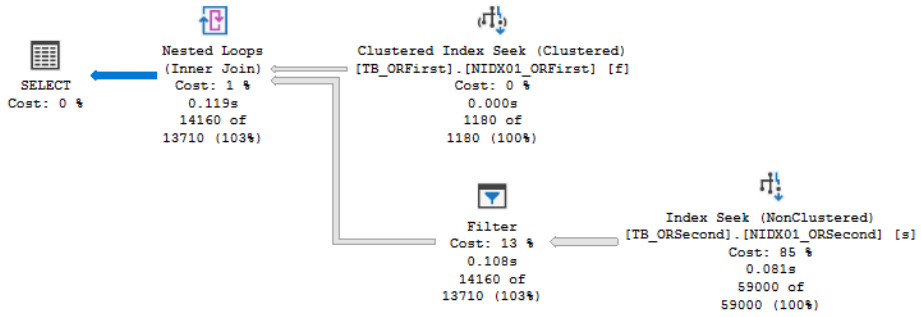
후행 테이블인 [TB_ORSecond] 테이블의 IN 절로 사용된 [gid] 컬럼을 고의적으로 가공하여 인덱스 탐색의 대상이 되지 못하도록 변경한다. [gid] 컬럼은 데이터 타입이 INT형이므로 데이터가 변하지 않는 값 (+0) 을 추가하여 가공한다. 후행 테이블은 선행 테이블의 결과 집합 1,180건 만큼만 검색되며 [seq] 컬럼으로 인덱스를 탐색한 뒤 IN 절의 조건은 Filter 연산자를 통해 동작하는 것을 확인할 수 있다.

■ 변경된 SQL 구문

```
SELECT f.date, s.txt
FROM TB_ORFirst f INNER JOIN TB_ORSecond s
ON f.seq = s.seq
WHERE f.date BETWEEN '2020-01-01' AND '2020-02-28'
AND s.gid + 0 IN (1,2,3,7,8,9,13,14,15,19,20,21,25,26,27,31,32,33)
GO
```

· 실행 계획

Rows	Executes	StmtText
14160	1	SELECT f.date, s.txt FROM TB_ORFirst f INNER JOIN TB_ORSecond s ON f.seq = s.seq WHERE f.date BETWEEN '2020-01-01' AND '2020-02-28' AND s.gid+0 IN (1,2,3,7,8,9,13,14,15,19,20,21,25,26,27,31,32,33)
14160	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([f].[seq], [Expr1005]) WITH UNORDERED PREFETCH)
1180	1	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_ORFirst].[NIDX01_ORFirst] AS [f]), SEEK:([f].[date] >= '2020-01-01 00:00:00.000' AND [f].[date] <= '2020-02-28 00:00:00.000') Ordered Forward)
14160	1180	¦--Filter(WHERE:(([Tuning].[dbo].[TB_ORSecond].[gid] as [s].[gid]+(0))=(33) OR ([Tuning].[dbo].[TB_ORSecond].[gid] as [s].[gid]+(0))=(32) OR ([Tuning].[dbo].[TB_ORSecond].[gid] as [s].[gid]+(0))=(31) OR ([Tuning].[dbo].[TB_ORSecond].[gid] as [s].[gid]+(0))=(27) OR ([Tuning].[dbo].[TB_ORSecond].[gid] as [s].[gid]+(0))=(26) OR ([Tuning].[dbo].[TB_ORSecond].[gid] as [s].[gid]+(0))=(25) OR ([Tuning].[dbo].[TB_ORSecond].[gid] as [s].[gid]+(0))=(21) OR ([Tuning].[dbo].[TB_ORSecond].[gid] as [s].[gid]+(0))=(20) OR ([Tuning].[dbo].[TB_ORSecond].[gid] as [s].[gid]+(0))=(19) OR ([Tuning].[dbo].[TB_ORSecond].[gid] as [s].[gid]+(0))=(15) OR ([Tuning].[dbo].[TB_ORSecond].[gid] as [s].[gid]+(0))=(14) OR ([Tuning].[dbo].[TB_ORSecond].[gid] as [s].[gid]+(0))=(13) OR ([Tuning].[dbo].[TB_ORSecond].[gid] as [s].[gid]+(0))=(9) OR ([Tuning].[dbo].[TB_ORSecond].[gid] as [s].[gid]+(0))=(8) OR ([Tuning].[dbo].[TB_ORSecond].[gid] as [s].[gid]+(0))=(7) OR ([Tuning].[dbo].[TB_ORSecond].[gid] as [s].[gid]+(0))=(3) OR ([Tuning].[dbo].[TB_ORSecond].[gid] as [s].[gid]+(0))=(2) OR ([Tuning].[dbo].[TB_ORSecond].[gid] as [s].[gid]+(0))=(1)))
59000	1180	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_ORSecond].[NIDX01_ORSecond] AS [s]), SEEK:([s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq]) Ordered Forward)




```
테이블 'TB_ORSecond'. 검색 수 1180, 논리적 읽기 수 4533
테이블 'TB_ORFirst'. 검색 수 1, 논리적 읽기 수 7
SQL Server 실행 시간:
CPU 시간 = 32ms, 경과 시간 = 37ms
```

2) Hash Join으로 유도하여 테이블의 검색 수를 최소화한다.

조인 조건을 제외한 조건절의 컬럼을 키로 인덱스를 생성하여 Hash Join으로 유도한다. IN 절의 조건절 개수만큼만 인덱스로 탐색을 수행하기 때문에 검색 횟수가 감소되며 조건절을 탐색하는 스캔 범위에 따라 I/O가 결정된다.

■ 인덱스 생성 구문

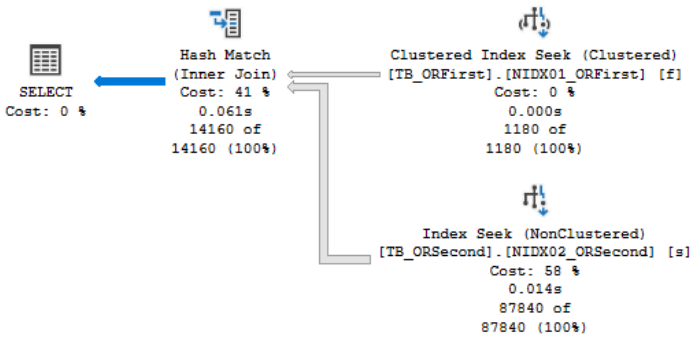
```
CREATE INDEX NIDX02_ORSecond ON TB_ORSecond(gid) INCLUDE (seq,txt)
GO
```

■ SQL 구문

```
SELECT f.date, s.txt
FROM TB_ORFirst f INNER JOIN TB_ORSecond s
ON f.seq = s.seq
WHERE f.date BETWEEN '2020-01-01' AND '2020-02-28'
AND s.gid IN (1,2,3,7,8,9,13,14,15,19,20,21,25,26,27,31,32,33)
GO
```

· 실행 계획

Rows	Executes	StmtText
14160	1	SELECT f.date, s.txt FROM TB_ORFirst f INNER JOIN TB_ORSecond s ON f.seq = s.seq WHERE f.date BETWEEN '2020-01-01' AND '2020-02-28' AND s.gid IN (1,2,3,7,8,9,13,14,15,19,20,21,25,26,27,31,32,33)
14160	1	¦--Hash Match(INNER JOIN, HASH:([f].[seq])=([s].[seq]), RESIDUAL:([Tuning].[dbo].[TB_ORSecond].[seq] as [s].[seq]=[Tuning].[dbo].[TB_ORFirst].[seq] as [f].[seq]))
1180	1	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_ORFirst].[NIDX01_ORFirst] AS [f]), SEEK:([f].[date] >= '2020-01-01 00:00:00.000' AND [f].[date] <= '2020-02-28 00:00:00.000') Ordered Forward)
87840	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_ORSecond].[NIDX02_ORSecond] AS [s]), SEEK:([s].[gid]=(1) OR [s].[gid]=(2) OR [s].[gid]=(3) OR [s].[gid]=(7) OR [s].[gid]=(8) OR [s].[gid]=(9) OR [s].[gid]=(13) OR [s].[gid]=(14) OR [s].[gid]=(15) OR [s].[gid]=(19) OR [s].[gid]=(20) OR [s].[gid]=(21) OR [s].[gid]=(25) OR [s].[gid]=(26) OR [s].[gid]=(27) OR [s].[gid]=(31) OR [s].[gid]=(32) OR [s].[gid]=(33)) Ordered Forward)




```
테이블 'Workfile'. 검색 수 0, 논리적 읽기 수 0
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_ORSecond'. 검색 수 18, 논리적 읽기 수 1145
테이블 'TB_ORFirst'. 검색 수 1, 논리적 읽기 수 7
SQL Server 실행 시간:
CPU 시간 = 15ms, 경과 시간 = 24ms
```

7. 개선 효과

· **리소스 사용률 비교**

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	47	56	64,404
TO-BE (1)	32	37	4,540
TO-BE (2)	15	24	1,152

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

13. IN 조건절의 성능 이슈

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT f.date, s.txt
FROM TB_ORFirst f INNER JOIN TB_ORSecond s
ON f.seq = s.seq
WHERE f.date BETWEEN '2020-01-01' AND '2020-02-28'
AND s.gid IN (1,2,3,7,8,9,13,14,15,19,20,21,25,26,27,31,32,33);
```

2. 분석 및 주요 최적화

1) 현재 테이블/인덱스 구조

테이블	행 수	클러스터형 인덱스	비클러스터형 인덱스
dbo.TB_ORFirst	7,320	CIDX01_ORFirst(date)	없음
dbo.TB_ORSecond	366,000	없음 (HEAP)	NIDX01_ORSecond(seq, gid) INCLUDE (txt)

2) 데이터 분포

항목	값
TB_ORFirst WHERE 조건 일치 행	1,180 / 7,320 (16%)
TB_ORSecond gid IN 조건 일치 행	87,840 / 366,000 (24%)
seq 당 TB_ORSecond 행 수	~50행 (gid 50개)

3) Query Store 실행 통계

plan_id	조인 전략	평균 Logical I/O	평균 Duration (μs)
15	기본 (Nested Loop 추정)	43,244	49,612
42	MERGE JOIN 힌트	38,812	147,943
45	HASH JOIN 힌트	4,408	103,986

4) 비효율 원인 분석

TB_ORSecond의 인덱스 키 순서 문제
(핵심 원인, 현재 인덱스: NIDX01_ORSecond(seq, gid) INCLUDE (txt))

1. 선행 키가 seq이므로, gid IN (...) 필터가 Index Seek가 아닌 잔여 필터(Residual Predicate)로 동작합니다.
2. Nested Loop 조인 시, TB_ORFirst에서 넘어온 1,180개 seq 값 각각에 대해 50개 행(모든 gid)을 읽은 후 18개 gid만 필터링합니다.
3. 실제 필요한 행: ~1,180 × 18 = ~21,240행, 하지만 읽는 행: ~1,180 × 50 = ~59,000행
   → 약 2.8배 과다 읽기

3. 개선안

```
-- 아래 인덱스를 생성하면 gid가 선행 키가 되어 Index Seek로 gid 필터링 → seq 조인이
-- 가능해지며, Logical I/O가 대폭 감소합니다.
-- [핵심] TB_ORSecond: gid를 선행 키로 하는 커버링 인덱스 생성
-- gid IN (...) 필터를 Index Seek로 처리 후, seq로 조인 가능

CREATE NONCLUSTERED INDEX NIDX02_ORSecond_gid_seq
ON dbo.TB_ORSecond (gid, seq)
INCLUDE (txt);
```

14 조건절 컬럼이 가공됐을 때의 영향

조건절에 사용된 컬럼이 가공되어 인덱스를 사용하지 못하는 사례이다.

1. 사전 구성 스크립트

```
SELECT CASE WHEN RIGHT(CONVERT(VARCHAR,a.number),1) <= 8 THEN 'A'
      ELSE 'B' END Col1
      , CONVERT(VARCHAR,a.number) Col2
      , CONVERT(VARCHAR,b.number) Col3
      , DATEADD(dd,a.number,'2020-01-01') Col4
INTO TB_cost
FROM master..spt_values a, (SELECT number
                            FROM master..spt_values
                            WHERE type ='P'
                            and number <= 500) b
WHERE a.type = 'P'

CREATE UNIQUE INDEX UIDX01_cost ON TB_cost (Col1, Col2, Col3)
GO
```

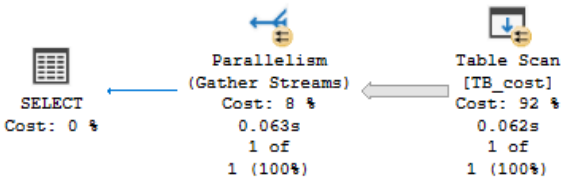
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT Col4
FROM TB_cost
WHERE Col1 = 'A'
AND Col2 + Col3 = '00'
GO
```

· 실행 계획

Rows	Executes	StmtText
1 1 1	1 1 8	SELECT [Col4] FROM [TB_cost] WHERE [Col1]=@1 AND ([Col2]+[Col3])=@2 ¦--Parallelism(Gather Streams) ¦--Table Scan(OBJECT:([tuning].[dbo].[TB_cost]), WHERE:([tuning].[dbo].[TB_cost].[Col1]='A' AND ([tuning].[dbo].[TB_cost].[Col2]+[tuning].[dbo].[TB_cost].[Col3])='00'))



테이블 'TB_cost'. 검색 수 9, 논리적 읽기 수 4097
SQL Server 실행 시간:
CPU 시간 = 545ms, 경과 시간 = 75ms

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_cost	UIDX01_cost	NONCLUSTERED	1	Col1, Col2, Col3	-

· Selectivity

	All density	Average Length	Columns
1	0.5	1	Col1
2	0.0004882813	4.458008	Col1, Col2
3	9.746133E-07	7.238447	Col1, Col2, Col3

· Histogram

	RANGE_HI_KEY	RANGE_ROWS	EQ_ROWS	DISTINCT_RANGE_ROWS	AVG_RANGE_ROWS
1	A	0	923844	0	1
2	B	0	102204	0	1

4. [AS-IS] 구문 설명

[TB_cost] 테이블의 [Col1], [Col2], [Col3] 컬럼에 대해 Equal 조건으로 하여 [Col4] 컬럼의 데이터를 조회하는 구문이다. 조건절에 사용된 세 컬럼은 유니크한 값이기 때문에 1건의 데이터만을 요구함에도 테이블 전체를 읽어내는 것으로 실행 계획이 생성되었다. 이유는 조건절에서 사용된 [Col2], [Col3] 두 컬럼이 가공되었기 때문인데 인덱스를 활용할 경우 [Col1] = 'A'의 조건에 대해서만 인덱스 탐색을 수행할 수 있고 해당 조건에 부합하는 데이터는 100만 건 중 약 90만 건에 해당된다. 최종 출력을 위한 [Col4] 컬럼이 인덱스에 포함되어 있지 않기 때문에 조건에 해당하는 90만 건의 데이터가 RID Lookup될 것을 고려하여 테이블 전체 스캔으로 선택된 것이다.

5. [TO-BE] 튜닝 포인트

[Col1], [Col2], [Col3] 세 컬럼은 유니크한 값을 갖기 때문에 [Col2]와 [Col3] 컬럼이 인덱스 탐색의 조건으로 활용될 수 있도록 가공되지 않은 구문으로 변경한다.

6. [TO-BE] 개선 방안

1) [Col2], [Col3] 컬럼에 사용된 연결 연산자(+)를 제거

[Col2] 컬럼과 [Col3] 컬럼에 사용된 연결 연산자를 제거하고 각각 Equal 조건으로 활용될 수 있도록 구문을 변경한다. 모든 컬럼이 인덱스를 탐색할 수 있는 상태가 되기 때문에 유니크 인덱스를 통해서 단 1건을 위한 탐색 비용만 발생한다.

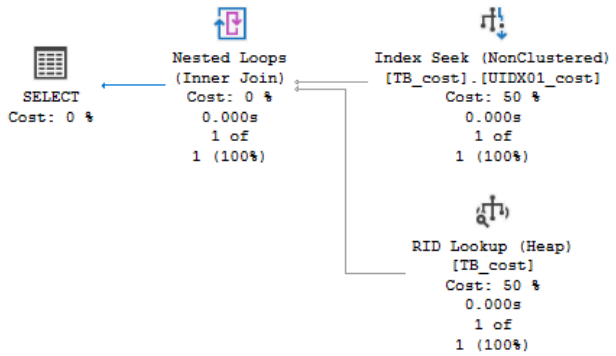
■ 변경된 SQL 구문

```
SELECT Col4
FROM TB_cost
WHERE Col1 = 'A'
AND Col2 = '0'
AND Col3 = '0'
GO
```

· 실행 계획

Rows	Executes	StmtText

1
1
1
 	
1
	1
	1
	1
	
	1
		SELECT [Col4] FROM [TB_cost] WHERE [col1]=@1 AND [Col2]=@2 AND [Col3]=@3
		  ¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([Bmk1000]))
		  ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_cost].[UIDX01_cost]), SEEK:([Tuning].[dbo].[TB_cost].
		  [Col1]=[@1] AND [Tuning].[dbo].[TB_cost].[Col2]=[@2] AND [Tuning].[dbo].[TB_cost].[Col3]=[@3])
		  Ordered Forward)
		  ¦--RID Lookup(OBJECT:([Tuning].[dbo].[TB_cost]), SEEK:([Bmk1000]=[Bmk1000]) LOOKUP Ordered
		  Forward)




```
테이블 'TB_cost'. 검색 수 0, 논리적 읽기 수 4
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

- **리소스 사용률 비교**

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	545	75	4,097
TO-BE	0	0	4

GitHub Copilot 채팅 결과

Claude Opus 4.6

14. 조건절 컬럼이 가공됐을 때의 영향

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```sql
[AS-IS]
SELECT Col4
FROM TB_cost
WHERE Col1 = 'A'
AND Col2 + Col3 = '00';
```

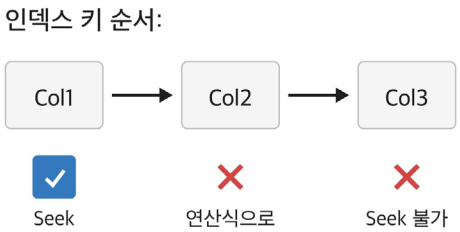
2. 분석 및 주요 최적화

1) 현재 테이블/인덱스 구조

테이블	타입	인덱스	키 컬럼	INCLUDE
dbo.TB_cost	Heap	UIDX01_cost (UNIQUE, NC)	Col1(varchar(1)), Col2(varchar(30)), Col3(varchar(30))	없음

2) 핵심 원인: Non-SARGable 식 Col2 + Col3 = '00'

인덱스 UIDX01_cost(Col1, Col2, Col3)가 존재하지만, 복합 인덱스는 선두 컬럼부터 순서대로 Seek합니다:



조건	SARGable 여부	인덱스 활용
Col1 = 'A'	☑ SARGable	Index Seek 가능
Col2 + Col3 = '00'	✖ Non-SARGable	Col2에 연산(+)이 적용되어 Seek 불가

3) Col2에서 Seek이 끊기면 Col3도 Seek할 수 없고, 결과적으로:

1. Col1 = 'A' 범위만 Index Seek → 나머지는 Residual Predicate로 전건 비교
2. Col4가 인덱스에 없으므로 매 건마다 RID Lookup 발생
3. Col1 = 'A'의 선택도가 낮으면(건수가 많으면) → 옵티마이저가 Table Scan을 선택

3. 개선안

```sql
-- 쿼리 개선 – 연산식 분리 (SARGable로 변환)
-- Col2 + Col3 = '00'을 개별 컬럼 조건으로 분리하면 인덱스 3개 키를
-- 모두 Seek할 수 있습니다:

SELECT Col4
FROM dbo.TB_cost
WHERE Col1 = 'A'
AND Col2 = '0'
AND Col3 = '0';
```

Section 02

실행 계획 및 연산 처리 최적화

실행 계획은 쿼리 성능을 결정짓는 핵심 요소다. 이 섹션에서는 주요 연산자의 동작 원리와 실행 계획 해석을 통해 비효율적인 처리 구조를 개선하는 방법을 다룬다.

15 SPOOL 연산자 성능 개선 - 1

NL Join을 수행하는 구문에 인덱스 부재로 인해 IndexSpool (*Lazy Spool)을 포함한 전체 스캔이 반복되면서 I/O가 많아진 사례이다.

*Lazy Spool : "필요할 때마다 데이터를 가져와 tempdb에 저장하고, 이미 저장된 값은 재사용하는 지연 로딩 방식의 Spool 연산자."

1. 사전 구성 스크립트

```
SELECT a.number id , NEWID() txt, DATEADD(dd,a.number-1,'2020-01-01') date
INTO TB_FSpool02
FROM master..spt_values a, master..spt_values b
WHERE a.type = 'P'
AND a.number BETWEEN 1 AND 10
AND b.type = 'P'
AND b.number BETWEEN 1 AND 5


SELECT ROW_NUMBER() OVER (ORDER BY a.number) id
      , CAST(LEFT(a.number*b.high,2) AS INT) value
INTO TB_SSpool02
FROM master..spt_values a, master..spt_values b
WHERE a.type = 'P'
AND a.number BETWEEN 1 AND 1000
AND b.type = 'P'
AND b.number BETWEEN 1 AND 1000

CREATE CLUSTERED INDEX CIDX_FSpool02 ON TB_FSpool02 (date)
GO
```

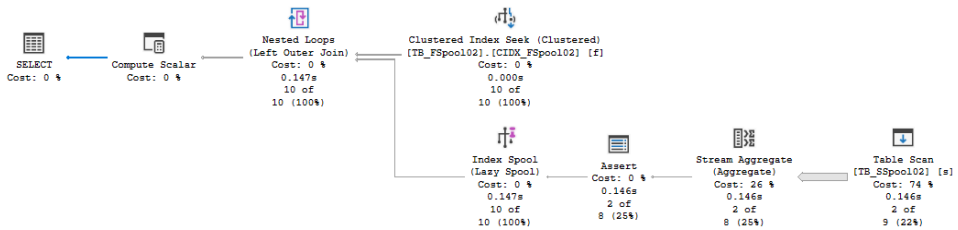
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT f.id , f.txt, (SELECT s.value
                      FROM TB_SSpool02 s
                      WHERE f.id = s.id) value
FROM TB_FSpool02 f
WHERE date BETWEEN '2020-01-01' AND '2020-01-02'
GO
```

· 실행 계획

Rows	Executes	StmtText
10	1	SELECT f.id, f.txt, (SELECT s.value FROM TB_SSpool02 s WHERE f.id = s.id) value FROM TB_FSpool02 f WHERE date BETWEEN '2020-01-01' AND '2020-01-02'
0	0	¦--Compute Scalar(DEFINE:([Expr1005]=[Expr1007]))
10	1	¦--Nested Loops(LEFT OUTER JOIN, OUTER REFERENCES:([f].[id]))
10	1	¦--Clustered Index Seek(OBJECT:([tuning].[dbo].[TB_FSpool02].[CIDX_FSpool02] AS [f]), SEEK:([f].[date] >= '2020-01-01 00:00:00.000' AND [f].[date] <= '2020-01-02 00:00:00.000') Ordered Forward)
10	10	¦--Index Spool(SEEK:([f].[id]=[tuning].[dbo].[TB_FSpool02].[id] as [f].[id]))
2	2	¦--Assert(WHERE:(CASE WHEN [Expr1006]>(1) THEN (0) ELSE NULL END))
2	2	¦--Stream Aggregate(DEFINE:([Expr1006]=Count(*), [Expr1007]=ANY([tuning].[dbo].[TB_SSpool02].[value] as [s].[value])))
2	2	¦--Table Scan(OBJECT:([tuning].[dbo].[TB_SSpool02] AS [s]), WHERE:([tuning].[dbo].[TB_FSpool02].[id] as [f].[id]=[tuning].[dbo].[TB_SSpool02].[id] as [s].[id]))



테이블 'Worktable'. 검색 수 12, 논리적 읽기 수 29
테이블 'TB_SSpool02'. 검색 수 2, 논리적 읽기 수 5196
테이블 'TB_FSpool02'. 검색 수 1, 논리적 읽기 수 2
SQL Server 실행 시간:
CPU 시간 = 63ms, 경과 시간 = 75ms

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_FSpool02	CIDX_FSpool02	CLUSTERED	0	date	-

· Selectivity

	All density	Average Length	Columns
1	0.1	4	id

· **Histogram**

	RANGE_HI_KEY	RANGE_ROWS	EQ_ROWS	DISTINCT_RANGE_ROWS	AVG_RANGE_ROWS
1	1	0	5	0	1
2	3	5	5	1	5
3	5	5	5	1	5
4	7	5	5	1	5
5	9	5	5	1	5
6	10	0	5	0	1

4. [AS-IS] 구문 설명

[TB_FSpool02] 테이블에서 [date] 컬럼 조건이 '2020-01-01 ~ 2020-01-02' 범위에 해당되는 데이터를 서브쿼리인 [TB_SSpool02] 테이블에 조인하여 [value] 값을 출력하는 구문이다. [TB_FSpool02] 테이블에서 [date] 컬럼 범위 조건에 해당하는 [id] 값은 1 또는 2로 중복되어 있으며 서브쿼리에 동일한 값으로 반복되어 조인된다. 조인하는 과정에서 서브쿼리의 인덱스 부재로 인해 Index Spool (Lazy Spool)을 동반한 전체 스캔이 반복되면서 I/O가 높게 사용된다.

Index Spool (Lazy Spool)은 쿼리가 수행되는 동안에만 반복적으로 사용될 데이터를 임시 테이블에 저장한다. 반복적으로 읽어야 하는 테이블을 매번 전체 스캔하는 대신 값이 전달될 때 조인된 결과를 Spool에 추가하고 동일한 값이 전달되면 기존에 생성한 Spool과 조인하면서 읽는 범위를 최소화 한다. 이렇게 생성된 Spool에 내부적으로 비클러스터형 인덱스를 생성하여 데이터를 재사용할 때 인덱스 탐색으로 수행한다.

5. [TO-BE] 튜닝 포인트

Index Spool (Lazy Spool)은 데이터를 반복해서 읽어야 하는 구문에서 인덱스가 부재인 경우에 나타날 수 있다. 필요한 데이터만 인덱스 탐색할 수 있도록 후행 테이블인 [TB_SSpool02]의 조인 조건 컬럼에 인덱스를 생성하면 읽어내는 I/O를 최소화할 수 있다.

6. [TO-BE] 개선 방안

1) [TB_SSpool02] 테이블의 조인 조건 컬럼에 인덱스 생성

[TB_SSpool02] 테이블의 조인 조건 컬럼 [id]를 키로 [value] 컬럼이 포괄 열로 포함된 인덱스를 생성하면 인덱스 탐색으로 I/O를 감소시킬 수 있다.

인덱스 생성 구문

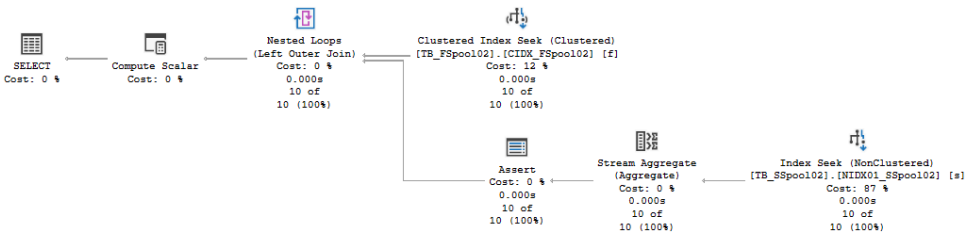
```
CREATE INDEX NIDX01_SSpool02 ON TB_SSpool02 (id) INCLUDE (value)
GO
```

SQL 구문

```
SELECT f.id , f.txt, (SELECT s.value
                      FROM TB_SSpool02 s
                      WHERE f.id = s.id) value
FROM TB_FSpool02 f
WHERE date BETWEEN '2020-01-01' AND '2020-01-02'
GO
```

· 실행 계획

Rows	Executes	StmtText
10	1	SELECT f.id, f.txt, (SELECT s.value FROM TB_SSpool02 s WHERE f.id = s.id) value FROM TB_FSpool02 f WHERE date BETWEEN '2020-01-01' AND '2020-01-02'
0	0	¦--Compute Scalar(DEFINE:([Expr1005]=[Expr1007]))
10	1	¦--Nested Loops(LEFT OUTER JOIN, OUTER REFERENCES:([f].[id]))
10	1	¦--Clustered Index Seek(OBJECT:([tuning].[dbo].[TB_FSpool02].[CIDX_FSpool02] AS [f]), SEEK:([f].[date] >= '2020-01-01 00:00:00.000' AND [f].[date] <= '2020-01-02 00:00:00.000') Ordered Forward)
10	10	¦--Assert(WHERE:(CASE WHEN [Expr1006]>(1) THEN (0) ELSE NULL END))
10	10	¦--Stream Aggregate(DEFINE:([Expr1006]=Count(*), [Expr1007]=ANY([tuning].[dbo].[TB_SSpool02].[value] as [s].[value])))
10	10	¦--Index Seek(OBJECT:([tuning].[dbo].[TB_SSpool02].[NIDX01_SSpool02] AS [s]), SEEK:([s].[id]=[tuning].[dbo].[TB_FSpool02].[id] as [f].[id]) Ordered Forward)




```
테이블 'TB_SSpool02'. 검색 수 10, 논리적 읽기 수 30
테이블 'TB_FSpool02'. 검색 수 1, 논리적 읽기 수 2
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	63	75	5,227
TO-BE	0	0	32

GitHub Copilot 채팅 결과

15. SPOOL 연산자 성능 개선 - 1

Claude Opus 4.6

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT f.id, f.txt, (SELECT s.value FROM TB_SSpool02 s WHERE f.id = s.id) value
FROM TB_FSpool02 f
WHERE date BETWEEN '2020-01-01' AND '2020-01-02';
```

2. 분석 및 주요 최적화

1) 현재 환경 요약

테이블	행 수	인덱스
dbo.TB_FSpool02	50	CIDX_FSpool02 (Clustered, date)
dbo.TB_SSpool02	1,000,000	없음 (Heap)

2) 비효율 원인 분석

1 상관 서브쿼리 (Correlated Subquery) → Table Spool 발생

```
(SELECT s.value FROM TB_SSpool02 s WHERE f.id = s.id)
```

- 이 서브쿼리는 외부 쿼리의 각 행마다 반복 실행됩니다.
- TB_FSpool02에서 필터 된 행 수만큼 TB_SSpool02(100만 행)를 반복 스캔하게 되어, 실행 계획에서 Table Spool 연산이 발생합니다.

2 TB_SSpool02에 인덱스 없음 (Heap Scan)

- TB_SSpool02.id에 인덱스가 없으므로, 서브쿼리가 실행될 때마다 100만 행 전체를 Table Scan 해야 합니다.
- SQL Server가 이를 최적화하기 위해 중간 결과를 임시 저장하는 Spool 연산을 사용하지만, 근본적인 비효율은 해결되지 않습니다.

3 암시적 형변환 (CONVERT_IMPLICIT)

· TB_FSpool02.id는 int이고, TB_SSpool02.id는 bigint입니다.

· 조인 시 암시적 형변환이 발생하여 인덱스가 있더라도 효율적으로 사용하지 못할 수 있습니다.

3. 개선안

```
-- TB_SSpool02에 인덱스가 전혀 없으므로, id 컬럼에 인덱스를 생성하면
-- JOIN 성능이 크게 향상됩니다.

CREATE INDEX IX_TB_SSpool02_id ON dbo.TB_SSpool02 (id) INCLUDE (value);
```

16 SPOOL 연산자 성능 개선 - 2

2개의 서로 다른 컬럼을 각각 중복 제거된 값으로 행 수를 조회할 때 Table Spool (*Eager Spool)이 사용되면서 I/O 비용이 높아진 사례이다.

*Eager Spool : "쿼리 실행 초기에 모든 데이터를 tempdb에 저장해 두는 즉시 로딩 방식의 Spool 연산자."

1. 사전 구성 스크립트

```
SELECT a.number num
      , CHAR(LEFT(b.high, 1) + 64) txt
      , LEFT(a.low,1) group_id
INTO TB_FSpool03
FROM master..spt_values a, master..spt_values b
WHERE a.type = 'P'
AND a.number BETWEEN 1 AND 100
AND b.type = 'P'
AND b.number BETWEEN 1 AND 1000

CREATE INDEX NIDX01_FSpool03 on TB_FSpool03 (group_id) INCLUDE (num, txt)
GO
```

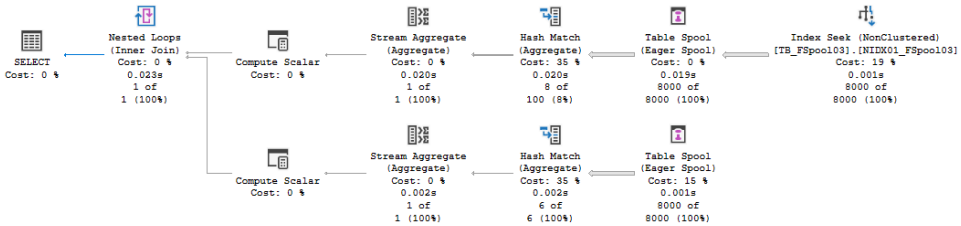
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT COUNT(DISTINCT num) num_cnt
      ,COUNT(DISTINCT txt) txt_cnt
FROM TB_FSpool03
WHERE group_id = '7'
GO
```

· 실행 계획

Rows	Executes	StmtText
1	1	SELECT COUNT(DISTINCT [num]) [num_cnt],COUNT(DISTINCT [txt]) [txt_cnt] FROM [TB_FSpool03] WHERE [group_id]=@1
1	1	┆--Nested Loops(INNER JOIN)
0	0	┆--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(int,[Expr1009],0)))
1	1	┆ ┆--Stream Aggregate(DEFINE:([Expr1009]=Count(*)))
8	1	┆ ┆--Hash Match(Aggregate, HASH:([Tuning].[dbo].[TB_FSpool03].[num]))
8000	1	┆ ┆--Table Spool
8000	1	┆ ┆--Index Seek(OBJECT:([Tuning].[dbo].[TB_FSpool03].[NIDX01_FSpool03]), SEEK:([Tuning].[dbo].[TB_FSpool03].[group_id]='7') Ordered Forward)
0	0	┆--Compute Scalar(DEFINE:([Expr1004]=CONVERT_IMPLICIT(int,[Expr1010],0)))
1	1	┆--Stream Aggregate(DEFINE:([Expr1010]=COUNT([Tuning].[dbo].[TB_FSpool03].[txt])))
6	1	┆--Hash Match(Aggregate, HASH:([Tuning].[dbo].[TB_FSpool03].[txt]), RESIDUAL:([Tuning].[dbo].[TB_FSpool03].[txt] = [Tuning].[dbo].[TB_FSpool03].[txt]))
8000	1	┆--Table Spool




```
테이블 'Worktable'. 검색 수 2, 논리적 읽기 수 16151
테이블 'Workfile'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_FSpool03'. 검색 수 1, 논리적 읽기 수 28
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 14ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_FSpool03	NIDX01_FSpool03	NONCLUSTERED	0	group_id	num, txt

4. [AS-IS] 구문 설명

[TB_FSpool03] 테이블의 [group_id] 컬럼 값이 '7' 인 데이터에서 [num] 컬럼과 [txt] 컬럼에 대해 각각 중복 값을 제거한 행 수를 출력하는 구문이다. 각 컬럼 별로 중복 제거된 행을 추출하기 위해선 컬럼마다 데이터를 새로 읽어내야 하는데, 이를 위해서 조건절로 탐색된 8천 건의 데이터를 임시 테이블로 저장하고 이를 재사용하면서 결과값을 추출한다. 중간 결과 집합 전체를 임시 테이블로 저장하고 재사용 하기 위해 Table Spool (Eager Spool)이 사용되는데 수행하는 과정에서 임시 테이블에 대한 I/O 비용이 높게 발생한다.

5. [TO-BE] 튜닝 포인트

두 컬럼에 대해 중복 값이 제거된 행을 추출하기 위해 인덱스로 탐색된 8천 건의 데이터를 임시 테이블로 저장하고 재사용하면서 많은 I/O가 사용되었다. COUNT(DISTINCT num)와 COUNT(DISTINCT txt)의 값을 각각 조회하도록 구문을 분리하고 두 결과를 조인하도록 변경하면 임시 데이터를 만들고 사용하는 비용을 제거할 수 있기 때문에 I/O를 개선할 수 있다.

6. [TO-BE] 개선 방안

1) 중복 값이 제거된 행 수의 추출 구문을 각 컬럼 별로 분리하여 조인하도록 구문 변경

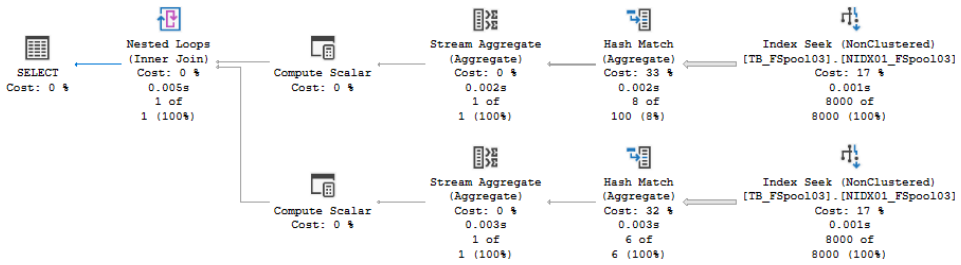
[num], [txt] 각 컬럼의 중복 제거된 행 수를 각각 조회하고 두 데이터를 CROSS JOIN한다. 인덱스를 통해 8천 건의 데이터를 탐색하는 비용만 2회 발생되기 때문에 전체적인 I/O 비용을 개선할 수 있다.

■ 변경된 SQL 구문

```
SELECT num_cnt, txt_cnt
FROM (SELECT COUNT(DISTINCT num) num_cnt
      FROM TB_FSpool03
      WHERE group_id = '7') a,
     (SELECT COUNT(DISTINCT txt) txt_cnt
      FROM TB_FSpool03
      WHERE group_id = '7') b
GO
```

· 실행 계획

Rows	Executes	StmtText
1	1	SELECT num_cnt, txt_cnt FROM (SELECT COUNT(DISTINCT num) num_cnt FROM TB_FSpool03 WHERE group_id = '7') a, (SELECT COUNT(DISTINCT txt) txt_cnt FROM TB_FSpool03 WHERE group_id = '7') b
1	1	┦--Nested Loops(INNER JOIN)
0	0	┦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(int,[Expr1012],0)))
1	1	┦ ┦--Stream Aggregate(DEFINE:([Expr1012]=Count(*)))
8	1	┦ ┦--Hash Match(Aggregate, HASH:([Tuning].[dbo].[TB_FSpool03].[num]))
8000	1	┦ ┦--Index Seek(OBJECT:([Tuning].[dbo].[TB_FSpool03].[NIDX01_FSpool03]), SEEK:([Tuning].[dbo].[TB_FSpool03].[group_id]='7') Ordered Forward)
0	0	┦--Compute Scalar(DEFINE:([Expr1007]=CONVERT_IMPLICIT(int,[Expr1013],0)))
1	1	┦--Stream Aggregate(DEFINE:([Expr1013]=COUNT([Tuning].[dbo].[TB_FSpool03].[txt])))
6	1	┦--Hash Match(Aggregate, HASH:([Tuning].[dbo].[TB_FSpool03].[txt]), RESIDUAL:([Tuning].[dbo].[TB_FSpool03].[txt] = [Tuning].[dbo].[TB_FSpool03].[txt]))
8000	1	┦--Index Seek(OBJECT:([Tuning].[dbo].[TB_FSpool03].[NIDX01_FSpool03]), SEEK:([Tuning].[dbo].[TB_FSpool03].[group_id]='7') Ordered Forward)



테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'Workfile'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_FSpool03'. 검색 수 2, 논리적 읽기 수 56
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 2ms

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	0	14	16,179
TO-BE	0	2	56

GitHub Copilot 채팅 결과

Claude Opus 4.6

16. SPOOL 연산자 성능 개선 - 2

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT COUNT(DISTINCT num) num_cnt, COUNT(DISTINCT txt) txt_cnt
FROM TB_FSpool03
WHERE group_id = '7'
```

2. 분석 및 주요 최적화

다중 COUNT(DISTINCT) → Table Spool 하나의 SELECT에 COUNT(DISTINCT)가 2개 있으면, SQL Server는 첫 번째 DISTINCT 집계 후 두 번째를 위해 8,000행을 tempdb에 Table Spool로 복사해서 재탐색합니다.

```
Index Seek (group_id='7') → 8,000행
├─ Table Spool (8,000행) → Stream Aggregate (DISTINCT num) ← 1차 처리
└─ Table Spool (8,000행) → Stream Aggregate (DISTINCT txt) ← 2차 처리
```

*커버링 인덱스가 있어 데이터 접근 자체는 효율적이지만, Table Spool이 8,000행을 tempdb에 기록하고 재읽기하는 과정에서 불필요한 I/O가 발생합니다.

3. 개선안

```
-- COUNT(DISTINCT) 분리, 단일 SELECT에 2개 독립 서브쿼리 2개
-- 각 서브쿼리가 Index Seek → Stream Aggregate를 독립 수행.
-- Table Spool 완전 제거

SELECT
  (SELECT COUNT(DISTINCT num) FROM dbo.TB_FSpool03 WHERE group_id = '7') AS num_cnt,
  (SELECT COUNT(DISTINCT txt) FROM dbo.TB_FSpool03 WHERE group_id = '7') AS txt_cnt;
```

*Index Seek가 1회 → 2회로 늘지만, 커버링 인덱스에서 동일 group_id = '7' 페이지들이 버퍼 캐시에 이미 적재되어 2차 Seek는 물리적 I/O 없이 메모리에서 처리됩니다. 반면 8,000행의 tempdb Table Spool이 완전히 제거되므로 전체 I/O가 감소합니다.

17 SPOOL 연산자 성능 개선 - 3

스칼라 서브쿼리에 반복적으로 조인될 때 인덱스 부재로 인해 Index Spool (*Eager Spool)이 사용되면서 I/O가 많아진 사례이다.

*Eager Spool : "쿼리 실행 초기에 모든 데이터를 tempdb에 저장해 두는 즉시 로딩 방식의 Spool 연산자."

1. 사전 구성 스크립트

```
SELECT a.number id , NEWID() txt, DATEADD(dd,a.number-1,'2020-01-01') date INTO
TB_FSpool04
FROM master..spt_values a
WHERE a.type = 'P'
AND a.number BETWEEN 1 AND 1000

SELECT ROW_NUMBER() OVER (ORDER BY a.number) id
      , CHAR(LEFT(CONVERT(NVARCHAR(4), high), 1) + 64) code
      , RIGHT(number * high * low, 2) value
INTO TB_SSpool04
FROM master..spt_values a
WHERE a.type = 'P'
AND a.number BETWEEN 1 AND 1000

CREATE CLUSTERED INDEX CIDX_FSpool04 ON TB_FSpool04 (date)
GO
```

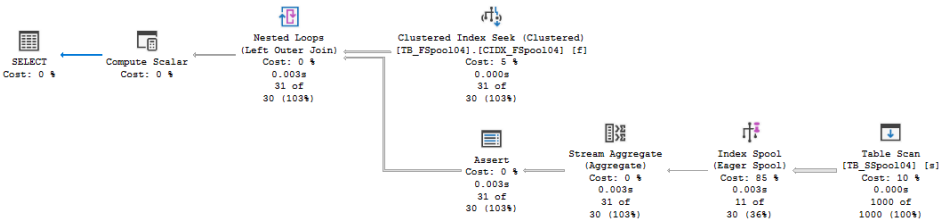
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT f.id, f.txt, (SELECT value
                     FROM TB_SSpool04 s
                     WHERE f.id=s.id
                     AND s.code ='A') value
FROM TB_FSpool04 f
WHERE f.date BETWEEN '2020-01-01' AND '2020-01-31'
GO
```

· 실행 계획

Rows	Executes	StmtText
31	1	SELECT f.id, f.txt, (SELECT value FROM TB_SSpool04 s WHERE f.id=s.id AND s.code ='A') value FROM TB_FSpool04 f WHERE f.date BETWEEN '2020-01-01' AND '2020-01-31'
0	0	┦--Compute Scalar(DEFINE:([Expr1005]=[Expr1007]))
31	1	┦--Nested Loops(LEFT OUTER JOIN, OUTER REFERENCES:([f].[id]))
31	1	┦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_FSpool04].[CIDX_FSpool04] AS [f]), SEEK:([f].[date] >= '2020-01-01 00:00:00.000' AND [f].[date] <= '2020-01-31 00:00:00.000') Ordered Forward)
31	31	┦--Assert(WHERE:(CASE WHEN [Expr1006]>(1) THEN (0) ELSE NULL END))
31	31	┦--Stream Aggregate(DEFINE:([Expr1006]=Count(*), [Expr1007]=ANY([Tuning].[dbo].[TB_SSpool04].[value] as [s].[value])))
11	31	┦--Index Spool(SEEK:([s].[id]=[Tuning].[dbo].[TB_FSpool04].[id] as [f].[id] AND [s].[code]='A'))
1000	1	┦--Table Scan(OBJECT:([Tuning].[dbo].[TB_SSpool04] AS [s]))




```
테이블 'Worktable'. 검색 수 31, 논리적 읽기 수 2077
테이블 'TB_SSpool04'. 검색 수 1, 논리적 읽기 수 3
테이블 'TB_FSpool04'. 검색 수 1, 논리적 읽기 수 2
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 1ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_FSpool04	CIDX_FSpool04	CLUSTERED	0	date	-

4. [AS-IS] 구문 설명

[TB_FSpool04] 테이블에서 [date] 컬럼이 2020년 1월 범위에 해당되는 데이터의 [id] 컬럼 기준으로 [TB_SSpool04] 테이블에 조인하고 [code] 컬럼이 'A'값에 일치하는 데이터의 [value] 값을 같이 출력되도록 서브쿼리로 작성된 구문이다. 서브쿼리인 [TB_SSpool04] 테이블에는 인덱스가 존재하지 않기 때문에 [TB_FSpool04] 테이블의 결과 건수만큼 테이블 전체 스캔을 반복해야 하는데, 매번 모든 데이터를 읽어내는 대신 테이블 전체를 읽어 Worktable(임시 테이블)에 저장하고 내부적으로 비클러스터형 인덱스를 생성하여 이후 입력값에 대해선 Spool에 인덱스로 탐색하는 것을 반복한다. Spool은 임시 테이블에 저장되고 사용되며 이로 인해 Worktable에 대한 I/O가 높게 사용된다.

5. [TO-BE] 튜닝 포인트

NL Join이 수행되면서 [TB_SSpool04] 테이블을 매번 전체 스캔해야 하기 때문에 이를 대신하여 Spool이 사용되었다. 조인 조건에 사용된 [id] 컬럼과 조건절인 [code] 컬럼에 인덱스를 생성하면 조인이 수행될 때 필요한 데이터만 인덱스로 탐색하기 때문에 Spool을 통해 저장하고 재사용하는 과정을 제거할 수 있다.

6. [TO-BE] 개선 방안

1) [TB_SSpool04] 테이블의 조건 컬럼에 신규 인덱스 생성

[TB_SSpool04] 테이블의 [id]와 [code] 컬럼에 인덱스를 생성하여 인덱스 탐색을 통해 읽어내는 I/O를 최소화한다.

■ 인덱스 생성 구문

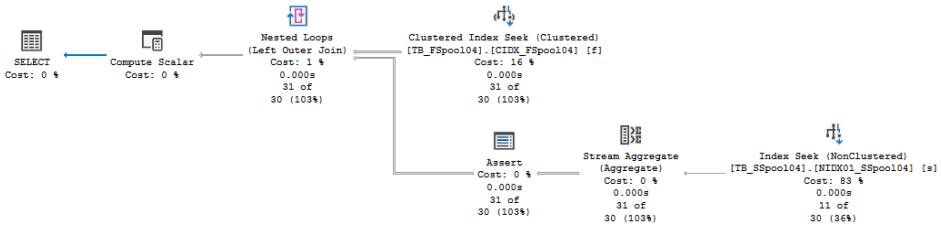
```
CREATE INDEX NIDX01_SSpool04 ON TB_SSpool04 (id, code) INCLUDE (value)
GO
```

■ SQL 구문

```
SELECT f.id, f.txt, ( SELECT value
                      FROM TB_SSpool04 s
                      WHERE f.id=s.id
                      AND s.code ='A') value
FROM TB_FSpool04 f
WHERE f.date BETWEEN '2020-01-01' AND '2020-01-31'
GO
```

- 실행 계획

Rows	Executes	StmtText
31	1	SELECT f.id, f.txt, (SELECT value FROM TB_SSpool04 s WHERE f.id=s.id AND s.code ='A') value FROM TB_FSpool04 f WHERE f.date BETWEEN '2020-01-01' AND '2020-01-31'
0	0	¦--Compute Scalar(DEFINE:([Expr1005]=[Expr1007]))
31	1	¦--Nested Loops(LEFT OUTER JOIN, OUTER REFERENCES:([f].[id]))
31	1	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_FSpool04].[CIDX_FSpool04] AS [f]), SEEK:([f].[date] >= '2020-01-01 00:00:00.000' AND [f].[date] <= '2020-01-31 00:00:00.000') Ordered Forward)
31	31	¦--Assert(WHERE:(CASE WHEN [Expr1006]>(1) THEN (0) ELSE NULL END))
31	31	¦--Stream Aggregate(DEFINE:([Expr1006]=Count(*), [Expr1007]=ANY([Tuning].[dbo].[TB_SSpool04].[value] as [s].[value]))
11	31	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_SSpool04].[NIDX01_SSpool04] AS [s]), SEEK:([s].[id]=[Tuning].[dbo].[TB_FSpool04].[id] as [f].[id] AND [s].[code]='A') Ordered Forward)




```
테이블 'TB_SSpool04'. 검색 수 31, 논리적 읽기 수 62
테이블 'TB_FSpool04'. 검색 수 1, 논리적 읽기 수 2
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	0	1	2,082
TO-BE	0	0	64

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

17. SPOOL 연산자 성능 개선 - 3

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인 분석 및 개선해줘.

```
[AS-IS]
SELECT f.id, f.txt,
(SELECT value
FROM TB_SSpool04 s
WHERE f.id = s.id AND s.code = 'A') value
FROM TB_FSpool04 f
WHERE f.date BETWEEN '2020-01-01' AND '2020-01-31';
```

2. 분석 및 주요 최적화

1) 테이블 구조 요약

테이블	컬럼	데이터 타입	인덱스
dbo.TB_FSpool04	id	int	없음
	date	datetime	Clustered (CIDX_FSpool04)
dbo.TB_SSpool04	id	bigint	없음
	code	char(1)	없음
	value	varchar(4)	-

2) 비효율 원인 분석

#	문제 유형	설명	영향도
1	상관 서브쿼리 (Correlated Subquery)	SELECT절의 서브쿼리가 TB_FSpool04의 매 행마다 반복 실행됩니다. 실행 계획에서 Table Spool 연산자가 발생하여 tempdb I/O가 증가합니다.	높음
2	암시적 형변환 (CONVERT_IMPLICIT)	TB_FSpool04.id는 int, TB_SSpool04.id는 bigint으로 데이터 타입이 다릅니다. 조인 시 int → bigint 암시적 변환이 발생하여 인덱스 활용이 제한됩니다.	중간
3	인덱스 부재	TB_SSpool04에 인덱스가 전혀 없어 조인 조건(id, code)에 대해 매번 Full Table Scan이 발생합니다.	높음

3. 개선안

```
-- 쿼리 개선 – 상관 서브쿼리 → LEFT JOIN 변환

SELECT f.id, f.txt, s.value
FROM dbo.TB_FSpool04 f
LEFT JOIN dbo.TB_SSpool04 s ON f.id = s.id AND s.code = 'A'
WHERE f.date BETWEEN '2020-01-01' AND '2020-01-31';

변경 사항
- 상관 서브쿼리를 LEFT JOIN으로 변환하여 Table Spool 제거
- 두 테이블을 한 번만 스캔하므로 반복 실행 비용 제거
- 2-part name(dbo.) 명시


-- 인덱스 생성 - TB_SSpool04 커버링 인덱스

CREATE NONCLUSTERED INDEX IX_SSpool04_id_code
ON dbo.TB_SSpool04 (id, code)
INCLUDE (value);


-- 데이터 타입 일치 – 암시적 형변환 해소
-- TB_SSpool04.id 컬럼의 타입을 bigint → int로 변경하거나, 설계상 bigint이
-- 필요한 경우 TB_FSpool04.id를 bigint으로 통일합니다.
-- 옵션 A: TB_SSpool04.id를 int로 변경 (TB_FSpool04와 동일하게)

ALTER TABLE dbo.TB_SSpool04 ALTER COLUMN id int;
```

· 개선 효과 요약

항목	AS-IS	TO-BE
조인 방식	상관 서브쿼리 (행마다 반복)	LEFT JOIN (1회 조인)
TB_SSpool04 접근	Full Table Scan × N회	Index Seek × 1회
형변환	암시적 변환 발생	타입 일치 (변환 없음)
Table Spool	발생	제거

18 TOP EXPRESSION (SUBQUERY)

스칼라 서브쿼리를 사용하여 가장 큰 값을 조회할 때 인덱스 부재로 인해 전체 데이터를 읽어 내면서 I/O 비용이 높아진 사례이다.

1. 사전 구성 스크립트

```
SELECT DISTINCT type
INTO TB_Type
FROM master..spt_values

SELECT a.*
INTO TB_Value
FROM master..spt_values a, master..spt_values b
WHERE a.number <= 1000 AND b.number <= 1000

CREATE INDEX NIDX01_Type ON TB_Type(type)
GO
```

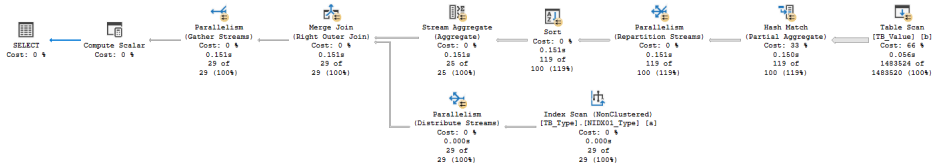
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT a.type, (SELECT MAX(number)
                FROM TB_Value b
                WHERE a.type = b.type) AS max_number
FROM TB_Type a
GO
```

· 실행 계획

Rows	Executes	StmtText
29	1	SELECT a.type, (SELECT MAX(number) FROM TB_Value b WHERE a.type = b.type) AS max_number FROM TB_Type a
0	0	¦--Compute Scalar(DEFINE:([Expr1006]=[Expr1004]))
29	1	¦--Parallelism(Gather Streams)
29	8	¦--Merge Join(Right Outer Join, MERGE:([b].[type])=([a].[type]), RESIDUAL:([tuning].[dbo].[TB_Type].[type] as [a].[type]=[tuning].[dbo].[TB_Value].[type] as [b].[type]))
25	8	¦--Stream Aggregate(GROUP BY:([b].[type]) DEFINE:([Expr1004]=MAX([partialagg1007])))
117	8	¦ ¦--Sort(ORDER BY:([b].[type] ASC))
117	8	¦ ¦--Parallelism(Repartition Streams, Hash Partitioning, PARTITION COLUMNS:([b].[type]))
117	8	¦ ¦--Hash Match(Partial Aggregate, HASH:([b].[type]), RESIDUAL:([tuning].[dbo].[TB_Value].[type] as [b].[type] = [tuning].[dbo].[TB_Value].[type] as [b].[type]) DEFINE:([partialagg1007]=MAX([tuning].[dbo].[TB_Value].[number] as [b].[number])))
1483524	8	¦ ¦--Table Scan(OBJECT:([tuning].[dbo].[TB_Value] AS [b]))
29	8	¦--Parallelism(Distribute Streams, Hash Partitioning, PARTITION COLUMNS:([a].[type]))
29	1	¦--Index Scan(OBJECT:([tuning].[dbo].[TB_Type].[NIDX01_Type] AS [a]), Ordered Forward)




```
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'Workfile'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_Value'. 검색 수 9, 논리적 읽기 수 6540
테이블 'TB_Type'. 검색 수 1, 논리적 읽기 수 2
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
SQL Server 실행 시간:
CPU 시간 = 813ms, 경과 시간 = 123ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Type	NIDX01_Type	NONCLUSTERED	0	type	-

4. [AS-IS] 구문 설명

[TB_Type] 테이블을 [TB_Value] 테이블과 [type] 컬럼으로 조인하여 [TB_Type] 테이블의 각 행에 대해 가장 큰 [number] 값을 조회하는 구문이다. 후행으로 조인되는 [TB_Value] 테이블에는 인덱스가 존재하지 않아 모든 데이터를 읽어내면서 I/O가 높게 발생된다.

5. [TO-BE] 튜닝 포인트

[TB_Value] 테이블의 조인 조건인 [type] 컬럼과 MAX 값 추출을 위한 [number] 컬럼에 인덱스가 부재된 것이 원인이다. [type], [number] 컬럼이 키로 구성된 인덱스를 사용하면 [type] 컬럼 기준으로 조인된 결과 집합이 [number] 컬럼 기준으로 정렬된 상태이기 때문에 가장 큰 값을 추출하기 위해 모든 데이터를 읽지 않아도 [type] 별 상위 1건씩만 읽어내어 I/O를 감소시킬 수 있다.

6. [TO-BE] 개선 방안

1) [TB_Value] 테이블의 [type], [number] 컬럼이 키로 구성된 인덱스를 생성

신규 인덱스를 사용하여 [TB_Type] 테이블의 데이터 29건에 대해 [TB_Value] 테이블을 TOP Expression 연산을 통해 1건씩만 읽어내어 개선된 것을 확인할 수 있다.

■ 인덱스 생성 구문

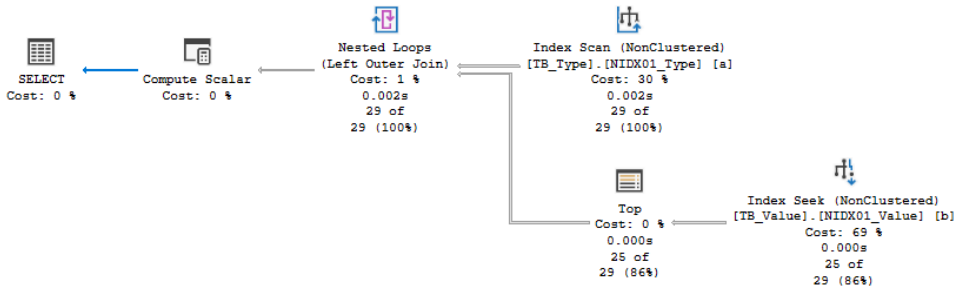
```
CREATE INDEX NIDX01_Value on TB_Value(type, number)
GO
```

■ SQL 구문

```
SELECT a.type, (SELECT MAX(number)
                FROM TB_Value b
                WHERE a.type = b.type) AS max_number
FROM TB_Type a
GO
```

· 실행 계획

Rows	Executes	StmtText
29 0 29 29 25 25	1 0 1 1 29 29	SELECT a.type, (SELECT MAX(number) FROM TB_Value b WHERE a.type = b.type) AS max_number FROM TB_Type a ¦--Compute Scalar(DEFINE:([Expr1006]=[Tuning].[dbo].[TB_Value].[number] as [b].[number])) ¦--Nested Loops(LEFT OUTER JOIN, OUTER REFERENCES:([a].[type])) ¦--Index Scan(OBJECT:([Tuning].[dbo].[TB_Type].[NIDX01_Type] AS [a])) ¦--Top(TOP EXPRESSION:((1))) ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_Value].[NIDX01_Value] AS [b]), SEEK:([b].[type]=[Tuning].[dbo].[TB_Type].[type] as [a].[type]) ORDERED Backward)



```
테이블 'TB_Value'. 스캔 수 29, 논리적 읽기 87
테이블 'TB_Type'. 스캔 수 1, 논리적 읽기 2
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· **리소스 사용률 비교**

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	813	123	6,542
TO-BE	0	0	89

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

18. TOP EXPRESSION (SUBQUERY)

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT a.type, (SELECT MAX(number)
                FROM TB_Value b
                WHERE a.type = b.type) AS max_number
FROM TB_Type a;
```

2. 분석 및 주요 최적화

1) 현재 테이블/인덱스 구조

테이블	행 수	크기	클러스터형 인덱스	비클러스터형 인덱스
dbo.TB_Type	29	< 1 MB	없음 (HEAP)	NIDX01_Type(type)
dbo.TB_Value	1,485,961	51 MB	없음 (HEAP)	없음

2) 데이터 분포

항목	값
TB_Value 고유 type 수	25
type 당 평균 행 수	~ 59,438행

3) Query Store 실행 통계 (query_id = 128)

plan_id	병렬 실행	평균 Logical I/O	평균 Duration (μs)	평균 CPU (μs)
27	✔	6,568	36,934	161,934
109	✔	6,566	111,254	505,920

4) 비효율 원인 분석

1 상관 서브쿼리(Correlated Subquery)로 인한 반복 실행 (핵심 원인)

```sql
SELECT a.type, (SELECT MAX(number)    -- ← TB_Type의 행마다 반복 실행
                FROM TB_Value b
                WHERE a.type = b.type) AS max_number
FROM TB_Type a
```

- TB_Type 29행 × 서브쿼리 매번 실행 → 옵티마이저가 Hash Aggregate + Table Scan으로 풀어도, 논리적으로 비효율 구조
- 옵티마이저가 자동 변환(Spool/Hash Match)을 하지만, 항상 최적 플랜이 보장되지 않음

2 dbo.TB_Value에 인덱스가 전혀 없음 (핵심 원인)

- 148만 행 HEAP 테이블에 인덱스 없음 → MAX(number) 계산 시 매번 Full Table Scan (6,556 데이터 페이지) 발생
- type 컬럼 필터링을 위한 인덱스가 없어 Seek 불가능
- Query Store 기록: Logical I/O ~6,566 = HEAP 전체 스캔 수준 병렬 실행으로 인한 CPU 과다 사용
- Duration ~37ms인데 CPU ~162ms → 병렬 처리 오버헤드(스레드 동기화 비용)가 실제 작업량보다 큼
- 29행 결과를 위해 과도한 리소스 소모

3. 개선안

```sql
-- 커버링 인덱스 생성 (근본 해결)
-- 쿼리 형태와 무관하게, TB_Value에 적절한 인덱스를 생성하면 Table Scan
-- → Index Seek로 전환됩니다.
-- TB_Value: type별 MAX(number) 조회를 위한 커버링 인덱스
-- type으로 Seek 후, number의 MAX를 인덱스 내에서 바로 계산 가능

CREATE NONCLUSTERED INDEX NIDX01_Value_type_number
ON dbo.TB_Value (type, number DESC);
```

19 TOP EXPRESSION (NOT EXISTS)

NOT EXISTS 구문을 수행할 때 인덱스 부재로 인해 스캔 범위가 많아지면서 높은 I/O 비용이 사용된 사례이다.

1. 사전 구성 스크립트

```
CREATE TABLE TB_Exist (no INT, name VARCHAR(20))
INSERT INTO TB_Exist VALUES (1,'대한민국'), (2,'중국'), (3,'미국'), (4,'일본'),
                            (5,'러시아'),(6,'런던'), (7,'프랑스'), (8,'베트남'),
                            (9,'필리핀'), (10,'호주')
SELECT CASE WHEN (c.number%10+1)=6 THEN `` ELSE (SELECT name
                                                 FROM TB_Exist
                                                 WHERE no = c.number%10+1) END AS NAME
INTO TB_List
FROM (SELECT ROW_NUMBER() OVER(ORDER BY a.name) AS number
      FROM master..spt_values a, master..spt_values b
      WHERE a.type = 'P' AND b.type='P'
      AND a.number < 1000 AND b.number < 1000
) c
GO
```

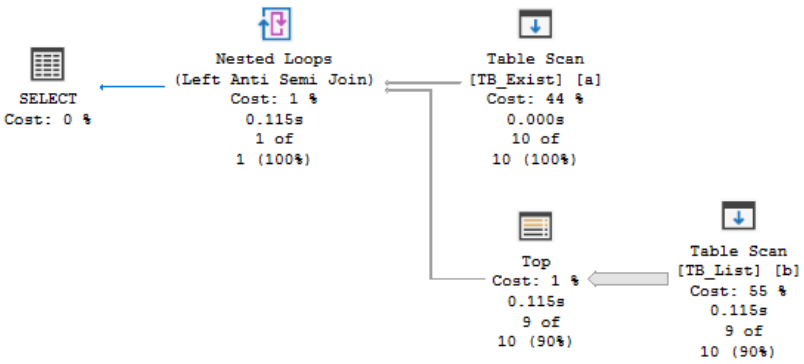
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT a.name
FROM TB_Exist a
WHERE NOT EXISTS (SELECT name
                  FROM TB_List b
                  WHERE b.name = a.name)
GO
```

· 실행 계획

Rows	Executes	StmtText
1 1 10 9 9	1 1 1 10 10	SELECT a.name FROM TB_Exist a WHERE NOT EXISTS (SELECT name FROM TB_List b WHERE b.name = a.name) ¦--Nested Loops(Left Anti Semi Join, OUTER REFERENCES:([a].[name])) ¦--Table Scan(OBJECT:([tuning].[dbo].[TB_Exist] AS [a])) ¦--Top(TOP EXPRESSION:((1))) ¦--Table Scan(OBJECT:([tuning].[dbo].[TB_List] AS [b]), WHERE:([tuning].[dbo].[TB_List].[NAME] as [b].[NAME]=[tuning].[dbo].[TB_Exist].[name] as [a].[name]))




```
테이블 'TB_List'. 검색 수 1, 논리적 읽기 수 2359
테이블 'TB_Exist'. 검색 수 1, 논리적 읽기 수 1
SQL Server 실행 시간:
CPU 시간 = 110ms, 경과 시간 = 115ms
```

3. [AS-IS] 통계 정보

- **인덱스 정보 없음**

4. [AS-IS] 구문 설명

[TB_Exist] 테이블 10건의 데이터를 [TB_List] 테이블에 NOT EXISTS 절을 이용하여 조인에 실패하는 데이터를 출력하는 구문이다. NOT EXISTS 구문은 Anti Semi Join으로 수행되는데, 이는 메인테이블 하나의 행에 대해 조인에 성공하는 행을 만날 때까지 서브 테이블을 탐색하는 방식이며 그 중 조인에 실패하는 행을 최종 결과 집합으로 포함한다. [TB_Exist] 테이블 10건에 대해 [TB_List] 테이블로 조인에 실패하는 행은 1건이며 해당 행을 찾기 위해 모든 데이터를 읽어내면서 높은 I/O를 사용하게 된다.

5. [TO-BE] 튜닝 포인트

NOT EXISTS 절의 서브 테이블로 사용된 [TB_List] 테이블의 데이터가 정렬되어 있지 않기 때문에 조인에 성공하는 행을 만날 때까지 모든 행들을 읽어내면서 I/O가 높게 사용되는 것이다. 조인 조건에 사용되는 [name] 컬럼에 인덱스를 생성하여 정렬된 데이터를 사용하면 인덱스로 탐색되는 데이터 상위 1건만으로 조인의 성공 실패 유무를 확인할 수 있기 때문에 I/O를 개선할 수 있다.

6. [TO-BE] 개선 방안

1) [TB_List] 테이블의 [name] 컬럼이 키로 구성된 인덱스 생성

인덱스를 통한 탐색만으로 조인의 성공 실패 유무가 확인되어 I/O가 개선된다.

■ 인덱스 생성 구문

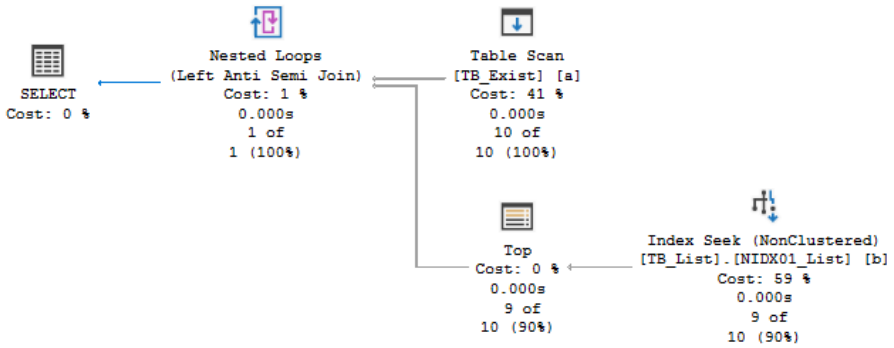
```
CREATE INDEX NIDX01_List ON TB_List(name)
GO
```

■ SQL 구문

```
SELECT a.name
FROM TB_Exist a
WHERE NOT EXISTS (SELECT name
                  FROM TB_List b
                  WHERE b.name = a.name)
GO
```

- 실행 계획

Rows	Executes	StmtText
1	1	SELECT a.name FROM TB_Exist a WHERE NOT EXISTS (SELECT name FROM TB_List b WHERE b.name = a.name)
1	1	¦--Nested Loops(Left Anti Semi Join, OUTER REFERENCES:([a].[name]))
10	1	¦--Table Scan(OBJECT:([tuning].[dbo].[TB_Exist] AS [a]))
9	10	¦--Top(TOP EXPRESSION:((1)))
9	10	¦--Index Seek(OBJECT:([tuning].[dbo].[TB_List].[NIDX01_List] AS [b]), SEEK:([b].[NAME]=[tuning].[dbo].[TB_Exist].[name] as [a].[name]) Ordered Forward)




```
테이블 'TB_List'. 검색 수 10, 논리적 읽기 수 40
테이블 'TB_Exist'. 검색 수 1, 논리적 읽기 수 1
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	110	115	2,360
TO-BE	0	0	41

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

19. TOP EXPRESSION (NOT EXISTS)

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT a.name
FROM TB_Exist a
WHERE NOT EXISTS (SELECT name
                  FROM TB_List b
                  WHERE b.name = a.name);
```

2. 분석 및 주요 최적화

```
100만 행 Heap에 인덱스 없음. NOT EXISTS 서브쿼리가 TB_List.NAME을 조건으로 사용하지
만, 인덱스가 없어 매번 Full Table Scan 발생.
TB_Exist 10행 × TB_List 100만행 = 최대 1,000만 행 탐색

TB_Exist (10행 Heap Scan)
└ 각 행마다 → TB_List (1,000,000행 Heap Full Scan)
= 10 × 1,000,000 = 10,000,000행 탐색
```

*NOT EXISTS 패턴 자체는 Anti-Join의 최적 패턴입니다. NOT IN보다 NULL-safe하고 옵티마이저 친화적이므로 쿼리 구조는 변경할 필요가 없습니다.

3. 개선안

```
-- 쿼리 구조 자체는 이미 최적이므로, 성능 개선의 핵심은 인덱스 추가입니다.

CREATE NONCLUSTERED INDEX NIDX01_List ON dbo.TB_List (NAME);
```

20 TOP EXPRESSION (TOP N)

TOP N 구문을 수행할 때 인덱스 누락으로 인해 높은 I/O 비용이 발생된 사례이다.

1. 사전 구성 스크립트

```
CREATE TABLE TB_People(num INT IDENTITY(1,1)
      , secureNo INT
      , cardno VARCHAR(50) DEFAULT NEWID())

INSERT INTO TB_People (secureNo)
SELECT ROW_NUMBER() over(ORDER BY a.number) as secureNo
FROM master..spt_values a, (SELECT number
                            FROM master..spt_values
                            WHERE number < 100
                            AND type = 'P') b
WHERE a.type = 'P'
GO
```

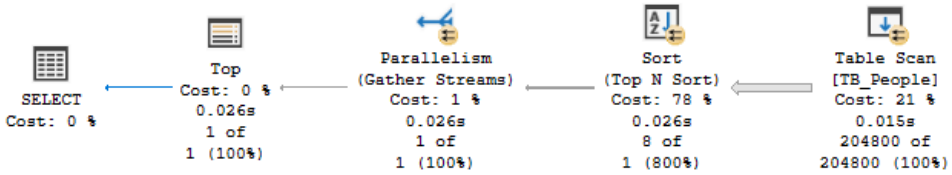
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT TOP 1 *
FROM TB_People
ORDER BY secureNo
GO
```

· 실행 계획

Rows	Executes	StmtText
1	1	SELECT TOP 1 * FROM TB_People ORDER BY secureNo
1	1	|--Top(TOP EXPRESSION:((1)))
1	1	|--Parallelism(Gather Streams, ORDER BY:([tuning].[dbo].[TB_People].[secureNo] ASC))
8	8	|--Sort(TOP 1, ORDER BY:([tuning].[dbo].[TB_People].[secureNo] ASC))
204800	8	|--Table Scan(OBJECT:([tuning].[dbo].[TB_People]))




```
테이블 'TB_People'. 스캔 수 9, 논리적 읽기 1443
SQL Server 실행 시간:
CPU 시간 = 111ms, 경과 시간 = 26ms
```

3. [AS-IS] 통계 정보

- **인덱스 없음**

4. [AS-IS] 구문 설명

[TB_People] 테이블에서 [secureNo] 컬럼 기준으로 가장 작은 값 1건을 출력하는 구문이다. [secureNo] 컬럼에는 인덱스가 존재하지 않아 데이터가 정렬되지 않은 상태이다. 이 같은 상황에서 가장 작은 값의 판별을 위해 모든 데이터를 읽으면서 정렬이 수행되고 읽어내야 하는 데이터가 많아지면서 높은 I/O 비용이 사용된다.

5. [TO-BE] 튜닝 포인트

[secureNo] 컬럼의 데이터가 정렬되어 있다면 첫 행만 읽어내도 가장 작은 값을 판별할 수 있다. 인덱스를 생성하여 데이터를 정렬하고 읽어내는 범위를 개선한다.

6. [TO-BE] 개선 방안

1) [secureNo] 컬럼이 키로 구성된 인덱스를 생성

인덱스를 생성하면 키를 기준으로 데이터가 정렬된다. ORDER BY 절에 사용된 [secureNo] 컬럼이 정렬된 상태이기 때문에 첫 행만 읽어내면서 가장 작은 값을 추출할 수 있다.

■ 인덱스 생성 구문

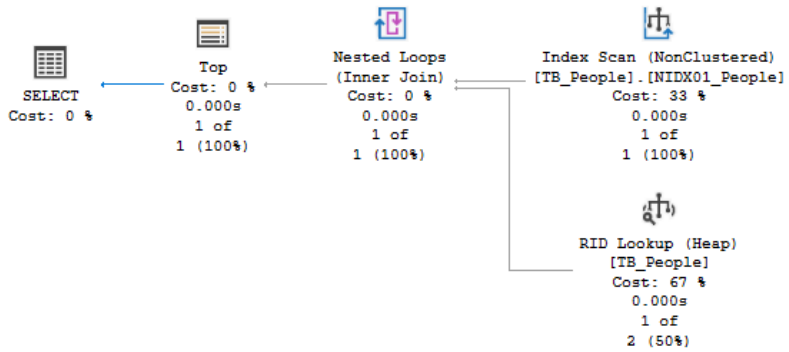
```
CREATE INDEX NIDX01_People ON TB_People(secureNo)
GO
```

SQL 구문

```
SELECT TOP 1 *
FROM TB_People
ORDER BY secureNo
GO
```

· 실행 계획

Rows	Executes	StmtText
1 1 1 1 1	1 1 1 1 1	SELECT TOP 1 * FROM TB_People ORDER BY secureNo ¦--Top(TOP EXPRESSION:((1))) ¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([Bmk1000], [Expr1003]) WITH ORDERED PREFETCH) ¦--Index Scan(OBJECT:([Tuning].[dbo].[TB_People].[NIDX01_People]), Ordered Forward) ¦--RID Lookup(OBJECT:([Tuning].[dbo].[TB_People]), SEEK:([Bmk1000]=[Bmk1000]) LOOKUP Ordered Forward)



테이블 'TB_People'. 스캔 수 1, 논리적 읽기 4
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	111	26	1,443
TO-BE	0	0	4

GitHub Copilot 채팅 결과

Claude Opus 4.6

20. TOP EXPRESSION (TOP N)

1. 프롬프트

```
[AS-IS]
구문에 성능 측면에서 비효율이 발생하는 원인 분석 및 개선해줘.
```

```
[AS-IS]
SELECT TOP 1 * FROM TB_People
ORDER BY secureNo;
```

2. 분석 및 주요 최적화

1) 인덱스가 전혀 없는 Heap 테이블입니다. 최적화된 쿼리를 검증하겠습니다.

2) 안티패턴 설명

1. SELECT * 모든 컬럼 반환. 명시적으로 필요한 컬럼만 지정해야 합니다
2. 인덱스 없는 ORDER BY secureNo에 인덱스가 없어 204,800행 전체를 Heap Scan → Sort 후 TOP 1 반환. 검색 횟수 13은 **병렬 처리(Parallelism)**가 동원된 것을 의미합니다

3) 병목 원인

Heap Full Scan (204,800행, 13 병렬 스레드) → Sort (secureNo 기준 전체 정렬) → Top 1 (1행만 반환) = 1행을 얻기 위해 204,800행 전체를 읽고 정렬

3. 개선안

```
-- 핵심 권장사항: 커버링 인덱스 생성 쿼리 구조 변경만으로는 Full Scan + Sort가
-- 제거되지 않습니다.
-- 근본적인 해결을 위해 아래 인덱스가 필수입니다.

CREATE NONCLUSTERED INDEX NIDX01_People ON dbo.TB_People (secureNo)
INCLUDE (num, cardno);
```

*secureNo 기준 인덱스가 있으면 TOP 1 ORDER BY secureNo는 인덱스의 첫 번째 행을 바로 읽는 것으로 해결됩니다. Full Scan, Sort, Parallelism이 모두 제거되어 논리적 읽기가 1,443 → ~3 (99.8% 감소)으로 줄어듭니다.

Section 03

쿼리 구조 및 서브쿼리 최적화

같은 결과를 반환하더라도 쿼리 구조에 따라 성능은 크게 달라질 수 있다. 이 섹션에서는 서브쿼리, 조인 구조, 쿼리 재작성 등을 통해 효율적인 SQL 작성 방법을 소개한다.

21 중복된 서브쿼리 사용으로 테이블을 여러 번 읽는 이슈

동일한 조건으로 이뤄진 서브쿼리로 인해 테이블을 여러 번 읽어내면서 불필요한 I/O가 발생한 사례이다.

1. 사전 구성 스크립트

```
SELECT a.seq
, a.seq%7 AS no1
, a.seq%13 AS no2 INTO TB_Subupdate
FROM (SELECT ROW_NUMBER() OVER (ORDER BY a.number) AS seq
       FROM master..spt_values a
       JOIN master..spt_values b
       ON a.type = 'P' AND b.type = 'P'
       AND a.number BETWEEN 1 AND 100 AND b.number BETWEEN 1 AND 10
) a

SELECT a.seq
      , DATEPART(YY, DATEADD(YY, -b.number%100, '2020-09-10 00:00:00.000')) AS year
      , low AS no1
      , high AS no2 INTO TB_Sublist
FROM TB_Subupdate a, master..spt_values b
WHERE b.type = 'P' and b.number BETWEEN 100 AND 110

ALTER TABLE TB_Subupdate ALTER COLUMN seq bigint NOT NULL;
ALTER TABLE TB_Subupdate ADD CONSTRAINT PK_TB_Subupdate_seq PRIMARY KEY (seq);

ALTER TABLE TB_Sublist ALTER COLUMN year int NOT NULL;
ALTER TABLE TB_Sublist ALTER COLUMN seq bigint NOT NULL;
ALTER TABLE TB_Sublist ADD CONSTRAINT PK_TB_Sublist_year_seq PRIMARY KEY (year, seq);

CREATE NONCLUSTERED INDEX NIDX_TB_Sublist_seq ON TB_Sublist (seq)

ALTER TABLE TB_Sublist
ADD CONSTRAINT FK_TB_Subupdate_seq FOREIGN KEY (seq)
REFERENCES TB_Subupdate(seq);
GO
```

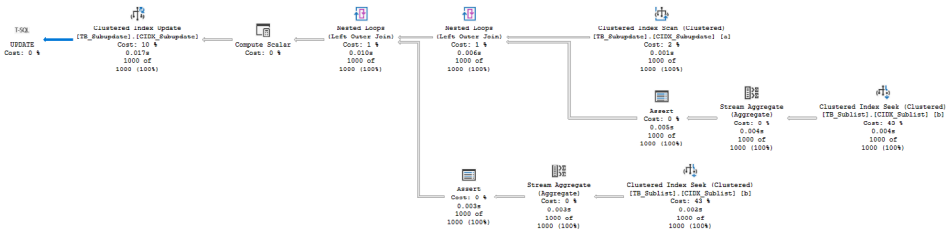
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
UPDATE TB_Subupdate
SET no1=(SELECT no1 FROM TB_Sublist b WHERE b.seq=a.seq AND year = 2019),
      no2=(SELECT no2 FROM TB_Sublist b WHERE b.seq=a.seq AND year = 2019)
FROM TB_Subupdate a
GO
```

· 실행 계획

Rows	Executes	StmtText
1000	1	UPDATE TB_Subupdate SET no1=(SELECT no1 FROM TB_Sublist b WHERE b.seq=a.seq AND year = 2019),
1000	1	no2=(SELECT no2 FROM TB_Sublist b WHERE b.seq=a.seq AND year = 2019) FROM TB_Subupdate a ┦--Clustered Index Update(OBJECT:([tuning].[dbo].[TB_Subupdate].[CIDX_Subupdate]), SET:([tuning].[
0	0	dbo].[TB_Subupdate].[no1] = [Expr1005],[tuning].[dbo].[TB_Subupdate].[no2] = [Expr1009])) ┦--Compute Scalar(DEFINE:([Expr1005]=CONVERT_IMPLICIT(bigint,[Expr1016],0), [Expr1009]=CONVERT
1000	1	_IMPLICIT(bigint,[Expr1018],0))) ┦--Nested Loops(LEFT OUTER JOIN, OUTER REFERENCES:([a].[seq]))
1000	1	┦--Nested Loops(LEFT OUTER JOIN, OUTER REFERENCES:([a].[seq]))
1000	1	┦ ┦--Clustered Index Scan(OBJECT:([tuning].[dbo].[TB_Subupdate].[CIDX_Subupdate] AS
1000	1000	[a]))
1000	1000	┦ ┦--Assert(WHERE:(CASE WHEN [Expr1015]>(1) THEN (0) ELSE NULL END))
1000	1000	┦ ┦--Stream Aggregate(DEFINE:([Expr1015]=Count(*), [Expr1016]=ANY([tuning].[dbo]
1000	1000	.[TB_Sublist].[no1] as [b].[no1])))
1000	1000	┦ ┦--Clustered Index Seek(OBJECT:([tuning].[dbo].[TB_Sublist].[CIDX_Sublist]
1000	1000	AS [b]), SEEK:([b].[seq]=[tuning].[dbo].[TB_Subupdate].[seq] as [a].[seq]
1000	1000	AND [b].[year]=(2019)) Ordered Forward) ┦--Assert(WHERE:(CASE WHEN [Expr1017]>(1) THEN (0) ELSE NULL END))
1000	1000	┦--Stream Aggregate(DEFINE:([Expr1017]=Count(*), [Expr1018]=ANY([tuning].[dbo].[TB
1000	1000	_Sublist].[no2] as [b].[no2])))
1000	1000	┦--Clustered Index Seek(OBJECT:([tuning].[dbo].[TB_Sublist].[CIDX_Sublist] AS
		[b]), SEEK:([b].[seq]=[tuning].[dbo].[TB_Subupdate].[seq] as [a].[seq] AND
		[b].[year]=(2019)) Ordered Forward)



테이블 'TB_Subupdate'. 검색 수 1, 논리적 읽기 수 7
테이블 'TB_Sublist'. 검색 수 2000, 논리적 읽기 수 4012
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 8ms

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Sublist	PK_TB_Sublist_year_seq	CLUSTERED	1	year,seq	-
TB_Sublist	NIDX_TB_Sublist_seq	NONCLUSTERED	0	seq	-
TB_Subupdate	PK_TB_Subupdate_seq	CLUSTERED	1	seq	-

4. [AS-IS] 구문 설명

[TB_Subupdate] 테이블의 [no1], [no2] 컬럼 데이터를 [TB_Sublist] 테이블로 조인하여 업데이트하는 구문이다. 각 컬럼 별로 동일 조건의 서브쿼리를 사용하여 업데이트를 수행하는데, NL Join으로 수행되면서 [TB_Sublist] 테이블을 반복적으로 탐색하게 되고 [no1], [no2] 두 컬럼에 대해 서브쿼리가 각각 조인되기 때문에 같은 테이블이 반복 사용되면서 불필요한 I/O가 발생한다.

5. [TO-BE] 튜닝 포인트

동일 조건의 [TB_Sublist] 테이블을 서브쿼리로 여러 번 사용하면서 불필요한 I/O가 사용된다. 조인문으로 구문을 변경하면 테이블을 한 번만 사용하면서 모든 컬럼을 업데이트할 수 있고, 두 테이블은 조인 조건에 클러스터형 인덱스 키가 존재하기 때문에 정렬을 생략한 Merge Join으로 수행하여 반복 탐색하는 비용도 개선할 수 있다.

6. [TO-BE] 개선 방안

1) 서브쿼리를 조인문으로 변경

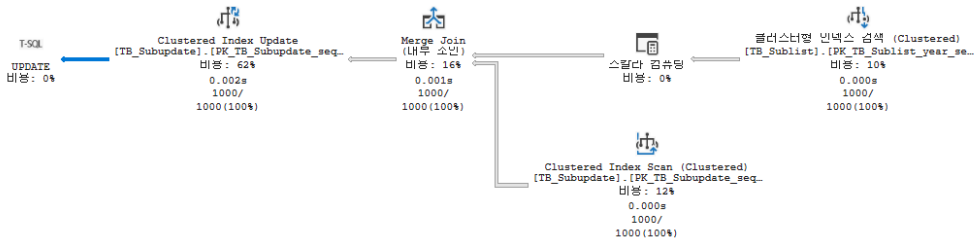
[TB_Subupdate] 테이블과 [TB_Sublist] 테이블을 INNER JOIN 구문으로 변경하여 각 테이블을 한 번만 읽어내도록 개선한다.

■ 변경된 SQL 구문

```
UPDATE a
SET no1 = b.no1,
    no2 = b.no2
FROM TB_Subupdate a
INNER JOIN TB_Sublist b ON a.seq=b.seq
WHERE b.year = 2019
GO
```

· 실행 계획

Rows	Executes	StmtText
1000	1	UPDATE a SET no1 = b.no1, no2 = b.no2 FROM TB_Subupdate a INNER JOIN TB_Sublist b ON a.seq=b.seq WHERE b.year = 2019
1000	1	¦--Clustered Index Update(OBJECT:([tuning].[dbo].[TB_Subupdate].[PK_TB_Subupdate_seq] AS [a]), SET:([maxgauge].[dbo].[TB_Subupdate].[no1] as [a].[no1] = [Expr1002],[tuning].[dbo].[TB_Subupdate].[no2] as [a].[no2] = [Expr1003]))
1000	1	¦--Merge Join(Inner Join, MERGE:([b].[seq])=([a].[seq]), RESIDU-AL:([tuning].[dbo].[TB_Sublist].[seq] as [b].[seq]=[tuning].[dbo].[TB_Subupdate].[seq] as [a].[seq]))
0	0	¦--Compute Sca-lar(DEFINE:([Expr1002]=CONVERT_IMPLICIT(bigint,[tuning].[dbo].[TB_Sublist].[no1] as [b].[no1],0), [Expr1003]=CONVERT_IMPLICIT(bigint,[tuning].[dbo].[TB_Sublist].[no2] as [b].[no2],0)))
1000	1	¦ ¦--Clustered Index Seek(OBJECT:([tuning].[dbo].[TB_Sublist].[PK_TB_Sublist_year_seq] AS [b]), SEEK:([b].[year]=(2019)) ORDERED FORWARD)
1000	1	¦--Clustered Index Scan(OBJECT:([tuning].[dbo].[TB_Subupdate].[PK_TB_Subupdate_seq] AS [a]), ORDERED FOR-WARD)
1000	1	¦--Clustered Index Scan(OBJECT:([tuning].[dbo].[TB_Subupdate].[CIDX_Subupdate] AS [a]), Ordered Forward)




```
테이블 'TB_Subupdate'. 검색 수 1, 논리적 읽기 수 7
테이블 'TB_Sublist'. 검색 수 1, 논리적 읽기 수 6
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 2ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	0	8	4,019
TO-BE	0	2	13

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

21. 중복된 서브쿼리 사용으로 테이블을 여러 번 읽는 이슈

1. 프롬프트

> [AS-IS]
> 구문을 실행하면 불필요하게 TB_Sublist 테이블을 두 번 탐색하고 있으며, 서브쿼리로 인해 NL Join에서 과도한 I/O가 발생한다. 이 부분을 확인하여 성능을 개선해줘.

```
[AS-IS]
UPDATE TB_Subupdate
SET no1=(SELECT no1 FROM TB_Sublist b WHERE b.seq = a.seq AND year = 2019),
    no2=(SELECT no2 FROM TB_Sublist b WHERE b.seq = a.seq AND year = 2019)
FROM TB_Subupdate a;
```

2. 분석 및 주요 최적화

1) 문제점 분석

항목	원본 쿼리	비고
TB_Sublist 접근 횟수	2회 (서브쿼리 2개)	동일 조건으로 중복 탐색
조인 방식	상관 서브쿼리 → NL Join	TB_Subupdate 행마다 반복 Seek
I/O	TB_Subupdate 건수 × 2 만큼 누적	불필요한 I/O 발생

2) 핵심 포인트

1. dbo.TB_Sublist의 클러스터드 인덱스가 (year, seq) 순이므로, WHERE b.year = 2019 조건이 선 두 컬럼에 해당하여 효율적인 Range Scan이 가능합니다.
2. 상관 서브쿼리를 JOIN으로 전환하면 옵티마이저가 Hash Join / Merge Join 등 더 효율적인 전략을 선택할 수 있는 여지도 생깁니다.

3. 개선안

```sql
-- 상관 서브쿼리 2개를 INNER JOIN 1회로 통합하여 TB_Sublist를 한 번만 읽도록
-- 변경합니다.

UPDATE a
SET a.no1 = b.no1, a.no2 = b.no2
FROM dbo.TB_Subupdate a
INNER JOIN dbo.TB_Sublist b ON b.seq = a.seq
WHERE b.year = 2019
```

22 스칼라 서브쿼리 성능 이슈

스칼라 서브쿼리가 NL Join으로 반복 호출되면서 많은 I/O가 사용된 사례이다.

1. 사전 구성 스크립트

```sql
SELECT rownum AS seq
      , NEWID() AS sub_code
      , DATEADD(ss,rownum,'2020-01-01 00:00:00.000') AS date
```

```
INTO TB_Sub05
FROM (SELECT ROW_NUMBER() OVER (ORDER BY A.number) AS rownum, A.number
      FROM master..spt_values A
      JOIN master..spt_values B
      ON A.type = 'P' AND B.type = 'P' AND A.number <= 1000 AND B.number <= 1000
) A

SELECT sub_code AS main_code
     , CASE WHEN seq < 30000 THEN 11 ELSE CAST(seq%10 AS INT) END AS value
INTO TB_Main05
FROM TB_Sub05

CREATE CLUSTERED INDEX CIDX_Main05 ON TB_Main05(main_code)
CREATE INDEX NIDX01_Main05 ON TB_Main05(value)
CREATE CLUSTERED INDEX CIDX_Sub05 ON TB_Sub05(date)
GO
```

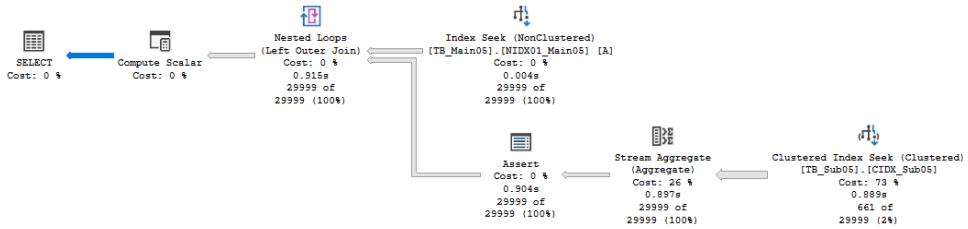
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT *, (SELECT sub_code
           FROM TB_Sub05
           WHERE sub_code = A.main_code
           AND date BETWEEN '2020-01-01 00:09:00.000' AND '2020-01-01 00:20:00.000')
FROM TB_Main05 A
WHERE A.value = 11
GO
```

· 실행 계획

Rows	Executes	StmtText
29999	1	SELECT *, (SELECT sub_code FROM TB_Sub05 WHERE sub_code = A.main_code AND date BETWEEN '2020-01-01 00:09:00.000' AND '2020-01-01 00:20:00.000') FROM TB_Main05 A WHERE A.value = 11
0	0	¦--Compute Scalar(DEFINE:([Expr1006]=[Expr1008]))
29999	1	¦--Nested Loops(LEFT OUTER JOIN, OUTER REFERENCES:([A].[main_code]))
29999	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_Main05].[NIDX01_Main05] AS [A]), SEEK:([A].[value]=(11)) Ordered Forward)
29999	29999	¦--Assert(WHERE:(CASE WHEN [Expr1007]>(1) THEN (0) ELSE NULL END))
29999	29999	¦--Stream Aggregate(DEFINE:([Expr1007]=Count(*), [Expr1008]=ANY([Tuning].[dbo].[TB_Sub05].[sub_code]))
661	29999	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Sub05].[CIDX_Sub05]), SEEK:([Tuning].[dbo].[TB_Sub05].[date] >= '2020-01-01 00:09:00.000' AND [Tuning].[dbo].[TB_Sub05].[date] <= '2020-01-01 00:20:00.000'), WHERE:([Tuning].[dbo].[TB_Sub05].[sub_code]=[Tuning].[dbo].[TB_Main05].[main_code] as [A].[main_code]) Ordered Forward)




```
테이블 'TB_Sub05'. 스캔 수 29999, 논리적 읽기 239992
테이블 'TB_Main05'. 스캔 수 1, 논리적 읽기 100
SQL Server 실행 시간:
CPU 시간 = 937ms, 경과 시간 = 1034ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Main05	CIDX_Main05	CLUSTERED	0	main_code	-
TB_Main05	NIDX01_Main05	NONCLUSTERED	0	value	-
TB_Sub05	CIDX_Sub05	CLUSTERED	0	date	-

4. [AS-IS] 구문 설명

[TB_Main05] 테이블의 [value] 컬럼 값이 11인 데이터를 스칼라 서브쿼리 내의 [TB_Sub05] 테이블과 조인하여 [date] 컬럼 '2020-01-01 00:09:00.000'부터 '2020-01-01 00:20:00.000'까지 범위 안에 포함된 데이터가 존재하면 함께 출력하는 구문이다. 스칼라 서브쿼리는 하나의 입력 값에 대해 출력되는 결과로 2개 이상을 가질 수 없는데, 데이터가 유일한 값이라는 것을 보장할 수 없기 때문에 데이터 검증을 위해 NL Join으로 수행된다. 이로 인해서 메인 쿼리의 결과 집합인 약 3만 건의 데이터가 스칼라 서브쿼리로 반복 호출되기 때문에 많은 I/O가 사용된다.

5. [TO-BE] 튜닝 포인트

다음 구문을 개선하기 위해 두 가지 방안을 고려할 수 있다.
첫째, 스칼라 서브쿼리 구문에 DISTINCT 추가
둘째, 스칼라 서브쿼리를 LEFT OUTER JOIN으로 구문 변경

첫 번째 방법은 스칼라 서브쿼리 구문에 [TB_Sub05] 테이블의 [sub_code] 컬럼 값에 중복 값이 없지만 DISTINCT를 추가하여 [sub_code] 값이 유일한 값임을 정의하여 NL Join이 아닌 Merge Join 방식으로 수행되도록 유도하는 방법이다.

두 번째 방법은 스칼라 서브쿼리로 작성된 구문을 LEFT OUTER JOIN 문으로 변경하여 다른 조인 방식으로 수행되도록 유도하는 방법이다. [TB_Sub05] 테이블의 [sub_code] 컬럼 값에 중복 값이 있는 경우라면 전체 건수의 차이가 발생됨으로 주의하여야 한다.

6. [TO-BE] 개선 방안

1) 스칼라 서브쿼리 구문에 DISTINCT를 추가

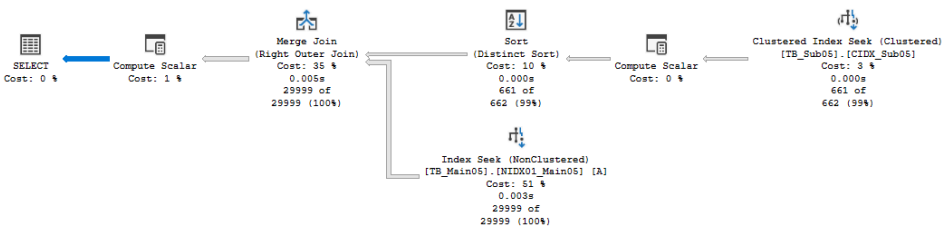
스칼라 서브쿼리 구문에 DISTINCT 추가로 유일한 값을 보장하여 Merge Join을 통해 반복 수행으로 인한 I/O를 개선한다.

■ 변경된 SQL 구문

```
SELECT *, (SELECT DISTINCT sub_code
           FROM TB_Sub05
           WHERE sub_code = A.main_code
           AND date BETWEEN '2020-01-01 00:09:00.000' AND '2020-01-01 00:20:00.000')
FROM TB_Main05 A
WHERE A.value = 11
GO
```

· 실행 계획

Rows	Executes	StmtText
29999	1	SELECT *, (SELECT DISTINCT sub_code FROM TB_Sub05 WHERE sub_code = A.main_code AND date BETWEEN '2020-01-01 00:09:00.000' AND '2020-01-01 00:20:00.000') FROM TB_Main05 A WHERE A.value = 11
0	0	¦--Compute Scalar(DEFINE:([Expr1006]=[Tuning].[dbo].[TB_Sub05].[sub_code]))
29999	1	¦--Merge Join(Right Outer Join, MERGE:([Tuning].[dbo].[TB_Sub05].[sub_code])=([A].[main_code]), RESIDUAL:([Tuning].[dbo].[TB_Sub05].[sub_code]=[Tuning].[dbo].[TB_Main05].[main_code] as [A].[main_code]))
661	1	¦--Sort(DISTINCT ORDER BY:([Tuning].[dbo].[TB_Sub05].[sub_code] ASC))
0	0	¦ ¦--Compute Scalar(DEFINE:([Expr1005]=[Tuning].[dbo].[TB_Sub05].[sub_code]))
661	1	¦ ¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Sub05].[CIDX_Sub05]), SEEK:([Tuning].[dbo].[TB_Sub05].[date] >= '2020-01-01 00:09:00.000' AND [Tuning].[dbo].[TB_Sub05].[date] <= '2020-01-01 00:20:00.000') Ordered Forward)
29999	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_Main05].[NIDX01_Main05] AS [A]), SEEK:([A].[value]=(11)) Ordered Forward)




```
테이블 'TB_Main05'. 검색 수 1, 논리적 읽기 수 100
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_Sub05'. 검색 수 1, 논리적 읽기 수 8
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 447ms
```

2) 스칼라 서브쿼리를 LEFT OUTER JOIN으로 구문 변경

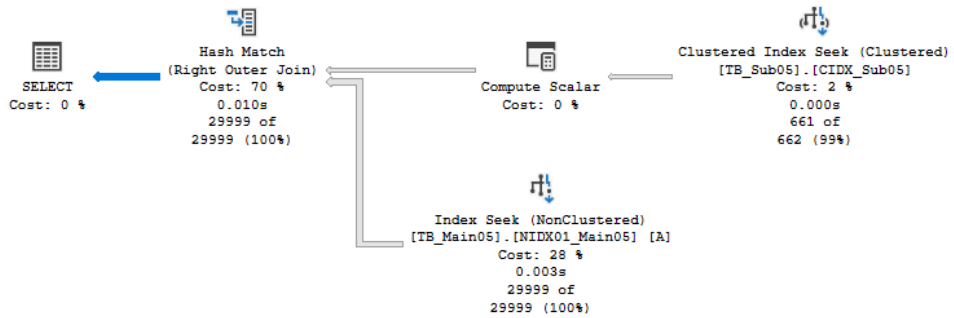
스칼라 서브쿼리를 LEFT OUTER JOIN으로 구문을 변경하여 Hash Join을 통해 [TB_Sub05] 테이블을 반복적으로 읽으면서 발생된 I/O 비용을 개선한다.

■ 변경된 SQL 구문

```
SELECT *
FROM TB_Main05 A
LEFT OUTER JOIN (SELECT sub_code
                 FROM TB_Sub05
                 WHERE date BETWEEN '2020-01-01 00:09:00.000'
                       AND '2020-01-01 00:20:00.000') B
ON A.main_code = B.sub_code
WHERE A.value = 11
GO
```

· 실행 계획

Rows	Executes	StmtText
29999	1	SELECT * FROM TB_Main05 A LEFT OUTER JOIN (SELECT sub_code FROM TB_Sub05 WHERE date BETWEEN '2020-01-01 00:09:00.000' AND '2020-01-01 00:20:00.000') B ON A.main_code = B.sub_code WHERE A.value = 11
29999	1	¦--Hash Match(Right Outer Join, HASH:([Tuning].[dbo].[TB_Sub05].[sub_code])=([A].[main_code]), RESIDUAL:([Tuning].[dbo].[TB_Main05].[main_code] as [A].[main_code]=[Tuning].[dbo].[TB_Sub05].[sub_code]))
0	0	¦--Compute Scalar(DEFINE:([Expr1005]=[Tuning].[dbo].[TB_Sub05].[sub_code]))
661	1	¦ ¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Sub05].[CIDX_Sub05]), SEEK:([Tuning].[dbo].[TB_Sub05].[date] >= '2020-01-01 00:09:00.000' AND [Tuning].[dbo].[TB_Sub05].[date] <= '2020-01-01 00:20:00.000') Ordered Forward)
29999	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_Main05].[NIDX01_Main05] AS [A]), SEEK:([A].[value]=(11)) Ordered Forward)




```
테이블 'Workfile'. 스캔 수 0, 논리적 읽기 0
테이블 'Worktable'. 스캔 수 0, 논리적 읽기 0
테이블 'TB_Main05'. 스캔 수 1, 논리적 읽기 100
테이블 'TB_Sub05'. 스캔 수 1, 논리적 읽기 8
SQL Server 실행 시간:
CPU 시간 = 16ms, 경과 시간 = 436ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	937	1,034	240,092
TO-BE (1)	0	447	108
TO-BE (2)	16	436	108

GitHub Copilot 채팅 결과

Claude Opus 4.6

22. 스칼라 서브쿼리 성능 이슈

1. 프롬프트

> [AS-IS]
> 구문은 서브쿼리 형식으로 인해 TB_Main05 테이블에서 value가 11에 해당하는 건 수만큼 서브쿼리가 반복 처리되어 성능 이슈가 발생해. 성능 개선해줘.

```
[AS-IS]
SELECT *, (SELECT sub_code
           FROM TB_Sub05
           WHERE sub_code = A.main_code
           AND date BETWEEN '2020-01-01 00:09:00.000' AND '2020-01-01 00:20:00.000')
FROM TB_Main05 A
WHERE A.value = 11;
```

2. 분석 및 주요 최적화

1) 문제점 분석

항목	원본 쿼리	비고
서브쿼리 방식	상관 서브쿼리	value = 11 건수만큼 반복 실행
TB_Sub05 인덱스	클러스터드: date only	sub_code 조건에 대한 인덱스 없음
SELECT *	안티패턴	필요 컬럼만 명시해야 함

2) 핵심 포인트

1. 원본 스칼라 서브쿼리는 매칭 없으면 NULL을 반환합니다.
   이 동작을 동일하게 유지하려면 INNER JOIN이 아닌 LEFT JOIN이 필요합니다.
2. LEFT JOIN 전환으로 옵티마이저가 Hash Join 등 효율적인 전략을 선택할 수 있게 됩니다.
3. dbo.TB_Sub05에 sub_code 컬럼 인덱스가 없으므로, 추가 인덱스 생성 시 더 큰 개선이 가능합니다.

3. 개선안

```
-- 상관 서브쿼리를 LEFT JOIN으로 전환하여 TB_Sub05를 한 번만 읽도록 변경합니다.

SELECT A.main_code, A.value, B.sub_code
```

```
FROM dbo.TB_Main05 A
LEFT JOIN dbo.TB_Sub05 B ON B.sub_code = A.main_code
AND B.date >= '2020-01-01 00:09:00.000'
AND B.date <= '2020-01-01 00:20:00.000'
WHERE A.value = 11;
```

23 NOT IN 조건으로 사용된 서브쿼리 성능 이슈

NOT IN 조건에 서브쿼리를 사용할 때 NL Join으로 인해 반복되는 조인 시도로 높은 I/O 비용이 발생된 사례이다.

1. 사전 구성 스크립트

```
SELECT c.id
      , CASE WHEN c.id <= 5 THEN 1
        WHEN c.id <= 10 THEN 2
        WHEN c.id <= 15 THEN 3
        WHEN c.id <= 20 THEN 4
        ELSE 5 END gid
      , NEWid() code1
      , NEWid() code2 INTO TB_Maind8
FROM (SELECT ROW_NUMBER() OVER ( ORDER BY a.number ) id
       FROM master..spt_values A, master..spt_values B
       WHERE a.type = 'P' AND a.number BETWEEN 1 AND 100
       AND b.type = 'P' AND b.number BETWEEN 1 AND 100
) c

SELECT a.* INTO TB_Subd8
FROM TB_Maind8 a

CREATE CLUSTERED INDEX CIDX_Maind8 ON TB_Maind8(gid)
CREATE INDEX NIDX01_Subd8 ON TB_Subd8(id)
GO
```

2. [AS-IS] SQL 구문 및 실행 계획

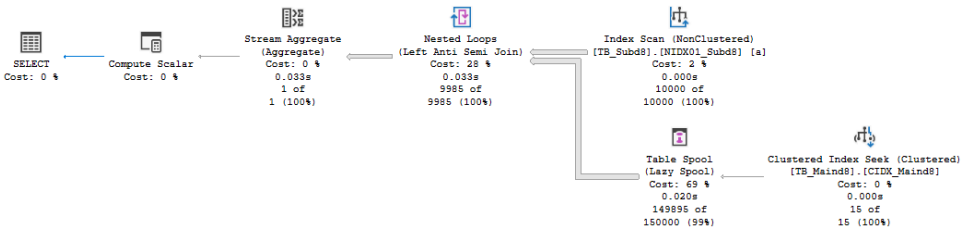
■ SQL 구문

```
SELECT COUNT(id)
FROM TB_Subd8 a
```

```
WHERE id NOT IN (SELECT DISTINCT id
                 FROM TB_Maind8
                 WHERE gid BETWEEN 2 AND 4)
GO
```

· 실행 계획

Rows	Executes	StmtText
1	1	SELECT COUNT(id) FROM TB_Subd8 a WHERE id NOT IN (SELECT DISTINCT id FROM TB_Maind8 WHERE gid BETWEEN 2 AND 4)
0	0	¦--Compute Scalar(DEFINE:([Expr1005]=CONVERT_IMPLICIT(int,[Expr1008],0)))
1	1	¦--Stream Aggregate(DEFINE:([Expr1008]=COUNT([Tuning].[dbo].[TB_Subd8].[id] as [a].[id])))
9985	1	¦--Nested Loops(Left Anti Semi Join, WHERE:([Tuning].[dbo].[TB_Subd8].[id] as [a].[id] IS NULL OR [Tuning].[dbo].[TB_Maind8].[id] IS NULL OR [Tuning].[dbo].[TB_Subd8].[id] as [a].[id]=[Tuning].[dbo].[TB_Maind8].[id]))
10000	1	¦--Index Scan(OBJECT:([Tuning].[dbo].[TB_Subd8].[NIDX01_Subd8] AS [a]))
149895	10000	¦--Table Spool
15	1	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Maind8].[CIDX_Maind8]), SEEK :([Tuning].[dbo].[TB_Maind8].[gid] >= (2) AND [Tuning].[dbo].[TB_Maind8].[gid] <= (4)) Ordered Forward)




```
테이블 'TB_Maind8'. 스캔 수 1, 논리적 읽기 2
테이블 'Worktable'. 스캔 수 1, 논리적 읽기 20029
테이블 'TB_Subd8'. 스캔 수 1, 논리적 읽기 30
SQL Server 실행 시간:
CPU 시간 = 31ms, 경과 시간 = 33ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Maind8	CIDX_Maind8	CLUSTERED	0	gid	-
TB_Subd8	NIDX01_Subd8	NONCLUSTERED	0	id	-

4. [AS-IS] 구문 설명

[TB_Subd8] 테이블의 [id] 컬럼을 기준으로 [TB_Maind8] 테이블 [gid] 컬럼 값이 2부터 4사이인 데이터의 [id] 값을 제외한 결과 행 수를 출력하는 구문이다. 데이터를 제외하기 위해 NOT IN 절이 사용되는데 두 테이블의 데이터를 비교하는 과정에서 NL Join으로 수행되어 선행 테이블 [TB_Subd8]의 결과 집합 10,000건에 대해 후행 테이블인 [TB_Maind8]에 반복 조인되면서 많은 I/O 비용이 발생된다.

5. [TO-BE] 튜닝 포인트

두 테이블을 LEFT OUTER JOIN으로 구문을 변경하여 다른 조인 방식으로 수행되도록 유도한다. [TB_Subd8] 테이블은 조인 키에 해당되는 [id] 컬럼이 인덱스로 인해 이미 정렬되어 있는 상태이며 [TB_Maind8] 테이블은 DISTINCT로 정렬을 선행하기 때문에 테이블을 한 번씩 읽어내면서 정렬된 데이터로 조인하는 Merge Join으로 성능을 개선할 수 있다.

추가적으로, 개선 전 구문에서는 [TB_Maind8] 테이블에 NULL 값이 포함되어 있을 경우 두 데이터 간의 비교가 불가능하여 데이터가 출력되지 않는데, 이와 같은 상황을 방지하기 위해 [id] 컬럼에 IS NOT NULL 조건을 명시하거나 NOT NULL 제약조건 적용 혹은 구문에 조인 조건을 명시하여 요구사항에 명확한 구문으로 작성해야 하며 이를 통해서도 좀 더 나은 실행 계획이 수립될 수 있다.

6. [TO-BE] 개선 방안

1) LEFT OUTER JOIN으로 구문 변경

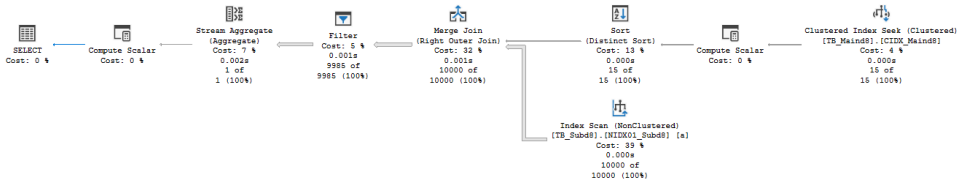
NOT IN 절을 LEFT OUTER JOIN 구문으로 변경하여 Merge Join을 통해 [TB_Maind8] 테이블을 반복적으로 읽으면서 발생된 I/O 비용을 개선한다.

■ 변경된 SQL 구문

```
SELECT COUNT(a.id)
FROM TB_Subd8 a
LEFT OUTER JOIN (SELECT DISTINCT id
                 FROM TB_Maind8
                 WHERE gid BETWEEN 2 AND 4) b
ON a.id = b.id
WHERE b.id IS NULL
GO
```

· 실행 계획

Rows	Executes	StmtText
1	1	SELECT COUNT(a.id) FROM TB_Subd8 a LEFT OUTER JOIN (SELECT DISTINCT id FROM TB_Maind8 WHERE gid BETWEEN 2 AND 4) b ON a.id = b.id WHERE b.id IS NULL
0	0	¦--Compute Scalar(DEFINE:([Expr1006]=CONVERT_IMPLICIT(int,[Expr1009],0)))
1	1	¦--Stream Aggregate(DEFINE:([Expr1009]=COUNT([Tuning].[dbo].[TB_Subd8].[id] as [a].[id])))
9985	1	¦--Filter(WHERE:([Expr1005] IS NULL))
10000	1	¦--Merge Join(Right Outer Join, MERGE:([Expr1005])=([a].[id]), RESIDUAL:([Tuning].[dbo].[TB_Subd8].[id] as [a].[id]=[Expr1005]))
15	1	¦--Sort(DISTINCT ORDER BY:([Expr1005] ASC))
0	0	¦ ¦--Compute Scalar(DEFINE:([Expr1005]=[Tuning].[dbo].[TB_Maind8].[id]))
15	1	¦ ¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Maind8].[CIDX_Maind8]), SEEK:([Tuning].[dbo].[TB_Maind8].[gid] >= (2) AND [Tuning].[dbo].[TB_Maind8].[gid] <= (4)) Ordered Forward)
10000	1	¦--Index Scan(OBJECT:([Tuning].[dbo].[TB_Subd8].[NIDX01_Subd8] AS [a]), Ordered Forward)




```
테이블 'TB_Subd8'. 스캔 수 1, 논리적 읽기 30
테이블 'Worktable'. 스캔 수 0, 논리적 읽기 0
테이블 'TB_Maind8'. 스캔 수 1, 논리적 읽기 2
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 1ms
```

7. 개선 효과

· **리소스 사용률 비교**

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	31	33	20,061
TO-BE	0	1	32

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

23. NOT IN 조건으로 사용된 서브쿼리 성능 이슈

1. 프롬프트

> [AS-IS]
> 구문의 NOT IN 처리를 하는 과정에서 비효율적인 I/O 비용이 발생해.
> 이 부분을 분석해서 대안을 알려줘.

```
[AS-IS]
SELECT COUNT(id)
FROM TB_Subd8 a
WHERE id NOT IN (SELECT DISTINCT id
                 FROM TB_Maind8
                 WHERE gid BETWEEN 2 AND 4);
```

2. 분석 및 주요 최적화

1) 현재 환경 요약

항목	값
TB_Subd8 행 수	10,000 (Heap, id에 비클러스터형 인덱스)
TB_Maind8 행 수	10,000 (클러스터형 인덱스: gid, id에 인덱스 없음)
서브쿼리 결과	15건 (DISTINCT id)
id 컬럼 Nullable	양쪽 모두 YES ⚠

2) 현재 쿼리의 문제점

#	문제	영향
1	NOT IN + Nullable 컬럼	서브쿼리에 NULL이 하나라도 포함되면 전체 결과가 0건이 됩니다. 옵티마이저는 이를 보장하기 위해 추가 NULL 검사 로직을 삽입하여 I/O가 증가합니다.
2	불필요한 DISTINCT	NOT IN은 내부적으로 중복을 무시하므로 DISTINCT는 정렬/해싱 비용만 추가합니다.
3	서브쿼리 전체 Materialization	NOT IN은 서브쿼리 결과를 메모리에 전부 적재한 후 비교합니다. TB_Maind8.id에 인덱스가 없어 비용이 더 큽니다.

3. 개선안

```sql
-- 방안 1. NOT EXISTS

SELECT COUNT(a.id) FROM dbo.TB_Subd8 a
WHERE NOT EXISTS (SELECT 1
                  FROM dbo.TB_Maind8 b
                  WHERE b.id = a.id AND b.gid BETWEEN 2 AND 4);
```

■ 장점

· NULL 안전 – 서브쿼리에 NULL이 있어도 정확한 결과 보장

· Semi-Join으로 변환되어 매칭되는 첫 행을 찾으면 즉시 중단 (short-circuit)

· DISTINCT 불필요

```sql
-- 방안 2. LEFT JOIN ... IS NULL

SELECT COUNT(a.id) FROM dbo.TB_Subd8 a
LEFT JOIN (SELECT DISTINCT id FROM dbo.TB_Maind8
          WHERE gid BETWEEN 2 AND 4) b ON a.id = b.id
WHERE b.id IS NULL;
```

■ 장점

· 실행 계획이 Hash/Merge Anti-Join으로 최적화될 수 있음

· 대량 데이터에서 안정적인 성능

```sql
-- 방안 3. EXCEPT

SELECT COUNT(*)
FROM (SELECT id FROM dbo.TB_Subd8
      EXCEPT
      SELECT id FROM dbo.TB_Maind8 WHERE gid BETWEEN 2 AND 4) x;
```

■ 장점

· NULL을 값으로 취급하여 비교 (NULL-safe)

· 자동 중복 제거

*EXCEPT는 중복을 제거하므로, 원본 쿼리와 의미가 약간 다를 수 있습니다 (동일 id가 여러 건이면 1건으로 카운트).

· 대안 비교 요약

방식	NULL 안전	Short-Circuit	DISTINCT 필요	권장도
NOT IN (현재)	✗	✗	불필요하지만 사용 중	–
NOT EXISTS	✓	✓	불필요	***
LEFT JOIN IS NULL	✓	✗	서브쿼리 내 필요	**
EXCEPT	✓	✗	자동 처리	* (의미 차이 주의)

24 WITH 절의 성능 이슈

WITH 절로 선언된 내부 구문이 반복적으로 수행되면서 I/O가 증가된 사례이다.

1. 사전 구성 스크립트

```sql
SELECT std_no
      , CASE WHEN std_no <= 1000 THEN '1'
        WHEN std_no <= 2000 THEN '2'
        WHEN std_no <= 3000 THEN '3'
        WHEN std_no <= 4000 THEN '4'
        ELSE '5' END class_cd
     , CONVERT(NVARCHAR(36),NEWID()) code INTO TB_Std
FROM (
      SELECT ROW_NUMBER() OVER ( ORDER BY a.number) std_no
      FROM master..spt_values a , master..spt_values b
      WHERE a.type = 'P'
      AND a.number BETWEEN 1 AND 1000
      AND b.type = 'P'
      AND b.number BETWEEN 1 AND 5
) c
```

```
SELECT a.std_no
      , CHAR(b.number+64) sbjt_cd
      , CASE WHEN a.std_no = 1 THEN 100
        ELSE CONVERT(INT,RIGHT(std_no * ASCII(CONVERT(NVARCHAR(36), NEWID())), 2))
        END point
INTO TB_Point
FROM TB_Std a , master..spt_values b
WHERE b.type = 'P' AND b.number BETWEEN 1 AND 26

CREATE INDEX NIDX01_Std ON TB_Std ( class_cd , std_no )
GO
```

2. [AS-IS] SQL 구문 및 실행 계획

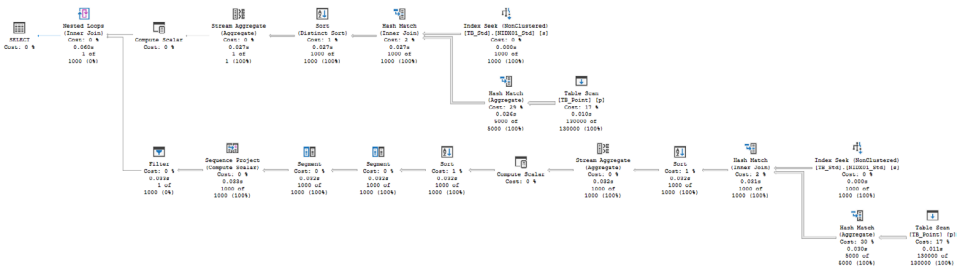
■ SQL 구문

```
WITH WT_Rank
AS (SELECT s.class_cd, s.std_no, SUM(p.point) total_point
    FROM TB_Std s INNER JOIN TB_Point p
    ON s.std_no = p.std_no
    WHERE s.class_cd = '1'
    GROUP BY s.class_cd, s.std_no)

SELECT class_cd, std_no, std_rank, total_point, total_std_cnt
FROM (SELECT class_cd, std_no, total_point
           , RANK() OVER (ORDER BY total_point DESC) std_rank
      FROM WT_Rank
) a, (SELECT COUNT(*) total_std_cnt FROM WT_Rank) b
WHERE a.std_rank = 1
GO
```

· 실행 계획

Rows	Executes	StmtText
1	1	WITH WT_Rank AS (SELECT s.class_cd, s.std_no, SUM(p.point) total_point FROM TB_Std s INNER JOIN TB_Point p ON s.std_no = p.std_no WHERE s.class_cd = '1' GROUP BY s.class_cd, s.std_no) SELECT class_cd, std_no, std_rank, total_point, total_std_cnt FROM (SELECT class_cd, std_no, total_point, RANK() OVER (ORDER BY total_point DESC) std_rank FROM WT_Rank) a, (SELECT COUNT(*) total_std_cnt FROM WT_Rank) b WHERE a.std_rank = 1
1	1	¦--Nested Loops(INNER JOIN)
0	0	¦--Compute Scalar(DEFINE:([Expr1011]=CONVERT_IMPLICIT(int,[Expr1018],0)))
1	1	¦ ¦--Stream Aggregate(DEFINE:([Expr1018]=Count(*)))
1000	1	¦ ¦--Sort(DISTINCT ORDER BY:([s].[std_no] ASC))
1000	1	¦ ¦--Hash Match(INNER JOIN, HASH:([s].[std_no])=([p].[std_no]), RESIDUAL:([Tuning].[dbo].[TB_Point].[std_no] as [p].[std_no]=[Tuning].[dbo].[TB_Std].[std_no] as [s].[std_no]))
1000	1	¦ ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_Std].[NIDX01_Std] AS [s]), SEEK:([s].[class_cd]='1') Ordered Forward)
5000	1	¦ ¦--Hash Match(Aggregate, HASH:([p].[std_no]), RESIDUAL:([Tuning].[dbo].[TB_Point].[std_no] as [p].[std_no] = [Tuning].[dbo].[TB_Point].[std_no] as [p].[std_no]))
130000	1	¦ ¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Point] AS [p]))
1	1	¦--Filter(WHERE:([Expr1005]=(1)))
1000	1	¦--Sequence Project(DEFINE:([Expr1005]=rank))
1000	1	¦--Segment
1000	1	¦--Segment
1000	1	¦--Sort(ORDER BY:([Expr1004] DESC))

0	0	¦--Compute Scalar(DEFINE:([Expr1004]=CASE WHEN [globalagg1013]=(0) THEN NULL ELSE [globalagg1015] END))
1000	1	¦--Stream Aggregate(GROUP BY:([s].[std_no]) DEFINE:([globalagg1013]=SUM([partialagg1012]), [globalagg1015]=SUM([partialagg1014]), [s].[class_cd]=ANY([Tuning].[dbo].[TB_Std].[class_cd] as [s].[class_cd])))
1000	1	¦--Sort(ORDER BY:([s].[std_no] ASC))
1000	1	¦--Hash Match(INNER JOIN, HASH:([s].[std_no])=([p].[std_no]), RESIDUAL:([Tuning].[dbo].[TB_Point].[std_no] as [p].[std_no]=[Tuning].[dbo].[TB_Std].[std_no] as [s].[std_no]))
1000	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_Std].[NIDX01_Std] AS [s]), SEEK:([s].[class_cd]='1') Ordered Forward)
5000	1	¦--Hash Match(Aggregate, HASH:([p].[std_no]), RESIDUAL:([Tuning].[dbo].[TB_Point].[std_no] as [p].[std_no] = [Tuning].[dbo].[TB_Point].[std_no] as [p].[std_no]) DEFINE:([partialagg1012]=COUNT_BIG([Tuning].[dbo].[TB_Point].[point] as [p].[point]), [partialagg1014]=SUM([Tuning].[dbo].[TB_Point].[point] as [p].[point])))
130000	1	¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Point] AS [p]))



```
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'Workfile'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_Point'. 검색 수 2, 논리적 읽기 수 726
테이블 'TB_Std'. 검색 수 2, 논리적 읽기 수 12
SQL Server 실행 시간:
CPU 시간 = 47ms, 경과 시간 = 54ms
```

3. [AS-IS] 통계 정보

- **인덱스 정보**

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Std	NIDX01_Std	NONCLUSTERED	0	class_cd, std_no	-

4. [AS-IS] 구문 설명

WITH 절에서 [TB_Std] 테이블과 [TB_Point] 테이블을 조인하여 각 [std_no] 컬럼 값마다 [point]의 합계를 구하고 하위 구문에서 합계가 1순위인 데이터와 전체 행 수를 구해 조인하는 구문이다. 순위 데이터와 행 수 데이터를 출력하기 위해 각각 WITH 절을 호출하여 사용하는데 이로 인해 WITH 절 내부 구문이 반복 수행되면서 I/O 비용이 2배로 발생한다.

5. [TO-BE] 튜닝 포인트

WITH 절은 하위 쿼리에서 호출된 수만큼 반복 수행되기 때문에 다수의 호출이 필요한 경우 내부 구문에서 발생하는 비용에 따라 더 많은 비효율이 포함될 수 있다. WITH 절 구문이 수행된 결과 데이터가 크지 않다면 해당 데이터를 임시 테이블에 저장하여 재사용이 필요할 때 임시로 저장된 데이터만 읽어내면서 I/O 비용을 감소시킬 수 있다.

6. [TO-BE] 개선 방안

1) 임시 테이블을 사용하여 WITH 절의 중간 결과 집합을 저장

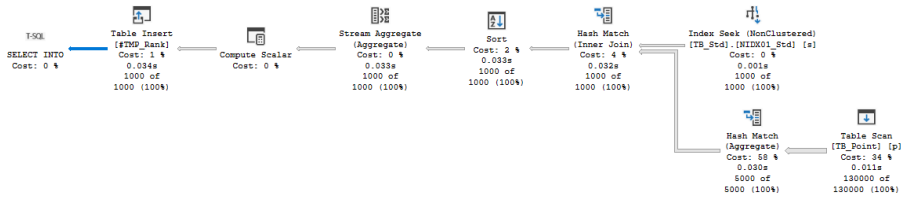
WITH 절 결과 데이터를 임시 테이블에 저장하고 해당 데이터를 재사용하여 전체적인 I/O 비용을 감소시킨다.

■ 변경된 SQL 구문

```
SELECT s.class_cd, s.std_no, SUM(p.point) total_point
INTO #TMP_Rank
FROM TB_Std s INNER JOIN TB_Point p
ON s.std_no = p.std_no
WHERE s.class_cd = '1'
GROUP BY s.class_cd, s.std_no

SELECT class_cd, std_no, std_rank, total_point, total_std_cnt
FROM (SELECT class_cd, std_no, total_point
           , RANK() OVER ( ORDER BY total_point DESC) std_rank
      FROM #TMP_Rank
) a, (SELECT COUNT(*) total_std_cnt FROM #TMP_Rank) b
WHERE a.std_rank = 1
GO
```

· 실행 계획 (Temp Table 생성)

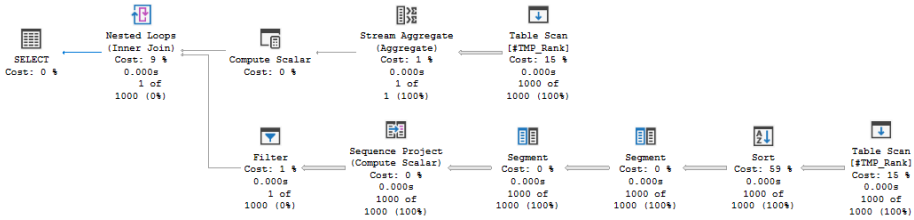
Rows	Executes	StmtText
1000	1	SELECT s.class_cd, s.std_no, SUM(p.point) total_point INTO #TMP_Rank FROM TB_Std s INNER JOIN TB_Point p ON s.std_no = p.std_no WHERE s.class_cd = '1' GROUP BY s.class_cd, s.std_no
1000	1	¦--Table INSERT(OBJECT:([#TMP_Rank]), SET:([#TMP_Rank].[class_cd] = [Tuning].[dbo].[TB_Std].[class_cd] as [s].[class_cd], [#TMP_Rank].[std_no] = [Tuning].[dbo].[TB_Std].[std_no] as [s].[std_no],[#TMP_Rank].[total_point] = [Expr1007]))
0	0	¦--Compute Scalar(DEFINE:([Expr1007]=CASE WHEN [globalagg1009]=(0) THEN NULL ELSE [globalagg1011] END))
1000	1	¦--Stream Aggregate(GROUP BY:([s].[std_no]) DEFINE:([globalagg1009]=SUM([partialagg1008]), [globalagg1011]=SUM([partialagg1010]), [s].[class_cd]=ANY([Tuning].[dbo].[TB_Std].[class_cd] as [s].[class_cd])))
1000	1	¦--Sort(ORDER BY:([s].[std_no] ASC))
1000	1	¦--Hash Match(INNER JOIN, HASH:([s].[std_no])=([p].[std_no]), RESIDUAL:([Tuning].[dbo].[TB_Point].[std_no] as [p].[std_no]=[Tuning].[dbo].[TB_Std].[std_no] as [s].[std_no]))
1000	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_Std].[NIDX01_Std] AS [s]), SEEK:([s].[class_cd]='1') Ordered Forward)
5000	1	¦--Hash Match(Aggregate, HASH:([p].[std_no]), RESIDUAL:([Tuning].[dbo].[TB_Point].[std_no] as [p].[std_no] = [Tuning].[dbo].[TB_Point].[std_no] as [p].[std_no]) DEFINE:([partialagg1008]=COUNT_BIG([Tuning].[dbo].[TB_Point].[point] as [p].[point]), [partialagg1010]=SUM([Tuning].[dbo].[TB_Point].[point] as [p].[point])))
130000	1	¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Point] AS [p]))



테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'Workfile'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_Point'. 검색 수 1, 논리적 읽기 수 363
테이블 'TB_Std'. 검색 수 1, 논리적 읽기 수 6
SQL Server 실행 시간:
CPU 시간 = 16ms, 경과 시간 = 25ms

· 실행 계획 (Temp Table 사용)

Rows	Executes	StmtText
1	1	SELECT class_cd, std_no, std_rank, total_point, total_std_cnt FROM (SELECT class_cd, std_no, total_point, RANK() OVER (ORDER BY total_point DESC) std_rank FROM #TMP_Rank) a, (SELECT COUNT(*) total_std_cnt FROM #TMP_Rank) b WHERE a.std_rank = 1
1	1	┦--Nested Loops(INNER JOIN)
0	0	┦--Compute Scalar(DEFINE:([Expr1007]=CONVERT_IMPLICIT(int,[Expr1010],0)))
1	1	┦--Stream Aggregate(DEFINE:([Expr1010]=Count(*)))
1000	1	┦--Table Scan(OBJECT:([tempdb].[dbo].[#TMP_Rank]))
1	1	┦--Filter(WHERE:([Expr1003]=(1)))
1000	1	┦--Sequence Project(DEFINE:([Expr1003]=rank))
1000	1	┦--Segment
1000	1	┦--Segment
1000	1	┦--Sort(ORDER BY:([tempdb].[dbo].[#TMP_Rank].[total_point] DESC))
1000	1	┦--Table Scan(OBJECT:([tempdb].[dbo].[#TMP_Rank]))



테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 '#TMP_Rank___000000000022'. 검색 수 2, 논리적 읽기 수 8
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	47	54	738
TO-BE	16	25	377

GitHub Copilot 채팅 결과

Claude Opus 4.6

24. WITH 절의 성능 이슈

1. 프롬프트

```
[AS-IS]
구문을 처리하는 과정에서 WITH 문이 여러 번 호출되어 불필요한 I/O 비용이 발생한다.
이 부분을 기존보다 낮은 I/O 비용으로 성능 개선해줘.
단, WITH문 및 Spool연산자를 사용하지 않고 I/O 비용을 줄이는 방식이어야해.
```

```
[AS-IS]
WITH WT_Rank
AS (SELECT s.class_cd, s.std_no, SUM(p.point) total_point
      FROM TB_Std s INNER JOIN TB_Point p
      ON s.std_no = p.std_no
      WHERE s.class_cd = '1'
      GROUP BY s.class_cd , s.std_no)

SELECT class_cd, std_no, std_rank, total_point, total_std_cnt
FROM (SELECT class_cd, std_no, total_point,
             RANK() OVER (ORDER BY total_point DESC) std_rank
       FROM WT_Rank) a, (SELECT COUNT(*) total_std_cnt FROM WT_Rank) b
WHERE a.std_rank = 1;
```

2. 분석 및 주요 최적화

현재 쿼리에서 CTE WT_Rank가 2번 참조(메인 쿼리 + COUNT 서브쿼리)되어 동일한 JOIN·GROUP BY가 2회 실행됩니다. SQL Server는 CTE를 구체화(materialize)하지 않고 인라인 확장하므로, 참조 횟수만큼 I/O가 반복됩니다. 아래 방안은 데이터를 1회만 스캔하여 I/O를 절감합니다.

3. 개선안

```
-- 방안 1. 임시 테이블에 중간 결과 저장 집계 결과를 임시 테이블에 1회만
-- 저장하고 재사용합니다.

SELECT s.class_cd, s.std_no, SUM(p.point) AS total_point
INTO #WT_Rank
FROM dbo.TB_Std s
INNER JOIN dbo.TB_Point p ON s.std_no = p.std_no
WHERE s.class_cd = '1'
GROUP BY s.class_cd, s.std_no;
```

```
SELECT a.class_cd, a.std_no, a.std_rank, a.total_point, b.total_std_cnt
FROM (SELECT class_cd, std_no, total_point,
             RANK() OVER (ORDER BY total_point DESC) AS std_rank
      FROM #WT_Rank) a
CROSS JOIN (SELECT COUNT(*) AS total_std_cnt FROM #WT_Rank) b
WHERE a.std_rank = 1;

DROP TABLE #WT_Rank;
```

25 EXCEPT 구문 성능 이슈

EXCEPT를 사용한 구문에서 인덱스에 키 컬럼이 누락되어 과도한 스캔 범위로 I/O가 높아진 사례이다.

1. 사전 구성 스크립트

```
SELECT number custid
      , NEWID() code
      , CASE WHEN number <= 10  THEN 5
        WHEN number <= 110 THEN 4
        WHEN number <= 410 THEN 3
        WHEN number <= 710 THEN 2
        ELSE 1 END level
INTO TB_Cust01
FROM master..spt_values
WHERE type = 'P'
AND number BETWEEN 1 AND 1000

SELECT a.custid, DATEADD(dd,b.number,'2000-01-01') date
INTO TB_Pay01
FROM TB_Cust01 a, master..spt_values b
WHERE b.type = 'P'
AND b.number BETWEEN 1 AND 1000
AND ((level IN (5,4) AND custid%2=0) OR (level IN (3,2,1) AND custid%100<>0 ))

CREATE INDEX NIDX01_Cust01 ON TB_Cust01 (level) INCLUDE (custid)
CREATE INDEX NIDX01_Pay01 ON TB_Pay01 (date) INCLUDE (custid)
GO
```

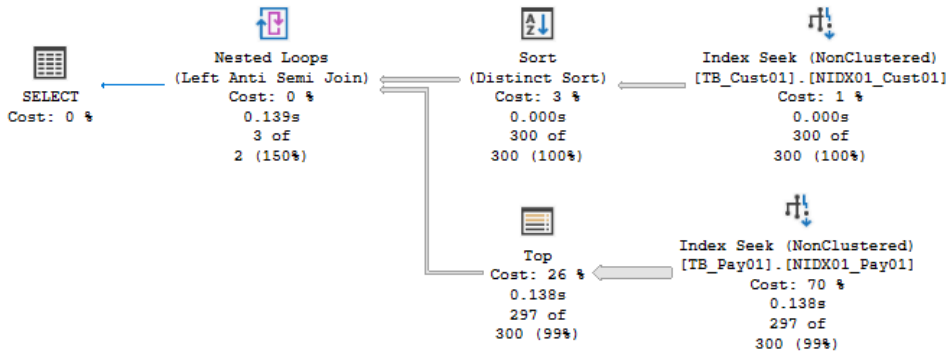
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT custid
FROM TB_Cust01
WHERE level = 2
EXCEPT
SELECT custid
FROM TB_Pay01
WHERE date >= '2000-01-01'
GO
```

· 실행 계획

Rows	Executes	StmtText
3	1	SELECT custid FROM TB_Cust01 WHERE level = 2 EXCEPT SELECT custid FROM TB_Pay01 WHERE date >= '2000-01-01'
3	1	¦--Nested Loops(Left Anti Semi Join, OUTER REFERENCES:([tuning].[dbo].[TB_Cust01].[custid]))
300	1	¦--Sort(DISTINCT ORDER BY:([tuning].[dbo].[TB_Cust01].[custid] ASC))
300	1	¦ ¦--Index Seek(OBJECT:([tuning].[dbo].[TB_Cust01].[NIDX01_Cust01]), SEEK:([tuning].[dbo].[TB_Cust01].[level]=(2)) Ordered Forward)
297	300	¦--Top(TOP EXPRESSION:((1)))
297	300	¦--Index Seek(OBJECT:([tuning].[dbo].[TB_Pay01].[NIDX01_Pay01]), SEEK:([tuning].[dbo].[TB_Pay01].[date] >= '2000-01-01 00:00:00.000'), WHERE:([tuning].[dbo].[TB_Cust01].[custid]=[tuning].[dbo].[TB_Pay01].[custid]) Ordered Forward)




```
테이블 'TB_Pay01'. 검색 수 300, 논리적 읽기 수 12882
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_Cust01'. 검색 수 1, 논리적 읽기 수 3
SQL Server 실행 시간:
CPU 시간 = 109ms, 경과 시간 = 110ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Cust01	NIDX01_Cust01	NONCLUSTERED	0	level	custid
TB_Pay01	NIDX01_Pay01	NONCLUSTERED	0	date	custid

4. [AS-IS] 구문 설명

[TB_Cust01] 테이블에서 [level] 컬럼이 2인 데이터 중 [TB_Pay01] 테이블에 '2000-01-01' 이후 이력이 없는 [custid]를 출력하는 구문이다. [TB_Cust01] 테이블에서 [TB_Pay01] 테이블의 데이터를 제외하기 위해 조건절을 필터하고 정렬을 수행한 뒤 Anti Semi Join으로 조인에 성공하지 못하는 데이터만을 최종 출력한다. [TB_Pay01] 테이블에서 사용된 인덱스에는 조인 조건인 [custid] 컬럼이 키로 구성되어 있지 않기 때문에 [date] >= '2000-01-01' 조건만 인덱스로 탐색되며 선행 테이블의 결과 집합 각 행마다 조인에 성공하는 데이터를 찾을 때까지 읽게 되면서 I/O가 높게 사용된다.

5. [TO-BE] 튜닝 포인트

NOT EXISTS 구문으로 변경하면 동일한 결과를 도출하면서 불필요한 정렬 과정을 제거할 수 있다. NOT EXISTS도 Anti Semi Join으로 수행되며 조인에 성공하는 데이터를 찾으면 해당 지점에서 중단하지만 조인에 성공하는 데이터를 찾지 못한다면 모든 데이터를 읽어내야 하기 때문에 많은 I/O가 사용될 수 있다. [TB_Pay01] 테이블의 조인 조건인 [custid] 컬럼이 인덱스에 선행 키로 포함되어 있다면 데이터가 정렬된 상태로 유지되기 때문에 첫 행만 읽어내도 조인되는 데이터가 있는지에 대한 판별이 가능하여 I/O를 개선할 수 있다

6. [TO-BE] 개선 방안

1) NOT EXISTS 구문 변경과 [custid] 컬럼이 포함된 신규 인덱스 생성

NOT EXISTS로 변경된 구문에 [custid], [date] 컬럼으로 구성된 인덱스를 생성하면 [TB_Cust01] 테이블의 결과 집합 각 행에 대해 [TB_Pay01] 테이블의 데이터 1건씩만을 탐색하여 I/O가 개선된다.

■ 인덱스 생성 구문

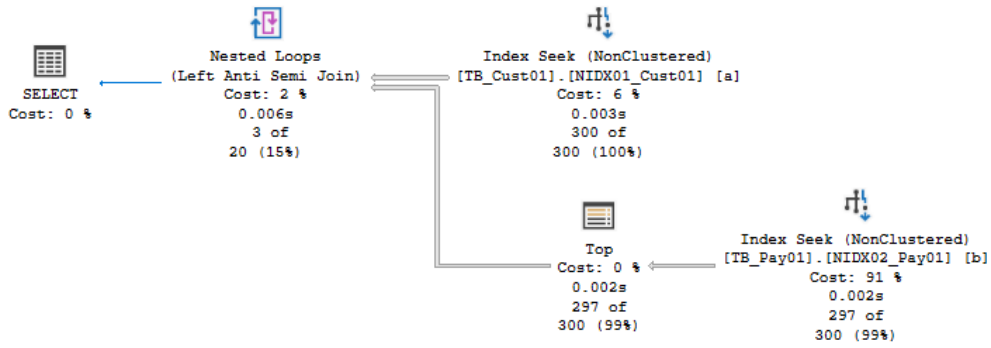
```
CREATE INDEX NIDX02_Pay01 ON TB_Pay01 (custid,date)
GO
```

■ 변경된 SQL 구문

```
SELECT custid
FROM TB_Cust01 a
WHERE level = 2
AND NOT EXISTS (SELECT 'x'
                FROM TB_Pay01 b
                WHERE a.custid = b.custid
                AND date >= '2000-01-01')
GO
```

· 실행 계획

Rows	Executes	StmtText
3	1	SELECT custid FROM TB_Cust01 a WHERE level = 2 AND NOT EXISTS (SELECT 'x' FROM TB_Pay01 b WHERE a.custid = b.custid AND date >= '2000-01-01')
3	1	¦--Nested Loops(Left Anti Semi Join, OUTER REFERENCES:([a].[custid], [Expr1005]) WITH UNORDERED PREFETCH)
300	1	¦--Index Seek(OBJECT:([tuning].[dbo].[TB_Cust01].[NIDX01_Cust01] AS [a]), SEEK:([a].[level]=(2)) Ordered Forward)
297	300	¦--Top(TOP EXPRESSION:((1)))
297	300	¦--Index Seek(OBJECT:([tuning].[dbo].[TB_Pay01].[NIDX02_Pay01] AS [b]), SEEK:([b].[custid]=[tuning].[dbo].[TB_Cust01].[custid] as [a].[custid] AND [b].[date] >= '2000-01-01 00:00:00.000') Ordered Forward)




```
테이블 'TB_Pay01'. 검색 수 300, 논리적 읽기 수 983
테이블 'TB_Cust01'. 검색 수 1, 논리적 읽기 수 3
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	109	110	12,885
TO-BE	0	0	986

GitHub Copilot 채팅 결과

Claude Opus 4.6

25. EXCEPT 구문 성능 이슈

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인 분석 및 개선해줘.

```
[AS-IS]
SELECT custid FROM TB_Cust01 WHERE level = 2
EXCEPT
SELECT custid FROM TB_Pay01 WHERE date >= '2000-01-01';
```

2. 분석 및 주요 최적화

1) 문제점 분석

항목	원본 쿼리	비고
EXCEPT	양쪽 결과를 정렬 + 중복 제거 후 비교	내부적으로 Hash/Sort 연산 발생
TB_Cust01	Heap (클러스터드 인덱스 없음)	custid 접근 시 RID Lookup 또는 Table Scan
TB_Pay01	Heap (클러스터드 인덱스 없음)	custid 인덱스 없음 → Table Scan
TB_Pay01 인덱스	date only	custid를 포함하지 않아 커버링 불가
EXCEPT 특성	양쪽 모두 DISTINCT 처리	불필요한 중복 제거 비용 발생

2) 핵심 병목

EXCEPT는 양쪽 집합 모두 정렬/해싱 후 비교하며, TB_Pay01에 custid 인덱스가 없어 Full Table Scan이 발생합니다.

3. 개선안

```
-- EXCEPT를 NOT EXISTS로 전환하여 불필요한 정렬/중복 제거를 제거합니다.

SELECT DISTINCT a.custid FROM dbo.TB_Cust01 a
WHERE a.level = 2 AND NOT EXISTS (SELECT 1 FROM dbo.TB_Pay01 b
WHERE b.custid = a.custid AND b.date >= '2000-01-01');
```

■ 변경 요약

변경	이전	이후	효과
집합 연산	EXCEPT (양쪽 정렬 + DISTINCT)	NOT EXISTS	정렬/중복 제거 비용 제거
TB_Pay01 탐색	전체 Scan 후 정렬	매칭 즉시 중단 (short-circuit)	I/O 감소
의미적 차이	EXCEPT는 양쪽 모두 DISTINCT	NOT EXISTS는 좌측 원본 유지	원본에 중복 없으면 결과 동일

· EXCEPT는 내부적으로 양쪽 집합 모두 DISTINCT 처리 후 차집합을 구하므로 Sort/Hash 비용이 큽니다.

· NOT EXISTS는 매칭 행을 찾는 즉시 탐색을 중단하여 I/O가 절감됩니다.

· 추가 인덱스 권장, dbo.TB_Pay01에 커버링 인덱스를 생성하면 더 큰 개선이 가능합니다.

```
CREATE NONCLUSTERED INDEX NIDX02_Pay01 ON dbo.TB_Pay01 (custid, date);
```

26 INTERSECT 구문 성능 이슈

INTERSECT를 사용한 구문에서 인덱스에 키 컬럼이 누락되어 과도한 스캔 범위로 I/O가 많아진 사례이다.

1. 사전 구성 스크립트

```
SELECT number custid
      , NEWID() code
      , CASE WHEN number <= 10  THEN 5
        WHEN number <= 110 THEN 4
        WHEN number <= 410 THEN 3
        WHEN number <= 710 THEN 2
        ELSE 1 END level
INTO TB_Cust02
FROM master..spt_values
WHERE type = 'P'
AND number BETWEEN 1 AND 1000

SELECT a.custid, DATEADD(dd,b.number,'2000-01-01') date
INTO TB_Pay02
FROM TB_Cust02 a, master..spt_values b
WHERE b.type = 'P'
AND b.number BETWEEN 1 AND 1000
AND ((level IN (5,4) AND custid%2=0) OR (level IN (3,2,1) AND custid%100<>0 ))

CREATE INDEX NIDX01_Cust02 ON TB_Cust02 (level) INCLUDE (custid)
CREATE INDEX NIDX01_Pay02 ON TB_Pay02 (date) INCLUDE (custid)
GO
```

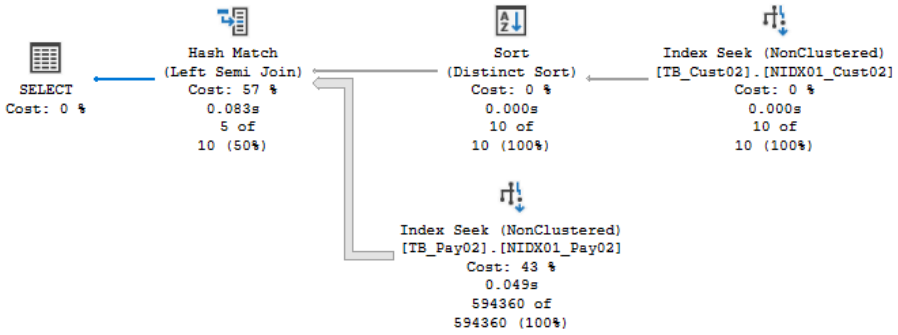
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT custid
FROM TB_Cust02
WHERE level = 5
INTERSECT
SELECT custid
FROM TB_Pay02
WHERE date >= '2001-01-01'
GO
```

· 실행 계획

Rows	Executes	StmtText
5	1	SELECT custid FROM TB_Cust02 WHERE level = 5 INTERSECT SELECT custid FROM TB_Pay02 WHERE date >= '2001-01-01'
5	1	┦--Hash Match(Left Semi Join, HASH:([tuning].[dbo].[TB_Cust02].[custid])=([tuning].[dbo].[TB_Pay02].[custid]))
10	1	┦--Sort(DISTINCT ORDER BY:([tuning].[dbo].[TB_Cust02].[custid] ASC))
10	1	┦ ┦--Index Seek(OBJECT:([tuning].[dbo].[TB_Cust02].[NIDX01_Cust02]), SEEK:([tuning].[dbo].[TB_Cust02].[level]=(5)) Ordered Forward)
594360	1	┦--Index Seek(OBJECT:([tuning].[dbo].[TB_Pay02].[NIDX01_Pay02]), SEEK:([tuning].[dbo].[TB_Pay02].[date] >= '2001-01-01 00:00:00.000') Ordered Forward)




```
테이블 'Workfile'. 검색 수 0, 논리적 읽기 수 0
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_Pay02'. 검색 수 1, 논리적 읽기 수 1923
테이블 'TB_Cust02'. 검색 수 1, 논리적 읽기 수 2
SQL Server 실행 시간:
CPU 시간 = 31ms, 경과 시간 = 44ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Cust02	NIDX01_Cust02	NONCLUSTERED	0	level	custid
TB_Pay02	NIDX01_Pay02	NONCLUSTERED	0	date	custid

4. [AS-IS] 구문 설명

[TB_Cust02] 테이블의 [level] 컬럼이 5인 데이터 기준으로 [TB_Pay02] 테이블에 '2001-01-01' 이후 이력이 존재하는 [custid]를 출력하는 구문이다. [TB_Pay02] 테이블의 데이터 존재 유무를 확인하기 위해 조건절을 필터하고 정렬을 수행한 뒤 Semi Join으로 조인에 성공한 데이터만을 최종 출력한다. [TB_Pay02] 테이블에서 사용된 인덱스에는 조인 조건인 [custid] 컬럼이 키로 구성되어 있지 않기 때문에 [date] 컬럼만 인덱스로 탐색하여 Hash Join으로 수행되었다. [date] 컬럼 조건에 해당되는 약 60만 건의 데이터를 읽어내면서 불필요한 I/O가 사용된다.

5. [TO-BE] 튜닝 포인트

INTERSECT는 비교 대상 두 테이블을 정렬한 뒤 Semi Join으로 수행되며 조인 조건은 SELECT 절에 기술된 컬럼이 된다. 최종적으로는 INTERSECT를 기준으로 상단 구문의 데이터만 출력되기 때문에 EXISTS로 구문을 변경하면 정렬 과정을 제거할 수 있고 조인 조건에 해당하는 [custid] 컬럼을 인덱스 키로 포함하면 인덱스 탐색 범위를 줄일 수 있다.

6. [TO-BE] 개선 방안

1) EXISTS 구문 변경과 [custid] 컬럼이 포함된 신규 인덱스 생성

EXISTS 구문은 서브쿼리의 조인되는 데이터가 존재하는지 여부만 판단하기 때문에 정렬하는 과정을 필요로 하지 않는다. 변경된 구문에 [custid], [date] 컬럼으로 구성된 인덱스를 생성하면[TB_Cust02] 테이블의 결과 집합 각 행에 대해 [TB_Pay02] 테이블의 데이터 1건씩만을 탐색하여 I/O를 개선할 수 있다.

■ 인덱스 생성 구문

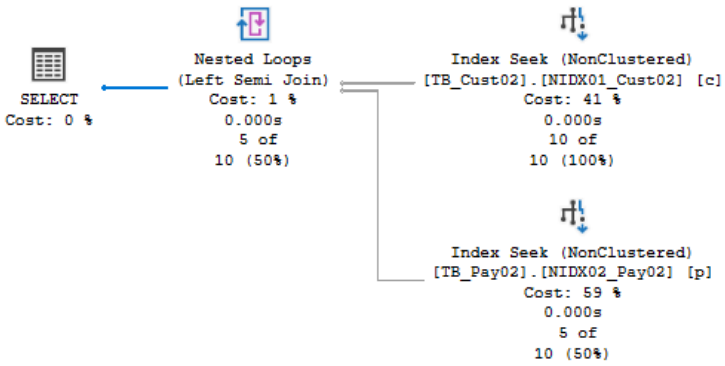
```
CREATE INDEX NIDX02_Pay02 ON TB_Pay02 (custid,date)
GO
```

■ 변경된 SQL 구문

```
SELECT custid
FROM TB_Cust02 c
WHERE level = 5
AND EXISTS (SELECT 'x'
             FROM TB_Pay02 p
             WHERE c.custid = p.custid
             AND date >= '2001-01-01')
GO
```

· 실행 계획

Rows	Executes	StmtText
5	1	SELECT custid FROM TB_Cust02 c WHERE level = 5 AND EXISTS (SELECT 'x' FROM TB_Pay02 p WHERE c.custid = p.custid AND date >= '2001-01-01')
5	1	¦--Nested Loops(Left Semi Join, OUTER REFERENCES:([c].[custid]))
10	1	¦--Index Seek(OBJECT:([tuning].[dbo].[TB_Cust02].[NIDX01_Cust02] AS [c]), SEEK:([c].[level]=(5)) Ordered Forward)
5	10	¦--Index Seek(OBJECT:([tuning].[dbo].[TB_Pay02].[NIDX02_Pay02] AS [p]), SEEK:([p].[custid]=[tuning].[dbo].[TB_Cust02].[custid] as [c].[custid] AND [p].[date] >= '2001-01-01 00:00:00.000') Ordered Forward)




```
테이블 'TB_Pay02'. 검색 수 10, 논리적 읽기 수 30
테이블 'TB_Cust02'. 검색 수 1, 논리적 읽기 수 2
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· **리소스 사용률 비교**

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	31	44	1,925
TO-BE	0	0	32

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

26. INTERSECT 구문 성능 이슈

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인 분석 및 개선해줘

```
[AS-IS]
SELECT custid FROM TB_Cust02 WHERE level = 5
INTERSECT
SELECT custid FROM TB_Pay02 WHERE date >= '2001-01-01';
```

2. 분석 및 주요 최적화

1) 문제점 분석

항목	원본 쿼리	비고
INTERSECT	양쪽 결과를 정렬 + DISTINCT 후 교집합	Sort/Hash 비용 발생
TB_Cust02	Heap, 인덱스: level only	custid 커버링 불가
TB_Pay02	Heap, 인덱스: date only	custid 인덱스 없음 → Full Table Scan
INTERSECT 특성	양쪽 모두 DISTINCT 처리	불필요한 중복 제거 비용

2) 핵심 병목

INTERSECT는 양쪽 집합 모두 정렬/해싱 후 비교하며, TB_Pay02에 custid 인덱스가 없어 Full Scan 병목(논리적 읽기 1,923)이 발생합니다

3. 개선안

```
-- 개선 쿼리 INTERSECT를 EXISTS로 전환하여 불필요한 정렬/중복 제거를 제거합니다.

SELECT DISTINCT a.custid FROM dbo.TB_Cust02 a
WHERE a.level = 5 AND EXISTS (SELECT 1 FROM dbo.TB_Pay02 b
                              WHERE b.custid = a.custid AND b.date >= '2001-01-01');
```

■ 변경 요약

변경	이전	이후	효과
집합 연산	INTERSECT (양쪽 정렬 + DISTINCT)	EXISTS	정렬 비용 제거
TB_Pay02 탐색	Full Scan 후 정렬	매칭 즉시 중단 (short-circuit)	I/O 감소
중복 제거	양쪽 모두 DISTINCT	외부 DISTINCT 1회만	비용 절감

· INTERSECT는 결과에 DISTINCT를 적용하므로, EXISTS 전환 시 외부에 DISTINCT를 추가하여 동일한 결과를 보장합니다.

· 추가 인덱스 권장, TB_Pay02에 커버링 인덱스를 추가하면 근본적으로 해결됩니다:

```
CREATE NONCLUSTERED INDEX NIDX02_Pay02 ON dbo.TB_Pay02 (custid, date);
```

27 데이터 중복 제거에 대한 스캔 범위 개선

데이터의 중복된 값을 제거하기 위해 모든 데이터를 읽어내면서 I/O가 높게 발생하는 사례이다.

1. 사전 구성 스크립트

```
CREATE TABLE TB_Vip01 (level INT, logtime DATETIME, fileId VARCHAR(50))

INSERT INTO TB_Vip01
SELECT seq%59+1 AS level
      , CASE WHEN (seq%59+1)%2=0 THEN DATEADD(SS, -seq, '2020-08-09 00:00:00.000')
        ELSE DATEADD(SS, -seq, '2020-09-09 00:00:00.000') END AS logtime
      , NEWID() AS fileId
FROM (SELECT ROW_NUMBER() OVER (ORDER BY a.number) AS seq
       FROM master..spt_values a JOIN master..spt_values b
       ON a.type ='P'
       AND b.type ='P'
       AND a.number < 1000
       AND b.number < 1000
) c

CREATE INDEX IDX01_Vip01 ON TB_Vip01(logtime,level)
GO
```

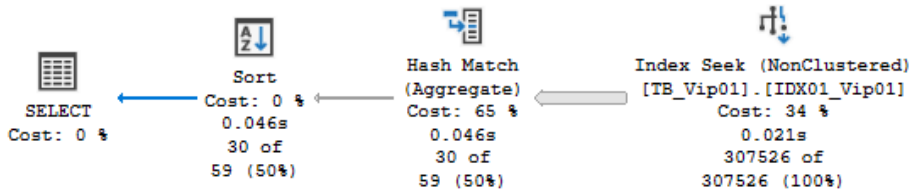
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT DISTINCT level
FROM TB_Vip01
WHERE logtime BETWEEN '20200901' AND '20200908'
ORDER BY level
GO
```

· 실행 계획

Rows	Executes	StmtText
30	1	SELECT DISTINCT level FROM TB_Vip01 WHERE logtime BETWEEN '20200901' AND '20200908' ORDER BY level
30	1	¦--Sort(ORDER BY:([tuning].[dbo].[TB_Vip01].[level] ASC))
30	1	¦--Hash Match(Aggregate, HASH:([tuning].[dbo].[TB_Vip01].[level]), RESIDUAL:([tuning].[dbo].[TB_Vip01].[level] = [tuning].[dbo].[TB_Vip01].[level]))
307526	1	¦--Index Seek(OBJECT:([tuning].[dbo].[TB_Vip01].[IDX01_Vip01]), SEEK:([tuning].[dbo].[TB_Vip01].[logtime] >= '2020-09-01 00:00:00.000' AND [tuning].[dbo].[TB_Vip01].[logtime] <= '2020-09-08 00:00:00.000') Ordered Forward)




```
테이블 'Worktable'. 스캔 수 0, 논리적 읽기 0
테이블 'Workfile'. 스캔 수 0, 논리적 읽기 0
테이블 'TB_Vip01'. 스캔 수 11, 논리적 읽기 998
SQL Server 실행 시간:
CPU 시간 = 62ms, 경과 시간 = 67ms
```

3. [AS-IS] 통계 정보

- **인덱스 정보**

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Vip01	IDX01_Vip01	NONCLUSTERED	0	logtime, level	-

4. [AS-IS] 구문 설명

[TB_Vip01] 테이블의 [logtime] 컬럼 값이 '20200901'부터 '20200908'사이에 해당하는 데이터를 [level] 컬럼으로 데이터를 중복 제거하여 조회하는 구문이다. [TB_Vip01] 테이블의 [level] 컬럼은 1부터 60까지 구성되어 있으며 그 중 중복 값이 제거된 최종 30건이 출력된다. 인덱스를 통해 조건절에 부합하는 데이터를 탐색한 후에 중복 제거가 수행되기 때문에 탐색된 30만 건의 데이터를 읽어내면서 불필요한 I/O가 발생한다.

5. [TO-BE] 튜닝 포인트

범위 조건 내에서 중복 값을 포함한 30만 건의 데이터를 모두 읽어내는 것에서 많은 I/O가 사용된다. 중복된 데이터를 모두 읽지 않도록 하는 방법으로 EXISTS 문을 고려할 수 있는데, [level] 컬럼 값의 범위에 해당되는 1부터 60까지의 데이터를 별도로 구성하여 [TB_Vip01] 테이블에 EXISTS 문으로 조인하면 각 입력 값은 조인되는 첫 행을 만나는 순간 스캔을 중단하고 결과 집합에 포함한다. 중복된 데이터를 모두 읽어내지 않는 부분에서 많은 I/O를 개선할 수 있다.

6. [TO-BE] 개선 방안

1) EXISTS로 구문 변경 및 신규 인덱스 생성

시스템 테이블(master..spt_values)을 이용하여 1부터 60까지의 데이터 리스트를 구성하고, EXISTS 문을 통해 [TB_Vip01] 테이블에 조인하는 것으로 구문을 변경한다. [TB_Vip01] 테이블의 조건에 해당되는 [level] 컬럼과 [logtime] 컬럼을 신규 인덱스 키로 생성하면 1부터 60까지의 값에 대해 [TB_Vip01] 테이블의 데이터 존재 유무를 인덱스로 탐색하기 때문에 전체적인 I/O를 감소시킬 수 있다.

■ 인덱스 생성 구문

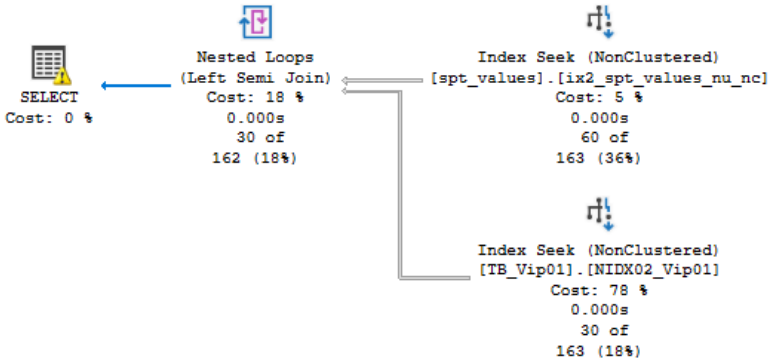
```
CREATE INDEX NIDX02_Vip01 ON TB_Vip01(level, logtime)
GO
```

■ 변경된 SQL 구문

```
SELECT a.level
FROM (SELECT number as level
       FROM master..spt_values WITH(NOLOCK)
       WHERE type = 'P' AND number BETWEEN 1 AND 60) a
WHERE EXISTS (SELECT level
              FROM TB_Vip01
              WHERE level = a.level AND logtime BETWEEN '20200901' AND '20200908')
ORDER BY a.level
GO
```

- **실행 계획**

Rows	Executes	StmtText
30	1	SELECT a.level FROM (SELECT number as level FROM master..spt_values WITH(nolock) WHERE type = 'P' AND number BETWEEN 1 and 60) a WHERE EXISTS (SELECT level FROM TB_Vip01 WHERE level = a.level and logtime BETWEEN '20200901' AND '20200908') ORDER BY a.level
30	1	¦--Nested Loops(Left Semi Join, OUTER REFERENCES:([mssqlsystemresource].[sys].[spt_values].[number], [Expr1008]) WITH ORDERED PREFETCH)
60	1	¦--Index Seek(OBJECT:([mssqlsystemresource].[sys].[spt_values].[ix2_spt_values_nu_nc]), SEEK:([mssqlsystemresource].[sys].[spt_values].[number] >= (1) AND [mssqlsystemresource].[sys].[spt_values].[number] <= (60)),WHERE:(CONVERT(nchar(3),[mssqlsystemresource].[sys].[spt_values].[type],0)=N'P') Ordered Forward)
30	60	¦--Index Seek(OBJECT:([tuning].[dbo].[TB_Vip01].[IDX02_Vip01]), SEEK:([tuning].[dbo].[TB_Vip01].[level]=[mssqlsystemresource].[sys].[spt_values].[number] AND [tuning].[dbo].[TB_Vip01].[logtime] >= '2020-09-01 00:00:00.000' AND [tuning].[dbo].[TB_Vip01].[logtime] <= '2020-09-08 00:00:00.000'), WHERE:([tuning].[dbo].[TB_Vip01].[level]>=(1) AND [tuning].[dbo].[TB_Vip01].[level]<=(60)) Ordered Forward)




```
테이블 'TB_Vip01'. 스캔 수 60, 논리적 읽기 220
테이블 'spt_values'. 스캔 수 1, 논리적 읽기 4
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

- **리소스 사용률 비교**

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	62	67	998
TO-BE	0	0	224

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

27. 데이터 중복 제거에 대한 스캔 범위 개선

1. 프롬프트

> [AS-IS]
> 구문은 조건에 해당하는 데이터는 30만건이지만, 중복을 제거하면 30건으로 출력되는 구문이다. level은 1~59로만 이뤄져 있기에 불필요한 중복 데이터를 탐색하는 과정에서 비효율이 발생하고 있다. 이런 비효율을 개선해줘.

```
[AS-IS]
SELECT DISTINCT level FROM TB_Vip01
WHERE logtime BETWEEN '20200901' AND '20200908'
ORDER BY level;
```

2. 분석 및 주요 최적화

1) 현재 상태

항목	값
테이블	dbo.TB_Vip01 (HEAP, 100만 건, 59MB)
기존 인덱스	IDX01_Vip01 (logtime, level)
조건 일치 건수	~300,000건
최종 결과 건수	30건 (DISTINCT 적용 후)
논리적 읽기 수	1,004

2) 비효율 원인

· SELECT DISTINCT는 logtime 범위에 해당하는 300,000건 전체를 스캔한 뒤 중복을 제거합니다.

· level 값이 1~59뿐인데도 30만 건을 모두 읽는 구조입니다.

3) 최적화 방안

· 인덱스 추가 + EXISTS 패턴 쿼리 재작성

· level 값(1~59)을 미리 생성하고, 각 값에 대해 존재 여부만 확인하면 됩니다.

· 이를 위해 level이 선행 컬럼인 새 인덱스가 필요합니다.

3. 개선안

```sql
-- 신규 인덱스: level 선행 컬럼으로 EXISTS 패턴 최적화

CREATE NONCLUSTERED INDEX IDX02_Vip01_Level_Logtime ON dbo.TB_Vip01 (level, logtime);


-- 최적화된 쿼리: EXISTS + 재귀 CTE로 level 값(1~59)별 존재 여부만 확인

WITH Levels AS (
    SELECT 1 AS level
    UNION ALL
    SELECT level + 1 FROM Levels WHERE level < 59
)
SELECT L.level
FROM Levels L
WHERE EXISTS (
    SELECT 1
    FROM dbo.TB_Vip01 t
    WHERE t.level = L.level
      AND t.logtime >= '20200901'
      AND t.logtime <= '20200908'
)
ORDER BY L.level
OPTION (MAXRECURSION 59);
```

Section 04

변수 및 임시 객체 활용 이슈

변수, 임시 테이블, 테이블 변수 등의 사용은 편의성을 제공하지만 성능 저하의 원인이 되기도 한다. 이 섹션에서는 이러한 객체 사용 시 발생할 수 있는 문제와 최적화 방안을 살펴본다.

28 변수 테이블의 통계 개선 - 1

변수 테이블을 사용했을 때 부정확한 통계 정보로 인해 잘못된 실행 계획이 수립된 사례이다.

1. 사전 구성 스크립트

```
SELECT ROW_NUMBER() OVER (ORDER BY a.number) seq
      , NEWID() main_code
INTO TB_Main01
FROM master..spt_values a , (SELECT number
                             FROM master..spt_values
                             WHERE type = 'P' AND number < 10) b
WHERE a.type = 'P'
AND a.number BETWEEN 1 AND 1000

SELECT c.sub_code, ASCII(SUBSTRING(sub_code,1,1))*c.number value
INTO TB_Sub01
FROM (SELECT b.number,CONVERT(NVARCHAR(36),main_code) sub_code
      FROM TB_Main01 a, (SELECT number
                         FROM master..spt_values
                         WHERE type = 'P'
                         AND number BETWEEN 1 AND 40) b
      WHERE a.seq%2 = 0
) c
GO
```

2. [AS-IS] SQL 구문 및 실행 계획

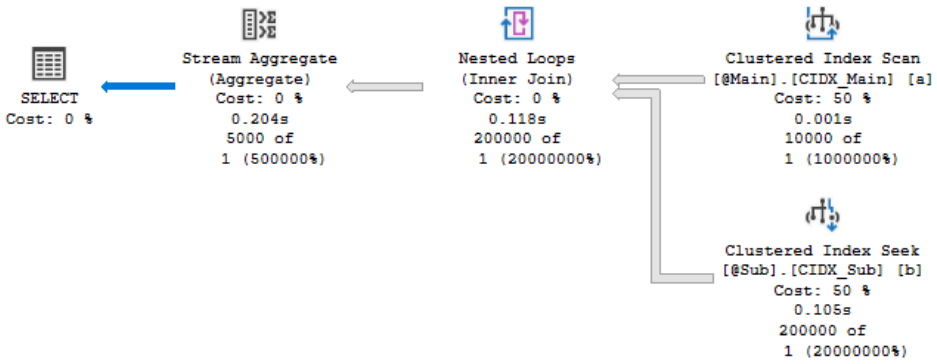
■ SQL 구문

```
DECLARE @Main TABLE (seq INT,  main_code NVARCHAR(50)
INDEX CIDX_Main CLUSTERED(seq,main_code) )
```

```
DECLARE @Sub TABLE (sub_code NVARCHAR(50), value INT
INDEX CIDX_Sub CLUSTERED(sub_code,value) )

INSERT INTO @Main SELECT * FROM TB_Main01
INSERT INTO @Sub SELECT * FROM TB_Sub01

SELECT a.seq, a.main_code, MAX(b.value) max_value
FROM @Main a INNER JOIN @Sub b
ON a.main_code = b.sub_code
GROUP BY a.seq, a.main_code
GO
```

· **실행 계획 (INSERT 구문 제외)**

Rows	Executes	StmtText	EstimateRows
5000	1	SELECT a.seq, a.main_code, MAX(b.value) max_value FROM @Main a INNER JOIN @Sub b ON a.main_code = b.sub_code GROUP BY a.seq, a.main_code	1
5000	1	¦--Stream Aggregate(GROUP BY:([a].[seq], [a].[main_code]) DEFINE:([Expr1004]=MAX(@Sub.[value] as [b].[value])))	1
200000	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([a].[main_code]))	1
10000	1	¦--Clustered Index Scan(OBJECT:(@Main AS [a]), Ordered Forward)	1
200000	10000	¦--Clustered Index Seek(OBJECT:(@Sub AS [b]), SEEK:([b].[sub_code]=@Main.[main_code] as [a].[main_code]) Ordered Forward)	1



테이블 '#A2C4BB92'. 검색 수 10000, 논리적 읽기 수 33145
테이블 '#A1D09759'. 검색 수 1, 논리적 읽기 수 118
SQL Server 실행 시간:
CPU 시간 = 250ms, 경과 시간 = 272ms

3. [AS-IS] 통계 정보

· **인덱스 정보**

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
@Main	CIDX_Main	CLUSTERED	0	seq, main_code	-
@Sub	CIDX_Sub	CLUSTERED	0	sub_code, value	-

4. [AS-IS] 구문 설명

[TB_Main01] 테이블과 [TB_Sub01] 테이블을 각각 변수 테이블인 [@Main]과 [@Sub]에 삽입하고 두 변수 테이블을 조인하여 [seq], [main_code] 컬럼의 데이터마다 MAX(value) 값을 구하는 구문이다. 별도의 조건절이 제공되지 않았기 때문에 선행 테이블의 모든 행을 읽고 처리를 하는데 실행 계획이 NL Join으로 수행되면서 후행 테이블로 10,000번의 조인 시도를 하게 된다. 여기서 NL Join으로 수행되는 이유는 변수 테이블의 통계 정보를 활용할 수 없기 때문이다. 변수 테이블은 통계 정보를 생성하지 않고 실행 계획을 생성하기 때문에 많은 양의 데이터를 처리하는 구문에서는 변수 테이블의 사용을 주의하여야 한다.

5. [TO-BE] 튜닝 포인트

변수 테이블을 사용할 경우 통계 정보를 생성하지 않으며 CREATE INDEX 구문도 사용이 제한된다. 반면 임시 테이블은 CREATE INDEX 및 통계 정보를 생성하여 관리하기 때문에 카디널리티 추정이 가능해진다.

6. [TO-BE] 개선 방안

1) 변수 테이블을 임시 테이블로 변경

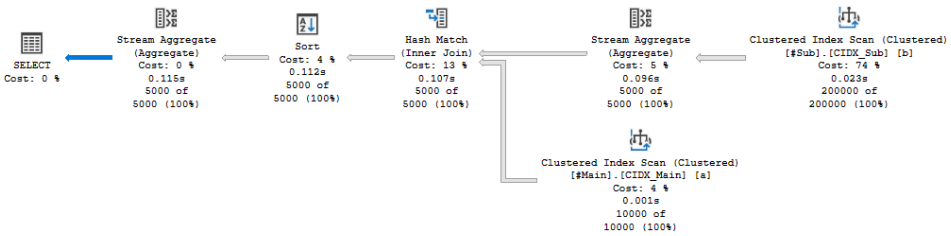
변수 테이블을 임시 테이블로 변경하여 통계 정보를 활용한 실행 계획을 생성할 수 있도록 유도한다.

■ 변경된 SQL 구문

```
CREATE TABLE #Main (seq INT, Main_Code nvarchar(50))
INSERT INTO #Main SELECT * FROM TB_Main01

CREATE TABLE #Sub (Sub_Code NVARCHAR(50), value INT)
INSERT INTO #Sub SELECT * FROM TB_Sub01

CREATE CLUSTERED INDEX CIDX_Main ON #Main(seq, main_code)
CREATE CLUSTERED INDEX CIDX_Sub ON #Sub(Sub_Code, value)

SELECT a.seq, a.main_code, MAX(b.value) max_value
FROM #Main a INNER JOIN #Sub b
ON a.main_code = b.sub_code
GROUP BY a.seq, a.main_code
GO
```

· **실행 계획 (INSERT 구문 제외)**

Rows	Executes	StmtText	EstimateRows
5000	1	SELECT a.seq, a.main_code, MAX(b.value) max_value FROM #Main a INNER JOIN #Sub b ON a.main_code = b.sub_code GROUP BY a.seq, a.main_code	4999
5000	1	¦--Stream Aggregate(GROUP BY:([a].[Main_Code], [a].[seq]) DEFINE:([Expr1004]=MAX([partialagg1005])))	4999
5000	1	¦--Sort(ORDER BY:([a].[Main_Code] ASC, [a].[seq] ASC))	4999
5000	1	¦--Hash Match(INNER JOIN, HASH:([b].[Sub_Code])=([a].[Main_Code]), RESIDUAL:([tempdb].[dbo].[#Sub].[Sub_Code] as [b].[Sub_Code]=[tempdb].[dbo].[#Main].[Main_Code] as [a].[Main_Code]))	4999
5000	1	¦--Stream Aggregate(GROUP BY:([b].[Sub_Code]) DEFINE:([partialagg1005]=MAX([tempdb].[dbo].[#Sub].[value] as [b].[value])))	5000
200000	1	¦ ¦--Clustered Index Scan(OBJECT:([tempdb].[dbo].[#Sub] AS [b]), Ordered Forward)	200000
10000	1	¦--Clustered Index Scan(OBJECT:([tempdb].[dbo].[#Main] AS [a]))	10000




```
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'Workfile'. 검색 수 0, 논리적 읽기 수 0
테이블 '#Main__0000000000E0'. 검색 수 1, 논리적 읽기 수 117
테이블 '#Sub___0000000000E1'. 검색 수 1, 논리적 읽기 수 2302
SQL Server 실행 시간:
CPU 시간 = 203ms, 경과 시간 = 204ms
```

7. 개선 효과

· **리소스 사용률 비교**

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	250	272	33,263
TO-BE	203	204	2,419

GitHub Copilot 채팅 결과

Claude Opus 4.6

28. 변수 테이블의 통계 개선 - 1

1. 프롬프트

> [AS-IS]
> 구문의 문제는 테이블 변수 사용으로 인해 잘못된 실행 계획(NL Join)으로 과도한 I/O가 발생한다. 성능 개선안을 알려줘.

```
[AS-IS]
DECLARE @Main TABLE (seq INT, main_code NVARCHAR(50)
INDEX CIDX_Main CLUSTERED(seq, main_code))

DECLARE @Sub TABLE (sub_code NVARCHAR(50), value INT
INDEX CIDX_Sub CLUSTERED(sub_code, value) )

INSERT INTO @Main SELECT * FROM TB_Main01
INSERT INTO @Sub SELECT * FROM TB_Sub01

SELECT a.seq, a.main_code, MAX(b.value) max_value
FROM @Main a INNER JOIN @Sub b
ON a.main_code = b.sub_code
GROUP BY a.seq, a.main_code;
```

2. 분석 및 주요 최적화

1) 현재 상태

항목	값
TB_Main01 행 수	10,000
TB_Sub01 행 수	200,000
원본 테이블 인덱스	없음
호환성 수준	140 (SQL 2017)

2) 발견된 문제점

#	문제	영향
1	테이블 변수 사용	옵티마이저가 행 수를 1로 추정 → NL Join 선택 → 200,000번 반복 I/O
2	데이터 타입 불일치	원본 seq는 bigint인데 변수는 INT, main_code는 uniqueidentifier인데 변수는 NVARCHAR(50) → CONVERT_IMPLICIT 발생
3	SELECT * 사용	불필요한 컬럼 포함 가능성 및 스키마 변경 시 오류 위험
4	불필요한 데이터 복사	원본 테이블을 그대로 조인할 수 있는데 테이블 변수로 복사하는 오버헤드

3. 개선안

```
-- 임시 테이블(#temp) 사용
-- 테이블 변수를 반드시 써야 한다면, 임시 테이블로 교체하세요.
-- 임시 테이블은 통계를 유지하므로
-- 옵티마이저가 정확한 카디널리티를 추정합니다.

CREATE TABLE #Main
(seq BIGINT,main_code UNIQUEIDENTIFIER,INDEX CIDX_Main CLUSTERED (seq, main_code));

CREATE TABLE #Sub
(sub_code NVARCHAR(50),value INT,INDEX CIDX_Sub CLUSTERED (sub_code, value));

INSERT INTO #Main SELECT seq, main_code FROM dbo.TB_Main01;
INSERT INTO #Sub SELECT sub_code, value FROM dbo.TB_Sub01;

SELECT a.seq, a.main_code, MAX(b.value) AS max_value
FROM #Main a INNER JOIN #Sub b ON CAST(a.main_code AS NVARCHAR(50)) = b.sub_code
GROUP BY a.seq, a.main_code;

DROP TABLE #Main;
DROP TABLE #Sub;
```

29 변수 테이블의 통계 개선 - 2

변수 테이블을 사용했을 때 부정확한 통계 정보로 인해 잘못된 실행 계획이 수립된 사례이다.

1. 사전 구성 스크립트

```
SELECT id, CONVERT(NVARCHAR,id)+CONVERT(NVARCHAR(50),NEWID()) main_code
INTO TB_Main02
FROM (SELECT ROW_NUMBER() OVER (ORDER BY a.number) id
      FROM master..spt_values a, (SELECT number
                                  FROM master..spt_values
                                  WHERE type = 'P' AND number BETWEEN 1 AND 1000) b
      WHERE a.type = 'P'
      AND a.number BETWEEN 1 AND 1000
) c
GO
```

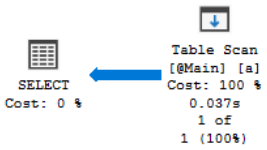
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
DECLARE @Main TABLE (id INT, main_code NVARCHAR(50) INDEX NIDX01_Main NONCLUSTERED
(id))
INSERT INTO @Main SELECT * FROM TB_Main02

SELECT a.id, a.main_code AS code
FROM @Main a
WHERE a.id = 127
GO
```

· 실행 계획 (INSERT 구문 제외)

Rows	Executes	StmtText	EstimateRows
1 1	1 1	SELECT a.id, a.main_code AS code FROM @Main a WHERE a.id = 127 ¦--Table Scan(OBJECT:(@Main AS [a]), WHERE:(@Main.[id] as [a].[id]=(127)))	1 1




```
테이블 '#AAB827D7'. 검색 수 1, 논리적 읽기 수 12482
SQL Server 실행 시간:
CPU 시간 = 78ms, 경과 시간 = 98ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
@Main	NIDX01_Main	NONCLUSTERED	0	Id	-

· 데이터 건수

	table_name	rowcnt
1	TB_Main02	1000000

4. [AS-IS] 구문 설명

[TB_Main02] 테이블을 변수 테이블인 [@Main]에 삽입하여 [id] = 127인 데이터를 출력하는 구문이다. [id] 컬럼이 키로 구성된 인덱스가 존재하며 조건에 해당하는 결과는 1건임에도 테이블 전체를 스캔하면서 불필요한 I/O 비용을 사용하였다. 변수 테이블은 통계 정보를 관리하지 않으며 테이블 내의 행이 없다고 가정하기 때문에 인덱스를 탐색하여 RID Lookup을 수행하는 것보다 테이블 전체를 읽는 것이 더 효율적이라 판단한 것이다.

5. [TO-BE] 튜닝 포인트

[@Main] 테이블의 총 데이터는 100만 건이며 그 중 [id] = 127 조건에 해당하는 데이터는 1건이다. 임시 테이블은 이러한 통계 정보들을 관리하기 때문에 테이블 전체를 읽어내는 것보다 하나의 행을 인덱스로 탐색하여 RID Lookup을 수행하는 것이 효율적인 것을 판단할 수 있다.

6. [TO-BE] 개선 방안

1) 변수 테이블을 임시 테이블로 변경

변수 테이블을 임시 테이블로 변경하여 통계 정보를 활용한 실행 계획을 생성할 수 있도록 유도한다.

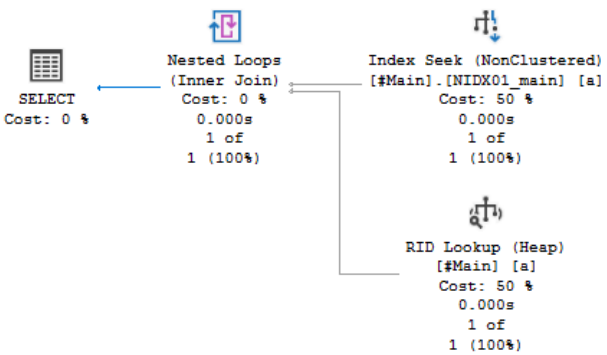
■ 변경된 SQL 구문

```
CREATE TABLE #Main (id INT,  main_code NVARCHAR(50))
INSERT INTO #Main SELECT * FROM TB_Main02

CREATE INDEX NIDX01_main ON #Main(id)

SELECT a.id, a.main_code AS code
FROM #Main a
WHERE a.id = 127
GO
```

· 실행 계획 (INSERT 구문 제외)

Rows	Executes	StmtText	EstimateRows

1
1
0
1
 	
1
	1
	1
	0
	1
	
	1
		SELECT a.id, a.main_code AS code FROM #Main a WHERE a.id = 127
		  ¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([Bmk1000]) OPTIMIZED)
		  ¦ ¦--Compute Scalar(DEFINE:([Expr1002]=BmkToPage([Bmk1000])))
		  ¦ ¦ ¦--Index Seek(OBJECT:([tempdb].[dbo].[#Main] AS [a]),
		  ¦ ¦ SEEK:([a].[id]=(127)) Ordered Forward)
		  ¦ ¦--RID Lookup(OBJECT:([tempdb].[dbo].[#Main] AS [a]),
		  ¦ SEEK:([Bmk1000]=[Bmk1000]) LOOKUP Ordered Forward)
			1
			1
			1
			1
			
			1




```
테이블 '#Main___000000000103'. 검색 수 1, 논리적 읽기 수 4
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	78	98	12,482
TO-BE	0	0	4

GitHub Copilot 채팅 결과

Claude Opus 4.6

29. 변수 테이블의 통계 개선 - 2

1. 프롬프트

> [AS-IS]
> 구문에 변수 테이블을 사용하여 통계 정보를 활용할 수 없어 비효율이 발생했다.
> 이 부분을 분석해서 성능 개선안을 알려줘.
> *조건 : 원본 테이블에 대한 직접 삭제는 허용되지 않는다.

```
[AS-IS]
DECLARE @Main
TABLE (id INT, main_code NVARCHAR(50) INDEX NIDX01_Main NONCLUSTERED (id))
INSERT INTO @Main SELECT * FROM TB_Main02

SELECT  a.id, a.main_code AS code
FROM @Main a
WHERE a.id = 127;
```

2. 분석 및 주요 최적화

1) 현재 환경

항목	값
원본 테이블	dbo.TB_Main02 (HEAP, 인덱스 없음)
행 수	1,000,000건
id 컬럼 데이터 타입	bigint
main_code 컬럼 데이터 타입	nvarchar(80)

2) 발견된 문제점

#	문제	영향
1	테이블 변수는 통계를 유지하지 않음	옵티마이저가 항상 1행으로 추정하여 비효율적 실행 계획 생성
2	데이터 타입 불일치 – 원본 id는 bigint인데 변수 테이블은 INT로 선언	CONVERT_IMPLICIT 발생, 인덱스 Seek 불가 → Scan 유발
3	100만건 전체를 복사 후 1건만 조회	불필요한 I/O 및 tempdb 부하
4	SELECT * 사용	컬럼 변경 시 오류 가능, 명시적 컬럼 지정 필요

3. 개선안

```
-- 임시 테이블은 통계가 자동 생성되므로 옵티마이저가 정확한 카디널리티를
-- 추정할 수 있습니다.

CREATE TABLE #Main
(id BIGINT NULL,
main_code NVARCHAR(80)  NULL,
INDEX NIDX01_Main NONCLUSTERED (id));

INSERT INTO #Main (id, main_code)
SELECT id, main_code
FROM dbo.TB_Main02;

SELECT a.id, a.main_code AS code
FROM #Main a
WHERE a.id = 127;

DROP TABLE #Main;
```

*@Main → #Main으로 전환하여 통계 자동 생성

30 변수 테이블의 기능 제약 개선

변수 테이블의 전체 데이터를 삭제할 때 DELETE 명령으로 인해 성능이 저하되는 사례이다.

1. 사전 구성 스크립트

```
SELECT a.number, a.type, a.low, a.high, a.status INTO TB_Del
FROM master..spt_values a, (SELECT number
                            FROM master..spt_values
                            WHERE type = 'P') b
WHERE a.type = 'P'
GO
```

2. [AS-IS] SQL 구문 및 실행 계획

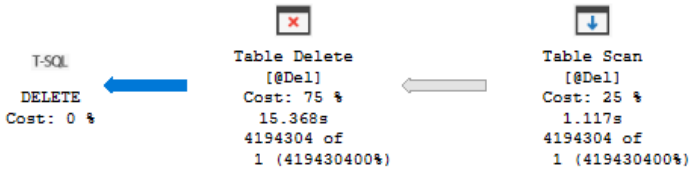
■ SQL 구문

```
DECLARE @Del TABLE (number INT,  type NVARCHAR(50), low INT, high INT, status INT )
INSERT INTO @Del SELECT * FROM TB_Del

DELETE FROM @Del
GO
```

· 실행 계획 (INSERT 구문 제외)

Rows	Executes	StmtText

4194304
4194304
4194304
	1
	1
	1
		DELETE FROM @Del
		  ¦--Table Delete(OBJECT:(@Del))
		  ¦--Table Scan(OBJECT:(@Del))




```
테이블 '#BAEE8FA0'. 검색 수 1, 논리적 읽기 수 4212462
SQL Server 실행 시간:
CPU 시간 = 20469ms, 경과 시간 = 23031ms
```

3. [AS-IS] 통계 정보

· 인덱스 없음

· 데이터 건수

	table_name	rowcnt
1	TB_Del	4194304

4. [AS-IS] 구문 설명

[@Del]에 삽입된 전체 데이터를 삭제하는 구문이다. 변수 테이블은 TRUNCATE 구문이 지원되지 않기 때문에 전체 데이터를 삭제하기 위해선 DELETE 구문으로 수행해야 한다. DELETE 구문은 데이터를 삭제함에 따라 트랜잭션 로그를 기록하며 삭제 대상 데이터를 검색하고 제거하는 과정에서 많은 I/O가 사용된다.

5. [TO-BE] 튜닝 포인트

전체 데이터를 삭제하는 것이 목적이기 때문에 DELETE 구문보다 최소한의 로그만으로 데이터를 일괄 삭제하는 TRUNCATE 구문을 사용하는 것이 효율적이다. 변수 테이블은 TRUNCATE 구문을 지원하지 않기 때문에 임시 테이블로 구문을 변경하는 것이 좋다.

6. [TO-BE] 개선 방안

1) 변수 테이블을 임시 테이블로 변경

임시 테이블로 변경하여 데이터를 TRUNCATE 구문으로 삭제한다.

■ 변경된 SQL 구문

```
CREATE TABLE #Del (number INT,  type NVARCHAR(50), low INT, high INT, status INT )
INSERT INTO #Del SELECT * FROM TB_Del

TRUNCATE TABLE #Del
GO
```

- 실행 계획 (INSERT 구문 제외)

EventClass	TextData	CPU	Duration	Reads	RowCounts
SQL:BatchCompleted	SET STATISTICS XML ON	0	0	0	0
SQL:BatchCompleted	TRUNCATE TABLE #Del	0	0	41	0
SQL:BatchCompleted	SET STATISTICS XML OFF	0	0	0	0

TRUNCATE TABLE #Del

```
테이블 '#Del'. 논리적 읽기 수 41
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

- 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	20,469	23,031	4,212,462
TO-BE	0	0	41

GitHub Copilot 채팅 결과

Claude Opus 4.6

30. 변수 테이블의 기능 제약 개선

1. 프롬프트

```
[AS-IS]
구문에 변수 테이블을 활용한 DELETE 부분에서 과도한 I/O가 발생하였다.
이 부분을 분석해서 성능 개선안을 알려줘.
* 조건 : 원본 테이블에 대한 직접 삭제는 허용되지 않는다.
```

```
[AS-IS]
DECLARE @Del TABLE (number INT, type NVARCHAR(50), low INT, high INT, status INT)
INSERT INTO @Del SELECT * FROM TB_Del

DELETE FROM @Del;
```

2. 분석 및 주요 최적화

1) 분석 결과

dbo.TB_Del 테이블은 약 419만 건의 데이터를 보유하고 있으며, 인덱스가 없습니다

2) 현재 스크립트의 문제점

#	문제	영향
1	불필요한 DELETE FROM @Del	GO(배치 종료) 시 테이블 변수는 자동 소멸됩니다. 419만 건을 행 단위로 삭제하므로 트랜잭션 로그에 대량 I/O 발생
2	테이블 변수 대량 데이터	테이블 변수는 통계가 없어 SQL Server가 항상 1행으로 추정 → 비효율적 실행 계획
3	SELECT * 사용	불필요한 컬럼 포함 가능, 명시적 컬럼 나열 권장
4	데이터 타입 불일치	원본 type은 nchar(3)인데 변수는 nvarchar(50) → 암시적 변환(CONVERT_IMPLICIT) 발생

3. 개선안

```
-- 임시 테이블 + TRUNCATE (배치 내에서 반복 사용 시)
-- 만약 동일 배치 내에서 데이터를 비우고 재사용해야 한다면,
-- 임시 테이블로 전환하고 TRUNCATE TABLE을 사용하세요.

CREATE TABLE #Del (number INT, type NCHAR(3), low INT, high INT, status INT)
INSERT INTO #Del
SELECT number, type, low, high, status
```

```
-- DELETE 대신 TRUNCATE 사용 (최소 로깅, I/O 대폭 감소)

TRUNCATE TABLE #Del;
```

개선 포인트 요약

항목	AS-IS	TO-BE
삭제 방식	DELETE (행 단위 로깅, 419만건)	TRUNCATE 또는 삭제 제거
로그 I/O	행당 1건 × 419만 = 대량 로그	페이지 할당 해제만 기록 → 최소 로깅
데이터 타입	nvarchar(50) (불일치)	nchar(3) (원본과 동일)
컬럼 지정	SELECT *	명시적 컬럼 나열
통계	테이블 변수 (통계 없음)	임시 테이블 (통계 자동 생성)

*DELETE는 모든 행에 대해 트랜잭션 로그를 기록하지만, TRUNCATE는 페이지 단위 할당 해제만 기록하여 I/O가 극적으로 줄어듭니다.

31 변수가 사용된 OR 절의 성능 이슈

입력되는 바인드 값에 따라 OR 절에 의해 서로 다른 조건이 사용되는 것을 의도한 구문에서 좋지 않은 실행 계획으로 고정되어 성능적 이슈가 발생되는 사례이다.

1. 사전 구성 스크립트

```
SELECT ROW_NUMBER() OVER (ORDER BY a.number) AS userid
      , NEWID() AS txt
      , DATEADD(dd,a.number,'2020-01-01') AS date
INTO TB_OR
FROM master..spt_values a, master..spt_values b
WHERE a.type = 'P'
AND b.type = 'P'
AND a.number BETWEEN 1 AND 1000
AND b.number BETWEEN 1 AND 1000

CREATE INDEX NIDX01_OR on TB_OR (userid)
GO
```

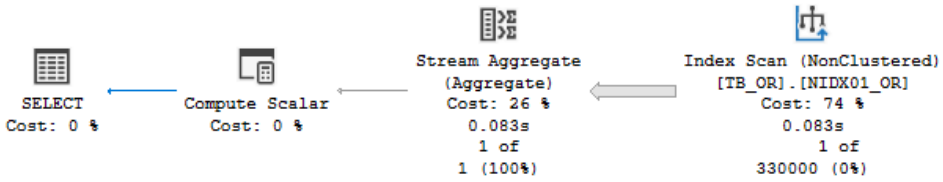
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
DECLARE @userid INT = 555
SELECT COUNT(*)
FROM TB_OR
WHERE (@userid IS NULL)
      OR (@userid IS NOT NULL AND userid = @userid)
GO
```

· 실행 계획

Rows	Executes	StmtText
1 0 1 1	1 0 1 1	SELECT COUNT(*) FROM TB_OR WHERE (@userid IS NULL) OR (@userid IS NOT NULL AND userid = @userid) ¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(int,[Expr1009],0))) ¦--Stream Aggregate(DEFINE:([Expr1009]=Count(*))) ¦--Index Scan(OBJECT:([Tuning].[dbo].[TB_OR].[NIDX01_OR]), WHERE:([@userid] IS NULL OR [@userid] IS NOT NULL AND [Tuning].[dbo].[TB_OR].[userid]=CONVERT_IMPLICIT(bigint,[@userid],0)))




```
테이블 'TB_OR'. 검색 수 1, 논리적 읽기 수 2730
SQL Server 실행 시간:
CPU 시간 = 93ms, 경과 시간 = 115ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_OR	NIDX01_OR	NONCLUSTERED	0	userid	-

4. [AS-IS] 구문 설명

조건문에 사용된 변수 @userid가 NULL이면 전체 데이터의 행 수를 출력하고, 특정 값이 지정되면 [userid] 컬럼 값에 해당하는 행의 수만 구하는 구문이다. 변수값에 따라 서로 다른 조건으로 실행 계획 수립이 필요한데, 이 중 하나의 실행 계획을 생성해야 하기 때문에 인덱스를 전체 스캔하는 것으로 계획이 고정된다. 1건의 [userid]를 필터하더라도 인덱스 전체를 스캔하기 때문에 성능 저하의 원인이 된다.

5. [TO-BE] 튜닝 포인트

변수값에 따라 인덱스 전체 스캔과 일부를 탐색하는 것을 구분할 수 있도록 구문을 변경해야 한다. UNION ALL을 통해 특정 [userid]를 검색하는 것과 전체 데이터를 스캔하는 구문을 분리하면 1건을 검색하기 위해 모든 데이터를 스캔하는 상황을 피할 수 있다.

6. [TO-BE] 개선 방안

1) UNION ALL을 사용하여 변수값에 따라 해당되는 조건의 구문이 수행될 수 있도록 변경

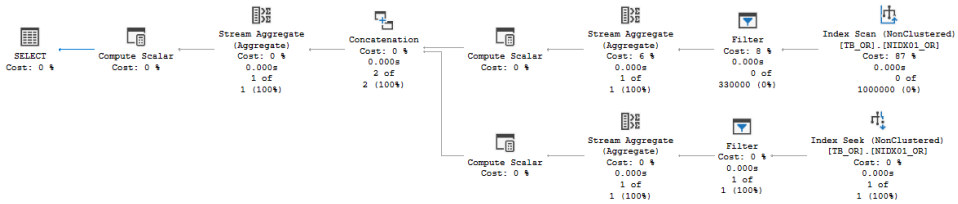
@userid 값에 따라 서로 다른 조건을 실행할 수 있도록 UNION ALL을 사용한다. UNION ALL로 연결된 두 구문이 실행되어 테이블을 두 번 액세스하는 것처럼 보이나 @userid에 특정 값이 존재할 때는 @userid IS NULL 조건에 해당하는 테이블은 참조하지 않는다. @userid가 NULL인 경우에는 [userid] 컬럼에 대한 조건이 존재하지 않아 인덱스 전체 스캔으로 수행되며, @userid가 NULL이 아닌 경우에는 [userid] = @userid 조건에 의해 인덱스 탐색으로 수행되기 때문에 잘못된 실행 계획으로 인한 비효율 I/O가 발생되는 것을 방지할 수 있다.

■ 변경된 SQL 구문

```
DECLARE @userid INT = 555
SELECT SUM(cnt) cnt
FROM (SELECT COUNT(*) cnt
       FROM TB_OR
       WHERE ( @userid IS NULL)
       UNION ALL
       SELECT COUNT(*) cnt
       FROM TB_OR
       WHERE ( @userid IS NOT NULL AND userid = @userid)
) a
GO
```

· 실행 계획

Rows	Executes	StmtText
1	1	SELECT SUM(cnt) cnt FROM (SELECT COUNT(*) cnt FROM TB_OR WHERE (@userid IS NULL) UNION ALL SELECT COUNT(*) cnt FROM TB_OR WHERE (@userid IS NOT NULL AND userid = @userid)) a
0	0	¦--Compute Scalar(DEFINE:([Expr1009]=CASE WHEN [Expr1023]=(0) THEN NULL ELSE [Expr1024] END))
1	1	¦--Stream Aggregate(DEFINE:([Expr1023]=COUNT_BIG([Union1008]), [Expr1024]=SUM([Union1008])))
2	1	¦--Concatenation
0	0	¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(int,[Expr1021],0)))
1	1	¦ ¦--Stream Aggregate(DEFINE:([Expr1021]=Count(*)))
0	1	¦ ¦--Filter(WHERE:(STARTUP EXPR([@userid] IS NULL)))
0	0	¦ ¦--Index Scan(OBJECT:([Tuning].[dbo].[TB_OR].[NIDX01_OR]))
0	0	¦--Compute Scalar(DEFINE:([Expr1007]=CONVERT_IMPLICIT(int,[Expr1022],0)))
1	1	¦--Stream Aggregate(DEFINE:([Expr1022]=Count(*)))
1	1	¦--Filter(WHERE:(STARTUP EXPR([@userid] IS NOT NULL)))
1	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_OR].[NIDX01_OR]), SEEK:([Tuning].[dbo].[TB_OR].[userid]=CONVERT_IMPLICIT(bigint,[@userid],0)) Ordered Forward)




```
테이블 'TB_OR'. 검색 수 1, 논리적 읽기 수 3
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· **리소스 사용률 비교**

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	93	115	2,730
TO-BE	0	0	3

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

31. 변수가 사용된 OR 절의 성능 이슈

1. 프롬프트

> [AS-IS]
> 구문에서 변수 값에 따라 서로 다른 조건으로 실행 계획 수립이 필요한데, 이 중 하나의 실행 계획을 생성해야 하기 때문에 인덱스를 전체 스캔하는 것으로 계획이 고정되는 문제가 있다. 이 문제를 분석해서 최적의 성능 개선안을 알려줘.

```
[AS-IS]
DECLARE @userid INT = 555
SELECT COUNT(*)
FROM TB_OR
WHERE (@userid IS NULL)
OR (@userid IS NOT NULL AND userid = @userid);
```

2. 분석 및 주요 최적화

이 쿼리는 "Catch-All Query" (만능 쿼리) 안티패턴입니다.

항목	설명
근본 원인	@userid가 NULL일 때 전체 행 반환, 값이 있으면 특정 행 필터링 – 두 시나리오를 하나의 실행 계획으로 커버해야 함
결과	옵티마이저가 안전하게 Index Scan을 선택 (Seek 불가)
인덱스 현황	NIDX01_OR (nonclustered on userid) – Seek가 가능하지만 활용 못함
데이터 타입 불일치	변수 @userid는 INT, 컬럼 userid는 BIGINT → 암시적 변환 발생 가능

3. 개선안

```
-- 방안 1. OPTION (RECOMPILE) – (가장 간결)
-- 매 실행 시 @userid의 실제 값을 보고 최적 계획을 수립합니다.

DECLARE @userid BIGINT = 555
SELECT COUNT(*)
FROM dbo.TB_OR
WHERE (@userid IS NULL)
OR (@userid IS NOT NULL AND userid = @userid)
OPTION (RECOMPILE);
```

· @userid = 555일 때 → 옵티마이저가 OR의 첫 분기를 컴파일 시점에 제거하고 userid = 555로 Index Seek 수행

· @userid = NULL일 때 → 조건 전체가 TRUE로 평가되어 전체 스캔 (의도된 동작)

```
-- 방안 2. IF/ELSE 분기 – 호출 빈도가 매우 높을 때
-- 재컴파일 비용이 부담될 경우, 쿼리를 분기하여 각각 별도 계획을 캐싱합니다.

DECLARE @userid BIGINT = 555

IF @userid IS NULL
BEGIN
    SELECT COUNT(*)
    FROM dbo.TB_OR
END
ELSE
BEGIN
    SELECT COUNT(*)
    FROM dbo.TB_OR
    WHERE userid = @userid
END;
```

■ 변경 요약

변경 사항	이유
INT → BIGINT	컬럼 userid가 BIGINT이므로 타입 일치시켜 암시적 변환(CONVERT_IMPLICIT) 방지
OPTION (RECOMPILE) 추가 (방안 1)	런타임 값 기반으로 불필요한 OR 분기를 제거하여 Index Seek 유도
IF/ELSE 분기 (방안 2)	각 시나리오별 독립된 실행 계획 캐싱 가능
dbo.TB_OR 2-part name 사용	스키마 명시로 불필요한 이름 해석 비용 제거

32 변수 사용에 따른 카디널리티 추정 이슈

변수를 사용한 SQL의 잘못된 카디널리티로 인해 성능이 저하되는 사례이다.

1. 사전 구성 스크립트

```
SELECT CASE WHEN seq  = 1 THEN 1
        WHEN seq <= 11 THEN 2
        WHEN seq <= 111 THEN 3
        WHEN seq <= 1111 THEN 4
        WHEN seq <= 11111 THEN 5
        WHEN seq <= 111111 THEN 6
        WHEN seq <= 1111111 THEN 7
        END AS gid
      , NEWID() AS txt
      , GETDATE() AS date INTO TB_Hist
FROM (SELECT ROW_NUMBER() OVER ( ORDER BY a.number) seq
      FROM master..spt_values a, (SELECT number
               FROM master..spt_values
               WHERE type = 'P'
               AND number BETWEEN 1 AND 1000) b
      WHERE a.type = 'P'
      AND a.number BETWEEN 1 AND 1200) c
WHERE seq BETWEEN 1 AND 1111111

CREATE INDEX NIDX01_Hist ON TB_Hist(gid)
GO
```

2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

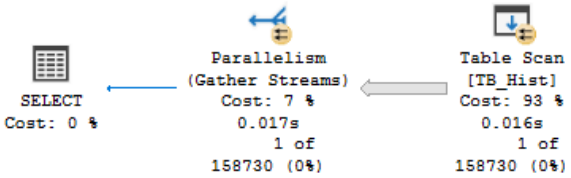
```
DECLARE @int INT
SET @int = 1
```

```
SELECT gid, txt
FROM TB_Hist
WHERE gid = @int
GO
```

· 실행 계획

Rows	Executes	StmtText	EstimateRows

1
1
1
	1
	1
	12
		SELECT gid, txt FROM TB_Hist WHERE gid = @int
		  ¦--Parallelism(Gather Streams)
		  ¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Hist]), WHERE:([Tuning].[dbo].[TB_Hist].[gid]=[@int]))
			158730.1
			158730.1
			158730.1




```
테이블 'TB_Hist'. 검색 수 13, 논리적 읽기 수 5097
SQL Server 실행 시간:
CPU 시간 = 173ms, 경과 시간 = 7ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Hist	NIDX01_Hist	NONCLUSTERED	0	gid	-

· SELECTIVITY

	All density	Average Length	Columns
1	0.1428571	4	gid

· HISTOGRAM

	RANGE_HI_KEY	RANGE_ROWS	EQ_ROWS	DISTINCT_RANGE_ROWS	AVG_RANGE_ROWS
1	1	0	1	0	1
2	2	0	10	0	1
3	3	0	100	0	1
4	4	0	1000	0	1
5	5	0	10000	0	1
6	6	0	100000	0	1
7	7	0	1000000	0	1

4. [AS-IS] 구문 설명

[TB_Hist] 테이블에서 [gid] 컬럼 값을 바인드 변수로 입력받아 조회하는 구문이다. [gid] 컬럼이 키로 구성된 인덱스가 존재하며 [gid] = 1에 해당하는 행은 1건이지만 테이블 전체를 스캔하는 실행 계획으로 수행되었다. 이는 바인드 변수가 사용되어 Histogram 데이터를 활용하지 못하고 카디널리티가 계산된 약 15만 건의 결과 집합이 생성될 것이라 예상되었으며 인덱스에 포함되어 있지 않은 [txt] 컬럼을 출력하기 위해 RID Lookup이 15만 번 수행될 것으로 예상하여 테이블 전체를 스캔하였다.

5. [TO-BE] 튜닝 포인트

카디널리티가 계산되어 많은 양의 데이터가 RID Lookup으로 수행될 것을 예상했기 때문에 최적의 실행 계획을 수립하지 못했다. 이를 개선하기 위해선 Histogram을 활용할 수 있도록 저장 프로시저를 사용하는 방법과 UNION ALL 연산을 통해 바인드 값에 따라 다른 구문으로 처리될 수 있도록 변경하는 방법, [txt] 컬럼을 인덱스에 포함시켜 RID Lookup이 발생되지 않도록 하는 방법을 고려해 볼 수 있다.

6. [TO-BE] 개선 방안

1) 저장 프로시저를 생성하여 HISTOGRAM을 활용

저장 프로시저는 매개변수를 조건절에 직접 사용하면 Histogram을 참조할 수 있다. 통계를 활용할 수 있기 때문에 [gid] = 1에 해당하는 데이터가 1건이라는 것을 확인하여 더 나은 실행 계획을 생성한다.

■ 저장 프로시저 생성

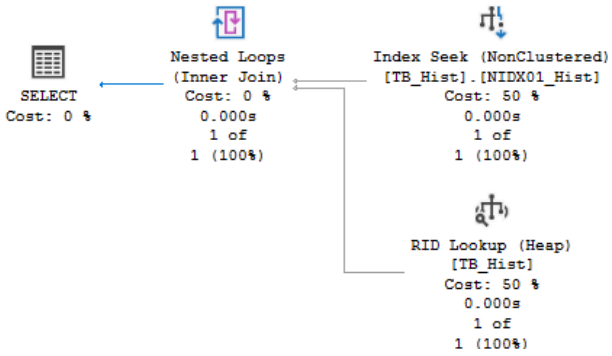
```
CREATE PROC USP_Hist @int INT
WITH RECOMPILE
AS
SELECT gid, txt
FROM TB_Hist
WHERE gid = @int
GO
```

■ 저장 프로시저 실행 구문

```
EXEC USP_Hist 1
GO
```

· 실행 계획

Rows	Executes	StmtText	EstimateRows
1	1	SELECT gid, txt FROM TB_Hist WHERE gid = @int	1
1	1	┆--Nested Loops(INNER JOIN, OUTER REFERENCES:([Bmk1000]))	1
1	1	┆--Index Seek(OBJECT:([Tuning].[dbo].[TB_Hist].[NIDX01_Hist]), SEEK:([Tuning].[dbo].[TB_Hist].[gid]=[@int]) Ordered Forward)	1
1	1	┆--RID Lookup(OBJECT:([Tuning].[dbo].[TB_Hist]), SEEK:([Bmk1000]=[Bmk1000]) LOOKUP Ordered Forward)	1




```
테이블 'TB_Hist'. 검색 수 1, 논리적 읽기 수 4
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

2) UNION ALL을 사용해 바인드 값에 따른 다른 구문 처리

UNION ALL을 사용하여 [gid] 컬럼의 데이터 분포도에 따라 다른 처리 방식으로 수행될 수 있도록 구문을 변경한다. UNION ALL을 기준으로 상단의 구문은 [gid] IN (1,2,3) 조건을 통해 *카디널리티가 다시 계산되면서 약 37건의 행을 예상하여 인덱스 탐색을 선택하고, 하단의 구문은 [gid] NOT IN (1,2,3) 조건으로 약 27만 건의 데이터를 예상하여 테이블 전체를 스캔하는 실행 계획으로 수립된다.

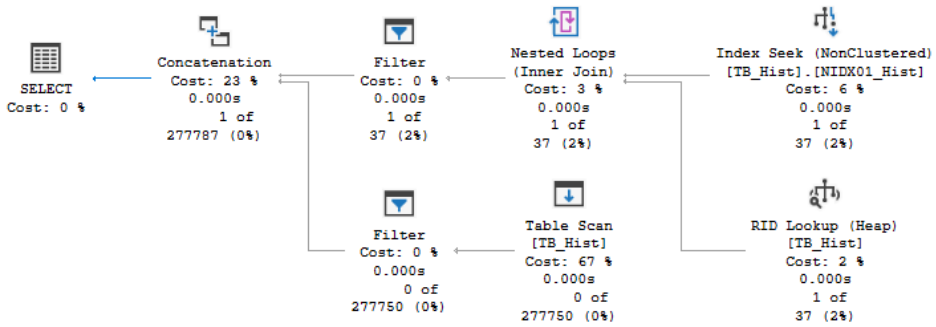
*카디널리티 계산 (1 / DISTINCT Count * Rows)
- [gid] IN (1,2,3) : 1/3 * 111 Rows = 37 Rows
- [gid] NOT IN (1,2,3) : 1/4 * 1,111,000 Rows = 277,750 Rows

■ 변경된 SQL 구문

```sql
DECLARE @int INT
SET @int = 1

SELECT gid, txt
FROM TB_Hist
WHERE gid IN (1, 2, 3) AND gid = @int
UNION ALL
SELECT gid, txt
FROM TB_Hist
WHERE gid NOT IN (1, 2, 3) AND gid = @int
GO
```

· 실행 계획

Rows	Executes	StmtText	EstimateRows
1	1	SELECT gid, txt FROM TB_Hist WHERE gid in (1,2,3) AND gid = @int UNION ALL SELECT gid, txt FROM TB_Hist WHERE gid NOT IN (1,2,3) AND gid = @int	277787
1	1	┦--Concatenation	277787
1	1	┦--Filter(WHERE:(STARTUP EXPR([@int]=(1) OR [@int]=(2) OR [@int]=(3))))	37
1	1	┦ ┦--Nested Loops(INNER JOIN, OUTER REFERENCES:([Bmk1000], [Expr1009]) WITH UNORDERED PREFETCH)	37
1	1	┦ ┦--Index Seek(OBJECT:([Tuning].[dbo].[TB_Hist].[NIDX01_Hist]), SEEK:([Tuning].[dbo].[TB_Hist].[gid]=[@int]), WHERE:([Tuning].[dbo].[TB_Hist].[gid]=(1) OR [Tuning].[dbo].[TB_Hist].[gid]=(2) OR [Tuning].[dbo].[TB_Hist].[gid]=(3)) Ordered Forward)	37
1	1	┦ ┦--RID Lookup(OBJECT:([Tuning].[dbo].[TB_Hist]), SEEK:([Bmk1000]=[Bmk1000]) LOOKUP Ordered Forward)	1
0	1	┦--Filter(WHERE:(STARTUP EXPR([@int]<>(1) AND [@int]<>(2) AND [@int]<>(3))))	277750
0	0	┦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Hist]), WHERE:([Tuning].[dbo].[TB_Hist].[gid]=[@int] AND [Tuning].[dbo].[TB_Hist].[gid]<>(1) AND [Tuning].[dbo].[TB_Hist].[gid]<>(2) AND [Tuning].[dbo].[TB_Hist].[gid]<>(3)))	277750




```
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_Hist'. 검색 수 1, 논리적 읽기 수 5
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

3) [txt] 컬럼이 포함된 인덱스로 재생성

RID Lookup 하는 과정을 생략할 수 있도록 인덱스에 누락된 [txt] 컬럼을 포함하여 재생성한다. 예상 행 수가 부정확하더라도 인덱스를 활용하기 때문에 I/O가 개선될 수 있다.

■ 인덱스 재생성

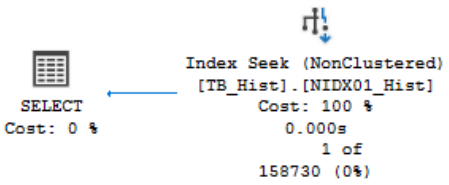
```
DROP INDEX NIDX01_Hist ON TB_Hist
CREATE INDEX NIDX01_Hist ON TB_Hist (gid) INCLUDE (txt)
GO
```

SQL 구문

```
DECLARE @int INT
SET @int = 1

SELECT gid, txt
FROM TB_Hist
WHERE gid = @int
GO
```

· 변경된 SQL 구문

Rows	Executes	StmtText	EstimateRows
1 1	1 1	SELECT gid, txt FROM TB_Hist WHERE gid = @int ¦--Index Seek(OBJECT:([tuning].[dbo].[TB_Hist].[NIDX01_Hist]), SEEK:([tuning].[dbo].[TB_Hist].[gid]=[@int]) Ordered Forward)	158730.1 158730.1




```
테이블 'TB_Hist'. 검색 수 1, 논리적 읽기 수 4
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	173	7	5,097
TO-BE (1)	0	0	4
TO-BE (2)	0	0	5
TO-BE (3)	0	0	4

GitHub Copilot 채팅 결과

Claude Opus 4.6

32. 변수 사용에 따른 카디널리티 추정 이슈

1. 프롬프트

> [AS-IS]
> 구문에서 변수에 해당하는 값은 1건이지만 바인드 변수가 사용되어 Histogram 데이터를 활용하지 못하며 테이블 스캔이 발생한다. 이 문제를 분석해서 최적의 성능 개선안을 알려줘.

```
[AS-IS]
DECLARE @int INT
SET @int = 1

SELECT gid, txt
FROM TB_Hist
WHERE gid = @int;
```

2. 분석 및 주요 최적화

1) 현재 테이블 상태

항목	값
테이블	dbo.TB_Hist
구조	HEAP (클러스터드 인덱스 없음)
총 행 수	1,111,111
인덱스	NIDX01_Hist (비클러스터드, gid)
컬럼	gid (int), txt (uniqueidentifier), date (datetime)

2) gid 데이터 분포 (심한 데이터 편향)

gid	행 수	비율
1	1	0.00009%
2	10	0.0009%
3	100	0.009%
4	1,000	0.09%
5	10,000	0.90%
6	100,000	9%
7	1,000,000	90%

3) 근본 원인

로컬 변수(@int) 사용 시, SQL Server는 컴파일 타임에 변수 값을 알 수 없습니다. 따라서 히스토그램(Histogram)의 EQ_ROWS를 참조하지 못하고, Density Vector를 사용하여 카디널리티를 추정합니다.

추정 방식	계산	예상 행 수
Density Vector 사용	0.1428 × 1,111,111	≈ 158,730 행
히스토그램 (gid=1)	EQ_ROWS 직접 참조	1 행

실제로는 1건만 반환되지만, 옵티마이저는 158,730건으로 추정합니다. 이 과대 추정으로 인해 Index Seek + RID Lookup 대신 Table Scan이 선택됩니다.

3. 개선안

```sql
-- 방안 1. OPTION (RECOMPILE) 사용 (즉시 적용 가능)
-- 실행 시점에 변수 값을 확인하여 히스토그램을 활용합니다.

DECLARE @int INT
SET @int = 1

SELECT gid, txt
FROM dbo.TB_Hist
WHERE gid = @int
OPTION (RECOMPILE);
```

장점	단점
매 실행마다 최적 플랜 생성	매번 컴파일 비용 발생
데이터 편향에 완벽 대응	실행 빈도가 매우 높으면 부담

```sql
-- 방안 2. 커버링 인덱스 생성 (근본 해결)
-- txt 컬럼을 포함(INCLUDE)하여 RID Lookup을 제거하면, 추정 행 수가 다소 과대해도
-- Index Seek가 선택될 가능성이 높아집니다.
-- 기존 인덱스를 커버링 인덱스로 교체

DROP INDEX NIDX01_Hist ON dbo.TB_Hist;

CREATE NONCLUSTERED INDEX NIDX01_Hist
ON dbo.TB_Hist (gid)
INCLUDE (txt);
```

장점 (Advantages)	단점 (Disadvantages)
RID Lookup 제거로 Tipping Point 문제 해소	인덱스 크기 증가 (txt가 uniqueidentifier = 16byte)
쿼리 힌트 없어도 성능 개선	INSERT/UPDATE 시 인덱스 유지 비용

Section 05

커서와 절차형 처리 개선

커서와 반복문 기반 처리는 직관적이지만 성능 측면에서는 비효율적일 수 있다. 이 섹션에서는 절차형 처리의 문제점을 분석하고, 집합 기반 처리로 개선하는 방법을 제시한다.

33 CURSOR 성능 이슈 - 1

CURSOR로 인해 테이블을 반복적으로 읽어내면서 많은 I/O 비용이 사용된 사례이다.

1. 사전 구성 스크립트

```
SELECT number id
      , CONVERT (NVARCHAR(36),NEWID()) code
INTO TB_Classlist
FROM master..spt_values
WHERE type = 'P' AND number BETWEEN 1 AND 500

SELECT id
      , code
      , GETDATE() date
INTO TB_Classstu
FROM TB_Classlist a, master..spt_values b
WHERE b.type = 'P' AND b.number BETWEEN 1 AND 100

CREATE INDEX NIDX01_Classlist ON TB_Classlist(id)
CREATE INDEX NIDX01_Classstu ON TB_Classstu(id) INCLUDE (code)
GO
```

2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
CREATE TABLE #tmp (id int, code nvarchar(36))

DECLARE @id VARCHAR(8)
DECLARE CUR CURSOR FOR SELECT id FROM TB_Classlist
OPEN CUR FETCH NEXT FROM CUR INTO @id
```

```
WHILE @@FETCH_STATUS = 0
BEGIN
INSERT INTO #tmp
SELECT id,code
FROM TB_Classstu
WHERE id = @id

FETCH NEXT FROM CUR INTO @id
END
CLOSE CUR
DEALLOCATE CUR

SELECT COUNT(*) FROM #tmp
GO
```

· **수행 비용 (Profiler)**

EventClass	TextData	CPU	Duration	Reads
SQL:BatchCompleted	CREATE TABLE #tmp (id int, code nvar...	94	190	69124

```
CREATE TABLE #tmp (id int, code nvarchar(36))

DECLARE @id VARCHAR(8)
DECLARE CUR CURSOR FOR SELECT id FROM TB_Classlist
OPEN CUR FETCH NEXT FROM CUR INTO @id

WHILE @@FETCH_STATUS = 0
BEGIN
INSERT INTO #tmp
SELECT id,code
FROM TB_Classstu
WHERE id = @id

FETCH NEXT FROM CUR INTO @id
END
CLOSE CUR
DEALLOCATE CUR

SELECT COUNT(*) FROM #tmp
```

```
논리적 읽기 수 69124
SQL Server 실행 시간:
CPU 시간 = 94ms, 경과 시간 = 190ms
```

3. [AS-IS] 통계 정보

· **인덱스 정보**

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Classlist	NIDX01_Classlist	NONCLUSTERED	0	id	-
TB_Classstu	NIDX01_Classstu	NONCLUSTERED	0	id	code

4. [AS-IS] 구문 설명

CURSOR로 선언된 [TB_Classlist] 테이블의 [id] 컬럼 값 전체를 [TB_Classstu] 테이블에 조건으로 탐색하고 결과 집합을 임시 테이블로 저장하여 저장된 누적 건수를 출력하는 구문이다. [TB_Classstu] 테이블의 [id] 컬럼은 하나의 입력 값에 대해 100건의 결과 집합을 추출하여 임시 테이블에 삽입하는데, 이런 과정이 [TB_Classlist] 테이블의 결과 건수(500건) 만큼 반복 수행되면서 많은 I/O를 사용하게 된다.

5. [TO-BE] 튜닝 포인트

CURSOR로 인해 [TB_Classstu] 테이블을 반복하여 탐색하고 각 결과 집합을 임시 테이블에 삽입하는 과정에서 많은 I/O 비용이 사용되었다. 불필요한 CURSOR를 제거하고 [TB_Classlist] 테이블과 [TB_Classstu] 테이블을 조인하는 구문으로 변경하면 두 테이블의 조인 조건으로 사용될 [id] 컬럼에 인덱스가 존재하기 때문에 정렬하는 과정이 생략되어 Merge Join으로 유도할 수 있다. 구문 변경으로 인해 반복적인 탐색에 의한 I/O 비용이 개선되며 테이블을 직접 조인하고 출력하기 때문에 임시 테이블의 비용도 제거할 수 있다.

6. [TO-BE] 개선 방안

1) CURSOR로 사용된 구문을 조인문으로 변경

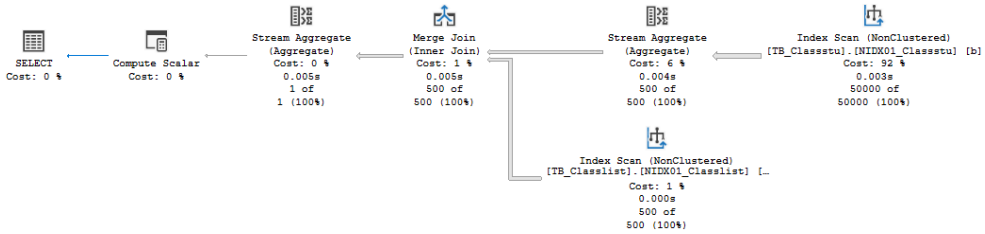
[TB_Classlist] 테이블의 [id] 컬럼과 [TB_Classstu] 테이블의 [id] 컬럼을 조인 조건으로 한 조인문으로 변경한다. 정렬된 데이터로 인해 Merge Join으로 수행되며, 테이블을 한 번씩 읽고 처리하게 되면서 전체적인 I/O가 개선된다.

■ 변경된 SQL 구문

```
SELECT COUNT(*)
FROM TB_Classlist a INNER JOIN TB_Classstu b
ON a.id = b.id
GO
```

· 실행 계획

Rows	Executes	StmtText
1	1	SELECT COUNT(*) FROM TB_Classlist a INNER JOIN TB_Classstu b ON a.id = b.id
0	0	¦--Compute Scalar(DEFINE:([Expr1004]=CONVERT_IMPLICIT(int,[globalagg1006],0)))
1	1	¦--Stream Aggregate(DEFINE:([globalagg1006]=SUM([partialagg1005])))
500	1	¦--Merge Join(INNER JOIN, MERGE:([b].[id])=([a].[id]), RESIDUAL:([Tuning].[dbo].[TB_Classstu].[id] as [b].[id]=[Tuning].[dbo].[TB_Classlist].[id] as [a].[id]))
500	1	¦--Stream Aggregate(GROUP BY:([b].[id]) DEFINE:([partialagg1005]=Count(*)))
50000	1	¦ ¦--Index Scan(OBJECT:([Tuning].[dbo].[TB_Classstu].[NIDX01_Classstu] AS [b]), Ordered Forward)
500	1	¦--Index Scan(OBJECT:([Tuning].[dbo].[TB_Classlist].[NIDX01_Classlist] AS [a]), Ordered Forward)




```
테이블 'TB_Classlist'. 검색 수 1, 논리적 읽기 수 4
테이블 'TB_Classstu'. 검색 수 1, 논리적 읽기 수 597
SQL Server 실행 시간: CPU 시간 = 0ms, 경과 시간 = 3ms
```

7. 개선 효과

· **리소스 사용률 비교**

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	94	190	69,124
TO-BE	0	3	601

AI GitHub Copilot 채팅 결과

GPT-5 mini

33. CURSOR 성능 이슈 – 1

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
CREATE TABLE #tmp (id INT, code NVARCHAR(36))

DECLARE @id VARCHAR(8)
DECLARE CUR CURSOR FOR SELECT id FROM TB_Classlist
OPEN CUR FETCH NEXT FROM CUR INTO @id

WHILE @@FETCH_STATUS = 0
```

```
BEGIN
INSERT INTO #tmp
SELECT id, code
FROM TB_Classstu
WHERE id = @id

FETCH NEXT FROM CUR INTO @id
END
CLOSE CUR
DEALLOCATE CUR

SELECT COUNT(*) FROM #tmp;
```

2. 분석 및 주요 최적화

1) 문제 요약

- 불필요한 커서 사용(행 단위 처리) — 성능 저하 및 스케일 문제.
- 불필요한 임시 테이블 사용(단순 집계 목적).
- 변수 타입 불일치: 커서에서 @id를 VARCHAR(8)로 선언했는데 TB_Classlist.id는 int.

2) 수정/최적화한 점

- 커서를 제거하고 집합 기반의 JOIN으로 한 번에 처리.
- 임시 테이블은 필요 없으면 제거.
- 모든 객체는 두-파트 이름(dbo.<object>) 사용.
- 데이터 타입 일치(조회 시 암묵적 변환 제거).

3. 개선안

```
-- 권장 쿼리:

SELECT COUNT(*) AS MatchingCount
FROM dbo.TB_Classstu AS t
INNER JOIN dbo.TB_Classlist AS c ON t.id = c.id;
```

34 CURSOR 성능 이슈 - 2

CURSOR로 인해 테이블을 반복적으로 읽어내면서 많은 I/O 비용이 사용된 사례이다.

1. 사전 구성 스크립트

```
SELECT rownum AS seq
      , NEWID() AS main_code
      , DATEADD(MINUTE, rownum, '2020-01-01 00:00:00.000') AS date
INTO TB_CURSOR001
FROM (SELECT ROW_NUMBER() OVER (ORDER BY a.number) AS rownum
           , a.number
      FROM master..spt_values a JOIN master..spt_values b
      ON a.type = 'P' AND b.type = 'P' AND a.number <= 1000 AND b.number <= 1000
) a

SELECT TOP 100000 main_code AS sub_code
      , CASE WHEN seq % 5 = 1 THEN seq
        WHEN seq % 5 = 2 THEN seq
        ELSE seq % 100 END AS value
      , CASE WHEN seq % 5 = 2 THEN seq
        ELSE seq % 200 END AS value02
INTO TB_CURSOR002
FROM TB_CURSOR001
ORDER BY seq

INSERT INTO TB_CURSOR002 SELECT * FROM TB_CURSOR002 WHERE value02 > 100
INSERT INTO TB_CURSOR002 SELECT * FROM TB_CURSOR002 WHERE value02 > 500

CREATE CLUSTERED INDEX CIDX_CURSOR001 ON TB_CURSOR001(main_code)
CREATE CLUSTERED INDEX CIDX_CURSOR002 ON TB_CURSOR002(sub_code)
CREATE INDEX NIDX01_CURSOR002 ON TB_CURSOR002 (sub_code,value,value02)
GO
```

2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
DECLARE @SUB_CODE uniqueidentifier
DECLARE @VALUE INT
DECLARE @VALUE02 INT

DECLARE CUR CURSOR FOR
      SELECT DISTINCT value, value02, sub_code
      FROM TB_CURSOR001 A
      INNER JOIN TB_CURSOR002 B
      ON A.main_code = B.sub_code
```

```
OPEN CUR

FETCH NEXT FROM CUR INTO @VALUE, @VALUE02, @SUB_CODE

WHILE @@FETCH_STATUS = 0

BEGIN
      DECLARE @CNT INT

      SELECT @CNT = COUNT(*)
      FROM TB_CURSOR002
      WHERE value = @VALUE
      AND value02 = @VALUE02
      AND sub_code = @SUB_CODE

      IF @CNT = 1
      BEGIN
              UPDATE A
              SET value = 0
              FROM TB_CURSOR002 A
              WHERE value = @VALUE AND value02 = @VALUE02 AND sub_code = @SUB_CODE
      END
FETCH NEXT FROM CUR INTO @VALUE, @VALUE02, @SUB_CODE

END

CLOSE CUR
DEALLOCATE CUR
GO
```

· **수행 비용 (Profiler)**

EventClass	TextData	Duration	Reads	CPU	RowCounts
SQL:BatchCompleted	DECLARE @SUB_CODE uniqueidentifier ...	70699	1532594	29158	240528

```
DECLARE @SUB_CODE uniqueidentifier
DECLARE @VALUE INT
DECLARE @VALUE02 INT

DECLARE CUR CURSOR FOR
                SELECT  value, value02, sub_code
                FROM TB_CURSOR001 A
                INNER JOIN TB_CURSOR002 B
                ON A.main_code = B.sub_code
                GROUP BY value, value02, sub_code
OPEN CUR

FETCH NEXT FROM CUR INTO @VALUE, @VALUE02, @SUB_CODE

WHILE @@FETCH_STATUS = 0

BEGIN
        DECLARE @CNT INT

        SELECT @CNT = COUNT(*)
        FROM TB_CURSOR002
        WHERE value = @VALUE
        AND value02 = @VALUE02
        AND sub_code = @SUB_CODE

        IF @CNT = 1
        BEGIN
                UPDATE A
                SET value = 0
                FROM TB_CURSOR002 A
                WHERE value = @VALUE AND value02 = @VALUE02 AND sub_code = @SUB_CODE
        END
FETCH NEXT FROM CUR INTO @VALUE, @VALUE02, @SUB_CODE

END
CLOSE CUR
DEALLOCATE CUR
```

```
논리적 읽기 수 1532594
SQL Server 실행 시간:
CPU 시간 = 29158ms, 경과 시간 = 70699ms
```


3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_CURSOR001	CIDX_CURSOR001	CLUSTERED	0	main_code	-
TB_CURSOR002	CIDX_CURSOR002	CLUSTERED	0	sub_code	-
TB_CURSOR002	NIDX01_CURSOR002	NONCLUSTERED	0	sub_code, value, value02	-

4. [AS-IS] 구문 설명

[TB_CURSOR001] 테이블과 [TB_CURSOR002] 테이블을 조인한 결과 집합의 [value], [value02], [sub_code] 컬럼에 대해 CURSOR로 선언하고, 각 행마다 [TB_CURSOR002] 테이블에 조건으로 검색한 결과가 1행인 경우 [value]를 0으로 업데이트하는 구문이다. [TB_CURSOR002] 테이블을 조건 검색 후 집계하는 구문과 업데이트하는 구문이 CURSOR의 행 수만큼 반복 수행되면서 I/O가 과다하게 발생한다.

5. [TO-BE] 튜닝 포인트

CURSOR를 사용하는 대신 두 테이블을 조인하면서 동일 값에 대한 행 수를 구하고, 행이 1개인 데이터를 기반으로 [TB_CURSOR002] 테이블을 업데이트하도록 구문을 변경하는 것을 고려할 수 있다. 구문을 변경하게 되면 CURSOR에 의해 반복적으로 수행되는 과정을 제거할 수 있기 때문에 전체적인 비용을 개선할 수 있다.

6. [TO-BE] 개선 방안

1) CURSOR로 사용된 구문을 조인문으로 변경

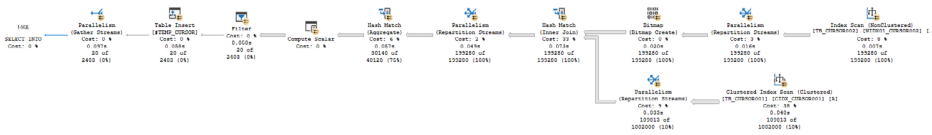
[TB_CURSOR001] 테이블과 [TB_CURSOR002] 테이블을 조인한 뒤 집계하여 행 수를 구한 데이터를 임시 테이블로 저장하고, 임시 테이블과 [TB_CURSOR002] 테이블을 조인하여 조건에 맞는 데이터를 업데이트하도록 구문을 변경한다. CURSOR에 의해 반복적으로 수행하는 과정을 제거하면서 전체적인 비용을 개선할 수 있다.

■ 변경된 SQL 구문

```
SELECT B.value, B.value02, B.sub_code
INTO #TEMP_CURSOR
FROM TB_CURSOR001 A
INNER JOIN TB_CURSOR002 B
ON A.main_code=B.sub_code
GROUP BY B.value, B.value02, B.sub_code
HAVING COUNT(*) = 1

UPDATE A SET A.value = 0
FROM TB_CURSOR002 A
INNER JOIN #TEMP_CURSOR B
ON A.value=B.value AND A.value02=B.value02 AND A.sub_code=B.sub_code
GO
```

· 실행 계획 (임시 테이블 생성문)

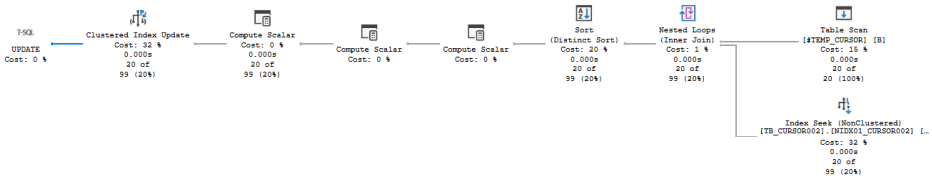
Rows	Executes	StmtText
40520	1	SELECT B.value, B.value02, B.sub_code INTO #TEMP_CURSOR FROM TB_CURSOR001 A INNER JOIN TB_CURSOR002 B ON A.main_code=B.sub_code GROUP BY B.value, B.value02, B.sub_code HAVING COUNT(*) = 1
40520	1	¦--Parallelism(Gather Streams)
40520	12	¦--Table Insert(OBJECT:([#TEMP_CURSOR]), SET:([#TEMP_CURSOR].[value] = [maxgauge].[dbo].[TB_CURSOR002].[value] as [B].[value],[#TEMP_CURSOR].[value02] = [maxgauge].[dbo].[TB_CURSOR002].[value02] as [B].[value02],[#TEMP_CURSOR].[sub_code] = [maxgauge].[dbo].[TB_CURSOR002].[sub_code] as [B].[sub_code]))
40520	12	¦--Filter(WHERE:([Expr1007]=(1)))
0	0	¦--Compute Scalar(DEFINE:([Expr1007]=CONVERT_IMPLICIT(int,[Expr1011],0)))
100000	12	¦--Hash Match(Aggregate, HASH:([B].[value], [B].[value02], [B].[sub_code]), RESIDU-AL:([maxgauge].[dbo].[TB_CURSOR002].[value] as [B].[value] = [maxgauge].[dbo].[TB_CURSOR002].[value] as [B].[value] AND [maxgauge].[dbo].[TB_CURSOR002].[value02] as [B].[value02] = [maxgauge].[dbo].[TB_CURSOR002].[value02] as [B].[value02] AND [maxgauge].[dbo].[TB_CURSOR002].[sub_code] as [B].[sub_code] = [maxgauge].[dbo].[TB_CURSOR002].[sub_code] as [B].[sub_code]) DE-FINE:([Expr1011]=COUNT(*)))
199280	12	¦--Parallelism(Repartition Streams, Hash Partitioning, PARTITION COLUMNS:([B].[value], [B].[value02], [B].[sub_code]))
199280	12	¦--Hash Match(Inner Join, HASH:([B].[sub_code])=([A].[main_code]), RESIDU-AL:([maxgauge].[dbo].[TB_CURSOR002].[sub_code] as [B].[sub_code]=[maxgauge].[dbo].[TB_CURSOR001].[main_code] as [A].[main_code]))
199280	12	¦--Bitmap(HASH:([B].[sub_code]), DEFINE:([Bitmap1010]))
199280	12	¦ ¦--Parallelism(Repartition Streams, Hash Partitioning, PARTITION COLUMNS:([B].[sub_code]))
199280	12	¦ ¦--Index Scan(OBJECT:([maxgauge].[dbo].[TB_CURSOR002].[NIDX01_CURSOR002] AS [B]))
102592	12	¦--Parallelism(Repartition Streams, Hash Partitioning, PARTITION COLUMNS:([A].[main_code]))
102592	12	¦--Clustered Index Scan(OBJECT:([maxgauge].[dbo].[TB_CURSOR001].[CIDX_CURSOR001] AS [A]), WHERE:(PROBE([Bitmap1010],[maxgauge].[dbo].[TB_CURSOR001].[main_code] as [A].[main_code]))



테이블 'TB_CURSOR002'. 검색 수 13, 논리적 읽기 수 1117
테이블 'TB_CURSOR001'. 검색 수 13, 논리적 읽기 수 5206
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
CPU 시간 = 424ms, 경과 시간 = 143ms

· 실행 계획 (Update문)

Rows	Executes	StmtText
40520	1	UPDATE A SET A.value = 0 FROM TB_CURSOR002 A INNER JOIN #TEMP_CURSOR B ON A.value=B.value AND A.value02=B.value02 AND A.sub_code=B.sub_code
40520	1	¦--Clustered Index Update(OBJECT:([maxgauge].[dbo].[TB_CURSOR002].[CIDX_CURSOR002] AS [A]), OB-JECT:([maxgauge].[dbo].[TB_CURSOR002].[NIDX01_CURSOR002] AS [A]), SET:([maxgauge].[dbo].[TB_CURSOR002].[value] as [A].[value] = [Expr1004]) WITH UNORDERED PREFETCH)
40520	1	¦--Compute Scalar(DEFINE:([Expr1008]=[Expr1008]))
40520	1	¦--Parallelism(Gather Streams)
40520	12	¦--Table Spool
0	0	¦--Compute Scalar(DEFINE:([Expr1008]=CASE WHEN [Expr1007] THEN (0) ELSE (1) END))
0	0	¦--Compute Scalar(DEFINE:([Expr1004]=(0), [Expr1007]=CASE WHEN [maxgauge].[dbo].[TB_CURSOR002].[value] as [A].[value] = (0) THEN (1) ELSE (0) END))
40520	12	¦--Hash Match(Right Semi Join, HASH:([B].[value], [B].[value02], [B].[sub_code])=([A].[value], [A].[value02], [A].[sub_code]), RESIDUAL:([maxgauge].[dbo].[TB_CURSOR002].[value] as [A].[value]=#TEMP_CURSOR.[value] as [B].[value] AND [maxgauge].[dbo].[TB_CURSOR002].[value02] as [A].[value02]=#TEMP_CURSOR.[value02] as [B].[value02] AND [maxgauge].[dbo].[TB_CURSOR002].[sub_code] as [A].[sub_code]=#TEMP_CURSOR.[sub_code] as [B].[sub_code]))
40520	12	¦--Bitmap(HASH:([B].[value], [B].[value02], [B].[sub_code]), DEFINE:([Bitmap1009]))
40520	12	¦ ¦--Parallelism(Repartition Streams, Hash Partitioning, PARTITION COLUMNS:([B].[value], [B].[value02], [B].[sub_code]))
40520	12	¦ ¦--Table Scan(OBJECT:([tempdb].[dbo].[#TEMP_CURSOR] AS [B]))
40758	12	¦--Parallelism(Repartition Streams, Hash Partitioning, PARTITION COLUMNS:([A].[value], [A].[value02], [A].[sub_code]))
40758	12	¦--Index Scan(OBJECT:([maxgauge].[dbo].[TB_CURSOR002].[NIDX01_CURSOR002] AS [A]), WHERE:(PROBE([Bitmap1009],[maxgauge].[dbo].[TB_CUSOR002].[value] as [A].[value],[maxgauge].[dbo].[TB_CURSOR002].[value02] as [A].[value02],[maxgauge].[dbo].[TB_CURSOR002].[sub_code] as [A].[sub_code])) ORDERED FORWARD)




```
테이블 '#TEMP_CURSOR_______00000000002B'. 검색 횟수 13, 논리적 읽기 214
테이블 'TB_CURSOR002'. 검색 횟수 13, 논리적 읽기 364704
테이블 'Worktable'. 검색 횟수 12, 논리적 읽기 82504
SQL Server 실행 시간:
CPU 시간 = 718ms, 경과 시간 = 797ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time(ms)	Elapsed Time(ms)	Logical Reads
AS-IS	29,158	70,699	1,532,594
TO-BE	1,142	940	453,745

GitHub Copilot 채팅 결과

Claude Opus 4.6

34. CURSOR 성능 이슈 - 2

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
DECLARE @SUB_CODE uniqueidentifier
DECLARE @VALUE INT
DECLARE @VALUE02 INT

DECLARE CUR CURSOR FOR
               SELECT  value, value02, sub_code
               FROM TB_CURSOR001 A
               INNER JOIN TB_CURSOR002 B
               ON A.main_code = B.sub_code
               GROUP BY value, value02, sub_code
OPEN CUR

FETCH NEXT FROM CUR INTO @VALUE, @VALUE02, @SUB_CODE

WHILE @@FETCH_STATUS = 0

BEGIN
      DECLARE @CNT INT

      SELECT @CNT = COUNT(*)
      FROM TB_CURSOR002
      WHERE value = @VALUE
      AND value02 = @VALUE02
      AND sub_code = @SUB_CODE

      IF @CNT = 1
      BEGIN
               UPDATE A
               SET value = 0
               FROM TB_CURSOR002 A
               WHERE value = @VALUE AND value02 = @VALUE02 AND sub_code = @SUB_CODE
      END
FETCH NEXT FROM CUR INTO @VALUE, @VALUE02, @SUB_CODE

END
CLOSE CUR
DEALLOCATE CUR;
```

2. 분석 및 주요 최적화

1) 현재 테이블 현황

테이블	행 수	클러스터형 인덱스	비클러스터형 인덱스
dbo.TB_CURSOR001	1,002,001	main_code	-
dbo.TB_CURSOR002	199,280	sub_code	value, value02

2) 식별된 성능 문제점

	안티패턴	설명	영향도
1	커서 사용 (Cursor Overuse)	100만+ 행을 행 단위로 순회하며 각 행마다 COUNT + UPDATE 쿼리 실행. 최대 2×N번의 쿼리 실행 발생	● 심각
2	반복적 COUNT 쿼리	커서 루프 내에서 매번 COUNT() 조회 후 조건 분기 → 단일 HAVING COUNT() = 1로 대체 가능	● 심각
3	암시적 데이터 타입 변환 (CONVERT_IMPLICIT)	@VALUE, @VALUE02는 INT로 선언되었으나, 실제 컬럼 value, value02는 bigint → 암시적 변환 발생	● 중간
4	루프 내 변수 선언	DECLARE @CNT INT가 WHILE 루프 내부에 위치 (기능상 문제는 없으나 가독성 저하)	● 경미

이 스크립트의 핵심 로직은:

TB_CURSOR001과 TB_CURSOR002를 조인한 결과에서, (value, value02, sub_code) 그룹의 건수가 정확히 1건인 행의 value를 0으로 UPDATE

*이것은 단일 집합 기반 UPDATE 문으로 완전히 대체 가능합니다.

3. 개선안

```sql
-- [TO-BE] 커서를 집합 기반 UPDATE로 변환
-- (value, value02, sub_code) 그룹 기준 1건만 존재하는 행의 value를 0으로 갱신

UPDATE B
SET B.value = 0
FROM dbo.TB_CURSOR002 B
INNER JOIN dbo.TB_CURSOR001 A
ON A.main_code = B.sub_code
WHERE EXISTS (SELECT 1
              FROM dbo.TB_CURSOR001 A2
              INNER JOIN dbo.TB_CURSOR002 B2
              ON A2.main_code = B2.sub_code
              WHERE B2.value = B.value AND B2.value02 = B.value02
              AND B2.sub_code = B.sub_code
              GROUP BY B2.value, B2.value02, B2.sub_code
              HAVING COUNT(*) = 1);
```

AS-IS vs TO-BE 비교

항목	AS-IS (커서)	TO-BE (집합 기반)
처리 방식	행 단위 순차 처리	집합 기반 일괄 처리
쿼리 실행 횟수	최대 ~2×N회 (N=그룹 수)	1회
Lock 점유	반복적 row lock 획득/해제	단일 트랜잭션
데이터 타입 불일치	INT vs bigint 암시적 변환	해소 (변수 미사용)
유지보수성	40줄+	~15줄

변경 요약

· 커서 → EXISTS + HAVING COUNT(*) = 1 서브쿼리로 대체하여 행 단위 루프 제거

· 암시적 타입 변환 제거 — INT 변수 선언이 불필요해짐

· 불필요한 변수(@CNT, @VALUE, @VALUE02, @SUB_CODE) 제거

35 CURSOR 성능 이슈 - 3

CURSOR로 인해 테이블을 반복적으로 읽어내면서 많은 I/O 비용이 사용된 사례이다.

1. 사전 구성 스크립트

```
SELECT rownum AS seq
      , NEWID() AS main_code
      , DATEADD(MINUTE, rownum, '2020-01-01 00:00:00.000') AS date
INTO TB_CURSOR003
FROM (SELECT ROW_NUMBER() OVER (ORDER BY a.number) AS rownum
             , a.number
      FROM master..spt_values a JOIN master..spt_values b
      ON a.type = 'P' AND b.type = 'P' AND a.number <=1000 AND b.number <= 1000
) a

SELECT TOP 100000 main_code AS sub_code
      , CASE WHEN seq % 5 = 1 THEN seq
        WHEN seq % 5 = 2 THEN seq
        ELSE seq % 100 END AS value
      , CASE WHEN seq % 5 = 2 THEN seq
        ELSE seq % 200 END AS value02
INTO TB_CURSOR004
FROM TB_CURSOR003
ORDER BY seq

INSERT INTO TB_CURSOR004 SELECT * FROM TB_CURSOR004 WHERE value02 > 100
INSERT INTO TB_CURSOR004 SELECT * FROM TB_CURSOR004 WHERE value02 > 500

CREATE CLUSTERED INDEX CIDX_CURSOR003 ON TB_CURSOR003(main_code)
CREATE CLUSTERED INDEX CIDX_CURSOR004 ON TB_CURSOR004(sub_code)
CREATE INDEX NIDX01_CURSOR004 ON TB_CURSOR004 (sub_code,value,value02)
GO
```

2. [AS-IS] SQL 구문 및 실행 계획

SQL 구문

```
DECLARE @SUB_CODE uniqueidentifier
DECLARE @VALUE INT
DECLARE @VALUE02 INT

DECLARE CUR CURSOR FOR
      SELECT DISTINCT value, value02, sub_code
      FROM TB_CURSOR003 A INNER JOIN TB_CURSOR004 B
      ON A.main_code=B.sub_code

OPEN CUR
FETCH NEXT FROM CUR INTO @VALUE, @VALUE02, @SUB_CODE
WHILE @@FETCH_STATUS = 0
BEGIN
      DECLARE @CNT INT

      SELECT @CNT = COUNT(*)
      FROM TB_CURSOR004
      WHERE value = @VALUE
      AND value02 = @VALUE02
      AND sub_code = @SUB_CODE

      IF @CNT = 1
      BEGIN
              UPDATE A
              SET value = 0
              FROM TB_CURSOR004 A
              WHERE value = @VALUE
              AND value02 = @VALUE02
              AND sub_code = @SUB_CODE
      END

      ELSE IF @CNT = 2
      BEGIN
              UPDATE A
              SET value = -1
              FROM TB_CURSOR004 A
              WHERE value = @VALUE
              AND value02 = @VALUE02
              AND sub_code = @SUB_CODE
      END

      ELSE IF @CNT = 4
      BEGIN
              UPDATE A
              SET value = -2
              FROM TB_CURSOR004 A
              WHERE value = @VALUE
              AND value02 = @VALUE02
              AND sub_code = @SUB_CODE
      END
```

```
FETCH NEXT FROM CUR INTO @VALUE, @VALUE02, @SUB_CODE

END
CLOSE CUR
DEALLOCATE CUR
GO
```

· 수행 비용 (Profiler)

EventClass	TextData	Duration	Reads	CPU	RowCounts
SQL:BatchCompleted	DECLARE @SUB_CODE uniqueidentifier DECLARE @VALUE INT D...	134142	3669416	50116	399312

```
DECLARE @SUB_CODE uniqueidentifier
DECLARE @VALUE INT
DECLARE @VALUE02 INT

DECLARE CUR CURSOR FOR
                SELECT value, value02, B.sub_code
                FROM TB_CURSOR003 A INNER JOIN TB_CURSOR004 B
                ON A.main_code = B.sub_code
                GROUP BY value, value02, sub_code
OPEN CUR
FETCH NEXT FROM CUR INTO @VALUE, @VALUE02, @SUB_CODE
WHILE @@FETCH_STATUS = 0
BEGIN
        DECLARE @CNT INT

        SELECT @CNT = COUNT(*)
        FROM TB_CURSOR004
        WHERE value = @VALUE
        AND value02 = @VALUE02
        AND sub_code = @SUB_CODE

        IF @CNT = 1
        BEGIN
                        UPDATE A
                        SET value = 0
                        FROM TB_CURSOR004 A
                        WHERE value = @VALUE
                        AND value02 = @VALUE02
                        AND sub_code = @SUB_CODE
        END

        ELSE IF @CNT = 2
        BEGIN
                        UPDATE A
                        SET value = -1
                        FROM TB_CURSOR004 A
                        WHERE value = @VALUE
                        AND value02 = @VALUE02
                        AND sub_code = @SUB_CODE
        END

        ELSE IF @CNT = 4
        BEGIN
                        UPDATE A
                        SET value = -2
                        FROM TB_CURSOR004 A
                        WHERE value = @VALUE
                        AND value02 = @VALUE02
                        AND sub_code = @SUB_CODE
        END
FETCH NEXT FROM CUR INTO @VALUE, @VALUE02, @SUB_CODE

END
CLOSE CUR
DEALLOCATE CUR
```

논리적 읽기 수 3669416
SQL Server 실행 시간:
CPU 시간 = 50116ms, 경과 시간 = 134142ms

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_CURSOR003	CIDX_CURSOR003	CLUSTERED	0	main_code	-
TB_CURSOR004	CIDX_CURSOR004	CLUSTERED	0	sub_code	-
TB_CURSOR004	NIDX01_CURSOR004	NONCLUSTERED	0	sub_code,value,value02	-

4. [AS-IS] 구문 설명

[TB_CURSOR003] 테이블과 [TB_CURSOR004] 테이블을 조인한 결과인 [value], [value02], [sub_code] 컬럼에 대해 CURSOR로 선언하고, 각 행마다 [TB_CURSOR004] 테이블에 조건으로 검색한 결과가 1, 2, 4행인 경우 [value] 컬럼을 각각 0, -1, -2로 업데이트하는 구문이다. [TB_CURSOR004] 테이블을 조건 검색 후 집계하는 구문과 업데이트하는 구문이 CURSOR의 행 수만큼 반복 수행되면서 I/O가 과다하게 발생하게 된다.

5. [TO-BE] 튜닝 포인트

CURSOR를 사용하는 대신 두 테이블을 조인하면서 동일 값에 대한 행 수를 구하고, 행이 1, 2, 4개인 데이터를 기반으로 [TB_CURSOR004] 테이블을 업데이트하도록 구문을 변경하는 것을 고려할 수 있다. 구문을 변경하게 되면 CURSOR에 의해 반복적으로 수행되는 과정을 제거할 수 있기 때문에 전체적인 비용을 개선할 수 있다.

6. [TO-BE] 개선 방안

1) CURSOR로 사용된 구문을 조인문으로 변경

[TB_CURSOR003] 테이블과 [TB_CURSOR004] 테이블 조인한 뒤 집계하여 행 수를 구한 데이터를 임시 테이블로 저장하고, 임시 테이블과 [TB_CURSOR004] 테이블을 조인하여 건수가 1, 2, 4인 경우에 대해서 각각 업데이트하도록 구문을 변경한다. CURSOR에 의해 반복적으로 수행하는 과정을 제거하면서 전체적인 비용을 개선할 수 있다.

■ 변경된 SQL 구문

```
SELECT B.value, B.value02, B.sub_code, COUNT(*) AS CNT
INTO #TEMP_CURSOR2
FROM TB_CURSOR003 A INNER JOIN TB_CURSOR004 B
ON A.main_code=B.sub_code
GROUP BY B.value, B.value02, B.sub_code

UPDATE A
SET A.value = 0
FROM TB_CURSOR004 A
INNER JOIN #TEMP_CURSOR2 B
ON A.value = B.value
AND A.value02 = B.value02
AND A.sub_code = B.sub_code
AND B.CNT = 1

UPDATE A
SET A.value = -1
FROM TB_CURSOR004 A
INNER JOIN #TEMP_CURSOR2 B
ON A.value = B.value
```

```
AND A.value02 = B.value02
AND A.sub_code = B.sub_code
AND B.CNT = 2

UPDATE A
SET A.value = -2
FROM TB_CURSOR004 A
INNER JOIN #TEMP_CURSOR2 B
ON A.value = B.value
AND A.value02 = B.value02
AND A.sub_code = B.sub_code
AND B.CNT = 4
GO
```

· 수행 비용 (Profiler)

EventClass	TextData	Duration	Reads	CPU	RowCounts
SQL:BatchCompleted	SELECT B.value, B.value02, B.sub...	3828	2240951	5375	299296

```
SELECT B.value, B.value02, B.sub_code, COUNT(*) AS CNT
INTO #TEMP_CURSOR2
FROM TB_CURSOR003 A INNER JOIN TB_CURSOR004 B
ON A.main_code=B.sub_code
GROUP BY B.value, B.value02, B.sub_code

UPDATE A
SET A.value = 0
FROM TB_CURSOR004 A
INNER JOIN #TEMP_CURSOR2 B
ON A.value = B.value
AND A.value02 = B.value02
AND A.sub_code = B.sub_code
AND B.CNT = 1

UPDATE A
SET A.value = -1
FROM TB_CURSOR004 A
INNER JOIN #TEMP_CURSOR2 B
ON A.value = B.value
AND A.value02 = B.value02
AND A.sub_code = B.sub_code
AND B.CNT = 2

UPDATE A
SET A.value = -2
FROM TB_CURSOR004 A
INNER JOIN #TEMP_CURSOR2 B
ON A.value = B.value
AND A.value02 = B.value02
AND A.sub_code = B.sub_code
AND B.CNT = 4
```

```
논리적 읽기 수 2240951
SQL Server 실행 시간:
CPU 시간 = 5375ms, 경과 시간 = 3828ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time(ms)	Elapsed Time(ms)	Logical Reads
AS-IS	50,116	134,142	3,669,416
TO-BE	5,375	3,828	2,240,951

GitHub Copilot 채팅 결과

35. CURSOR 성능 이슈 - 3

Claude Opus 4.6

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
DECLARE @SUB_CODE uniqueidentifier
DECLARE @VALUE INT
DECLARE @VALUE02 INT

DECLARE CUR CURSOR FOR
              SELECT value, value02, B.sub_code
              FROM TB_CURSOR003 A INNER JOIN TB_CURSOR004 B
              ON A.main_code = B.sub_code
              GROUP BY value, value02, sub_code
OPEN CUR
FETCH NEXT FROM CUR INTO @VALUE, @VALUE02, @SUB_CODE
WHILE @@FETCH_STATUS = 0
BEGIN
      DECLARE @CNT INT

      SELECT @CNT = COUNT(*)
      FROM TB_CURSOR004
      WHERE value = @VALUE
      AND value02 = @VALUE02
      AND sub_code = @SUB_CODE

      IF @CNT = 1
      BEGIN
                     UPDATE A
                     SET value = 0
                     FROM TB_CURSOR004 A
                     WHERE value = @VALUE
                     AND value02 = @VALUE02
                     AND sub_code = @SUB_CODE
      END

      ELSE IF @CNT = 2
      BEGIN
                     UPDATE A
                     SET value = -1
                     FROM TB_CURSOR004 A
                     WHERE value = @VALUE
                     AND value02 = @VALUE02
                     AND sub_code = @SUB_CODE
      END
```

```
        ELSE IF @CNT = 4
        BEGIN
                        UPDATE A
                        SET value = -2
                        FROM TB_CURSOR004 A
                        WHERE value = @VALUE
                        AND value02 = @VALUE02
                        AND sub_code = @SUB_CODE
        END
FETCH NEXT FROM CUR INTO @VALUE, @VALUE02, @SUB_CODE

END
CLOSE CUR
DEALLOCATE CUR;
```

2. 분석 및 주요 최적화

1) 테이블 스키마 요약

테이블	컬럼	Clustered Index	Nonclustered Index
dbo.TB_CURSOR003	seq (bigint), main_code (uniqueidentifier), date (datetime)	main_code	-
dbo.TB_CURSOR004	sub_code (uniqueidentifier), value (bigint), value02 (bigint)	sub_code	value, value02

2) 식별된 성능 문제점

#	문제 유형	설명	영향도
1	커서 남용	전체 로직이 행 단위(Row-by-Row) 처리로, 반복마다 COUNT + UPDATE 총 2회 쿼리 실행	● 높음
2	암시적 형변환 (CONVERT_IMPLICIT)	@VALUE, @VALUE02가 INT로 선언되었으나 실제 컬럼은 bigint → 암시적 변환 발생, 인덱스 사용 저해 가능	● 중간
3	반복적 테이블 스캔	커서 행마다 TB_CURSOR004를 COUNT 조회 후 동일 조건으로 다시 UPDATE → 동일 I/O 2배 발생	● 높음
4	불필요한 변수	@SUB_CODE, @VALUE, @VALUE02, @CNT는 set-based 변환 시 불필요	● 낮음

3. 개선안

```sql
-- 최적화된 스크립트
-- (TO-BE) 커서를 제거하고 단일 set-based UPDATE로 변환했습니다.

WITH CountCTE AS (
    SELECT B.sub_code, B.value, B.value02,
           COUNT(*) AS cnt
    FROM dbo.TB_CURSOR004 B
    WHERE EXISTS (
        SELECT 1 FROM dbo.TB_CURSOR003 A
        WHERE A.main_code = B.sub_code
    )
    GROUP BY B.sub_code, B.value, B.value02
    HAVING COUNT(*) IN (1, 2, 4)
)
UPDATE T
SET value = CASE
    WHEN C.cnt = 1 THEN 0
    WHEN C.cnt = 2 THEN -1
    WHEN C.cnt = 4 THEN -2
END
FROM dbo.TB_CURSOR004 T
INNER JOIN CountCTE C
ON T.sub_code = C.sub_code AND T.value = C.value AND T.value02 = C.value02;
```

■ 변경 사항 요약

항목	AS-IS	TO-BE
처리 방식	커서 (Row-by-Row)	CTE + set-based UPDATE (단일 구문)
TB_CURSOR004 접근 횟수	행 수 × 2회 (COUNT + UPDATE)	2회 (CTE 1회 + UPDATE 1회)
데이터 형변환	INT ↔ bigint 암시적 변환	발생하지 않음 (컬럼 직접 비교)
HAVING 필터	IF 분기로 처리	HAVING COUNT(*) IN (1, 2, 4)로 불필요 행 사전 제거
변수/커서	4개 변수 + 커서	없음

※ 커서의 N회 반복 쿼리가 단일 UPDATE 구문으로 대체되어, 데이터 규모가 커질수록 성능 차이가 크게 벌어집니다.

Section 06

집계 및 함수 처리 최적화

집계 함수와 사용자 정의 함수는 쿼리 성능에 큰 영향을 미친다. 이 섹션에서는 집계 처리 방식과 함수 사용 시의 성능 이슈를 분석하고 개선 전략을 다룬다.

36 COUNT 함수의 OVER 절 성능 이슈 - 1

COUNT 함수의 OVER 절을 사용했을 때 Spool로 인한 임시 테이블 비용이 높아진 사례이다.

1. 사전 구성 스크립트

```
SELECT NEWID() AS code, CAST(rownum%12 AS INT) AS value
INTO TB_OVER
FROM (SELECT ROW_NUMBER() OVER (ORDER BY a.number) AS rownum, a.number
       FROM master..spt_values a
       JOIN master..spt_values b
       ON a.type = 'P' AND b.type = 'P' AND a.number <= 1000 AND b.number <= 1000
) A

CREATE CLUSTERED INDEX CIDX_OVER ON TB_OVER(code)
CREATE INDEX NIDX01_OVER ON TB_OVER(value)
GO
```

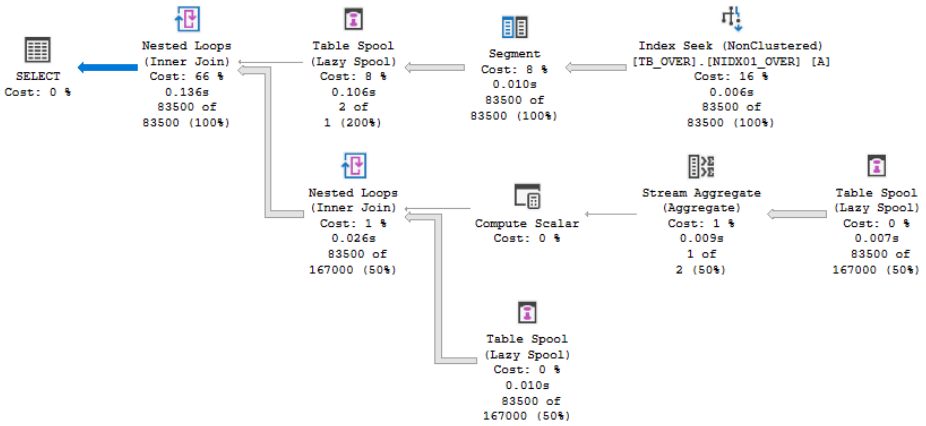
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT A.*, COUNT(1) OVER() AS 'totalcount'
FROM TB_OVER A
WHERE A.value = 11
GO
```

· **실행 계획**

Rows	Executes	StmtText
83500	1	SELECT A.*, COUNT(1) OVER() AS 'totalcount' FROM TB_OVER A WHERE A.value = 11
83500	1	┦--Nested Loops(INNER JOIN)
2	1	┦--Table Spool
83500	1	┦ ┦--Segment
83500	1	┦ ┦--Index Seek(OBJECT:([Tuning].[dbo].[TB_OVER].[NIDX01_OVER] AS [A]), SEEK:([A].[value]=(11)) Ordered Forward)
83500	2	┦--Nested Loops(INNER JOIN, WHERE:((1)))
0	0	┦--Compute Scalar(DEFINE:([Expr1002]=CONVERT_IMPLICIT(int,[Expr1004],0)))
1	2	┦ ┦--Stream Aggregate(DEFINE:([Expr1004]=Count(*)))
83500	2	┦ ┦--Table Spool
83500	1	┦--Table Spool




```
테이블 'Worktable'. 스캔 수 3, 논리적 읽기 169685
테이블 'TB_OVER'. 스캔 수 1, 논리적 읽기 274
SQL Server 실행 시간:
CPU 시간 = 281ms, 경과 시간 = 1285ms
```

3. [AS-IS] 통계 정보

· **인덱스 정보**

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_OVER	CIDX_OVER	CLUSTERED	0	code	-
TB_OVER	NIDX01_OVER	NONCLUSTERED	0	value	-

4. [AS-IS] 구문 설명

[TB_OVER] 테이블의 [value] 컬럼 값이 11인 모든 행에 결과 집합 전체 행 수를 동시에 출력하기 위해 OVER 절에 COUNT 함수를 사용한 구문이다. 결과 집합 모든 행에 전체 행 수를 조인하기 위해 Spool이 사용되는데, 이는 조건절에 해당하는 결과 집합을 임시 데이터로 저장하여 집합에 대한 전체 행 수를 집계할 때 임시 데이터를 재사용하기 위함이다. Spool은 내부적으로 임시 테이블을 만들어 저장하기 때문에 Spool을 생성하고 재사용하는 과정에서 임시 테이블에 대한 I/O 비용이 높게 발생된 것이다.

5. [TO-BE] 튜닝 포인트

임시 테이블에 중간 결과 집합을 저장하고 재사용하는 과정에서 I/O가 높게 사용되었다. 내부적으로 Spool을 생성하지 않도록 전체 행 수를 구하는 구문을 별도로 작성하여 CROSS JOIN을 통해 결과 집합 모든 행에 전체 행 수가 조인되도록 구문을 변경한다.

6. [TO-BE] 개선 방안

1) CROSS JOIN으로 구문을 변경

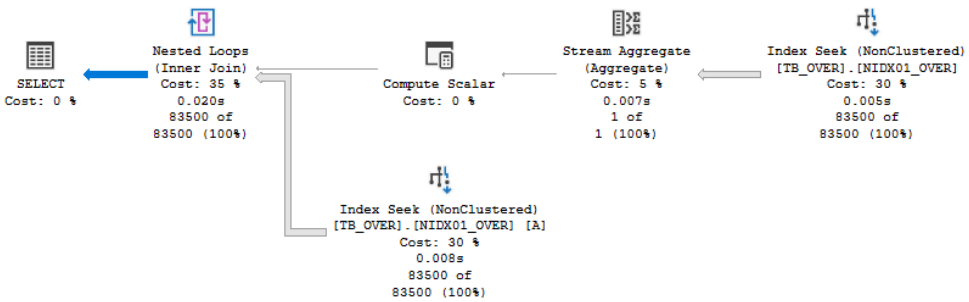
[TB_OVER] 테이블의 [value] = 11인 결과 행 수를 집계하는 구문을 별도로 작성하여 CROSS JOIN 하면 데이터를 재사용하는 과정이 제거되기 때문에 Spool 연산에 의한 I/O를 개선할 수 있다.

■ 변경된 SQL 구문

```
SELECT A.*, totalcount
FROM TB_OVER A
CROSS JOIN (SELECT COUNT(1) AS 'totalcount'
            FROM TB_OVER
            WHERE value = 11) B
WHERE A.value = 11
GO
```

· 실행 계획

Rows	Executes	StmtText
83500	1	SELECT A.*, totalcount FROM TB_OVER A CROSS JOIN (SELECT COUNT(1) AS 'totalcount' FROM TB_OVER WHERE value = 11) B WHERE A.value = 11
83500	1	¦--Nested Loops(INNER JOIN)
0	0	¦--Compute Scalar(DEFINE:([Expr1005]=CONVERT_IMPLICIT(int,[Expr1008],0)))
1	1	¦ ¦--Stream Aggregate(DEFINE:([Expr1008]=Count(*)))
83500	1	¦ ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_OVER].[NIDX01_OVER]), SEEK:([Tuning].[dbo].[TB_OVER].[value]=(11)) Ordered Forward)
83500	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_OVER].[NIDX01_OVER] AS [A]), SEEK:([A].[value]=(11)) Ordered Forward)




```
테이블 'TB_OVER'. 스캔 수 2, 논리적 읽기 548
SQL Server 실행 시간:
CPU 시간 = 16ms, 경과 시간 = 1046ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	281	1,285	169,959
TO-BE	16	1,046	548

AI

GitHub Copilot 채팅 결과

Claude Opus 4.6

36. COUNT 함수의 OVER 절 성능 이슈 - 1

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT A.*, COUNT(1) OVER() AS 'totalcount'
FROM TB_OVER A
WHERE A.value = 11;
```

2. 분석 및 주요 최적화

1) 테이블 현황

항목	값
테이블	dbo.TB_OVER
컬럼	code (uniqueidentifier), value (int)
전체 행 수	1,002,001
value = 11 행 수	83,500
클러스터드 인덱스	CIDX_OVER (code)
논클러스터드 인덱스	NIDX01_OVER (value)

2) 비효율 원인 분석

COUNT(1) OVER() — 핵심 비효율 포인트 : 이 부분이 가장 큰 성능 병목입니다.

문제	설명
전체 윈도우 스캔	OVER()에 PARTITION BY가 없으므로, WHERE 필터 후 결과 집합 전체(83,500행)를 대상으로 카운트를 수행합니다.
행마다 동일 값 반복	83,500행 **모든 행에 동일한 totalcount 값(83,500)**이 반복 출력됩니다. 이는 불필요한 I/O와 네트워크 전송을 유발합니다.
실행 계획상 Window Spool	SQL Server는 내부적으로 Window Spool (또는 Table Spool) 연산자를 추가하여 중간 결과를 임시 저장하므로 tempdb 사용이 증가할 수 있습니다.

3. 개선안

```sql
-- 목표: totalcount를 얻으면서도 Window Spool 비용을 제거
-- 방안 1. 서브쿼리로 totalcount 분리 (권장)

SELECT A.code,
       A.value,
       B.totalcount
FROM dbo.TB_OVER A
CROSS JOIN (
    SELECT COUNT(1) AS totalcount
    FROM dbo.TB_OVER
    WHERE value = 11
) B
WHERE A.value = 11;


-- 방안 2. 변수 사용 (결과를 애플리케이션에서 처리 가능한 경우)

DECLARE @totalcount INT;

SELECT @totalcount = COUNT(1)
FROM dbo.TB_OVER
WHERE value = 11;

SELECT A.code,
       A.value,
       @totalcount AS totalcount
FROM dbo.TB_OVER A
WHERE A.value = 11;
```

개선 요약

항목	AS-IS	TO-BE
SELECT *	사용 ✗	컬럼 명시 ✓
totalcount 계산	COUNT(1) OVER() — 매 행마다 Window Spool 발생	CROSS JOIN 서브쿼리 — 1회만 계산
Window Spool	발생 (tempdb 부하)	제거됨
결과 동일성	동일한 결과 반환	동일한 결과 반환 ✓

*COUNT(1) OVER()는 편리하지만, 대량 데이터에서는 Window Spool 비용이 크므로 CROSS JOIN 서브쿼리나 변수 방식으로 분리하는 것이 성능상 유리합니다.

37 COUNT 함수의 OVER 절 성능 이슈 - 2

COUNT 함수의 OVER 절을 사용했을 때 Spool로 인한 임시 테이블 비용이 높아진 사례이다.

1. 사전 구성 스크립트

```
CREATE TABLE TB_Std02 (std_no INT NOT NULL
      ,class_cd VARCHAR(2) NOT NULL
      ,code NVARCHAR(36) NOT NULL
      ,CONSTRAINT PK_Std02 PRIMARY KEY NONCLUSTERED(std_no))
CREATE TABLE TB_Point02 (std_no INT NOT NULL
      ,sbjt_cd varchar(2) NOT NULL
      ,point INT NOT NULL)
INSERT INTO TB_Std02
SELECT std_no
      , CASE WHEN std_no <= 200 THEN '1'
        WHEN std_no <= 500 THEN '2'
        WHEN std_no <= 650 THEN '3'
        ELSE '4' END class_cd
     , CONVERT(NVARCHAR(36),NEWID()) code
FROM (SELECT ROW_NUMBER() OVER ( ORDER BY a.number) std_no
      FROM master..spt_values a
      WHERE a.type = 'P'
      AND a.number BETWEEN 1 AND 1000) b

INSERT INTO TB_Point02
SELECT a.std_no, CHAR(b.number + 64) sbjt_cd
      , CASE WHEN a.std_no = 1 THEN 100
      ELSE CONVERT(INT, RIGHT(std_no * ASCII(CONVERT(NVARCHAR(36), NEWID())),2)) END
point
FROM TB_Std02 a, master..spt_values b
WHERE b.type = 'P'
AND b.number BETWEEN 1 AND 4
GO
```

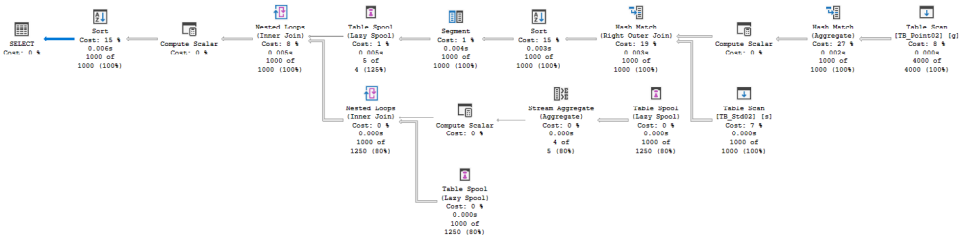
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT s.std_no, s.class_cd
     , ISNULL(AVG(g.point),0) avg_point
     , COUNT(1) OVER (PARTITION BY class_cd) total_cnt
FROM TB_Std02 s LEFT OUTER JOIN TB_Point02 g
ON s.std_no = g.std_no
GROUP BY s.std_no,s.class_cd
ORDER BY s.class_cd, avg_point DESC, s.std_no
GO
```

· 실행 계획

Rows	Executes	StmtText
1000	1	SELECT s.std_no, s.class_cd, ISNULL(AVG(g.point),0) avg_point, COUNT(1) OVER (PARTITION BY class_cd) total_cnt FROM TB_Std02 s LEFT OUTER JOIN TB_Point02 g ON s.std_no = g.std_no GROUP BY s.std_no,s.class_cd ORDER BY s.class_cd, avg_point DESC, s.std_no
1000	1	¦--Sort(ORDER BY:([s].[class_cd] ASC, [Expr1006] DESC, [s].[std_no] ASC))
0	0	¦--Compute Scalar(DEFINE:([Expr1006]=isnull([Expr1004],(0))))
1000	1	¦--Nested Loops(INNER JOIN)
5	1	¦--Table Spool
1000	1	¦ ¦--Segment
1000	1	¦ ¦--Sort(ORDER BY:([s].[class_cd] ASC))
1000	1	¦ ¦--Hash Match(Right Outer Join, HASH:([g].[std_no])=([s].[std_no]))
0	0	¦ ¦--Compute Scalar(DEFINE:([Expr1004]=CASE WHEN [Expr1011]=(0) THEN NULL ELSE [Expr1012]/CONVERT_IMPLICIT(int,[Expr1011],0) END))
1000	1	¦ ¦ ¦--Hash Match(Aggregate, HASH:([g].[std_no]) DEFINE:([Expr1011]=COUNT(*), [Expr1012]=SUM([Tuning].[dbo].[TB_Point02].[point] as [g].[point]))
4000	1	¦ ¦ ¦ ¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Point02] AS [g]))
1000	1	¦ ¦ ¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Std02] AS [s]))
1000	5	¦--Nested Loops(INNER JOIN, WHERE:((1)))
0	0	¦--Compute Scalar(DEFINE:([Expr1005]=CONVERT_IMPLICIT(int,[Expr1014],0)))
4	5	¦ ¦--Stream Aggregate(DEFINE:([Expr1014]=Count(*)))
1000	5	¦ ¦--Table Spool
1000	4	¦--Table Spool



```
테이블 'Worktable'. 검색 수 3, 논리적 읽기 수 2029
테이블 'Workfile'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_Std02'. 검색 수 1, 논리적 읽기 수 12
테이블 'TB_Point02'. 검색 수 1, 논리적 읽기 수 11
SQL Server 실행 시간:
CPU 시간 = 16ms, 경과 시간 = 170ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Std02	PK_Std02	NONCLUSTERED	1	std_no	-

4. [AS-IS] 구문 설명

[TB_Std02] 테이블과 [TB_Point02] 테이블을 조인한 데이터로 [std_no], [class_cd] 컬럼을 기준하여 [point] 컬럼의 평균값을 구하고, OVER (Partition By class_cd) 절에 COUNT 함수를 적용하여 [class_cd] 그룹별로 집계한 행 수를 동시에 출력하는 구문이다. [class_cd] 컬럼을 그룹별로 집계하는 과정에서 Spool이 사용되는데, 이는 두 테이블을 조인하여 평균값이 구해진 데이터를 임시 테이블에 저장하고 다시 읽어내면서 그룹별 행을 집계하기 위해 사용된다. 이렇게 Spool은 재사용이 필요한 데이터를 저장하고 읽어내기 위해 사용되는데, 임시 테이블을 이용하기 때문에 Worktable에 대한 I/O 비용이 발생된다.

5. [TO-BE] 튜닝 포인트

해당 구문은 [TB_Std02] 테이블의 [class_cd] 그룹별 행 수를 집계하기 위해 데이터를 재사용하는 과정에서 I/O가 사용된다. COUNT 함수가 포함된 OVER 절을 사용하는 대신, [TB_Std02] 테이블을 별도로 집계하고 조인하도록 구문을 변경하면 데이터 재사용의 필요성이 없어지기 때문에 임시 테이블로 인한 I/O를 개선할 수 있다.

6. [TO-BE] 개선 방안

1) [class_cd] 그룹 별 행 수를 별도로 추출하여 조인하도록 구문 변경

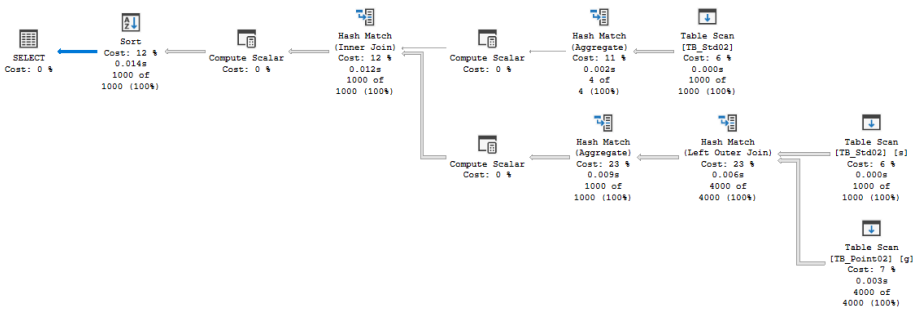
[TB_Std02] 테이블의 [class_cd] 컬럼을 기준하여 그룹별 행 수를 구하고 평균 데이터를 구하는 구문에 조인한다. 재사용에 필요한 데이터를 테이블로 읽어 내기 때문에 Spool에 의한 수행 과정이 제거된다.

■ 변경된 SQL 구문

```
SELECT a.std_no, a.class_cd, a.avg_point, b.total_cnt
FROM (SELECT s.std_no,s.class_cd,  ISNULL(AVG(g.point), 0) avg_point
       FROM TB_Std02 s LEFT OUTER JOIN TB_Point02 g
       ON s.std_no = g.std_no
       GROUP BY s.std_no, s.class_cd) a
INNER JOIN
     (SELECT class_cd, COUNT(1) total_cnt
       FROM TB_Std02
       GROUP BY class_cd) b
ON a.class_cd = b.class_cd
ORDER BY a.class_cd, avg_point DESC, a.std_no
GO
```

· 실행 계획

Rows	Executes	StmtText
1000	1	SELECT a.std_no, a.class_cd, a.avg_point, b.total_cnt FROM (SELECT s.std_no,s.class_cd, ISNULL(AVG(g.point),0) avg_point FROM TB_Std02 s LEFT OUTER JOIN TB_Point02 g ON s.std_no = g.std_no GROUP BY s.std_no,s.class_cd) a INNER JOIN (SELECT class_cd, COUNT(1) total_cnt FROM TB_Std02 GROUP BY class_cd) b ON a.class_cd = b.class_cd ORDER BY a.class_cd, avg_point DESC, a.std_no
1000	1	¦--Sort(ORDER BY:([s].[class_cd] ASC, [Expr1005] DESC, [s].[std_no] ASC))
0	0	¦--Compute Scalar(DEFINE:([Expr1005]=isnull([Expr1004],(0))))
1000	1	¦--Hash Match(INNER JOIN, HASH:([Tuning].[dbo].[TB_Std02].[class_cd])=([s].[class_cd]), RESIDUAL:([Tuning].[dbo].[TB_Std02].[class_cd]=[Tuning].[dbo].[TB_Std02].[class_cd] as [s].[class_cd]))
0	0	¦--Compute Scalar(DEFINE:([Expr1009]=CONVERT_IMPLICIT(int,[Expr1012],0)))
4	1	¦ ¦--Hash Match(Aggregate, HASH:([Tuning].[dbo].[TB_Std02].[class_cd]), RESIDUAL :([Tuning].[dbo].[TB_Std02].[class_cd] = [Tuning].[dbo].[TB_Std02].[class_cd]) DEFINE:([Expr1012]=COUNT(*)))
1000	1	¦ ¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Std02]))
0	0	¦--Compute Scalar(DEFINE:([Expr1004]=CASE WHEN [Expr1013]=(0) THEN NULL ELSE [Expr1014]/CONVERT_IMPLICIT(int,[Expr1013],0) END))
1000	1	¦--Hash Match(Aggregate, HASH:([s].[std_no]) DEFINE:([Expr1013]=COUNT_BIG([Tuning].[dbo].[TB_Point02].[point] as [g].[point]), [Expr1014]=SUM([Tuning].[dbo].[TB_Point02].[point] as [g].[point]), [s].[class_cd]=ANY([Tuning].[dbo].[TB_Std02].[class_cd] as [s].[class_cd])))
4000	1	¦--Hash Match(LEFT OUTER JOIN, HASH:([s].[std_no])=([g].[std_no]))
1000	1	¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Std02] AS [s]))
4000	1	¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Point02] AS [g]))



테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'Workfile'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_Point02'. 검색 수 1, 논리적 읽기 수 11
테이블 'TB_Std02'. 검색 수 2, 논리적 읽기 수 24
SQL Server 실행 시간:
CPU 시간 = 15ms, 경과 시간 = 143ms

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	16	170	2,052
TO-BE	15	143	35

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

37. COUNT 함수의 OVER 절 성능 이슈 - 2

1. 프롬프트

> [AS-IS]
> 구문에서 COUNT(1) OVER (PARTITION BY class_cd)는 TB_Std02 테이블의 전체 데이터를 파티션별로 집계하므로, Spool 연산자가 발생한다.
> Spool이 발생하지 않도록 최적의 개선안을 알려줘.

```
[AS-IS]
SELECT s.std_no, s.class_cd
       , ISNULL(AVG(g.point),0) avg_point
       , COUNT(1) OVER (PARTITION BY class_cd) total_cnt
FROM TB_Std02 s LEFT OUTER JOIN TB_Point02 g
ON s.std_no = g.std_no
GROUP BY s.std_no, s.class_cd
ORDER BY s.class_cd, avg_point DESC, s.std_no;
```

2. 분석 및 주요 최적화

원인 분석

COUNT(1) OVER (PARTITION BY class_cd)는 GROUP BY 이후 윈도우 함수로 동작합니다.

SQL Server는 이미 집계된 결과 셋을 다시 파티션별로 스캔해야 하므로 Table Spool (Eager/Lazy)

연산자가 발생합니다.

3. 개선안

```
-- 윈도우 함수 대신 CTE로 class_cd별 카운트를 미리 집계한 뒤 JOIN하면,
-- Spool 없이 Hash/Merge Join으로 처리됩니다.
-- Spool 제거: Window Function 대신 CTE로 class_cd별 학생 수를 미리 집계 후 JOIN

WITH StudentAvg AS (
    SELECT s.std_no, s.class_cd
        , ISNULL(AVG(g.point), 0) AS avg_point
    FROM dbo.TB_Std02 s
    LEFT OUTER JOIN dbo.TB_Point02 g
        ON s.std_no = g.std_no
    GROUP BY s.std_no, s.class_cd
),
ClassCount AS (
    SELECT class_cd, COUNT(1) AS total_cnt
    FROM dbo.TB_Std02
    GROUP BY class_cd
)
SELECT sa.std_no, sa.class_cd, sa.avg_point, cc.total_cnt
FROM StudentAvg sa
INNER JOIN ClassCount cc
ON sa.class_cd = cc.class_cd
ORDER BY sa.class_cd, sa.avg_point DESC, sa.std_no;
```

■ 변경 사항 요약

항목	AS-IS	TO-BE
total_cnt 계산	COUNT(1) OVER (PARTITION BY class_cd) – 윈도우 함수	ClassCount CTE에서 GROUP BY class_cd로 사전 집계
Spool 발생	✔ 발생 (윈도우 함수가 집계 결과를 재스캔)	✖ 제거 (JOIN으로 대체)
테이블 참조	dbo.TB_Std02 1회 스캔	dbo.TB_Std02 2회 스캔 (단, Spool 비용보다 효율적)
스키마 표기	1-part name	2-part name (dbo. 접두사)

38 FUNCTION 성능 이슈 - 1

스칼라 반환 함수를 사용했을 때 인덱스 부재로 인해 테이블 전체 스캔이 반복되면서 I/O가 증가된 사례이다.

1. 사전 구성 스크립트

```
SELECT rownum AS seq, NEWID() AS main_code
      , DATEADD(MINUTE, rownum, '2020-01-01 00:00:00.000') AS date
```

```
INTO TB_Main04
FROM (SELECT ROW_NUMBER() OVER (ORDER BY a.number) AS rownum, a.number
       FROM master..spt_values a
       JOIN master..spt_values b
       ON a.type = 'P' AND b.type = 'P' AND a.number <= 1000 AND b.number <= 1000
) a

SELECT main_code AS sub_code, CAST(seq%5326 AS INT) AS value
INTO TB_Sub04
FROM TB_Main04

CREATE CLUSTERED INDEX CIDX_Main04 ON TB_Main04(date)
GO

/* 함수 생성문 */
CREATE FUNCTION [dbo].[FN_Code]
(@a UNIQUEIDENTIFIER)
RETURNS UNIQUEIDENTIFIER
AS
BEGIN
      DECLARE @b UNIQUEIDENTIFIER
      SELECT @b = sub_code FROM TB_Sub04 WHERE sub_code = @a
      RETURN @b
END
GO
```

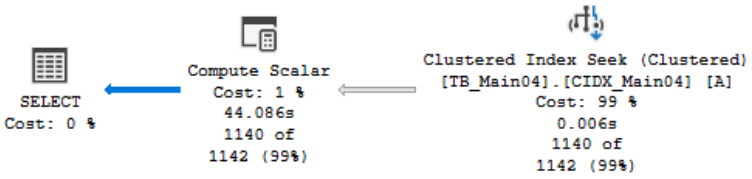
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT *, [dbo].[FN_Code](Main_Code) AS f
FROM TB_Main04 A
WHERE A.date BETWEEN '2020-01-30 00:00:00.000' AND '2020-01-30 18:59:59.000'
GO
```

· 실행 계획

Rows	Executes	StmtText
1140	1	SELECT *, [dbo].[FN_Code](Main_Code) AS f FROM TB_Main04 A WHERE A.date BETWEEN '2020-01-30 00:00:00.000' AND '2020-01-30 18:59:59.000'
1140	1	¦--Compute Scalar(DEFINE:([Expr1002]=[Tuning].[dbo].[FN_Code]([Tuning].[dbo].[TB_Main04].[main_code] as [A].[main_code])))
1140	1	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Main04].[CIDX_Main04] AS [A]), SEEK:([A].[date] >= '2020-01-30 00:00:00.000' AND [A].[date] <= '2020-01-30 18:59:59.000') Ordered Forward)




```
테이블 'TB_Main04'. 검색 수 1, 논리적 읽기 수 10
SQL Server 실행 시간:
CPU 시간 = 46812ms, 경과 시간 = 47878ms
```

- **SQL 구문 (Profiler)**

EventClass	TextData	CPU	Reads	Duration
SQL:StmtCompleted	SELECT *, [dbo].[FN_Code](Main_Code,...	46812	4106292	47878

```
SELECT *, [dbo].[FN_Code](Main_Code) AS f
FROM TB_Main04 A
WHERE A.date BETWEEN '2020-01-30 00:00:00.000' AND '2020-01-30 18:59:59.000'
```

- **함수 내부 실행 계획 (Profiler)**

```
Table Scan(OBJECT:([Tuning].[dbo].[TB_Sub04]), WHERE:([Tuning].[dbo].[TB_Sub04].[sub_code]=[@a]))
```

3. [AS-IS] 통계 정보

- **인덱스 정보**

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Main04	CIDX_Main04	CLUSTERED	0	date	-

4. [AS-IS] 구문 설명

[TB_Main04] 테이블의 [date] 컬럼이 '2020-01-30 00:00:00.000'부터 '2020-01-30 18:59:59.000'사이에 해당되는 Main_Code 컬럼 값은 매개변수로 [FN_Code] 스칼라 반환 함수를 호출하여 [sub_code] 값을 함께 추출하는 구문이다. 해당 구문의 수행 시간은 40초 이상이 소요됐지만 I/O는 상당히 낮은 수치로 확인되는데, 이는 실행 계획을 포함해 함수가 사용한 비용이 출력되지 않기 때문이다. SQL Profiler를 통해 확인해 보면 함수 내부에 사용된 [TB_Sub04] 테이블에는 조건절로 사용한 컬럼에 인덱스가 존재하지 않아 테이블 전체 스캔이 발생됨을 확인할 수 있다. 선행되는 결과 집합 1,140건만큼 함수가 호출되면서 테이블의 전체 스캔이 반복되어 I/O가 과다하게 발생된 것이다.

5. [TO-BE] 튜닝 포인트

선행되는 [TB_Main04] 테이블의 결과 집합 건수만큼 함수 내부의 테이블 전체 스캔이 수행된다. 함수가 호출될 때마다 인덱스 탐색으로 수행될 수 있도록 [TB_Sub04] 테이블에 사용된 조건절에 인덱스를 생성한다.

6. [TO-BE] 개선 방안

1) [TB_Sub04] 테이블 조건절 컬럼에 인덱스를 생성한다.

함수 내부의 [TB_Sub04] 테이블 조건절인 [sub_code] 컬럼에 인덱스를 생성하면 필요한 데이터만 탐색할 수 있기 때문에 전체적인 비용을 감소시킬 수 있다.

■ 인덱스 생성 구문

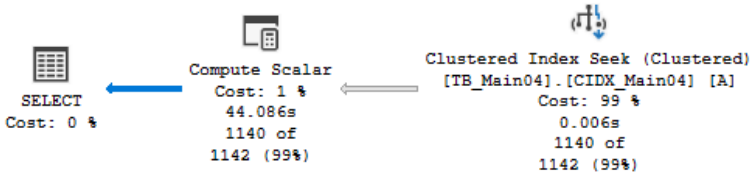
```
CREATE INDEX NIDX01_Sub04 ON TB_Sub04(sub_code)
GO
```

■ SQL 구문

```
SELECT *, [dbo].[FN_Code](Main_Code) AS f
FROM TB_Main04 A
WHERE A.date BETWEEN '2020-01-30 00:00:00.000' AND '2020-01-30 18:59:59.000'
GO
```

· 실행 계획

Rows	Executes	StmtText
1140	1	SELECT *, [dbo].[FN_Code](Main_Code) AS f FROM TB_Main04 A WHERE A.date BETWEEN '2020-01-30 00:00:00.000' AND '2020-01-30 18:59:59.000'
1140	1	¦--Compute Scalar(DEFINE:([Expr1002]=[Tuning].[dbo].[FN_Code]([Tuning].[dbo].[TB_Main04].[main_code] as [A].[main_code])))
1140	1	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Main04].[CIDX_Main04] AS [A]), SEEK:([A].[date] >= '2020-01-30 00:00:00.000' AND [A].[date] <= '2020-01-30 18:59:59.000') Ordered Forward)




```
테이블 'TB_Main04'. 검색 수 1, 논리적 읽기 수 10
SQL Server 실행 시간:
CPU 시간 = 219ms, 경과 시간 = 3327ms
```

· SQL 구문 (Profiler)

EventClass	TextData	CPU	Reads	Duration
SQL:StmtCompleted	SELECT *, [dbo].[FN_Code](Main_Code,...	219	3438	3327

```
SELECT *, [dbo].[FN_Code](Main_Code) AS f
FROM TB_Main04 A
WHERE A.date BETWEEN '2020-01-30 00:00:00.000' AND '2020-01-30 18:59:59.000'
```

· 함수 내부 실행 계획 (Profiler)

```
Index Seek(OBJECT:([Tuning].[dbo].[TB_Sub04].[NIDX01_Sub04]), SEEK:([Tuning].[dbo].[TB_Sub04].[sub_code]=[@a]) ORDERED FORWARD)
```

7. 개선 효과

· 리소스 사용률 비교

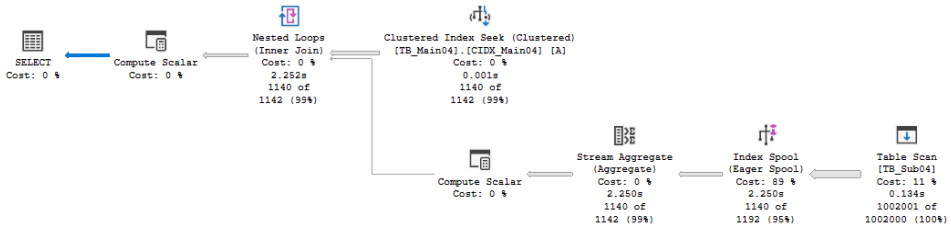
	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	46,812	47,878	4,106,292
TO-BE	219	3,327	3,438

8. 기타

SQL Server 2019에서는 일부 함수에 대해 SQL Profiler를 사용하지 않고 SET STATISTICS PROFILE ON, SET STATISTICS IO ON 옵션만으로도 함수 내부적으로 처리되는 비용을 확인할 수 있다.

· [AS-IS] 실행 계획

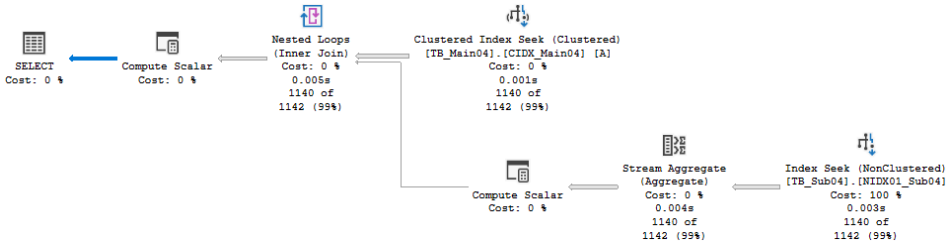
Rows	Executes	StmtText
1140	1	SELECT *, [dbo].[FN_Code](Main_Code) AS f FROM TB_Main04 A WHERE A.date BETWEEN '2020-01-30 00:00:00.000' AND '2020-01-30 18:59:59.000'
0	0	¦--Compute Scalar(DEFINE:([Expr1011]=CONVERT_IMPLICIT(uniqueidentifier,CASE WHEN [Expr1005]=(0) THEN NULL ELSE [Expr1006] END,0)))
1140	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([A].[main_code]))
1140	1	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Main04].[CIDX_Main04] AS [A]), SEEK:([A].[date] >= '2020-01-30 00:00:00.000' AND [A].[date] <= '2020-01-30 18:59:59.000') Ordered Forward)
0	0	¦--Compute Scalar(DEFINE:([Expr1005]=CONVERT_IMPLICIT(int,[Expr1015],0)))
1140	1140	¦--Stream Aggregate(DEFINE:([Expr1015]=Count(*), [Expr1006]=ANY([Tuning].[dbo].[TB_Sub04].[sub_code])))
1140	1140	¦--Index Spool(SEEK:([Tuning].[dbo].[TB_Sub04].[sub_code]=[Tuning].[dbo].[TB_Main04].[main_code] as [A].[main_code]))
1002001	1	¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Sub04]))



테이블 'Worktable'. 스캔 수 1140, 논리적 읽기 2964869
테이블 'TB_Sub04'. 스캔 수 1, 논리적 읽기 3595
테이블 'TB_Main04'. 스캔 수 1, 논리적 읽기 10
SQL Server 실행 시간:
CPU 시간 = 3344ms, 경과 시간 = 3483ms

· [TO-BE] 실행 계획

Rows	Executes	StmtText
1140	1	SELECT *, [dbo].[FN_Code](Main_Code) AS f FROM TB_Main04 A WHERE A.date BETWEEN '2020-01-30 00:00:00.000' AND '2020-01-30 18:59:59.000'
0	0	┦--Compute Scalar(DEFINE:([Expr1011]=CONVERT_IMPLICIT(uniqueidentifier,CASE WHEN [Expr1005]=(0) THEN NULL ELSE [Expr1006] END,0)))
1140	1	┦--Nested Loops(INNER JOIN, OUTER REFERENCES:([A].[main_code], [Expr1016]) WITH UNORDERED PREFETCH)
1140	1	┦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Main04].[CIDX_Main04] AS [A]), SEEK:([A].[date] >= '2020-01-30 00:00:00.000' AND [A].[date] <= '2020-01-30 18:59:59.000') Ordered Forward)
0 1140	0 1140	┦--Compute Scalar(DEFINE:([Expr1005]=CONVERT_IMPLICIT(int,[Expr1015],0))) ┦--Stream Aggregate(DEFINE:([Expr1015]=Count(*), [Expr1006]=ANY([Tuning].[dbo].[TB_Sub04].[sub_code]))
1140	1140	┦--Index Seek(OBJECT:([Tuning].[dbo].[TB_Sub04].[NIDX01_Sub04]), SEEK:([Tuning].[dbo].[TB_Sub04].[sub_code]=[Tuning].[dbo].[TB_Main04].[main_code] as [A].[main_code]) Ordered Forward)



테이블 'TB_Sub04'. 스캔 수 1140, 논리적 읽기 3668
테이블 'TB_Main04'. 스캔 수 1, 논리적 읽기 10
SQL Server 실행 시간:
CPU 시간 = 16ms, 경과 시간 = 117ms

GitHub Copilot 채팅 결과

Claude Opus 4.6

38. FUNCTION 성능 이슈 - 1

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT *, [dbo].[FN_Code](Main_Code) AS f
FROM TB_Main04 A
WHERE A.date BETWEEN '2020-01-30 00:00:00.000' AND '2020-01-30 18:59:59.000';
```

2. 분석 및 주요 최적화

1) 현재 실행 통계 요약

항목	값
CPU 시간	66,969ms (~67초)
경과 시간	67,205ms
반환 행 수	1,140행
논리적 읽기	10 (매우 적음)
TB_Main04 행 수	1,002,001
TB_Sub04 행 수	1,002,001

*논리적 읽기가 10으로 매우 적은데도 CPU가 67초나 소요된 것은 스칼라 UDF가 원인입니다.

2) 발견된 문제점

#	문제	심각도	설명
1	스칼라 UDF dbo.FN_Code 행별 호출	● 치명적	1,140행 각각에 대해 함수가 호출되며, 매 호출마다 인덱스 없는 TB_Sub04 (100만 행)를 풀 스캔합니다.
2	TB_Sub04.sub_code에 인덱스 없음	● 치명적	함수 내부의 조회가 매번 테이블을 풀 스캔할 수 있습니다.
3	SELECT * 사용	● 경고	불필요한 컬럼까지 반환하며, 코드 가독성이 떨어집니다.
4	BETWEEN 시간 경계값 누락 가능	● 경고	18:59:59.000 ~ 18:59:59.999 사이의 데이터가 누락될 수 있습니다.
5	UDF 로직 자체가 불필요	● 참고	함수는 sub_code를 받아 같은 sub_code를 반환하므로, 존재 여부 확인 용도로만 사용됩니다.

3) dbo.FN_Code 함수 분석

```
-- 현재 함수: sub_code를 받아서 동일한 sub_code가 있으면 그대로 반환

CREATE FUNCTION [dbo].[FN_Code](@a UNIQUEIDENTIFIER)
RETURNS UNIQUEIDENTIFIER
AS
BEGIN
    DECLARE @b UNIQUEIDENTIFIER
    SELECT @b = sub_code FROM TB_Sub04 WHERE sub_code = @a
    RETURN @b  -- 입력값과 동일한 값을 반환 (없으면 NULL)
END;
```

*이 함수는 입력값이 TB_Sub04에 존재하면 그 값을 그대로 반환하고, 없으면 NULL을 반환합니다.
LEFT JOIN으로 완벽히 대체할 수 있습니다.

3. 개선안

```
-- 스칼라 UDF를 LEFT JOIN으로 대체, SELECT * 제거, BETWEEN 경계값 수정

SELECT A.seq, A.main_code, A.date, B.sub_code AS f
FROM dbo.TB_Main04 A
LEFT JOIN dbo.TB_Sub04 B
ON A.main_code = B.sub_code
WHERE A.date >= '2020-01-30 00:00:00.000' AND A.date <  '2020-01-30 19:00:00.000'
```

■ 변경 사항 요약

변경 (Change)	AS-IS	TO-BE	효과 (Effect)
UDF 제거	dbo.FN_Code (Main_Code) 행별 호출	LEFT JOIN dbo.TB_Sub04	1,140회 풀 스캔 → 1회 세트 기반 JOIN
SELECT * 제거	SELECT *	SELECT seq, main_code, date	명시적 컬럼 지정
시간 경계 수정	BETWEEN ... 18:59:59.000	< '19:00:00.000'	밀리초 데이터 누락 방지

*SQL Server에서 스칼라 UDF는 행마다 개별 호출되어 병렬 처리가 차단되고, 특히 내부에 테이블 조회가 있을 경우 성능이 급격히 저하됩니다. LEFT JOIN으로 전환하면 옵티마이저가 세트 기반으로 최적화할 수 있어 극적인 성능 향상이 기대됩니다.

39 FUNCTION 성능 이슈 - 2

스칼라 반환 함수가 반복 호출되면서 많은 I/O가 사용된 사례이다.

1. 사전 구성 스크립트

```
/* 데이터 생성 */

SELECT seq
      , CONVERT(NVARCHAR(36),NEWID()) code
      , DATEADD(mi, seq-1, '2001-01-01') date
INTO TB_FMain
FROM (SELECT ROW_NUMBER() OVER (ORDER BY a.number) seq
       FROM master..spt_values a, master..spt_values b
       WHERE a.type = 'P'
       AND a.number BETWEEN 1 AND 1000
       AND b.type = 'P'
       AND b.number BETWEEN 1 AND 100
) c
SELECT seq
      , CASE WHEN seq <= 25000 THEN 1
        WHEN seq <= 50000 THEN 2
        WHEN seq <= 75000 THEN 3
        ELSE 4 END group_id
      , CONVERT(INT, LEFT(seq * num * ASCII(SUBSTRING(code, 1, 1)), 2)) value
INTO TB_FSub
FROM TB_FMain a, (SELECT 1 num UNION ALL SELECT 2 num UNION ALL SELECT 3 num) b

CREATE CLUSTERED INDEX CIDX_FMain ON TB_FMain(date)
CREATE INDEX NIDX01_FSub ON TB_FSub(group_id,seq) INCLUDE (value)
GO

/* 함수 생성문 */
CREATE FUNCTION FN_SUM_VALUE ( @seq INT )
RETURNS INT
AS
BEGIN
DECLARE @value INT
      SELECT @value = SUM(value)
      FROM TB_FSub
      WHERE seq = @seq AND group_id  IN (1,2)
      RETURN @value
END
GO
```

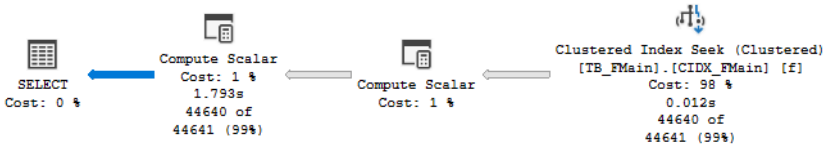
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT f.seq, f.code, dbo.FN_SUM_VALUE(seq) sum_value
FROM TB_FMain f
WHERE date BETWEEN '2001-01-01 00:00:00.000' AND '2001-01-31 23:59:59.997'
GO
```

· 실행 계획

Rows	Executes	StmtText
44640	1	SELECT f.seq, f.code ,dbo.FN_SUM_VALUE(seq) sum_value FROM TB_FMain f WHERE date BETWEEN '2001-01-01 00:00:00.000' AND '2001-01-31 23:59:59.997'
44640	1	¦--Compute Scalar(DEFINE:([Expr1002]=[Tuning].[dbo].[FN_SUM_VALUE]([Expr1003])))
0	0	¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(int,[Tuning].[dbo].[TB_FMain].[seq] as [f].[seq],0)))
44640	1	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_FMain].[CIDX_FMain] AS [f]), SEEK:([f].[date] >= '2001-01-01 00:00:00.000' AND [f].[date] <= '2001-01-31 23:59:59.997') Ordered Forward)



테이블 'TB_FMain'. 스캔 수 1, 논리적 읽기 577
SQL Server 실행 시간:
CPU 시간 = 1891ms, 경과 시간 = 65390ms

· 수행 비용 (Profiler)

EventClass	TextData	CPU	Duration	Reads	RowCounts
SP:StmtCompleted	RETURN @value	0	0	0	1
SQL:BatchCompleted	SELECT f.seq, f.code ,dbo.FN_SUM_VAL...	1891	65390	269361	133920

```
SELECT f.seq, f.code ,dbo.FN_SUM_VALUE(seq) sum_value
FROM TB_Fmain f
WHERE date BETWEEN '2001-01-01 00:00:00.000' AND '2001-01-31 23:59:59.997'
```

· 함수 내부 SQL 구문 (Profiler)

EventClass	TextData	CPU	Duration	Reads	RowCounts
SP:StmtCompleted	RETURN @value	0	0	0	1
SP:StmtCompleted	SELECT @value = SUM(value) FROM TB_FSub WH...	0	0	7	1

```
SELECT @value = SUM(value)
FROM TB_FSub
WHERE seq = @seq
AND group_id IN (1,2)
```

· 함수 내부 실행 계획 (Profiler)

```
Execution Tree
Compute Scalar(DEFINE:([Expr1003]=CASE WHEN [Expr1009]=(0) THEN NULL ELSE [Expr1010] END))
    ¦--Stream Aggregate(DEFINE:([Expr1009]=COUNT_BIG([Tuning].[dbo].[TB_FSub].[value]), [Expr1010]=SUM([Tuning].[dbo].[TB_FSub].[val
    ue])))
        ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_FSub].[NIDX01_FSub]), SEEK:([Tuning].[dbo].[TB_FSub].[group_id]=(1) AND [Tuning].[d
        bo].[TB_FSub].[seq]=CONVERT_IMPLICIT(bigint,[@seq],0) OR [Tuning].[dbo].[TB_FSub].[group_id]=(2) AND [Tuning].[dbo].[TB_FSub
        ].[seq]=CONVERT_IMPLICIT(bigint,[@seq],0)) Ordered Forward)
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_FMain	CIDX_FMain	CLUSTERED	0	date	-
TB_FSub	NIDX01_FSub	NONCLUSTERED	0	group_id, seq	value

4. [AS-IS] 구문 설명

[TB_FMain] 테이블의 [date] 컬럼이 '2001-01-01 00:00:00.000'부터 '2001-01-31 23:59:59.997'사이에 해당되는 seq 컬럼 값은 매개변수로 [FN_SUM_VALUE] 스칼라 반환 함수를 호출하여 SUM(value) 값을 함께 추출하는 구문이다. 해당 구문은 I/O가 577로 상당히 적은 I/O를 사용하는 것처럼 보이지만 이는 구문에 사용된 함수가 비용에 포함되지 않았기 때문이다. SQL Profiler를 통해 확인해 보면 함수 내부에서 수행된 구문을 포함하여 약 26만의 I/O 비용이 발생된 것을 알 수 있는데, 함수가 한 번 호출될 때 사용되는 비용은 7로 적은 양이지만 선행되는 테이블의 결과 집합 약 4만 5천 건만큼 호출이 반복되면서 26만의 I/O가 사용된 것이다.

5. [TO-BE] 튜닝 포인트

스칼라 반환 함수가 NL Join과 동일한 방식으로 수행되기 때문에 반복 호출되면서 과다한 I/O를 발생시켰다. 함수 내부 구문을 스칼라 서브쿼리로 변경하면 다른 조인 방식을 고려할 수 있고 Hash Join을 통해서 반복적인 호출에 의한 I/O 비용을 제거할 수 있다.

6. [TO-BE] 개선 방안

1) [FN_SUM_VALUE] 함수의 내부 구문을 스칼라 서브쿼리로 변환

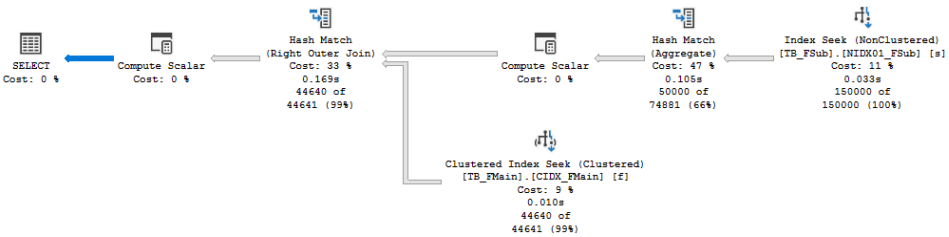
함수 내부 구문은 [TB_FSub] 테이블에 하나의 값을 입력받아 SUM(value) 값을 출력한다. 해당 구문을 스칼라 서브쿼리로 변경하면 Hash Join을 통해 전체 구문에 대한 비용을 감소시킬 수 있다.

■ 변경된 SQL 구문

```
SELECT f.seq, f.code ,(SELECT SUM(value) sum_value
                       FROM TB_FSub s
                       WHERE s.seq = f.seq
                       AND s.group_id IN (1,2)) sum_value
FROM TB_Fmain f
WHERE date BETWEEN '2001-01-01 00:00:00.000' AND '2001-01-31 23:59:59.997'
GO
```

· 실행 계획

Rows	Executes	StmtText
44640	1	SELECT f.seq, f.code ,(SELECT SUM(value) sum_value FROM TB_FSub s WHERE s.seq = f.seq AND s.group_id IN (1,2)) sum_value FROM TB_Fmain f WHERE date BETWEEN '2001-01-01 00:00:00.000' AND '2001-01-31 23:59:59.997'
0	0	¦--Compute Scalar(DEFINE:([Expr1006]=[Expr1004]))
44640	1	¦--Hash Match(Right Outer Join, HASH:([s].[seq])=([f].[seq]), RESIDUAL:([Tuning].[dbo].[TB_FSub].[seq] as [s].[seq]=[Tuning].[dbo].[TB_FMain].[seq] as [f].[seq]))
0	0	¦--Compute Scalar(DEFINE:([Expr1004]=CASE WHEN [Expr1011]=(0) THEN NULL ELSE [Expr1012] END))
50000	1	¦ ¦--Hash Match(Aggregate, HASH:([s].[seq]), RESIDUAL:([Tuning].[dbo].[TB_FSub].[seq] as [s].[seq] = [Tuning].[dbo].[TB_FSub].[seq] as [s].[seq]) DEFINE:([Expr1011]=COUNT_BIG([Tuning].[dbo].[TB_FSub].[value] as [s].[value]), [Expr1012]=SUM([Tuning].[dbo].[TB_FSub].[value] as [s].[value])))
150000	1	¦ ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_FSub].[NIDX01_FSub] AS [s]), SEEK:([s].[group_id]=(1) OR [s].[group_id]=(2)) Ordered Forward)
44640	1	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_FMain].[CIDX_FMain] AS [f]), SEEK:([f].[date] >= '2001-01-01 00:00:00.000' AND [f].[date] <= '2001-01-31 23:59:59.997') Ordered Forward)




```
테이블 'Workfile'. 스캔 수 0, 논리적 읽기 0
테이블 'Worktable'. 스캔 수 0, 논리적 읽기 0
테이블 'TB_FMain'. 스캔 수 5, 논리적 읽기 597
테이블 'TB_FSub'. 스캔 수 7, 논리적 읽기 604
SQL Server 실행 시간: CPU 시간 = 297ms, 경과 시간 = 210ms
```

7. 개선 효과

· 리소스 사용률 비교

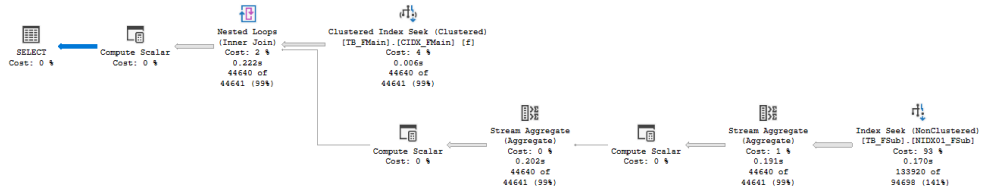
	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	1,891	65,390	269,361
TO-BE	297	210	1,201

8. 기타

SQL Server 2019에서는 일부 함수에 대해 내부적으로 쿼리 변환이 발생되며 정확한 비용에 대한 확인이 가능하다.

· [AS-IS] 실행 계획

Rows	Executes	StmtText
44640	1	SELECT f.seq, f.code ,dbo.FN_SUM_VALUE(seq) sum_value FROM TB_Fmain f WHERE date BETWEEN '2001-01-01 00:00:00.000' AND '2001-01-31 23:59:59.997'
0	0	¦--Compute Scalar(DEFINE:([Expr1012]=CONVERT_IMPLICIT(int,CASE WHEN [Expr1006]=(0) THEN NULL ELSE [Expr1007] END,0)))
44640	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([f].[seq], [Expr1023]) WITH UNORDERED PREFETCH)
44640	1	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_FMain].[CIDX_FMain] AS [f]), SEEK:([f].[date] >= '2001-01-01 00:00:00.000' AND [f].[date] <= '2001-01-31 23:59:59.997') Ordered Forward)
0	0	¦--Compute Scalar(DEFINE:([Expr1006]=CONVERT_IMPLICIT(int,[Expr1022],0)))
44640	44640	¦--Stream Aggregate(DEFINE:([Expr1022]=Count(*), [Expr1007]=ANY([Expr1005])))
0	0	¦--Compute Scalar(DEFINE:([Expr1005]=CASE WHEN [Expr1020]=(0) THEN NULL ELSE [Expr1021] END))
44640	44640	¦--Stream Aggregate(DEFINE:([Expr1020]=COUNT_BIG([Tuning].[dbo].[TB_FSub].[value]), [Expr1021]=SUM([Tuning].[dbo].[TB_FSub].[value])))
133920	44640	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_FSub].[NIDX01_FSub]), SEEK:([Tuning].[dbo].[TB_FSub].[group_id]=(1) AND [Tuning].[dbo].[TB_FSub].[seq]=CONVERT_IMPLICIT(bigint,CONVERT_IMPLICIT(int,[Tuning].[dbo].[TB_FMain].[seq] as [f].[seq],0),0) OR [Tuning].[dbo].[TB_FSub].[group_id]=(2) AND [Tuning].[dbo].[TB_FSub].[seq]=CONVERT_IMPLICIT(bigint,CONVERT_IMPLICIT(int,[Tuning].[dbo].[TB_FMain].[seq] as [f].[seq],0),0)) Ordered Forward)



테이블 'TB_FSub'. 스캔 수 89280, 논리적 읽기 276899
테이블 'TB_FMain'. 스캔 수 1, 논리적 읽기 577
SQL Server 실행 시간:
CPU 시간 = 219ms, 경과 시간 = 541ms

1. 프롬프트

> [AS-IS]
> 구문에서 함수가 한 번 호출될 때 사용되는 I/O 비용은 7로 적은 양이지만 선행되는 테이블의 결과 집합 약 4만 5천 건만큼 호출이 반복되면서 31만의 I/O가 사용된다.
> 이 문제를 분석하여 성능 최적화해줘.

```
[AS-IS]
SELECT f.seq, f.code, dbo.FN_SUM_VALUE(seq) sum_value
FROM TB_FMain f
WHERE date BETWEEN '2001-01-01 00:00:00.000' AND '2001-01-31 23:59:59.997';
```

2. 분석 및 주요 최적화

1) 근본 원인: 스칼라 UDF의 행 단위 반복 호출 (RBAR)

항목	값
반환 항 수	~44,640건
함수 1회 호출 I/O	약 7 (논리적 읽기)
총 I/O (추정)	44,640 × 7 ≈ 312,480
호환성 수준	140 (스칼라 UDF 인라인 미지원)

- dbo.FN_SUM_VALUE는 스칼라 UDF로, TB_FMain에서 반환된 매 행마다 TB_FSub를 개별 조회합니다.
- SQL Server 2017(호환성 수준 140)에서는 스칼라 UDF 인라인이 지원되지 않아, 옵티마이저가 이를 집합 기반으로 변환할 수 없습니다.

2) 최적화 방안

- 핵심 전략: 스칼라 UDF를 제거하고, 사전 집계된 서브쿼리와 JOIN으로 대체하여 TB_FSub를 1회만 스캔합니다.

3. 개선안

```
-- 스칼라 UDF를 LEFT JOIN으로 대체, SELECT * 제거, BETWEEN 경계값 수정

SELECT f.seq, f.code, s.sum_value
FROM dbo.TB_FMain f
LEFT JOIN (SELECT seq, SUM(value) AS sum_value
    FROM dbo.TB_FSub
    WHERE group_id IN (1, 2)
    GROUP BY seq
) s ON f.seq = s.seq
WHERE f.date >= '2001-01-01 00:00:00.000' AND f.date <  '2001-02-01 00:00:00.000'
ORDER BY f.seq;
```

■ 변경 사항 요약

변경 항목	AS-IS	TO-BE
UDF 호출	항마다 FN_SUM_VALUE 호출 (~44,640회)	❌ 제거 → LEFT JOIN 서브쿼리
TB_FSub 스캔	~44,640회 반복	1회 사전 집계
예상 I/O	~312,480 논리적 읽기	TB_FSub 1회 스캔 수준
날짜 필터	BETWEEN ... 23:59:59.997	>= / < 범위 (반개방 구간, 더 안전)
JOIN 유형	없음	LEFT JOIN (UDF가 NULL 반환 가능하므로)

40 FUNCTION 성능 이슈 - 3

스칼라 반환 함수 내부의 조인 구문에서 테이블 전체 스캔이 반복되면서 I/O가 증가된 사례이다.

1. 사전 구성 스크립트

```
SELECT rownum AS seq, NEWID() AS main_code
      , DATEADD(MINUTE, rownum, '2020-01-01 00:00:00.000') AS date
INTO TB_FMain02
FROM (SELECT ROW_NUMBER() OVER (ORDER BY A.number) AS rownum, A.number
       FROM master..spt_values A
       JOIN master..spt_values B
       ON A.type = 'P' AND B.type = 'P' AND A.number <= 1000 AND B.number <= 1000
) A
```

```
SELECT main_code AS sub_code
      , CAST(seq%5326 AS INT) AS value
INTO TB_FSub02
FROM TB_FMain02

SELECT main_code AS code
INTO TB_FCode02
FROM TB_FMain02

CREATE CLUSTERED INDEX CIDX_FMain02 ON TB_FMain02(date)
CREATE INDEX NIDX01_FSub02 ON TB_FSub02(sub_code)
GO

/* 함수 생성문 */
CREATE FUNCTION [dbo].[FN_RTCode]
(@a UNIQUEIDENTIFIER)
RETURNS UNIQUEIDENTIFIER
AS
BEGIN
      DECLARE @b UNIQUEIDENTIFIER
               SELECT @b = sub_code
               FROM TB_FSub02 A INNER JOIN TB_FCode02 B
               ON A.sub_code = B.code
               WHERE A.sub_code = @a
      RETURN @b
END
GO
```

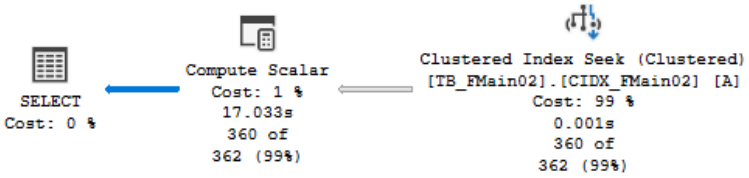
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT *, [dbo].[FN_RTCode](Main_Code) AS f
FROM TB_FMain02 A
WHERE A.date BETWEEN '2020-01-30 00:00:00.000' AND '2020-01-30 05:59:59.000'
GO
```

· 실행 계획

Rows	Executes	StmtText
360	1	SELECT *, [dbo].[FN_RTCode](Main_Code) AS f FROM TB_FMain02 A WHERE A.date BETWEEN '2020-01-30 00:00:00.000' AND '2020-01-30 05:59:59.000'
360	1	¦--Compute Scalar(DEFINE:([Expr1002]=[Tuning].[dbo].[FN_RTCode]([Tuning].[dbo].[TB_FMain02].[main_code] as [A].[main_code])))
360	1	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_FMain02].[CIDX_FMain02] AS [A]), SEEK:([A].[date] >= '2020-01-30 00:00:00.000' AND [A].[date] <= '2020-01-30 05:59:59.000') Ordered Forward)




```
테이블 'TB_FMain02'. 검색 수 1, 논리적 읽기 수 6
SQL Server 실행 시간:
CPU 시간 = 136360ms, 경과 시간 = 43888ms
```

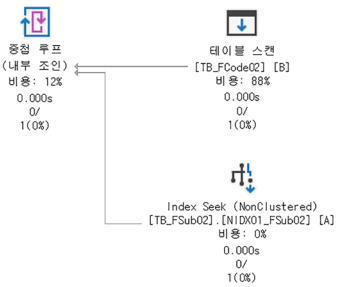
· SQL 구문 (Profiler)

EventClass	TextData	CPU	Duration	Reads	RowCounts
SQL:StmtCompleted	SELECT *, [dbo].[FN_RTCode](Ma...	136360	43888	1747449	1080
SQL:BatchCompleted	SELECT *, [dbo].[FN_RTCode](Ma...	136360	43888	1747449	1080

```
SELECT *, [dbo].[FN_RTCode](Main_Code) AS f
FROM TB_FMain02 A
WHERE A.date BETWEEN '2020-01-30 00:00:00.000' AND '2020-01-30 05:59:59.000'
```

· 함수 내부 실행 계획 (Profiler)

```
Nested Loops(Inner Join)
  |--Index Seek(OBJECT:([Tuning].[dbo].[TB_FSub02].[NIDX01_FSub02] AS [A]), SEEK:([A].[sub_code]=[@a]) ORDERED FORWARD)
  |--Table Scan(OBJECT:([Tuning].[dbo].[TB_FCode02] AS [B]), WHERE:([Tuning].[dbo].[TB_FCode02].[code] as [B].[code]=[@a]))
```



3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_FMain02	CIDX_FMain02	CLUSTERED	0	date	-
TB_FSub02	NIDX01_FSub02	NONCLUSTERED	0	sub_code	-

4. [AS-IS] 구문 설명

[TB_FMain02] 테이블의 [date] 컬럼이 '2020-01-30 00:00:00.000'부터 '2020-01-30 05:59:59.000' 사이에 해당되는 데이터를 [FN_RTCode] 스칼라 반환 함수에 매개변수로 전달하여 함수 결과를 함께 추출하는 구문이다. 해당 구문은 결과 집합 360건의 데이터를 클러스터형 인덱스로 탐색하면서 상당히 낮은 수치의 I/O가 발생되지만 수행되는 시간은 약 15초 소요된다. 이는 실행 계획을 포함해 함수가 사용한 비용이 출력되지 않기 때문인데, SQL Profiler를 통해 확인해 보면 함수 내부 구문에서 사용된 [TB_FCode02] 테이블에 인덱스가 존재하지 않아 테이블 전체 스캔이 발생되는 것을 확인할 수 있다. [TB_FMain02] 테이블의 결과 집합 360건만큼 함수가 호출되고 함수 내부에서 전달된 값마다 [TB_FSub02] 테이블의 조건절로 적용된 결과 건수만큼 [TB_FCode02] 테이블 전체 스캔이 반복되면서 I/O가 과다하게 발생된 것을 확인할 수 있다.

5. [TO-BE] 튜닝 포인트

선행되는 [TB_FCode02] 테이블의 결과 집합 건수만큼 함수 내부의 테이블 전체 스캔이 수행된다. 함수가 호출될 때마다 인덱스 탐색으로 수행될 수 있도록 [TB_FCode02] 테이블에 사용된 조건절에 인덱스를 생성한다.

6. [TO-BE] 개선 방안

1) [TB_FCode02] 테이블에 인덱스를 생성한다.

[TB_FCode02] 테이블의 [code] 컬럼에 인덱스를 생성하면 함수 내부에서 조인이 발생할 때 테이블 전체 스캔이 아닌 필요한 데이터만 탐색하기 때문에 비용을 감소시킬 수 있다.

■ 인덱스 생성 구문

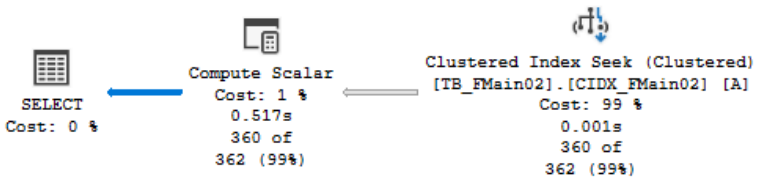
```
CREATE INDEX NIDX01_FCode02 ON TB_FCode02(code)
GO
```

■ SQL 구문

```
SELECT *, [dbo].[FN_RTCode](Main_Code) AS f
FROM TB_FMain02 A
WHERE A.date BETWEEN '2020-01-30 00:00:00.000' AND '2020-01-30 05:59:59.000'
GO
```

· 실행 계획

Rows	Executes	StmtText
360	1	SELECT *, [dbo].[FN_RTCode](Main_Code) AS f FROM TB_FMain02 A WHERE A.date BETWEEN '2020-01-30 00:00:00.000' AND '2020-01-30 05:59:59.000'
360	1	¦--Compute Scalar(DEFINE:([Expr1002]=[Tuning].[dbo].[FN_RTCode]([Tuning].[dbo].[TB_FMain02].[main_code] as [A].[main_code]))
360	1	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_FMain02].[CIDX_FMain02] AS [A]), SEEK:([A].[date] >= '2020-01-30 00:00:00.000' AND [A].[date] <= '2020-01-30 05:59:59.000') Ordered Forward)



테이블 'TB_FMain02'. 검색 수 1, 논리적 읽기 수 6
SQL Server 실행 시간:
CPU 시간 = 109ms, 경과 시간 = 244ms

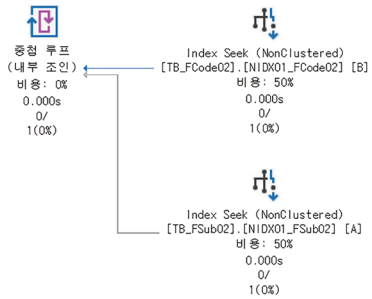
· SQL 구문 (Profiler)

EventClass	TextData	CPU	Duration	Reads	RowCounts
SQL:StmtCompleted	SELECT *, [dbo].[FN_RTCode](Ma...	109	244	2192	1080
SQL:BatchCompleted	SELECT *, [dbo].[FN_RTCode](Ma...	109	244	2192	1080

```
SELECT *, [dbo].[FN_RTCode](Main_Code) AS f
FROM TB_FMain02 A
WHERE A.date BETWEEN '2020-01-30 00:00:00.000' AND '2020-01-30 05:59:59.000'
```

· 함수 내부 실행 계획 (Profiler)

```
Nested Loops(Inner Join)
  |--Index Seek(OBJECT:([Tuning].[dbo].[TB_FCode02].[NIDX01_FCode02] AS [B]), SEEK:([B].[code]=[@a]) ORDERED FORWARD)
  |--Index Seek(OBJECT:([Tuning].[dbo].[TB_FSub02].[NIDX01_FSub02] AS [A]), SEEK:([A].[sub_code]=[@a]) ORDERED FORWARD)
```



7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	136,360	43,888	1,747,449
TO-BE (1)	109	244	2,192

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT *, [dbo].[FN_RTCode](Main_Code) AS f
FROM TB_FMain02 A
WHERE A.date BETWEEN '2020-01-30 00:00:00.000' AND '2020-01-30 05:59:59.000';
```

2. 분석 및 주요 최적화

1) 현재 환경 요약

항목	내용
테이블	dbo.TB_FMain02 (약 100만 행)
컬럼	seq (bigint), main_code (uniqueidentifier), date (datetime)
인덱스	CIDX_FMain02 – date 컬럼 클러스터드 인덱스 ☑
함수	dbo.FN_RTCode – 스칼라 함수 (내부에서 TB_FSub02, TB_FCode02 JOIN)

2) 성능 문제점 분석

스칼라 UDF (FN_RTCode) — 가장 큰 성능 병목

· 스칼라 함수는 행마다 1회씩 호출됩니다. 100만 행 중 조건에 맞는 모든 행에 대해 반복 실행됩니다.

· 함수 내부에서 TB_FSub02 JOIN TB_FCode02 쿼리가 실행되므로 행당 별도의 쿼리 실행과 동일한 효과입니다.

· SQL Server 2017(호환성 레벨 140)에서는 Scalar UDF Inlining이 지원되지 않습니다

3. 개선안

```
-- 스칼라 함수를 JOIN으로 대체하여 세트 기반 처리로 변환합니다.

SELECT A.seq,
       A.main_code,
```

```
       A.date,
       B.sub_code AS f
FROM dbo.TB_FMain02 A
LEFT JOIN dbo.TB_FSub02 B ON A.main_code = B.sub_code
INNER JOIN dbo.TB_FCode02 C ON B.sub_code = C.code
WHERE A.date >= '2020-01-30 00:00:00.000'
AND A.date <  '2020-01-30 06:00:00.000';
```

■ 변경 사항 요약

#	AS-IS 문제	TO-BE 개선	기대 효과
1	SELECT *	컬럼 명시 (seq, main_code, date)	불필요한 I/O 감소
2	스칼라 함수 FN_RTCode 행별 호출	LEFT JOIN + INNER JOIN으로 대체	행별 쿼리 제거 → 대폭 성능 향상
3	BETWEEN ... 05:59:59.000	>= ... AND < 06:00:00.000	datetime 정밀도 누락 방지
4	스키마 미지정 (TB_FMain02)	dbo.TB_FMain02 2-part name 사용	스키마 해석 비용 제거

*스칼라 UDF → JOIN 변환이 가장 큰 성능 개선 포인트입니다. 100만 행 테이블에서 행별 함수 호출을 제거하면 수십~수백 배 성능 향상이 가능합니다.

41 FUNCTION 성능 이슈 - 4

함수가 불필요하게 많이 호출되면서 I/O 비용이 높게 발생한 사례이다.

1. 사전 구성 스크립트

```
SELECT ROW_NUMBER() OVER (ORDER BY a.number) AS id
      , CAST(a.number AS VARCHAR)+LEFT(a.type, 1)+CAST(b.number AS VARCHAR) AS code
INTO TB_Ptype
FROM master..spt_values A, master..spt_values B
WHERE a.type = 'P' AND b.type = 'P'
AND a.number BETWEEN 1 AND 20
AND b.number BETWEEN 1 AND 20

SELECT id
      , CAST(LEFT(id * n1* n2 * h1*36 ,2) AS INT) AS st1
      , CAST(LEFT(id * n1* n2 * h2*56 ,2) AS INT) AS st2
      , CAST(LEFT(id * n1* n2 * h1* h2*84 ,2) AS INT) AS st3
      , CASE WHEN id <= 200 THEN 'A' ELSE 'B' END AS lev
INTO TB_Stype
FROM (SELECT ROW_NUMBER() OVER (ORDER BY a.number) AS id
            , a.number AS n1
            , b.number AS n2
            , a.high AS h1
            , b.high AS h2
```

```
        FROM master..spt_values a, master..spt_values b
        WHERE a.type = 'P' AND b.type = 'P'
        AND a.number BETWEEN 1 AND 20
        AND b.number BETWEEN 1 AND 20
) a

CREATE TABLE TB_Ctype (id INT, c_name NVARCHAR(100))

INSERT INTO TB_Ctype
SELECT id
      , CASE WHEN id <= 100 THEN 'A'
        WHEN id <= 200 THEN 'B'
        WHEN id <= 300 THEN 'C'
        ELSE 'D' END AS c_name
FROM TB_Ptype

CREATE INDEX NIDX01_Ctype on TB_Ctype(id) include (c_name)
CREATE CLUSTERED INDEX CIDX_Ptype ON TB_Ptype(id)
CREATE CLUSTERED INDEX CIDX_Stype ON TB_Stype(id)
GO

CREATE FUNCTION FN_GetCname(@id INT)
RETURNS NVARCHAR(100) AS
BEGIN
      DECLARE @result NVARCHAR(100)
      SELECT @result = c_name FROM TB_Ctype WHERE id = @id
      RETURN @result
END
GO
```

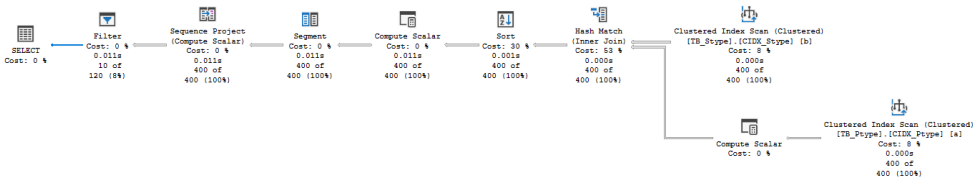
2. [AS-IS] SQL 구문 및 실행 계획

SQL 구문

```
SELECT *
FROM (SELECT ROW_NUMBER() OVER (PARTITION BY c.lev ORDER BY c.st3 DESC) seq
            , c.*
       FROM (SELECT a.*
                  , b.st3
                  , b.lev
                  , dbo.FN_GetCname(a.id) AS cname
             FROM TB_Ptype a JOIN TB_Stype b
             ON a.id = b.id
       ) c
) d
WHERE seq <= 5
GO
```

· 실행 계획

Rows	Executes	StmtText
10	1	SELECT * FROM (SELECT ROW_NUMBER() OVER (PARTITION BY c.lev ORDER BY c.st3 DESC) seq, c.* FROM (SELECT a.*, b.st3, b.lev, dbo.FN_GetCname(a.id) AS cname FROM TB_Ptype a JOIN TB_Stype b ON a.id = b.id) c) d WHERE seq <= 5
10	1	¦--Filter(WHERE:([Expr1005]<=(5)))
400	1	¦--Sequence Project(DEFINE:([Expr1005]=row_number))
400	1	¦--Segment
400	1	¦--Compute Scalar(DEFINE:([Expr1004]=[tuning].[dbo].[FN_GetCname]([Expr1006])))
400	1	¦--Sort(ORDER BY:([b].[lev] ASC, [b].[st3] DESC))
400	1	¦--Hash Match(INNER JOIN, HASH:([b].[id])=([a].[id]), RESIDUAL:([tuning].[dbo].[TB_Stype].[id] as [b].[id]=[tuning].[dbo].[TB_Ptype].[id] as [a].[id]))
400	1	¦--Clustered Index Scan(OBJECT:([tuning].[dbo].[TB_Stype].[CIDX_Stype] AS [b]))
0	0	¦--Compute Scalar(DEFINE:([Expr1006]=CONVERT_IMPLICIT(int,[tuning].[dbo].[TB_Ptype].[id] as [a].[id],0)))
400	1	¦--Clustered Index Scan(OBJECT:([tuning].[dbo].[TB_Ptype].[CIDX_Ptype] AS [a]))



테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'Workfile'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_Ptype'. 검색 수 1, 논리적 읽기 수 4
테이블 'TB_Stype'. 검색 수 1, 논리적 읽기 수 4
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 11ms

· 수행 비용 (Profiler)

EventClass	TextData	CPU	Duration	Reads	RowCounts
SQL:BatchCompleted	SELECT * FROM (SELECT ROW_NUMBE...	125	188	812	810
SQL:BatchCompleted	SET STATISTICS XML OFF	0	0	0	0

```
SELECT *
FROM (SELECT ROW_NUMBER() OVER (PARTITION BY c.lev ORDER BY c.st3 DESC) seq
           , c.*
      FROM (SELECT a.*, b.st3, b.lev, dbo.FN_GetCname(a.id) AS cname
            FROM TB_Ptype a JOIN TB_Stype b
            ON a.id = b.id) c
) d
WHERE seq <= 5
```

· 함수 내부 수행 비용 (Profiler)

EventClass	TextData	CPU	Duration	Reads	RowCounts
SP:StmtCompleted	SELECT @result = c_name FROM TB_Ctype WHER...	0	0	2	1
SP:StmtCompleted	RETURN @result	0	0	0	1

```
SELECT @result = c_name
FROM TB_Ctype
WHERE id = @id
```

· 함수 내부 실행 계획 (Profiler)

Execution Tree
Index Seek(OBJECT:([tuning].[dbo].[TB_Ctype].[NIDX01_Ctype]), SEEK:([tuning].[dbo].[TB_Ctype].[id]=[@id]) Ordered Forward)

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Ctype	NIDX01_Ctype	NONCLUSTERED	0	id	c_name
TB_Ptype	CIDX_Ptype	CLUSTERED	0	id	-
TB_Stype	CIDX_Stype	CLUSTERED	0	id	-

4. [AS-IS] 구문 설명

[TB_Ptype] 테이블과 [TB_Stype] 테이블을 각 [id] 컬럼으로 조인한 결과 집합에 [FN_GetCname] 함수를 적용하여 [cname] 컬럼 값을 추출하고, [lev] 컬럼의 동일한 값마다 [st3] 컬럼을 기준으로 하여 상위 5건씩 데이터를 출력하는 구문이다. 최종 출력되는 행은 10건이지만 데이터가 필터되기 전에 조인된 결과 집합 400건만큼 함수가 호출되기 때문에 불필요한 I/O가 발생된다.

5. [TO-BE] 튜닝 포인트

조인된 결과 집합(400건) 만큼 함수가 호출되는 과정에서 불필요한 I/O가 발생되기 때문에 조인 후가 아닌 최종 10건에 대해서만 함수가 호출되도록 구문을 변경한다.

6. [TO-BE] 개선 방안

1) 필터 후에 함수가 호출되도록 구문 변경

함수 호출 위치를 필터가 적용된 다음으로 변경하여 최종 결과 집합에 대해서만 함수가 호출되도록 유도한다. 함수가 최종 10건에 대해서만 호출되기 때문에 불필요한 I/O가 개선된다.

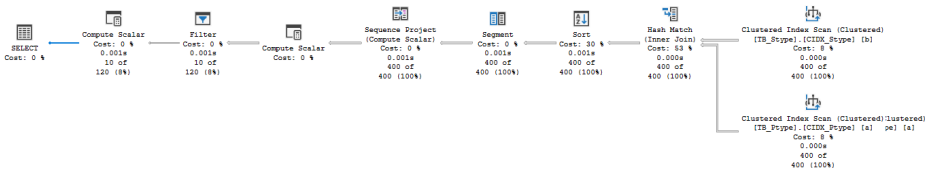
■ 변경된 SQL 구문

```
SELECT *, dbo.FN_GetCname(d.id) AS cname
FROM (SELECT ROW_NUMBER() OVER (PARTITION BY c.lev ORDER BY c.st3 desc ) seq
           , c.*
      FROM (SELECT a.*
                 , b.st3
```

```
                , b.lev
            FROM TB_Ptype a JOIN TB_Stype b
            ON a.id = b.id
        ) c
) d
WHERE seq <= 5
GO
```

· **실행 계획**

Rows	Executes	StmtText
10	1	SELECT *, dbo.FN_GetCname(d.id) AS cname FROM (SELECT ROW_NUMBER() OVER (PARTITION BY c.lev ORDER BY c.st3 desc) seq, c.* FROM (SELECT a.*, b.st3, b.lev FROM TB_Ptype a JOIN TB_Stype b ON a.id = b.id) c) d WHERE seq <= 5
10	1	¦--Compute Scalar(DEFINE:([Expr1005]=[tuning].[dbo].[FN_GetCname]([Expr1006])))
10	1	¦--Filter(WHERE:([Expr1004]<=(5)))
0	0	¦--Compute Scalar(DEFINE:([Expr1006]=CONVERT_IMPLICIT(int,[tuning].[dbo].[TB_Ptype].[id] as [a].[id],0)))
400	1	¦--Sequence Project(DEFINE:([Expr1004]=row_number))
400	1	¦--Segment
400	1	¦--Sort(ORDER BY:([b].[lev] ASC, [b].[st3] DESC))
400	1	¦--Hash Match(INNER JOIN, HASH:([b].[id])=([a].[id]), RESIDUAL:([tuning].[dbo].[TB_Stype].[id] as [b].[id]=[tuning].[dbo].[TB_Ptype].[id] as [a].[id]))
400	1	¦--Clustered Index Scan(OBJECT:([tuning].[dbo].[TB_Stype].[CIDX_Stype] AS [b]))
400	1	¦--Clustered Index Scan(OBJECT:([tuning].[dbo].[TB_Ptype].[CIDX_Ptype] AS [a]))




```
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'Workfile'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_Ptype'. 검색 수 1, 논리적 읽기 수 4
테이블 'TB_Stype'. 검색 수 1, 논리적 읽기 수 4
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 13ms
```

· **함수 내부 수행 비용 (Profiler)**

EventClass	TextData	CPU	Duration	Reads	RowCounts
SQL:BatchCompleted	SELECT *, dbo.FN_GetCname(d.id) AS cname FR...	0	13	30	30
SQL:BatchCompleted	SET STATISTICS XML OFF	0	0	0	0

```
SELECT *, dbo.FN_GetCname(d.id) AS cname
FROM (SELECT ROW_NUMBER() OVER (PARTITION BY c.lev ORDER BY c.st3 desc ) seq
            , c.*
      FROM (SELECT a.*, b.st3, b.lev
            FROM TB_Ptype a JOIN TB_Stype b
            ON a.id = b.id
      ) c
) d
WHERE seq <= 5
```

· 함수내부 I/O 비용 (SQL Profile)

EventClass	TextData	CPU	Duration	Reads	RowCounts
SP:StmtCompleted	RETURN @result	0	0	0	1
SP:StmtCompleted	SELECT @result = c_name FROM TB_Ctype WHER...	0	0	2	1

```
SELECT @result = c_name
FROM TB_Ctype
WHERE id = @id
```

· 함수내부 실행 계획 (Profiler)

```
Execution Tree
Index Seek(OBJECT:([tuning].[dbo].[TB_Ctype].[NIDX01_Ctype]), SEEK:([tuning].[dbo].[TB_Ctype].[id]=[@id]) Ordered Forward)
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	125	188	812
TO-BE	0	16	30

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

41. FUNCTION 성능 이슈 - 4

1. 프롬프트

> [AS-IS]
> 구문에서 최종 출력되는 행은 10건이지만 데이터가 필터되기 전에 조인된 결과 집합 400건 만큼 함수가 호출되기 때문에 불필요한 I/O가 발생된다. 해당 문제점을 분석하여 최적의 개선안을 알려줘.

```
[AS-IS]
SELECT *
FROM (SELECT ROW_NUMBER() OVER (PARTITION BY c.lev ORDER BY c.st3 DESC) seq
               , c.*
        FROM (SELECT a.*
                   , b.st3
                   , b.lev
                   , dbo.FN_GetCname(a.id) AS cname
              FROM TB_Ptype a JOIN TB_Stype b
              ON a.id = b.id
       ) c
) d
WHERE seq <= 5;
```

2. 분석 및 주요 최적화

1) 현재 환경 분석

항목	값
TB_Ptype 행 수	400
TB_Stype 행 수	400
lev 고유값 수	2
최종 출력 행 수	최대 10건 (lev 2개 × 5건)
FN_GetCname 호출 횟수	조인 결과 전체 (360~400건)
호환성 수준	140 (Scalar UDF Inlining 미지원)

2) 문제점 분석

1 스칼라 함수 조기 호출 (핵심 문제)

· dbo.FN_GetCname(a.id)가 가장 안쪽 서브쿼리에 위치하여, ROW_NUMBER() 필터(seq <= 5) 적용 이전에 전체 조인 결과(~400건)에 대해 실행됩니다.

· 최종적으로 10건만 필요하지만 400번 호출되어 TB_Ctype 테이블에 3,112회 논리적 읽기가 발생합니다.

2 스칼라 UDF 자체의 비효율

· 호환성 수준 140에서는 Scalar UDF Inlining이 불가합니다.

· 스칼라 함수는 행 단위(row-by-row) 실행되어 병렬 처리를 차단합니다.

3. 개선안

```
-- 방안 1. 함수 호출을 필터 이후로 이동 (최소 변경)
-- 함수 호출을 ROW_NUMBER() 필터 바깥으로 이동하여 10건에 대해서만 호출합니다.

SELECT d.seq, d.id, d.code, d.st3, d.lev, dbo.FN_GetCname(d.id) AS cname
FROM (SELECT ROW_NUMBER() OVER (PARTITION BY b.lev ORDER BY b.st3 DESC) AS seq,
a.id, a.code, b.st3, b.lev
FROM dbo.TB_Ptype a
    INNER JOIN dbo.TB_Stype b ON a.id = b.id) d
WHERE d.seq <= 5;
```

*효과: 함수 호출 400회 → 10회로 감소

```
-- 방안 2. 스칼라 함수를 JOIN으로 대체
-- 스칼라 UDF를 완전히 제거하고 TB_Ctype 테이블을 직접 JOIN합니다.

SELECT d.seq, d.id, d.code, d.st3, d.lev, ct.c_name AS cname
FROM (
    SELECT ROW_NUMBER() OVER (PARTITION BY b.lev ORDER BY b.st3 DESC) AS seq
         , a.id, a.code, b.st3, b.lev
    FROM dbo.TB_Ptype a
    INNER JOIN dbo.TB_Stype b
    ON a.id = b.id
) d
LEFT JOIN dbo.TB_Ctype ct
ON d.id = ct.id
WHERE d.seq <= 5;
```

*효과: 스칼라 함수 오버헤드 완전 제거 + 옵티마이저가 세트 기반 최적화 가능

■ 개선 효과 비교

항목	AS-IS	개선안 1	개선안 2 (권장)
FN_GetCname 호출 횟수	~400회	10회	0회
TB_Ctype 논리적 읽기	3,112	대폭 감소	최소 (세트 기반)
병렬 처리 가능 여부	✗ (UDF 차단)	✗	✓

42 ISNULL() 함수가 사용된 조건절의 성능 이슈

조건절의 컬럼이 ISNULL() 함수로 가공되어 인덱스 탐색을 못하게 되면서 불필요한 I/O가 발생되는 사례이다.

1. 사전 구성 스크립트

```
SELECT seq
      , NEWID() AS txt
      , CASE WHEN seq = 1 THEN 0 WHEN seq = 2 THEN NULL ELSE number END AS value
INTO TB_Null
FROM (SELECT a.number, ROW_NUMBER() OVER (ORDER BY a.number) AS seq
       FROM master..spt_values a, master..spt_values b
       WHERE a.type = 'P'
       AND b.type = 'P'
       AND a.number BETWEEN 1 AND 1000
       AND b.number BETWEEN 1 AND 1000
) c

CREATE INDEX NIDX01_Null ON TB_Null ( value )
GO
```

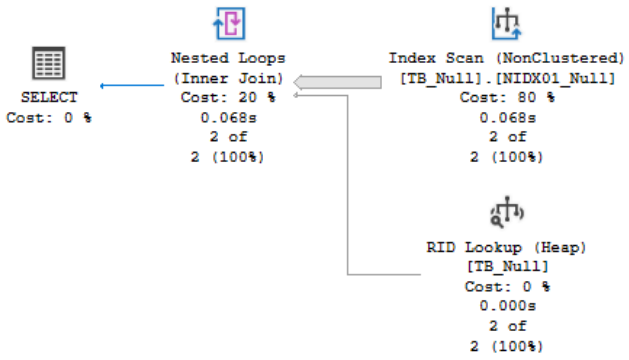
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT seq, txt
FROM TB_Null
WHERE ISNULL(value,0) = 0
GO
```

· 실행 계획

Rows	Executes	StmtText
2	1	SELECT seq, txt FROM TB_Null WHERE ISNULL(value,0) = 0
2	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([Bmk1000]))
2	1	¦--Index Scan(OBJECT:([Tuning].[dbo].[TB_Null].[NIDX01_Null]), WHERE:(isnull([Tuning].[dbo].[TB_Null].[value],(0))=(0)))
2	2	¦--RID Lookup(OBJECT:([Tuning].[dbo].[TB_Null]), SEEK:([Bmk1000]=[Bmk1000]) LOOKUP Ordered Forward)




```
테이블 'TB_Null'. 검색 수 1, 논리적 읽기 수 2239
SQL Server 실행 시간:
CPU 시간 = 78ms, 경과 시간 = 86ms
```

3. [AS-IS] 통계 정보

- **인덱스 정보**

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Null	NIDX01_Null	NONCLUSTERED	0	value	-

4. [AS-IS] 구문 설명

[TB_Null] 테이블 [value] 컬럼 데이터 중 NULL 값을 0으로 치환했을 때 [value] = 0 조건에 해당되는 데이터를 출력하는 구문이다. 전체 데이터 건수는 100만 건이며 조건에 부합하는 결과 건수는 그 중 단 2건에 해당한다. 조건으로 사용되는 [value] 컬럼에는 인덱스가 존재하지만 ISNULL() 함수의 사용으로 인해 조건 컬럼이 가공되었고 이로 인해 인덱스 전체 스캔을 수행하게 되면서 불필요한 I/O를 사용하게 된다.

5. [TO-BE] 튜닝 포인트

인덱스 키로 구성된 컬럼은 스캔 범위에 큰 영향을 주기 때문에 가공하지 않는 것이 성능 면에서 좋다. 구문에 사용된 ISNULL(value,0) = 0 조건은 [value] 컬럼 값이 NULL이거나 0인 것을 의미하기 때문에 컬럼을 가공하지 않은 두 조건으로 분리할 수 있다. 구문 변경으로 조건을 분리하면 인덱스로 필요한 데이터만을 탐색할 수 있기 때문에 많은 I/O 비용을 개선할 수 있다.

6. [TO-BE] 개선 방안

1) 컬럼이 가공되지 않도록 구문 변경

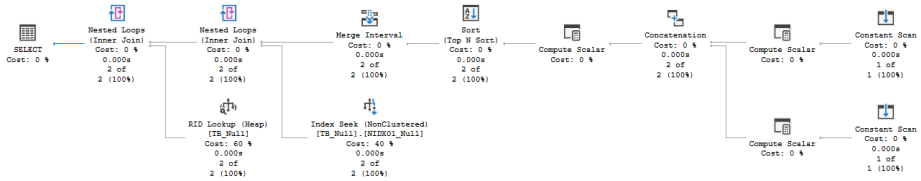
ISNULL(value,0) = 0 조건을 [value] IS NULL OR [value] = 0으로 변경하면 인덱스를 통해 두 조건을 각각 탐색하여 많은 비용이 개선된다.

■ 변경된 SQL 구문

```
SELECT seq, txt
FROM TB_Null
WHERE value IS NULL OR value = 0
GO
```

· 실행 계획

Rows	Executes	StmtText
2	1	SELECT seq, txt FROM TB_Null WHERE value IS NULL OR value = 0
2	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([Bmk1000]))
2	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([Expr1009], [Expr1010], [Expr1011]))
2	1	¦ ¦--Merge Interval
2	1	¦ ¦ ¦--Sort(TOP 2, ORDER BY:([Expr1012] DESC, [Expr1013] ASC, [Expr1009] ASC, [Expr1014] DESC))
0	0	¦ ¦ ¦--Compute Scalar(DEFINE:([Expr1012]=((4)&[Expr1011]) = (4) AND NULL = [Expr1009], [Expr1013]=(4)&[Expr1011], [Expr1014]=(16)&[Expr1011]))
2	1	¦ ¦ ¦--Concatenation
0	0	¦ ¦ ¦--Compute Scalar(DEFINE:([Expr1004]=NULL, [Expr1005]=NULL, [Expr1003]=(60)))
1	1	¦ ¦ ¦ ¦--Constant Scan
0	0	¦ ¦ ¦--Compute Scalar(DEFINE:([Expr1007]=(0), [Expr1008]=(0), [Expr1006]=(62)))
1	1	¦ ¦ ¦ ¦--Constant Scan
2	2	¦ ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_Null].[NIDX01_Null]), SEEK:([Tuning].[dbo].[TB_Null].[value] > [Expr1009] AND [Tuning].[dbo].[TB_Null].[value] < [Expr1010]) Ordered Forward)
2	2	¦--RID Lookup(OBJECT:([Tuning].[dbo].[TB_Null]), SEEK:([Bmk1000]=[Bmk1000]) LOOKUP Ordered Forward)




```
테이블 'TB_Null'. 검색 수 2, 논리적 읽기 수 8
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	78	86	2,239
TO-BE	0	0	8

GitHub Copilot 채팅 결과

42. ISNULL() 함수가 사용된 조건절의 성능 이슈

GPT-5 mini

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT seq, txt
FROM TB_Null
WHERE ISNULL(value,0) = 0;
```

2. 분석 및 주요 최적화

1) WHERE ISNULL(value,0) = 0 — 컬럼에 함수(ISNULL)를 적용하므로
인덱스가 있어 시크(Seek)를 방해하는 비-SARGable 표현입니다.

2) 의미상 ISNULL(value,0)=0 은 value = 0 OR value IS NULL 과 동일하므로,
이를 SARGable 한 표현으로 바꿔야 성능이 개선됩니다.

3. 개선안

```
-- 권장 수정 (간단하고 SARGable)

SELECT seq, txt
FROM dbo.TB_Null
WHERE value = 0 OR value IS NULL;
```

43 MAX() 함수의 성능 개선 - 1

Max 값을 구하는 구문에서 인덱스 키가 누락되어 많은 스캔 범위로 인해 높은 I/O 비용이 발생된 사례이다.

1. 사전 구성 스크립트

```
SELECT CASE WHEN edge2 <= 500000 THEN 1 ELSE 2 END edge1, *
INTO TB_MaxExp
FROM (SELECT ROW_NUMBER() OVER (ORDER BY a.number) AS edge2
            , a.number*ROW_NUMBER() OVER (ORDER BY a.number)/2.0 AS area
      FROM master..spt_values a, (SELECT number
                                  FROM master..spt_values
                                  WHERE number > 0 AND number <= 1000 AND type = 'P')
b
      WHERE a.number > 0 AND a.number <= 1000 AND a.type = 'P'
) c

CREATE INDEX NIDX01_MaxExp ON TB_MaxExp(edge1) INCLUDE (area)
GO
```

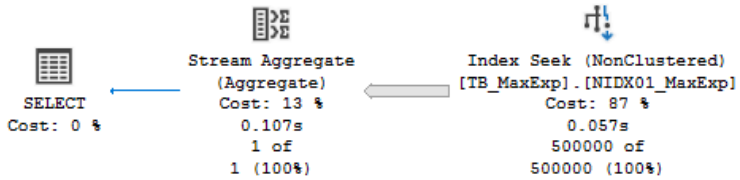
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT MAX(area) AS area
FROM TB_MaxExp
WHERE edge1 = 2
GO
```

· 실행 계획

Rows	Executes	StmtText
1 1 500000	1 1 1	SELECT MAX([area]) [area] FROM [TB_MaxExp] WHERE [edge1]=@1 ¦--Stream Aggregate(DEFINE:([Expr1003]=MAX([tuning].[dbo].[TB_MaxExp].[area]))) ¦--Index Seek(OBJECT:([tuning].[dbo].[TB_MaxExp].[NIDX01_MaxExp]), SEEK:([tuning].[dbo].[TB_MaxExp].[edge1]=(2)) Ordered Forward)



```
테이블 'TB_MaxExp'. 스캔 수 1, 논리적 읽기 1931
SQL Server 실행 시간:
CPU 시간 = 93ms, 경과 시간 = 95ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_MaxExp	NIDX01_MaxExp	NONCLUSTERED	0	edge1	area

4. [AS-IS] 구문 설명

[TB_MaxExp] 테이블에서 [edge1] = 2에 해당하는 데이터 중 [area] 컬럼 값이 가장 큰 데이터를 조회하는 구문이다. 조건에 만족하는 데이터 50만 건 중 Max 데이터 1건을 추출하기 위해 해당 데이터를 모두 읽어내면서 많은 I/O를 사용하고 있다.

5. [TO-BE] 튜닝 포인트

가장 큰 값 1건을 추출하기 위해 모든 데이터를 읽어내면서 값의 크기를 비교하는 것에 문제가 있다. MAX() 함수에 사용된 컬럼을 미리 정렬시켜 둔다면 서로 비교하지 않고도 가장 큰 값을 가진 데이터를 추출해 낼 수 있다.

6. [TO-BE] 개선 방안

1) [area] 컬럼이 키로 추가된 인덱스 생성

[edge1], [area] 컬럼 순서로 신규 인덱스를 생성하면 [edge1] = 2 조건에 해당하는 데이터 내에서 [area] 컬럼 데이터가 정렬된 상태로 유지되어 모든 데이터를 비교하지 않아도 가장 큰 값을 추출할 수 있다.

■ 인덱스 생성 구문

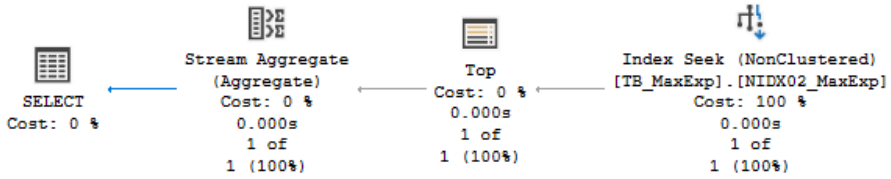
```
CREATE INDEX NIDX02_MaxExp ON TB_MaxExp(edge1, area)
GO
```

■ SQL 구문

```
SELECT MAX(area) AS area
FROM TB_MaxExp
WHERE edge1 = 2
GO
```

· 실행 계획

Rows	Executes	StmtText
1 1 1 1	1 1 1 1	SELECT MAX([area]) [area] FROM [TB_MaxExp] WHERE [edge1]=@1 ¦--Stream Aggregate(DEFINE:([Expr1003]=MAX([tuning].[dbo].[TB_MaxExp].[area]))) ¦--Top(TOP EXPRESSION:((1))) ¦--Index Seek(OBJECT:([tuning].[dbo].[TB_MaxExp].[NIDX02_MaxExp]), SEEK:([tuning].[dbo].[TB_MaxExp].[edge1]=(2)), WHERE:([tuning].[dbo].[TB_MaxExp].[area] IS NOT NULL) ORDERED Backward)



테이블 'TB_MaxExp'. 스캔 수 1, 논리적 읽기 3
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	93	95	1,931
TO-BE	0	0	3

GitHub Copilot 채팅 결과

GPT-5 mini

43. MAX() 함수의 성능 개선 - 1

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT MAX(area) AS area
FROM TB_MaxExp
WHERE edge1 = 2;
```

2. 분석 및 주요 최적화

1) 문제 요약

· 원본: SELECT MAX(area) FROM TB_MaxExp WHERE edge1 = 2

· 현재 인덱스는 edge1를 키로 가지고 area를 포함열(included)로만 정의함.

· 포함열은 정렬 키가 아니므로 ORDER BY area DESC를 효율적으로 지원하지 못함.

· 따라서 빠른 단일값 검색(Top 1 by area) 위해서는 area를 인덱스 키로 포함한 인덱스가 필요합니다.

2) 최적화 방안

· 인덱스 생성 (권장)

· edge1, area DESC를 키로 하는 비클러스터드 인덱스.

· 이 인덱스가 있으면 단일 SEEK + 첫 행 읽기로 결과를 즉시 얻습니다.

3. 개선안

```
-- 권장 인덱스 – 일반(모든 edge1 값에 대해)

CREATE NONCLUSTERED INDEX NIDX_MaxExp_edge1_area_desc
ON dbo.TB_MaxExp (edge1, area DESC)
INCLUDE (edge2);
```

44 MAX() 함수의 성능 개선 - 2

MAX() 함수를 사용했을 때 NULL 값에 대한 처리 과정으로 인해 불필요한 I/O가 발생된 사례이다.

1. 사전 구성 스크립트

```
SELECT CASE WHEN a.number<10 THEN '가'
        ELSE '나' END AS A
      , ROW_NUMBER() OVER (ORDER BY a.number) AS B
INTO TB_TOP
FROM master..spt_values a
JOIN master..spt_values b
ON a.type = 'P' AND b.type = 'P' AND a.number <= 100 AND b.number <= 1000

CREATE INDEX NIDX01_TOP ON TB_TOP(A,B)
GO
```

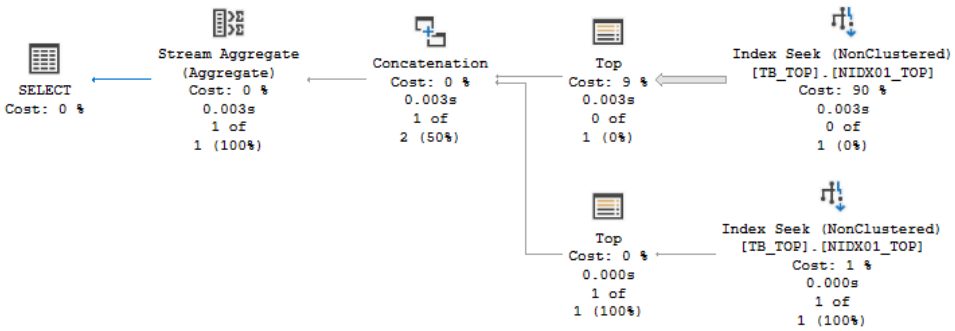
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT A,MAX(B) AS 'MAX(B)'
FROM TB_TOP
WHERE A = '나'
GROUP BY A
GO
```

· 실행 계획

Rows	Executes	StmtText
1 1 1 0 0 1 1	1 1 1 1 1 1 1	SELECT A,MAX(B) AS 'MAX(B)' FROM TB_TOP WHERE A = '나' GROUP BY A |--Stream Aggregate(DEFINE:([Expr1003]=MAX([Tuning].[dbo].[TB_TOP].[B]), [Tuning].[dbo].[TB_TOP] .[A]=ANY([Tuning].[dbo].[TB_TOP].[A]))) |--Concatenation |--Top(TOP EXPRESSION:((1))) | |--Index Seek(OBJECT:([Tuning].[dbo].[TB_TOP].[NIDX01_TOP]), SEEK:([Tuning].[dbo].[T B_TOP].[A]='나'), WHERE:([Tuning].[dbo].[TB_TOP].[B] IS NULL) ORDERED Backward) |--Top(TOP EXPRESSION:((1))) | |--Index Seek(OBJECT:([Tuning].[dbo].[TB_TOP].[NIDX01_TOP]), SEEK:([Tuning].[dbo].[T B_TOP].[A]='나'), WHERE:([Tuning].[dbo].[TB_TOP].[B] IS NOT NULL) ORDERED Backward)




```
테이블 'TB_TOP'. 스캔 수 2, 논리적 읽기 323
SQL Server 실행 시간:
CPU 시간 = 15ms, 경과 시간 = 8ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_TOP	NIDX01_TOP	NONCLUSTERED	0	A,B	-

4. [AS-IS] 구문 설명

[TB_TOP] 테이블의 [A] 컬럼이 '나' 인 데이터 중에서 [B] 컬럼의 최대값을 구하는 구문이다. 테이블의 [B] 컬럼은 NULL 값이 허용되어 있기 때문에 IS NULL과 IS NOT NULL 두 경우를 각각 실행하여 병합한다. 각각의 가장 큰 데이터 1건을 찾기 위해 인덱스를 Backward 방향으로 스캔하는데 NULL 값은 인덱스 내에서 가장 선두에 존재하기 때문에 IS NULL 조건에 해당되는 데이터를 찾는 과정에서 [A] = '나'로 탐색된 데이터를 모두 읽게 된다. 실제 NULL 값이 존재하지는 않지만 IS NULL 조건인 경우에 대한 처리로 인해 불필요한 I/O가 생기게 된다.

5. [TO-BE] 튜닝 포인트

다음 구문을 개선하기 위해 두 가지 방안을 제시한다.
첫째, [B] 컬럼에 대한 IS NOT NULL 조건을 명시한다.
둘째, [B] 컬럼에 NOT NULL 제약조건을 적용한다.

첫 번째 방법은 조건에 IS NOT NULL을 명시하여 NULL 값의 존재 유무를 고려하지 않도록 제외시킨다. 두 번째 방법은 IS NOT NULL 제약조건으로 NULL에 대한 불필요한 처리를 제거한다.

6. [TO-BE] 개선 방안

1) 조건절에 IS NOT NULL을 명시한다.

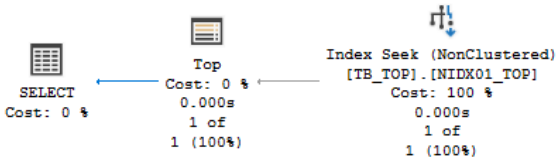
[B] 컬럼에 대한 IS NOT NULL 조건을 명시하여 IS NULL 처리 과정을 제거한다.

■ 변경된 SQL 구문

```
SELECT A,MAX(B) AS 'MAX(B)'
FROM TB_TOP
WHERE A = '나' AND B IS NOT NULL
GROUP BY A
GO
```

· 실행 계획

Rows	Executes	StmtText
1 1 1	1 1 1	SELECT A,MAX(B) AS 'MAX(B)' FROM TB_TOP WHERE A = '나' AND B IS NOT NULL GROUP BY A ¦--Top(TOP EXPRESSION:((1))) ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_TOP].[NIDX01_TOP]), SEEK:([Tuning].[dbo].[TB_TOP].[A]='나' AND [Tuning].[dbo].[TB_TOP].[B] IsNotNull) ORDERED Backward)




```
테이블 'TB_TOP'. 스캔 수 1, 논리적 읽기 3
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

2) [B] 컬럼에 NOT NULL 제약조건을 적용한다.

[B] 컬럼에 NOT NULL 제약조건을 적용하여 NULL에 대한 고려 없이 [A] 컬럼이 '나' 인 것 중 가장 큰 [B] 값을 TOP Expression 처리하도록 한다.

■ 제약조건 추가

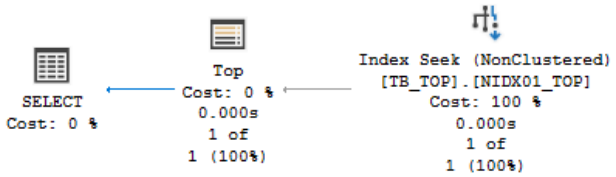
```
DROP INDEX NIDX01_TOP ON TB_TOP
ALTER TABLE TB_TOP ALTER COLUMN B INT NOT NULL
CREATE INDEX NIDX01_TOP ON TB_TOP(A,B)
GO
```

■ 변경된 SQL 구문

```
SELECT A,MAX(B) AS 'MAX(B)'
FROM TB_TOP
WHERE A = '나'
GROUP BY A
GO
```

· 실행 계획

Rows	Executes	StmtText
1 1 1	1 1 1	SELECT A,MAX(B) AS 'MAX(B)' FROM TB_TOP WHERE A = '나' GROUP BY A ¦--Top(TOP EXPRESSION:((1))) ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_TOP].[NIDX01_TOP]), SEEK:([Tuning].[dbo].[TB_TOP].[A]='나') ORDERED Backward)




```
테이블 'TB_TOP'. 스캔 수 1, 논리적 읽기 3
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	15	8	323
TO-BE (1)	0	0	3
TO-BE (2)	0	0	3

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

44. MAX() 함수의 성능 개선 - 2

1. 프롬프트

> [AS-IS]
> 구문 실행 시 [B] 컬럼이 NULL을 허용함에 따라 IS NULL과 IS NOT NULL 조건이
> 각각 수행되어 불필요한 반복 연산이 발생한다. 이에 대한 원인 분석과 성능 개선안을
> 알려줘.

```
[AS-IS]
SELECT A,MAX(B) AS 'MAX(B)'
FROM TB_TOP
WHERE A = '나'
GROUP BY A;
```

2. 분석 및 주요 최적화

1) 현재 상태

항목	값
테이블	dbo.TB_TOP
인덱스	NIDX01_TOP (A, B) - Nonclustered
B 컬럼	bigint, NULL 허용
A='나' 건수	91,091건
B IS NULL 건수	0건 (실제 NULL 없음)

2) 문제 원인

· B 컬럼이 NULL 허용이기 때문에, MAX(B) 연산 시 SQL Server 옵티마이저는 다음 두 가지 경우를 모두 탐색하는 실행 계획을 생성합니다.

· B IS NOT NULL → 최댓값 탐색

· B IS NULL → 모든 값이 NULL인 경우 대비

*이로 인해 인덱스를 두 번 탐색하는 비효율이 발생합니다 (Concatenation + 2개의 Index Seek/Scan).

3. 개선안

```
-- 방안 1. WHERE 조건에 B IS NOT NULL 추가 (가장 간단)

SELECT A, MAX(B) AS 'MAX(B)'
FROM dbo.TB_TOP
WHERE A = '나'
AND B IS NOT NULL
GROUP BY A;
```

*B IS NOT NULL 조건을 명시하면 옵티마이저가 NULL 분기를 제거하고, 인덱스 NIDX01_TOP(A, B)에서 **단일 Seek(역방향)**로 최댓값을 바로 찾습니다.

```
-- 방안 2. 컬럼 NOT NULL 제약 추가 (근본적 해결)

ALTER TABLE dbo.TB_TOP ALTER COLUMN B bigint NOT NULL;
```

*실제 데이터에 NULL이 0건이므로, 비즈니스 로직상 허용된다면 컬럼 자체를 NOT NULL로 변경하는 것이 근본적인 해결책입니다.

45 MIN() 함수의 성능 개선

MIN() 함수를 사용했을 때 NULL 값에 대한 처리 과정으로 인해 불필요한 I/O가 발생된 사례이다.

1. 사전 구성 스크립트

```
SELECT A, ROW_NUMBER() OVER(PARTITION BY A ORDER BY number) AS B
INTO TB_TOP02
```

```
FROM (SELECT B.number, CASE WHEN B.number % 2 = 0 THEN '가' ELSE '나' END AS A
      FROM master..spt_values A, (SELECT number
                                  FROM master..spt_values
                                  WHERE type = 'P' AND number < 15) B
      WHERE A.type = 'P'
) A
CREATE INDEX NIDX01_TOP02 ON TB_TOP02(A,B)
GO
```

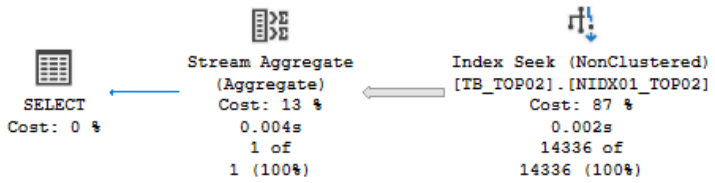
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT A, MIN(B) AS 'MIN(B)'
FROM TB_TOP02
WHERE A = '나'
GROUP BY A
GO
```

· 실행 계획

Rows	Executes	StmtText
1	1	SELECT A, MIN(B) AS 'MIN(B)' FROM TB_TOP02 WHERE A = '나' GROUP BY A
1	1	¦--Stream Aggregate(DEFINE:([Expr1003]=MIN([Tuning].[dbo].[TB_TOP02].[B]), [Tuning].[dbo].[TB_TOP02].[A]=ANY([Tuning].[dbo].[TB_TOP02].[A]))
14336	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_TOP02].[NIDX01_TOP02]), SEEK:([Tuning].[dbo].[TB_TOP02].[A]='나') Ordered Forward)



테이블 'TB_TOP02'. 스캔 수 1, 논리적 읽기 53
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 8ms

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_TOP02	NIDX01_TOP02	NONCLUSTERED	0	A, B	-

4. [AS-IS] 구문 설명

[TB_TOP02] 테이블의 [A] 컬럼이 '나' 인 데이터 중 [B] 컬럼 최소값을 구하는 구문이다. [B] 컬럼이 인덱스 키 값에 포함되어 있기 때문에 정렬되어 있음에도 TOP Expression으로 수행되지 않아 인덱스를 통해 조건절로 탐색된 데이터를 전부 읽게 된다.

5. [TO-BE] 튜닝 포인트

[B] 컬럼은 NULL 값이 허용되어 있기 때문에 인덱스를 Ordered Forward로 읽어내면서 상위 1건만 읽고 제일 작은 값임을 판별할 수가 없다. [B] IS NOT NULL 조건을 명시하면 인덱스 탐색을 통해 NULL인 데이터를 배제할 수 있고, 상위 1건만으로도 제일 작은 값을 판별할 수 있기 때문에 TOP Expression으로 수행할 수 있다.

다음 구문을 개선하기 위해 두 가지 방안을 고려할 수 있다.
첫째, [B] 컬럼에 대한 IS NOT NULL 조건을 명시한다.
둘째, [B] 컬럼에 NOT NULL 제약조건을 적용한다.

첫 번째 방법은 조건에 IS NOT NULL을 명시하여 NULL 값의 존재 유무를 고려하지 않도록 제외시킨다. 두 번째 방법은 IS NOT NULL 제약조건으로 NULL에 대한 불필요한 처리를 제거한다.

6. [TO-BE] 개선 방안

1) 조건절에 IS NOT NULL을 명시한다.

[B] 컬럼에 대한 IS NOT NULL 조건을 명시하여 인덱스 탐색을 통해 NULL을 배제하고 TOP Expression으로 1건의 데이터만 탐색하도록 개선한다.

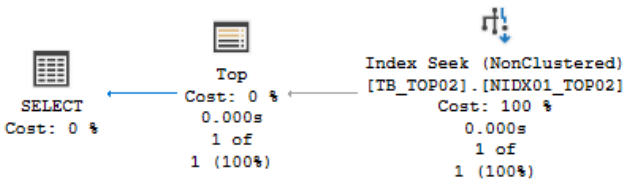
■ 변경된 SQL 구문

```
SELECT A, MIN(B) AS 'MIN(B)'
FROM TB_TOP02
WHERE A = '나' AND B IS NOT NULL
GROUP BY A
GO
```

· 실행 계획

Rows	Executes	StmtText

1
1
1
	1
	1
	1
		SELECT A, MIN(B) AS 'MIN(B)' FROM TB_TOP02 WHERE A = '나' AND B IS NOT NULL GROUP BY A
		  ¦--Top(TOP EXPRESSION:((1)))
		  ¦ ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_TOP02].[NIDX01_TOP02]), SEEK:([Tuning].[dbo].[TB_TOP 02].[A]='나' AND [Tuning].[dbo].[TB_TOP02].[B] IsNotNull) Ordered Forward)




```
테이블 'TB_TOP02'. 스캔 수 1, 논리적 읽기 2
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

2) 컬럼에 NOT NULL 제약조건을 적용한다.

[B] 컬럼에 NOT NULL 제약조건을 적용하여 NULL에 대한 고려 없이 [A] 컬럼이 '나' 인 것 중 가장 작은 [B] 컬럼 값을 TOP Expression으로 수행한다.

■ 제약조건 추가

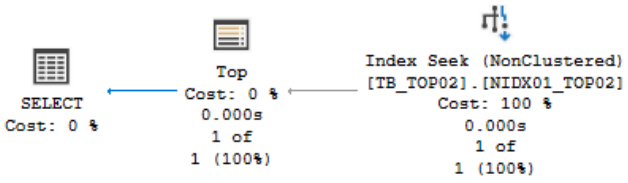
```
DROP INDEX NIDX01_TOP02 ON TB_TOP02
ALTER TABLE TB_TOP02 ALTER COLUMN B INT NOT NULL
CREATE INDEX NIDX01_TOP02 ON TB_TOP02(A,B)
GO
```

■ SQL 구문

```
SELECT A, MIN(B) AS 'MIN(B)'
FROM TB_TOP02
WHERE A = '나'
GROUP BY A
GO
```

· 실행 계획

Rows	Executes	StmtText
1 1 1	1 1 1	SELECT A, MIN(B) AS 'MIN(B)' FROM TB_TOP02 WHERE A = '나' GROUP BY A ¦--Top(TOP EXPRESSION:((1))) ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_TOP02].[NIDX01_TOP02]), SEEK:([Tuning].[dbo].[TB_TOP02].[A]='나') Ordered Forward)




```
테이블 'TB_TOP02'. 스캔 수 1, 논리적 읽기 2
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	0	8	53
TO-BE (1)	0	0	2
TO-BE (2)	0	0	2

AI

GitHub Copilot 채팅 결과

Claude Opus 4.6

45. MIN() 함수의 성능 개선

1. 프롬프트

> [AS-IS]
> 구문 실행 시 [B] 컬럼이 NULL을 허용함에 따라 IS NULL과 IS NOT NULL 조건이
> 각각 수행되어 불필요한 반복 연산이 발생한다. 이에 대한 원인 분석과 성능 개선안을
> 알려줘.

```
[AS-IS]
SELECT A, MIN(B) AS 'MIN(B)'
FROM TB_TOP02
WHERE A = '나'
GROUP BY A;
```

2. 분석 및 주요 최적화

1) 현재 테이블/인덱스 구조

항목	값
테이블	dbo.TB_TOP02
컬럼 A	varchar, NOT NULL
컬럼 B	bigint, NULL 허용
인덱스	NIDX01_TOP02 (A ASC, B ASC) – NONCLUSTERED
A='나' 행 수	14,336건
B가 NULL인 행	0건

2) 왜 성능 문제가 발생하는가?

MIN(B) + GROUP BY A는 옵티마이저가 이상적으로는 인덱스를 Ordered Forward 스캔하여 첫번째 NOT NULL 행 1건만 읽으면 최솟값을 구할 수 있습니다 (Top 1 + MIN 최적화, 일명 MIN/MAX 최적화). 그러나 B 컬럼이 NULL 허용이기 때문에 인덱스 정렬 순서상 NULL 값이 가장 앞에 위치합니다 (SQL Server에서 NULL은 가장 작은 값으로 취급). MIN(B)는 NULL을 무시해야 하므로, 옵티마이저는 첫 번째 행이 MIN 결과라고 확신할 수 없습니다. 결과적으로 Top Expression 최적화가 불가능해지고, A='나'에 해당하는 14,336건 전체를 Stream Aggregate로 읽게 됩니다.

*현재 실제 NULL 데이터가 0건이지만, 컬럼 정의 자체가 NULL 허용이므로 옵티마이저는 이를 보장할 수 없어 최적화를 적용하지 않습니다.

3. 개선안

```
-- 방안 1. 쿼리 레벨 – TOP 1 + ORDER BY로 변환 (즉시 적용 가능)
-- NULL을 명시적으로 제외하고 TOP 1을 사용하면, 옵티마이저가 인덱스를
-- Ordered Forward로 1건만 읽을 수 있습니다

SELECT '나' AS A,
       (SELECT TOP 1 B
        FROM dbo.TB_TOP02
        WHERE A = '나' AND B IS NOT NULL
        ORDER BY B ASC) AS [MIN(B)];
```

*인덱스 NIDX01_TOP02를 Seek 후 1건만 읽고 바로 반환. I/O가 14,336 → 1~2로 대폭 감소.

```
-- 방안 2: 스키마 레벨 – B 컬럼에 NOT NULL 제약 추가
-- 실제로 NULL 데이터가 0건이므로, B 컬럼을 NOT NULL로 변경하면 원본 쿼리 그대로도
-- MIN/MAX 최적화가 적용됩니다.

ALTER TABLE dbo.TB_TOP02
ALTER COLUMN B bigint NOT NULL;
```

■ 방안 비교

방안	변경 범위	예상 I/O	원본 쿼리 수정 필요	리스크
방안 1: 쿼리 변화	쿼리만	1~2 페이지	✅ 예	낮음
방안 2: NOT NULL 제약	DDL 변경	1~2 페이지	❌ 아니오	중간 (앱 영향 확인 필요)

46 NULL 값을 포함하지 않는 함수 개선

NULL 값을 포함하지 않도록 의도된 함수를 사용할 때 스캔 범위에 NULL 값이 포함되어 불필요한 I/O를 발생시키는 사례이다.

1. 사전 구성 스크립트

```
SELECT ROW_NUMBER() OVER (ORDER BY a.number) seq
      , CASE WHEN a.number = 1 THEN LEFT(a.type, 1)
        ELSE NULL END txt
INTO TB_Cnt
FROM master..spt_values a , master..spt_values b
WHERE a.TYPE= 'P'
AND a.number BETWEEN 1 AND 1000
AND b.type = 'P'
AND b.number BETWEEN 1 AND 1000

CREATE INDEX NIDX01_Cnt ON TB_Cnt (txt)
GO
```

2. [AS-IS] SQL 구문 및 실행 계획

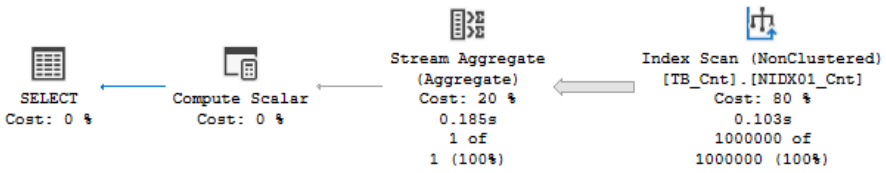
■ SQL 구문

```
SELECT COUNT(txt) AS [count]
FROM TB_Cnt
GO
```

· 실행 계획

Rows	Executes	StmtText

1
0
1
1000000
	1
	0
	1
	1
		SELECT COUNT(txt) AS [count] FROM TB_Cnt
		  ¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(int,[Expr1004],0)))
		  ¦--Stream Aggregate(DEFINE:([Expr1004]=COUNT([Tuning].[dbo].[TB_Cnt].[txt])))
		  ¦--Index Scan(OBJECT:([Tuning].[dbo].[TB_Cnt].[NIDX01_Cnt]))




```
경고: 집계 또는 다른 SET 작업에 의해 Null 값이 제거되었습니다.
테이블 'TB_Cnt'. 검색 수 1, 논리적 읽기 수 1738
SQL Server 실행 시간:
CPU 시간 = 172ms, 경과 시간 = 172ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Cnt	NIDX01_Cnt	NONCLUSTERED	0	txt	-

· [txt] 컬럼의 데이터 분포도

	txt	Count
1	NULL	999000
2	P	1000

4. [AS-IS] 구문 설명

[TB_Cnt] 테이블에서 [txt] 컬럼을 기준으로 NULL 값을 제외한 행 수를 출력하는 구문이다. [txt] 컬럼의 100만 건 데이터 중 NULL이 아닌 값은 1,000건에 해당되는데, 최종 출력에 필요한 데이터는 1,000건임에도 100만 건의 데이터를 모두 읽어내기 때문에 불필요한 I/O가 발생된다.

5. [TO-BE] 튜닝 포인트

행 수를 출력하기 위해 COUNT(컬럼명)을 사용할 때는 NULL 값을 제외한 행의 수를 출력하기 때문에 모든 데이터를 읽어낼 필요가 없다. 함수에 사용된 [txt] 컬럼이 키로 구성된 인덱스가 존재하기 때문에 조건절을 추가하여 NULL이 아닌 값을 인덱스로 탐색한다면 불필요한 I/O 비용을 제거할 수 있다.

6. [TO-BE] 개선 방안

1) IS NOT NULL 조건을 추가하여 인덱스 탐색을 수행하도록 변경

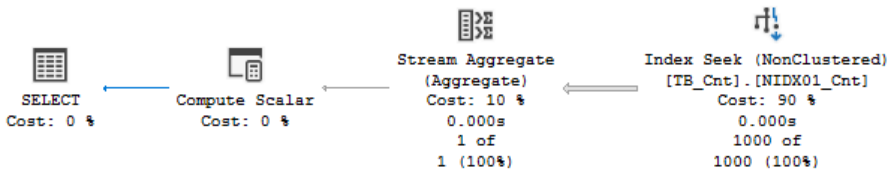
NULL이 아닌 데이터 1,000건만 읽어낼 수 있도록 [txt] 컬럼에 IS NOT NULL 조건절을 추가한다. 최종 결과에 필요한 데이터만을 미리 탐색하기 때문에 I/O 비용이 개선된다.

■ SQL 구문

```
SELECT COUNT(txt) AS [count]
FROM TB_Cnt
WHERE txt IS NOT NULL
GO
```

· 실행 계획

Rows	Executes	StmtText
1 0 1 1000	1 0 1 1	SELECT COUNT(txt) AS [count] FROM TB_Cnt WHERE txt IS NOT NULL ¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(int,[Expr1004],0))) ¦--Stream Aggregate(DEFINE:([Expr1004]=Count(*))) ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_Cnt].[NIDX01_Cnt]), SEEK:([Tuning].[dbo].[TB_Cnt].[txt] IsNotNull) Ordered Forward)




```
테이블 'TB_Cnt'. 검색 수 1, 논리적 읽기 수 6
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	172	172	1,738
TO-BE	0	0	6

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

46. NULL 값을 포함하지 않는 함수 개선

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT COUNT(txt) AS [count]
FROM TB_Cnt;
```

2. 분석 및 주요 최적화

1) 문제 분석

항목	내용
테이블	dbo.TB_Cnt (Heap – 클러스터드 인덱스 없음)
컬럼	seq (bigint), txt (nvarchar)
인덱스	NIDX01_Cnt – txt 컬럼 비클러스터드 (필터 없음)
데이터 분포	100만 건 중 txt IS NOT NULL은 1,000건 (0.1%)

2) 왜 느린가?

COUNT(txt)는 txt가 NULL이 아닌 행만 카운트합니다. 현재 NIDX01_Cnt 인덱스에는 NULL 값을 포함한 100만 건 전체가 들어 있으므로, SQL Server는 인덱스 전체를 스캔하면서 NULL 여부를 판별해야 합니다.

*실제 필요한 건 1,000건인데, 100만 건을 모두 읽는 불필요한 I/O가 발생합니다.

3. 개선안

```
-- 성능 개선: 필터드 인덱스 (Filtered Index)
-- txt IS NOT NULL 조건의 필터드 인덱스를 생성하면,
-- 인덱스에는 NULL이 아닌 1,000건만 저장됩니다.
-- SQL Server는 이 작은 인덱스만 스캔하여 결과를 반환합니다.


-- 1) 기존 인덱스 삭제 (필터 없는 전체 인덱스)

DROP INDEX IF EXISTS NIDX01_Cnt ON dbo.TB_Cnt;


-- 2) 필터드 인덱스 생성 (NULL이 아닌 1,000건만 포함)

CREATE NONCLUSTERED INDEX NIDX01_Cnt
ON dbo.TB_Cnt (txt)
WHERE txt IS NOT NULL;


-- 3) 쿼리 수정 (WHERE 절 추가)
-- 필터드 인덱스가 사용되려면 쿼리에도 동일한 필터 조건이 있어야 합니다:

SELECT COUNT(txt) AS [count]
FROM dbo.TB_Cnt
WHERE txt IS NOT NULL;
```

47 데이터 형식 우선순위에 의한 암시적 CONVERT

두 테이블을 조인하는 과정에서 후행 테이블의 조인 조건 컬럼에 암시적 형 변환이 발생되어 성능이 저하되는 사례이다.

1. 사전 구성 스크립트

```
CREATE TABLE TB_Main (seq INT
                    , main_code UNIQUEIDENTIFIER
                    , date DATE )
CREATE TABLE TB_Sub (value INT NOT NULL
                   , sub_code NVARCHAR(50)
                   , date DATE
                   , txt UNIQUEIDENTIFIER )

INSERT INTO TB_Main
SELECT ROW_NUMBER() OVER (ORDER BY a.number) AS seq
      , NEWID() AS main_code
      , DATEADD(d, a.number, '2020-01-01') date
FROM master..spt_values a (NOLOCK) , (SELECT number
                                      FROM master..spt_values (NOLOCK)
                                      WHERE type = 'p' AND number < 20) b
WHERE type = 'p'
AND a.number <= 365

INSERT INTO TB_Sub
SELECT ISNULL(CONVERT(INT, RIGHT(a.seq * b.high * b.number, 2)), 0) AS value
      , CONVERT(NVARCHAR(50),a.main_code) AS sub_code
      , GETDATE() AS date
      , NEWID() AS txt
FROM TB_Main a, master..spt_values (NOLOCK) b
WHERE b.type = 'P'
AND b.number BETWEEN 1 AND 10

CREATE CLUSTERED INDEX CIDX_Main ON TB_Main(date)
CREATE INDEX NIDX01_Sub ON TB_Sub(sub_code,value)
GO
```

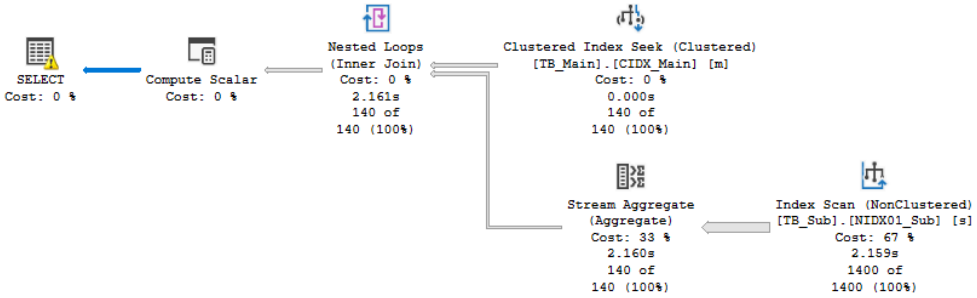
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT m.seq, m.main_code, (SELECT MAX(value)
                            FROM TB_Sub s
                            WHERE s.sub_code = m.main_code) AS value
FROM TB_Main m
WHERE date BETWEEN '2020-01-01' AND '2020-01-07'
GO
```

· 실행 계획

Rows	Executes	StmtText
140	1	SELECT m.seq,m.main_code,(SELECT MAX(value) FROM TB_Sub s WHERE s.sub_code = m.main_code) AS value FROM TB_Main m WHERE date BETWEEN '2020-01-01' AND '2020-01-07'
0	0	¦--Compute Scalar(DEFINE:([Expr1006]=[Expr1004]))
140	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([m].[main_code]))
140	1	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Main].[CIDX_Main] AS [m]), SEEK:([m].[date] >= '2020-01-01 00:00:00.000' AND [m].[date] <= '2020-01-07 00:00:00.000') Ordered Forward)
140	140	¦--Stream Aggregate(DEFINE:([Expr1004]=MAX([Tuning].[dbo].[TB_Sub].[value] as [s].[value])))
1400	140	¦--Index Scan(OBJECT:([Tuning].[dbo].[TB_Sub].[NIDX01_Sub] AS [s]), WHERE:(CONVERT_IMPLICIT(uniqueidentifier,[Tuning].[dbo].[TB_Sub].[sub_code] as [s].[sub_code],0)=[Tuning].[dbo].[TB_Main].[main_code] as [m].[main_code]))




```
테이블 'TB_Sub'. 검색 수 140, 논리적 읽기 수 121100
테이블 'TB_Main'. 검색 수 1, 논리적 읽기 수 2
SQL Server 실행 시간:
CPU 시간 = 2531ms, 경과 시간 = 2678ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Main	CIDX_Main	CLUSTERED	0	date	-
TB_Sub	NIDX01_Sub	NONCLUSTERED	0	sub_code, value	-

· 데이터 타입 우선순위

우선순위	데이터 형식	우선순위	데이터 형식	우선순위	데이터 형식
1	sql_variant	11	decimal	21	image
2	xml	12	money	22	timestamp
3	datetimeoffset	13	smallmoney	23	uniqueidentifier
4	datetime2	14	bigint	24	nvarchar(nvarchar(max) 포함)
5	datetime	15	int	25	nchar
6	smalldatetime	16	smallint	26	varchar(varchar(max) 포함)
7	date	17	tinyint	27	char
8	time	18	bit	28	varbinary(varbinary(max) 포함)
9	float	19	ntext	29	binary(가장 낮음)
10	real	20	text	-	-

4. [AS-IS] 구문 설명

[TB_Main] 테이블의 [date] 컬럼이 '2020-01-01'부터 일주일간 해당되는 데이터를 서브쿼리 [TB_Sub] 테이블에 조인하여 MAX(value) 값을 함께 출력하는 구문이다. 후행 테이블 [TB_Sub]에는 조인 조건에 해당하는 [sub_code] 컬럼과 Max 값을 찾아내기 위한 [value] 컬럼이 인덱스로 생성되어 있다. 후행 테이블에 인덱스가 존재함에도 인덱스 탐색을 하지 못하고 선행 테이블 결과 집합 140건에 대해 인덱스 전체 스캔을 반복하여 불필요한 I/O가 발생되는 현상이다.

5. [TO-BE] 튜닝 포인트

인덱스를 탐색하지 못하고 전체를 스캔하는 이유는 [sub_code] 컬럼에 CONVERT_IMPLICIT가 발생됐기 때문이다. 구문이 실행될 때 두 테이블을 조인하기 위해 조인 조건 컬럼의 데이터 타입을 동일한 형태로 변경하는데, [sub_code] (Nvarchar) 컬럼이 [main_code] (Uniqueidentifier) 컬럼보다 타입 우선순위가 낮기 때문에 [sub_code] 컬럼에 형 변환이 발생되는 것이다. 암시적 형 변환이 발생되지 않도록 [main_code] 컬럼의 데이터 타입을 명시적으로 변환하면 인덱스 탐색이 가능해지면서 I/O를 개선할 수 있다.

6. [TO-BE] 개선 방안

1) [main_code] 컬럼의 데이터 타입을 명시적으로 변경

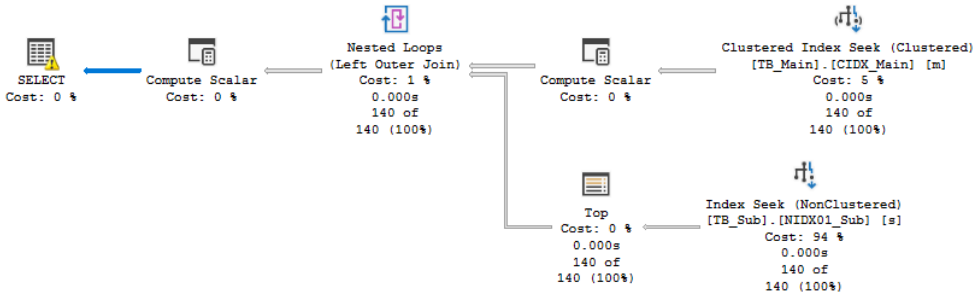
[sub_code] 컬럼에 암시적 형 변환이 발생되지 않도록 [main_code] 컬럼을 Nvarchar 타입으로 명시적 변경한다. 인덱스를 통해 상위 1건의 데이터만을 탐색하는 비용만 발생하기 때문에 많은 I/O를 개선할 수 있다.

■ 변경된 SQL 구문

```
SELECT m.seq, m.main_code, (SELECT MAX(value)
                            FROM TB_Sub s
                            WHERE s.sub_code=CONVERT(NVARCHAR(50), m.main_code)
                            ) AS value
FROM TB_Main m
WHERE date BETWEEN '2020-01-01' AND '2020-01-07'
GO
```

· 실행 계획

Rows	Executes	StmtText
140	1	SELECT m.seq,m.main_code,(SELECT MAX(value) FROM TB_Sub s WHERE s.sub_code = CONVERT(NVARCHAR(50),m.main_code)) AS value FROM TB_Main m WHERE date BETWEEN '2020-01-01' AND '2020-01-07'
0	0	¦--Compute Scalar(DEFINE:([Expr1006]=[Tuning].[dbo].[TB_Sub].[value] as [s].[value]))
140	1	¦--Nested Loops(LEFT OUTER JOIN, OUTER REFERENCES:([Expr1007]))
0	0	¦--Compute Scalar(DEFINE:([Expr1007]=CONVERT(nvarchar(50),[Tuning].[dbo].[TB_Main].[main_code] as [m].[main_code],0)))
140	1	¦ ¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_Main].[CIDX_Main] AS [m]), SEEK:([m].[date] >= '2020-01-01 00:00:00.000' AND [m].[date] <= '2020-01-07 00:00:00.000') Ordered Forward)
140	140	¦--Top(TOP EXPRESSION:((1)))
140	140	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_Sub].[NIDX01_Sub] AS [s]), SEEK:([s].[sub_code]=[Expr1007]) ORDERED Backward)




```
테이블 'TB_Sub'. 검색 수 140, 논리적 읽기 수 420
테이블 'TB_Main'. 검색 수 1, 논리적 읽기 수 2
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	2,531	2,678	121,102
TO-BE	0	0	422

GitHub Copilot 채팅 결과

Claude Opus 4.6

47. 데이터 형식 우선순위에 의한 암시적 CONVERT

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생하는 원인을 분석하여 개선해줘.

```
[AS-IS]
SELECT m.seq, m.main_code, (SELECT MAX(value)
                            FROM TB_Sub s
                            WHERE s.sub_code = m.main_code) AS value
FROM TB_Main m
WHERE date BETWEEN '2020-01-01' AND '2020-01-07';
```

2. 분석 및 주요 최적화

1) 현재 테이블 구조

테이블	컬럼	데이터 타입	행 수
dbo.TB_Main	main_code	uniqueidentifier	7,320
dbo.TB_Sub	sub_code	nvarchar	73,200

2) 인덱스 현황

테이블	인덱스명	유형	키 컬럼
dbo.TB_Main	CIDX_Main	Clustered	date
dbo.TB_Sub	NIDX01_Sub	Nonclustered	sub_code, value

3) 암시적 형변환 (CONVERT_IMPLICIT) — 가장 심각

main_code는 uniqueidentifier, sub_code는 nvarchar입니다. WHERE s.sub_code = m.main_code 비교 시 SQL Server가 암시적 형변환을 수행합니다. uniqueidentifier가 nvarchar보다 데이터 형식 우선 순위가 높으므로, sub_code 컬럼에 CONVERT(uniqueidentifier, sub_code) 형태의 변환이 발생합니다. 이로 인해 NIDX01_Sub 인덱스의 Index Seek가 불가능해지고, Index Scan으로 전환됩니다. TB_Main의 매 행(row)마다 TB_Sub 73,200건을 Full Scan 하게 됩니다.

4) 상관 서브쿼리 (Correlated Subquery)

SELECT 절의 스칼라 서브쿼리는 TB_Main의 각 행마다 반복 실행됩니다. 옵티마이저가 경우에 따라 JOIN으로 변환하지만, 암시적 형변환과 결합되면 최적화가 어렵습니다.

3. 개선안

```
-- 최적화 스크립트 (TO-BE)

SELECT m.seq, m.main_code, s.max_value AS value
FROM dbo.TB_Main m
LEFT JOIN (SELECT sub_code, MAX(value) AS max_value
           FROM dbo.TB_Sub
           GROUP BY sub_code) s
ON s.sub_code = CONVERT(NVARCHAR(50), m.main_code)
WHERE m.date BETWEEN '2020-01-01' AND '2020-01-07';
```

■ 변경 사항 요약

항목	AS-IS	TO-BE	효과
조인 방식	상관 서브쿼리 (행마다 실행)	LEFT JOIN + 파생 테이블	TB_Sub를 1회만 집계
형변환	암시적 변환 (sub_code에 함수 적용)	CONVERT(NVARCHAR(50), m.main_code) 명시	변환을 main_code 쪽으로 이동 → NIDX01_Sub 인덱스 Seek 가능
스키마 표기	1-part name	2-part name (dbo.)	명확한 객체 참조

*형변환 방향이 중요합니다. CONVERT를 sub_code (인덱스가 있는 컬럼)에 적용하면 인덱스를 사용할 수 없습니다. 반면 main_code 쪽에 CONVERT를 적용하면 NIDX01_Sub 인덱스의 Seek가 유지됩니다.

48 함수를 이용한 문자열 분할 성능 개선

사용자 정의 함수(테이블 반환)를 사용하여 문자열 분할 시 성능 이슈 사례이다.

1. 사전 구성 스크립트

```
CREATE TABLE TB_SPLIT (Seq BIGINT, Code VARCHAR(100))

INSERT INTO TB_SPLIT
SELECT rownum, NEWID()
FROM (
```

```
      SELECT ROW_NUMBER() OVER (ORDER BY A.number) AS rownum, A.number
      FROM master..spt_values A
      JOIN master..spt_values B
      ON A.type = 'P' AND B.type = 'P' AND A.number <= 1000 AND B.number <= 100
) A

CREATE INDEX NIDX01_SPLIT ON TB_SPLIT(Code)
GO

/* 함수 생성문 */
CREATE FUNCTION [dbo].FN_SPLIT (
      @strList VARCHAR(MAX),
      @strDelimiter VARCHAR(10))
RETURNS @Array TABLE (Value varchar(4000))

BEGIN
IF @strList IS NULL RETURN

DECLARE @strLen int = LEN(@strList)
DECLARE @byLenDelimiter TINYINT

SET @byLenDelimiter = LEN(@strDelimiter)
SET @strList = @strList + @strDelimiter;

WITH WT_TRECT (RN, CUT, REMAIN) AS (
      SELECT 1 RN
           , SUBSTRING(@strList, 1, ABS(CHARINDEX (@strDelimiter, @strList) - 1)) AS
CUT
           , SUBSTRING(@strList, LEN(SUBSTRING(@strList,1
           , ABS(CHARINDEX(@strDelimiter, @strList)-1)))+1+@byLenDelimiter, @strLen
           ) AS REMAIN
      UNION ALL
      SELECT RN + 1 AS RN
           , SUBSTRING(REMAIN, 1, ABS(CHARINDEX(@strDelimiter, REMAIN) - 1)) AS CUT
           , SUBSTRING(REMAIN, LEN(SUBSTRING(REMAIN, 1, ABS(CHARINDEX(@strDelimiter,
             REMAIN)-1)))+1+ @byLenDelimiter, @strLen) AS REMAIN
      FROM WT_TRECT
      WHERE CUT <> @strDelimiter AND LEN(REMAIN) > 0
)
INSERT INTO @Array SELECT CUT FROM WT_TRECT WHERE LEN(CUT) > 0 ORDER BY RN
OPTION (maxrecursion 0)
RETURN
END
GO
```

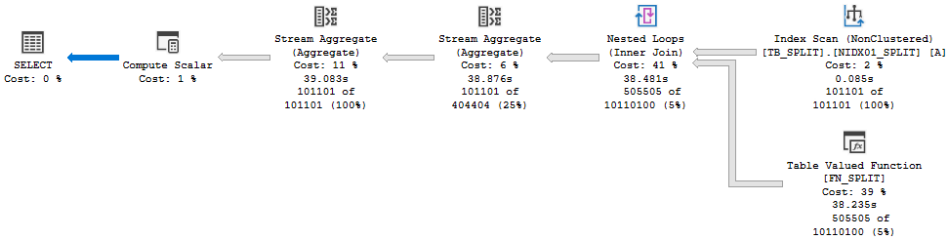
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT Code, COUNT(*) AS Cnt
FROM TB_SPLIT A CROSS APPLY FN_SPLIT(A.Code, '-')
GROUP BY Code
GO
```

· **실행 계획**

Rows	Executes	StmtText
101101	1	SELECT Code, COUNT(*) AS Cnt FROM TB_SPLIT A CROSS APPLY FN_SPLIT(A.Code, '-') GROUP BY Code
0	0	¦--Compute Scalar(DEFINE:([Expr1004]=CONVERT_IMPLICIT(int,[globalagg1007],0)))
101101	1	¦--Stream Aggregate(GROUP BY:([A].[Code]) DEFINE:([globalagg1007]=SUM([partialagg1006])))
101101	1	¦--Stream Aggregate(GROUP BY:([A].[Code]) DEFINE:([partialagg1006]=Count(*)))
505505	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([A].[Code]))
101101	1	¦--Index Scan(OBJECT:([Tuning].[dbo].[TB_SPLIT].[NIDX01_SPLIT] AS [A]), Ordered Forward)
505505	101101	¦--Table-valued function(OBJECT:([Tuning].[dbo].[FN_SPLIT]))




```
테이블 '#BB685E46'. 스캔 수 101101, 논리적 읽기 101101
테이블 'TB_SPLIT'. 스캔 수 1, 논리적 읽기 687
SQL Server 실행 시간:
CPU 시간 = 171515ms, 경과 시간 = 750824ms
```

· **SQL 구문 (Profiler)**

EventClass	TextData	CPU	Duration	Reads	RowCounts
SP:StmtCompleted	RETURN	0	0	0	0
SQL:BatchCompleted	SELECT Code, COUNT(*) AS Cnt FROM TB_SPL...	171515	750832	6168368	909909

```
SELECT Code, COUNT(*) AS Cnt
FROM TB_SPLIT A CROSS APPLY FN_SPLIT(A.Code, '-')
GROUP BY Code
```

3. [AS-IS] 통계 정보

· **인덱스 정보**

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_SPLIT	NIDX01_SPLIT	NONCLUSTERED	0	Code	-

4. [AS-IS] 구문 설명

[TB_SPLIT] 테이블을 [FN_SPLIT] 함수에 조인하여 반환되는 행 집합을 [Code] 값 별로 집계하여 행 수를 출력하는 구문이다. [FN_SPLIT] 함수는 입력되는 문자열을 특정 문자 기준으로 잘라내어 다중 행으로 반환하도록 재귀 질의로 작성되어 있다. [TB_SPLIT] 테이블 약 10만 건의 데이터에 대해 [FN_SPLIT] 함수를 반복하여 호출하고, 각 행마다 [FN_SPLIT] 함수 내에서 재귀 호출되면서 많은 I/O를 사용한다.

5. [TO-BE] 튜닝 포인트

[FN_SPLIT] 함수는 재귀 호출하면서 잘라내기 때문에 입력받은 문자열을 반복하여 읽어내면서 오랜 시간이 소요된다. SQL Server 2016 이상의 버전에서는 문자열을 적은 비용으로 처리할 수 있도록 시스템 함수가 제공되는데 구분 기호 문자가 단일 문자라면 시스템 함수를 사용하는 것으로 구문 변경을 고려할 수 있다.

6. [TO-BE] 개선 방안

1) STRING_SPLIT() 함수 사용으로 구문을 변경

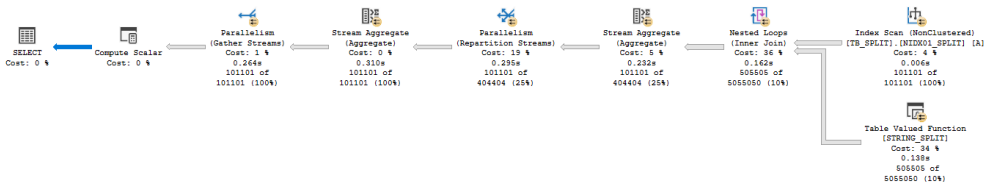
사용자 정의 함수 대신 STRING_SPLIT() 시스템 함수를 사용하는 구문으로 변경하여 재귀 호출에 의한 비용을 개선한다.

■ 변경된 SQL 구문

```
SELECT Code, COUNT(*) AS Cnt
FROM TB_SPLIT A CROSS APPLY STRING_SPLIT(A.Code, '-')
GROUP BY Code
GO
```

· 실행 계획

Rows	Executes	StmtText
101101	1	SELECT Code, COUNT(*) AS Cnt FROM TB_SPLIT A CROSS APPLY STRING_SPLIT(a.Code, '-') GROUP BY Code
0	0	¦--Compute Scalar(DEFINE:([Expr1002]=CONVERT_IMPLICIT(int,[globalagg1004],0)))
101101	1	¦--Parallelism(Gather Streams)
101101	4	¦--Stream Aggregate(GROUP BY:([A].[Code]) DEFINE:([globalagg1004]=SUM([partialagg1003])))
101101	4	¦--Parallelism(Repartition Streams, Hash Partitioning, PARTITION COLUMNS:([A].[Code]), ORDER BY:([A].[Code] ASC))
101101	4	¦--Stream Aggregate(GROUP BY:([A].[Code]) DEFINE:([partialagg1003]=Count(*)))
505505	4	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([A].[Code]))
101101	4	¦--Index Scan(OBJECT:([Tuning].[dbo].[TB_SPLIT].[NIDX01_SPLIT] AS [A]), Ordered Forward)
505505	101101	¦--Table-valued function




```
테이블 'TB_SPLIT'. 스캔 수 5, 논리적 읽기 727
SQL Server 실행 시간:
CPU 시간 = 1547ms, 경과 시간 = 1610ms
```

· SQL 구문 (Profiler)

EventClass	TextData	CPU	Duration	Reads	RowCounts
Showplan XML	<ShowPlanXML xmlns="http://schemas.microsoft...				
SQL:BatchCompleted	SELECT Code, COUNT(*) AS Cnt FROM TB_SPLIT ...	2046	1187	724	101101

```
SELECT Code, COUNT(*) AS Cnt
FROM TB_SPLIT A CROSS APPLY STRING_SPLIT(A.Code, '-')
GROUP BY Code
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	171,515	750,824	6,168,368
TO-BE	2,046	1,187	724

AI

GitHub Copilot 채팅 결과

Claude Opus 4.6

48. 함수를 이용한 문자열 분할 성능 개선

1. 프롬프트

> [AS-IS]
> 구문은 split 사용자 함수를 통해 code 값에서 '-'를 기준으로 분리하여 코드 값의 포함 횟수를 확인한다. 하지만 실행 시간이 지나치게 길어 성능 저하가 발생하고 있다. 이에 대해 사용자 함수, 구문, 통계를 종합적으로 분석하여 성능 개선안을 알려줘.
> * 함수 예시> 0A209131-58F0-4780-BFE1-49034BC6BE8B -> 5

```
[AS-IS]
SELECT Code, COUNT(*) AS Cnt
FROM TB_SPLIT A CROSS APPLY FN_SPLIT(A.Code, '-')
GROUP BY Code;
```

2. 분석 및 주요 최적화

1) 현재 환경 분석

항목	내용
테이블	dbo.TB_SPLIT (101,101건, HEAP - 클러스터형 인덱스 없음)
인덱스	NIDX01_SPLIT (Code 컬럼, 비클러스터)
통계	최신 상태 (수정 0건)
함수 유형	Multi-Statement TVF (재귀 CTE 사용)

2) 성능 저하 원인

1 Multi-Statement TVF (MSTVF): FN_SPLIT은 다중문 테이블 반환 함수입니다.

- SQL Server는 이를 **행 단위(row-by-row)**로 실행하며, 카디널리티 추정이 부정확합니다.

2 재귀 CTE: 함수 내부에서 재귀 CTE로 문자열을 분리합니다.

- 101,101건 × 각 행마다 재귀 극심한 오버헤드.

3 불필요한 행 확장: 단순히 구분자 개수(세그먼트 수)만 세면 되는데, 실제 문자열을 분리한 뒤 다시 GROUP BY로 집계하고 있습니다.

3. 개선안

```sql
-- STRING_SPLIT (SQL 2016+)
-- 내장 함수라 MSTVF보다 훨씬 효율적입니다.

SELECT A.Code, COUNT(*) AS Cnt
FROM dbo.TB_SPLIT A
CROSS APPLY STRING_SPLIT(A.Code, '-') S
GROUP BY A.Code;
```

Section 07

동시성 및 잠금

동시성 문제와 잠금은 성능 저하뿐만 아니라 장애로 이어질 수 있다. 이 섹션에서는 잠금, 블로킹, 데드락 등의 원인을 이해하고 이를 최소화하는 방법을 설명한다.

49 스캔 범위에 따른 블로킹 이슈 - 1

두 세션이 서로 다른 행을 참조함에도 클러스터형 인덱스 전체 스캔으로 인해 대기가 발생되는 사례이다.

1. 사전 구성 스크립트

```
CREATE TABLE TB_LOCK01 (NO INT, EmpID CHAR(900), NAME CHAR(900))

INSERT INTO TB_LOCK01 (NO, EmpID, NAME) VALUES (1, 1001, 'Ken')
INSERT INTO TB_LOCK01 (NO, EmpID, NAME) VALUES (2, 1002, 'Terri')
INSERT INTO TB_LOCK01 (NO, EmpID, NAME) VALUES (3, 1003, 'Roberto')
INSERT INTO TB_LOCK01 (NO, EmpID, NAME) VALUES (4, 1004, 'Rob')
INSERT INTO TB_LOCK01 (NO, EmpID, NAME) VALUES (5, 1101, 'Jossef')
INSERT INTO TB_LOCK01 (NO, EmpID, NAME) VALUES (6, 1102, 'Brian')
INSERT INTO TB_LOCK01 (NO, EmpID, NAME) VALUES (7, 1103, 'Michael')
INSERT INTO TB_LOCK01 (NO, EmpID, NAME) VALUES (8, 1104, 'Sharon')
INSERT INTO TB_LOCK01 (NO, EmpID, NAME) VALUES (9, 1201, 'David')
INSERT INTO TB_LOCK01 (NO, EmpID, NAME) VALUES (10, 1202, 'Kevin')
INSERT INTO TB_LOCK01 (NO, EmpID, NAME) VALUES (11, 1203, 'John')
INSERT INTO TB_LOCK01 (NO, EmpID, NAME) VALUES (12, 1204, 'Mary')

CREATE CLUSTERED INDEX CIDX_LOCK01 ON TB_LOCK01(EmpID)
CREATE INDEX NIDX01_LOCK01 ON TB_LOCK01(NO)
GO
```

2. [AS-IS] SQL 구문 및 실행 계획

- **SQL 구문**

<Session 54>	<Session 55>
BEGIN TRAN UPDATE TB_LOCK01 SET EmpID = '1501' FROM TB_LOCK01 WHERE EmpID = '1201'	SELECT NO, EmpID, NAME FROM TB_LOCK01 WHERE NO = 8

· 실행 계획

<Session 54>

Rows	Executes	StmtText
1 1	1 1	UPDATE TB_LOCK01 SET EmpID = '1501' FROM TB_LOCK01 WHERE EmpID = '1201' ¦--Clustered Index Update(OBJECT:([Tuning].[dbo].[TB_LOCK01].[CIDX_LOCK01]), OBJECT:([Tuning].[dbo].[TB_LOCK01].[NIDX01_LOCK01]), SET:([Tuning].[dbo].[TB_LOCK01].[EmpID] = [Expr1003]), DEFINE:([Expr1003]='1501'), WHERE:([Tuning].[dbo].[TB_LOCK01].[EmpID]='1201'))

테이블 'TB_LOCK01'. 스캔 수 1, 논리적 읽기 16

3. [AS-IS] 통계 및 Lock 정보

· 인덱스 정보

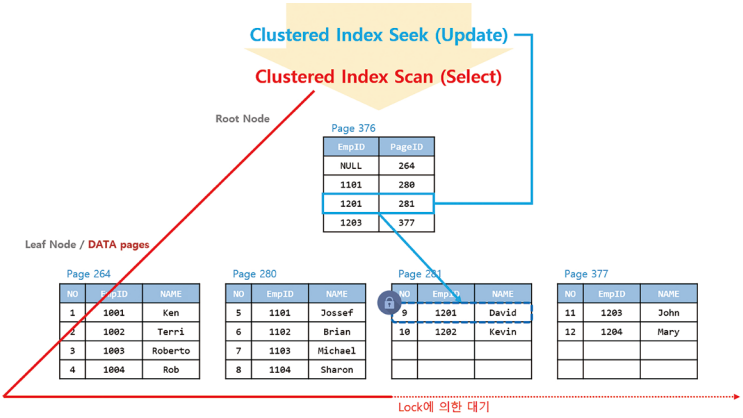
Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_LOCK01	CIDX_LOCK01	CLUSTERED	0	EmpID	-
TB_LOCK01	NIDX01_LOCK01	NONCLUSTERED	0	NO	-

· Lock 정보

spid	dbid	ObjId	IndId	Type	Resource	Mode	Status
54	25	0	0	DB		S	GRANT
54	25	1013578649	1	PAG	1:281	IX	GRANT
54	25	1013578649	1	PAG	1:280	IX	GRANT
54	25	1013578649	1	PAG	1:377	IX	GRANT
54	25	1013578649	1	KEY	(a52240a4b25f)	X	GRANT
54	25	1013578649	2	PAG	1:8480	IX	GRANT
54	25	1013578649	2	KEY	(9c4e8c5a017f)	X	GRANT
54	25	1013578649	0	TAB		IX	GRANT
54	25	1013578649	1	KEY	(ffaa72d2deb7)	X	GRANT
54	25	1013578649	2	KEY	(c6c6be2c6d97)	X	GRANT
55	25	0	0	DB		S	GRANT
55	25	1013578649	1	PAG	1:281	IS	GRANT
55	25	1013578649	1	KEY	(a52240a4b25f)	S	WAIT
55	25	1013578649	0	TAB		IS	GRANT

4. [AS-IS] 구문 설명

Session 54에서는 [EmpID] 컬럼이 1201인 데이터를 1501로 업데이트하는 구문이며, Session 55에서는 [NO] 컬럼이 8인 데이터의 [NO], [EmpID], [NAME] 컬럼을 조회하는 구문이다. 선행되는 Session 54에서 업데이트 구문에 의해 [EmpID] 컬럼이 1201인 데이터를 대상으로 배타 잠금(X)을 획득하게 되고 이후에 Session 55에서 조회할 때 공유 잠금(S)을 획득하기 위해 대기가 발생한다. 서로 다른 행을 참조하는데도 대기가 발생되는 이유는 Session 55에서 [NO] 컬럼에 인덱스가 존재함에도 [NAME] 컬럼에 의한 Key Lookup의 비용을 고려하여 클러스터형 인덱스를 전체 스캔했기 때문이다.



5. [TO-BE] 튜닝 포인트

다음 구문을 개선하기 위해 두 가지 방안을 고려할 수 있다.
첫째, 포괄 열이 있는 인덱스를 생성하여 클러스터형 인덱스에 액세스하지 않도록 한다.
둘째, WITH(NOLOCK) 힌트를 명시한다.

첫 번째 방법은 인덱스의 포괄 열에 [NAME] 컬럼을 추가하여 클러스터형 인덱스에 액세스하지 않고 비클러스터형 인덱스만을 액세스하여 블로킹을 방지하는 방법이다.

두 번째 방법은 SELECT 절에 WITH(NOLOCK) 힌트를 추가해 COMMIT 되지 않은 데이터를 읽도록 하여 블로킹을 방지하는 방법이다.

6. [TO-BE] 개선 방안

1) 포괄 열이 있는 인덱스를 생성하여 클러스터형 인덱스에 액세스하지 않도록 한다.

인덱스의 포괄 열에 [NAME] 컬럼을 추가하면 Session 55가 비클러스터형 인덱스만 액세스하기 때문에 Session 54와 서로 다른 인덱스를 참조하게 되어 블로킹을 방지할 수 있다.

■ 인덱스 생성 구문

```
CREATE INDEX NIDX02_LOCK01 ON TB_LOCK01(NO) INCLUDE(NAME)
GO
```

· SQL 구문

<Session 54>	<Session 55>
BEGIN TRAN UPDATE TB_LOCK01 SET EmpID = '1501' FROM TB_LOCK01 WHERE EmpID = '1201'	SELECT NO, EmpID, NAME FROM TB_LOCK01 WHERE NO = 8

· Lock 정보

spid	dbid	ObjId	IndId	Type	Resource	Mode	Status
54	25	0	0	DB		S	GRANT
54	25	1029578706	1	PAG	1:281	IX	GRANT
54	25	1029578706	1	PAG	1:280	IX	GRANT
54	25	1029578706	3	PAG	1:353	IX	GRANT
54	25	1029578706	1	PAG	1:377	IX	GRANT
54	25	1029578706	3	PAG	1:433	IX	GRANT
54	25	1029578706	1	KEY	(a52240a4b25f)	X	GRANT
54	25	1029578706	2	PAG	1:8480	IX	GRANT
54	25	1029578706	2	KEY	(9c4e8c5a017f)	X	GRANT
54	25	1029578706	3	KEY	(9c4e8c5a017f)	X	GRANT
54	25	1029578706	1	KEY	(ffaa72d2deb7)	X	GRANT
54	25	1029578706	0	TAB		IX	GRANT
54	25	1029578706	3	KEY	(c6c6be2c6d97)	X	GRANT
54	25	1029578706	2	KEY	(c6c6be2c6d97)	X	GRANT
55	25	0	0	DB		S	GRANT

· 실행 계획

<Session 54>

Rows	Executes	StmtText
1 1	1 1	UPDATE A SET NUM = 9 FROM TB_LOCK01 A WHERE NUM=10 ¦--Clustered Index Update(OBJECT:([Tuning].[dbo].[TB_LOCK01].[CIDX_LOCK01] AS [A]), OBJECT:([Tuning].[dbo].[TB_LOCK01].[NIDX01_LOCK01] AS [A]), SET:([Tuning].[dbo].[TB_LOCK01].[NUM] as [A].[NUM] = [Expr1002]), DEFINE:([Expr1002]=(9)), WHERE:([Tuning].[dbo].[TB_LOCK01].[NUM]=(10)))

테이블 'TB_LOCK01'. 스캔 수 1, 논리적 읽기 19

<Session 55>

Rows	Executes	StmtText
1 1	1 1	SELECT [NO],[TXT] FROM [TB_LOCK01] WHERE [NO]=@1 ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_LOCK01].[NIDX02_LOCK01]), SEEK:([Tuning].[dbo].[TB_LOCK01].[NO]=CONVERT_IMPLICIT(int,[@1],0)) Ordered Forward)

테이블 'TB_LOCK01'. 스캔 수 1, 논리적 읽기 3

2) WITH(NOLOCK) 힌트를 명시한다.

Session 55에 WITH(NOLOCK) 힌트를 명시하면 아직 COMMIT 되지 않은 데이터를 읽을 수 있도록 허용하기 때문에 Session 54의 업데이트 작업과 관계없이 블로킹을 방지할 수 있다.

· HINT가 적용된 SQL 구문

<Session 54>	<Session 55>
BEGIN TRAN UPDATE TB_LOCK01 SET EmpID = '1501' FROM TB_LOCK01 WHERE EmpID = '1201'	SELECT NO, EmpID, NAME FROM TB_LOCK01 WITH(NOLOCK) WHERE NO = 8

· **Lock 정보**

spid	dbid	ObjId	IndId	Type	Resource	Mode	Status
54	25	0	0	DB		S	GRANT
54	25	1013578649	1	PAG	1:281	IX	GRANT
54	25	1013578649	1	PAG	1:280	IX	GRANT
54	25	1013578649	1	PAG	1:377	IX	GRANT
54	25	1013578649	1	KEY	(a52240a4b25f)	X	GRANT
54	25	1013578649	2	PAG	1:8480	IX	GRANT
54	25	1013578649	2	KEY	(9c4e8c5a017f)	X	GRANT
54	25	1013578649	0	TAB		IX	GRANT
54	25	1013578649	1	KEY	(ffaa72d2deb7)	X	GRANT
54	25	1013578649	2	KEY	(c6c6be2c6d97)	X	GRANT
55	25	0	0	DB		S	GRANT

· **실행 계획**

<Session 54>

Rows	Executes	StmtText
1 1	1 1	UPDATE TB_LOCK01 SET EmpID = '1501' FROM TB_LOCK01 WHERE EmpID = '1201' ¦--Clustered Index Update(OBJECT:([Tuning].[dbo].[TB_LOCK01].[CIDX_LOCK01]), OBJECT:([Tuning].[dbo].[TB_LOCK01].[NIDX01_LOCK01]), SET:([Tuning].[dbo].[TB_LOCK01].[EmpID] = [Expr1003]), DEFINE:([Expr1003]='1501'), WHERE:([Tuning].[dbo].[TB_LOCK01].[EmpID]='1201'))

테이블 'TB_LOCK01'. 스캔 수 1, 논리적 읽기 16

<Session 55>

Rows	Executes	StmtText
1 1	1 1	SELECT [NO],[EmpID],[NAME] FROM [TB_LOCK01] WITH(nolock) WHERE [NO]=@1 ¦--Clustered Index Scan(OBJECT:([Tuning].[dbo].[TB_LOCK01].[CIDX_LOCK01]), WHERE:([Tuning].[dbo] .[TB_LOCK01].[NO]=(8)))

테이블 'TB_LOCK01'. 스캔 수 1, 논리적 읽기 6

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

49. 스캔 범위에 따른 블로킹 이슈 - 1

1. 프롬프트

> [AS-IS]
> 구문 수행 시 지속적으로 대기가 발생한다. 문제 원인을 분석하여 개선안을 알려줘.

```
[AS-IS]
SELECT NO, EmpID, NAME
FROM TB_LOCK01
WHERE NO = 8;
```

2. 분석 및 주요 최적화

1) 문제 분석

인덱스	유형	키 컬럼
CIDX_LOCK01	Clustered	EmpID
NIDX01_LOCK01	NonClustered	NO

2) 세션 58이 보유 중인 잠금 (TB_LOCK01)

리소스 유형	잠금 모드	상태	설명
KEY (4건)	X (Exclusive)	GRANT	클러스터드 인덱스 키에 배타 잠금
PAGE (3건)	IX	GRANT	페이지 수준 의도 배타 잠금
OBJECT	IX	GRANT	테이블 수준 의도 배타 잠금

*대기 발생 원인: Key Lookup 블로킹

귀하의 쿼리 실행 흐름은 다음과 같습니다:

① WHERE NO = 8 → NIDX01_LOCK01 (비클러스터드 인덱스) 탐색 성공
② EmpID, NAME 조회 → CIDX_LOCK01 (클러스터드 인덱스)로 Key Lookup 필요
③ Key Lookup 시 해당 KEY에 S 잠금 요청 → 세션 58의 X 잠금과 충돌 대기!

*비클러스터드 인덱스(NO)에는 EmpID, NAME 컬럼이 없어 반드시 클러스터드 인덱스로 Key Lookup해야 합니다. 이때 세션 1이 해당 KEY에 X 잠금을 보유 중이므로 S ↔ X 충돌로 무한 대기가 발생합니다.

3. 개선안

```
-- 방안 1) 커버링 인덱스 생성 (가장 효과적)
-- Key Lookup을 제거하면 클러스터드 인덱스의 잠금과 충돌하지 않습니다:

DROP INDEX NIDX01_LOCK01 ON dbo.TB_LOCK01;

CREATE NONCLUSTERED INDEX NIDX01_LOCK01
ON dbo.TB_LOCK01 (NO)
INCLUDE (EmpID, NAME);
```

*INCLUDE로 EmpID, NAME을 포함하면 비클러스터드 인덱스만으로 쿼리가 완료되어 Key Lookup이 제거됩니다.

```
-- 방안 2) RCSI (Read Committed Snapshot Isolation) 활성화

ALTER DATABASE Tuning SET READ_COMMITTED_SNAPSHOT ON;
```

*읽기 시 행 버전을 사용하므로 X 잠금과 충돌하지 않습니다.

```
-- 방안 3. NOLOCK 힌트 (임시 조치)

SELECT NO, EmpID, NAME
FROM dbo.TB_LOCK01 WITH (NOLOCK)
WHERE NO = 8;
```

*Dirty Read가 허용되므로 커밋되지 않은 데이터가 조회될 수 있습니다.

■ 개선 우선순위

순위	조치	효과	부작용
1	커버링 인덱스	Key Lookup 제거 → 잠금 충돌 원천 해소	인덱스 저장 공간 소량 증가
2	RCSI 활성화	모든 읽기·쓰기 블로킹 해소	tempdb 사용량 증가
3	NOLOCK 힌트	즉시 대기 해소	Dirty Read 위험

*방안 1(커버링 인덱스)**이 이 시나리오에 가장 적합합니다. Key Lookup 자체를 제거하여 클러스터드 인덱스의 잠금에 영향을 받지 않게 됩니다.

50 스캔 범위에 따른 블로킹 이슈 - 2

두 세션이 서로 다른 행을 참조함에도 테이블 전체 스캔으로 인해 대기가 발생되는 사례이다.

1. 사전 구성 스크립트

```
SELECT TOP 1000 NO, DATEADD (DD,NO,'20200101') AS DATE
INTO TB_FLock
FROM (SELECT ROW_NUMBER () OVER (ORDER BY A.number) AS NO
      FROM master..spt_values A, (SELECT number
                                  FROM master..spt_values B
                                  WHERE type = 'P' AND number <= 100) B
      WHERE A.type = 'P'
) A
GO
```

2. [AS-IS] SQL 구문 및 실행 계획

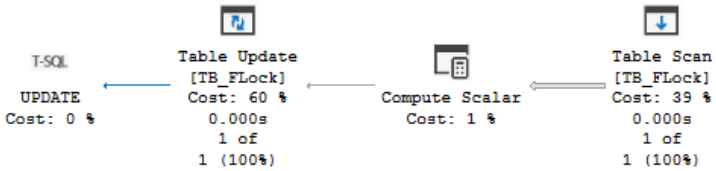
· SQL 구문

<Session 53>	<Session 54>
BEGIN TRAN UPDATE TB_FLock SET NO = 100 WHERE DATE = '2020-01-03 00:00:00.000'	BEGIN TRAN SELECT NO FROM TB_FLock WHERE DATE = '2020-01-05 00:00:00.000'

· 실행 계획

<Session 53>

Rows	Executes	StmtText
1 1 0 1	1 1 0 1	UPDATE [TB_FLock] set [NO] = @1 WHERE [DATE]=@2 ¦--Table Update(OBJECT:([Tuning].[dbo].[TB_FLock]), SET:([Tuning].[dbo].[TB_FLock].[NO] = [Expr1003])) ¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(bigint,[@1],0))) ¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_FLock]), WHERE:([Tuning].[dbo].[TB_FLock].[DATE] = CONVERT_IMPLICIT(datetime,[@2],0)) ORDERED)



테이블 'TB_FLock'. 스캔 수 1, 논리적 읽기 4

3. [AS-IS] 통계 및 Lock 정보

· 인덱스 없음

· Lock 정보

spid	dbid	ObjId	IndId	Type	Resource	Mode	Status
53	6	0	0	DB		S	GRANT
53	6	2.67E+08	0	PAG	1:155752	IX	GRANT
53	6	2.67E+08	0	RID	1:155752:1	X	GRANT
53	6	2.67E+08	0	TAB		IX	GRANT
54	6	2.67E+08	0	TAB		IS	GRANT
54	6	2.67E+08	0	RID	1:155752:1	S	WAIT
54	6	2.67E+08	0	PAG	1:155752	IS	GRANT
54	6	0	0	DB		S	GRANT

4. [AS-IS] 구문 설명

Session 53에서는 [DATE] 컬럼이 '2020-01-03 00:00:00.000' 인 데이터의 [NO] 컬럼을 100으로 업데이트하는 구문이며, Session 54에서는 [DATE] 컬럼이 '2020-01-05 00:00:00.000' 인 데이터의 [NO] 컬럼을 조회하는 구문이다. 선행되는 Session 53이 업데이트 구문을 수행하면서 조건에 해당되는 데이터에 배타 잠금(X)을 획득하게 되는데 이후에 Session 54가 업데이트 대상이 아닌 데이터를 조회함에도 공유 잠금(S)을 획득하기 위해 대기가 발생된다. 원인은 Session 54에서 데이터를 조회할 때 테이블 전체 스캔으로 수행되면서 Session 53이 배타 잠금을 획득하고 있는 행에 공유 잠금 획득을 시도하기 때문이다.

5. [TO-BE] 튜닝 포인트

다음 구문을 개선하기 위해 다음과 같은 방법을 고려할 수 있다.

첫째, 신규 인덱스 생성

둘째, READ_COMMITTED_SNAPSHOT 데이터베이스 옵션이 ON으로 설정된 READ COMMITTED 격리 수준 사용

셋째, WITH(NOLOCK) 힌트 사용

넷째, WITH(READPAST) 힌트 사용

첫 번째 방법은 [DATE] 컬럼을 키값으로 하는 신규 인덱스를 생성하는 방법이다. Session 54에서 데이터를 조회할 때 인덱스 탐색으로 수행하면 Session 53에서 업데이트 중인 행은 읽지 않기 때문에 블로킹을 방지할 수 있다.

두 번째 방법은 데이터베이스에 READ_COMMITTED_SNAPSHOT 옵션을 적용하여 업데이트된 행 버전이 유지 관리되도록 하는 방법이다. Session 53이 데이터를 변경 중이라도 Session 54에서 조회할 때 이전 버전의 행을 읽어내기 때문에 블로킹을 방지할 수 있다.

세 번째 방법은 SELECT 절에 WITH(NOLOCK) 힌트 사용으로 COMMIT 되지 않은 데이터를 읽도록 허용하여 블로킹을 방지하는 방법이다.

네 번째 방법은 SELECT 절에 WITH(READPAST) 힌트 사용으로 잠긴 행을 읽지 않도록 하여 블로킹을 방지하는 방법이다.

6. [TO-BE] 개선 방안

1) 신규 인덱스 생성

조건 절에 사용된 [DATE] 컬럼을 키값으로 인덱스를 생성하여 Session 54에서 조회 구문 수행 시 인덱스 탐색으로 필요한 데이터만 읽어내기 때문에 블로킹을 방지할 수 있다.

■ 인덱스 생성 구문

```
CREATE INDEX NIDX01_FLock ON TB_FLock(DATE)
GO
```

· SQL 구문

<Session 53>	<Session 54>
BEGIN TRAN UPDATE TB_FLock SET NO = 100 WHERE DATE = '2020-01-03 00:00:00.000'	BEGIN TRAN SELECT NO FROM TB_FLock WHERE DATE = '2020-01-05 00:00:00.000'

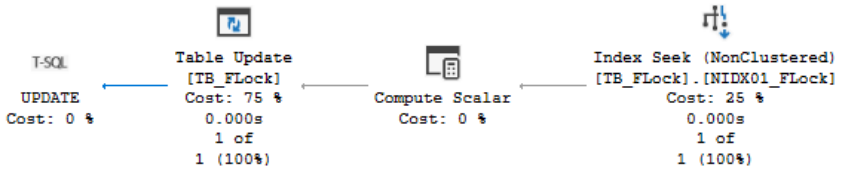
· **Lock 정보**

spid	dbid	ObjId	IndId	Type	Resource	Mode	Status
53	6	0	0	DB		S	GRANT
53	6	2.67E+08	0	TAB		IX	GRANT
53	6	2.67E+08	0	PAG	1:155752	IX	GRANT
53	6	2.67E+08	0	RID	1:155752:1	X	GRANT
54	6	0	0	DB		S	GRANT

· **실행 계획**

<Session 53>

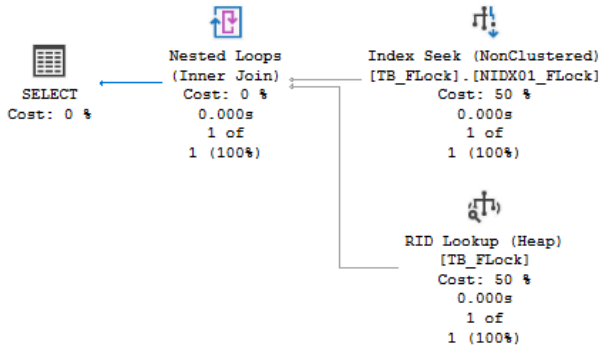
Rows	Executes	StmtText
1 1 0 1	1 1 0 1	UPDATE [TB_FLock] set [NO] = @1 WHERE [DATE]=@2 ¦--Table Update(OBJECT:([Tuning].[dbo].[TB_FLock]), SET:([Tuning].[dbo].[TB_FLock].[NO] = [Expr1003])) ¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(bigint,[@1],0))) ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_FLock].[NIDX01_FLock]), SEEK:([Tuning].[dbo].[TB_FLock].[DATE]=CONVERT_IMPLICIT(datetime,[@2],0)) Ordered Forward)



테이블 'TB_FLock'. 스캔 수 1, 논리적 읽기 3

<Session 54>

Rows	Executes	StmtText
1 1 1 1	1 1 1 1	SELECT [NO] FROM [TB_FLock] WHERE [DATE]=@1 ¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([Bmk1000])) ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_FLock].[NIDX01_FLock]), SEEK:([Tuning].[dbo].[TB_FLock].[DATE]='2020-01-05 00:00:00.000') Ordered Forward) ¦--RID Lookup(OBJECT:([Tuning].[dbo].[TB_FLock]), SEEK:([Bmk1000]=[Bmk1000]) LOOKUP Ordered Forward)



테이블 'TB_FLock'. 스캔 수 1, 논리적 읽기 3

2) READ_COMMITTED_SNAPSHOT 데이터베이스 옵션이 ON으로 설정된 READ COMMITTED 격리 수준 사용

READ_COMMITTED_SNAPSHOT 데이터베이스 옵션 활성화로 업데이트 중인 행의 이전 버전 데이터를 참조하도록 하여 블로킹을 방지할 수 있다.

인덱스 삭제

```
DROP INDEX NIDX01_FLock ON TB_FLock --TO-BE(1)에서 생성한 인덱스 삭제
GO
```

READ_COMMITTED_SNAPSHOT 옵션 활성화

```
-- READ_COMMITTED_SNAPSHOT 옵션 활성화

ALTER DATABASE Tuning SET READ_COMMITTED_SNAPSHOT ON
SET TRANSACTION ISOLATION LEVEL READ COMMITTED
GO
```

· SQL 구문

<Session 53>	<Session 54>
BEGIN TRAN UPDATE TB_FLock SET NO = 100 WHERE DATE = '2020-01-03 00:00:00.000'	BEGIN TRAN SELECT NO FROM TB_FLock WHERE DATE = '2020-01-05 00:00:00.000'

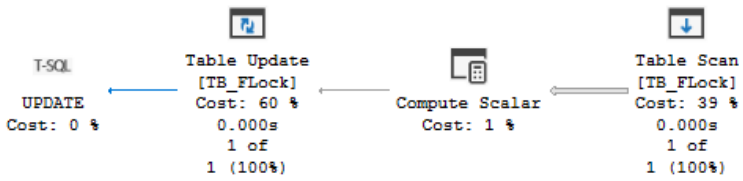
· Lock 정보

spid	dbid	ObjId	IndId	Type	Resource	Mode	Status
53	6	0	0	DB		S	GRANT
53	6	2.67E+08	0	TAB		IX	GRANT
53	6	2.67E+08	0	PAG	1:155752	IX	GRANT
53	6	2.67E+08	0	RID	1:155752:1	X	GRANT
54	6	0	0	DB		S	GRANT

· 실행 계획

<Session 53>

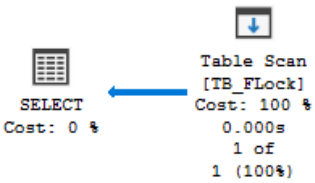
Rows	Executes	StmtText
1	1	UPDATE [TB_FLock] set [NO]=@1 WHERE [DATE]=@2
1	1	¦--Table Update(OBJECT:([Tuning].[dbo].[TB_FLock]), SET:([Tuning].[dbo].[TB_FLock].[NO] = [Expr1003]))
0	0	¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(bigint,[@1],0)))
1	1	¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_FLock]), WHERE:([Tuning].[dbo].[TB_FLock].[DATE]=CONVERT_IMPLICIT(datetime,[@2],0)) ORDERED)



테이블 'TB_FLock'. 스캔 수 1, 논리적 읽기 4

<Session 54>

Rows	Executes	StmtText
1 1	1 1	SELECT [NO] FROM [TB_FLock] WHERE [DATE]=@1 ¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_FLock]), WHERE:([Tuning].[dbo].[TB_FLock].[DATE]=CONVERT_IMPLICIT(datetime,[@1],0)))



테이블 'TB_FLock'. 스캔 수 1, 논리적 읽기 4

3) WITH (NOLOCK) 힌트 사용

Session 54에 WITH(NOLOCK) 힌트를 적용하면 COMMIT 되지 않은 데이터를 읽도록 허용하기 때문에 Session 53의 업데이트 작업과 관계없이 블로킹을 방지할 수 있다.

■ READ_COMMITTED_SNAPSHOT 옵션 비활성화

```
-- READ_COMMITTED_SNAPSHOT 옵션 비활성화

ALTER DATABASE Tuning SET READ_COMMITTED_SNAPSHOT OFF
GO
```

· HINT가 적용된 SQL 구문

<Session 53>	<Session 54>
BEGIN TRAN UPDATE TB_FLock SET NO = 100 WHERE DATE = '2020-01-03 00:00:00.000'	BEGIN TRAN SELECT NO FROM TB_FLock WITH(NOLOCK) WHERE DATE = '2020-01-05 00:00:00.000'

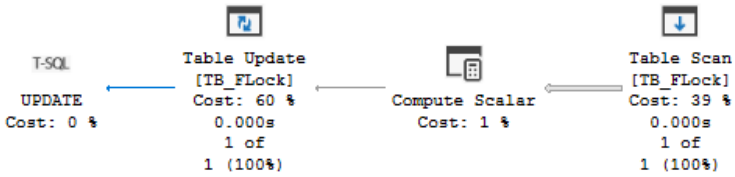
· Lock 정보

spid	dbid	ObjId	IndId	Type	Resource	Mode	Status
53	6	0	0	DB		S	GRANT
53	6	2.67E+08	0	TAB		IX	GRANT
53	6	2.67E+08	0	PAG	1:155752	IX	GRANT
53	6	2.67E+08	0	RID	1:155752:1	X	GRANT
54	6	0	0	DB		S	GRANT

· 실행 계획

<Session 53>

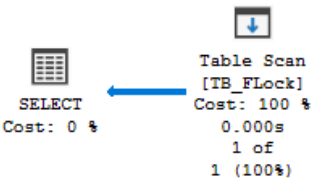
Rows	Executes	StmtText
1 1 0 1	1 1 0 1	UPDATE [TB_FLock] set [NO]=@1 WHERE [DATE]=@2 ¦--Table Update(OBJECT:([Tuning].[dbo].[TB_FLock]), SET:([Tuning].[dbo].[TB_FLock].[NO] = [Expr1003])) ¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(bigint,[@1],0))) ¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_FLock]), WHERE:([Tuning].[dbo].[TB_FLock].[DATE]=CONVERT_IMPLICIT(datetime,[@2],0)) ORDERED)



테이블 'TB_FLock'. 스캔 수 1, 논리적 읽기 4

<Session 54>

Rows	Executes	StmtText
1 1	1 1	SELECT [NO] FROM [TB_FLock] WITH(nolock) WHERE [DATE]=@1 ¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_FLock]), WHERE:([Tuning].[dbo].[TB_FLock].[DATE]=CONVER T_IMPLICIT(datetime,[@1],0)))



테이블 'TB_FLock'. 스캔 수 1, 논리적 읽기 4

4) WITH (READPAST) 힌트 사용

Session 54에 WITH(READPAST) 힌트를 적용하면 다른 트랜잭션에 의해 잠긴 행은 읽지 않기 때문에 블로킹을 방지할 수 있다.

· HINT가 적용된 SQL 구문

<Session 53>	<Session 54>
BEGIN TRAN UPDATE TB_FLock SET NO = 100 WHERE DATE = '2020-01-03 00:00:00.000'	BEGIN TRAN SELECT NO FROM TB_FLock WITH(READPAST) WHERE DATE = '2020-01-05 00:00:00.000'

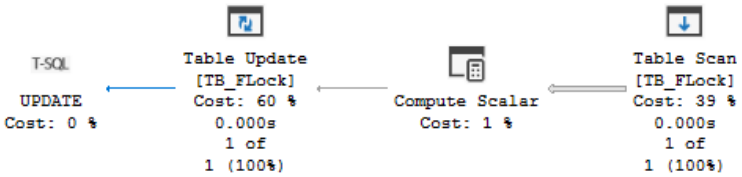
· Lock 정보

spid	dbid	ObjId	IndId	Type	Resource	Mode	Status
53	6	0	0	DB		S	GRANT
53	6	2.67E+08	0	TAB		IX	GRANT
53	6	2.67E+08	0	PAG	1:155752	IX	GRANT
53	6	2.67E+08	0	RID	1:155752:1	X	GRANT
54	6	0	0	DB		S	GRANT

· 실행 계획

<Session 53>

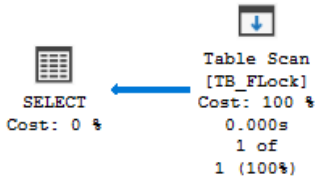
Rows	Executes	StmtText
1 1 0 1	1 1 0 1	UPDATE [TB_FLock] set [NO] = @1 WHERE [DATE]=@2 ¦--Table Update(OBJECT:([Tuning].[dbo].[TB_FLock]), SET:([Tuning].[dbo].[TB_FLock].[NO] = [Expr1003])) ¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(bigint,[@1],0))) ¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_FLock]), WHERE:([Tuning].[dbo].[TB_FLock].[DATE]=CONVERT_IMPLICIT(datetime,[@2],0)) ORDERED)



테이블 'TB_FLock'. 스캔 수 1, 논리적 읽기 4

<Session 54>

Rows	Executes	StmtText
1 1	1 1	SELECT [NO] FROM [TB_FLock] WITH(readpast) WHERE [DATE]=@1 ¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_FLock]), WHERE:([Tuning].[dbo].[TB_FLock].[DATE]=CONVERT_IMPLICIT(datetime,[@1],0)))



테이블 'TB_FLock'. 스캔 수 1, 논리적 읽기 4

1. 프롬프트

> [AS-IS]
> 구문에서 대기가 무한히 발생한다. 문제 원인을 분석하여 개선안을 알려줘.

```
[AS-IS]
BEGIN TRAN
SELECT NO
FROM TB_FLock
WHERE DATE = '2020-01-05 00:00:00.000';
```

2. 분석 및 주요 최적화

1) 문제 분석 결과

현재 세션 79(귀하의 SELECT)가 세션 58에 의해 블로킹되고 있습니다.

2) 블로킹 상황 요약

항목	세션 58 (블로커)	세션 79 (대기 중)
동작	BEGIN TRAN + DML (INSERT/UPDATE/DELETE) 후 미커밋	BEGIN TRAN + SELECT
보유 잠금	RID에 X (Exclusive) 잠금	RID에 S (Shared) 잠금 요청 중
상태	유휴 (idle) – 커밋/롤백 없이 열려 있음	suspended (무한 대기)
대기 유형	–	LCK_M_S (공유 잠금 대기)
lock_timeout	–	-1 (무한 대기)

3) 근본 원인

1. 세션 58이 dbo.TB_FLock 테이블의 특정 행(RID)에 X 잠금을 걸고 COMMIT/ROLLBACK 없이 트랜잭션을 열어 놓았습니다.
2. 귀하의 쿼리(세션 79)도 BEGIN TRAN으로 시작하여 동일 행에 S 잠금을 요청하지만, X ↔ S 잠금은 호환되지 않으므로 무한 대기합니다.
3. 테이블이 Heap (클러스터드 인덱스 없음)이므로, SELECT 시 테이블 전체 스캔이 발생하여 잠금 충돌 확률이 더 높습니다.

3. 개선안

```
-- 방안 1. 읽기 시 블로킹 회피 - NOLOCK 힌트 또는 RCSI 활성화
-- 방법 A: 쿼리에 NOLOCK 힌트 사용 (Dirty Read 허용)

SELECT NO
FROM dbo.TB_FLock WITH (NOLOCK)
WHERE DATE = '2020-01-05 00:00:00.000';


-- 방법 B: 데이터베이스에 Read Committed Snapshot Isolation (RCSI) 활성화 (권장)

ALTER DATABASE Tuning SET READ_COMMITTED_SNAPSHOT ON;
```

*RCSI를 활성화하면 SELECT가 행 버전을 읽으므로 X 잠금과 충돌하지 않습니다.

```
-- 방안 2. 인덱스 추가- Heap 제거
-- 현재 테이블에 인덱스가 전혀 없어 Full Table Scan이 발생합니다.

CREATE CLUSTERED INDEX IX_TB_FLock_DATE ON dbo.TB_FLock (DATE);
```

*클러스터드 인덱스를 추가하면 필요한 행만 접근하여 잠금 범위가 줄어듭니다.

51 DEADLOCK 이슈 - 1

동일 테이블에 대해 SELECT와 UPDATE가 수행되면서 나타나는 DeadLock 사례이다.

1. 사전 구성 스크립트

```
SELECT ROW_NUMBER() OVER(ORDER BY B.number) AS NO
      , RAND() * 100 AS NUM, GETDATE() AS DATE
INTO TB_DLock
FROM master..spt_values A, (SELECT number
                            FROM master..spt_values
                            WHERE type = 'P' AND number < 5) B
WHERE A.type = 'P'

CREATE CLUSTERED INDEX CIDX_DLock ON TB_DLock(DATE)
CREATE INDEX NIDX01_DLock ON TB_DLock(NO)
GO
```

2. [AS-IS] SQL 구문 및 실행 계획

· SQL 구문

<Session 54>	<Session 55>

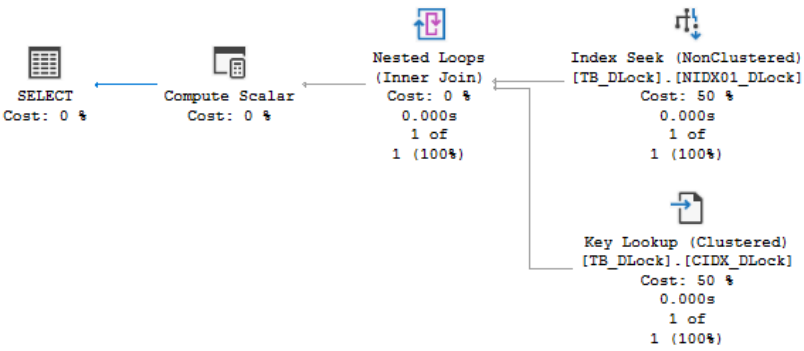
SET TRANSACTION
ISOLATION LEVEL REPEATABLE READ
 	
DECLARE @A INT, @B INT, @C DATETIME
WHILE 1=1
BEGIN
  SELECT @A = NO,@B = NUM,@C = DATE
  FROM TB_DLOCK
  WHERE NO = 7412
END
GO
	WHILE 1 = 1
	BEGIN
	  UPDATE TB_DLock
	  SET DATE = GETDATE()
	  WHERE NO = 7412
	END
	GO

· 실행 계획

<Session 54>

Rows	Executes	StmtText

1
0
 	
1
1
 	
1
	1
	0
	
	1
	1
	
	1
		SELECT @A=NO, @B=NUM, @C=DATE FROM TB_DLOCK WHERE NO=7412
		  ¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(int,[Tuning].[dbo].[TB_DLock].[NO],0), [Ex
		pr1004]=CONVERT_IMPLICIT(int,[Tuning].[dbo].[TB_DLock].[NUM],0)))
		  ¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([Uniq1001], [Tuning].[dbo].[TB_DLock].[DATE]))
		  ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_DLock].[NIDX01_DLock]), SEEK:([Tuning].[dbo].[T
		B_DLock].[NO]=(7412)) Ordered Forward)
		  ¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_DLock].[CIDX_DLock]), SEEK:([Tuning
		].[dbo].[TB_DLock].[DATE]=[Tuning].[dbo].[TB_DLock].[DATE] AND [Uniq1001]=[Uniq1001])
		LOOKUP Ordered Forward)



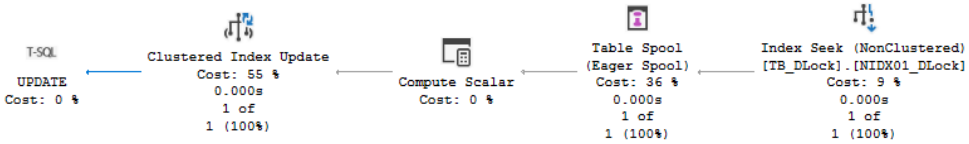
테이블 'TB_DLock'. 검색 수 1, 논리적 읽기 수 5

메시지 1205, 수준 13, 상태 51, 줄 77
트랜잭션(프로세스 ID 54)이 잠금 리소스에서 다른 프로세스와의 교착 상태가 발생하여 실행이 중지되었습니다. 트랜잭션을 다시 실행하십시오.

<Session 55>

Rows	Executes	StmtText

1
1
 	
0
1
1
	1
	1
	
	0
	1
	1
		UPDATE [TB_DLock] set [DATE]=getdate() WHERE [NO]=@1
		  ¦--Clustered Index Update(OBJECT:([Tuning].[dbo].[TB_DLock].[CIDX_DLock]), OBJECT:(
		[Tuning].[dbo].[TB_DLock].[NIDX01_DLock]), SET:([Tuning].[dbo].[TB_DLock].[DATE] =
		RaiseIfNullUpdate([Expr1003])))
		  ¦--Compute Scalar(DEFINE:([Expr1003]=getdate()))
		  ¦--Table Spool
		  ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_DLock].[NIDX01_DLock]), SEEK:([Tuning].[dbo
		].[TB_DLock].[NO]=(7412)) Ordered Forward)



테이블 'TB_DLock'. 검색 수 1, 논리적 읽기 수 12
테이블 'Worktable'. 검색 수 1, 논리적 읽기 수 5

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_DLock	CIDX_DLock	CLUSTERED	0	DATE	-
TB_DLock	NIDX01_DLock	NONCLUSTERED	0	NO	-

· Lock 정보

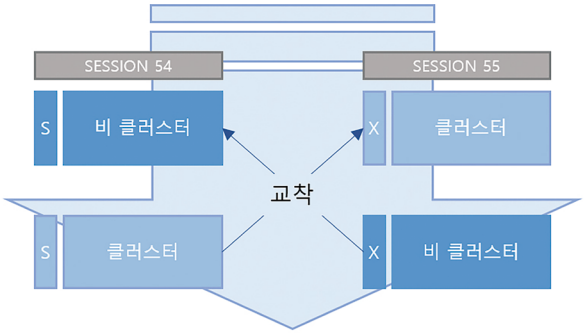
spid	dbid	ObjId	IndId	Type	Resource	Mode	Status
54	6	1.43E+08	2	KEY	(b7c98bb69a76)	S	GRANT
54	6	0	0	DB		S	GRANT
54	6	1.43E+08	0	TAB		IS	GRANT
54	6	1.43E+08	1	KEY	(d55bfb1ffdac)	S	WAIT
54	6	1.43E+08	1	PAG	1:127419	IS	GRANT
55	6	1.43E+08	1	PAG	1:127419	IX	GRANT
55	6	1.43E+08	2	PAG	1:112019	IX	GRANT
55	6	1.43E+08	2	PAG	1:112031	IU	GRANT
55	6	1.43E+08	1	KEY	(a37c6842c588)	X	GRANT
55	6	1.43E+08	1	KEY	(d55bfb1ffdac)	X	GRANT
55	6	1.43E+08	0	TAB		IX	GRANT
55	6	1.43E+08	2	KEY	(b7c98bb69a76)	X	CNVT
55	6	1.43E+08	2	KEY	(b7c98bb69a76)	U	GRANT
55	6	0	0	DB		S	GRANT

4. [AS-IS] 구문 설명

Session 54에서는 REPEATABLE READ 격리 수준으로 [NO] 컬럼이 7412인 데이터를 반복적으로 조회하는 구문이며, Session 55에서는 [NO] 컬럼이 7412인 데이터의 [DATE] 컬럼을 반복적으로 업데이트하는 구문이다.

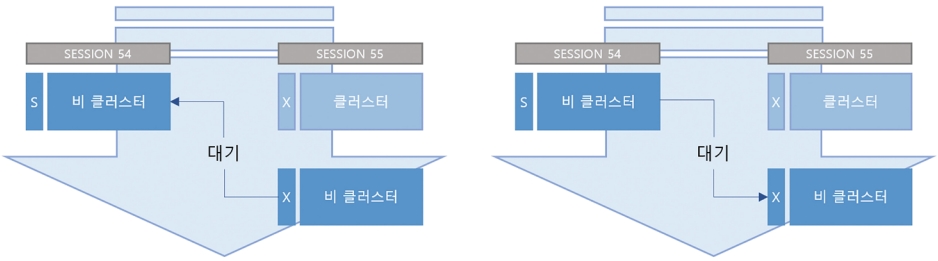
Session 54에서는 비클러스터형 인덱스 탐색을 수행하면서 [NO] 컬럼이 7412인 데이터를 대상으로 공유 잠금(S)을 획득하게 된다. 이후 [NUM] 컬럼을 조회하기 위해 클러스터형 인덱스에 Key Lookup이 발생하면서 공유 잠금(S) 획득을 시도한다. 반면 Session 55에서는 [NO] 컬럼이 7412인 데이터를 업데이트하기 위해 클러스터형 인덱스 키에 배타 잠금(X)을 획득하게 되며, 이후 관련된 비클러스터형 인덱스에 대해서도 배타 잠금(X) 획득을 시도한다.

두 구문이 반복 수행되면서 Session 54는 Session 55의 클러스터형 인덱스 키 배타 잠금(X)으로 인해 대기하게 되고, Session 55는 Session 54의 비클러스터형 인덱스 공유 잠금(S)으로 인해 대기하게 된다. 결과적으로 서로가 상대방의 잠금 해제를 기다리는 상태가 되어 교착 상태가 발생한다.



5. [TO-BE] 튜닝 포인트

Session 54에서 비클러스터형 인덱스 탐색 후 클러스터형 인덱스 참조(Key Lookup) 과정을 생략할 수 있다면 두 세션이 교착되는 것을 방지할 수 있다. Key Lookup이 발생되지 않도록 [NUM] 컬럼을 비클러스터형 인덱스에 포괄 열로 포함하여 클러스터형 인덱스를 읽어내지 않도록 한다.



6. [TO-BE] 개선 방안

[NUM] 컬럼을 포괄 열로 하는 인덱스를 생성하고 Session 54에서 SELECT 구문을 수행할 때 해당 인덱스를 사용하면 Key Lookup에 의해 클러스터형 인덱스를 참조하지 않기 때문에 Session 55와 교착 상태가 발생되지 않는다.

■ 인덱스 생성 구문

```
CREATE INDEX NIDX02_DLock ON TB_DLock(NO) INCLUDE(NUM)
GO
```

· SQL 구문

<Session 54>

```
SET TRANSACTION
ISOLATION LEVEL REPEATABLE READ

DECLARE @A INT, @B INT, @C DATETIME

WHILE 1 = 1
BEGIN
  SELECT @A = NO,@B = NUM,@C = DATE
  FROM TB_DLOCK
  WHERE NO = 7412
END
GO
```

<Session 55>

```
WHILE 1 = 1
BEGIN
  UPDATE TB_DLock
  SET DATE = GETDATE()
  WHERE NO = 7412
END
GO
```

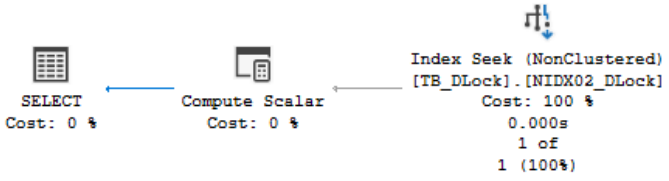
· Lock 정보

spid	dbid	ObjId	IndId	Type	Resource	Mode	Status
54	6	1.43E+08	3	PAG	1:127489	IS	GRANT
54	6	1.43E+08	0	TAB		IS	GRANT
54	6	1.43E+08	3	KEY	(e1e64bb5d2ec)	S	GRANT
54	6	1.43E+08	3	KEY	(3f285c7ea991)	S	GRANT
54	6	0	0	DB		S	GRANT
55	6	1.43E+08	1	PAG	1:127419	IX	GRANT
55	6	1.43E+08	3	KEY	(e1e64bb5d2ec)	U	GRANT
55	6	1.43E+08	3	KEY	(e1e64bb5d2ec)	X	CNVT
55	6	1.43E+08	2	KEY	(e1e64bb5d2ec)	X	GRANT
55	6	1.43E+08	0	TAB		IX	GRANT
55	6	1.43E+08	3	PAG	1:127489	IX	GRANT
55	6	0	0	DB		S	GRANT
55	6	1.43E+08	1	KEY	-9.42178E+11	X	GRANT
55	6	1.43E+08	2	PAG	1:112019	IX	GRANT
55	6	1.43E+08	2	PAG	1:112031	IX	GRANT
55	6	1.43E+08	1	KEY	(832e14c0348f)	X	GRANT
55	6	1.43E+08	2	KEY	(f6e928ec90e3)	X	GRANT

· 실행 계획

<Session 54>

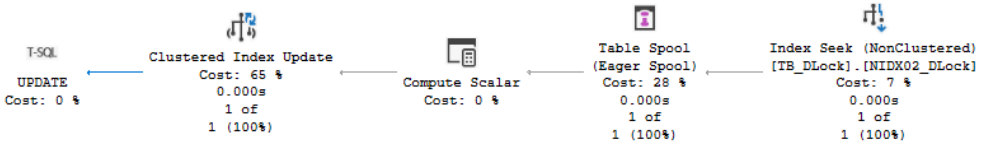
Rows	Executes	StmtText
1	1	SELECT @A=NO, @B=NUM, @C=DATE FROM TB_DLOCK WHERE NO=7412
0	0	¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(int,[Tuning].[dbo].[TB_DLock].[NO],0), [Expr1004]=CONVERT_IMPLICIT(int,[Tuning].[dbo].[TB_DLock].[NUM],0)))
1	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_DLock].[NIDX02_DLock]), SEEK:([Tuning].[dbo].[TB_DLock].[NO]=(7412)) Ordered Forward)



테이블 'TB_DLock'. 검색 수 1, 논리적 읽기 수 3

<Session 55>

Rows	Executes	StmtText
1	1	UPDATE [TB_DLock] set [DATE]=getdate() WHERE [NO]=@1
1	1	¦--Clustered Index Update(OBJECT:([Tuning].[dbo].[TB_DLock].[CIDX_DLock]), OBJECT:([Tuning].[dbo].[TB_DLock].[NIDX01_DLock]), OBJECT:([Tuning].[dbo].[TB_DLock].[NIDX02_DLock]), SET:([Tuning].[dbo].[TB_DLock].[DATE] = RaiseIfNullUpdate([Expr1003])))
0	0	¦--Compute Scalar(DEFINE:([Expr1003]=getdate()))
1	1	¦--Table Spool
1	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_DLock].[NIDX02_DLock]), SEEK:([Tuning].[dbo].[TB_DLock].[NO]=(7412)) Ordered Forward)



테이블 'TB_DLock'. 검색 수 1, 논리적 읽기 수 17
테이블 'Worktable'. 검색 수 1, 논리적 읽기 수 5

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

51. DEADLOCK 이슈 - 1

1. 프롬프트

> [AS-IS]
> 두 구문 수행 간, 데드락이 발생하는 원인과 함께 개선안을 알려줘.

```
[AS-IS]
<Session 1>
SET TRANSACTION
ISOLATION LEVEL REPEATABLE READ

DECLARE @A INT, @B INT, @C DATETIME
WHILE 1=1
BEGIN
    SELECT @A = NO, @B = NUM, @C = DATE
    FROM TB_DLOCK
    WHERE NO = 7412
END;

<Session 2>
WHILE 1 = 1
BEGIN
    UPDATE TB_DLock
    SET DATE = GETDATE()
    WHERE NO = 7412
END;
```

2. 분석 및 주요 최적화

1) 인덱스 구조 (핵심)

인덱스	유형	키 컬럼
CIDX_DLock	Clustered	DATE
NIDX01_DLock	Nonclustered	NO

2) 데드락 원인

```
sequenceDiagram
participant S1 as Session 1 (SELECT, REPEATABLE READ)
participant NC as NIDX01_DLock (NO)
participant CI as CIDX_DLock (DATE)
participant S2 as Session 2 (UPDATE)

S1->>NC: ① S Lock 획득 ✅
S1->>CI: ② S Lock 획득 (Key Lookup) ✅
Note over S1: REPEATABLE READ → 양쪽 S Lock 유지
S2->>CI: ③ X Lock 획득 ✅ (DATE 변경)
S2->>NC: ④ X Lock 필요 (NO 인덱스 갱신) → S1 대기 ⛔
S1->>CI: ⑤ 다음 반복 Key Lookup → S2 대기 ⛔
Note over S1,S2: 💀 DEADLOCK
```

리소스	Session 1 보유	Session 2 보유
NC Index (NO)	S Lock ✅	X Lock 대기 ⛔
Clustered (DATE)	S Lock 대기 ⛔	X Lock ✅

- WHERE NO = 7412는 NC 인덱스를 타지만, 필요한 컬럼(NUM, DATE)이 NC 인덱스에 없어 Key Lookup이 발생합니다.
- UPDATE가 DATE를 변경 → Clustered Key가 DATE이므로 행의 물리적 위치가 변경됨
  → NC 인덱스의 포인터도 갱신 필요 → NC 인덱스에 X Lock 필요 → 교차 잠금 발생

3. 개선안

```
-- 방안 1. 커버링 인덱스 (근본 해결)
-- NC 인덱스에 NUM, DATE를 포함시켜 Clustered Index 접근(Key Lookup)을 제거합니다.

DROP INDEX NIDX01_DLock ON dbo.TB_DLock;

CREATE NONCLUSTERED INDEX NIDX01_DLock
ON dbo.TB_DLock (NO)
INCLUDE (NUM, DATE);
```

```sql
-- 방안 2:  SNAPSHOT 격리수준 읽기 세션이 행 버전을 통해 잠금 없이 읽습니다.
-- 데이터베이스 수준 설정 필요 (1회)

ALTER DATABASE Tuning SET READ_COMMITTED_SNAPSHOT ON;


-- Session 1: 격리수준 변경 불필요, 잠금 없이 읽기

DECLARE @A INT, @B INT, @C DATETIME
WHILE 1=1
BEGIN
    SELECT @A = NO, @B = NUM, @C = DATE
    FROM dbo.TB_DLock
    WHERE NO = 7412
END;
```

항목	방안 1 (커버링 인덱스)	방안 2 (SNAPSHOT)
데드락 해결	✅	✅
동시성	높음	매우 높음
SELECT 성능	향상 (Lookup 제거)	변화 없음
영향 범위	이 테이블만	DB 전체
tempdb 부하	없음	버전 저장소 사용

*방안 1이 권장됩니다. Key Lookup 제거로 데드락 원인을 근본적으로 없애면서 SELECT 성능도 개선됩니다.
DB 전체 설정 변경이 불필요하므로 부작용도 없습니다.

52 DEADLOCK 이슈 - 2

동일 테이블에 대해 SELECT와 UPDATE가 수행되면서 나타나는 DeadLock 사례이다.

1. 사전 구성 스크립트

```sql
CREATE TABLE TB_DLock02(A INT, B DATETIME, C VARCHAR(10));

DECLARE @CNT INT, @DATE CHAR(8)
SET @CNT = 1
SET @DATE = '20200101'

WHILE @CNT <11
BEGIN
INSERT INTO TB_DLock02 VALUES (@CNT, @DATE,'ABC')
SET @CNT = @CNT + 1
SET @DATE = DATEADD(DD,1,@DATE)
END
```

```
CREATE CLUSTERED INDEX CIDX_DLock02 ON TB_DLock02(A)
CREATE INDEX NIDX01_DLock02 ON TB_DLock02(B)
GO
```

2. [AS-IS] SQL 구문 및 실행 계획

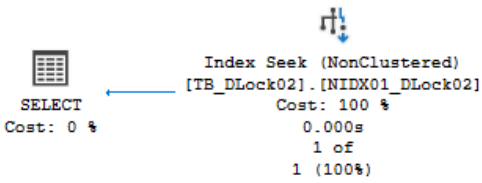
· SQL 구문

<Session 53>	<Session 54>
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ BEGIN TRAN SELECT A FROM TB_DLock02 WHERE B = '2020-01-03 00:00:00.000' WAITFOR DELAY '00:00:05' SELECT C FROM TB_DLock02 WHERE B = '2020-01-03 00:00:00.000'	BEGIN TRAN UPDATE TB_DLock02 SET A = '100' WHERE B = '2020-01-03 00:00:00.000'

· 실행 계획

<Session 53>

Rows	Executes	StmtText
1 1	1 1	SELECT [A] FROM [TB_DLock02] WHERE [B]=@1 ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_DLock02].[NIDX01_DLock02]), SEEK:([Tuning].[dbo].[TB_DLock02].[B]=CONVERT_IMPLICIT(datetime,[@1],0)) Ordered Forward)



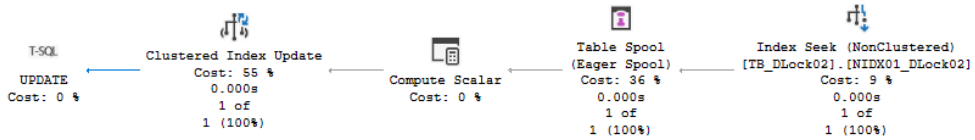
테이블 'TB_DLock02'. 검색 수 1, 논리적 읽기 수 2

메시지 1205, 수준 13, 상태 51, 줄 13
트랜잭션(프로세스 ID 53)이 잠금 리소스에서 다른 프로세스와의 교착 상태가 발생하여 실행이 중지되었습니다. 트랜잭션을 다시 실행하십시오.

<Session 54>

Rows	Executes	StmtText

1
1
 	
0
1
1
	1
	1
	
	0
	1
	1
		UPDATE [TB_DLock02] set [A]=@1 WHERE [B]=@2
		  ¦--Clustered Index Update(OBJECT:([Tuning].[dbo].[TB_DLock02].[CIDX_DLock02]), OBJECT:([Tuning].[dbo].[TB_DLock02].[NIDX01_DLock02]), SET:([Tuning].[dbo].[TB_DLock02].[A] = [Expr1003]))
		  ¦--Compute Scalar(DEFINE:([Expr1003]=(100)))
		  ¦--Table Spool
		  ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_DLock02].[NIDX01_DLock02]), SEEK:([Tuning].[dbo].[TB_DLock02].[B]='2020-01-03 00:00:00.000') Ordered Forward)




```
테이블 'TB_DLock02'. 검색 수 1, 논리적 읽기 수 12
테이블 'Worktable'. 검색 수 1, 논리적 읽기 수 5
```

3. [AS-IS] 통계 정보

· 인덱스 정보

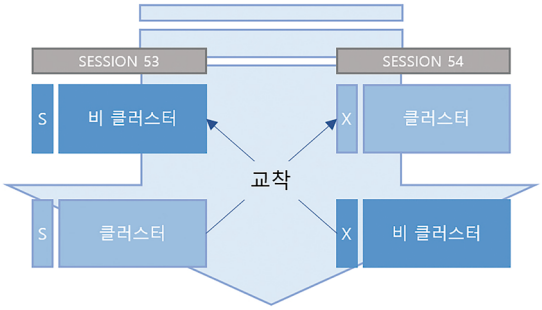
Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_DLock02	CIDX_DLock02	CLUSTERED	0	A	-
TB_DLock02	NIDX01_DLock02	NONCLUSTERED	0	B	-

· Lock 정보

spid	dbid	ObjId	IndId	Type	Resource	Mode	Status
53	6	4.95E+08	0	TAB		IS	GRANT
53	6	4.95E+08	2	PAG	1:120080	IS	GRANT
53	6	4.95E+08	1	PAG	1:111984	IS	GRANT
53	6	0	0	DB		S	GRANT
53	6	4.95E+08	1	KEY	(052c8c7d9727)	S	WAIT
53	6	4.95E+08	2	KEY	(605acc022110)	S	GRANT
54	6	4.95E+08	2	KEY	(605acc022110)	X	CNVT
54	6	4.95E+08	2	KEY	(605acc022110)	U	GRANT
54	6	0	0	DB		S	GRANT
54	6	4.95E+08	1	KEY	(052c8c7d9727)	X	GRANT
54	6	4.95E+08	1	PAG	1:111984	IX	GRANT
54	6	4.95E+08	0	TAB		IX	GRANT
54	6	4.95E+08	2	PAG	1:120080	IX	GRANT
54	6	4.95E+08	1	KEY	(dbff0d4a5b42)	X	GRANT

4. [AS-IS] 구문 설명

[TB_DLock02] 테이블의 [B] 컬럼이 '2020-01-03 00:00:00.000' 인 데이터에 대해 Session 53에서는 REPEATABLE READ 격리 수준으로 [A] 값을 조회 후 5초 뒤 [C] 값을 조회하는 구문이며, Session 54에서는 [A] 값을 업데이트하는 구문이다. Session 53에서 비클러스터형 인덱스 탐색을 하면서 키값에 공유 잠금(S)을 획득하게 된다. 이후 Session 54에서 업데이트하기 위해 클러스터형 인덱스 키값에 배타 잠금(X)을 획득하고 비클러스터형 인덱스 키값에도 배타 잠금(X) 획득을 시도하는데 Session 53의 공유 잠금으로 인해 대기가 발생한다. 5초 뒤 Session 53에서 [C] 컬럼을 조회하기 위해 클러스터형 인덱스 키값에 공유 잠금 획득을 시도하지만 Session 54 배타 잠금으로 인해 충돌이 생기면서 교착 상태가 발생한다.



5. [TO-BE] 튜닝 포인트

다음 구문을 개선하기 위해 두 가지 방안을 제시한다.
첫째, READ_COMMITTED_SNAPSHOT 데이터베이스 옵션이 ON으로 설정된 READ COMMITTED 격리 수준
둘째, 세션에 READ UNCOMMITTED 격리 수준 적용

첫 번째 방법은 데이터베이스에 READ_COMMITTED_SNAPSHOT 옵션을 적용하여 업데이트된 행 버전이 유지 관리되도록 하는 것이다. Session 54가 클러스터형 인덱스에 배타 잠금을 획득하고 있어도 Session 53에서 행 버전 포인터를 이용해 이전 버전의 데이터를 참조하기 때문에 대기가 발생되지 않아 교착 상태를 방지할 수 있다.

두 번째 방법은 Session 53의 SELECT 구문에 READ UNCOMMITTED 격리 수준 설정으로 아직 COMMIT 되지 않은 행을 읽을 수 있도록 허용하면서 교착 상태를 방지하는 방법이다.

6. [TO-BE] 개선 방안

1) READ_COMMITTED_SNAPSHOT 데이터베이스 옵션이 ON으로 설정된 READ COMMITTED 격리 수준

READ_COMMITTED_SNAPSHOT 데이터베이스 옵션을 적용하여 업데이트 중인 행의 이전 버전 데이터를 참조하도록 하여 대기가 발생되지 않도록 한다.

■ READ_COMMITTED_SNAPSHOT 옵션 활성화

```
-- READ_COMMITTED_SNAPSHOT 옵션 활성화

ALTER DATABASE Tuning SET READ_COMMITTED_SNAPSHOT ON
SET TRANSACTION ISOLATION LEVEL READ COMMITTED
GO
```

· 변경된 SQL 구문

<Session 53>	<Session 54>
BEGIN TRAN SELECT A FROM TB_DLock02 WHERE B = '2020-01-03 00:00:00.000' WAITFOR DELAY '00:00:05' SELECT C FROM TB_DLock02 WHERE B = '2020-01-03 00:00:00.000'	BEGIN TRAN UPDATE TB_DLock02 SET A = '100' WHERE B = '2020-01-03 00:00:00.000'

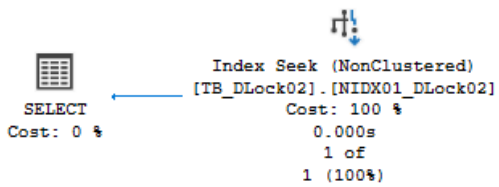
· Lock 정보

spid	dbid	ObjId	IndId	Type	Resource	Mode	Status
53	6	0	0	DB		S	GRANT
54	6	0	0	DB		S	GRANT
54	6	4.95E+08	2	PAG	1:120080	IX	GRANT
54	6	4.95E+08	2	KEY	(be894d35ed75)	X	GRANT
54	6	4.95E+08	1	KEY	(dbff0d4a5b42)	X	GRANT
54	6	4.95E+08	1	KEY	(052c8c7d9727)	X	GRANT
54	6	4.95E+08	2	KEY	(605acc022110)	X	GRANT
54	6	4.95E+08	1	PAG	1:111984	IX	GRANT
54	6	4.95E+08	0	TAB		IX	GRANT

· 실행 계획

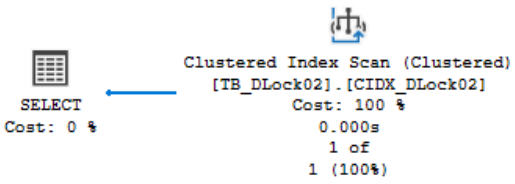
<Session 53>

Rows	Executes	StmtText
1 1	1 1	SELECT [A] FROM [TB_DLock02] WHERE [B]=@1 ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_DLock02].[NIDX01_DLock02]),SEEK:([Tuning].[dbo].[TB_DLock02].[B]=CONVERT_IMPLICIT(datetime,[@1],0)) Ordered Forward)



테이블 'TB_DLock02'. 검색 수 1, 논리적 읽기 수 2

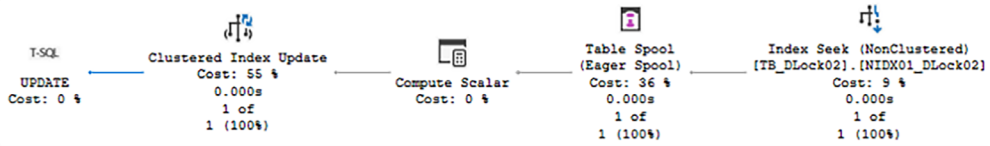
Rows	Executes	StmtText
1 1	1 1	SELECT [C] FROM [TB_DLock02] WHERE [B]=@1 ¦--Clustered Index Scan(OBJECT:([Tuning].[dbo].[TB_DLock02].[CIDX_DLock02]), WHERE:([Tuning].[dbo].[TB_DLock02].[B]='2020-01-03 00:00:00.000'))



테이블 'TB_DLock02'. 검색 수 1, 논리적 읽기 수 2

<Session 54>

Rows	Executes	StmtText
1 1 0 1 1	1 1 0 1 1	UPDATE [TB_DLock02] set [A]=@1 WHERE [B]=@2 ¦--Clustered Index Update(OBJECT:([Tuning].[dbo].[TB_DLock02].[CIDX_DLock02]), OBJECT:([Tuning]. [dbo].[TB_DLock02].[NIDX01_DLock02]),SET:([Tuning].[dbo].[TB_DLock02].[A] = [Expr1003])) ¦--Compute Scalar(DEFINE:([Expr1003]=(100))) ¦--Table Spool ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_DLock02].[NIDX01_DLock02]), SEEK:([Tuning]. [dbo].[TB_DLock02].[B]='2020-01-03 00:00:00.000') Ordered Forward)



테이블 'TB_DLock02'. 검색 수 1, 논리적 읽기 수 10
테이블 'Worktable'. 검색 수 1, 논리적 읽기 수 5

2) 세션에 READ UNCOMMITTED 격리 수준 적용

Session 53에 READ UNCOMMITTED 격리 수준을 설정하여 Session 54가 클러스터형 인덱스에 배타 잠금을 획득하고 있더라도 변경되기 전 행을 읽기 때문에 대기가 발생하지 않아 교착 상태가 발생하지 않는다.

READ_COMMITTED_SNAPSHOT 옵션 비활성화

```
-- 스냅샷 격리 비활성화

ALTER DATABASE Tuning SET READ_COMMITTED_SNAPSHOT OFF
SET TRANSACTION ISOLATION LEVEL READ COMMITTED
GO
```

· 변경된 SQL 구문

<Session 53>	<Session 54>
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED BEGIN TRAN SELECT A FROM TB_DLock02 WHERE B = '2020-01-03 00:00:00.000' WAITFOR DELAY '0:0:5' SELECT C FROM TB_DLock02 WHERE B = '2020-01-03 00:00:00.000'	BEGIN TRAN UPDATE TB_DLock02 SET A = '100' WHERE B = '2020-01-03 00:00:00.000'

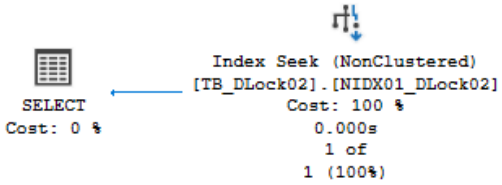
· **Lock 정보**

spid	dbid	ObjId	IndId	Type	Resource	Mode	Status
53	6	0	0	DB		S	GRANT
54	6	0	0	DB		S	GRANT
54	6	4.95E+08	2	PAG	1:120080	IX	GRANT
54	6	4.95E+08	2	KEY	(be894d35ed75)	X	GRANT
54	6	4.95E+08	1	KEY	(dbff0d4a5b42)	X	GRANT
54	6	4.95E+08	1	KEY	(052c8c7d9727)	X	GRANT
54	6	4.95E+08	2	KEY	(605acc022110)	X	GRANT
54	6	4.95E+08	1	PAG	1:111984	IX	GRANT
54	6	4.95E+08	0	TAB		IX	GRANT

· **실행 계획**

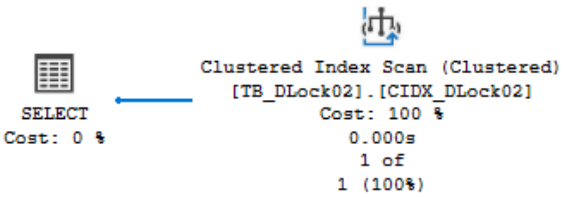
<Session 53>

Rows	Executes	StmtText
1 1	1 1	SELECT [A] FROM [TB_DLock02] WHERE [B]=@1 ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_DLock02].[NIDX01_DLock02]),SEEK:([Tuning].[dbo].[TB_DLock02].[B]=CONVERT_IMPLICIT(datetime,[@1],0)) Ordered Forward)



테이블 'TB_DLock02'. 검색 수 1, 논리적 읽기 수 2

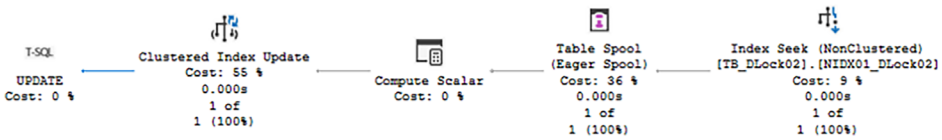
Rows	Executes	StmtText
1 1	1 1	SELECT [C] FROM [TB_DLock02] WHERE [B]=@1 ¦--Clustered Index Scan(OBJECT:([Tuning].[dbo].[TB_DLock02].[CIDX_DLock02]), WHERE:([Tuning].[dbo].[TB_DLock02].[B]='2020-01-03 00:00:00.000'))



테이블 'TB_DLock02'. 검색 수 1, 논리적 읽기 수 2

<Session 54>

Rows	Executes	StmtText
1 1 0 1 1	1 1 0 1 1	UPDATE [TB_DLock02] set [A]=@1 WHERE [B]=@2 ¦--Clustered Index Update(OBJECT:([Tuning].[dbo].[TB_DLock02].[CIDX_DLock02]), OBJECT:([Tuning].[dbo].[TB_DLock02].[NIDX01_DLock02]),SET:([Tuning].[dbo].[TB_DLock02].[A] = [Expr1003])) ¦--Compute Scalar(DEFINE:([Expr1003]=(100))) ¦--Table Spool ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_DLock02].[NIDX01_DLock02]), SEEK:([Tuning].[dbo].[TB_DLock02].[B]='2020-01-03 00:00:00.000') Ordered Forward)




```
테이블 'TB_DLock02'. 검색 수 1, 논리적 읽기 수 10
테이블 'Worktable'. 검색 수 1, 논리적 읽기 수 5
```

GitHub Copilot 채팅 결과

Claude Opus 4.6

52. DEADLOCK 이슈 - 2

1. 프롬프트

> [AS-IS]
> 두 세션 간 데드락이 발생한 원인을 분석하고 개선안을 알려줘.

```
[AS-IS]
<Session 1>
SET TRANSACTION
ISOLATION LEVEL REPEATABLE READ

BEGIN TRAN
SELECT A
FROM TB_DLock02
WHERE B = '2020-01-03 00:00:00.000'
WAITFOR DELAY '00:00:05'
SELECT C
FROM TB_DLock02
WHERE B = '2020-01-03 00:00:00.000'

<Session 2>
BEGIN TRAN
   UPDATE TB_DLock02
   SET A = '100'
   WHERE B = '2020-01-03 00:00:00.000'
```

2. 분석 및 주요 최적화

1) 인덱스 구조 (핵심)

인덱스	유형	키 컬럼
CIDX_DLock02	Clustered	A
NIDX01_DLock02	Nonclustered	B

*SELECT A와 SELECT C는 NC 인덱스(B)에 포함되지 않아 Clustered Index로 Key Lookup이 필요합니다.

2) 데드락 원인

두 세션이 서로 다른 인덱스의 잠금을 역순으로 요청하면서 교착 상태가 발생합니다.

```
세션 1 (REPEATABLE READ):
  SELECT A WHERE B = '...'
    ① NIDX01_DLock02 (B) → S Lock 획득
    ② CIDX_DLock02 (A)   → S Lock 획득 (Key Lookup)
    → REPEATABLE READ이므로 두 잠금 모두 트랜잭션 끝까지 유지

세션 2:
  UPDATE SET A = '100' WHERE B = '...'
    ③ NIDX01_DLock02 (B) → X Lock 필요 ← 세션1 S Lock 대기 ⛔
    ④ CIDX_DLock02 (A)   → X Lock 획득 ✅
```

```
SequenceDiagram
    participant S1 as Session 1 (SELECT)
    participant NC as NIDX01_DLock02 (B)
    participant CI as CIDX_DLock02 (A)
    participant S2 as Session 2 (UPDATE)

    S1->>NC: S Lock 획득 ✅
    S1->>CI: S Lock 획득 (Key Lookup) ✅
    Note over S1: WAITFOR 5초 (잠금 유지)
    S2->>CI: X Lock 획득 ✅
    S2->>NC: X Lock 필요 → S1 대기 ⛔
    S1->>CI: 2번째 SELECT → Key Lookup 필요 → S2 대기 ⛔
    Note over S1,S2: 💀 DEADLOCK
```

리소스	Session 1 보유	Session 2 보유
NC Index (B)	S Lock ✅	X Lock 대기 ⛔
Clustered (A)	S Lock 대기 ⛔	X Lock ✅

3. 개선안

```
-- 방안 1. 커버링 인덱스 (근본 해결)
-- NC 인덱스에 A, C를 포함시켜 Key Lookup을 제거하면,
-- Clustered Index 접근 자체가 없어져 잠금 충돌이 사라집니다.

DROP INDEX NIDX01_DLock02 ON dbo.TB_DLock02;

CREATE NONCLUSTERED INDEX NIDX01_DLock02
ON dbo.TB_DLock02 (B)
INCLUDE (A, C);


-- 방안 2. UPDLOCK 힌트
-- 읽기 시점에 U Lock을 획득하여 잠금 순서를 직렬화합니다.

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ
BEGIN TRAN

SELECT A
FROM dbo.TB_DLock02 WITH (UPDLOCK)
WHERE B = '2020-01-03 00:00:00.000'

WAITFOR DELAY '00:00:05'

SELECT C
FROM dbo.TB_DLock02
WHERE B = '2020-01-03 00:00:00.000'

COMMIT TRAN
```

- **비교**

항목	방안 1 (커버링 인덱스)	방안 2 (UPDLOCK)
데드락 해결	✅	✅
동시성	높음 (잠금 범위 축소)	낮음 (직렬화)
SELECT 성능	향상 (Lookup 제거)	변화 없음
추가 비용	인덱스 저장 공간	없음

*방안 1이 최적입니다. Key Lookup 자체를 제거하면 데드락 원인이 근본적으로 사라지고, SELECT 성능도 동시에 개선됩니다.

53 잠금이 전환될 때 발생하는 DEADLOCK 이슈

공유 잠금(S-Lock)에서 배타 잠금(X-Lock)으로 전환될 때 발생하는 DeadLock 사례이다.

1. 사전 구성 스크립트

```
CREATE TABLE TB_DATA03 ( NUM INT, NO INT )

DECLARE @INT INT = 1
WHILE @INT <= 2000
BEGIN
INSERT INTO TB_DATA03(NUM,NO) VALUES (@INT, @INT % 10 )
SET @INT += 1
END

CREATE CLUSTERED INDEX CIDX_DATA03 ON TB_DATA03 (NUM)
GO
```

2. [AS-IS] SQL 구문 및 실행 계획

· SQL 구문

<Session 54>	<Session 55>
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ DECLARE @NUM INT BEGIN TRAN SELECT @NUM = NO + 1 FROM TB_DATA03 WHERE NUM = 10 WAITFOR DELAY '00:00:05' UPDATE TB_DATA03 SET NO = @NUM WHERE NUM = 10	SET TRANSACTION ISOLATION LEVEL REPEATABLE READ DECLARE @NUM INT BEGIN TRAN SELECT @NUM = NO + 1 FROM TB_DATA03 WHERE NUM = 10 WAITFOR DELAY '00:00:05' UPDATE TB_DATA03 SET NO = @NUM WHERE NUM = 10

· 실행 계획

<Session 54>

Rows	Executes	StmtText

1
0
 	
1
	1
	0
	
	1
		SELECT @NUM=NO+1 FROM TB_DATA03 WHERE NUM = 10
		  ┆--Compute Scalar(DEFINE:([Expr1003]=[Tuning].[dbo].[TB_DATA03].[NO]+(1)))
		  ┆--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_DATA03].[CIDX_DATA03]), SEEK:([Tuning].[dbo].[TB_DATA03].[NUM]=(10)) Ordered Forward)

테이블 'TB_DATA03'. 스캔 수 1, 논리적 읽기 2

Rows	Executes	StmtText
1 1	1 1	UPDATE TB_DATA03 SET NO=@NUM WHERE NUM=10 ┆--Clustered Index Update(OBJECT:([Tuning].[dbo].[TB_DATA03].[CIDX_DATA03]), SET:([Tuning].[dbo].[TB_DATA03].[NO] = [@NUM]), WHERE:([Tuning].[dbo].[TB_DATA03].[NUM]=(10)))

테이블 ' TB_DATA03'. 스캔 수 1, 논리적 읽기 4

<Session 55>

Rows	Executes	StmtText
1 0 1	1 0 1	SELECT @NUM=NO+1 FROM TB_DATA03 WHERE NUM = 10 ¦--Compute Scalar(DEFINE:([Expr1003]=[Tuning].[dbo].[TB_DATA03].[NO]+(1))) ¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_DATA03].[NIDX01_DATA03]), SEEK:([Tuning].[dbo].[TB_DATA03].[NUM]=(10)) Ordered Forward)

테이블 'TB_DATA03'. 검색 수 1, 논리적 읽기 2

메시지 1205, 수준 13, 상태 51, 줄 19
트랜잭션(프로세스 ID 55)이 잠금 리소스에서 다른 프로세스와의 교착 상태가 발생하여 실행이 중지되었습니다. 트랜잭션을 다시 실행하십시오.

3. [AS-IS] 통계 및 Lock 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_DATA03	CIDX_DATA03	CLUSTERED	0	NUM	-

· Lock 정보

spid	dbid	ObjId	IndId	Type	Resource	Mode	Status
54	7	98099390	1	PAG	1:295184	IX	GRANT
54	7	98099390	0	TAB		IX	GRANT
54	7	0	0	DB		S	GRANT
54	**7**	**98099390**	**1**	**KEY**	**(241332e1ddb0)**	**S**	**GRANT**
54	**7**	**98099390**	**1**	**KEY**	**(241332e1ddb0)**	**X**	**CNVT**
55	**7**	**98099390**	**1**	**KEY**	**(241332e1ddb0)**	**X**	**CNVT**
55	**7**	**98099390**	**1**	**KEY**	**(241332e1ddb0)**	**S**	**GRANT**
55	7	98099390	1	PAG	1:295184	IX	GRANT
55	7	98099390	0	TAB		IX	GRANT
55	7	0	0	DB		S	GRANT

4. [AS-IS] 구문 설명

REPEATABLE READ 독립성 레벨로 SELECT와 UPDATE 구문을 하나의 트랜잭션에서 수행시키는 구문이다. 이 트랜잭션의 SELECT와 UPDATE는 [TB_DATA03] 테이블에서 [NUM] = 10인 데이터를 대상으로 잠금을 획득한다. 동일 구문을 서로 다른 세션에서 동시에 수행시키면 먼저 Session 54와 Session 55는 SELECT 절에서 공유 잠금(S)을 획득한다. 이어서 UPDATE를 수행하기 위한 배타 잠금(X)을 획득하려 시도하지만 Session 54는 Session 55의 공유 잠금으로 인해, Session 55는 Session 54의 공유 잠금으로 인해 서로 대기하게 되면서 DeadLock이 발생되는 문제가 생긴다.

5. [TO-BE] 튜닝 포인트

다음 구문을 개선하기 위해 두 가지 방안을 고려할 수 있다.

첫째, WITH(UPDLOCK) 잠금 힌트를 사용하여 DeadLock을 방지한다.

둘째, 애플리케이션 잠금(sp_getapplock)을 활용하여 DeadLock을 방지한다.

첫 번째 방법은 SELECT 절에 WITH(UPDLOCK) 힌트를 추가하여 의도적으로 업데이트 잠금(U)을 획득하는 방법이다. 업데이트 잠금 간에는 호환이 되지 않기 때문에 먼저 수행 중인 세션이 완료될 때까지 후행 세션을 대기시키면서 DeadLock을 회피할 수 있다.

두 번째 방법은 애플리케이션 잠금(sp_getapplock)을 활용하는 것이다. 이는 트랜잭션에 임의의 잠금을 획득하고 다른 트랜잭션이 동일한 잠금을 획득하려 할 때 대기하도록 의도할 수 있다. 즉, 해당 쿼리문을 배타적으로 실행하게 하여 먼저 수행 중인 세션의 구문이 완료될 때까지 후행 세션을 대기시키는 것이다.

6. [TO-BE] 개선 방안

1) WITH(UPDLOCK) 잠금 힌트를 사용하여 DeadLock을 방지한다.

SELECT 구문에 WITH(UPDLOCK) 힌트를 사용하면 공유 잠금(S)이 아닌 업데이트 잠금(U)을 획득한다. Session 54가 처음 트랜잭션을 시작하고 SELECT 구문을 수행할 때부터 업데이트 잠금을 획득하므로 Session 55는 Session 54의 구문이 완료될 때까지 대기하게 된다.

· 변경된 SQL 구문

<Session 54>

```
SET TRANSACTION
ISOLATION LEVEL REPEATABLE READ

DECLARE @NUM INT

BEGIN TRAN

SELECT @NUM = NO + 1
FROM TB_DATA03 WITH(UPDLOCK)
WHERE NUM = 10

WAITFOR DELAY '00:00:05'

UPDATE TB_DATA03
SET NO = @NUM WHERE NUM = 10
```

<Session 55>

```
SET TRANSACTION
ISOLATION LEVEL REPEATABLE READ

DECLARE @NUM INT

BEGIN TRAN

SELECT @NUM = NO + 1
FROM TB_DATA03 WITH(UPDLOCK)
WHERE NUM = 10

WAITFOR DELAY '00:00:05'

UPDATE TB_DATA03
SET NO = @NUM WHERE NUM = 10
```

· Lock 정보

spid	dbid	ObjId	IndId	Type	Resource	Mode	Status
54	7	0	0	DB		S	GRANT
54	**7**	**98099390**	**1**	**KEY**	**(241332e1ddb0)**	**U**	**GRANT**
54	7	98099390	0	TAB		IX	GRANT
54	7	98099390	1	PAG	1:295184	IU	GRANT
55	7	98099390	1	PAG	1:295184	IU	GRANT
55	7	98099390	0	TAB		IX	GRANT
55	**7**	**98099390**	**1**	**KEY**	**(241332e1ddb0)**	**U**	**WAIT**
55	7	0	0	DB		S	GRANT

2) 애플리케이션 잠금 (sp_getapplock)을 활용하여 DeadLock을 방지한다.

트랜잭션 내에서 sp_getapplock을 사용하면 애플리케이션 잠금에 의해서 트랜잭션끼리 서로 영향받지 않도록 의도할 수 있다. 프로시저의 매개변수 @Resource는 동일한 리소스임을 확인하기 위해 사용되며 @LockMode에서는 잠금의 수준을 지정한다. 아래 예시는 트랜잭션 내에서 애플리케이션 잠금을 통해 'TESTLOCK' 리소스임을 명시하고 배타 잠금(X, Exclusive)을 획득하도록 지정하였다. 배타 잠금 간에는 호환이 되지 않기 때문에 후행 트랜잭션은 선행 트랜잭션이 애플리케이션 잠금을 반환할 때까지 대기하게 된다.

- **SQL 구문**

<Session 54>

```
SET TRANSACTION
ISOLATION LEVEL REPEATABLE READ

DECLARE @NUM INT

BEGIN TRAN
EXEC sp_getapplock
  @Resource = 'TESTLOCK'
  , @LockMode = 'Exclusive'

SELECT @NUM = NO + 1
FROM TB_DATA03
WHERE NUM = 10

WAITFOR DELAY '00:00:05'

UPDATE TB_DATA03
SET NO = @NUM WHERE NUM = 10
EXEC sp_releaseapplock
  @Resource = 'TESTLOCK'
```

<Session 55>

```
SET TRANSACTION
ISOLATION LEVEL REPEATABLE READ

DECLARE @NUM INT

BEGIN TRAN
EXEC sp_getapplock
  @Resource = 'TESTLOCK'
  , @LockMode = 'Exclusive'

SELECT @NUM = NO + 1
FROM TB_DATA03
WHERE NUM = 10

WAITFOR DELAY '00:00:05'

UPDATE TB_DATA03
SET NO = @NUM WHERE NUM = 10
EXEC sp_releaseapplock
  @Resource = 'TESTLOCK'
```

- **Lock 정보**

spid	dbid	ObjId	IndId	Type	Resource	Mode	Status
54	7	98099390	1	KEY	(241332e1ddb0)	S	GRANT
54	7	0	0	DB		S	GRANT
54	7	98099390	0	TAB		IS	GRANT
54	7	98099390	1	PAG	1:295184	IS	GRANT
54	**7**	**0**	**0**	**APP**	**0:[TESTLOCK]:(e07866b6)**	**X**	**GRANT**
55	**7**	**0**	**0**	**APP**	**0:[TESTLOCK]:(e07866b6)**	**X**	**WAIT**
55	7	0	0	DB		S	GRANT

1. 프롬프트

> [AS-IS]
> 구문을 서로 다른 세션에서 수행하니까 Dead Lock이 발생해.
> 문제 원인 및 개선안을 알려줘.

```
[AS-IS]
SET TRANSACTION
ISOLATION LEVEL REPEATABLE READ

DECLARE @NUM INT

BEGIN TRAN

SELECT @NUM = NO + 1
FROM TB_DATA03
WHERE NUM = 10

WAITFOR DELAY '00:00:05'

UPDATE TB_DATA03
SET NO = @NUM WHERE NUM = 10
```

2. 분석 및 주요 최적화

1) 잠금 흐름 (두 세션 동시 실행)

시점	Session A	Session B
T1	SELECT → S Lock 획득 ✅	SELECT → S Lock 획득 ✅
T2	WAITFOR 5초 (S Lock 유지)	WAITFOR 5초 (S Lock 유지)
T3	UPDATE → X Lock 필요 → ⛔ B의 S Lock 대기	UPDATE → X Lock 필요 → ⛔ A의 S Lock 대기
결과	💀 DEADLOCK	💀 DEADLOCK

2) 핵심 원인

REPEATABLE READ에서 S Lock(공유 잠금)은 트랜잭션 종료까지 유지됩니다. 두 세션이 동일 행에 S Lock을 동시에 잡은 후, 둘 다 X Lock(배타적 잠금)으로 승격이 불가능한 교착 상태에 빠집니다.

```
S Lock + S Lock = 호환 ✅ (둘 다 SELECT 성공)
S Lock → X Lock = 비호환 ⛔ (상대방 S Lock이 해제되어야 함)
→ 서로 상대방을 기다림 = DEADLOCK
```

3. 개선안

```sql
-- UPDLOCK 힌트 (권장)
-- SELECT 시점에 **U Lock(업데이트 잠금)**을 획득하여,
-- 두 번째 세션이 읽기 단계부터 대기하도록 합니다.

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ

DECLARE @NUM INT

BEGIN TRAN

SELECT @NUM = NO + 1
FROM dbo.TB_DATA03 WITH (UPDLOCK)
WHERE NUM = 10

WAITFOR DELAY '00:00:05'

UPDATE dbo.TB_DATA03
SET NO = @NUM WHERE NUM = 10

COMMIT TRAN
```

시점	Session A	Session B
T1	SELECT → U Lock 획득 ✅	SELECT → U Lock 대기 ⛔
T2	UPDATE → X Lock 승격 ✅	대기 중...
T3	COMMIT → 잠금 해제	U Lock 획득 → 정상 진행 ✅

Section 08

DML 최적화

대량 데이터의 INSERT, UPDATE, DELETE 작업은 시스템에 큰 부하를 줄 수 있다. 이 섹션에서는 DML 작업 시 발생하는 성능 이슈와 이를 효율적으로 처리하는 방법을 다룬다.

54 DELETE 구문의 성능 개선 - 1

DELETE 구문 사용 시 인덱스 누락으로 높은 I/O가 발생한 사례이다.

1. 사전 구성 스크립트

```
SELECT rownum AS seq
      , NEWID() AS main_code
      , DATEADD(ss, rownum, '2020-01-01 00:00:00.000') AS date
      , NEWID() AS no2
      , NEWID() AS no3
      , NEWID() AS no4
INTO TB_Maind2
FROM (SELECT ROW_NUMBER() OVER (ORDER BY a.number) AS rownum, a.number
FROM master..spt_values a
JOIN master..spt_values b ON a.type = 'P' AND b.type = 'P'
AND a.number <=1000 AND b.number <= 1000 ) a

--TB_Maind2 PK 설정
ALTER TABLE TB_Maind2 ALTER COLUMN main_code UNIQUEIDENTIFIER NOT NULL
ALTER TABLE TB_Maind2 ADD CONSTRAINT PK_Maind2 PRIMARY KEY (main_code)

-- TB_Ex 생성 및 FK 설정
SELECT * INTO TB_Exd2
FROM TB_Maind2

ALTER TABLE TB_Exd2 ADD CONSTRAINT FK_Maind2_Exd2 FOREIGN KEY (main_code)
REFERENCES TB_Maind2 (main_code) ON DELETE CASCADE

CREATE CLUSTERED INDEX CIDX_Exd2 ON TB_Exd2(main_code)
GO
```

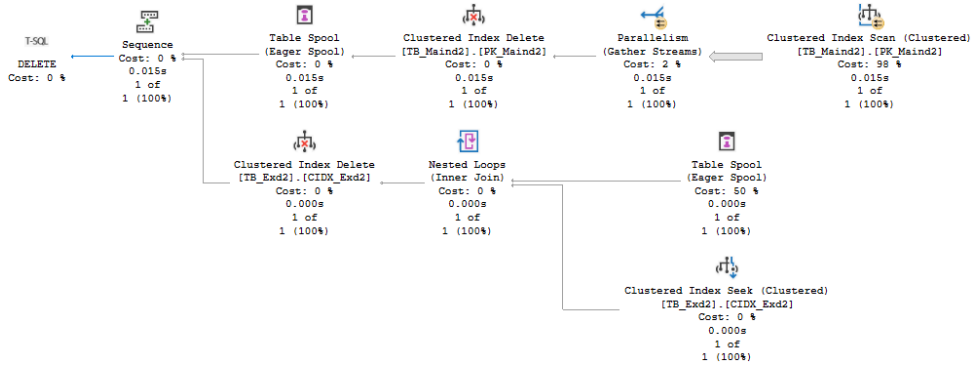
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
DELETE FROM TB_Maind2 WHERE seq = 64570
GO
```

· 실행 계획

Rows	Executes	StmtText
1	1	DELETE [TB_Maind2] WHERE [seq]=@1
1	1	┆--Sequence
1	1	┆ ┆--Table Spool
1	1	┆ ┆ ┆--Clustered Index Delete(OBJECT:([tuning].[dbo].[TB_Maind2].[PK_Maind2]))
1	1	┆ ┆ ┆--Parallelism(Gather Streams)
1	8	┆ ┆ ┆--Clustered Index Scan(OBJECT:([tuning].[dbo].[TB_Maind2].[PK_Maind2]), WHERE: ([tuning].[dbo].[TB_Maind2].[seq]=(64570)) ORDERED)
1	1	┆ ┆--Clustered Index Delete(OBJECT:([tuning].[dbo].[TB_Exd2].[CIDX_Exd2]))
1	1	┆ ┆--Nested Loops(INNER JOIN, OUTER REFERENCES:([tuning].[dbo].[TB_Maind2].[main_code]))
1	1	┆ ┆ ┆--Table Spool
1	1	┆ ┆ ┆ ┆--Clustered Index Seek(OBJECT:([tuning].[dbo].[TB_Exd2].[CIDX_Exd2]), SEEK:([tuning].[dbo].[TB_Exd2].[main_code]=[tuning].[dbo].[TB_Maind2].[main_code]) Ordered Forward)




```
테이블 'TB_Maind2'. 검색 수 9, 논리적 읽기 수 11332
테이블 'TB_Exd2'. 검색 수 1, 논리적 읽기 수 3
테이블 'Worktable'. 검색 수 2, 논리적 읽기 수 7
SQL Server 실행 시간:
CPU 시간 = 144ms, 경과 시간 = 21ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Exd2	CIDX_Exd2	CLUSTERED	0	main_code	-
TB_Maind2	PK_Maind2	CLUSTERED	1	main_code	-

· 제약조건 정보

Table_Name	Constraint_Type	Constraint_Name	Constraint_Keys
TB_Exd2	FOREIGN KEY	FK_Maind2_Exd2	main_code REFERENCES dbo.TB_Maind2 (main_code)

4. [AS-IS] 구문 설명

[TB_Maind2] 테이블의 [seq] 컬럼 값이 64570인 데이터를 삭제하는 구문이다. [TB_Maind2] 테이블의 [main_code] 컬럼을 기준으로 [TB_Exd2] 테이블의 [main_code] 컬럼이 CASCADE 옵션 설정으로 FK 제약조건이 구성되어 있기 때문에 [TB_Maind2]의 데이터가 삭제될 때마다 연결된 [TB_Exd2]의 데이터도 같이 삭제가 수행된다. DELETE 구문을 수행하는 과정에서 [TB_Maind2] 테이블의 [seq] 컬럼에 대한 인덱스 누락으로 클러스터형 인덱스를 전체 스캔하면서 높은 I/O가 발생한다.

5. [TO-BE] 튜닝 포인트

[TB_Maind2] 테이블을 삭제하는 과정에서 조건절의 [seq] 컬럼에 대한 값을 찾기 위해 클러스터형 인덱스 전체 스캔이 수행된다. 조건절에 있는 컬럼 대상으로 인덱스를 생성하면 필요한 행만 탐색하기 때문에 I/O를 감소시킬 수 있다.

6. [TO-BE] 개선 방안

1) [seq] 컬럼에 대한 신규 인덱스 생성

[seq] 컬럼에 인덱스를 생성하여 전체 스캔하는 비용을 제거하고 인덱스 탐색으로 I/O 비용을 감소시킨다.

■ 인덱스 생성 구문

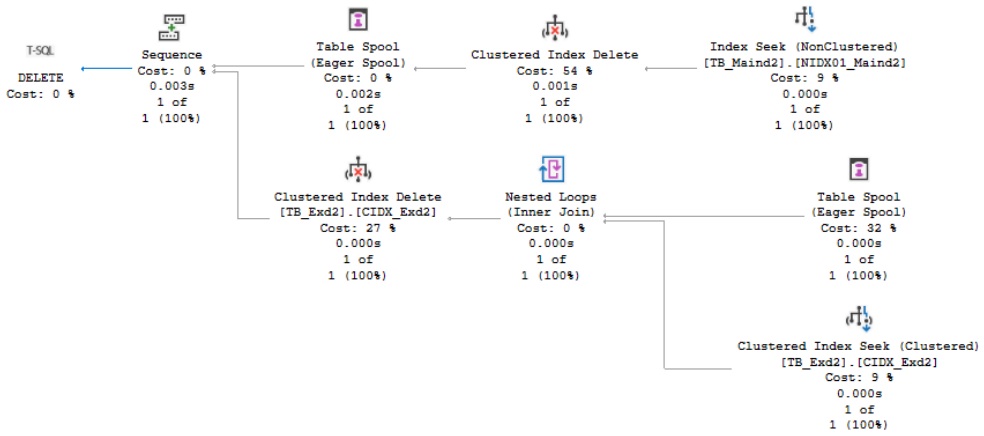
```
CREATE INDEX NIDX01_Maind2 ON TB_Maind2(seq)
GO
```

■ SQL 구문

```
DELETE FROM TB_Maind2 WHERE seq = 64570
GO
```

· 실행 계획

Rows	Executes	StmtText
1	1	DELETE [TB_Maind2] WHERE [seq]=@1
1	1	┆--Sequence
1	1	┆--Table Spool
1	1	┆ ┆--Clustered Index Delete(OBJECT:([tuning].[dbo].[TB_Maind2].[PK_Maind2]), OBJECT:([tuning].[dbo].[TB_Maind2].[NIDX01_Maind2]))
1	1	┆ ┆--Index Seek(OBJECT:([tuning].[dbo].[TB_Maind2].[NIDX01_Maind2]), SEEK:([tuning].[dbo].[TB_Maind2].[seq]=(64570)) Ordered Forward)
1	1	┆--Clustered Index Delete(OBJECT:([tuning].[dbo].[TB_Exd2].[CIDX_Exd2]))
1	1	┆--Nested Loops(INNER JOIN, OUTER REFERENCES:([tuning].[dbo].[TB_Maind2].[main_code]))
1	1	┆--Table Spool
1	1	┆--Clustered Index Seek(OBJECT:([tuning].[dbo].[TB_Exd2].[CIDX_Exd2]), SEEK:([tuning].[dbo].[TB_Exd2].[main_code]=[tuning].[dbo].[TB_Maind2].[main_code]) Ordered Forward)



테이블 'TB_Exd2'. 검색 수 1, 논리적 읽기 수 3
테이블 'Worktable'. 검색 수 2, 논리적 읽기 수 7
테이블 'TB_Maind2'. 검색 수 1, 논리적 읽기 수 12
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	144	21	11,342
TO-BE	0	0	22

1. 프롬프트

> [AS-IS]
> 구문에서 과도한 I/O가 발생되는 문제 원인 및 개선안을 알려줘.

```
[AS-IS]
DELETE FROM TB_Maind2 WHERE seq = 64570;
```

2. 분석 및 주요 최적화

1) 테이블 구조

```
ErDiagram
    TB_Maind2 ¦¦--o{ TB_Exd2 : "FK_Maind2_Exd2 (ON DELETE CASCADE)"
    TB_Maind2 {
        bigint seq "인덱스 없음 ⚠"
        uniqueidentifier main_code "PK (Clustered)"
        datetime date
        uniqueidentifier no2
        uniqueidentifier no3
        uniqueidentifier no4
    }
    TB_Exd2 {
        bigint seq
        uniqueidentifier main_code "FK → Clustered Index"
        datetime date
        uniqueidentifier no2
        uniqueidentifier no3
        uniqueidentifier no4
    }
```

2) Query Store 실측

항목	값
Logical Reads	11,369 pages
Duration	67,445 µs (~67ms)
대상 행	0건 (seq=64570 없음)

```
DELETE FROM TB_Maind2 WHERE seq = 64570
|
├─ 1단계: TB_Maind2에서 seq = 64570 찾기
|  └─ PK_Maind2는 main_code 기준 → seq로 Seek 불가
|  └─ ⚠ Clustered Index Scan → 11,293 pages (88MB) 전체 읽기
|
└─ 2단계: CASCADE DELETE → TB_Exd2에서 연쇄 삭제
   └─ CIDX_Exd2 (main_code) → Index Seek → 빠름 ✔

자식 테이블(TB_Exd2)이 아닌, 부모 테이블(TB_Maind2) 자체의 seq 검색이 병목
입니다.
```

*seq 컬럼에 인덱스 없음

3. 개선안

```
-- seq 컬럼에 넌클러스터드 인덱스 생성

CREATE NONCLUSTERED INDEX NIDX_TB_Maind2_seq ON dbo.TB_Maind2 (seq);
```

*클러스터드 인덱스(PK_Maind2)는 이미 main_code에 존재하므로, seq에는 넌클러스터드 인덱스를 추가합니다.

- **예상 개선 효과**

항목	현재	개선 후
TB_Maind2 탐색	Clustered Index Scan (11,293 pages)	Index Seek (~3 pages)
Logical Reads	11,369	~6 (Seek + RID/Key Lookup)
0건 매칭 시	88MB 전체 스캔	즉시 0건 확인

55 DELETE 구문의 성능 개선 - 2

다른 테이블과 외래 키 관계가 있는 테이블의 데이터를 삭제할 때 발생하는 성능 이슈 사례이다.

1. 사전 구성 스크립트

```
CREATE TABLE TB_Main03 (seq BIGINT
      , main_code UNIQUEIDENTIFIER NOT NULL
      , date DATETIME)

INSERT INTO TB_Main03
```

```
SELECT rownum AS seq, NEWID() AS main_code
      , DATEADD(MINUTE,rownum,'2020-01-01 00:00:00.000') AS date
FROM (SELECT ROW_NUMBER() OVER (ORDER BY a.number) AS rownum, a.number
       FROM master..spt_values a
       JOIN master..spt_values b
       ON a.type = 'P' AND b.type = 'P' AND a.number <= 1000 AND b.number <= 1000
) a

SELECT main_code AS sub_code, CAST(seq%5326 AS INT) AS value
INTO TB_Sub03
FROM TB_Main03

ALTER TABLE TB_Main03 ADD CONSTRAINT PK_Main03_main_code PRIMARY KEY
NONCLUSTERED(main_code)
ALTER TABLE TB_Sub03 ADD CONSTRAINT FK_TB_Sub03 FOREIGN KEY (sub_code)
REFERENCES TB_Main03(main_code) ON DELETE CASCADE
UPDATE STATISTICS TB_Main03(PK_Main03_main_code) WITH FULLSCAN
GO
```

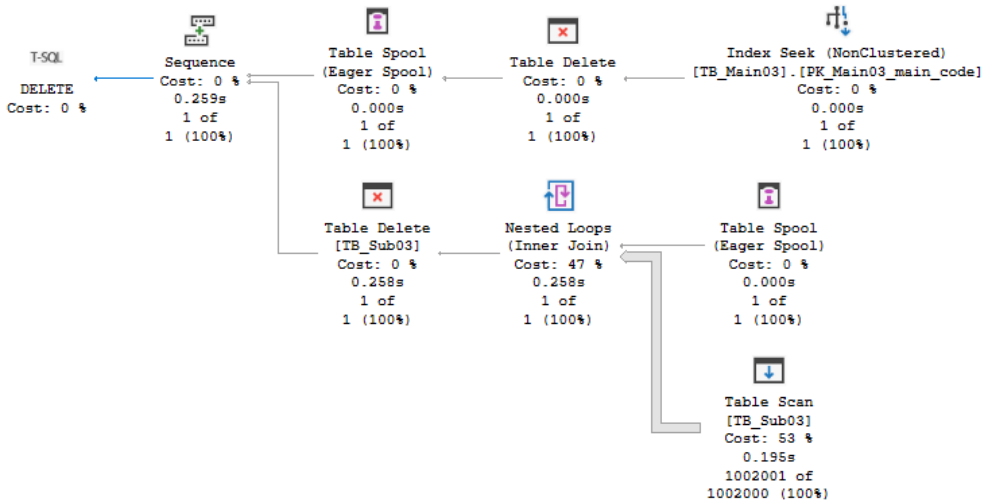
2. [AS-IS] SQL 구문 및 실행 계획

SQL 구문

```
/* TB_Main03 테이블의 임의의 Main_Code값을 선택하여 실행 */

DELETE FROM TB_Main03 WHERE Main_Code = '970A8E62-2F25-4D14-B167-F049BBA42620'
GO
```

- 실행 계획

Rows	Executes	StmtText
1	1	DELETE [TB_Main03] WHERE [Main_Code]=@1
1	1	¦--Sequence
1	1	¦--Table Spool
1	1	¦ ¦--Table Delete(OBJECT:([Tuning].[dbo].[TB_Main03]), OBJECT:([Tuning].[dbo].[TB_Main03].[PK_Main03_main_code]))
1	1	¦ ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_Main03].[PK_Main03_main_code]), SEEK:([Tuning].[dbo].[TB_Main03].[main_code]={guid'970A8E62-2F25-4D14-B167-F049BBA42620'}) Ordered Forward)
1	1	¦--Table Delete(OBJECT:([Tuning].[dbo].[TB_Sub03]))
1	1	¦--Nested Loops(INNER JOIN, WHERE:([Tuning].[dbo].[TB_Sub03].[sub_code]=[Tuning].[dbo].[TB_Main03].[main_code]))
1	1	¦--Table Spool
1002001	1	¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Sub03]))




```
테이블 'TB_Sub03'. 스캔 수 1, 논리적 읽기 3594
테이블 'Worktable'. 스캔 수 2, 논리적 읽기 7
테이블 'TB_Main03'. 스캔 수 0, 논리적 읽기 7
SQL Server 실행 시간:
CPU 시간 = 218ms, 경과 시간 = 217ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Main03	PK_Main03_main_code	NONCLUSTERED	1	main_code	-

· 제약조건 정보

Table_Name	Constraint_Name	Type_Desc	DELETE_Action	Constraint_Key
TB_Sub03	FK_TB_Sub03	FOREIGN KEY	Cascade	sub_code REFERENCES Tuning.dbo.TB_Main03 (main_code)

4. [AS-IS] 구문 설명

[TB_Main03] 테이블 [main_code] 컬럼의 특정 값 1건을 삭제하는 구문이다. [TB_Main03] 테이블의 [main_code] 컬럼은 [TB_Sub03] 테이블의 [sub_code] 컬럼에 외래 키로 참조되고 있으며, [TB_Sub03] 테이블의 외래 키는 CASCADE 옵션이 적용되어 기준 테이블인 [TB_Main03]의 데이터가 삭제될 때 [TB_Sub03] 테이블 데이터도 자동으로 삭제된다. 해당 구문에 의해 [TB_Sub03] 테이블이 삭제되는 과정에서 인덱스가 존재하지 않아 테이블을 전체 스캔하면서 높은 I/O 비용이 사용된다.

5. [TO-BE] 튜닝 포인트

참조 데이터를 찾는 과정에서 인덱스로 탐색할 수 있도록 외래 키로 참조하고 있는 [sub_code] 컬럼에 인덱스를 생성한다.

6. [TO-BE] 개선 방안

1) [sub_code] 컬럼이 키로 구성된 신규 인덱스 생성

[TB_Sub03] 테이블의 [sub_code] 컬럼에 인덱스를 생성하면 삭제 대상 데이터를 찾는 과정에서 인덱스로 탐색하기 때문에 I/O 비용이 감소된다.

■ 인덱스 생성 구문

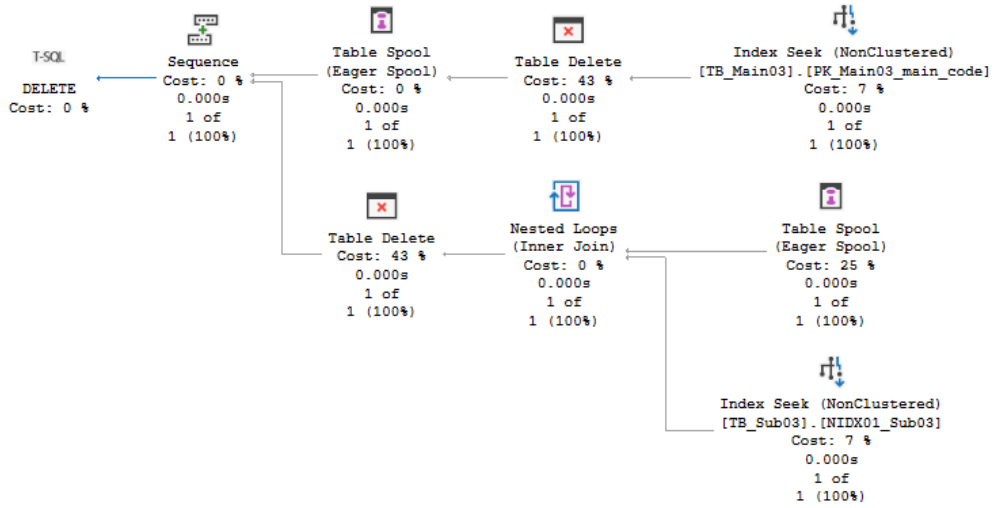
```
CREATE INDEX NIDX01_Sub03 ON TB_Sub03(sub_code)
GO
```

■ SQL 구문

```
/* TB_Main03 테이블의 임의의 Main_Code값을 선택하여 실행 */

DELETE FROM TB_Main03 WHERE Main_Code = 'B2AF8B45-A54C-439F-A99D-6138090FCB0A'
GO
```

· 실행 계획

Rows	Executes	StmtText
1	1	DELETE [TB_Main03] WHERE [Main_Code]=@1
1	1	┣--Sequence
1	1	┃ ┣--Table Spool
1	1	┃ ┃ ┣--Table Delete(OBJECT:([Tuning].[dbo].[TB_Main03]), OBJECT:([Tuning].[dbo].[TB_Main03].[PK_Main03_main_code]))
1	1	┃ ┃ ┗--Index Seek(OBJECT:([Tuning].[dbo].[TB_Main03].[PK_Main03_main_code]), SEEK:([Tuning].[dbo].[TB_Main03].[main_code]={guid'B2AF8B45-A54C-439F-A99D-6138090FCB0A'}) Ordered Forward)
1	1	┣--Table Delete(OBJECT:([Tuning].[dbo].[TB_Sub03]), OBJECT:([Tuning].[dbo].[TB_Sub03].[NIDX01_Sub03]))
1	1	┗--Nested Loops(INNER JOIN, OUTER REFERENCES:([Tuning].[dbo].[TB_Main03].[main_code]))
1	1	┣--Table Spool
1	1	┗--Index Seek(OBJECT:([Tuning].[dbo].[TB_Sub03].[NIDX01_Sub03]), SEEK:([Tuning].[dbo].[TB_Sub03].[sub_code]=[Tuning].[dbo].[TB_Main03].[main_code]) Ordered Forward)




```
테이블 'TB_Sub03'. 스캔 수 1, 논리적 읽기 10
테이블 'Worktable'. 스캔 수 2, 논리적 읽기 7
테이블 'TB_Main03'. 스캔 수 0, 논리적 읽기 7
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 17ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	218	217	3,608
TO-BE	0	17	24

GitHub Copilot 채팅 결과

Claude Opus 4.6

55. DELETE 구문의 성능 개선 - 2

1. 프롬프트

> [AS-IS]
> 구문에서 과도한 I/O가 발생되는 문제 원인 및 개선안을 알려줘.

```
[AS-IS]
DELETE FROM TB_Main03 WHERE main_code = 'E7523874-300A-4B28-82E6-BDEBA143EBAB';
```

2. 분석 및 주요 최적화

1) 테이블 구조

```
erDiagram
    TB_Main03 ¦¦--o{ TB_Sub03 : "FK_TB_Sub03 (ON DELETE CASCADE)"
    TB_Main03 {
        bigint seq
        uniqueidentifier main_code PK
        datetime date
    }
    TB_Sub03 {
        uniqueidentifier sub_code FK
        int value
    }
```

2) 현황 요약

항목	TB_Main03	TB_Sub03
행 수	1,002,000	1,002,000
구조	Heap + NC PK (main_code)	Heap (인덱스 없음)
FK 관계	부모 (main_code)	자식 (sub_code) → CASCADE DELETE

3) Query Store 실측 (DELETE 실행)

항목	값
Logical Reads	3,623 pages
Duration	224,832 µs (~225ms)

*자식 테이블 TB_Sub03에 인덱스 없음

```
DELETE 실행 시 다음 과정이 발생합니다:
DELETE FROM TB_Main03 WHERE main_code = '455830C1-...'
│
├─ 1단계: TB_Main03에서 행 찾기
│  └─ PK_Main03_main_code (NC Index Seek) → 빠름 ✅
│
└─ 2단계: CASCADE DELETE → TB_Sub03에서 연쇄 삭제
   └─ sub_code = '455830C1-...' 인 행 찾기
   └─ TB_Sub03에 인덱스 없음 (Heap)
   └─ ⚠️ Full Table Scan → 100만 행 전체 읽기 (28MB)
```

*TB_Main03 자체는 PK 인덱스로 빠르게 찾지만, CASCADE DELETE가 TB_Sub03 전체를 스캔하면서 대부분의 I/O가 발생합니다.

3. 개선안

```
-- TB_Sub03.sub_code에 인덱스 생성

CREATE CLUSTERED INDEX CIDX_TB_Sub03_sub_code ON dbo.TB_Sub03 (sub_code);
```

· 예상 개선 효과

항목	현재 (Heap)	개선 후 (Clustered Index)
CASCADE DELETE 방식	Table Scan (100만 행)	Index Seek
Logical Reads (추정)	~3,600 pages	~3 pages
자식 테이블 I/O 비중	전체 I/O의 대부분	거의 무시 가능

*FK의 자식 테이블 FK 컬럼에 인덱스가 없으면, 부모 행 DELETE/UPDATE 시마다 자식 테이블을 Full Scan합니다. 이는 CASCADE 여부와 관계없이 FK 무결성 검증 과정에서 항상 발생하는 문제입니다.

56 DELETE 구문의 성능 개선 - 3

데이터를 삭제할 때 인덱스 부재로 인해 테이블 전체를 읽어내면서 I/O가 증가된 사례이다.

1. 사전 구성 스크립트

```
SELECT ROW_NUMBER() OVER (ORDER BY a.number) id
     , CONVERT(NVARCHAR(36),NEWID()) code
     , DATEADD(dd,a.number,'2001-01-01') date
INTO TB_Del01
FROM master..spt_values a, master..spt_values b
WHERE a.type = 'P'
AND a.number BETWEEN 1 AND 1000
AND b.type = 'P'
AND b.number BETWEEN 1 AND 1000
GO
```

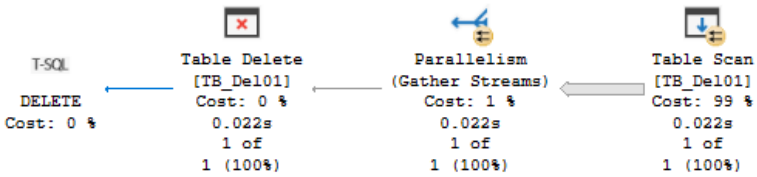
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
DELETE TB_Del01
WHERE id = 57
GO
```

· 실행 계획

Rows	Executes	StmtText
1 1 1 1	1 1 1 12	DELETE [TB_Del01] WHERE [id]=@1 ¦--Table Delete(OBJECT:([Tuning].[dbo].[TB_Del01])) ¦--Parallelism(Gather Streams) ¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Del01]), WHERE:([Tuning].[dbo].[TB_Del01].[id]=CONVERT_IMPLICIT(bigint,[@1],0)) ORDERED)




```
테이블 'TB_Del01'. 검색 수 13, 논리적 읽기 수 12506
SQL Server 실행 시간:
CPU 시간 = 174ms, 경과 시간 = 13ms
```

3. [AS-IS] 통계 정보

· 인덱스 없음

4. [AS-IS] 구문 설명

[TB_Del01] 테이블의 [id] 컬럼 값이 57인 데이터를 삭제하는 구문이다. 테이블의 총 데이터는 100만 건이고, 이 중 조건절에 해당되는 데이터 1건이지만 [id] 컬럼에 인덱스가 존재하지 않아 테이블의 모든 데이터를 읽어내면서 많은 I/O가 사용된다.

5. [TO-BE] 튜닝 포인트

조건에 해당되는 데이터를 탐색할 수 있도록 [id] 컬럼에 인덱스를 생성한다. 인덱스 탐색을 통해 테이블 전체를 읽어내면서 사용된 I/O를 개선할 수 있다.

6. [TO-BE] 개선 방안

1) [id] 컬럼에 신규 인덱스 생성

신규 인덱스를 생성하여 삭제 대상 행만 인덱스로 탐색한다.

■ 인덱스 생성 구문

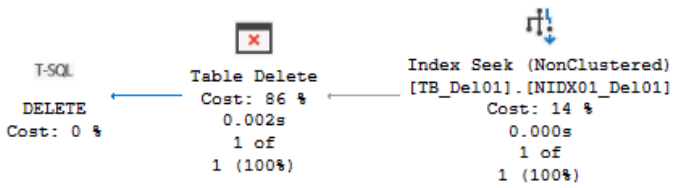
```
CREATE INDEX NIDX01_Del01 ON TB_Del01(id)
GO
```

■ SQL 구문

```
DELETE TB_Del01
WHERE id = 57
GO
```

· 실행 계획

Rows	Executes	StmtText
1	1	DELETE [TB_Del01] WHERE [id]=@1
1	1	¦--Table Delete(OBJECT:([Tuning].[dbo].[TB_Del01]), OBJECT:([Tuning].[dbo].[TB_Del01].[NIDX01_Del01]))
1	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_Del01].[NIDX01_Del01]), SEEK:([Tuning].[dbo].[TB_Del01].[id]=CONVERT_IMPLICIT(bigint,[@1],0)) Ordered Forward)



테이블 'TB_Del01'. 검색 수 1, 논리적 읽기 수 10
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	174	13	12,506
TO-BE	0	0	10

1. 프롬프트

> [AS-IS]
> 구문에 성능 측면에서 비효율이 발생되는 문제 원인 및 개선안을 알려줘.

```
[AS-IS]
DELETE TB_Del01
WHERE id = 57;
```

2. 분석 및 주요 최적화

1) 테이블 현황

항목	값
테이블	dbo.TB_Del01
행 수	999,999
인덱스	없음 (Heap)
트리거	없음
FK 참조	없음
id = 57 대상 행	0건

2) Query Store에서 확인된 I/O

쿼리	Logical Reads
WHERE id = 57 (COUNT)	12,512
SELECT MIN/MAX(id)	12,512
GROUP BY id	12,512

*모든 쿼리가 동일한 12,512 페이지를 읽고 있습니다. → Full Table Scan

id 컬럼에 인덱스가 전혀 없습니다. Heap 테이블이므로 WHERE id = 57 조건이 있더라도 100만 행 전체를 스캔해야 합니다. 심지어 일치하는 행이 0건인데도 전체 테이블을 읽습니다.

```
DELETE 실행 흐름:
    TB_Del01 (Heap, 999,999 rows)
    └── Table Scan → 12,512 pages 전체 읽기
        └── id = 57 필터링 → 0건 발견 → 삭제할 것 없음
```

3. 개선안

```
-- 해결: 클러스터드 인덱스 생성

CREATE CLUSTERED INDEX CIDX_TB_Del01_id ON dbo.TB_Del01 (id);
```

· **예상 개선 효과**

항목	현재 (Heap)	개선 후 (Clustered Index)
Scan 방식	Table Scan	Index Seek
Logical Reads	12,512 pages	~3 pages
시간 복잡도	O(n) - 100만 행	O(log n) - 약 3단계
0건 매칭 시	전체 스캔 후 0건	즉시 0건 확인

*id가 NULL 허용(IS_NULLABLE = YES)으로 설정되어 있습니다. id가 실제로 고유 식별자 역할을 한다면, NOT NULL 제약 조건 추가 후 Primary Key로 지정하는 것이 더 바람직합니다.

57 UPDATE / INSERT를 MERGE 문으로 통합

UPDATE와 INSERT를 수행하기 위해서 동일한 테이블을 여러 번 읽어내야 하는 비효율을 MERGE 문으로 통합하여 개선하는 사례이다.

1. 사전 구성 스크립트

```
/* 변경을 위해 기준이 되는 테이블 생성 */

SELECT ROW_NUMBER() OVER (ORDER BY a.number) id
      , NEWID() code
      , DATEADD(dd,a.number,'2020-01-01') date
INTO TB_Old
FROM master..spt_values a, master..spt_values b
WHERE a.type = 'P' AND a.number BETWEEN 1 AND 1000
AND b.type = 'P' AND b.number BETWEEN 1 AND 1000


/* 변경 대상이 되는 테이블 생성 */

SELECT id
      , CASE WHEN id%700 = 0 THEN NEWID()
        ELSE code END code
      , date
```

```
INTO TB_New
FROM TB_Old
WHERE id % 1000 <> 0
GO
```

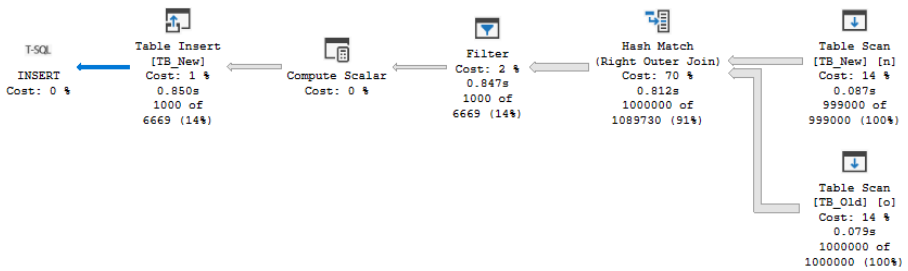
2. [AS-IS] SQL 구문 및 실행 계획

SQL 구문

```
/* TB_Old과 TB_New의 조인 되지 않는 Row를 TB_New 테이블에 추가 */

INSERT INTO TB_New
SELECT id, code, GETDATE() AS date
FROM (SELECT o.*, n.id as nid
      FROM TB_Old o
      LEFT OUTER JOIN TB_New n
      ON o.id = n.id
) o
WHERE o.nid IS NULL


/* TB_Old과 TB_New의 조인되는 Row를 TB_Old 테이블 기준으로 Update */

UPDATE n
SET n.code = o.code, n.date = GETDATE()
FROM TB_Old o, TB_New n
WHERE o.id = n.id AND o.code <> n.code
GO
```

- **실행 계획 (INSERT)**

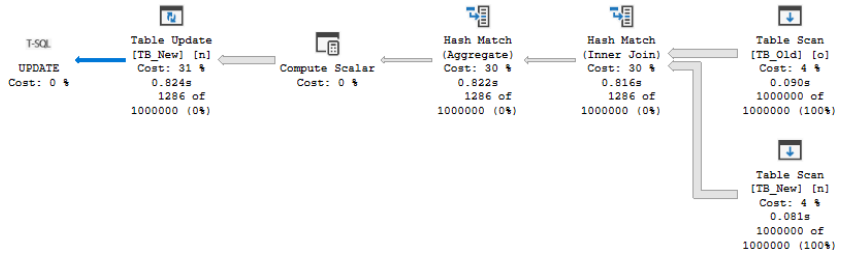
Rows	Executes	StmtText
1000	1	INSERT INTO TB_New SELECT id, code, GETDATE() AS date FROM (SELECT o.*,n.id as nid FROM TB_Old o LEFT OUTER JOIN TB_New n ON o.id = n.id) o WHERE o.nid IS NULL
1000	1	¦--Table INSERT(OBJECT:([Tuning].[dbo].[TB_New]), SET:([Tuning].[dbo].[TB_New].[id] = [Tuning].[dbo].[TB_Old].[id] as [o].[id],[Tuning].[dbo].[TB_New].[code] = [Tuning].[dbo].[TB_Old].[code] as [o].[code],[Tuning].[dbo].[TB_New].[date] = [Expr1007]))
0	0	¦--Compute Scalar(DEFINE:([Expr1007]=getdate()))
1000	1	¦--Filter(WHERE:([Tuning].[dbo].[TB_New].[id] as [n].[id] IS NULL))
1000000	1	¦--Hash Match(Right Outer Join, HASH:([n].[id])=([o].[id]), RESIDUAL:([Tuning].[dbo].[TB_Old].[id] as [o].[id]=[Tuning].[dbo].[TB_New].[id] as [n].[id]))
999000	1	¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_New] AS [n]))
1000000	1	¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Old] AS [o]))



```
테이블 'TB_New'. 검색 수 1, 논리적 읽기 수 6077
테이블 'Workfile'. 검색 수 0, 논리적 읽기 수 0
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_Old'. 검색 수 1, 논리적 읽기 수 5077
SQL Server 실행 시간:
CPU 시간 = 1328ms, 경과 시간 = 1678ms
```

· **실행 계획 (UPDATE)**

Rows	Executes	StmtText
1286	1	UPDATE n SET n.code=o.code, n.date=GETDATE() FROM TB_Old o, TB_New n WHERE o.id=n.id AND o.code <> n.code
1286	1	¦--Table Update(OBJECT:([Tuning].[dbo].[TB_New] AS [n]), SET:([Tuning].[dbo].[TB_New].[code] as [n].[code] = [Tuning].[dbo].[TB_Old].[code] as [o].[code],[Tuning].[dbo].[TB_New].[date] as [n].[date] = [Expr1004]))
0	0	¦--Compute Scalar(DEFINE:([Expr1004]=getdate()))
1286	1	¦--Hash Match(Aggregate, HASH:([Bmk1002]), RESIDUAL:([Bmk1002] = [Bmk1002]) DEFINE:([o].[code]=ANY([Tuning].[dbo].[TB_Old].[code] as [o].[code])))
1286	1	¦--Hash Match(INNER JOIN, HASH:([n].[id])=([o].[id]), RESIDUAL:([Tuning].[dbo].[TB_Old].[id] as [o].[id]=[Tuning].[dbo].[TB_New].[id] as [n].[id] AND [Tuning].[dbo].[TB_Old].[code] as [o].[code]<>[Tuning].[dbo].[TB_New].[code] as [n].[code]))
1000000	1	¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_New] AS [n]))
1000000	1	¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Old] AS [o]))




```
테이블 'TB_New'. 검색 수 1, 논리적 읽기 수 6363
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'Workfile'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_Old'. 검색 수 1, 논리적 읽기 수 5077
SQL Server 실행 시간:
CPU 시간 = 1484ms, 경과 시간 = 1516ms
```

3. [AS-IS] 구문 설명

[TB_New] 테이블을 [TB_Old] 테이블에 맞춰 데이터 동기화를 위해 DML을 사용하는 구문이다. [TB_New] 테이블에 존재하지 않는 데이터는 INSERT를 수행하고 [TB_Old] 테이블과 [code] 값이 다른 데이터는 UPDATE를 수행한다. 데이터를 삽입, 수정하는 과정에서 동일 테이블을 각 2회씩 읽어내야 하기 때문에 비효율이 존재한다.

4. [TO-BE] 튜닝 포인트

두 테이블을 조인하여 조건에 만족하는 데이터는 UPDATE를 수행하고 만족하지 않는 데이터는 INSERT를 수행하는 경우라면 테이블을 한 번씩만 읽어내면서 동시에 수행할 수 있는 MERGE 문을 활용할 수 있다. 두 구문을 MERGE 문으로 통합하여 비효율적인 I/O를 감소시킨다.

5. [TO-BE] 개선 방안

1) INSERT와 UPDATE를 MERGE 문으로 통합

INSERT와 UPDATE 구문을 MERGE 문으로 통합하여 동시에 처리하도록 개선한다.

■ 변경된 SQL 구문

```
MERGE INTO TB_New n
USING (SELECT * FROM TB_Old) o
ON  o.id = n.id
WHEN MATCHED AND o.code <> n.code THEN
UPDATE SET n.code = o.code, n.date = GETDATE()
WHEN NOT MATCHED THEN
INSERT (id, code, date) VALUES (o.id, o.code, GETDATE());
GO
```

· 실행 계획

Rows	Executes	StmtText
2286	1	MERGE INTO TB_New n USING (SELECT * FROM TB_Old) o ON o.id = n.id WHEN MATCHED AND o.code <> n.code THEN UPDATE SET n.code = o.code, n.date = GETDATE() WHEN NOT MATCHED THEN INSERT (id, code, date) VALUES (o.id, o.code, GETDATE())
2286	1	┦--Table Merge(OBJECT:([Tuning].[dbo].[TB_New] AS [n]), SET:(INSERT, [Tuning].[dbo].[TB_New].[date] as [n].[date] = [Expr1008],[Tuning].[dbo].[TB_New].[id] as [n].[id] = [Expr1009],[Tuning].[dbo].[TB_New].[code] as [n].[code] = [Expr1010]), SET:(Update, [Tuning].[dbo].[TB_New].[date] as [n].[date] = [Expr1008],[Tuning].[dbo].[TB_New].[code] as [n].[code] = [Expr1010]) ACTION:([Action1007]))
2286	1	┦--Assert(WHERE:(CASE WHEN [Expr1017]>(1) THEN (0) ELSE NULL END))
2286	1	┦--Sequence Project(DEFINE:([Expr1017]=conditional_row_number))
2286	1	┦--Segment
2286	1	┦--Sort(ORDER BY:([Bmk1003] ASC))
0	0	┦--Compute Scalar(DEFINE:([Expr1008]=getdate(), [Expr1009]=CASE WHEN [Action1007]=(4) THEN [Tuning].[dbo].[TB_Old].[id] ELSE [Tuning].[dbo].[TB_New].[id] as [n].[id] END))
2286	1	┦--Filter(WHERE:([Action1007] IS NOT NULL))
0	0	┦--Compute Scalar(DEFINE:([Expr1010]=[Tuning].[dbo].[TB_Old].[code]))
1000000	1	┦--Compute Scalar(DEFINE:([Action1007]=ForceOrder(CASE WHEN [TrgPrb1005] IS NOT NULL THEN CASE WHEN [Tuning].[dbo].[TB_Old].[code]<>[Tuning].[dbo].[TB_New].[code] as [n].[code] THEN (1) ELSE NULL END ELSE (4) END)))
1000000	1	┦--Hash Match(LEFT OUTER JOIN, HASH:([Tuning].[dbo].[TB_Old].[id])=([n].[id]), RESIDUAL:([Tuning].[dbo].[TB_Old].[id] = [Tuning].[dbo].[TB_New].[id] as [n].[id]))
1000000	1	┦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Old]))
0	0	┦--Compute Scalar(DEFINE:([TrgPrb1005]=(1)))
999000	1	┦--Table Scan(OBJECT:([Tuning].[dbo].[TB_New] AS [n]))

```
테이블 'TB_New'. 검색 수 1, 논리적 읽기 수 7363
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'Workfile'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_Old'. 검색 수 1, 논리적 읽기 수 5077
SQL Server 실행 시간:
CPU 시간 = 1469ms, 경과 시간 = 2201ms
```

6. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	2,812	3,194	22,594
TO-BE	1,469	2,201	12,440

AI

GitHub Copilot 채팅 결과

Claude Opus 4.6

57. UPDATE / INSERT를 MERGE 문으로 통합

1. 프롬프트

> [AS-IS]
> 구문은 TB_Old 테이블과 TB_New 테이블을 동기화하기 위한 것으로, 신규 데이터 삽입(INSERT)과 기존 데이터 변경(UPDATE) 처리 시 동일 테이블을 두 차례 조회하여 성능 비효율이 발생한다. 이 문제를 분석하고 성능 개선안을 알려줘.

```
[AS-IS]
INSERT INTO TB_New
SELECT id, code, GETDATE() AS date
FROM (SELECT o.*, n.id as nid
              FROM TB_Old o
              LEFT OUTER JOIN TB_New n
              ON o.id = n.id
) o
WHERE o.nid IS NULL

UPDATE n
SET n.code = o.code, n.date = GETDATE()
FROM TB_Old o, TB_New n
WHERE o.id = n.id AND o.code <> n.code;
```

2. 분석 및 주요 최적화

· 현재 문제점

구분	현재 방식	문제
INSERT	TB_Old LEFT JOIN TB_New → 1회 스캔	TB_Old 1차 스캔
UPDATE	TB_Old, TB_New → 1회 스캔	TB_Old 2차 스캔
합계	TB_Old 2번, TB_New 2번 Full Scan	각 100만 행 × 2회 = 불필요한 중복 I/O

두 테이블 모두 인덱스가 전혀 없는 Heap 테이블이므로, 매번 Full Table Scan이 발생합니다.

3. 개선안

```
-- 해결: MERGE 문으로 단일 패스 처리 MERGE를 사용하면 한 번의 JOIN으로 INSERT와
-- UPDATE를 동시에 처리합니다.

MERGE dbo.TB_New AS n
USING dbo.TB_Old AS o
ON n.id = o.id
WHEN MATCHED AND o.code <> n.code THEN
UPDATE SET n.code = o.code, n.date = GETDATE()
WHEN NOT MATCHED BY TARGET THEN
INSERT (id, code, date) VALUES (o.id, o.code, GETDATE());
```

· 변경 사항 요약

항목	기존	개선
테이블 스캔 횟수	TB_Old 2회 + TB_New 2회	TB_Old 1회 + TB_New 1회
구문 수	INSERT + UPDATE (2개)	MERGE (1개)
JOIN 방식	ANSI-89 암시적 JOIN	명시적 ON 절
트랜잭션 일관성	INSERT/UPDATE 사이 데이터 변경 가능	단일 원자적 작업

58 저장 프로시저 성능 이슈

저장 프로시저의 실행 계획 재사용으로 인해 성능 이슈가 발생되는 사례이다.

1. 사전 구성 스크립트

```
SELECT NO, DATE
      , CASE WHEN high=1 THEN 'Berlin' WHEN high=8 AND RIGHT(NO,3)=000 THEN 'Paris'
        ELSE 'London' END AS CITY
```

```
INTO TB_CITY
FROM (SELECT ROW_NUMBER() OVER (ORDER BY A.number) AS NO
            , CONVERT(CHAR(8), DATEADD (DD, A.number,'2020-01-01'), 112) AS DATE
            , high
       FROM master..spt_values A, (SELECT number
                                   FROM master..spt_values
                                   WHERE type = 'P' AND number < 500) B
       WHERE A.type = 'P'
) A

CREATE CLUSTERED INDEX CIDX_CITY ON TB_CITY(NO)
CREATE INDEX NIDX01_CITY ON TB_CITY(CITY)
GO

/* 저장 프로시저 생성문 */
CREATE PROCEDURE SP_CITY @CITY VARCHAR(20)
AS
SELECT CITY, COUNT(*)
FROM TB_CITY
WHERE CITY IN (@CITY)
AND DATE > '20200130'
GROUP BY CITY
GO
```

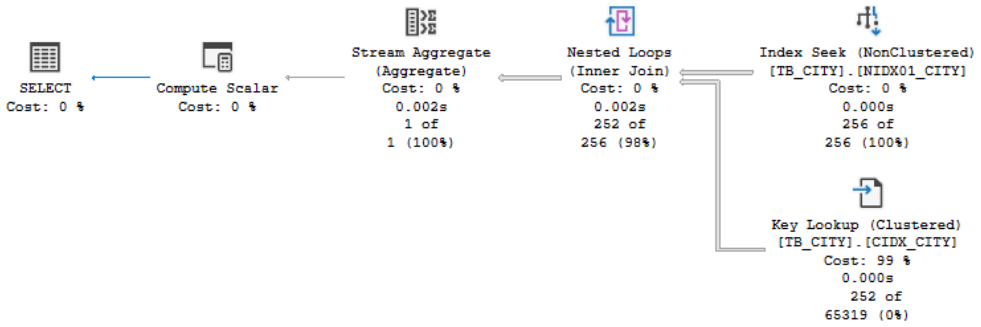
2. [AS-IS] SQL 구문 및 실행 계획

SQL 구문

```
EXEC SP_CITY 'Paris'
EXEC SP_CITY 'London'
GO
```

- 실행 계획 ('Paris' 매개변수 적용)

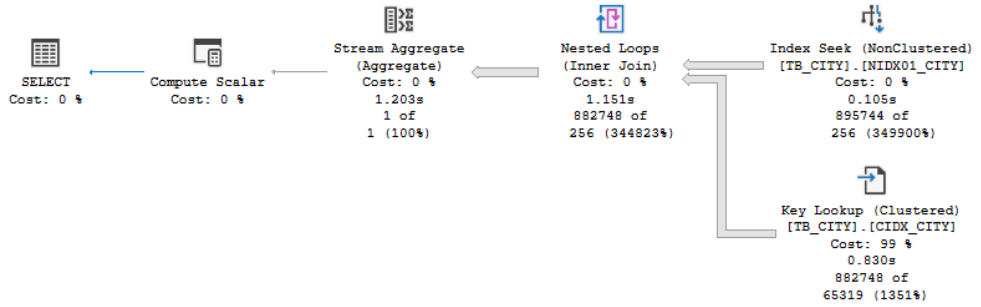
Rows	Executes	StmtText
1	1	SELECT CITY, COUNT(*) FROM TB_CITY WHERE CITY IN (@CITY) AND DATE > '20200130' GROUP BY CITY
0	0	¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(int,[Expr1007],0)))
1	1	¦--Stream Aggregate(DEFINE:([Expr1007]=Count(*), [Tuning].[dbo].[TB_CITY].[CITY]=ANY([Tuning].[dbo].[TB_CITY].[CITY])))
252	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([Uniq1001], [Tuning].[dbo].[TB_CITY].[NO], [Expr1006]) WITH UNORDERED PREFETCH)
256	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_CITY].[NIDX01_CITY]), SEEK:([Tuning].[dbo].[TB_CITY].[CITY]=[@CITY]) Ordered Forward)
252	256	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_CITY].[CIDX_CITY]), SEEK:([Tuning].[dbo].[TB_CITY].[NO]=[Tuning].[dbo].[TB_CITY].[NO] AND [Uniq1001]=[Uniq1001]), WHERE:([Tuning].[dbo].[TB_CITY].[DATE]>'20200130') LOOKUP Ordered Forward)



테이블 'TB_CITY'. 검색 수 1, 논리적 읽기 수 798
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms

· 실행 계획 ('London' 매개변수 적용)

Rows	Executes	StmtText
1	1	SELECT CITY, COUNT(*) FROM TB_CITY WHERE CITY IN (@CITY) AND DATE > '20200130' GROUP BY CITY
0	0	¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(int,[Expr1007],0)))
1	1	¦--Stream Aggregate(DEFINE:([Expr1007]=Count(*), [Tuning].[dbo].[TB_CITY].[CITY]=ANY([Tuning].[dbo].[TB_CITY].[CITY])))
882748	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([Uniq1001], [Tuning].[dbo].[TB_CITY].[NO], [Expr1006]) WITH UNORDERED PREFETCH)
895744	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_CITY].[NIDX01_CITY]), SEEK:([Tuning].[dbo].[TB_CITY].[CITY]=[@CITY]) Ordered Forward)
882748	895744	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_CITY].[CIDX_CITY]), SEEK:([Tuning].[dbo].[TB_CITY].[NO]=[Tuning].[dbo].[TB_CITY].[NO] AND [Uniq1001]=[Uniq1001]), WHERE:([Tuning].[dbo].[TB_CITY].[DATE]>'20200130') LOOKUP Ordered Forward)



테이블 'TB_CITY'. 검색 수 1, 논리적 읽기 수 2745897
SQL Server 실행 시간:
CPU 시간 = 1063ms, 경과 시간 = 1064ms

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_CITY	CIDX_CITY	CLUSTERED	0	NO	-
TB_CITY	NIDX01_CITY	NONCLUSTERED	0	CITY	-

· [TB_CITY] 테이블의 인덱스 PAGE 정보 (sys.sysindexes)

	Table_Name	Index_Name	dpages	rows
1	TB_CITY	CIDX_CITY	4702	1024000
2	TB_CITY	NIDX01_CITY	3039	1024000

4. [AS-IS] 구문 설명

저장 프로시저 [SP_CITY]에 매개변수를 서로 다른 값으로 하여 2회 실행시키는 구문이다. 프로시저 내부로 입력받는 매개변수는 [TB_CITY] 테이블 [CITY] 컬럼의 조건절로 사용되며 조건에 해당하는 값 중 [DATE] 컬럼이 '20200130' 이후에 해당되는 데이터를 추출하여 [CITY] 컬럼을 기준으로 집계한 행 수를 출력한다. 첫 번째로 입력받은 'Paris' 매개변수로 프로시저가 수행되면서 실행 계획이 생성되는데, [TB_CITY] 테이블에서 [CITY] = 'Paris'에 해당하는 데이터 256건을 인덱스로 탐색한 후 [DATE] 컬럼의 조건절을 적용하기 위해 클러스터형 인덱스에 Key Lookup을 수행한다. 이어서 'London'을 매개변수로 입력받아 수행하게 되는데, [CITY] = 'London'에 해당되는 데이터 약 80만 건에 대해 이전에 생성된 실행 계획이 재사용되면서 많은 양의 Key Lookup으로 과다한 I/O 비용이 발생한다.

5. [TO-BE] 튜닝 포인트

[DATE] 컬럼이 인덱스 키로 존재하지 않아서 Key Lookup 연산이 추가적으로 요구되는데, 프로시저 매개변수를 'London'으로 처음 실행했다면 많은 양의 Key Lookup 수행 비용을 고려하여 클러스터형 인덱스 전체를 읽어내는 것으로 실행 계획이 생성됐을 것이다. [DATE] 컬럼을 인덱스 키로 포함하면, 두 경우 모두 인덱스만 읽고 탐색하는 동일한 실행 계획으로 생성되기 때문에 실행 계획이 재사용될 때 데이터 분포 차이에 대한 성능적인 이슈가 발생되는 것을 해결할 수 있다.

6. [TO-BE] 개선 방안

1) [CITY] 컬럼과 [DATE] 컬럼이 키로 구성된 인덱스를 생성한다.

[TB_CITY] 테이블에 조건절로 사용된 [CITY] 컬럼과 [DATE] 컬럼을 키로 포함한 인덱스를 새로 생성한다. 이후 프로시저를 다시 실행하면 각 매개변수값에 대해 모두 동일한 실행 계획으로 수행되기 때문에 실행 계획 재사용으로 인한 과다한 I/O 사용을 개선할 수 있다.

■ 인덱스 생성 구문

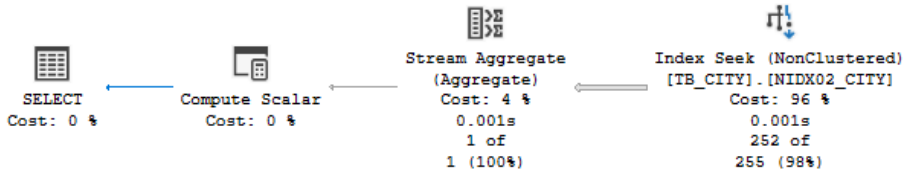
```
CREATE INDEX NIDX02_CITY ON TB_CITY(CITY, DATE)
GO
```

SQL 구문

```
EXEC SP_CITY 'Paris'
EXEC SP_CITY 'London'
GO
```

· 실행 계획 ('Paris')

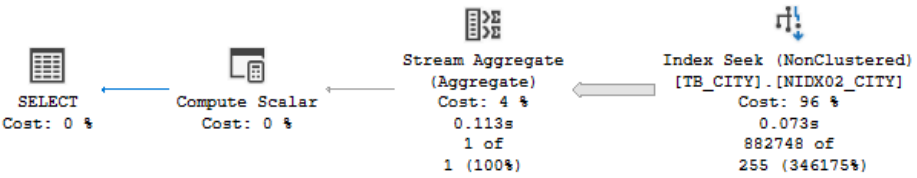
Rows	Executes	StmtText
1 0 1 252	1 0 1 1	SELECT CITY, COUNT(*) FROM TB_CITY WHERE CITY IN (@CITY) AND DATE > '20200130' GROUP BY CITY ¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(int,[Expr1006],0))) ¦--Stream Aggregate(DEFINE:([Expr1006]=Count(*), [Tuning].[dbo].[TB_CITY].[CITY]=ANY([Tuning].[dbo].[TB_CITY].[CITY]))) ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_CITY].[NIDX02_CITY]), SEEK:([Tuning].[dbo].[TB_CITY].[CITY] = [@CITY] AND [Tuning].[dbo].[TB_CITY].[DATE] > '20200130') Ordered Forward)



테이블 'TB_CITY'. 검색 수 1, 논리적 읽기 수 5
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms

· 실행 계획 ('London')

Rows	Executes	StmtText
1 0 1 882748	1 0 1 1	SELECT CITY, COUNT(*) FROM TB_CITY WHERE CITY IN (@CITY) AND DATE > '20200130' GROUP BY CITY ¦--Compute Scalar(DEFINE:([Expr1003]=CONVERT_IMPLICIT(int,[Expr1006],0))) ¦--Stream Aggregate(DEFINE:([Expr1006]=Count(*), [Tuning].[dbo].[TB_CITY].[CITY]=ANY([Tuning].[dbo].[TB_CITY].[CITY]))) ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_CITY].[NIDX02_CITY]), SEEK:([Tuning].[dbo].[TB_CITY].[CITY]=[@CITY] AND [Tuning].[dbo].[TB_CITY].[DATE] > '20200130') Ordered Forward)



테이블 'TB_CITY'. 검색 수 1, 논리적 읽기 수 3510
SQL Server 실행 시간:
CPU 시간 = 63ms, 경과 시간 = 73ms

7. 개선 효과

- 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	0	0	798
	1,063	1,064	2,745,897
TO-BE	0	0	5
	63	73	3,510

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

58. 저장 프로시저 성능 이슈

1. 프롬프트

> [AS-IS]
> 구문에서 첫번째 매개변수에 대한 실행 계획이 두번째 매개변수 처리에 재사용되면서
> 과도한 I/O가 발생하는 상황이야, 성능 개선해줘.

```
[AS-IS]
EXEC SP_CITY 'Paris'     <--- 첫번째 매개변수
EXEC SP_CITY 'London'    <--- 두번째 매개변수
```

2. 분석 및 주요 최적화

전형적인 Parameter Sniffing(매개변수 스니핑) 문제입니다.

- 데이터 분포 (심각한 편향)

CITY	행 수	비율
Paris	256	0.02%
Berlin	128,000	12.50%
London	895,744	87.50%

'Paris'(256건)로 먼저 실행하면 Index Seek 계획이 캐시되지만, 'London'(895,744건)에서 동일 계획이 재사용되면서 대량의 Key Lookup으로 과도한 I/O가 발생합니다.

3. 개선안

```
-- 커버링 인덱스 추가 (근본적 해결)
-- 현재 NIDX01_CITY는 CITY만 포함하므로, DATE 컬럼을 추가한 커버링 인덱스를 만들면
-- Key Lookup을 제거할 수 있습니다.

CREATE NONCLUSTERED INDEX NIDX02_CITY_DATE
ON dbo.TB_CITY (CITY, DATE);
```


Section 09

통계 정보

> 옵티마이저는 통계 정보를 기반으로 실행 계획을 수립한다. 이 섹션에서는 통계 정보의 중요성과 관리 방법, 그리고 잘못된 통계로 인한 성능 문제를 살펴본다.

59 잘못된 통계 정보로 인한 성능 이슈 - 1

통계가 생성된 이후의 DML 작업으로 인해 통계가 부정확하여 의도하지 않은 실행 계획이 수립되는 사례이다.

1. 사전 구성 스크립트

```
/* 통계 부정확성 테스트를 위해 자동 통계 업데이트 기능은 OFF 된 상태로 진행 */
/* ALTER DATABASE [DB명] SET AUTO_UPDATE_STATISTICS OFF WITH NO_WAIT */

SELECT no
      , CASE WHEN no <= 250000 THEN 1
        WHEN no <= 500000 THEN 2
        WHEN no <= 750000 THEN 3
        ELSE 4 END AS num
     , CONVERT(NVARCHAR(36),NEWID()) AS txt
INTO TB_Stat
FROM (SELECT ROW_NUMBER() OVER ( ORDER BY a.number ) AS no
       FROM master..spt_values a, master..spt_values b
```

```
        WHERE a.type = 'P'
        AND a.number BETWEEN 1 AND 1000
        AND b.type = 'P'
        AND b.number BETWEEN 1 AND 1000
) c

CREATE INDEX NIDX01_Stat ON TB_Stat(num)

DELETE TB_Stat WHERE no >= 750002
GO
```

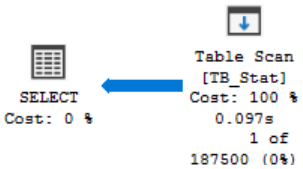
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT no,num,txt
FROM TB_Stat
WHERE num = 4
GO
```

- 실행 계획

Rows	Executes	StmtText	EstimateRows
1 1	1 1	SELECT [no],[num],[txt] FROM [TB_Stat] WHERE [num]=@1 ¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_Stat]), WHERE:([Tuning].[dbo].[TB_Stat].[num]=(4)))	187500.3 187500.3




```
테이블 'TB_Stat'. 검색 수 1, 논리적 읽기 수 12049
SQL Server 실행 시간:
CPU 시간 = 62ms, 경과 시간 = 70ms
```

3. [AS-IS] 통계 정보

- 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_Stat	NIDX01_Stat	NONCLUSTERED	0	num	-

- [NIDX01_Stat] 통계 데이터 (Selectivity, Histogram)

	All density	Average Length	Columns
1	0.25	4	num

	RANGE_HI_KEY	RANGE_ROWS	EQ_ROWS	DISTINCT_RANGE_ROWS	AVG_RANGE_ROWS
1	1	0	250000	0	1
2	2	0	250000	0	1
3	3	0	250000	0	1
4	4	0	250000	0	1

· **실제 데이터 분포도**

	num	EQ_ROWS
1	1	250000
2	2	250000
3	3	250000
4	4	1

4. [AS-IS] 구문 설명

[TB_Stat] 테이블에서 [num] 컬럼의 값이 4인 행을 출력하는 구문이다. 조건에 사용된 [num] 컬럼에는 인덱스가 존재하고 최종 출력되는 데이터는 1건임에도 테이블 전체를 스캔하면서 많은 I/O 비용을 사용한다.

5. [TO-BE] 튜닝 포인트

[TB_Stat] 테이블 조건에 해당되는 데이터 1건만을 출력함에도 인덱스를 사용하지 않은 이유는 부정확한 통계 정보로 인해 예상 행 수(Estimate Rows)가 높게 측정되었기 때문이다. [NIDX01_Stat] 인덱스를 사용하여 탐색을 할 경우 [no], [txt] 컬럼을 출력하기 위한 RID Lookup이 필요한데, 예상 행 수 약 18만 건만큼 RID Lookup에 대한 비용을 고려하여 테이블 전체 스캔하는 것을 선택한 것이다. 통계가 생성된 이후에 데이터가 삭제되면서 통계 데이터와 실제 데이터의 차이가 발생된 것이기 때문에 현재 데이터를 기준으로 통계를 업데이트하여 정확한 정보를 제공할 수 있도록 해야 한다.

6. [TO-BE] 개선 방안

1) 통계 정보 업데이트

UPDATE STATISTICS 구문을 통해 기존 통계 정보를 업데이트하고 정확한 통계 정보로 실행 계획을 수립할 수 있도록 개선한다.

■ [NIDX01_Stat] 통계 정보 업데이트

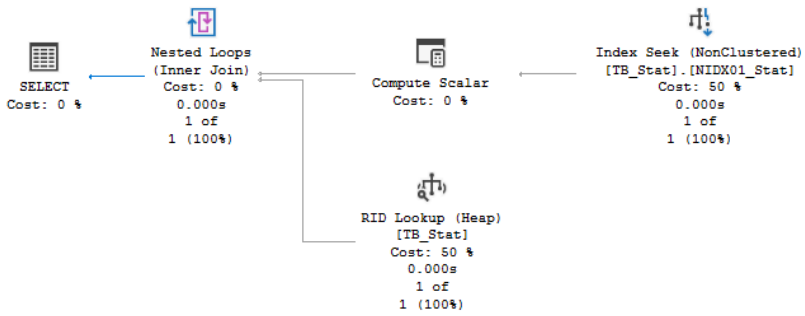
```
UPDATE STATISTICS TB_Stat(NIDX01_Stat) WITH FULLSCAN
GO
```

SQL 구문

```
SELECT no,num,txt
FROM TB_Stat
WHERE num = 4
GO
```

· 실행 계획

Rows	Executes	StmtText	EstimateRows
1 1 0 1 1	1 1 0 1 1	SELECT [no],[num],[txt] FROM [TB_Stat] WHERE [num]=@1 ¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([Bmk1000]) OPTIMIZED) ¦--Compute Scalar(DEFINE:([Expr1004]=BmkToPage([Bmk1000]))) ¦ ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_Stat].[NIDX01_Stat]), SE EK:([Tuning].[dbo].[TB_Stat].[num]=(4)) Ordered Forward) ¦--RID Lookup(OBJECT:([Tuning].[dbo].[TB_Stat]), SEEK:([Bmk1000]=[Bmk1000]) LOOKUP Ordered Forward)	1 1 1 1 1




```
테이블 'TB_Stat'. 검색 수 1, 논리적 읽기 수 7
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	62	70	12,049
TO-BE	0	0	7

GitHub Copilot 채팅 결과

Claude Opus 4.6

59.잘못된 통계 정보로 인한 성능 이슈 - 1

1. 프롬프트

> [AS-IS]
> 구문의 결과 건 수는 1건이며 적합한 인덱스가 있는데, Table Scan이 발생해.
> 문제 원인 분석 및 성능 개선해줘.

```
[AS-IS]
SELECT no,num,txt
FROM TB_Stat
WHERE num = 4;
```

2. 분석 및 주요 최적화

원인을 찾았습니다. 통계(Statistics)가 오래되어 옵티마이저가 잘못된 예측을 하고 있습니다.

1) 문제 원인 분석

항목	통계 기준 (오래된)	실제 현재 값
테이블 총 행 수	1,000,000	750,001
num = 4 예상 행 수	250,000	1
변경된 행 수	–	249,999 (25%)

2) 옵티마이저의 판단 과정:

1. 통계 히스토그램에서 num = 4의 EQ_ROWS = 250,000 (전체의 25%)
2. 인덱스 NIDX01_Stat은 비클러스터드 인덱스이고, 테이블은 Heap (클러스터드 인덱스 없음)
3. 250,000건의 RID Lookup 비용이 Table Scan 비용보다 높다고 판단
4. → Table Scan 선택 (논리적 읽기 9,102 발생)

*실제로는 1건뿐이므로 Index Seek + RID Lookup이 훨씬 효율적이지만, 오래된 통계 때문에 잘못된 플랜을 선택한 것입니다.

3. 개선안

```
-- 방안 1. 통계 업데이트 (즉시 효과)

UPDATE STATISTICS [dbo].[TB_Stat] [NIDX01_Stat] WITH FULLSCAN;
```

통계를 갱신하면 옵티마이저가 num = 4의 실제 행 수(1건)를 정확히 인식하여 Index Seek를 선택합니다.

```
-- 방안 2. Heap 제거 – 클러스터드 인덱스 추가 (권장)
-- 현재 테이블이 Heap이므로 비클러스터드 인덱스 사용 시 비효율적인 RID Lookup이
-- 발생합니다.
-- 적절한 클러스터드 인덱스를 추가하면 Key Lookup으로 전환되어 성능이 개선됩니다.
-- no 컬럼이 고유하다면 PK로 지정, 아니라면 적합한 컬럼 선택 필요

CREATE CLUSTERED INDEX [CIX_TB_Stat] ON [dbo].[TB_Stat] ([no]);


-- 방안 3. (선택) 커버링 인덱스로 변경

DROP INDEX [NIDX01_Stat] ON [dbo].[TB_Stat];
CREATE NONCLUSTERED INDEX [NIDX01_Stat] ON [dbo].[TB_Stat] ([num])
INCLUDE ([no], [txt]);
```

· **요약**

개선 방안	효과	우선순위
UPDATE STATISTICS	즉시 올바른 플랜 선택	필수
클러스터드 인덱스 추가	Heap 제거, 전반적 성능 향상	권장
커버링 인덱스	Lookup 제거, 최적 성능	선택

60 잘못된 통계 정보로 인한 성능 이슈 - 2

통계가 생성된 이후의 DML 작업으로 인해 통계가 부정확하여 의도하지 않은 실행 계획이 수립되는 사례이다.

1. 사전 구성 스크립트

```
/* 통계 부정확성 테스트를 위해 자동 통계 업데이트 기능은 OFF 된 상태로 진행 */
/* ALTER DATABASE [DB명] SET AUTO_UPDATE_STATISTICS OFF WITH NO_WAIT */

SELECT ROW_NUMBER () OVER (ORDER BY A.number) AS ID, A.number AS CODE01
```

```
      , NULL AS CODE02
      , LEFT(CONVERT(CHAR(8), DATEADD(DD, A.high,'2020-01-01'), 112), 8) AS DATE
INTO TB_STAT01
FROM master..spt_values A, (SELECT number
                            FROM master..spt_values B
                            WHERE type = 'P' AND number <= 500) B
WHERE A.type = 'P'

CREATE CLUSTERED INDEX CIDX_STAT01 ON TB_STAT01(ID)
CREATE INDEX NIDX01_STAT01 ON TB_STAT01(DATE, CODE02)
CREATE INDEX NIDX02_STAT01 ON TB_STAT01(CODE01)
GO
```

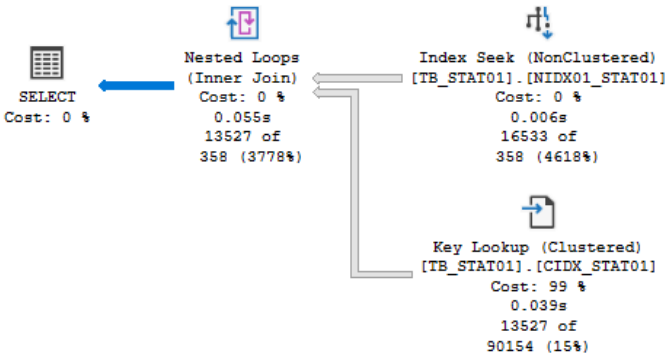
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT ID, CODE01, CODE02, DATE
FROM TB_STAT01
WHERE DATE = '20200103'
AND CODE02 IN ('2','3')
AND CODE01 < '500'

UPDATE A
SET A.CODE02 = LEFT(A.CODE01,1)
FROM TB_STAT01 A

SELECT ID, CODE01, CODE02, DATE
FROM TB_STAT01
WHERE DATE = '20200103'
AND CODE02 IN ('2','3')
AND CODE01 < '500'
GO
```

· 실행 계획 (UPDATE 이후 SELECT된 구문)

Rows	Executes	StmtText	EstimateRows
13527	1	SELECT ID, CODE01, CODE02, DATE FROM TB_STAT01 WHERE DATE ='20200103' AND CODE02 IN ('2','3') AND CODE01 \<'500'	292.4107
13527	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([Uniq1001], [Tuning].[dbo].[TB_STAT01].[ID], [Expr1003]) WITH UNORDERED PREFETCH)	292.4107
16533	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_STAT01].[NIDX01_STAT01]), SEEK:([Tuning].[dbo].[TB_STAT01].[DATE]='20200103' AND [Tuning].[dbo].[TB_STAT01]. [CODE02]=(2) OR [Tuning].[dbo].[TB_STAT01].[DATE]='20200103' AND [Tuning].[dbo].[TB_STAT01].[CODE02]=(3)) Ordered Forward)	292.4107
13527	16533	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_STAT01].[CIDX_STAT01]), SEEK:([Tuning].[dbo].[TB_STAT01].[ID]=[Tuning].[dbo].[TB_STAT01].[ID] AND [Uniq1001]=[Uniq1001]), WHERE:([Tuning].[dbo].[TB_STAT01].[CODE01]<(500)) LOOKUP Ordered Forward)	205.5433




```
테이블 'TB_STAT01'. 검색 수 2, 논리적 읽기 수 50712
SQL Server 실행 시간:
CPU 시간 = 94ms, 경과 시간 = 205ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_STAT01	CIDX_STAT01	CLUSTERED	0	ID	-
TB_STAT01	NIDX01_STAT01	NONCLUSTERED	0	DATE, CODE02	-
TB_STAT01	NIDX02_STAT01	NONCLUSTERED	0	CODE01	-

· [CODE02] 자동 통계 데이터 (Selectivity, Histogram)

	All density	Average Length	Columns
1	1	0	CODE02

	RANGE_HI_KEY	RANGE_ROWS	EQ_ROWS	DISTINCT_RANGE_ROWS	AVG_RANGE_ROWS
1	NULL	0	1026048	0	1

· [NIDX01_STAT01] 통계 데이터 (Selectivity, Histogram)

	All density	Average Length	Columns
1	0.125	8	DATE
2	0.125	8	DATE, CODE02
3	9.746133E-07	16	DATE, CODE02, ID

	RANGE_HI_KEY	RANGE_ROWS	EQ_ROWS	DISTINCT_RANGE_ROWS	AVG_RANGE_ROWS
1	20200102	0	128256	0	1
2	20200103	0	128256	0	1
3	20200105	0	128256	0	1
4	20200109	0	128256	0	1
5	20200117	0	128256	0	1
6	20200202	0	128256	0	1
7	20200305	0	128256	0	1
8	20200508	0	128256	0	1

4. [AS-IS] 구문 설명

[TB_STAT01] 테이블 [DATE] 컬럼이 '20200103'이고 [CODE02] 컬럼이 '2' 와 '3', [CODE01] 컬럼이 '500' 미만인 데이터를 출력한 뒤 [CODE02] 컬럼을 업데이트 후 다시 출력하는 구문이다. 첫 번째 SELECT 구문을 실행할 때 인덱스 선두 키 컬럼으로 존재하지 않는 [CODE02] 컬럼에 대해 자동 통계가 작성되었고 이때 값은 모두 NULL인 상태이다. [CODE02] 컬럼을 UPDATE한 뒤 다시 SELECT 구문을 실행하면 업데이트되지 않은 통계로 인해 예상 행 수가 적게 예측되어 NL Join으로 수행되면서 Key Lookup I/O가 과다하게 발생하였다.

5. [TO-BE] 튜닝 포인트

[CODE02] 컬럼이 업데이트되었지만 통계는 업데이트가 되지 않아 예상 행 수를 적게 판단하여 NL Join으로 실행 계획이 생성되었다. 따라서 해당 통계 업데이트를 통해 예상 행 수를 보다 정확히 예측하여 Hash Join으로 실행되도록 유도한다.

6. [TO-BE] 개선 방안

1) 통계 업데이트 수행

[CODE02] 컬럼에 대한 자동 통계를 업데이트한다. 기존에 NULL 값이었던 컬럼에 대한 통계가 실제 데이터로 갱신되면서 Hash Join으로 수행되어 I/O가 감소된다.

■ 통계 업데이트 구문

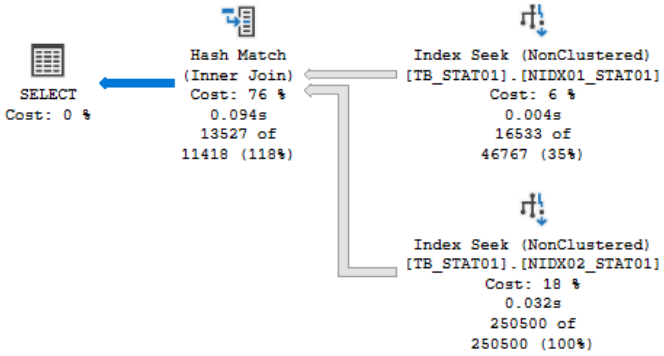
```
DBCC SHOW_STATISTICS('TB_STAT01', CODE02) --통계명 조회
UPDATE STATISTICS TB_STAT01(_WA_Sys_00000003_05A3D694) --해당 통계명 업데이트
```

■ SQL 구문

```
SELECT ID, CODE01, CODE02, DATE
FROM TB_STAT01
WHERE DATE ='20200103' AND CODE02 IN ('2','3') AND CODE01 <'500'
GO
```

· 실행 계획

Rows	Executes	StmtText	EstimateRows
13527	1	SELECT ID, CODE01, CODE02, DATE FROM TB_STAT01 WHERE DATE ='20200103' AND CODE02 IN ('2','3') AND CODE01 <'500'	32998.48
13527	1	┼--Hash Match(INNER JOIN, HASH:([Tuning].[dbo].[TB_STAT01].[ID], [Uniq1001])=([Tuning].[dbo].[TB_STAT01].[ID], [Uniq1001]), RESIDUAL:([Tuning].[dbo].[TB_STAT01].[ID] = [Tuning].[dbo].[TB_STAT01].[ID] AND [Uniq1001] = [Uniq1001]))	11461.04
16533	1	┼--Index Seek(OBJECT:([Tuning].[dbo].[TB_STAT01].[NIDX01_STAT01]), SEEK:([Tuning].[dbo].[TB_STAT01].[DATE]='20200103' AND [Tuning].[dbo].[TB_STAT01].[CODE02]=(2) OR [Tuning].[dbo].[TB_STAT01].[DATE]='20200103' AND [Tuning].[dbo].[TB_STAT01].[CODE02]=(3)) Ordered Forward)	46944.42
250500	1	┼--Index Seek(OBJECT:([Tuning].[dbo].[TB_STAT01].[NIDX02_STAT01]), SEEK:([Tuning].[dbo].[TB_STAT01].[CODE01] < (500)) Ordered Forward)	250500



테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'Workfile'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_STAT01'. 검색 수 3, 논리적 읽기 수 633
SQL Server 실행 시간:
CPU 시간 = 78ms, 경과 시간 = 197ms

· [CODE02] 컬럼의 통계 데이터 (Selectivity, Histogram)

	All density	Average Length	Columns
1	0.1111111	4	CODE02

	RANGE_HI_KEY	RANGE_ROWS	EQ_ROWS	DISTINCT_RANGE_ROWS	AVG_RANGE_ROWS
1	1	0	563737.4	0	1
2	2	0	74214.63	0	1
3	3	0	63246.61	0	1
4	4	0	55090.18	0	1
5	5	0	58454.4	0	1
6	6	0	53803.38	0	1
7	7	0	43787.35	0	1
8	8	0	61007.82	0	1
9	9	0	52706.18	0	1

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	94	205	50,712
TO-BE	78	197	633

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

60. 잘못된 통계 정보로 인한 성능 이슈 - 2

1. 프롬프트

> [AS-IS]
> 구문 수행 시 업데이트 이후 동일한 쿼리임에도 과도하게 I/O 많이 발생해.
> 문제의 원인을 찾고, 성능 개선해줘.

```
[AS-IS]
SELECT ID, CODE01, CODE02, DATE
FROM TB_STAT01
WHERE DATE = '20200103'
AND CODE02 IN ('2','3')
AND CODE01 < '500'

UPDATE A
SET A.CODE02 = LEFT(A.CODE01,1)
FROM TB_STAT01 A

SELECT ID, CODE01, CODE02, DATE
FROM TB_STAT01
WHERE DATE = '20200103'
AND CODE02 IN ('2','3')
AND CODE01 < '500';
```

2. 분석 및 주요 최적화

핵심 원인: AUTO_UPDATE_STATISTICS = OFF + 대량 UPDATE → 통계 불일치

1) 현재 상태 요약

항목	값
AUTO_UPDATE_STATISTICS	❌ OFF
테이블 행 수	1,026,048
UPDATE 영향 행 수	1,026,048 (100%)
통계 modification_counter (CODE02)	1,026,048 (100%)

2) 인덱스 & 통계 구성

인덱스명	키 컬럼	modification_counter
CIDX_STAT01	ID	0
NIDX01_STAT01	DATE, CODE02	0 ⚠
NIDX02_STAT01	CODE01	0
_WA_Sys (auto)	CODE02	1,026,048

3) 문제 흐름

```
flowchart TD
A["인덱스 생성 시 통계 수집 CODE02 = 전부 NULL"] --> B["1차 SELECT 실행
CODE02 IN (2,3)
→ 0건 반환 ✅ 빠름"]
B --> C["UPDATE 실행 CODE02 = LEFT(CODE01,1) 102만건 전부 변경"]
C --> D{"AUTO_UPDATE_STATISTICS = OFF"}
D -->|"통계 갱신 안됨"| E["2차 SELECT 실행 옵티마이저: CODE02=전부 NULL로
인식"]
E --> F["카디널리티 예상: ~1건 실제: 16,533건"]
F --> G["Nested Loops + Key Lookup 16,533 × 3 depth ≈ 50,000+ I/O ❌ 과도한
I/O"]
```

4) 원인 상세

1 AUTO_UPDATE_STATISTICS = OFF (근본 원인)

- 이 데이터베이스는 통계 자동 갱신이 꺼져 있습니다.
- UPDATE가 102만 행의 CODE02를 전부 변경했지만, 통계가 갱신되지 않습니다.

2 통계-실제 데이터 불일치

구분	통계가 인식하는 CODE02	실제 CODE02
분포	전부 NULL (1,026,048건)	0~9의 정수 값
CODE02 IN (2,3) 예상 행	~1건 (값이 없다고 판단)	16,533건

3 NIDX01_STAT01의 modification_counter = 0 함정

· 복합 통계의 modification_counter는 선행 컬럼(DATE) 변경만 추적합니다.

· UPDATE는 CODE02만 변경했으므로 NIDX01_STAT01의 카운터는 0으로 유지.

· 즉, AUTO_UPDATE_STATISTICS를 켜더라도 이 통계는 자동 갱신 트리거되지 않습니다!

4 잘못된 계획 선택 → Key Lookup 폭발

· 이 옵티마이저: "1건만 Key Lookup하면 되겠다" → Nested Loops 선택

· 실제: 16,533건 × 클러스터드 인덱스 depth 3 = ~50,000+ I/O

3. 개선안

```
-- ================================================
-- 방안 1. AUTO_UPDATE_STATISTICS 활성화 (근본 해결)
-- ================================================

ALTER DATABASE [Tuning] SET AUTO_UPDATE_STATISTICS ON;


-- ================================================
-- 방안 2. UPDATE 이후 관련 통계 수동 갱신
-- (대량 UPDATE 직후 즉시 반영이 필요한 경우)
-- ================================================

UPDATE STATISTICS [dbo].[TB_STAT01] [NIDX01_STAT01] WITH FULLSCAN;
UPDATE STATISTICS [dbo].[TB_STAT01] [_WA_Sys_00000003_4865BE2A] WITH FULLSCAN;


-- ================================================
-- 방안 3. 커버링 인덱스로 변경 (Key Lookup 완전 제거)
-- CODE01을 키에 추가하고 ID를 INCLUDE하여
-- 인덱스만으로 쿼리를 완전히 처리
-- ================================================

CREATE NONCLUSTERED INDEX [NIDX01_STAT01]
ON [dbo].[TB_STAT01] ([DATE], [CODE02], [CODE01])
INCLUDE ([ID])
WITH (DROP_EXISTING = ON, ONLINE = ON);
```

· **조치 요약**

우선순위	조치	효과
1	AUTO_UPDATE_STATISTICS ON	통계 자동 갱신 활성화 → 향후 동일 문제 방지
2	대량 UPDATE 후 UPDATE STATISTICS ... WITH FULLSCAN	즉시 통계 반영 → 정확한 카디널리티 추정
3	커버링 인덱스 (CODE01 키 추가 + ID INCLUDE)	Key Lookup 제거 → I/O 99% 감소

*AUTO_UPDATE_STATISTICS를 켜더라도, 복합 인덱스 통계(NIDX01_STAT01)는 선행 컬럼(DATE)이 변경되지 않으면 자동 갱신되지 않습니다. 대량 UPDATE 후에는 방안 2의 수동 통계 갱신을 반드시 함께 수행하세요.

61 통계 부재로 인한 성능 이슈

조건절 컬럼에 통계가 존재하지 않아 성능이 저하된 실행 계획이 만들어진 사례이다.

1. 사전 구성 스크립트

```
/* 통계 부정확성 테스트를 위해 자동 통계 작성 기능은 OFF 된 상태로 진행 */
/* ALTER DATABASE [DB명] SET AUTO_CREATE_STATISTICS OFF WITH NO_WAIT */

SELECT ROW_NUMBER () OVER (ORDER BY A.number) AS ID
      , A.number AS CODE01
      , LEFT(A.number,1) AS CODE02
      , LEFT(CONVERT(CHAR(8), DATEADD (DD, A.high,'2020-01-01'), 112), 8) AS DATE
INTO TB_DATA12
FROM master..spt_values A, (SELECT number
                            FROM master..spt_values B
                            WHERE type = 'P' AND number <= 500) B
WHERE A.type = 'P'

CREATE CLUSTERED INDEX CIDX_DATA12 ON TB_DATA12(ID)
CREATE INDEX NIDX01_DATA12 ON TB_DATA12(DATE, CODE02)
CREATE INDEX NIDX02_DATA12 ON TB_DATA12(CODE01)
GO
```

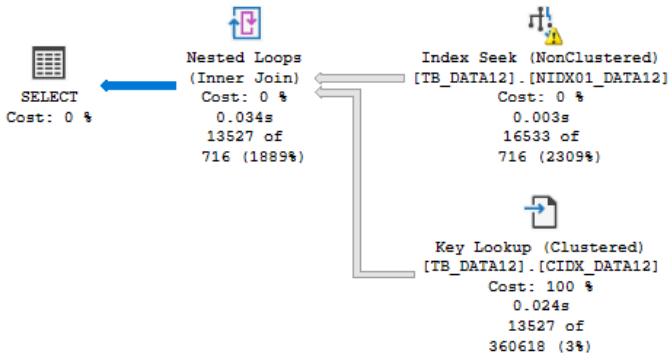
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT ID, CODE01, CODE02, DATE
FROM TB_DATA12
WHERE DATE = '20200103'
AND CODE02 IN ('2','3')
AND CODE01 < '500'
GO
```

- 실행 계획

Rows	Executes	StmtText	EstimateRows
13527	1	SELECT ID, CODE01, CODE02, DATE FROM TB_DATA12 WHERE DATE ='20200103' AND CODE02 IN ('2','3') AND CODE01<'500'	716.257
13527	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([Uniq1001], [Tuning].[dbo].[TB_DATA12].[ID], [Expr1003]) WITH UNORDERED PREFETCH)	716.257
16533	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_DATA12].[NIDX01_DATA12]), SEEK:([Tuning].[dbo].[TB_DATA12].[DATE]='20200103' AND [Tuning].[dbo].[TB_DATA12].[CODE02]='2' OR [Tuning].[dbo].[TB_DATA12].[DATE]='20200103' AND [Tuning].[dbo].[TB_DATA12].[CODE02]='3') Ordered Forward)	716.257
13527	16533	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_DATA12].[CIDX_DATA12]), SEEK:([Tuning].[dbo].[TB_DATA12].[ID]=[Tuning].[dbo].[TB_DATA12].[ID] AND [Uniq1001]=[Uniq1001]), WHERE:([Tuning].[dbo].[TB_DATA12].[CODE01]<(500)) LOOKUP Ordered Forward)	503.4761



테이블 'TB_DATA12'. 검색 수 2, 논리적 읽기 수 50708
SQL Server 실행 시간:
CPU 시간 = 94ms, 경과 시간 = 309ms

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_DATA12	CIDX_DATA12	CLUSTERED	0	ID	-
TB_DATA12	NIDX01_DATA12	NONCLUSTERED	0	DATE, CODE02	-
TB_DATA12	NIDX02_DATA12	NONCLUSTERED	0	CODE01	-

· [NIDX01_DATA12] 통계 데이터 (Selectivity, Histogram)

	All density	Average Length	Columns
1	0.125	8	DATE
2	0.01369863	9	DATE, CODE02
3	9.746133E-07	17	DATE, CODE02, ID

	RANGE_HI_KEY	RANGE_ROWS	EQ_ROWS	DISTINCT_RANGE_ROWS	AVG_RANGE_ROWS
1	20200102	0	128256	0	1
2	20200103	0	128256	0	1
3	20200105	0	128256	0	1
4	20200109	0	128256	0	1
5	20200117	0	128256	0	1
6	20200202	0	128256	0	1
7	20200305	0	128256	0	1
8	20200508	0	128256	0	1

4. [AS-IS] 구문 설명

[TB_DATA12] 테이블 [DATE] 컬럼이 '20200103'이고 [CODE02] 컬럼이 '2' 또는 '3' , [CODE01] 컬럼이 '500' 미만인 데이터를 출력하는 구문이다. [NIDX01_DATA12] 인덱스를 통해 [DATE], [CODE02] 컬럼에 대한 데이터를 탐색하는데 [CODE02] 컬럼에 대한 통계가 존재하지 않아 탐색한 실제 행 수가 16,533건임에도 예상 행 수가 적다고 판단하여 클러스터형 인덱스에 Key Lookup이 반복되면서 I/O가 과다하게 발생되었다.

실행 계획을 보면 Estimate Rows가 716.257건으로 예상됐으며 이는 '20200103'의 EQ_ROWS 제곱근을 통해 계산되고, [CODE02] 컬럼이 2와 3으로 두 번 탐색하므로, 앞에서 구한 제곱근에 2를 곱하여 구해진다.

```
SQRT(EQ_ROWS) * 2 = SQRT(128,256) * 2 = 358.128 * 2 = 716.257
```

5. [TO-BE] 튜닝 포인트

[CODE02] 컬럼의 통계 부재로 인해 부정확한 통계로 예상 행 수를 적게 판단하면서 과다한 Key Lookup이 발생되었다. 정확한 통계가 제공된다면, [NIDX01_DATA12] 인덱스와 [NIDX02_DATA12] 인덱스를 이용하여 탐색한 뒤 Hash Join을 통해 I/O를 개선할 수 있다. 정확한 통계를 제공하기 위해 [CODE02] 컬럼을 키값으로 인덱스를 생성한다.

6. [TO-BE] 개선 방안

1) [CODE02] 컬럼이 키로 구성된 신규 인덱스로 통계 생성

[CODE02] 컬럼에 대한 인덱스를 생성하면 해당 인덱스의 통계가 자동으로 생성된다. 실행 계획을 생성할 때 해당 통계를 함께 사용하여 예상 행 수를 측정하게 되고 인덱스 간의 Hash Join으로 실행되어 I/O를 개선하였다.

실행 계획을 보면 Estimate Rows가 46,568.75건으로 변경되었다. [NIDX03_DATA12] 통계에서 '2'의 EQ_ROWS인 79,659를 전체 ROWS 1,026,048로 나눈 값인 0.077636718을 구한다. 같은 방식으로 3에 대해서도 구하면 0.054199218이다. 두 값을 더한 수의 제곱근에 '20200103'의 EQ_ROWS인 128,256를 곱하면 예상 행 수가 계산된다.

```
SQRT(0.077636718 +0.054199218) * 128256 = 46568.75
```

인덱스 생성 구문

```
CREATE INDEX NIDX03_DATA12 ON TB_DATA12(CODE02)
GO
```

SQL 구문

```
SELECT ID, CODE01, CODE02, DATE
FROM TB_DATA12
WHERE DATE = '20200103'
AND CODE02 IN ('2','3')
AND CODE01 < '500'
GO
```

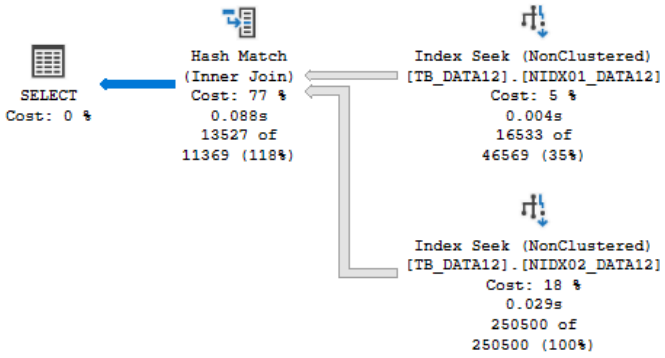
· [NIDX03_DATA12] 통계 데이터 (Selectivity, Histogram)

	All density	Average Length	Columns
1	0.1	1	CODE02
2	9.746133E-07	9	CODE02, ID

	RANGE_HI_KEY	RANGE_ROWS	EQ_ROWS	DISTINCT_RANGE_ROWS	AVG_RANGE_ROWS
1	0	0	501	0	1
2	1	0	556611	0	1
3	2	0	79659	0	1
4	3	0	55611	0	1
5	4	0	55611	0	1
6	5	0	55611	0	1
7	6	0	55611	0	1
8	7	0	55611	0	1
9	8	0	55611	0	1
10	9	0	55611	0	1

· 실행 계획

Rows	Executes	StmtText	EstimateRows
13527	1	SELECT ID, CODE01, CODE02, DATE FROM TB_DATA12 WHERE DATE ='20200103' AND CODE02 IN ('2','3') AND CODE01<'500'	32734.42
13527	1	¦--Hash Match(INNER JOIN, HASH:([Tuning].[dbo].[TB_DATA12].[ID], [Uniq1001])=([Tuning].[dbo].[TB_DATA12].[ID], [Uniq1001]), RESIDUAL:([Tuning].[dbo].[TB_DATA12].[ID] = [Tuning].[dbo].[TB_DATA12].[ID] AND [Uniq1001] = [Uniq1001]))	11369.32
16533	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_DATA12].[NIDX01_DATA12]), SEEK:([Tuning].[dbo].[TB_DATA12].[DATE]='20200103' AND [Tuning].[dbo].[TB_DATA12]. [CODE02]='2' OR [Tuning].[dbo].[TB_DATA12].[DATE]='20200103' AND [Tuning].[dbo].[TB_DATA12].[CODE02]='3') Ordered Forward)	46568.75
250500	1	¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_DATA12].[NIDX02_DATA12]), SEEK:([Tuning].[dbo].[TB_DATA12].[CODE01] < (500)) Ordered Forward)	250500




```
테이블 'Worktable'. 검색 수 0, 논리적 읽기 수 0
테이블 'Workfile'. 검색 수 0, 논리적 읽기 수 0
테이블 'TB_DATA12'. 검색 수 3, 논리적 읽기 수 629
SQL Server 실행 시간:
CPU 시간 = 93ms, 경과 시간 = 298ms
```

7. 개선 효과

- **리소스 사용률 비교**

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	94	309	50,708
TO-BE	93	298	629

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

61. 통계 부재로 인한 성능 이슈

1. 프롬프트

> [AS-IS]
> 구문에 적합한 인덱스도 있는데 과도한 I/O가 발생하네.
> 문제 원인을 찾고 성능 개선해줘.

```
[AS-IS]
SELECT ID,CODE01, CODE02, DATE
```

```
FROM TB_DATA12
WHERE DATE = '20200103'
AND CODE02 IN ('2','3')
AND CODE01 < '500';
```

2. 분석 및 주요 최적화

핵심 문제: Key Lookup (키 조회)로 인한 과도한 I/O

실행 계획과 실제 데이터를 분석한 결과, 핵심 문제는 다음과 같습니다

1) 현재 인덱스 구성

인덱스명	유형	키 컬럼	페이지 수
CIDX_DATA12	Clustered	ID	4,824
NIDX01_DATA12	Nonclustered	DATE, CODE02	3,680
NIDX02_DATA12	Nonclustered	CODE01	2,291

2) 실행 계획 분석

항목	예상값	실제값
Index Seek 반환 행 (DATE + CODE02)	716	16,533
최종 결과 행 (CODE01 < 500 적용 후)	503	13,527
Logical I/O Reads	-	52,792 페이지

3) 원인 상세

```
flowchart LR
A["Index Seek NIDX01_DATA12 (DATE, CODE02)"] -->|"16,533행"| B["Key Lookup
CIDX_DATA12 (CODE01 조회)"]
B -->|"CODE01 < 500 필터"| C["결과 13,527행"]
```

1 Key Lookup (Bookmark Lookup) – 주원인

- NIDX01_DATA12 인덱스에 CODE01 컬럼이 없어서, 매칭된 16,533건마다 클러스터드 인덱스로 돌아가 CODE01 값을 가져옵니다.
- 클러스터드 인덱스 깊이가 3이므로: 16,533 × 3 = ~49,599 I/O → 실제 52,792 I/O와 거의 일치합니다.

2 카디널리티 추정 오류

- 옵티마이저가 716행을 예상했지만 실제는 16,533행(23배 차이)으로, Nested Loops + Key Lookup 전략이 선택되었습니다.
- 실행 계획에 CODE02 컬럼에 대한 통계 누락 경고가 있습니다.

3. 개선안

```
-- 방안 1. (권장): 커버링 인덱스로 변경 – Key Lookup 완전 제거
-- CODE01을 키 컬럼에 추가하면 인덱스만으로 쿼리를 완전히 처리할 수 있습니다:
-- 기존 NIDX01_DATA12를 커버링 인덱스로 재생성
-- CODE01을 키 컬럼으로 추가하여 범위 필터(< 500)를 인덱스 내에서 처리

CREATE NONCLUSTERED INDEX [NIDX01_DATA12]
ON [dbo].[TB_DATA12] ([DATE], [CODE02], [CODE01])
WITH (DROP_EXISTING = ON, ONLINE = ON);


-- 방안 2. CODE02 컬럼 통계 생성 - 카디널리티 추정 개선

CREATE STATISTICS [STAT_CODE02] ON [dbo].[TB_DATA12] ([CODE02]);
```

*방안 1을 적용하면 인덱스 Seek만으로 모든 컬럼을 반환할 수 있어, Key Lookup이 완전히 제거되고 I/O가 극적으로 줄어듭니다.

Section 10

파티션 및 원격 쿼리

대용량 데이터 처리와 분산 환경에서는 파티션과 원격 쿼리가 중요한 역할을 한다. 이 섹션에서는 파티션 설계와 원격 쿼리 처리 시 고려해야 할 성능 요소를 설명한다.

62 REMOTE QUERY - 1

4-Part Name 방식의 Remote Query 성능 이슈 사례이다.

1. 사전 구성 스크립트

```
/* 로컬 서버 */

USE [master]
```

```
GO
EXEC master.dbo.sp_addlinkedserver
@server = N'LOCALHOST,1444' --연결된 서버 별칭 (접속 주소, 포트)
, @srvproduct = N'SQL Server'
EXEC master.dbo.sp_addlinkedsrvlogin
@rmtsrvname = N'LOCALHOST,1444' --연결된 서버 별칭 (접속 주소, 포트)
, @useself = 'false'
, @rmtuser = N'sa' --로그인 사용자
, @rmtpassword = N'P@ssW0rd!' --로그인 암호
GO

USE [Tuning]
GO
SELECT TOP 100000 ROW_NUMBER () OVER (ORDER BY A.number) + 800000 AS ID
      , A.number AS CODE01, RIGHT(A.number,1) AS CODE02
      , CASE WHEN B.number < 80
        THEN LEFT(CONVERT(CHAR(8), DATEADD(DD, A.high,'2020-01-01') ,112), 8)
        ELSE LEFT(CONVERT(CHAR(8), DATEADD(DD, A.number,'2020-01-01'),112), 8)
        END AS DATE
INTO TB_DATA
FROM master..spt_values A, (SELECT number
                            FROM master..spt_values B
                            WHERE type = 'P' AND number <= 500) B
WHERE A.type = 'P'

CREATE CLUSTERED INDEX CIDX_DATA ON TB_DATA(ID)
GO
```

```
/* 원격 서버 */

USE [TEST]
GO
SELECT ROW_NUMBER () OVER (ORDER BY A.number) AS ID
      , A.number AS CODE01, RIGHT(A.number,1) AS CODE02
      , CASE WHEN B.number < 80
        THEN LEFT(CONVERT(CHAR(8), DATEADD(DD, A.high, '2020-01-01'), 112) ,8)
        ELSE LEFT(CONVERT(CHAR(8), DATEADD(DD, A.number, '2020-01-01'), 112) ,8)
        END AS DATE
INTO TB_DATA13
FROM master..spt_values A, (SELECT number
                            FROM master..spt_values B
                            WHERE type = 'P' AND number <= 500) B
WHERE A.type = 'P'

CREATE CLUSTERED INDEX CIDX_DATA13 ON TB_DATA13(ID)
CREATE INDEX NIDX01_DATA13 ON TB_DATA13(DATE, CODE02)
GO
```

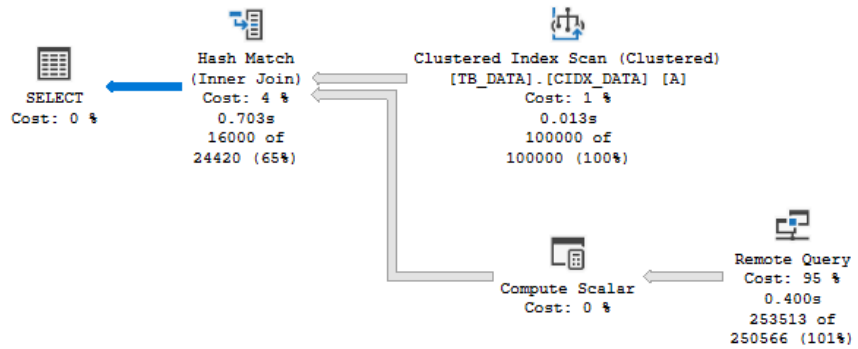
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT A.ID, A.CODE02, B.DATE
FROM TB_DATA A
INNER JOIN [LOCALHOST,1444].TEST.DBO.TB_DATA13 B
ON A.ID = B.ID
AND B.DATE  <= '20200731'
GO
```

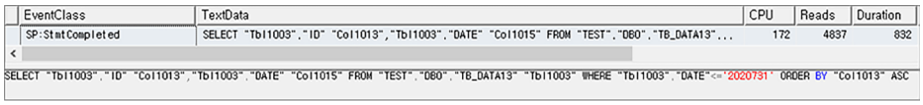
· 실행 계획

Rows	Executes	StmtText
16000	1	SELECT A.ID, A.CODE02, B.DATE FROM TB_DATA A INNER JOIN [LOCALHOST,1444].TEST.DBO.TB_DATA13 B ON A.ID=B.ID AND B.DATE <='20200731'
16000	1	¦--Hash Match(INNER JOIN, HASH:([A].[ID])=([LOCALHOST,1444].[TEST].[DBO].[TB_DATA13].[ID]), RESIDUAL:([Tuning].[dbo].[TB_DATA].[ID] as [A].[ID]=[LOCALHOST,1444].[TEST].[DBO].[TB_DATA13].[ID] as [B].[ID]))
100000	1	¦--Clustered Index Scan(OBJECT:([Tuning].[dbo].[TB_DATA].[CIDX_DATA] AS [A]))
0	0	¦--Compute Scalar(DEFINE:([LOCALHOST,1444].[TEST].[DBO].[TB_DATA13].[ID]=[LOCALHOST,1444].[TEST].[DBO].[TB_DATA13].[ID] as [B].[ID], [LOCALHOST,1444].[TEST].[DBO].[TB_DATA13].[DATE]=[LOCALHOST,1444].[TEST].[DBO].[TB_DATA13].[DATE] as [B].[DATE]))
253513	1	¦--Remote Query(SOURCE:(LOCALHOST,1444), QUERY:(SELECT "Tbl1003"."ID" "Col1013","Tbl1003"."DATE" "Col1015" FROM "TEST"."DBO"."TB_DATA13" "Tbl1003" WHERE "Tbl1003"."DATE"<='20200731' ORDER BY "Col1013" ASC))

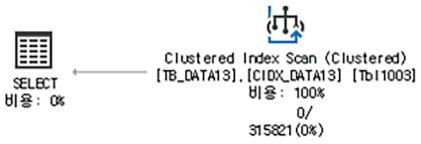


테이블 'Workfile'. 스캔 수 0, 논리적 읽기 0
테이블 'Worktable'. 스캔 수 0, 논리적 읽기 0
테이블 'TB_DATA'. 스캔 수 1, 논리적 읽기 476
SQL Server 실행 시간:
CPU 시간 = 562ms, 경과 시간 = 747ms

· 원격 서버 SQL 구문 (Profiler)



· 원격 서버 실행 계획 (Profiler)



3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_DATA	CIDX_DATA	CLUSTERED	0	ID	-
TB_DATA13	CIDX_DATA13	CLUSTERED	0	ID	-
TB_DATA13	NIDX01_DATA13	NONCLUSTERED	0	DATE, CODE02	-

4. [AS-IS] 구문 설명

4-Part Name 방식을 사용하여 원격 서버 [TB_DATA13] 테이블의 [DATE] 컬럼이 '20200731' 이하인 데이터와 로컬 서버의 [TB_DATA] 테이블을 조인하는 구문이다. 두 테이블은 Hash Join으로 실행되어 조인 과정에서 정렬이 필요 없음에도 원격 서버에서 실행되는 구문에 변형이 일어나 [ID] 컬럼 순으로 정렬하도록 ORDER BY가 추가되었고 [NIDX01_DATA13] 인덱스가 아닌 [CIDX_DATA13] 클러스터형 인덱스를 사용하였다.

5. [TO-BE] 튜닝 포인트

원격 서버에 접근하는 방식을 4-Part Name 방식에서 OPENQUERY 방식으로 구문을 변경한다. OPENQUERY로 작성하면 4-Part Name과 달리 구문 변경이 발생하지 않고 명시한 구문 그대로 원격 서버에서 실행된다. 따라서 원격 서버에 존재하는 적합한 인덱스를 탐색할 수 있다.

6. [TO-BE] 개선 방안

1) OPENQUERY 방식으로 구문을 변경한다.

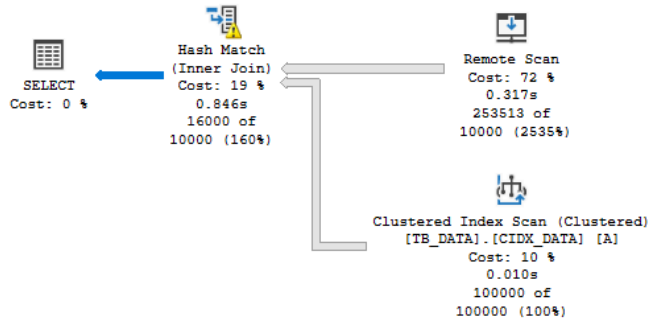
아래와 같이 OPENQUERY를 사용하여 원격 서버 [TB_DATA13] 테이블의 [DATE] 컬럼을 조건으로 조회한 데이터와 로컬 서버 [TB_DATA] 테이블과 조인하도록 구문을 재작성하였다. Profiler를 통해 원격 서버에서 실행된 구문 및 실행 계획을 확인해 보면 4-Part Name 방식과 달리 명시해 준 구문 그대로 실행됨을 확인할 수 있다. 따라서 WHERE 조건절의 [DATE] 컬럼에 대해 [NIDX01_DATA13] 인덱스를 탐색하여 원격 서버의 I/O가 감소된다.

■ 변경된 SQL 구문

```
SELECT A.ID, A.CODE02, B.DATE
FROM TB_DATA A
INNER JOIN (SELECT ID, DATE
            FROM OPENQUERY([LOCALHOST,1444], 'SELECT ID, DATE
                                              FROM TEST.DBO.TB_DATA13
                                              WHERE DATE <= ''20200731''')
) B
ON A.ID = B.ID
```

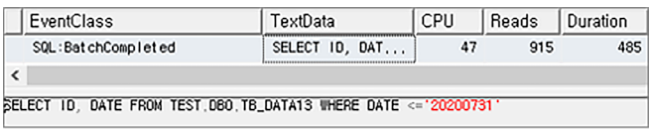
· 실행 계획

Rows	Executes	StmtText
16000	1	SELECT A.ID, A.CODE02, B.DATE FROM TB_DATA A INNER JOIN (SELECT ID, DATE FROM OPENQUERY([LOCALHOST,1444], 'SELECT ID, DATE FROM TEST.DBO.TB_DATA13 WHERE DATE <=''20200731'' ')) b ON A.ID=B.ID
16000	1	¦--Hash Match(INNER JOIN, HASH:([SQLNCLI11].[ID])=([A].[ID]), RESIDUAL:([SQLNCLI11].[ID]=[Tuning].[dbo].[TB_DATA].[ID] as [A].[ID]))
253513	1	¦--Remote Scan(SOURCE:(LOCALHOST,1444), OBJECT:(SELECT ID, DATE FROM TEST.DBO.TB_DATA13 WHERE DATE <='20200731'))
100000	1	¦--Clustered Index Scan(OBJECT:([Tuning].[dbo].[TB_DATA].[CIDX_DATA] AS [A]))

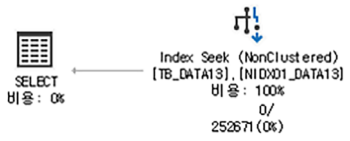



```
테이블 'Workfile'. 스캔 수 12, 논리적 읽기 912
테이블 'Worktable'. 스캔 수 0, 논리적 읽기 0
테이블 'TB_DATA'. 스캔 수 1, 논리적 읽기 476
SQL Server 실행 시간:
CPU 시간 = 453ms, 경과 시간 = 686ms
```

· 원격 서버 SQL 구문 (Profiler)



· 원격 서버 실행 계획 (Profiler)



7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	562	747	476
TO-BE	453	686	1,388

· 원격지 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	172	832	4,837
TO-BE	47	485	915

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

62. REMOTE QUERY - 1

1. 프롬프트

> [AS-IS]
> 구문은 원격지 테이블에 DATE 인덱스가 존재함에도 불구하고, ID 기준으로 ORDER BY 처리를 수행하면서 조건 범위 탐색이 아닌 클러스터형 인덱스(ID) 전체 스캔이 발생하여 과도한 I/O가 발생한다. 이 문제를 분석하고 성능 개선안을 알려줘.

```
[AS-IS]
SELECT A.ID, A.CODE02, B.DATE
FROM TB_DATA A
INNER JOIN [LOCALHOST,1444].TEST.DBO.TB_DATA13 B
ON A.ID = B.ID
AND B.DATE <= '20200731';
```

2. 분석 및 주요 최적화

1) 로컬 테이블 구조

항목	값
테이블	dbo.TB_DATA (100,000건)
클러스터드 인덱스	CIDX_DATA (ID)
ID 범위	800,001 ~ 900,000

2) 문제 원인

4-part naming으로 원격 테이블을 직접 조인하면, SQL Server 옵티마이저가 원격 쿼리를 생성할 때 다음과 같이 동작합니다:

```
로컬 TB_DATA → ID 기준 Clustered Index Scan (정렬된 순서)
  → 옵티마이저가 원격 쿼리에 ORDER BY ID 주입
    → 원격 서버: DATE 인덱스 무시, ID 클러스터드 인덱스 Full Scan
      → DATE <= '20200731' 은 Residual Predicate로 전락
```

옵티마이저는 양쪽을 ID 순서로 정렬하려다 보니, 원격 서버에서 DATE 인덱스 대신 ID 클러스터드 인덱스를 선택하게 되어 전체 테이블을 스캔하는 것입니다.

3. 개선안

```sql
-- OPENQUERY로 DATE 필터를 원격에서 먼저 수행하는 것입니다:

SELECT A.ID, A.CODE02, B.[DATE]
FROM dbo.TB_DATA A
INNER JOIN OPENQUERY([LOCALHOST,1444],
    'SELECT ID, DATE
     FROM TEST.DBO.TB_DATA13
     WHERE DATE <= ''20200731'''
) B ON A.ID = B.ID;
```

· 변경 요약

항목	원래 쿼리	개선 쿼리
원격 접근	4-part naming	OPENQUERY + WHERE DATE 필터
원격 인덱스	✗ CI Scan (ORDER BY ID 강제)	✓ DATE 인덱스 Seek
네트워크 전송	원격 전체 데이터	DATE 조건 만족 건만
SELECT *	미사용 (양호)	✓ 동일
DATE 예약어	미이스케이프	✓ [DATE] 이스케이프

63 REMOTE QUERY - 2

원격 서버의 데이터를 가져오기 위해 전체 데이터를 읽어내면서 성능이 저하되는 사례이다.

1. 사전 구성 스크립트

```
/*로컬 서버*/

USE [master]
EXEC master.dbo.sp_addlinkedserver @server = N'LOCALHOST,1444' --연결된 서버 별칭
(서버 주소, 포트)
, @srvproduct = N'SQL Server'
EXEC master.dbo.sp_addlinkedsrvlogin
@rmtsrvname = N'LOCALHOST,1444' --연결된 서버 별칭 (서버 주소, 포트)
, @useself = 'false'
, @rmtuser = N'sa' --로그인 사용자
, @rmtpassword = N'P@ssW0rd!' --로그인 암호
GO
USE [Tuning]
SELECT ROW_NUMBER () OVER (ORDER BY A.number) AS ID
      , A.number AS CODE01, RIGHT(A.number,1) AS CODE02
      , CASE WHEN B.number < 80
        THEN LEFT(CONVERT(CHAR(8), DATEADD(DD, A.high,'2020-01-01'),112),8)
        ELSE LEFT(CONVERT(CHAR(8), DATEADD(DD, A.number,'2020-01-01'),112),8)
        END AS DATE
INTO TB_REMOTE01
FROM master..spt_values A, (SELECT number
                            FROM master..spt_values B
                            WHERE type = 'P' AND number <= 500) B
WHERE A.type = 'P'

SELECT number AS ID, number +1700 AS CODE
INTO TB_CODE01
FROM master..spt_values
WHERE type = 'P' AND number BETWEEN 1 AND 5

CREATE CLUSTERED INDEX CIDX_REMOTE01 ON TB_REMOTE01(ID)
CREATE INDEX NIDX01_REMOTE01 ON TB_REMOTE01(DATE,CODE01)
CREATE INDEX NIDX02_REMOTE01 ON TB_REMOTE01(CODE01,DATE)
GO
```

```
/*원격 서버*/

USE [TEST]
SELECT TOP 1000000 ROW_NUMBER () OVER (ORDER BY A.number) + 800000 AS ID
      , A.number AS CODE01, RIGHT(A.number,1) AS CODE02
      , CASE WHEN B.number < 80
        THEN LEFT(CONVERT(CHAR(8), DATEADD(DD, A.high,'2020-01-01'),112),8)
        ELSE LEFT(CONVERT(CHAR(8), DATEADD(DD, A.number,'2020-01-01'),112),8) END AS
DATE
INTO TB_REMOTE02
```

```
FROM master..spt_values A, (SELECT number
                            FROM master..spt_values B
                            WHERE type = 'P' AND number <= 500) B
WHERE A.type = 'P'

CREATE CLUSTERED INDEX CIDX_REMOTE02 ON TB_REMOTE02(ID)
GO
```

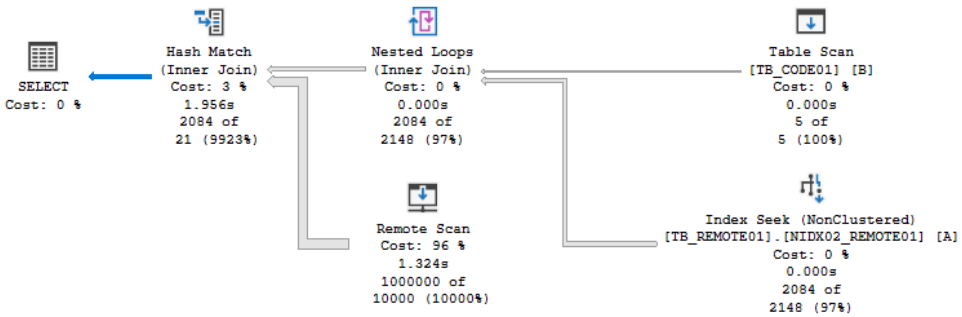
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
/*로컬 서버 실행쿼리 */

USE [Tuning]
SELECT A.ID, C.CODE01, C.DATE
FROM TB_REMOTE01 A INNER JOIN TB_CODE01 B
ON A.CODE01 = B.CODE AND A.DATE BETWEEN '20200101' AND '20240831'
INNER JOIN (SELECT *
            FROM OPENQUERY([LOCALHOST,1444], 'SELECT ID,CODE01,DATE
                                              FROM TEST.DBO.TB_REMOTE02')
) C
ON A.ID = C.ID
GO
```

· 실행 계획

Rows	Executes	StmtText
2084	1	SELECT A.ID, C.CODE01, C.DATE FROM TB_REMOTE01 A INNER JOIN TB_CODE01 B ON A.CODE01=B.CODE AND A.DATE BETWEEN '20200101' AND '20240831' INNER JOIN (SELECT * FROM OPENQUERY([LOCALHOST,1444], 'SELECT ID,CODE01,DATE FROM TEST.DBO.TB_REMOTE02')) C ON A.ID=C.ID
2084	1	¦--Hash Match(INNER JOIN, HASH:([A].[ID])=([SQLNCLI11].[ID]), RESIDUAL:([SQLNCLI11].[ID]=[Tuning].[dbo].[TB_REMOTE01].[ID] as [A].[ID]))
2084	1	¦--Nested Loops(INNER JOIN, OUTER REFERENCES:([B].[CODE]))
5	1	¦ ¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_CODE01] AS [B]))
2084	5	¦ ¦--Index Seek(OBJECT:([Tuning].[dbo].[TB_REMOTE01].[NIDX02_REMOTE01] AS [A]), SEEK:([A].[CODE01]=[Tuning].[dbo].[TB_CODE01].[CODE] as [B].[CODE] AND [A].[DATE] >= '20200101' AND [A].[DATE] <= '20240831') Ordered Forward)
1000000	1	¦--Remote Scan(SOURCE:(LOCALHOST,1444), OBJECT:(SELECT ID,CODE01,DATE FROM TEST.DBO.TB_REMOTE02))



테이블 'Workfile'. 스캔 수 0, 논리적 읽기 0
테이블 'Worktable'. 스캔 수 0, 논리적 읽기 0

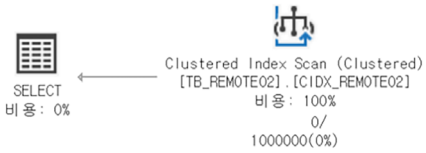
```
테이블 'TB_REMOTE01'. 스캔 수 5, 논리적 읽기 28
테이블 'TB_CODE01 '. 스캔 수 1, 논리적 읽기 1
SQL Server 실행 시간:
CPU 시간 = 1578ms, 경과 시간 = 2011ms
```

· **원격 서버 SQL 구문 (Profiler)**

EventClass	TextData	CPU	Reads	Duration
SQL:BatchCompleted	SELECT ID,CODE01,DATE FROM TEST.DBO.TB_REMOTE02	531	4710	1958

· **원격 서버 실행 계획 (Profiler)**

Clustered Index Scan(OBJECT:([TEST].[dbo].[TB_REMOTE02].[CIDX_REMOTE02]))



3. [AS-IS] 통계 및 Lock 정보

· **인덱스 정보**

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_REMOTE01	CIDX_REMOTE01	CLUSTERED	0	ID	-
TB_REMOTE01	NIDX01_REMOTE01	NONCLUSTERED	0	DATE_CODE01	-
TB_REMOTE01	NIDX02_REMOTE01	NONCLUSTERED	0	CODE01,DATE	-
TB_REMOTE02	CIDX_REMOTE02	CLUSTERED	0	ID	-

4. [AS-IS] 구문 설명

로컬 서버에서 [TB_REMOTE01] 테이블과 [TB_CODE01] 테이블을 조인하고 OPENQUERY를 사용하여 원격 서버의 [TB_REMOTE02] 테이블을 조인하는 구문이다. 원격지로부터 [TB_REMOTE02] 테이블을 참조할 때 조인 조건인 [ID] 컬럼으로 인덱스 탐색을 하지 못하고 모든 데이터를 로컬 서버로 읽어 온 후에 조인이 수행되면서 성능이 저하된다.

5. [TO-BE] 튜닝 포인트

원격 서버 데이터를 가져오는 과정에서 많은 데이터를 읽어오면서 처리가 지연된다. 로컬 서버의 두 테이블을 조인한 결과 건수가 적기 때문에 반대로 원격 서버에서 로컬 서버의 데이터를 호출해서 조인하는 프로시저를 생성하여 원격지에서 구문 처리를 수행하도록 한다.

6. [TO-BE] 개선 방안

1) 원격 서버에서 로컬 서버의 데이터를 가져오도록 프로시저를 생성한다.

원격 서버에서 로컬 서버를 Linked Server로 연결하고 [TB_REMOTE01], [TB_CODE01] 테이블을 조인한 결과 집합인 2,084 건을 원격 서버로 가져와 [TB_REMOTE02] 테이블과 조인하도록 프로시저를 생성한다. 로컬 서버에서 결과를 얻기 위해 OPENQUERY를 이용하여 프로시저를 실행한다. 선행 집합 2,084 건과 원격 서버 [TB_REMOTE02] 테이블이 조인될 때 [CIDX_REMOTE02] 인덱스를 Seek 하여 성능이 개선되었다.

■ 원격 서버 프로시저 생성문

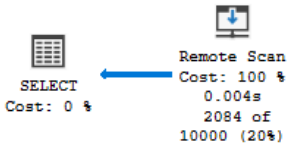
```
/* 원격 서버 */

CREATE PROCEDURE UDP_REMOTE
AS
SELECT A.ID, B.CODE01, B.DATE
FROM (SELECT *
       FROM OPENQUERY([LOCALHOST,1444],'SELECT A.ID
                                        FROM Tuning.DBO.TB_REMOTE01 A
                                        INNER JOIN Tuning.DBO.TB_CODE01 B
                                        ON A.CODE01 = B.CODE
                                        AND A.DATE BETWEEN ''20200101''
                                        AND ''20240831''')
) A
INNER JOIN TEST.DBO.TB_REMOTE02 B
ON A.ID=B.ID
GO
```

■ SQL 구문

```
SELECT * FROM OPENQUERY([LOCALHOST,1444], 'EXEC TEST.DBO.UDP_REMOTE')
GO
```

· 실행 계획

Rows	Executes	StmtText
2084 2084	1 1	SELECT * FROM OPENQUERY([LOCALHOST,1444], ' EXEC TEST.DBO.UDP_REMOTE ') ¦--Remote Scan(SOURCE:([LOCALHOST,1444]), OBJECT:(EXEC TEST.DBO.UDP_REMOTE))




```
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 146ms
```

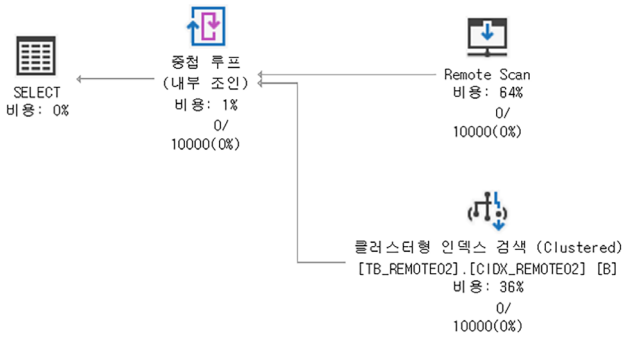
· 원격 서버 SQL 구문 (Profiler)

EventClass	TextData	CPU	Reads	Duration
SP:StmtCompleted	SELECT A.ID, B.CODE01, B.DATE FROM (SELECT * FROM OPENQUERY(...	0	6922	98

```
SELECT A.ID, B.CODE01, B.DATE
FROM ( SELECT * FROM OPENQUERY([LOCALHOST,1444],'SELECT A.ID FROM Tuning.DBO.TB_REMOTE01 A
                                                 INNER JOIN Tuning.DBO.TB_CODE01 B
                                                 ON A.CODE01=B.CODE AND A.DATE BETWEEN ''20200101'' AND ''20240831'''')) A
INNER JOIN TEST.DBO.TB_REMOTE02 B
ON A.ID=B.ID
```

· 원격 서버 실행 계획 (Profiler)

```
Nested Loops(Inner Join, OUTER REFERENCES:([SQLNCLI11].[ID], [Expr1004]) WITH UNORDERED PREFETCH)
  |--Remote Scan(SOURCE:(LOCALHOST,1444), OBJECT:(SELECT A.ID FROM Tuning.DBO.TB_REMOTE01 A
                                                  INNER JOIN Tuning.DBO.TB_CODE01 B ON A.CODE01=B.CODE
                                                  AND A.DATE BETWEEN '20200101' AND '20240831'))
  |--Clustered Index Seek(OBJECT:([TEST].[dbo].[TB_REMOTE02].[CIDX_REMOTE02] AS [B]), SEEK:([B].[ID]=[SQLNCLI11].[ID]) ORDERED FORWARD)
```



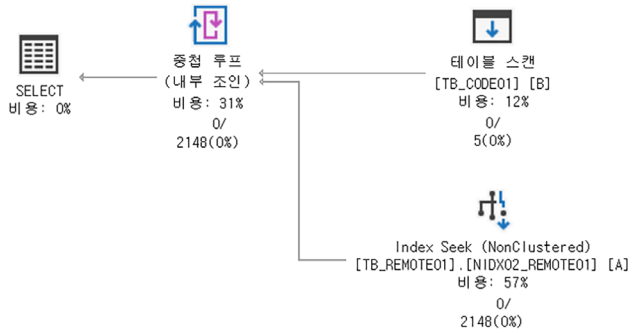
· 로컬 서버 SQL 구문 (Profiler)

EventClass	TextData	CPU	Reads	Duration
SQL:StmtCompleted	SELECT A.ID FROM Tuning.DBO.TB_REMOTE01 A INNE...	0	31	7

```
SELECT A.ID FROM Tuning.DBO.TB_REMOTE01 A
INNER JOIN Tuning.DBO.TB_CODE01 B
ON A.CODE01=B.CODE AND A.DATE BETWEEN '20200101' AND '20240831'
```

· 로컬 서버 실행 계획 (Profiler)

```
Nested Loops(Inner Join, OUTER REFERENCES:([B].[CODE]))
  |--Table Scan(OBJECT:([Tuning].[dbo].[TB_CODE01] AS [B]))
  |--Index Seek(OBJECT:([Tuning].[dbo].[TB_REMOTE01].[NIDX02_REMOTE01] AS [A]),
      SEEK:([A].[CODE01]=[Tuning].[dbo].[TB_CODE01].[CODE] as [B].[CODE] AND [A].[DATE] >= '20200101' AND [A].[DATE] <= '20240831') ORDERED FORWARD)
```



7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	1,578	2,011	29
TO-BE	0	146	-

· 원격지 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	531	1,958	4,710
TO-BE	0	112	6,922

AI GitHub Copilot 채팅 결과

Claude Opus 4.6

63. REMOTE QUERY - 2

1. 프롬프트

> [AS-IS]
> 해당 구문은 원격 서버의 TB_REMOTE02 테이블에 적합한 인덱스가 존재함에도 불구하고, 테이블 전체를 읽어 Hash Join으로 처리되고 있다. 이로 인해 불필요한 I/O가 발생하므로 문제를 분석하고 성능 개선안을 알려줘.

```
[AS-IS]
SELECT A.ID, C.CODE01, C.DATE
FROM TB_REMOTE01 A INNER JOIN TB_CODE01 B
ON A.CODE01 = B.CODE AND A.DATE BETWEEN '20200101' AND '20240831'
INNER JOIN (SELECT *
            FROM OPENQUERY([LOCALHOST,1444], 'SELECT ID, CODE01, DATE
                                              FROM TEST.DBO.TB_REMOTE02')
) C
ON A.ID = C.ID;
```

2. 분석 및 주요 최적화

1) 현재 데이터 흐름

테이블	위치	건수	비고
dbo.TB_REMOTE01	로컬	1,026,048	DATE 필터 후 881,645건
dbo.TB_CODE01	로컬	5	CODE 필터 역할
A ⋈ B 결과	로컬	2,084	실제 원격 조인에 필요한 ID 수
TB_REMOTE02	원격	전체	OPENQUERY로 전건 전송 ⊗

2) 문제 원인

```
OPENQUERY → TB_REMOTE02 전체를 로컬로 전송 (WHERE 없음)
→ 로컬에서 Hash Join (A.ID =C.ID)
  → 실제 필요한 건 2,084건뿐인데 원격 테이블 전체를 가져옴
```

*OPENQUERY는 원격에서 쿼리 결과를 통째로 가져오므로, 원격 테이블에 인덱스가 있어도 활용 불가합니다.

3. 개선안

```
-- 개선: REMOTE Join Hint + 4-part Naming
-- OPENQUERY 대신 4-part 이름으로 변경하고, REMOTE 조인 힌트를 사용하면:
-- 1) 로컬에서 필터링된 2,084개 ID만 원격 서버로 전송
-- 2) 원격 서버가 자신의 인덱스를 사용하여 해당 ID만 조회
-- 3) 매칭 결과만 로컬로 반환

SELECT A.ID, C.CODE01, C.[DATE]
FROM dbo.TB_REMOTE01 A
INNER JOIN dbo.TB_CODE01 B
    ON A.CODE01 = B.CODE
INNER REMOTE JOIN [LOCALHOST,1444].TEST.DBO.TB_REMOTE02 C
    ON A.ID = C.ID
WHERE A.[DATE] BETWEEN '20200101' AND '20240831';
```

· 변경 내용

항목	원래 쿼리	개선 쿼리
원격 접근 방식	OPENQUERY (전체 전송)	4-part naming + REMOTE hint
원격 인덱스 활용	✗ 불가	✓ 원격 서버에서 Index Seek
네트워크 전송	TB_REMOTE02 전체	2,084건 결과만
조인 방식	Hash Join (로컬)	Nested Loop (원격 실행)
SELECT *	서브쿼리에 사용 ✗	제거 ✓
DATE 예약어	미처리	[DATE]로 이스케이프 ✓

· INNER REMOTE JOIN 동작 원리

```
[원래 - OPENQUERY]
원격: SELECT ID, CODE01, DATE FROM TB_REMOTE02 → 전체 N건 전송
로컬: Hash Join (A.ID = C.ID) → 2,084건 반환

[개선 - REMOTE JOIN]
로컬: A ⋈ B 필터 → 2,084개 ID 추출
원격: 2,084개 ID로 TB_REMOTE02 Index Seek → 2,084건만 반환
```

*REMOTE 힌트는 조인 연산을 오른쪽 테이블(원격)에서 수행하도록 지시합니다. 로컬에서 필터링된 ID 목록이 원격 서버로 전달되고, 원격 서버는 자신의 인덱스를 활용하여 필요한 행만 반환합니다.

64 REMOTE QUERY - 3

원격 서버의 데이터와 조인하기 위해 OPENQUERY를 사용하여 성능 이슈가 발생된 사례이다.

1. 사전 구성 스크립트

```
/*로컬 서버*/

USE [master]
GO
EXEC master.dbo.sp_addlinkedserver
@server = N'LOCALHOST,1444' --연결된 서버 별칭 (접속 주소, 포트)
, @srvproduct = N'SQL Server'
EXEC master.dbo.sp_addlinkedsrvlogin
@rmtsrvname = N'LOCALHOST,1444' --연결된 서버 별칭 (접속 주소, 포트)
, @useself = 'false'
, @rmtuser = N'sa' --로그인 사용자
, @rmtpassword = N'P@ssW0rd!' --로그인 암호
GO
```

```
USE [Tuning]
SELECT number AS ID , NUMBER+1700 AS CODE
INTO TB_CODE02
FROM master..spt_values B
WHERE type = 'P' AND number BETWEEN 1 AND 5
GO
```

```
/* 원격 서버 */

USE [TEST]
GO
SELECT ROW_NUMBER () OVER (ORDER BY A.number) AS ID
      , A.number AS CODE01, RIGHT(A.number,1) AS CODE02
      , CASE WHEN B.number < 80
        THEN LEFT(CONVERT(CHAR(8), DATEADD(DD, A.high,'2020-01-01'),112),8)
        ELSE LEFT(CONVERT(CHAR(8), DATEADD(DD, A.number,'2020-01-01'),112),8) END AS
DATE
INTO TB_REMOTE03
FROM master..spt_values A, (SELECT number
                            FROM master..spt_values B
                            WHERE type = 'P' AND number <= 500) B
WHERE A.type = 'P'

CREATE CLUSTERED INDEX CIDX_REMOTE03 ON TB_REMOTE03(ID)
CREATE INDEX NIDX01_REMOTE03 ON TB_REMOTE03(CODE01)
GO
```

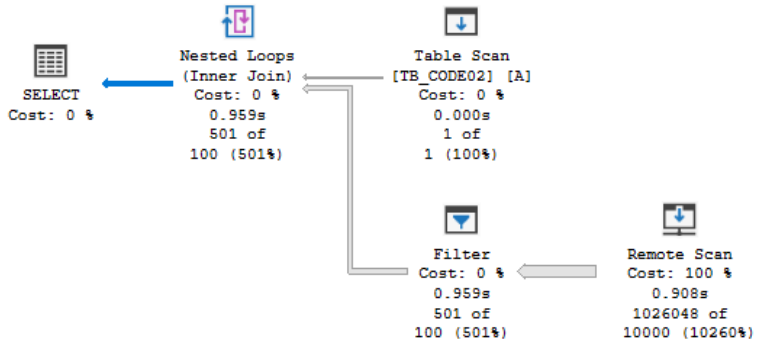
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
DECLARE @CODE VARCHAR(10)
SET @CODE = '1701'

SELECT A.* , B.CODE01
FROM TB_CODE02 A
INNER JOIN (SELECT *
            FROM OPENQUERY([LOCALHOST,1444], 'SELECT ID, CODE01
                                              FROM TEST.DBO.TB_REMOTE03')
) B
ON A.CODE = B.CODE01 AND A.CODE = @CODE
GO
```

· 실행 계획

Rows	Executes	StmtText
501	1	SELECT A.* , B.CODE01 FROM TB_CODE02 A INNER JOIN (SELECT * FROM NQUERY([LOCALHOST,1444], 'SELECT ID, CODE01 FROM TEST.DBO.TB_REMOTE03')) B ON A.CODE=B.CODE01 AND A.CODE =@CODE
501	1	¦--Nested Loops(INNER JOIN)
1	1	¦--Table Scan(OBJECT:([Tuning].[dbo].[TB_CODE02] AS [A]), WHERE:([Tuning].[dbo].[TB_CODE02].[CODE] as [A].[CODE]=CONVERT_IMPLICIT(int,[@CODE],0)))
501	1	¦--Filter(WHERE:([SQLNCLI11].[CODE01]=CONVERT_IMPLICIT(int,[@CODE],0)))
1026048	1	¦--Remote Scan(SOURCE:(LOCALHOST,1444), OBJECT:(SELECT ID, CODE01 FROM TEST.DBO.TB_REMOTE03))




```
테이블 'TB_CODE02'. 스캔 수 1, 논리적 읽기 1
SQL Server 실행 시간:
CPU 시간 = 500ms, 경과 시간 = 2162ms
```

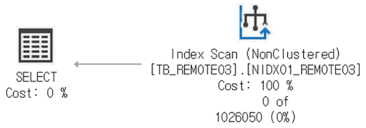
· 원격 서버 SQL 구문 (Profiler)

EventClass	TextData	CPU	Duration	Reads	RowCounts
Showplan XML	<ShowPlanXML xmlns="http://schemas.m...				
SQL:BatchCompleted	SELECT ID, CODE01 FROM TEST.dbo.T...	235	807	2295	1026048

```
SELECT ID, CODE01
FROM TEST.dbo.TB_REMOTE03
```

· 원격 서버 실행 계획 (Profiler)

```
StmtText
--------
Index Scan(OBJECT:([test].[dbo].[TB_REMOTE03].[NIDX01_REMOTE03]))
```



3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_REMOTE03	CIDX_REMOTE03	CLUSTERED	0	ID	-
TB_REMOTE03	NIDX01_REMOTE03	NONCLUSTERED	0	CODE01	-

4. [AS-IS] 구문 설명

로컬 서버 [TB_CODE02] 테이블의 [CODE] 컬럼을 변수로 '1701' 값을 입력받고 OPENQUERY를 통해 원격 서버 [TB_REMOTE03] 테이블과 조인하는 구문이다. 원격 서버에서 실행되는 구문에 WHERE 조건절이 없기 때문에 [TB_REMOTE03] 테이블에 대해 인덱스의 모든 데이터를 읽게 되면서 많은 I/O가 발생된다.

5. [TO-BE] 튜닝 포인트

원격 서버에서 실행되는 OPENQUERY에 WHERE 조건을 추가한다. OPENQUERY로 원격에서 수행할 때, 조건절에서 제시된 컬럼으로 인덱스가 존재한다면 해당 조건까지 포함하여 OPENQUERY로 수행하는 것으로 I/O를 개선할 수 있다.

6. [TO-BE] 개선 방안

1) OPENQUERY에 WHERE 조건으로 'CODE01 = 1701'을 추가한다.

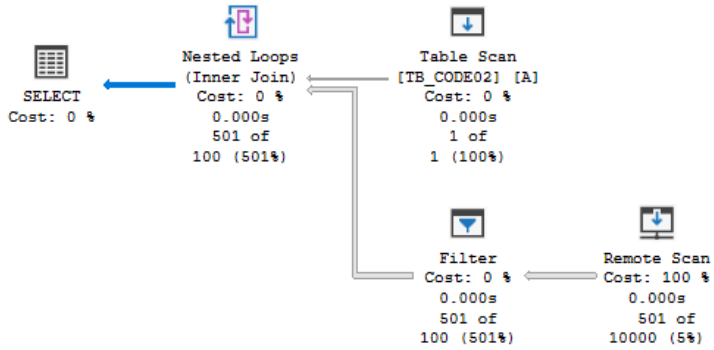
원격 서버 [TB_REMOTE03] 테이블에 접근하는 구문에 [CODE01] = 1701 조건을 추가하여 실행한다. 데이터를 인덱스 스캔으로 모두 읽어서 가져오는 부분이 인덱스 탐색 후 결과만 가져오고 해당 데이터만 조인하면서 I/O가 감소한다.

■ 변경된 SQL 구문

```
DECLARE @CODE VARCHAR(10), @SQL VARCHAR(8000)
SET @CODE = '1701'
SET @SQL = 'SELECT A.*, B.CODE01
             FROM TB_CODE02 A
             INNER JOIN (SELECT *
                         FROM OPENQUERY([LOCALHOST,1444], ''SELECT ID, CODE01
                                                           FROM TEST.DBO.TB_REMOTE03
                                                           WHERE CODE01 = ' + @CODE + ''')
                        ) B
             ON A.CODE = B.CODE01 AND A.CODE = ' + @CODE

EXEC (@SQL)
```

· 실행 계획

Rows	Executes	StmtText
501 501 1 501 501	1 1 1 1 1	SELECT A.*, B.CODE01 FROM TB_CODE02 A INNER JOIN (SELECT * FROM OPENQUERY([LOCALHOST,1444], 'SELECT ID, CODE01 FROM TEST.DBO.TB_REMOTE03 WHERE CODE01 = 1701')) B ON A.CODE = B.CODE01 AND A.CODE = 1701 ¦--Nested Loops(INNER JOIN) ¦--Table Scan(OBJECT:([tuning].[dbo].[TB_CODE02] AS [A]), WHERE:([tuning].[dbo].[TB_CODE02].[CODE] as [A].[CODE]=(1701))) ¦--Filter(WHERE:([SQLNCLI11].[CODE01]=(1701))) ¦--Remote Scan(SOURCE:(LOCALHOST,1444), OBJECT:(SELECT ID, CODE01 FROM TEST.DBO.TB_REMOTE03 WHERE CODE01 = 1701))



테이블 'TB_CODE02'. 스캔 수 1, 논리적 읽기 1
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 113ms

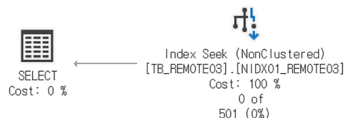
· 원격 서버 SQL 구문 (Profiler)

EventClass	TextData	CPU	Duration	Reads	RowCounts
Showplan XML	<ShowPlanXML xmlns="http://schemas.m...				
SQL:BatchCompleted	SELECT ID, CODE01 FROM test.dbo.T...	16	0	6	501

```
SELECT ID, CODE01
 FROM test.dbo.TB_REMOTE03
 WHERE CODE01 = 1701
```

· 원격 서버 실행 계획 (Profiler)

```
StmtText
Index Seek(OBJECT:([test].[dbo].[TB_REMOTE03].[NIDX01_REMOTE03]), SEEK:([test].[dbo].[TB_REMOTE03].[CODE01]=CONVERT_IMPLICIT(int,[@1],0)) ORDERED FORWARD)
```



7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	500	2,162	1
TO-BE	0	113	1

· 원격 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	343	2,092	2,295
TO-BE	0	1	6

GitHub Copilot 채팅 결과

64. REMOTE QUERY - 3

Claude Opus 4.6

1. 프롬프트

> [AS-IS]
> 구문에서 @CODE에 해당하는 데이터는 501건에 불과하지만, 원격 서버에서 전체 테이블을 읽어 들이면서 불필요한 I/O가 과도하게 발생한다. 이 문제를 분석하고 성능 개선안을 알려줘.

```
[AS-IS]
DECLARE @CODE VARCHAR(10)
SET @CODE = '1701'

SELECT A.* , B.CODE01
FROM TB_CODE02 A
INNER JOIN (SELECT *
FROM OPENQUERY([LOCALHOST,1444], 'SELECT ID,CODE01
FROM
TEST.DBO.TB_REMOTE03')
) B
ON A.CODE = B.CODE01 AND A.CODE = @CODE;
```

2. 분석 및 주요 최적화

1) 현재 상태

항목	값
로컬 테이블	dbo.TB_CODE02 (5건, CODE 컬럼은 int 타입)
CODE=1701 매칭	1건 (로컬)
원격 테이블	TEST.DBO.TB_REMOTE03
최종 결과	약 501건

2) 문제 원인

```
OPENQUERY → 원격 테이블 TB_REMOTE03 **전체** 를 로컬로 가져옴
→ 그 후 로컬에서 JOIN + WHERE 필터링
```

*OPENQUERY는 원격 서버에서 고정 쿼리를 실행한 뒤 결과 전체를 네트워크로 전송합니다.

현재 쿼리는 WHERE 조건 없이 전체를 가져오므로, 원격 테이블이 클수록 불필요한 네트워크 I/O

메모리 사용이 증가합니다.

3. 개선안

```
-- @CODE 값을 OPENQUERY 패스스루 쿼리 문자열에 포함시키면, 원격 서버에서 필터링 후
-- 결과만 전송합니다.

DECLARE @CODE INT = 1701;

DECLARE @SQL NVARCHAR(4000);
SET @SQL = N'SELECT A.ID, A.CODE, B.CODE01
FROM dbo.TB_CODE02 A
INNER JOIN OPENQUERY([LOCALHOST,1444],
    ''SELECT ID, CODE01
      FROM TEST.DBO.TB_REMOTE03
      WHERE CODE01 = ' + CAST(@CODE AS NVARCHAR(10)) + N''') B
ON A.CODE = B.CODE01
WHERE A.CODE = @CODE';

EXEC sp_executesql @SQL, N'@CODE INT', @CODE = @CODE;
```

*OPENQUERY 안에서 WHERE 조건을 직접 전달

65 PARTITION을 활용한 성능 개선 - 1

2개 이상의 범위 조건으로 인해 인덱스의 탐색 범위가 많아지면서 I/O가 증가된 사례이다.

1. 사전 구성 스크립트

```
CREATE TABLE TB_PartitionTable
      (ID INT, ProductName VARCHAR(64), ModifiedDate date, Time time)

DECLARE @i INT, @date varchar(16), @time varchar(16)
SET @i = 1
SET @date = NULL
SET @time = '10:00:00'
WHILE (@i < 1000)
      BEGIN
              SET @date = '2020-01-10'
              INSERT INTO TB_PartitionTable VALUES (@i, NEWID(), @date, @time)
              SET @date = '2020-02-10'
              INSERT INTO TB_PartitionTable VALUES (@i+1, NEWID(), @date, @time)
              SET @date = '2020-03-10'
              INSERT INTO TB_PartitionTable VALUES (@i+2, NEWID(), @date, @time)
              SET @date = '2020-04-10'
              INSERT INTO TB_PartitionTable VALUES (@i+3, NEWID(), @date, @time)
```

```
                SET @date = '2020-05-10'
                INSERT INTO TB_PartitionTable VALUES (@i+4, NEWID(), @date, @time)
                SET @date = '2020-06-10'
                INSERT INTO TB_PartitionTable VALUES (@i+5, NEWID(), @date, @time)
                SET @date = '2020-07-10'
                INSERT INTO TB_PartitionTable VALUES (@i+6, NEWID(), @date, @time)
                SET @date = '2020-08-10'
                INSERT INTO TB_PartitionTable VALUES (@i+7, NEWID(), @date, @time)
                SET @i = @i + 8
        END
WHILE (@i < 149800)
        BEGIN
                SET @date = '2020-09-03'
                SET @time = '02:00:00'
                INSERT INTO TB_PartitionTable VALUES (@i, NEWID(), @date, @time)
                SET @i = @i + 1
        END
WHILE (@i < 150000)
        BEGIN
                SET @date = '2020-09-10'
                SET @time = '10:00:00'
                INSERT INTO TB_PartitionTable VALUES (@i, NEWID(), @date, @time)
                SET @i = @i + 1
        END
CREATE CLUSTERED INDEX CIDX_PartitionTable ON TB_PartitionTable(ModifiedDate, Time)
GO
```

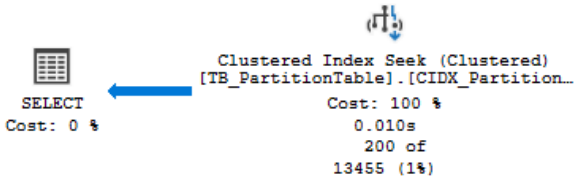
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
SELECT ProductName
FROM TB_PartitionTable
WHERE ModifiedDate BETWEEN '2020-09-01' AND '2020-09-30'
AND Time BETWEEN '09:00:00' AND '18:00:00'
GO
```

· 실행 계획

Rows	Executes	StmtText
200	1	SELECT [ProductName] FROM [TB_PartitionTable] WHERE [ModifiedDate]>=@1 AND [ModifiedDate]<=@2 AND [Time]>=@3 AND [Time]<=@4
200	1	¦--Clustered Index Seek(OBJECT:([Tuning].[dbo].[TB_PartitionTable].[CIDX_PartitionTable]), SEEK: (([Tuning].[dbo].[TB_PartitionTable].[ModifiedDate], [Tuning].[dbo].[TB_PartitionTable].[Time]) >= (CONVERT_IMPLICIT(date,[@1],0), CONVERT_IMPLICIT(time(7),[@3],0)) AND ([Tuning].[dbo].[TB_PartitionTable].[ModifiedDate], [Tuning].[dbo].[TB_PartitionTable].[Time]) <= (CONVERT_IMPLICIT(date,[@2],0), CONVERT_IMPLICIT(time(7),[@4],0))), WHERE:([Tuning].[dbo].[TB_PartitionTable].[Time]>=CONVERT_IMPLICIT(time(7),[@3],0) AND [Tuning].[dbo].[TB_PartitionTable].[Time]<=CONVERT_IMPLICIT(time(7),[@4],0)) Ordered Forward)




```
테이블 'TB_PartitionTable'. 스캔 수 1, 논리적 읽기 1255
SQL Server 실행 시간:
CPU 시간 = 16ms, 경과 시간 = 50ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_PartitionTable	CIDX_PartitionTable	CLUSTERED	0	ModifiedDate, Time	-

4. [AS-IS] 구문 설명

[TB_PartitionTable] 테이블에서 [ModifiedDate] 컬럼이 2020년 9월에 해당하면서 [Time] 컬럼이 09시~18시 사이에 있는 데이터를 출력하는 구문이다. [ModifiedDate], [Time] 두 컬럼이 클러스터형 인덱스 키로 구성되어 있지만 각각 범위 조건으로 사용되기 때문에 선행 컬럼인 [ModifiedDate]으로만 인덱스 탐색이 수행되며, [Time] 컬럼은 탐색된 모든 범위를 읽은 후에 필터가 이루어지기 때문에 읽어야 하는 스캔 범위가 많아지면서 높은 I/O가 사용된다.

5. [TO-BE] 튜닝 포인트

많은 양의 데이터에서 두 범위 조건을 모두 사용해야만 충분히 많은 행이 필터 된다면 두 조건 모두 탐색될 수 있는 방법을 고려해야 하는데 파티션 분할 기능은 그 중 하나의 방법이 될 수 있다. 날짜 조건으로 사용된 [ModifiedDate] 컬럼을 월 단위 기준으로 파티션을 분할한다면 Partition Elimination 기능을 통해서 일부 파티션만을 읽어 낼 수 있다. 또한 [Time] 컬럼에 인덱스를 생성하여 특정 파티션 내에서 인덱스 탐색을 수행하게 된다면 두 조건에 대해 필요한 데이터를 읽어내면서 효율적인 I/O를 사용할 수 있게 된다.

6. [TO-BE] 개선 방안

1) [TB_PartitionTable] 테이블을 파티션 테이블로 재구성 및 신규 인덱스 생성

[ModifiedDate] 컬럼을 월 기준으로 분할된 파티션 테이블로 재구성하고 특정 파티션 내에서 인덱스 탐색을 수행할 수 있도록 [Time] 컬럼이 키로 구성된 인덱스를 생성한다.

테이블 분할

```
DROP INDEX CIDX_PartitionTable ON TB_PartitionTable

CREATE PARTITION FUNCTION Partition_FN_TB_PT (date)
AS RANGE LEFT FOR VALUES (N'2020-01-01', N'2020-02-01', N'2020-03-01'
                        , N'2020-04-01', N'2020-05-01', N'2020-06-01'
                        , N'2020-07-01', N'2020-08-01', N'2020-09-01', N'2020-10-01')

CREATE PARTITION SCHEME Partition_Scheme_TB_PT
AS PARTITION Partition_FN_TB_PT TO ([PRIMARY], [PRIMARY], [PRIMARY], [PRIMARY]
                                  , [PRIMARY], [PRIMARY], [PRIMARY], [PRIMARY]
                                  , [PRIMARY], [PRIMARY], [PRIMARY])

CREATE CLUSTERED INDEX CIDX_PT_Scheme ON dbo.TB_PartitionTable
(ModifiedDate) ON Partition_Scheme_TB_PT(ModifiedDate)
GO
```

인덱스 생성 구문

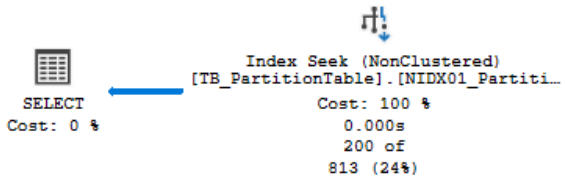
```
CREATE INDEX NIDX01_PartitionTable ON TB_PartitionTable(Time) INCLUDE (ProductName)
GO
```

SQL 구문

```
SELECT ProductName
FROM TB_PartitionTable
WHERE ModifiedDate BETWEEN '2020-09-01' AND '2020-09-30'
AND Time BETWEEN '09:00:00' AND '18:00:00'
GO
```

· 실행 계획

Rows	Executes	StmtText
200	1	SELECT [ProductName] FROM [TB_PartitionTable] WHERE [ModifiedDate]>=@1 AND [ModifiedDate]<=@2 AND [Time]>=@3 AND [Time]<=@4
200	1	|--Index Seek(OBJECT:([Tuning].[dbo].[TB_PartitionTable].[NIDX01_PartitionTable]), SEEK:([PtnId1000] >= (9) AND [PtnId1000] <= (10) AND ([Tuning].[dbo].[TB_PartitionTable].[Time], [Tuning].[dbo].[TB_PartitionTable].[ModifiedDate]) >= ('09:00:00.0000000', '2020-09-01') AND ([Tuning].[dbo].[TB_PartitionTable].[Time], [Tuning].[dbo].[TB_PartitionTable].[ModifiedDate]) <= ('18:00:00.0000000', '2020-09-30')), WHERE:([Tuning].[dbo].[TB_PartitionTable].[ModifiedDate]>='2020-09-01' AND [Tuning].[dbo].[TB_PartitionTable].[ModifiedDate]<='2020-09-30') Ordered Forward)



테이블 'TB_PartitionTable'. 스캔 수 2, 논리적 읽기 수 8
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 0ms

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	16	50	1,255
TO-BE	0	0	8

GitHub Copilot 채팅 결과

Claude Opus 4.6

65. PARTITION을 활용한 성능 개선 - 1

1. 프롬프트

> [AS-IS]
> 구문은 ModifiedDate와 Time을 키로 클러스터드 인덱스가 존재하지만, 두 조건 모두 범위 조건이어서 ModifiedDate만 탐색 키로 사용된다. 그 결과 불필요하게 넓은 범위를 읽게 되어 성능 비효율이 발생한다. 이 문제를 분석하고 성능 개선안을 알려줘.

```
[AS-IS]
SELECT ProductName
FROM TB_PartitionTable
WHERE ModifiedDate BETWEEN '2020-09-01' AND '2020-09-30'
AND Time BETWEEN '09:00:00' AND '18:00:00';
```

2. 분석 및 주요 최적화

1) 현재 테이블 및 인덱스 구조

항목	값
테이블	dbo.TB_PartitionTable
전체 행 수	149,999
클러스터드 인덱스	CIDX_PartitionTable (ModifiedDate, Time)
파티셔닝	미적용

2) 데이터 분포 (핵심)

ModifiedDate	Time	건수
2020.9.3	02:00:00	148,799
2020.9.10	10:00:00	200
(기타 8개 날짜)	10:00:00	각 125

3) 문제 원인

말씀하신 대로 **두 컬럼 모두 범위 조건(BETWEEN)**이므로, 복합 인덱스의 두 번째 키(Time)는
Seek 키로 사용 불가합니다.

Clustered Index Seek: ModifiedDate BETWEEN '2020-09-01' AND '2020-09-30'
→ 148,999건 스캔 (09-03 + 09-10 데이터 모두)
Residual Predicate: Time BETWEEN '09:00:00' AND '18:00:00'
→ 148,799건 버리고 200건만 반환 ✖

*148,999건을 읽고 200건만 반환 → 불필요한 I/O가 99.87% 입니다.

3. 개선안

```
-- 쿼리 리라이트 – CROSS APPLY + 날짜 등치 조건 (스키마 변경 없음, 권장)

-- 핵심 아이디어: 첫 번째 키를 등치(=) 조건으로 바꾸면,
-- 두 번째 키(Time)도 Seek 키로 사용됩니다.
-- 등치 조건 시 인덱스 동작:
-- ModifiedDate = '2020-09-10' AND Time BETWEEN '09:00:00' AND '18:00:00'
-- → 정확히 200건만 Seek

-- 날짜 CTE로 각 일자를 등치 조건으로 풀어서 CROSS APPLY하면,
-- 기존 클러스터드 인덱스를 최대한 활용할 수 있습니다:

;WITH Dates AS (
    SELECT CAST('2020-09-01' AS date) AS dt
    UNION ALL
    SELECT DATEADD(DAY, 1, dt) FROM Dates WHERE dt < '2020-09-30'
) -- 2020-09-01 ~ 2020-09-30 dt 테이블
SELECT pt.ProductName
FROM Dates -- 09-01 ~ 09-30 테이블을 Cross Apply로 TB_PartitionTable에 JOIN
CROSS APPLY (
    SELECT ProductName
    FROM dbo.TB_PartitionTable
    WHERE ModifiedDate = Dates.dt -- 기존 범위 조건이었던 날짜를 이퀄 조건으로 변경
    AND Time BETWEEN '09:00:00' AND '18:00:00' -- 09~18 범위만 탐색 가능
) pt
OPTION (MAXRECURSION 31); -- 재귀 함수 반복 횟수 제한 (32회 부터 에러)
```

비교	원래 쿼리	CROSS APPLY 리라이트
ModifiedDate 조건	범위 (BETWEEN)	등치 (=) × 30회
Time Seek 사용	✖ Residual	✔ Seek 키로 사용
읽는 행 수	~148,999	~200
스키마 변경	-	불필요

*각 날짜별로 (ModifiedDate = ?, Time BETWEEN ...) 형태의 정밀 Seek이 30회 발생하므로, 데이터가 없는 날짜는 즉시 빈 결과를 반환하고, 있는 날짜만 정확한 범위를 탐색합니다.

66 PARTITION을 활용한 성능 개선 - 2

DELETE 구문을 수행할 때 범위 조건에 해당하는 데이터의 양이 많아서 I/O 비용이 높게 발생되는 사례이다.

1. 사전 구성 스크립트

```
SELECT CASE WHEN RIGHT(CONVERT(VARCHAR,a.number),1) <= 8 THEN 'A' ELSE 'B' END ID
      , CONVERT(VARCHAR,a.number) ProductNumber
      , CONVERT(VARCHAR,b.number) CatalogNumber
      , CONVERT(DATE,DATEADD(dd,a.number,'2015-01-01'),0) ModifiedDate
INTO TB_PartitionDelete
FROM master..spt_values a, (SELECT number
                            FROM master..spt_values
                            WHERE type = 'P') b
WHERE a.type = 'P'

CREATE CLUSTERED INDEX CIDX_PartitionDelete ON TB_PartitionDelete(ModifiedDate)
GO
```

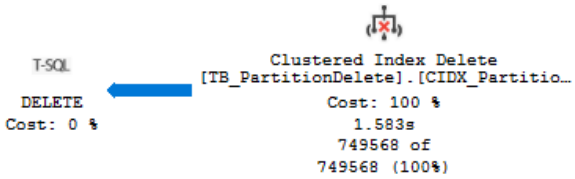
2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
DELETE
FROM TB_PartitionDelete
WHERE ModifiedDate BETWEEN '2015-01-01' AND '2015-12-31'
GO
```

· 실행 계획

Rows	Executes	StmtText
749568 749568	1 1	DELETE [TB_PartitionDelete] WHERE [ModifiedDate]>=@1 AND [ModifiedDate]<=@2 ¦--Clustered Index Delete(OBJECT:([Tuning].[dbo].[TB_PartitionDelete].[CIDX_PartitionDelete]), WHERE:([Tuning].[dbo].[TB_PartitionDelete].[ModifiedDate] >= CONVERT_IMPLICIT(date,[@1],0) AND [Tuning].[dbo].[TB_PartitionDelete].[ModifiedDate] <= CONVERT_IMPLICIT(date,[@2],0)))




```
테이블 'TB_PartitionDelete'. 스캔 수 1, 논리적 읽기 35649
SQL Server 실행 시간:
CPU 시간 = 781ms, 경과 시간 = 2206ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_PartitionDelete	CIDX_PartitionDelete	CLUSTERED	0	ModifiedDate	-

4. [AS-IS] 구문 설명

[TB_PartitionDelete] 테이블의 [ModifiedDate] 컬럼에서 2015년(2015.01.01~2015.12.31)에 해당하 는 데이터를 삭제하는 구문이다. 클러스터형 인덱스를 통해 [ModifiedDate] 조건에 해당하는 데이터인 약 75만 건을 탐색하고 삭제하면서 I/O 비용이 높게 발생되며 배타적 락으로 인한 블록킹 이슈를 발생시킬 수 있다.

5. [TO-BE] 튜닝 포인트

특정 범위의 데이터만을 삭제하기 위해 DELETE 구문을 사용하면서 성능적 이슈가 발생되었다. TRUNCATE 구문을 통해 데이터를 삭제한다면 데이터를 탐색하는 과정을 최소화할 수 있지만 특정 범위를 제한하는 조건을 지정할 수 없다. 이러한 단점을 해소할 수 있는 하나의 방안으로 파티션 분할 기능을 고려할 수 있다. 조건으로 사용된 날짜 데이터 기준으로 파티션을 분할한다면 특정 범위의 데이터가 포함된 파티션만을 스위칭하여 TRUNCATE 명령을 통해 신속하게 삭제할 수 있다.

6. [TO-BE] 개선 방안

1) [TB_PartitionDelete] 테이블을 파티션 테이블로 재구성 및 스위칭용 파티션 테이블 생성

[ModifiedDate] 컬럼을 연도 기준으로 분할된 파티션 테이블로 재구성하고, 스위칭용 테이블인 [TB_DeletePD]를 [TB_PartitionDelete] 테이블과 동일한 구조로 생성한다. 삭제하고자 하는 데이터를 [TB_DeletePD] 테이블로 스위칭하고 TRUNCATE 명령을 통해 데이터를 삭제한다.

■ 테이블 분할

```
/* 기존 클러스터형 인덱스 삭제 */

DROP INDEX CIDX_PartitionDelete ON TB_PartitionDelete


/* 파티션 함수 생성 */

CREATE PARTITION FUNCTION [Partition_FN_TB_PD](date)
AS RANGE RIGHT FOR VALUES (N'2016-01-01', N'2017-01-01', N'2018-01-01',
                           N'2019-01-01', N'2020-01-01')

/* 파티션 구성표 생성 */

CREATE PARTITION SCHEME [Partition_Scheme_TB_PD]
AS PARTITION [Partition_FN_TB_PD] TO ([PRIMARY], [PRIMARY], [PRIMARY],
                                      [PRIMARY], [PRIMARY], [PRIMARY])


/* 클러스터형 인덱스를 파티션을 적용하여 생성 */

CREATE CLUSTERED INDEX CIDX_PartitionDelete ON TB_PartitionDelete(ModifiedDate)
                                            ON Partition_Scheme_TB_PD(ModifiedDate)
GO
```

■ 삭제할 데이터 스위치 테이블 생성

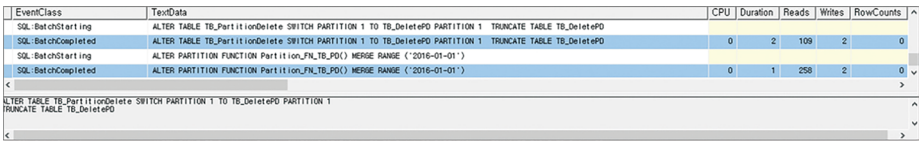
```
CREATE TABLE TB_DeletePD (ID VARCHAR NOT NULL
      , ProductNumber VARCHAR(30)
      , CatalogNumber VARCHAR(30)
      , ModifiedDate DATE)

CREATE CLUSTERED INDEX CIDX_DeletePD ON TB_DeletePD(ModifiedDate)
ON Partition_Scheme_TB_PD(ModifiedDate)
GO
```

■ 변경된 SQL 구문

```
/* 삭제 대상 데이터 스위칭 [TB_PartitionDelete] -> [TB_DeletePD] */

ALTER TABLE TB_PartitionDelete SWITCH PARTITION 1 TO TB_DeletePD PARTITION 1


/* 스위칭된 데이터 삭제 */

TRUNCATE TABLE TB_DeletePD


/* 삭제된 파티션 병합 */

ALTER PARTITION FUNCTION Partition_FN_TB_PD() MERGE RANGE ('2016-01-01')
GO
```

*SQL Server 2016부터는 TRUNCATE TABLE 구문에서 특정 파티션만 선택적으로 잘라내는 기능을 지원한다.
예) `TRUNCATE TABLE TB_PartitionDelete WITH (PARTITIONS (1));`

· 실행 계획 (Profiler)



EventClass	TextData	CPU	Duration	Reads	Writes	RowCounts
SQL:BatchStarting	ALTER TABLE TB_PartitionDelete SWITCH PARTITION 1 TO TB_DeletePD PARTITION 1 TRUNCATE TABLE TB_DeletePD					
SQL:BatchCompleted	ALTER TABLE TB_PartitionDelete SWITCH PARTITION 1 TO TB_DeletePD PARTITION 1 TRUNCATE TABLE TB_DeletePD	0	2	109	2	0
SQL:BatchStarting	ALTER PARTITION FUNCTION Partition_FN_TB_PD() MERGE RANGE ('2016-01-01')					
SQL:BatchCompleted	ALTER PARTITION FUNCTION Partition_FN_TB_PD() MERGE RANGE ('2016-01-01')	0	1	258	2	0

```
ALTER TABLE TB_PartitionDelete SWITCH PARTITION 1 TO TB_DeletePD PARTITION 1
TRUNCATE TABLE TB_DeletePD
```

```
테이블 'TB_PartitionDelete'. 스캔 수 0, 논리적 읽기 367
SQL Server 실행 시간:
CPU 시간 = 0ms, 경과 시간 = 3ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	781	2,206	35,649
TO-BE	0	3	367

GitHub Copilot 채팅 결과

Claude Opus 4.6

66. PARTITION을 활용한 성능 개선 - 2

1. 프롬프트

> [AS-IS]
> 구문은 조건을 만족하는 약 75만 건의 데이터를 탐색하며 DELETE를 수행하기 때문에 과도한 I/O가 발생한다. 이 문제를 분석하고 최소한의 I/O 비용으로 처리할 수 있도록 성능 개선안을 알려줘.

```
[AS-IS]
DELETE
FROM TB_PartitionDelete
WHERE ModifiedDate BETWEEN '2015-01-01' AND '2015-12-31';
```

2. 분석 및 주요 최적화

1) 현재 테이블 상태를 분석한 결과입니다:

항목	값
총 행 수	4,194,304
2015년 대상 행 수	747,520
파티션 여부	비파티션
클러스터형 인덱스	ModifiedDate 기준
날짜 범위	2015-01-01 ~ 2020-08-09

2) 문제점

현재 DELETE + WHERE 방식은 747,520건을 한 건씩 찾아서 삭제하므로 대량의 I/O와 트랜잭션 로그가 발생합니다.

3. 개선안

```
-- 파티션 TRUNCATE
-- 클러스터형 인덱스가 이미 ModifiedDate에 있으므로, 테이블을 연도별로 파티셔닝한 후
-- 해당 파티션만 TRUNCATE하면 메타데이터 연산만으로 즉시 삭제됩니다.
-- I/O가 사실상 0에 가깝습니다.
```

```
-------------------------------------------------------
-- 1) 파티션 함수 생성 (연도별 RIGHT 범위)
-------------------------------------------------------

CREATE PARTITION FUNCTION pf_ModifiedDate_Year (date)
AS RANGE RIGHT FOR VALUES (
    '2015-01-01',
    '2016-01-01',
    '2017-01-01',
    '2018-01-01',
    '2019-01-01',
    '2020-01-01',
    '2021-01-01'
);


-------------------------------------------------------
-- 2) 파티션 스킴 생성 (모든 파티션을 PRIMARY에 배치)
-------------------------------------------------------

CREATE PARTITION SCHEME ps_ModifiedDate_Year
AS PARTITION pf_ModifiedDate_Year
ALL TO ([PRIMARY]);

-------------------------------------------------------
-- 3) 기존 클러스터형 인덱스를 파티션 스킴 위로 재생성
--    (데이터가 연도별 파티션으로 재배치됨)
-------------------------------------------------------

DROP INDEX CIDX_PartitionDelete ON dbo.TB_PartitionDelete;


CREATE CLUSTERED INDEX CIDX_PartitionDelete
ON dbo.TB_PartitionDelete (ModifiedDate)
ON ps_ModifiedDate_Year (ModifiedDate);


-------------------------------------------------------
-- 4) 2015년 파티션(파티션 2)만 TRUNCATE → I/O 거의 0
-------------------------------------------------------

TRUNCATE TABLE dbo.TB_PartitionDelete
WITH (PARTITIONS (2));
```

67 PARTITION을 활용한 성능 개선 - 3

변경된 일부 데이터의 통계를 변경하기 위해 전체 데이터를 읽어내면서 I/O가 높게 발생되는 사례이다.

1. 사전 구성 스크립트

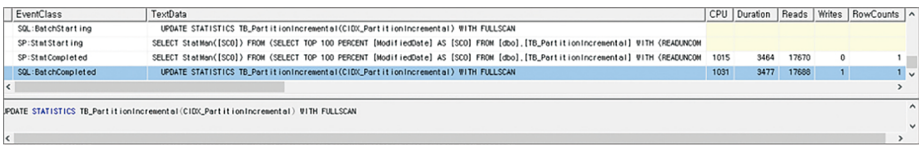
```
SELECT CASE WHEN RIGHT(CONVERT(VARCHAR,a.number),1) <= 8 THEN 'A' ELSE 'B' END ID
      , CONVERT(VARCHAR,a.number) ProductNumber
      , CONVERT(VARCHAR,b.number) CatalogNumber
      , CONVERT(DATE,DATEADD(DD,a.number+b.number,'2010-01-01'),0) ModifiedDate
INTO TB_PartitionIncremental
FROM master..spt_values a, (SELECT number
                            FROM master..spt_values
                            WHERE type ='P') b
WHERE a.type = 'P'

CREATE CLUSTERED INDEX CIDX_PartitionIncremental ON TB_PartitionIncremental(ModifiedDa
te)
GO
```

2. [AS-IS] SQL 구문 및 실행 계획

■ SQL 구문

```
UPDATE TB_PartitionIncremental SET ModifiedDate = '2020-04-12'
WHERE ModifiedDate BETWEEN '2020-01-01' AND '2020-12-31'

UPDATE STATISTICS TB_PartitionIncremental (CIDX_PartitionIncremental) WITH FULLSCAN
GO
```

· 실행 계획 (Profiler)



EventClass	TextData	CPU	Duration	Reads	Writes	RowCounts
SQL:BatchStarting	UPDATE STATISTICS TB_PartitionIncremental(CIDX_PartitionIncremental) WITH FULLSCAN					
SP:StmtStarting	SELECT StatMan([SC0]) FROM (SELECT TOP 100 PERCENT [ModifiedDate] AS [SC0] FROM [dbo].[TB_PartitionIncremental] WITH (READUNCOM					
SP:StmtCompleted	SELECT StatMan([SC0]) FROM (SELECT TOP 100 PERCENT [ModifiedDate] AS [SC0] FROM [dbo].[TB_PartitionIncremental] WITH (READUNCOM	1015	3464	17670	0	1
SQL:BatchCompleted	UPDATE STATISTICS TB_PartitionIncremental(CIDX_PartitionIncremental) WITH FULLSCAN	1031	3477	17688	1	1

```
UPDATE STATISTICS TB_PartitionIncremental(CIDX_PartitionIncremental) WITH FULLSCAN
```

```
테이블 'TB_PartitionIncremental'. 스캔 수 0, 논리적 읽기 17688
SQL Server 실행 시간:
CPU 시간 = 1031ms, 경과 시간 = 3477ms
```

3. [AS-IS] 통계 정보

· 인덱스 정보

Table_Name	Index_Name	Type_Desc	Is_Unique	Key_List	Include_List
TB_PartitionIncremental	CIDX_PartitionIncremental	CLUSTERED	0	ModifiedDate	-

4. [AS-IS] 구문 설명

[TB_PartitionIncremental] 테이블의 [ModifiedDate] 컬럼 값이 2020년도 (2020.01.01~2020.12.31) 에 해당하는 데이터를 업데이트하고 변경된 데이터의 통계를 업데이트하는 구문이다. 테이블의 총 데 이터 건수 약 400만 건 중 변경된 데이터는 약 9만 건이지만, 정확한 통계 데이터를 유지하기 위해선 테이블의 모든 데이터를 읽어내야 하기 때문에 통계 업데이트의 많은 I/O 비용이 발생된다

5. [TO-BE] 튜닝 포인트

일부 데이터 변경에 따라 통계를 업데이트하기 위해 모든 데이터를 읽어내는 부분에서 성능 이슈가 발생된다. 이러한 단점을 개선할 수 있는 방법으로 테이블 파티션을 고려할 수 있다. 조건으로 사용된 날짜 데이터를 기준으로 테이블을 파티션 분할한다면 INCREMENTAL 옵션을 적용하여 통계 데이터도 분할할 수 있고 이로 인해 변경된 데이터가 포함된 파티션의 통계만을 업데이트할 수 있게 되면서 I/O 비용을 개선할 수 있다.

6. [TO-BE] 개선 방안

1) [TB_PartitionIncremental] 테이블을 파티션 테이블로 재구성 및 인덱스 통계 구조 변경

[ModifiedDate] 컬럼을 월 기준으로 파티션 테이블로 재구성하고, 클러스터형 인덱스의 통계 데이터 분할을 위해 INCREMENTAL 옵션을 적용하여 통계를 업데이트한다. 적용된 통계 데이터는 분할되어 관리되기 때문에 특정 파티션의 통계만을 업데이트할 수 있게 된다.

■ 파티션 테이블로 재구성

```
BEGIN TRANSACTION

DROP INDEX CIDX_PartitionIncremental ON TB_PartitionIncremental

CREATE PARTITION FUNCTION [Partition_FN_TB_PI](DATE)
AS RANGE RIGHT FOR VALUES (N'2011-01-01', N'2012-01-01', N'2013-01-01', N'2014-01-01'
                         , N'2015-01-01', N'2016-01-01', N'2017-01-01', N'2018-01-01'
                         , N'2019-01-01', N'2020-01-01', N'2021-01-01')
```

```
CREATE PARTITION SCHEME [Partition_Scheme_TB_PI]
AS PARTITION [Partition_FN_TB_PI] TO ([PRIMARY], [PRIMARY], [PRIMARY], [PRIMARY]
                                    , [PRIMARY], [PRIMARY], [PRIMARY], [PRIMARY]
                                    , [PRIMARY], [PRIMARY], [PRIMARY], [PRIMARY])

CREATE CLUSTERED INDEX CIDX_PartitionIncremental
ON TB_PartitionIncremental(ModifiedDate) ON Partition_Scheme_TB_PI(ModifiedDate)
COMMIT TRANSACTION
GO
```

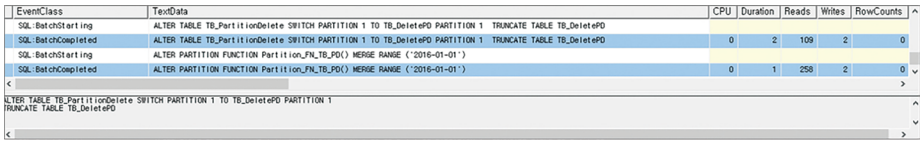
■ 파티션된 클러스터형 인덱스에 INCREMENTAL 옵션을 적용하여 통계 업데이트

```
UPDATE STATISTICS TB_PartitionIncremental(CIDX_PartitionIncremental)
WITH FULLSCAN, INCREMENTAL = ON
GO
```

■ 변경된 SQL 구문

```
UPDATE TB_PartitionIncremental SET ModifiedDate = '2020-04-12'
WHERE ModifiedDate BETWEEN '2020-01-01' AND '2020-12-31'

UPDATE STATISTICS TB_PartitionIncremental (CIDX_PartitionIncremental)
WITH RESAMPLE ON PARTITIONS (11)
GO
```

· 실행 계획 (Profiler)




```
테이블 'TB_PartitionIncremental'. 스캔 수 0, 논리적 읽기 978
SQL Server 실행 시간:
CPU 시간 = 16ms, 경과 시간 = 16ms
```

7. 개선 효과

· 리소스 사용률 비교

	CPU Time (ms)	Elapsed Time (ms)	Logical Reads
AS-IS	1,031	3,477	17,688
TO-BE	16	16	978

1. 프롬프트

> [AS-IS]
> 구문은 일부 데이터만 업데이트했음에도 불구하고 통계 업데이트가 전체 데이터를 기준으로 수행되어 과도한 I/O가 발생한다. 이 문제를 분석하고 기존 통계를 활용하여 성능 개선안을 알려줘.
> * 조건 : 기존 통계 사용

```
[AS-IS]
UPDATE TB_PartitionIncremental SET ModifiedDate = '2020-04-12'
WHERE ModifiedDate BETWEEN '2020-01-01' AND '2020-12-31'

UPDATE STATISTICS TB_PartitionIncremental (CIDX_PartitionIncremental) WITH FULLSCAN;
```

2. 분석 및 주요 최적화

1) 현재 상태 분석

항목	값
전체 행 수	4,194,304
UPDATE 대상 행 수	95,420 (약 2.3%)
데이터 범위	2010-01-01 ~ 2021-03-18
파티션 여부	비파티션 (단일)
증분 통계 (is_incremental)	FALSE
현재 통계 업데이트 방식	FULLSCAN (전체 스캔)

2) 문제점

구분	설명
데이터 변경	전체 419만 건 중 약 9.5만 건(2.3%)만 수정
통계 업데이트	WITH FULLSCAN으로 419만 건 전체를 스캔
비효율	변경되지 않은 97.7%의 데이터까지 불필요하게 읽어 대량의 I/O 발생

*변경 범위는 일부분이지만, 비파티션 테이블에서 FULLSCAN을 사용하면 전체 데이터를 재스캔할 수밖에 없습니다.

3. 개선안

```
-- 개선 방안: 파티션 + 증분 통계(Incremental Statistics)
-- 증분 통계는 파티션 테이블에서 변경된 파티션만 재스캔하고, 나머지 파티션의 기존 통계는
-- 그대로 병합(merge)합니다. 이를 통해 I/O를 대폭 줄일 수 있습니다.

-- ============================================================
-- STEP 1: 파티션 함수 생성 (연도별 경계)
-- ============================================================

CREATE PARTITION FUNCTION PF_PartitionIncremental (DATE)
AS RANGE RIGHT FOR VALUES
(
    '2011-01-01', '2012-01-01', '2013-01-01', '2014-01-01',
    '2015-01-01', '2016-01-01', '2017-01-01', '2018-01-01',
    '2019-01-01', '2020-01-01', '2021-01-01'
);

-- ============================================================
-- STEP 2: 파티션 스키마 생성 (모두 PRIMARY 파일그룹 사용)
-- ============================================================

CREATE PARTITION SCHEME PS_PartitionIncremental
AS PARTITION PF_PartitionIncremental
ALL TO ([PRIMARY]);


-- ============================================================
-- STEP 3: 클러스터형 인덱스를 파티션 스키마로 재구성
-- ============================================================

DROP INDEX CIDX_PartitionIncremental ON dbo.TB_PartitionIncremental;


CREATE CLUSTERED INDEX CIDX_PartitionIncremental
ON dbo.TB_PartitionIncremental (ModifiedDate)
ON PS_PartitionIncremental (ModifiedDate);



-- ============================================================
-- STEP 4: 증분 통계 활성화
-- ============================================================

UPDATE STATISTICS dbo.TB_PartitionIncremental (CIDX_PartitionIncremental)
WITH FULLSCAN, INCREMENTAL = ON;
```

```sql
-- ============================================================
-- STEP 5: 데이터 업데이트 (기존과 동일)
-- ============================================================

UPDATE dbo.TB_PartitionIncremental
SET ModifiedDate = '2020-04-12'
WHERE ModifiedDate BETWEEN '2020-01-01' AND '2020-12-31';


-- ============================================================
-- STEP 6: 증분 통계 업데이트 (변경된 파티션만 재스캔)
-- ============================================================

UPDATE STATISTICS dbo.TB_PartitionIncremental (CIDX_PartitionIncremental)
WITH RESAMPLE ON PARTITIONS (11);
```

마치며

04

총정리 & 에필로그

SQL 성능 최적화는 오랜 시간 동안 경험과 직관에 크게 의존해 온 영역입니다. 실행 계획을 해석하고 병목 지점을 추적하며 다양한 가설을 검증하는 과정은 많은 시간과 노력을 요구합니다. 이러한 특성 때문에 SQL 튜닝은 일부 숙련된 전문가의 영역으로 여겨지기도 했습니다.

그러나 GitHub Copilot 채팅의 등장으로 이러한 흐름에 변화가 나타나고 있습니다. 이제는 단순한 코드 자동화 도구를 넘어, 문제를 이해하고 해결 방향을 탐색하는 데 도움을 주는 협업 도구로 활용되고 있습니다. 이러한 변화 속에서 가장 두드러지는 점은 '속도'와 '시야의 확장'입니다. 쿼리와 실행 계획을 기반으로 문제를 제시하면, 다양한 관점에서의 개선 방향을 빠르게 확인할 수 있습니다. 과거에는 사람이 직접 모든 가능성을 검토해야 했다면, 이제는 여러 접근 방식을 동시에 고려할 수 있는 환경이 마련된 것입니다. 인덱스 설계 변경, 쿼리 재작성, 통계 정보 점검, 실행 계획 분석 등 다양한 해결 방안이 함께 제시되며, 이는 기존에 떠올리지 못했던 새로운 가능성을 발견하게 만듭니다. 이러한 경험은 자연스럽게 SQL 튜닝에 대한 사고의 폭을 넓혀줍니다.

특히 반복적인 분석 작업에서 그 가치는 더욱 분명하게 드러납니다. 실행 계획 해석이나 대기 유형 분석과 같은 작업에서 핵심을 빠르게 정리해주고, 문제의 원인을 추정하는 데 도움을 줍니다. 그 결과, 사용자는 단순한 정보 해석에 소모되는 시간을 줄이고 보다 중요한 의사결정에 집중할 수 있습니다.

이 책에서 다룬 다양한 사례는 이러한 활용 방식의 출발점에 해당합니다. 다만 중요한 점은, 제시되는 결과가 언제나 정답은 아니라는 사실입니다. 이는 어디까지나 가능한 방향일 뿐이며, 최종적인 판단과 선택은 여전히 사용자의 몫입니다.

최근 LLM 모델의 급격한 발전은 문제 해결 방식 자체에 변화를 가져오고 있습니다. 과거에는 질문의 수준에 따라 결과의 품질이 크게 좌우되었지만, 이제는 비교적 단순한 질문에도 높은 수준의 답변을 얻을 수 있는 환경이 되었습니다. 이로 인해 더욱 중요해진 것은 질문의 정교함이 아니라, 결과를 해석하고 판단하는 능력입니다. 다양한 가능성 중에서 어떤 선택이 적절한지를 결정하는 과정이 성능 최적화의 핵심이 되었기 때문입니다.

그럼에도 불구하고, 효과적인 활용을 위해서는 문제 상황을 명확하게 정의하고 필요한 정보를 정확하게 전달하는 기본적인 질문 역량이 여전히 중요합니다. 다만 이제는 질문 자체보다, 그 결과를 어떻게 이해하고 활용하느냐가 더 큰 차이를 만듭니다.

앞으로 SQL 튜닝의 방식은 점점 더 AI와의 협업 형태로 발전해 나갈 것입니다. 단순한 자동화를 넘어, 문제 해결을 함께 고민하는 파트너로서 활용되는 방향으로 변화하고 있습니다. 이러한 흐름 속에서 GitHub Copilot 채팅은 개발자와 DBA의 생산성을 높이는 핵심 도구로 자리 잡을 것입니다. 데이터의 규모는 계속해서 증가하고 시스템 구조는 더욱 복잡해지고 있습니다. 이러한 환경에서 모든 문제를 사람의 경험만으로 해결하는 데에는 분명한 한계가 존재합니다. AI는 이러한 한계를 보완하며, 보다 효율적인 문제 해결을 가능하게 하는 중요한 수단이 될 것입니다.

결국 중요한 것은 도구 그 자체가 아니라, 그것을 어떻게 활용하느냐에 있습니다. 단순한 자동화 도구로 사용하는 것과, 문제 해결을 위한 협업 파트너로 받아들이는 것은 전혀 다른 결과를 만들어냅니다.

이 책을 통해 독자 여러분이 SQL 성능 문제를 보다 빠르고 체계적으로 해결할 수 있는 기반을 마련하고, 변화하는 개발 환경 속에서 경쟁력을 갖춘 엔지니어로 성장할 수 있기를 기대합니다.

SQL 튜닝은 더 이상 혼자 고민하는 작업이 아닙니다. 이제는 GitHub Copilot 채팅과 함께 문제를 해결하는 시대가 시작되었습니다.

찾아보기

숫자

- 4-Part Name 370

A - B

- Anti Semi Join 127
- AUTO_CREATE_STATISTICS 364
- AUTO_UPDATE_STATISTICS 351
- Backward 263
- B-Tree 59

C

- CASCADE 327
- CONVERT_IMPLICIT 111
- CROSS APPLY 66
- CROSS JOIN 72

D

- DeadLock 302
- Density Vector 195
- Dirty Read 293
- DISTINCT 82

E - H

- Estimate Rows 353
- Foreign Key 67
- Hash Join 96
- Heap 41
- Histogram 190

I

- Include 36
- INCREMENTAL 403
- INNER JOIN 60
- IS NOT NULL 146
- ISNULL 254
- ISOLATION LEVEL 297

J

- Join Hint 383

K

- Key Lookup 42

L

- Leaf 39
- LEFT OUTER JOIN 140
- Linked Server 380
- Lock 288

M

- MERGE 340
- Merge Join 136

N

- Nested Loop Join 35
- NOLOCK 289
- NOT EXISTS 126
- ntext 277
- NVARCHAR 277

O

- OPENQUERY 373
- OVER 216

P

- Partition Elimination 392

R

- READ COMMITTED 295
- READ_COMMITTED_SNAPSHOT 292
- READPAST 295
- READ UNCOMMITTED 312
- REPEATABLE READ 304
- RID Lookup 34

S

- Selectivity 102
- Semi Join 79
- SHOW_STATISTICS 359
- sp_getapplock 321
- SPOOL 106
- Statistics 230
- STRING_SPLIT 284

찾아보기

T

- Table Scan 49
- text 277
- TOP EXPRESSION 121
- TRUNCATE 180

U

- uniqueidentifier 277
- UPDLOCK 321

W

- Worktable 117

ㄱ

- 교착 상태 303

ㄷ

- 동시성 207

ㅂ

- 바인드 변수 190
- 배타 잠금 288
- 비클러스터형 인덱스 39

ㅅ

- 서브 테이블 127
- 스칼라 서브쿼리 138
- 스칼라 함수 245
- 실행 계획 106

ㅇ

- 인라인 뷰 69

ㅈ

- 자동 통계 업데이트 351

ㅋ

- 클러스터형 인덱스 39

ㅌ

- 통계 업데이트 359

ㅍ

- 페이지 분할 54

ㅎ

- 행 버전 295

67개 실전 사례로 배우는
AI 기반 Microsoft SQL Server 성능 최적화 ✦

GitHub Copilot 채팅과 함께하는 SQL 튜닝 가이드

초판 1쇄 인쇄	2026년 6월 12일
지은이	김성식, 이병찬
기획 및 감수	김성식, 이병찬
마케팅·디자인	(주)엑셈 BX팀
펴낸곳	(주)엑셈
출판등록	제 2009-000006호
주소	서울시 강서구 마곡중앙8로 5길 40 (우)07792 (주)엑셈
전화	02. 6203. 6300
팩스	02. 6203. 6301
종이책 ISBN	979-11-88427-26-0
전자책 ISBN	979-11-88427-27-7

이 책은 저작권법에 의하여 보호를 받는 저작물이므로 무단 복제 및 무단 전재를 금합니다.
잘못된 책은 구입하신 서점에서 교환해 드립니다. 책값은 뒤표지에 표시되어 있습니다.

Copyright © 2026 EXEM Co., Ltd. All rights reserved.
Printed in KOREA.

스크립트 다운로드

엑셈 DB 기술 콘텐츠

QR을 스캔하여 이 책의 3장 SQL 실습 스크립트와
다양한 종류의 DB 기술 콘텐츠를 확인하세요.

67개 실전 사례로 배우는
✦ AI 활용 SQL Server 튜닝 실전 가이드

GitHub Copilot 채팅을
코드 생성 도구로만 활용하고 계신가요?

이 책은 GitHub Copilot 채팅을 단순한 도구가 아닌
SQL 성능 문제를 함께 분석하고 해결하는 실전 협업 파트너로 활용할 수 있도록 안내합니다.

- GitHub Copilot 채팅 활용 가이드부터 효과적인 질문 방법 제공
- 실제 현장 SQL 성능 문제를 10개 유형별 섹션으로 체계화 구성
- 67개 실전 SQL 구문 기반의 문제 해결 과정 수록

↳ 각 사례별 기존 방식의 수동 튜닝 과정과 AI 활용 개선 방안을 함께 담았습니다.

67개 사례
실제 성능 이슈 SQL 구문 제시

기존 방식
실행 계획과
경험 기반의 수동 튜닝

+

AI 활용법
AI와 협업하는
SQL 성능 개선

exem

값 28,000원 | ISBN 979-11-88427-27-7
95500